

Numerische Grundlagen der Data Science

LUKAS HOLLENSTEIN

June 22, 2026

Contents

0.1	Übersicht	1
0.1.1	Numerische Grundlagen der Data Science	2
0.1.2	Ein Durchgängiges Beispiel	3
0.2	Grundlagen	11
0.2.1	Grundlagen - Lösungen	12
0.2.2	NumPy & Matplotlib	41
0.2.3	NumPy & Matplotlib	57
0.3	Funktionen Darstellen	67
0.3.1	Darstellung von Funktionen mit mehreren Variablen	68
0.3.2	Darstellung von Funktionen mit mehreren Variablen	74
0.3.3	Darstellung von Funktionen mit mehreren Variablen	83
0.4	Interpolation	88
0.4.1	Interpolation	89
0.4.2	Interpolation	94
0.4.3	Interpolation	106
0.5	Nullstellen	116
0.5.1	Nullstellen	117
0.5.2	Nullstellen	123
0.5.3	Nullstellen	131
0.6	Ableitung	139
0.6.1	Numerische Ableitung	140
0.6.2	Numerische Ableitung	144
0.6.3	Numerische Ableitung	154
0.7	Ableitung Mehrere Variablen	162
0.7.1	Ableitung in mehreren Variablen	163
0.7.2	Ableitung mehrere Variablen	169
0.7.3	Ableitung mit mehreren Variablen	181
0.8	Integration	191
0.8.1	Numerische Integration	192
0.8.2	Numerische Integration	196
0.8.3	Numerische Integration	205
0.9	Anwendung Integration	211
0.9.1	Anwendungen der Integration	212
0.9.2	Anwendungen der Integration	219
0.9.3	Anwendungen der Integration	230
0.10	Best Practices Performance	245
0.10.1	Best Practices und Performance	246
0.10.2	Best Practices und Performance	252
0.11	Zufallszahlen Erzeugen	276
0.11.1	Zufallszahlen Erzeugen	277
0.11.2	Zufallszahlen Erzeugen	282
0.11.3	Zufallszahlen Erzeugen	295
0.11.4	Bootstrapping	306
0.12	Stochastische Prozesse	310

0.12.1	Zufallszahlen Simulieren	311
0.12.2	Stochastischer Prozess	318
0.12.3	Stochastische Prozesse	328

0.1 Übersicht

0.1.1 Numerische Grundlagen der Data Science

Unterrichtsmaterial für das gleichnamige Modul des Bachelorstudiengangs Applied Digital Life Sciences (ADLS) am Departement Life Sciences und Facility Management (LSFM) der Zürcher Hochschule für Angewandte Wissenschaften (ZHAW).

Abstract

In diesem Modul werden unterschiedliche numerische Methoden und deren konkrete Implementationen erarbeitet. Dabei geht es inhaltlich um ausgewählte Themen der numerischen Analysis und linearen Algebra, sowie um das Erzeugen von Zufallszahlen und ihre Bedeutung in der Simulation stochastischer Prozesse. Die Studierenden lernen, die Grundstrukturen numerischer Verfahren zu erkennen, um sie selbst zu programmieren und die Lösungen zu beurteilen, lernen aber auch den Umgang mit bestehenden Open-Source-Programmbibliotheken. Die Wissensvermittlung erfolgt hauptsächlich im asynchronen Onlineunterricht mittels Arbeitsblätter, Lernvideos und kurzen Texten. Im Präsenzunterricht werden die Inhalte anhand von umfangreicheren Übungsaufgaben und numerischen Experimenten weiter vertieft.

Lernziele

Fachkompetenzen Die Studierenden

- können die Grundbegriffe und -konzepte der numerischen Mathematik erklären und verwenden.
- kennen das systematische Vorgehen für iterative, numerische Algorithmen.
- können grundlegende Methoden und Algorithmen der numerischen Analysis, linearen Algebra und Simulation von Zufallsvariablen nennen, erklären und rudimentär implementieren.
- kennen Open-Source Programmbibliotheken, welche die kennengelernten Methoden professionell implementieren, und können damit arbeiten.

Sachkompetenzen Die Studierenden sind in der Lage,

- verschiedene Techniken zur Programmierung von Algorithmen zu verwenden und zu vergleichen.
- die Python-Programmbibliotheken NumPy, SciPy und Matplotlib effektiv in eigenem Code einzusetzen.
- sich beim Programmieren anhand von Dokumentation und Online-Ressourcen selbst zu helfen.
- über die Arbeitsweise von Programmcode zu reflektieren.

Selbst- und Sozialkompetenzen Die Studierenden sind in der Lage,

- strukturiert bei der Lösung eines Problems vorzugehen.
- sich Inhalte mit der für sie am besten geeigneten Methode anzueignen.
- Resultate kritisch zu hinterfragen.
- in kleinen Teams zusammenzuarbeiten.

Zugangsvoraussetzungen

Module aus dem 1. Semester: Analysis und Algebra, Programmieren, Daten und Informationen. Empfehlung: Datenzentriertes Programmieren im gleichen Semester absolvieren.

Lizenz

Diese Unterrichtsmaterialien und die zugehörigen Lernvideos werden als Open Educational Resources unter der CC-BY-SA-4.0 Lizenz auf der Plattform Switch OER zur Verfügung gestellt: <https://oer-repository.switch.ch/edu-sharing/components/collections?q=&id=41e67403-33d1-49b7-a674-0333d1f9b78c>

Die Videos sind auch direkt im folgenden Kanal des ZHAW Mediaspace auf Switch Cast zu finden: <https://zhaw.mediaspace.cast.switch.ch/channel/channelid/108707>

0.1.2 Ein Durchgängiges Beispiel

```
import numpy as np
import matplotlib.pyplot as plt
```

Daten importieren und darstellen

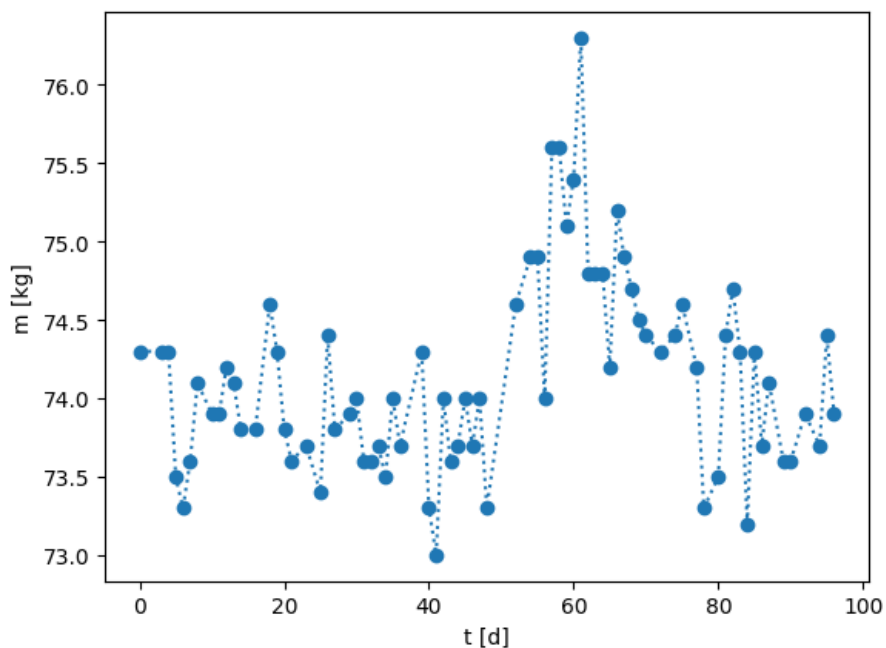
Daten importieren mit `numpy.loadtxt()` und darstellen mit `matplotlib.pyplot.plot()`:

```
data = np.loadtxt("weight.csv", delimiter=";", skiprows=1, usecols=(1, 2))
data[:5]
```

```
array([[ 0. , 74.3],
       [ 3. , 74.3],
       [ 4. , 74.3],
       [ 5. , 73.5],
       [ 6. , 73.3]])
```

```
t, m = data.T
```

```
plt.plot(t, m, "o:")
plt.xlabel("t [d]")
plt.ylabel("m [kg]")
plt.show()
```



Interpolation

Daten können mit `scipy.interpolate.make_interp_spline` linear, quadratisch oder kubisch interpoliert werden, siehe 1-D Interpolation (SciPy Manual).

```
from scipy.interpolate import make_interp_spline
```

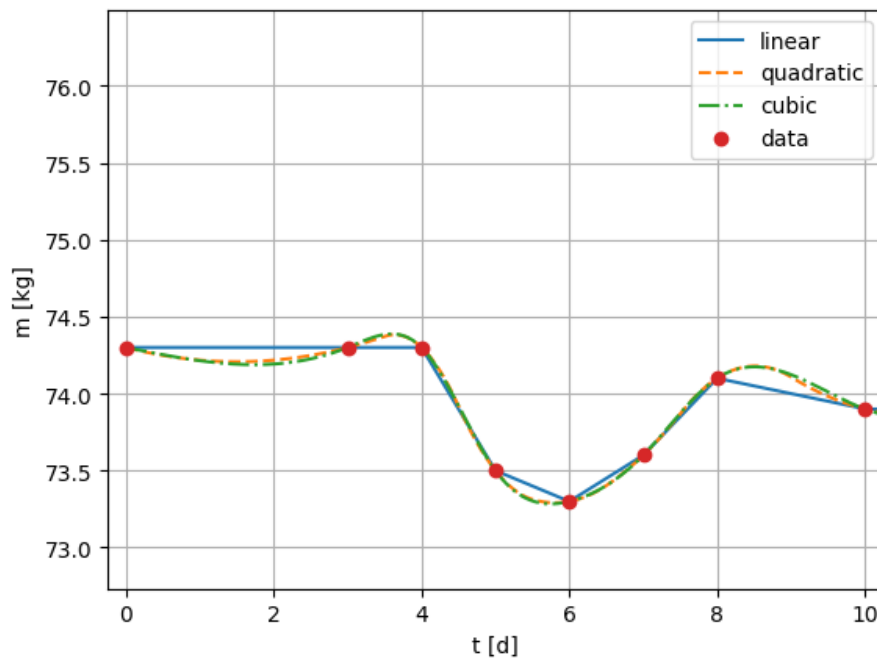
```
# Interpolationsfunktionen
f_linear = make_interp_spline(t, m, k=1)
f_quadratic = make_interp_spline(t, m, k=2)
f_cubic = make_interp_spline(t, m, k=3, bc_type="natural")
```

```

# Interpolationswerte
n = int(t.max() - t.min()) * 100 + 1
t_interp = np.linspace(t.min(), t.max(), n)
m_linear = f_linear(t_interp)
m_quadratic = f_quadratic(t_interp)
m_cubic = f_cubic(t_interp)

plt.plot(t_interp, m_linear, " -", label="linear")
plt.plot(t_interp, m_quadratic, " - -", label="quadratic")
plt.plot(t_interp, m_cubic, " - .", label="cubic")
plt.plot(t, m, "o", label="data")
plt.xlim(-0.25, 10.25)
plt.xlabel("t [d]")
plt.ylabel("m [kg]")
plt.legend()
plt.grid()
plt.show()

```



Nullstellen bestimmen / Gleichungen lösen

Wann sind 76 kg erreicht?

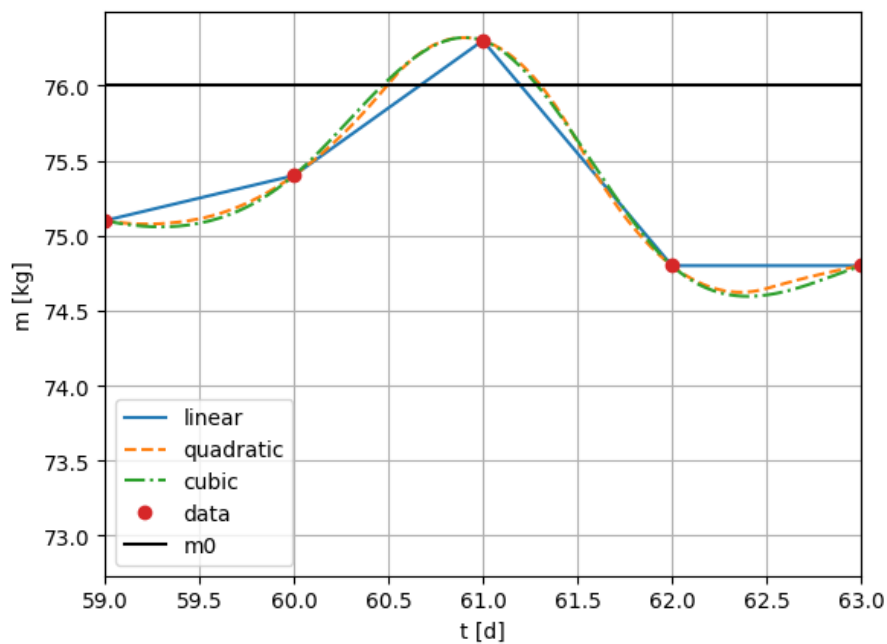
```

m0 = 76
m0_array = np.full_like(t, m0)

plt.plot(t_interp, m_linear, " -", label="linear")
plt.plot(t_interp, m_quadratic, " - -", label="quadratic")
plt.plot(t_interp, m_cubic, " - .", label="cubic")
plt.plot(t, m, "o", label="data")
plt.plot(t, m0_array, "k", label="m0")
plt.xlim(5.9, 6.3)
plt.xlabel("t [d]")
plt.ylabel("m [kg]")
plt.legend()

```

```
plt.grid()
plt.show()
```



Wir interpolieren $m(t) - m_0$ und berechnen die Nullstellen davon.

```
from scipy.optimize import bisect

a, b = 60, 61
# a, b = 61, 62
for i, method in enumerate(["linear", "quadratic", "cubic"]):
    f = make_interp_spline(t, m - m0, k=i + 1)
    t0 = bisect(f, a, b)
    print(method)
    print(f"t0 = {t0}, \t f0 = {f(t0)}\n")

linear
t0 = 60.666666666666666, f0 = -5.457097736657092e -13

quadratic
t0 = 60.49306609025916, f0 = -5.919172570743043e -13

cubic
t0 = 60.467816586946356, f0 = 2.1724976922076636e -12
```

Ableitung

Für die numerische Ableitung können wir `numpy.gradient()` verwenden.

```
dmdt = np.gradient(m, t)
dmdt_linear = np.gradient(m_linear, t_interp)
dmdt_quadratic = np.gradient(m_quadratic, t_interp)
dmdt_cubic = np.gradient(m_cubic, t_interp)

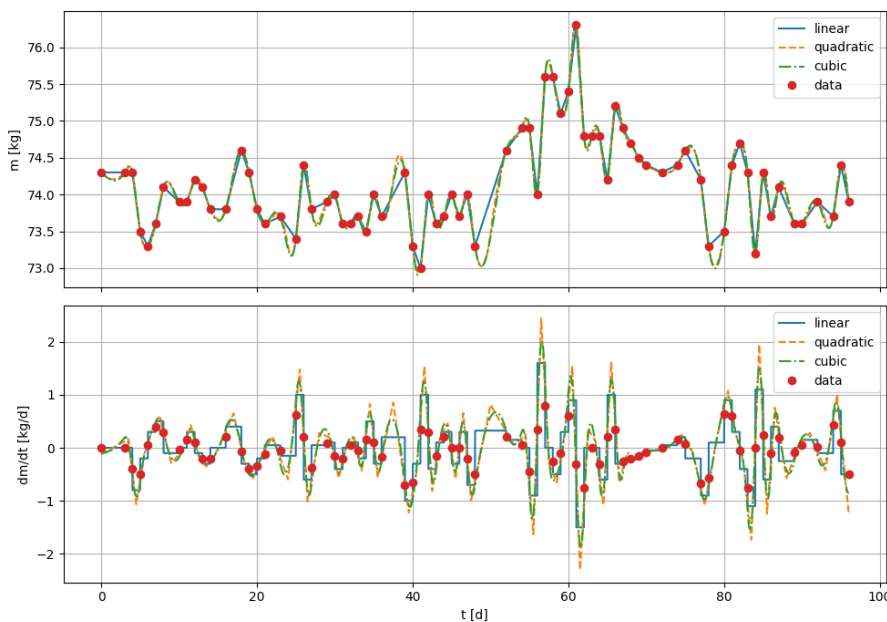
fig, (ax0, ax1) = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
ax0.plot(t_interp, m_linear, " -", label="linear")
ax0.plot(t_interp, m_quadratic, " - -", label="quadratic")
```



```

ax0.plot(t_interp, m_cubic, " -.", label="cubic")
ax0.plot(t, m, "o", label="data")
ax0.set_ylabel("m [kg]")
ax0.legend()
ax0.grid()
ax1.plot(t_interp, dmdt_linear, " -", label="linear")
ax1.plot(t_interp, dmdt_quadratic, " - -", label="quadratic")
ax1.plot(t_interp, dmdt_cubic, " -.", label="cubic")
ax1.plot(t, dmdt, "o", label="data")
ax1.set_xlabel("t [d]")
ax1.set_ylabel("dm/dt [kg/d]")
ax1.legend()
ax1.grid()
plt.tight_layout()
plt.show()

```

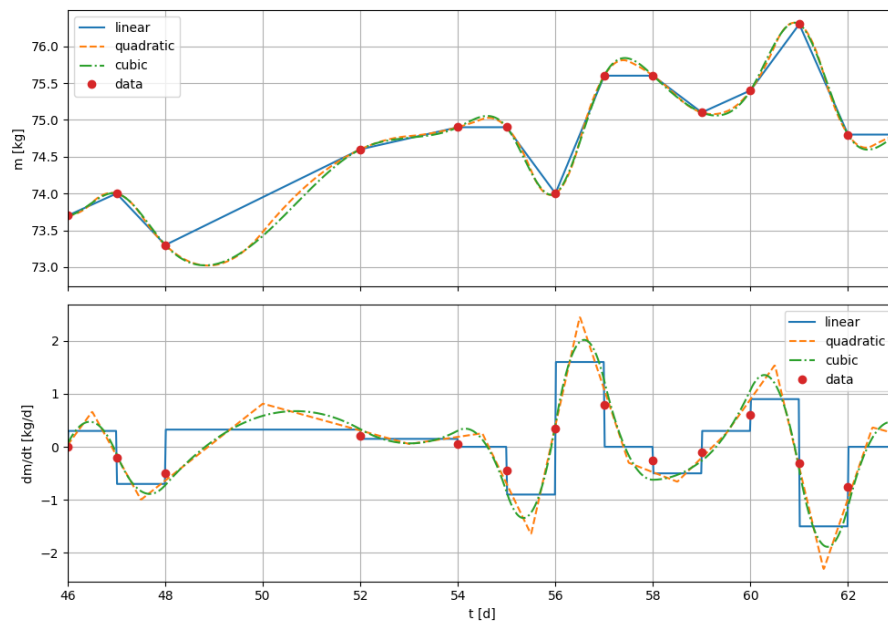


Zoom in einen stark schwankenden Bereich.

```

fig, (ax0, ax1) = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
ax0.plot(t_interp, m_linear, " -", label="linear")
ax0.plot(t_interp, m_quadratic, " - -", label="quadratic")
ax0.plot(t_interp, m_cubic, " -.", label="cubic")
ax0.plot(t, m, "o", label="data")
ax0.set_xlim(46, 63)
ax0.set_ylabel("m [kg]")
ax0.legend()
ax0.grid()
ax1.plot(t_interp, dmdt_linear, " -", label="linear")
ax1.plot(t_interp, dmdt_quadratic, " - -", label="quadratic")
ax1.plot(t_interp, dmdt_cubic, " -.", label="cubic")
ax1.plot(t, dmdt, "o", label="data")
ax1.set_xlabel("t [d]")
ax1.set_ylabel("dm/dt [kg/d]")
ax1.legend()
ax1.grid()
plt.tight_layout()
plt.show()

```



- Die lineare Interpolation ist an den Stützstellen nicht differenzierbar.
- Die Ableitung der quadratischen Interpolation hat Knicks zwischen den Stützstellen und ist daher nur einmal differenzierbar.
- Die kubische Interpolation ist zweimal differenzierbar, kann aber an den Rändern ggf. zu starken Schwankungen führen.

Integration

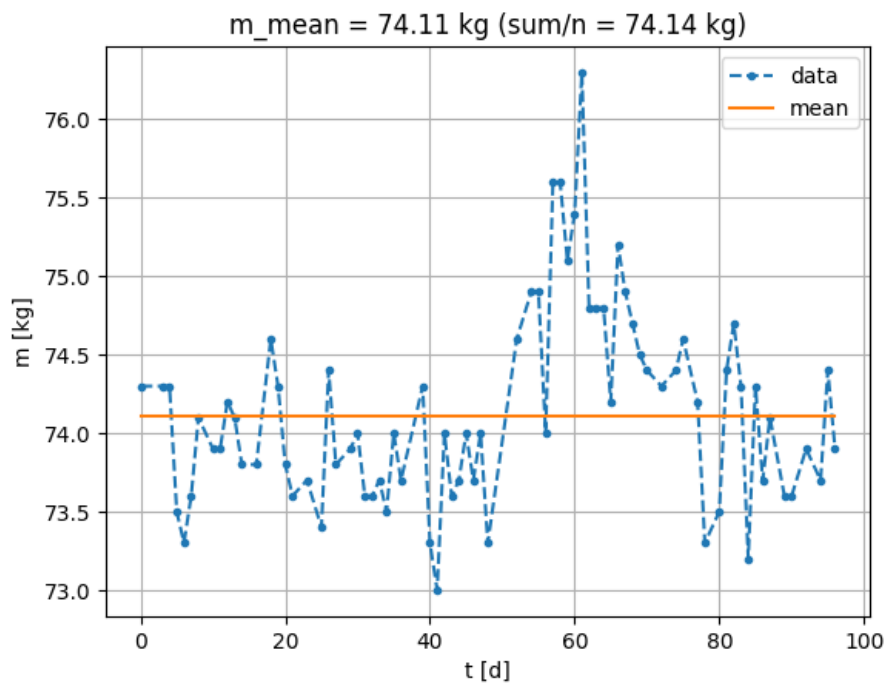
Für die Integration von Funktionen können wir `scipy.integrate.quad()` verwenden und für die Integration von Daten `scipy.integrate.trapezoid()`, siehe Integration (SciPy Manual).

Mittelwert der Daten

```
# integriere über m
from scipy.integrate import trapezoid

t_span = t.max() - t.min()
m_mean = trapezoid(m, t) / t_span

plt.plot(t, m, ". - -", label="data")
plt.plot(t, np.full_like(t, m_mean), " -", label="mean")
plt.xlabel("t [d]")
plt.ylabel("m [kg]")
plt.legend()
plt.grid()
plt.title(f"m_mean = {m_mean:.2f} kg (sum/n = {m.sum()/len(m):.2f} kg)")
plt.show()
```



Mittelwert der Interpolationsfunktion

```
# integriere über m
from scipy.integrate import quad

print("linear: ", quad(f_linear, t.min(), t.max(), limit=1000) / t_span)
print("quadratic: ", quad(f_quadratic, t.min(), t.max(), limit=1000) / t_span)
print("cubic: ", quad(f_cubic, t.min(), t.max(), limit=1000) / t_span)

linear: [7.41088542e+01 1.98951966e -13]
quadratic: [7.40939637e+01 1.10052370e -06]
cubic: [7.40908585e+01 1.06193120e -06]
```

Glättung mithilfe der Faltung

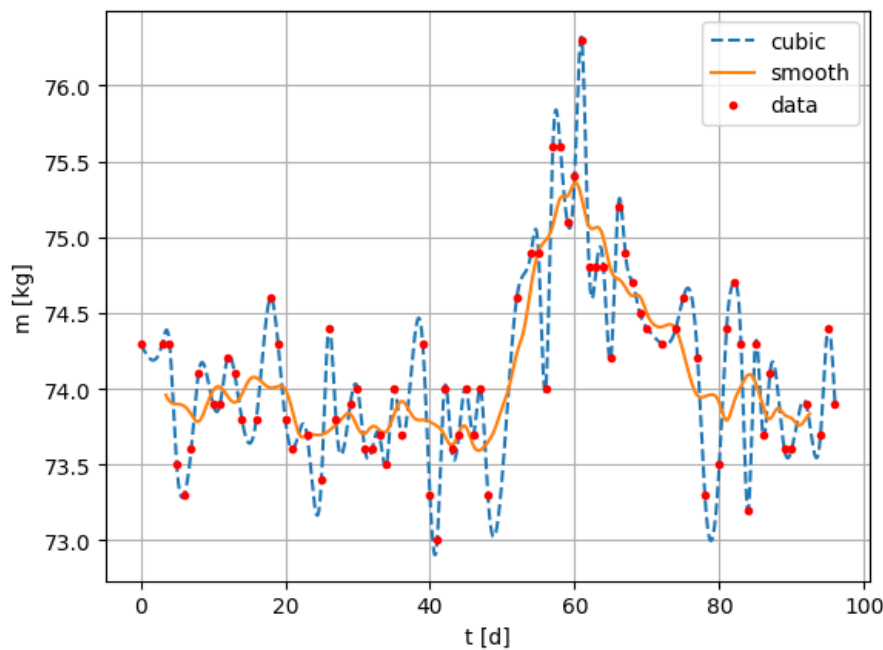
```
width = 7
half_width = width / 2

def tophat(t):
    filter = ( -half_width <= t ) & ( t <= half_width )
    return np.where(filter, 1.0 / width, 0.0)

@np.vectorize
def m_smooth(t):
    return quad(
        lambda tau: tophat(tau) * f_cubic(t - tau),
        -half_width,
        half_width,
        limit=1000,
    )[0]
```

```
t_smooth = np.linspace(t.min() + half_width, t.max() - half_width, 500)
m_smooth_array = m_smooth(t_smooth)
```

```
plt.plot(t_interp, m_cubic, "--", label="cubic")
plt.plot(t_smooth, m_smooth_array, "-", label="smooth")
plt.plot(t, m, "r.", label="data")
# plt.xlim(half_width, t.max() - half_width)
plt.xlabel("t [d]")
plt.ylabel("m [kg]")
plt.legend()
plt.grid()
plt.show()
```



```
width = 28
half_width = width / 2
```

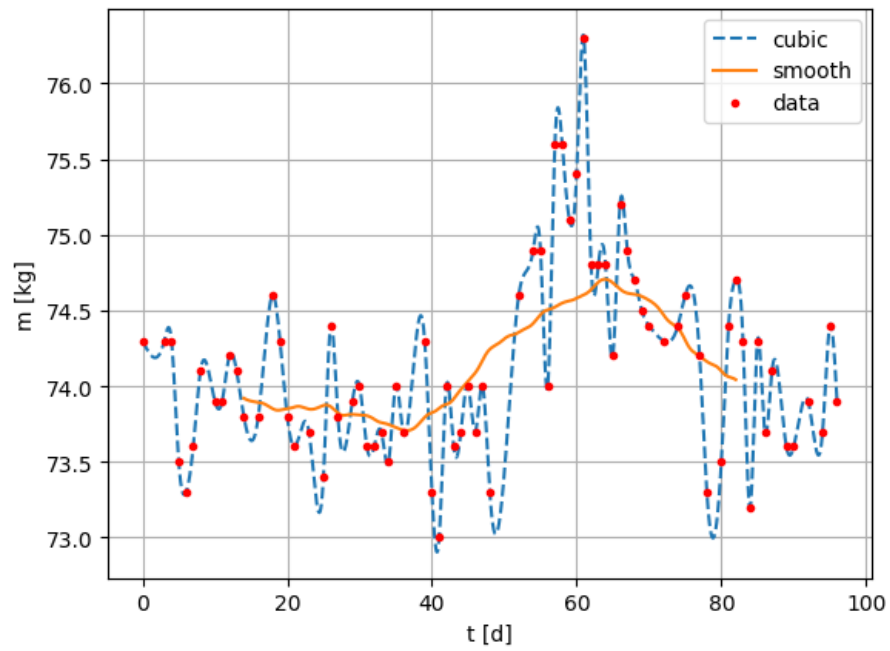
```
def tophat(t):
    filter = ( -half_width <= t ) & ( t <= half_width )
    return np.where(filter, 1.0 / width, 0.0)
```

```
@np.vectorize
def m_smooth(t):
    return quad(
        lambda tau: tophat(tau) * f_cubic(t - tau), -half_width, half_width, limit=1000
    )[0]
```

```
t_smooth = np.linspace(t.min() + half_width, t.max() - half_width, 500)
m_smooth_array = m_smooth(t_smooth)
```

```
plt.plot(t_interp, m_cubic, "--", label="cubic")
plt.plot(t_smooth, m_smooth_array, "-", label="smooth")
plt.plot(t, m, "r.", label="data")
```

```
plt.xlabel("t [d]")  
plt.ylabel("m [kg]")  
plt.legend()  
plt.grid()  
plt.show()
```



0.2 Grundlagen

0.2.1 Grundlagen - Lösungen

Übersicht

Dieses Notebook behandelt die Grundlagen der numerischen Mathematik:

1. Kontinuierliche und diskrete Funktionen, Folgen und iterative Prozesse
2. Zahlendarstellung im Computer (Dezimal-, Binär- und Gleitkommadarstellung)
3. Verschiedene Fehlertypen (Darstellungs-, Rechen-, Diskretisierungs- und Messfehler)

1. Kontinuierlich, Diskret, Iterativ

Kontinuierliche Funktionen / Prozesse Physikalische Größen sind sehr häufig **kontinuierlich**, d.h. sie ändern sich nicht sprunghaft. Beispiele:

- Temperatur einer Flüssigkeit
- Konzentration eines Stoffes in einer Lösung
- Leistung einer Batterie

In der Mathematik sprechen wir dann von **stetigen** Funktionen. Dabei macht ein **stetiger** Funktionsgraph **keine Sprünge** und hat **keine Lücken**.

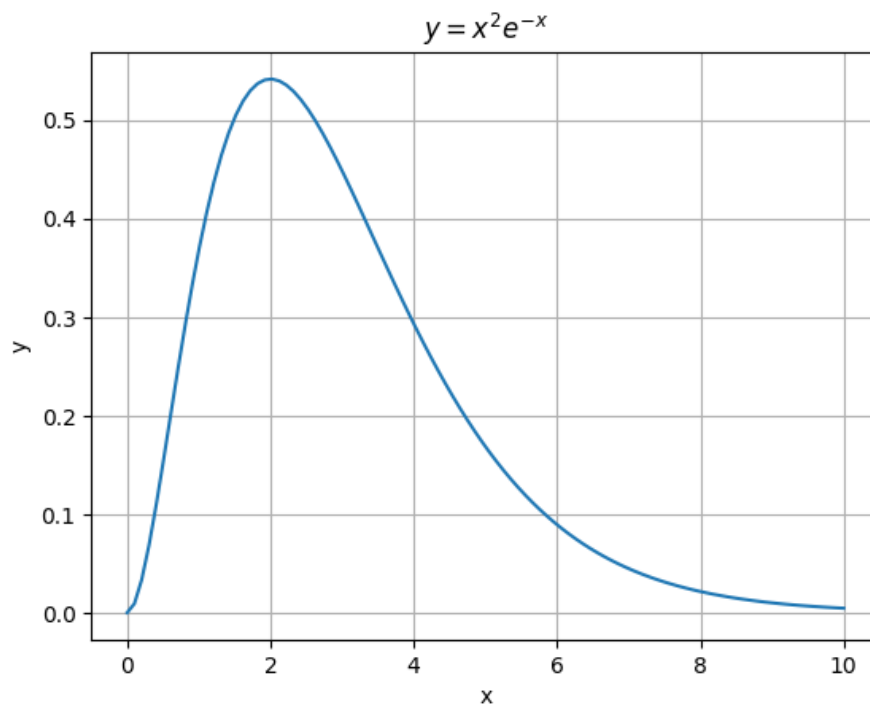
Als Beispiel betrachten wir die Funktion

$$f(x) = x^2 e^{-x} \quad (1)$$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, num=100)
y = x**2 * np.exp(-x)

plt.plot(x, y)
plt.grid()
plt.xlabel("x")
plt.ylabel("y")
plt.title(r"$y = x^2 e^{-x}$")
plt.show()
```



Bemerkung: Berechnet man die Funktion nur an an wenigen Stellen, so wird ersichtlich, dass `matplotlib.pyplot.plot` die Punkte mit Geraden miteinander verbindet, also **linear interpoliert**. Für kontinuierliche Funktionen ist dies sinnvoll, weil ja wirklich alle Werte zwischen zwei Punkten irgendwann mal angenommen werden (Zwischenwertsatz).

Diskrete Funktionen / Prozesse Andere Grössen können nur **spezifische Werte** (meist ganze Zahlen) annehmen. Wir sprechen von **diskreten Werten**. Beispiele:

- Anzahl Bäume in einem Waldstück
- Anzahl weisse Blutkörperchen in einer Blutprobe
- Anzahl Blutprobenröhrchen in einer Warteschlange vor einem Analysegerät
- Energieniveaus von Elektronen in Atomen
- Zustand des Analysegeräts (0=idle, 1=busy, 2=blocked)

Die entsprechenden mathematischen Funktionen können **nur ganzzahlige Werte** annehmen (Wertebereich $W \subseteq \mathbb{Z}$), müssen daher **Sprungstellen** haben und können somit **nicht stetig** sein.

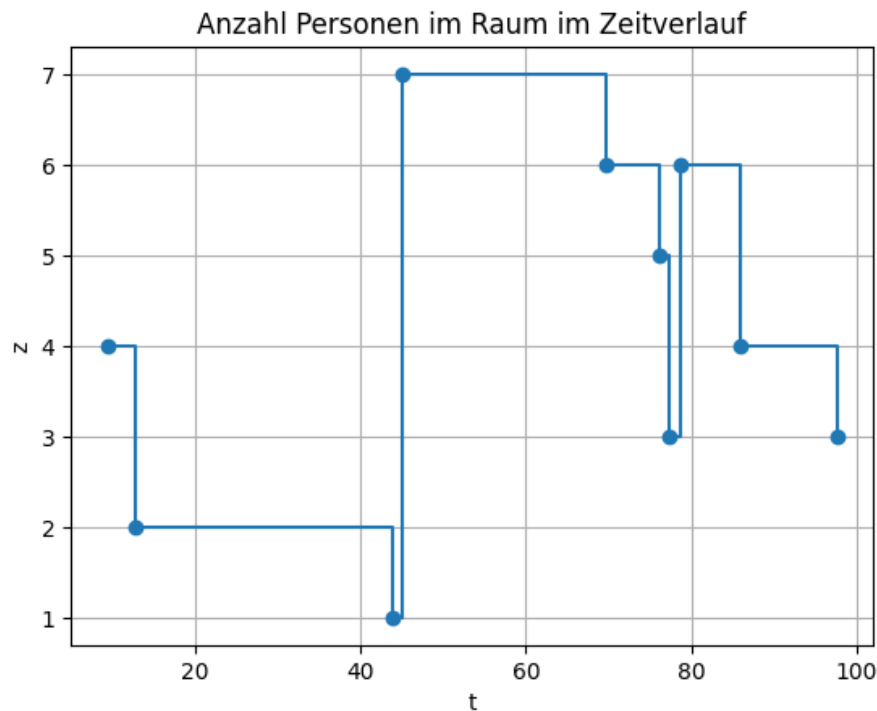
Als Beispiel simulieren wir einen stochastischen Prozess, bei dem sich der diskrete Zustand z an zufälligen Zeitpunkten t ändert. Dies könnte z.B. die Anzahl Personen in einem Raum im Zeitverlauf sein. Hier ist die Zeit kontinuierlich und der Zustand diskret.

```
rng = np.random.default_rng(42)
rng.uniform(low=3, high=5, size=10)

array([4.5479121 , 3.87775688, 4.71719584, 4.39473606, 3.1883547 ,
       4.9512447 , 4.522794 , 4.57212861, 3.25622727, 3.90077188])

rng = np.random.default_rng(42)
t = np.sort(rng.uniform(high=100, size=10))
z = rng.integers(low=0, high=8, size=len(t))

plt.step(t, z, "o -", where="post")
plt.xlabel("t")
plt.ylabel("z")
plt.title("Anzahl Personen im Raum im Zeitverlauf")
plt.grid()
plt.show()
```

Bemerkung: Für die Darstellung einer diskreten Funktion benutzen wir `matplotlib.pyplot.step`, weil Sprünge als senkrechte Linien dargestellt werden, anstatt die Punkte diagonal zu verbinden (also linear zu interpolieren) wie dies `matplotlib.pyplot.plot` macht.

Folgen / Iterative Prozesse Schliesslich gibt es auch Prozesse, die mit konstanten (diskreten) Schritten verlaufen. Hier ist der Fortschritt des Prozesses eine Anzahl Schritte, auch **Iterationen** genannt. In manchen Fällen sind dies Zeitschritte gleicher Schrittlänge, in anderen Fällen aber auch nicht. Solche Prozesse sind aus der Mathematik als **Folgen** bekannt.

Ein Beispiel ist die **logistische Gleichung**

$$x_{n+1} = rx_n(1 - x_n) \quad (2)$$

Sie ist ein Modell von beschränktem Populationswachstum mit einer Wachstumsrate r . Für r -Werte grösser als ca. 3.6 kann die Folge spannendes chaotisches Verhalten zeigen.

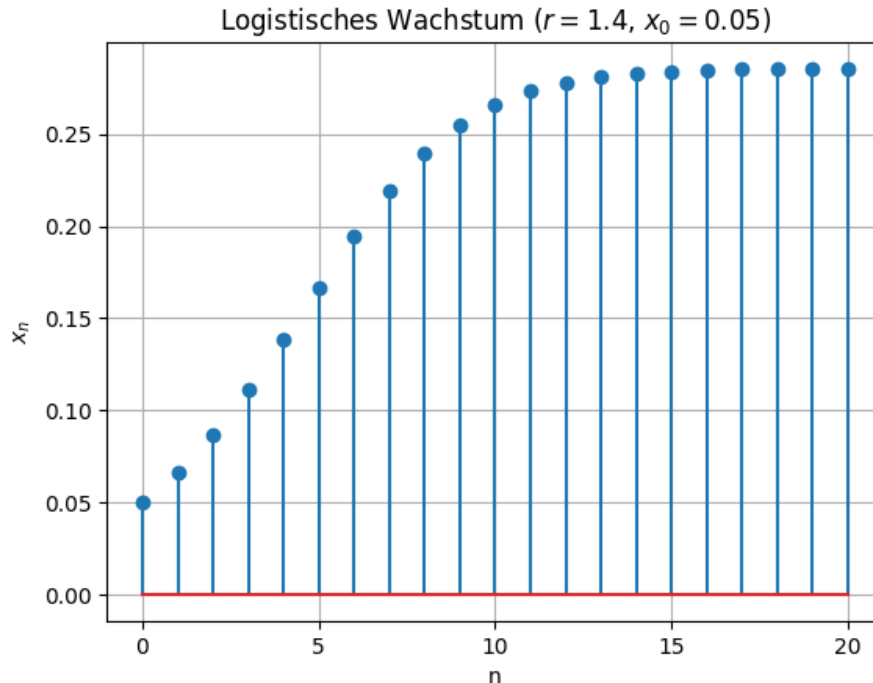
```
r = 1.4
x0 = 0.05
N = 20
```

```
def logistisch(r, x0, N):
    xn = x0
    x = [x0]
    for _ in range(N):
        xn = r * xn * (1 - xn)
        x.append(xn)
    return x
```

```
x = logistisch(r, x0, N)
```

```
fig, ax = plt.subplots()
ax.stem(x)
# ax.plot(x, 'o')
```

```
ax.xaxis.set_ticks(np.arange(N + 1, step=round(N / 4)))
ax.grid()
ax.set_xlabel("n")
ax.set_ylabel("$x_n$")
plt.title(f"Logistisches Wachstum ($r={r}$, $x_0={x0}$)")
plt.show()
```



Hier sind die n -Werte auf der horizontalen Achse **diskret** und x_n -Werte auf der vertikalen Achse **kontinuierlich**. Das heisst, zwischen den dargestellten Punkten existieren keine weiteren Punkte. Darum sollten die Punkte auch nicht verbunden werden.

Für die Darstellung bietet sich daher die Funktion `matplotlib.pyplot.stem` an, welche mit den senkrechten Stäben etwas verdeutlicht, dass es sich um diskrete n -Schritte handelt. Alternativ können auch einfach nicht-verbundene Punkte verwendet werden: `plt.plot(x, 'o')`.

Daten Data Scientists arbeiten häufig mit Daten und nicht nur mit Modellen, die in Form von stetigen, stochastischen oder iterativen Prozessen daher kommen. Daten sind allerdings Beobachtungen von Variablen, deren Ausprägungen entweder kontinuierlich oder diskret sind, siehe *Skalenniveau* im Modul *Daten und Information*. Beim Darstellen von und Rechnen mit Daten ist es daher wichtig, die geeigneten Tools und Methoden einzusetzen.

2. Zahlendarstellung

Wir werfen einen kleinen Blick auf die Darstellung von Zahlen im Computer, um etwas besser zu verstehen, wie genau oder eben ungenau Computer rechnen können.

Dezimalsystem, Binärsystem, b-adisches System Die Zahl **173** lässt sich im **Dezimalsystem** schreiben als

$$173.02 = 1 \cdot 10^2 + 7 \cdot 10^1 + 3 \cdot 10^0 + 1 \cdot 10^{-2} \quad (3)$$

Analog funktioniert das **Binärsystem**, in dem ein Computer Zahlen darstellen und damit rechnen kann:

$$1101_{(2)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 13_{(10)} \quad (4)$$

Allgemein können reelle Zahlen $x \in \mathbb{R}$ bezüglich einer beliebigen Basis $b \in \mathbb{N}$ mit $b \geq 2$ dargestellt werden

$$x = \pm \sum_{k=-\infty}^{\infty} z_k \cdot b^k \quad (5)$$

Hier sind $z_k \in \{0, 1, \dots, b-1\}$ die Ziffern. Für eine allgemeine Basis b spricht man vom **b-adischen Stellenwertsystem**.

Noch ein Beispiel im Binärsystem ($b = 2$ und $z_k \in \{0, 1\}$): Wir können auch binäre (oder dyadische) Brüche darstellen

$$\begin{aligned} 100110.01_{(2)} &= 1 \cdot 2^5 & + 0 \cdot 2^4 & + 0 \cdot 2^3 & + 1 \cdot 2^2 & + 1 \cdot 2^1 & + 0 \cdot 2^0 & + 0 \cdot 2^{-1} & + 1 \cdot 2^{-2} \\ &= 1 \cdot 32 & + 0 \cdot 16 & + 0 \cdot 8 & + 1 \cdot 4 & + 1 \cdot 2 & + 0 \cdot 1 & + 0 \cdot \frac{1}{2} & + 1 \cdot \frac{1}{4} \\ &= 32 & + 0 & + 0 & + 4 & + 2 & + 0 & + \frac{1}{4} \\ &= 38.25_{(10)} \end{aligned}$$

Die Umrechnung vom Dezimal- ins Binärsystem geschieht über die fortgesetzte Division mit Rest. Als Beispiel nehmen wir $x = 12.5_{(10)}$:

1. Suche den grössten Exponenten n mit $2^n \leq x$. Wir finden $n = 3$, weil $2^3 = 8 < x$ und $2^4 = 16 > x$.
2. Die erste 1 in der gesuchten Binärdarstellung ist also an 4. Stelle, $1000_{(2)}$, da zuhinterst die 0. Stelle ist.
3. Wir wiederholen den Vorgang mit dem Rest $x_1 = x - 2^n = 12.5 - 8 = 4.5$: Wir finden $n = 2$ mit $2^2 = 4 = 100_{(2)}$.
4. Und nochmals mit dem $x_2 = x_1 - 2^n = 4.5 - 4 = 0.5$: Wir finden $n = -1$ mit $2^{-1} = 0.5 = 0.1_{(2)}$.

Alles in allem erhalten wir:

$$x = 12.5 = 1000_{(2)} + 100_{(2)} + 0.1_{(2)} = 1100.1_{(2)} \quad (6)$$

Weitere Beispiele sind hier zu finden:

- Video “p-adische Zahlendarstellung - Daten- und Zeichendarstellung 2”, simpleclub
- Numerik 1, Wikiversity

Online-Rechner für Umrechnungen: <https://www.matheretter.de/rechner/dezimalbinar>

Gleitkommadarstellung (Floating Point) In Computern wird mit der Gleitkommadarstellung reeller Zahlen bezüglich der Basis 2 gearbeitet:

$$x = v \cdot m \cdot b^e \quad (7)$$

Da der Speicher endlich ist, kann nur eine gewisse Anzahl Bits (binäre Ziffern) verwendet werden. Eine Konvention ist die Folgende (IEEE 754 double):

Symbol	Bezeichnung	Bereich	Bits
v	Vorzeichen	$\{+1, -1\}$	1
m	Mantisse	$\mathbb{N}_0$	52
b	Basis	$\{2, 10\}$	0
e	Exponent	$\mathbb{Z}$	11

- 52 Bit für die Mantisse bedeutet, dass die grösste darstellbare Mantisse $m_{\max} = 2^{52} - 1 = 10^{52 \lg(2)} - 1 \approx 10^{15.7}$ ist. Das ist eine **Dezimalzahl mit 16 Stellen**.
- Weil der Exponent e auch negativ sein kann, wird eines der 11 Bits für das Vorzeichen verwendet. Die restlichen 10 Bit ergeben den maximalen Exponenten $2^{10} - 1 = 1023$ zur Basis 2, was ca. $1023 \lg(2) \approx 308$ als maximalen Exponenten zur Basis 10 entspricht.

Die Diskussion hier ist etwas vereinfacht, reicht aber für das Grundverständnis. Mehr Details finden sich hier:

- Gleitkommazahl, Wikipedia
- IEEE_754-2008, Wikipedia
- Numerik 1, Wikiversity

```
print(52 * np.log10(2))
print(1023 * np.log10(2))
```

```
15.653559774527022
307.9536855642528
```

Lernaufgabe 1: Kleinste Zahl Finde heraus, welches (ungefähr) die kleinste darstellbare Gleitkommazahl ist, indem Du die Zahl 1 immer wieder durch 2 teilst, bis das Ergebnis nicht mehr grösser Null ist. (While-Schleife verwenden!)

```
# Lösung
x = 1
while x > 0:
    x = x / 2
    print(x)
```

```
0.5
0.25
0.125
0.0625
0.03125
0.015625
0.0078125
0.00390625
0.001953125
0.0009765625
0.00048828125
0.000244140625
0.0001220703125
6.103515625e -05
3.0517578125e -05
1.52587890625e -05
7.62939453125e -06
3.814697265625e -06
1.9073486328125e -06
9.5367431640625e -07
4.76837158203125e -07
2.384185791015625e -07
1.1920928955078125e -07
5.960464477539063e -08
2.9802322387695312e -08
1.4901161193847656e -08
7.450580596923828e -09
```

3.725290298461914e -09
1.862645149230957e -09
9.313225746154785e -10
4.656612873077393e -10
2.3283064365386963e -10
1.1641532182693481e -10
5.820766091346741e -11
2.9103830456733704e -11
1.4551915228366852e -11
7.275957614183426e -12
3.637978807091713e -12
1.8189894035458565e -12
9.094947017729282e -13
4.547473508864641e -13
2.2737367544323206e -13
1.1368683772161603e -13
5.684341886080802e -14
2.842170943040401e -14
1.4210854715202004e -14
7.105427357601002e -15
3.552713678800501e -15
1.7763568394002505e -15
8.881784197001252e -16
4.440892098500626e -16
2.220446049250313e -16
1.1102230246251565e -16
5.551115123125783e -17
2.7755575615628914e -17
1.3877787807814457e -17
6.938893903907228e -18
3.469446951953614e -18
1.734723475976807e -18
8.673617379884035e -19
4.336808689942018e -19
2.168404344971009e -19
1.0842021724855044e -19
5.421010862427522e -20
2.710505431213761e -20
1.3552527156068805e -20
6.776263578034403e -21
3.3881317890172014e -21
1.6940658945086007e -21
8.470329472543003e -22
4.235164736271502e -22
2.117582368135751e -22
1.0587911840678754e -22
5.293955920339377e -23
2.6469779601696886e -23
1.3234889800848443e -23
6.617444900424222e -24
3.308722450212111e -24
1.6543612251060553e -24
8.271806125530277e -25

4.1359030627651384e	-25
2.0679515313825692e	-25
1.0339757656912846e	-25
5.169878828456423e	-26
2.5849394142282115e	-26
1.2924697071141057e	-26
6.462348535570529e	-27
3.2311742677852644e	-27
1.6155871338926322e	-27
8.077935669463161e	-28
4.0389678347315804e	-28
2.0194839173657902e	-28
1.0097419586828951e	-28
5.048709793414476e	-29
2.524354896707238e	-29
1.262177448353619e	-29
6.310887241768095e	-30
3.1554436208840472e	-30
1.5777218104420236e	-30
7.888609052210118e	-31
3.944304526105059e	-31
1.9721522630525295e	-31
9.860761315262648e	-32
4.930380657631324e	-32
2.465190328815662e	-32
1.232595164407831e	-32
6.162975822039155e	-33
3.0814879110195774e	-33
1.5407439555097887e	-33
7.703719777548943e	-34
3.851859888774472e	-34
1.925929944387236e	-34
9.62964972193618e	-35
4.81482486096809e	-35
2.407412430484045e	-35
1.2037062152420224e	-35
6.018531076210112e	-36
3.009265538105056e	-36
1.504632769052528e	-36
7.52316384526264e	-37
3.76158192263132e	-37
1.88079096131566e	-37
9.4039548065783e	-38
4.70197740328915e	-38
2.350988701644575e	-38
1.1754943508222875e	-38
5.877471754111438e	-39
2.938735877055719e	-39
1.4693679385278594e	-39
7.346839692639297e	-40
3.6734198463196485e	-40
1.8367099231598242e	-40
9.183549615799121e	-41

4.591774807899561e	-41
2.2958874039497803e	-41
1.1479437019748901e	-41
5.739718509874451e	-42
2.8698592549372254e	-42
1.4349296274686127e	-42
7.174648137343064e	-43
3.587324068671532e	-43
1.793662034335766e	-43
8.96831017167883e	-44
4.484155085839415e	-44
2.2420775429197073e	-44
1.1210387714598537e	-44
5.605193857299268e	-45
2.802596928649634e	-45
1.401298464324817e	-45
7.006492321624085e	-46
3.503246160812043e	-46
1.7516230804060213e	-46
8.758115402030107e	-47
4.3790577010150533e	-47
2.1895288505075267e	-47
1.0947644252537633e	-47
5.473822126268817e	-48
2.7369110631344083e	-48
1.3684555315672042e	-48
6.842277657836021e	-49
3.4211388289180104e	-49
1.7105694144590052e	-49
8.552847072295026e	-50
4.276423536147513e	-50
2.1382117680737565e	-50
1.0691058840368783e	-50
5.345529420184391e	-51
2.6727647100921956e	-51
1.3363823550460978e	-51
6.681911775230489e	-52
3.3409558876152446e	-52
1.6704779438076223e	-52
8.352389719038111e	-53
4.176194859519056e	-53
2.088097429759528e	-53
1.044048714879764e	-53
5.22024357439882e	-54
2.61012178719941e	-54
1.305060893599705e	-54
6.525304467998525e	-55
3.2626522339992623e	-55
1.6313261169996311e	-55
8.156630584998156e	-56
4.078315292499078e	-56
2.039157646249539e	-56
1.0195788231247695e	-56

5.0978941156238473e	-57
2.5489470578119236e	-57
1.2744735289059618e	-57
6.372367644529809e	-58
3.1861838222649046e	-58
1.5930919111324523e	-58
7.965459555662261e	-59
3.982729777831131e	-59
1.9913648889155653e	-59
9.956824444577827e	-60
4.9784122222889134e	-60
2.4892061111444567e	-60
1.2446030555722283e	-60
6.223015277861142e	-61
3.111507638930571e	-61
1.5557538194652854e	-61
7.778769097326427e	-62
3.8893845486632136e	-62
1.9446922743316068e	-62
9.723461371658034e	-63
4.861730685829017e	-63
2.4308653429145085e	-63
1.2154326714572542e	-63
6.077163357286271e	-64
3.0385816786431356e	-64
1.5192908393215678e	-64
7.596454196607839e	-65
3.7982270983039195e	-65
1.8991135491519597e	-65
9.495567745759799e	-66
4.7477838728798994e	-66
2.3738919364399497e	-66
1.1869459682199748e	-66
5.934729841099874e	-67
2.967364920549937e	-67
1.4836824602749686e	-67
7.418412301374843e	-68
3.7092061506874214e	-68
1.8546030753437107e	-68
9.273015376718553e	-69
4.636507688359277e	-69
2.3182538441796384e	-69
1.1591269220898192e	-69
5.795634610449096e	-70
2.897817305224548e	-70
1.448908652612274e	-70
7.24454326306137e	-71
3.622271631530685e	-71
1.8111358157653425e	-71
9.055679078826712e	-72
4.527839539413356e	-72
2.263919769706678e	-72
1.131959884853339e	-72

5.659799424266695e	-73
2.8298997121333476e	-73
1.4149498560666738e	-73
7.074749280333369e	-74
3.5373746401666845e	-74
1.7686873200833423e	-74
8.843436600416711e	-75
4.421718300208356e	-75
2.210859150104178e	-75
1.105429575052089e	-75
5.527147875260445e	-76
2.7635739376302223e	-76
1.3817869688151111e	-76
6.908934844075556e	-77
3.454467422037778e	-77
1.727233711018889e	-77
8.636168555094445e	-78
4.3180842775472223e	-78
2.1590421387736112e	-78
1.0795210693868056e	-78
5.397605346934028e	-79
2.698802673467014e	-79
1.349401336733507e	-79
6.747006683667535e	-80
3.3735033418337674e	-80
1.6867516709168837e	-80
8.433758354584419e	-81
4.2168791772922093e	-81
2.1084395886461046e	-81
1.0542197943230523e	-81
5.271098971615262e	-82
2.635549485807631e	-82
1.3177747429038154e	-82
6.588873714519077e	-83
3.2944368572595385e	-83
1.6472184286297693e	-83
8.236092143148846e	-84
4.118046071574423e	-84
2.0590230357872116e	-84
1.0295115178936058e	-84
5.147557589468029e	-85
2.5737787947340145e	-85
1.2868893973670072e	-85
6.434446986835036e	-86
3.217223493417518e	-86
1.608611746708759e	-86
8.043058733543795e	-87
4.021529366771898e	-87
2.010764683385949e	-87
1.0053823416929744e	-87
5.026911708464872e	-88
2.513455854232436e	-88
1.256727927116218e	-88

6.28363963558109e	-89
3.141819817790545e	-89
1.5709099088952725e	-89
7.854549544476363e	-90
3.9272747722381812e	-90
1.9636373861190906e	-90
9.818186930595453e	-91
4.909093465297727e	-91
2.4545467326488633e	-91
1.2272733663244316e	-91
6.136366831622158e	-92
3.068183415811079e	-92
1.5340917079055395e	-92
7.670458539527698e	-93
3.835229269763849e	-93
1.9176146348819244e	-93
9.588073174409622e	-94
4.794036587204811e	-94
2.3970182936024055e	-94
1.1985091468012028e	-94
5.992545734006014e	-95
2.996272867003007e	-95
1.4981364335015035e	-95
7.490682167507517e	-96
3.745341083753759e	-96
1.8726705418768793e	-96
9.363352709384397e	-97
4.6816763546921983e	-97
2.3408381773460992e	-97
1.1704190886730496e	-97
5.852095443365248e	-98
2.926047721682624e	-98
1.463023860841312e	-98
7.31511930420656e	-99
3.65755965210328e	-99
1.82877982605164e	-99
9.1438991302582e	-100
4.5719495651291e	-100
2.28597478256455e	-100
1.142987391282275e	-100
5.714936956411375e	-101
2.8574684782056875e	-101
1.4287342391028437e	-101
7.143671195514219e	-102
3.5718355977571093e	-102
1.7859177988785547e	-102
8.929588994392773e	-103
4.464794497196387e	-103
2.2323972485981933e	-103
1.1161986242990967e	-103
5.5809931214954833e	-104
2.7904965607477417e	-104
1.3952482803738708e	-104

6.976241401869354e	-105
3.488120700934677e	-105
1.7440603504673385e	-105
8.720301752336693e	-106
4.3601508761683463e	-106
2.1800754380841732e	-106
1.0900377190420866e	-106
5.450188595210433e	-107
2.7250942976052165e	-107
1.3625471488026082e	-107
6.812735744013041e	-108
3.4063678720065206e	-108
1.7031839360032603e	-108
8.515919680016301e	-109
4.257959840008151e	-109
2.1289799200040754e	-109
1.0644899600020377e	-109
5.3224498000101884e	-110
2.6612249000050942e	-110
1.3306124500025471e	-110
6.653062250012736e	-111
3.326531125006368e	-111
1.663265562503184e	-111
8.31632781251592e	-112
4.15816390625796e	-112
2.07908195312898e	-112
1.03954097656449e	-112
5.19770488282245e	-113
2.598852441411225e	-113
1.2994262207056124e	-113
6.497131103528062e	-114
3.248565551764031e	-114
1.6242827758820155e	-114
8.121413879410078e	-115
4.060706939705039e	-115
2.0303534698525194e	-115
1.0151767349262597e	-115
5.075883674631299e	-116
2.5379418373156492e	-116
1.2689709186578246e	-116
6.344854593289123e	-117
3.1724272966445615e	-117
1.5862136483222808e	-117
7.931068241611404e	-118
3.965534120805702e	-118
1.982767060402851e	-118
9.913835302014255e	-119
4.9569176510071274e	-119
2.4784588255035637e	-119
1.2392294127517818e	-119
6.196147063758909e	-120
3.0980735318794546e	-120
1.5490367659397273e	-120

7.745183829698637e	-121
3.8725919148493183e	-121
1.9362959574246591e	-121
9.681479787123296e	-122
4.840739893561648e	-122
2.420369946780824e	-122
1.210184973390412e	-122
6.05092486695206e	-123
3.02546243347603e	-123
1.512731216738015e	-123
7.563656083690075e	-124
3.7818280418450374e	-124
1.8909140209225187e	-124
9.454570104612593e	-125
4.727285052306297e	-125
2.3636425261531484e	-125
1.1818212630765742e	-125
5.909106315382871e	-126
2.9545531576914354e	-126
1.4772765788457177e	-126
7.386382894228589e	-127
3.6931914471142943e	-127
1.8465957235571472e	-127
9.232978617785736e	-128
4.616489308892868e	-128
2.308244654446434e	-128
1.154122327223217e	-128
5.770611636116085e	-129
2.8853058180580424e	-129
1.4426529090290212e	-129
7.213264545145106e	-130
3.606632272572553e	-130
1.8033161362862765e	-130
9.016580681431383e	-131
4.5082903407156913e	-131
2.2541451703578456e	-131
1.1270725851789228e	-131
5.635362925894614e	-132
2.817681462947307e	-132
1.4088407314736535e	-132
7.044203657368268e	-133
3.522101828684134e	-133
1.761050914342067e	-133
8.805254571710335e	-134
4.4026272858551673e	-134
2.2013136429275836e	-134
1.1006568214637918e	-134
5.503284107318959e	-135
2.7516420536594796e	-135
1.3758210268297398e	-135
6.879105134148699e	-136
3.4395525670743494e	-136
1.7197762835371747e	-136

8.598881417685874e	-137
4.299440708842937e	-137
2.1497203544214684e	-137
1.0748601772107342e	-137
5.374300886053671e	-138
2.6871504430268355e	-138
1.3435752215134178e	-138
6.717876107567089e	-139
3.3589380537835444e	-139
1.6794690268917722e	-139
8.397345134458861e	-140
4.1986725672294305e	-140
2.0993362836147152e	-140
1.0496681418073576e	-140
5.248340709036788e	-141
2.624170354518394e	-141
1.312085177259197e	-141
6.560425886295985e	-142
3.2802129431479926e	-142
1.6401064715739963e	-142
8.200532357869981e	-143
4.100266178934991e	-143
2.0501330894674953e	-143
1.0250665447337477e	-143
5.1253327236687384e	-144
2.5626663618343692e	-144
1.2813331809171846e	-144
6.406665904585923e	-145
3.2033329522929615e	-145
1.6016664761464807e	-145
8.008332380732404e	-146
4.004166190366202e	-146
2.002083095183101e	-146
1.0010415475915505e	-146
5.0052077379577523e	-147
2.5026038689788762e	-147
1.2513019344894381e	-147
6.256509672447191e	-148
3.1282548362235952e	-148
1.5641274181117976e	-148
7.820637090558988e	-149
3.910318545279494e	-149
1.955159272639747e	-149
9.775796363198735e	-150
4.887898181599368e	-150
2.443949090799684e	-150
1.221974545399842e	-150
6.10987272699921e	-151
3.054936363499605e	-151
1.5274681817498023e	-151
7.637340908749012e	-152
3.818670454374506e	-152
1.909335227187253e	-152

9.546676135936265e	-153
4.7733380679681323e	-153
2.3866690339840662e	-153
1.1933345169920331e	-153
5.966672584960166e	-154
2.983336292480083e	-154
1.4916681462400413e	-154
7.458340731200207e	-155
3.7291703656001034e	-155
1.8645851828000517e	-155
9.322925914000258e	-156
4.661462957000129e	-156
2.3307314785000646e	-156
1.1653657392500323e	-156
5.826828696250162e	-157
2.913414348125081e	-157
1.4567071740625404e	-157
7.283535870312702e	-158
3.641767935156351e	-158
1.8208839675781755e	-158
9.104419837890877e	-159
4.552209918945439e	-159
2.2761049594727193e	-159
1.1380524797363597e	-159
5.6902623986817984e	-160
2.8451311993408992e	-160
1.4225655996704496e	-160
7.112827998352248e	-161
3.556413999176124e	-161
1.778206999588062e	-161
8.89103499794031e	-162
4.445517498970155e	-162
2.2227587494850775e	-162
1.1113793747425387e	-162
5.556896873712694e	-163
2.778448436856347e	-163
1.3892242184281734e	-163
6.946121092140867e	-164
3.4730605460704336e	-164
1.7365302730352168e	-164
8.682651365176084e	-165
4.341325682588042e	-165
2.170662841294021e	-165
1.0853314206470105e	-165
5.426657103235053e	-166
2.7133285516175262e	-166
1.3566642758087631e	-166
6.783321379043816e	-167
3.391660689521908e	-167
1.695830344760954e	-167
8.47915172380477e	-168
4.239575861902385e	-168
2.1197879309511924e	-168

1.0598939654755962e	-168
5.299469827377981e	-169
2.6497349136889905e	-169
1.3248674568444952e	-169
6.624337284222476e	-170
3.312168642111238e	-170
1.656084321055619e	-170
8.280421605278095e	-171
4.140210802639048e	-171
2.070105401319524e	-171
1.035052700659762e	-171
5.17526350329881e	-172
2.587631751649405e	-172
1.2938158758247024e	-172
6.469079379123512e	-173
3.234539689561756e	-173
1.617269844780878e	-173
8.08634922390439e	-174
4.043174611952195e	-174
2.0215873059760975e	-174
1.0107936529880487e	-174
5.053968264940244e	-175
2.526984132470122e	-175
1.263492066235061e	-175
6.317460331175305e	-176
3.1587301655876523e	-176
1.5793650827938261e	-176
7.896825413969131e	-177
3.9484127069845653e	-177
1.9742063534922827e	-177
9.871031767461413e	-178
4.935515883730707e	-178
2.4677579418653533e	-178
1.2338789709326767e	-178
6.169394854663383e	-179
3.084697427331692e	-179
1.542348713665846e	-179
7.71174356832923e	-180
3.855871784164615e	-180
1.9279358920823073e	-180
9.639679460411536e	-181
4.819839730205768e	-181
2.409919865102884e	-181
1.204959932551442e	-181
6.02479966275721e	-182
3.012399831378605e	-182
1.5061999156893026e	-182
7.530999578446513e	-183
3.7654997892232564e	-183
1.8827498946116282e	-183
9.413749473058141e	-184
4.706874736529071e	-184
2.3534373682645353e	-184

1.1767186841322676e	-184
5.883593420661338e	-185
2.941796710330669e	-185
1.4708983551653345e	-185
7.354491775826673e	-186
3.6772458879133364e	-186
1.8386229439566682e	-186
9.193114719783341e	-187
4.5965573598916705e	-187
2.2982786799458352e	-187
1.1491393399729176e	-187
5.745696699864588e	-188
2.872848349932294e	-188
1.436424174966147e	-188
7.182120874830735e	-189
3.5910604374153675e	-189
1.7955302187076838e	-189
8.977651093538419e	-190
4.4888255467692094e	-190
2.2444127733846047e	-190
1.1222063866923024e	-190
5.611031933461512e	-191
2.805515966730756e	-191
1.402757983365378e	-191
7.01378991682689e	-192
3.506894958413445e	-192
1.7534474792067224e	-192
8.767237396033612e	-193
4.383618698016806e	-193
2.191809349008403e	-193
1.0959046745042015e	-193
5.479523372521008e	-194
2.739761686260504e	-194
1.369880843130252e	-194
6.84940421565126e	-195
3.42470210782563e	-195
1.712351053912815e	-195
8.561755269564074e	-196
4.280877634782037e	-196
2.1404388173910186e	-196
1.0702194086955093e	-196
5.351097043477547e	-197
2.6755485217387732e	-197
1.3377742608693866e	-197
6.688871304346933e	-198
3.3444356521734666e	-198
1.6722178260867333e	-198
8.361089130433666e	-199
4.180544565216833e	-199
2.0902722826084166e	-199
1.0451361413042083e	-199
5.225680706521042e	-200
2.612840353260521e	-200

1.3064201766302604e	-200
6.532100883151302e	-201
3.266050441575651e	-201
1.6330252207878255e	-201
8.165126103939127e	-202
4.082563051969564e	-202
2.041281525984782e	-202
1.020640762992391e	-202
5.103203814961955e	-203
2.5516019074809773e	-203
1.2758009537404886e	-203
6.379004768702443e	-204
3.1895023843512216e	-204
1.5947511921756108e	-204
7.973755960878054e	-205
3.986877980439027e	-205
1.9934389902195135e	-205
9.967194951097568e	-206
4.983597475548784e	-206
2.491798737774392e	-206
1.245899368887196e	-206
6.22949684443598e	-207
3.11474842221799e	-207
1.557374211108995e	-207
7.786871055544975e	-208
3.8934355277724873e	-208
1.9467177638862437e	-208
9.733588819431218e	-209
4.866794409715609e	-209
2.4333972048578046e	-209
1.2166986024289023e	-209
6.083493012144512e	-210
3.041746506072256e	-210
1.520873253036128e	-210
7.60436626518064e	-211
3.80218313259032e	-211
1.90109156629516e	-211
9.5054578314758e	-212
4.7527289157379e	-212
2.37636445786895e	-212
1.188182228934475e	-212
5.940911144672375e	-213
2.9704555723361872e	-213
1.4852277861680936e	-213
7.426138930840468e	-214
3.713069465420234e	-214
1.856534732710117e	-214
9.282673663550585e	-215
4.641336831775293e	-215
2.3206684158876463e	-215
1.1603342079438231e	-215
5.801671039719116e	-216
2.900835519859558e	-216

1.450417759929779e	-216
7.252088799648895e	-217
3.6260443998244473e	-217
1.8130221999122236e	-217
9.065110999561118e	-218
4.532555499780559e	-218
2.2662777498902796e	-218
1.1331388749451398e	-218
5.665694374725699e	-219
2.8328471873628494e	-219
1.4164235936814247e	-219
7.082117968407124e	-220
3.541058984203562e	-220
1.770529492101781e	-220
8.852647460508905e	-221
4.4263237302544523e	-221
2.2131618651272261e	-221
1.1065809325636131e	-221
5.5329046628180653e	-222
2.7664523314090327e	-222
1.3832261657045163e	-222
6.916130828522582e	-223
3.458065414261291e	-223
1.7290327071306454e	-223
8.645163535653227e	-224
4.322581767826614e	-224
2.161290883913307e	-224
1.0806454419566534e	-224
5.403227209783267e	-225
2.7016136048916335e	-225
1.3508068024458167e	-225
6.754034012229084e	-226
3.377017006114542e	-226
1.688508503057271e	-226
8.442542515286355e	-227
4.2212712576431773e	-227
2.1106356288215886e	-227
1.0553178144107943e	-227
5.276589072053972e	-228
2.638294536026986e	-228
1.319147268013493e	-228
6.595736340067465e	-229
3.2978681700337323e	-229
1.6489340850168661e	-229
8.244670425084331e	-230
4.1223352125421653e	-230
2.0611676062710827e	-230
1.0305838031355413e	-230
5.152919015677707e	-231
2.5764595078388533e	-231
1.2882297539194267e	-231
6.441148769597133e	-232
3.220574384798567e	-232

1.6102871923992833e	-232
8.051435961996417e	-233
4.0257179809982083e	-233
2.0128589904991042e	-233
1.0064294952495521e	-233
5.0321474762477604e	-234
2.5160737381238802e	-234
1.2580368690619401e	-234
6.290184345309701e	-235
3.1450921726548502e	-235
1.5725460863274251e	-235
7.862730431637126e	-236
3.931365215818563e	-236
1.9656826079092814e	-236
9.828413039546407e	-237
4.914206519773204e	-237
2.457103259886602e	-237
1.228551629943301e	-237
6.142758149716505e	-238
3.0713790748582522e	-238
1.5356895374291261e	-238
7.678447687145631e	-239
3.8392238435728152e	-239
1.9196119217864076e	-239
9.598059608932038e	-240
4.799029804466019e	-240
2.3995149022330095e	-240
1.1997574511165048e	-240
5.998787255582524e	-241
2.999393627791262e	-241
1.499696813895631e	-241
7.498484069478155e	-242
3.7492420347390774e	-242
1.8746210173695387e	-242
9.373105086847693e	-243
4.686552543423847e	-243
2.3432762717119234e	-243
1.1716381358559617e	-243
5.858190679279809e	-244
2.9290953396399042e	-244
1.4645476698199521e	-244
7.322738349099761e	-245
3.6613691745498803e	-245
1.8306845872749401e	-245
9.153422936374701e	-246
4.5767114681873503e	-246
2.2883557340936752e	-246
1.1441778670468376e	-246
5.720889335234188e	-247
2.860444667617094e	-247
1.430222333808547e	-247
7.151111669042735e	-248
3.5755558345213674e	-248

1.7877779172606837e	-248
8.938889586303419e	-249
4.4694447931517093e	-249
2.2347223965758547e	-249
1.1173611982879273e	-249
5.586805991439637e	-250
2.7934029957198183e	-250
1.3967014978599092e	-250
6.983507489299546e	-251
3.491753744649773e	-251
1.7458768723248864e	-251
8.729384361624432e	-252
4.364692180812216e	-252
2.182346090406108e	-252
1.091173045203054e	-252
5.45586522601527e	-253
2.727932613007635e	-253
1.3639663065038175e	-253
6.819831532519088e	-254
3.409915766259544e	-254
1.704957883129772e	-254
8.52478941564886e	-255
4.26239470782443e	-255
2.131197353912215e	-255
1.0655986769561075e	-255
5.327993384780537e	-256
2.6639966923902686e	-256
1.3319983461951343e	-256
6.659991730975672e	-257
3.329995865487836e	-257
1.664997932743918e	-257
8.32498966371959e	-258
4.162494831859795e	-258
2.0812474159298974e	-258
1.0406237079649487e	-258
5.2031185398247434e	-259
2.6015592699123717e	-259
1.3007796349561859e	-259
6.503898174780929e	-260
3.2519490873904646e	-260
1.6259745436952323e	-260
8.129872718476162e	-261
4.064936359238081e	-261
2.0324681796190404e	-261
1.0162340898095202e	-261
5.081170449047601e	-262
2.5405852245238005e	-262
1.2702926122619002e	-262
6.351463061309501e	-263
3.1757315306547506e	-263
1.5878657653273753e	-263
7.939328826636877e	-264
3.9696644133184383e	-264

1.9848322066592191e	-264
9.924161033296096e	-265
4.962080516648048e	-265
2.481040258324024e	-265
1.240520129162012e	-265
6.20260064581006e	-266
3.10130032290503e	-266
1.550650161452515e	-266
7.753250807262575e	-267
3.8766254036312874e	-267
1.9383127018156437e	-267
9.691563509078218e	-268
4.845781754539109e	-268
2.4228908772695546e	-268
1.2114454386347773e	-268
6.057227193173887e	-269
3.0286135965869433e	-269
1.5143067982934716e	-269
7.571533991467358e	-270
3.785766995733679e	-270
1.8928834978668395e	-270
9.464417489334198e	-271
4.732208744667099e	-271
2.3661043723335494e	-271
1.1830521861667747e	-271
5.915260930833874e	-272
2.957630465416937e	-272
1.4788152327084684e	-272
7.394076163542342e	-273
3.697038081771171e	-273
1.8485190408855855e	-273
9.242595204427927e	-274
4.621297602213964e	-274
2.310648801106982e	-274
1.155324400553491e	-274
5.776622002767455e	-275
2.8883110013837273e	-275
1.4441555006918637e	-275
7.220777503459318e	-276
3.610388751729659e	-276
1.8051943758648296e	-276
9.025971879324148e	-277
4.512985939662074e	-277
2.256492969831037e	-277
1.1282464849155185e	-277
5.641232424577593e	-278
2.8206162122887962e	-278
1.4103081061443981e	-278
7.051540530721991e	-279
3.5257702653609953e	-279
1.7628851326804976e	-279
8.814425663402488e	-280
4.407212831701244e	-280

2.203606415850622e	-280
1.101803207925311e	-280
5.509016039626555e	-281
2.7545080198132776e	-281
1.3772540099066388e	-281
6.886270049533194e	-282
3.443135024766597e	-282
1.7215675123832985e	-282
8.607837561916492e	-283
4.303918780958246e	-283
2.151959390479123e	-283
1.0759796952395615e	-283
5.379898476197808e	-284
2.689949238098904e	-284
1.344974619049452e	-284
6.72487309524726e	-285
3.36243654762363e	-285
1.681218273811815e	-285
8.406091369059075e	-286
4.2030456845295373e	-286
2.1015228422647686e	-286
1.0507614211323843e	-286
5.253807105661922e	-287
2.626903552830961e	-287
1.3134517764154804e	-287
6.567258882077402e	-288
3.283629441038701e	-288
1.6418147205193505e	-288
8.209073602596753e	-289
4.1045368012983762e	-289
2.0522684006491881e	-289
1.0261342003245941e	-289
5.1306710016229703e	-290
2.5653355008114852e	-290
1.2826677504057426e	-290
6.413338752028713e	-291
3.2066693760143564e	-291
1.6033346880071782e	-291
8.016673440035891e	-292
4.008336720017946e	-292
2.004168360008973e	-292
1.0020841800044864e	-292
5.010420900022432e	-293
2.505210450011216e	-293
1.252605225005608e	-293
6.26302612502804e	-294
3.13151306251402e	-294
1.56575653125701e	-294
7.82878265628505e	-295
3.914391328142525e	-295
1.9571956640712625e	-295
9.785978320356312e	-296
4.892989160178156e	-296

2.446494580089078e	-296
1.223247290044539e	-296
6.116236450222695e	-297
3.0581182251113476e	-297
1.5290591125556738e	-297
7.645295562778369e	-298
3.8226477813891845e	-298
1.9113238906945923e	-298
9.556619453472961e	-299
4.778309726736481e	-299
2.3891548633682403e	-299
1.1945774316841202e	-299
5.972887158420601e	-300
2.9864435792103004e	-300
1.4932217896051502e	-300
7.466108948025751e	-301
3.7330544740128755e	-301
1.8665272370064378e	-301
9.332636185032189e	-302
4.6663180925160944e	-302
2.3331590462580472e	-302
1.1665795231290236e	-302
5.832897615645118e	-303
2.916448807822559e	-303
1.4582244039112795e	-303
7.291122019556398e	-304
3.645561009778199e	-304
1.8227805048890994e	-304
9.113902524445497e	-305
4.5569512622227484e	-305
2.2784756311113742e	-305
1.1392378155556871e	-305
5.696189077778436e	-306
2.848094538889218e	-306
1.424047269444609e	-306
7.120236347223045e	-307
3.5601181736115222e	-307
1.7800590868057611e	-307
8.900295434028806e	-308
4.450147717014403e	-308
2.2250738585072014e	-308
1.1125369292536007e	-308
5.562684646268003e	-309
2.781342323134e	-309
1.390671161567e	-309
6.953355807835e	-310
3.4766779039175e	-310
1.73833895195875e	-310
8.691694759794e	-311
4.345847379897e	-311
2.1729236899484e	-311
1.086461844974e	-311
5.43230922487e	-312

2.716154612436e -312
1.35807730622e -312
6.7903865311e -313
3.39519326554e -313
1.69759663277e -313
8.487983164e -314
4.243991582e -314
2.121995791e -314
1.0609978955e -314
5.304989477e -315
2.65249474e -315
1.32624737e -315
6.63123685e -316
3.3156184e -316
1.6578092e -316
8.289046e -317
4.144523e -317
2.0722615e -317
1.036131e -317
5.180654e -318
2.590327e -318
1.295163e -318
6.4758e -319
3.2379e -319
1.61895e -319
8.095e -320
4.0474e -320
2.0237e -320
1.012e -320
5.06e -321
2.53e -321
1.265e -321
6.3e -322
3.16e -322
1.6e -322
8e -323
4e -323
2e -323
1e -323
5e -324
0.0

3. Fehlertypen

Wir diskutieren kurz unterschiedliche Arten von Fehlern, die beim numerischen Rechnen mit Computern und im Kontext von Messdaten auftreten können. Für dieses Modul sind die Darstellungs- und die Rechenfehler relevant. Diskretisierungsfehler und Messfehler werden in den höheren Semestern näher behandelt.

Darstellungsfehler Bei der Darstellung von Zahlen im Computer ergeben sich Rundungsfehler, weil nicht unendlich viel Speicher zur Verfügung steht. Stellt man die Zahl x im Computer dar, so wird effektiv auf die nächstgelegene exakt darstellbare Zahl $\text{rd}(x)$ gerundet. Der absolute Rundungsfehler beträgt $\text{rd}(x) - x$. Der relative Fehler

$$\frac{\text{rd}(x) - x}{x} \quad (8)$$

ist durch die **Maschinengenauigkeit** (*engl. machine epsilon / precision*) ε beschränkt. Diese hängt direkt mit der Anzahl Bits für die Mantisse r und der Basis b der Gleitkommadarstellung zusammen:

$$\left| \frac{\text{rd}(x) - x}{x} \right| \leq \varepsilon = \frac{1}{2} b^{1-r} \quad (9)$$

Für eine 64-bit Gleitkommazahlen (`float64`) ($b = 2, r = 52$) ergibt sich $\varepsilon \approx 10^{-16}$.

```
b, r = 2, 52
# b, r = 10, 16
b ** (1 - r) / 2
```

```
2.220446049250313e -16
```

Lernaufgabe 2: Genauigkeit der Mantisse Finde ähnlich wie bei Aufgabe 1 heraus, welches (ungefähr) die kleinste Gleitkommazahl ist, die von 1 unterschieden werden kann.

```
# Lösung
x = 1
while 1 + x > 1:
    x = x / 2
    print((1 + x, x))

(1.5, 0.5)
(1.25, 0.25)
(1.125, 0.125)
(1.0625, 0.0625)
(1.03125, 0.03125)
(1.015625, 0.015625)
(1.0078125, 0.0078125)
(1.00390625, 0.00390625)
(1.001953125, 0.001953125)
(1.0009765625, 0.0009765625)
(1.00048828125, 0.00048828125)
(1.000244140625, 0.000244140625)
(1.0001220703125, 0.0001220703125)
(1.00006103515625, 6.103515625e -05)
(1.000030517578125, 3.0517578125e -05)
(1.0000152587890625, 1.52587890625e -05)
(1.0000076293945312, 7.62939453125e -06)
(1.0000038146972656, 3.814697265625e -06)
(1.0000019073486328, 1.9073486328125e -06)
(1.0000009536743164, 9.5367431640625e -07)
(1.0000004768371582, 4.76837158203125e -07)
(1.000000238418579, 2.384185791015625e -07)
(1.0000001192092896, 1.1920928955078125e -07)
(1.0000000596046448, 5.960464477539063e -08)
(1.0000000298023224, 2.9802322387695312e -08)
(1.0000000149011612, 1.4901161193847656e -08)
(1.0000000074505806, 7.450580596923828e -09)
(1.0000000037252903, 3.725290298461914e -09)
```

```
(1.00000000018626451, 1.862645149230957e -09)
(1.0000000009313226, 9.313225746154785e -10)
(1.0000000004656613, 4.656612873077393e -10)
(1.0000000002328306, 2.3283064365386963e -10)
(1.0000000001164153, 1.1641532182693481e -10)
(1.0000000000582077, 5.820766091346741e -11)
(1.0000000000291038, 2.9103830456733704e -11)
(1.000000000014552, 1.4551915228366852e -11)
(1.000000000007276, 7.275957614183426e -12)
(1.000000000003638, 3.637978807091713e -12)
(1.000000000001819, 1.8189894035458565e -12)
(1.0000000000009095, 9.094947017729282e -13)
(1.0000000000004547, 4.547473508864641e -13)
(1.0000000000002274, 2.2737367544323206e -13)
(1.0000000000001137, 1.1368683772161603e -13)
(1.0000000000000568, 5.684341886080802e -14)
(1.0000000000000284, 2.842170943040401e -14)
(1.0000000000000142, 1.4210854715202004e -14)
(1.000000000000007, 7.105427357601002e -15)
(1.0000000000000036, 3.552713678800501e -15)
(1.0000000000000018, 1.7763568394002505e -15)
(1.0000000000000009, 8.881784197001252e -16)
(1.0000000000000004, 4.440892098500626e -16)
(1.0000000000000002, 2.220446049250313e -16)
(1.0, 1.1102230246251565e -16)
```

Noch ein Beispiel Die Dezimalzahl 0.1 kann nicht exakt im Binärsystem dargestellt werden. Das führt dazu, dass z.B. bei der Multiplikation mit 3 bereits ein Rundungsfehler entsteht:

```
0.1 * 3
```

```
0.30000000000000004
```

Der relative Fehler ist wieder durch $\varepsilon \approx 10^{-16}$ gegeben:

```
(0.1 * 3 - 0.3) / 0.3
```

```
1.8503717077085943e -16
```

Sehr wichtige Konsequenz: **Vergleiche von Gleitkommazahlen können gefährlich sein!**

```
0.1 * 3 == 0.3
```

```
False
```

Zur Sicherheit sollte man mit den absoluten Fehlern bzw. Toleranzen arbeiten, also z.B. $|x - y| \leq \varepsilon$ statt $x = y$.

```
tol = 1e -16
```

```
abs(0.1 * 3 - 0.3) < tol
```

```
True
```

Rechenfehler (Gleitkomma-Arithmetik) Das Resultat einer Rechnung mit Gleitkommazahlen im Computer führt nicht zwingend wieder zu einer exakt darstellbaren Gleitkommazahl. Es entstehen wieder Rundungsfehler. Die Addition und die Multiplikation sind dabei nicht problematisch, weil die relativen Fehler nur addiert werden.

Bei der Subtraktion zweier etwa gleich grosser Zahlen können die Rundungsfehler allerdings sehr relevant werden. Es ergibt sich das Phänomen der **Auslöschung** von korrekten Stellen.

Numerische Algorithmen sollten daher mit Blick auf die **Stabilität** entwickelt werden. Das heisst, es darf keine unkontrollierbare Verfälschung des Resultats durch Rundungsfehler der Gleitkomma-Arithmetik vorkommen. Dies ist ein wichtiger Bereich der **Numerik**, aber für diesen Kurs nicht so zentral.

Mehr zum Thema: Numerik 1, Wikiversity

0.2.2 NumPy & Matplotlib

Einleitung

Das Package NumPy stellt mehrdimensionale Arrays und Funktionen zum Rechnen mit diesen Arrays zur Verfügung. Matplotlib ist eine umfangreiche Programmbibliothek, um statische, animierte und interaktive Visualisierungen in Python zu erzeugen. Beide bilden die Basis für viele weiterführende Packages im Bereich der Data Sciences.

Im Modul *Programmieren* wurden die Grundlagen von NumPy bereits vermittelt. Zusätzlich zu diesen Unterlagen gibt es im Internet sehr viele Tutorials zu Numpy. Diese drei Websites erklären die Grundlagen recht gut:

- “Python für Naturwissenschaftler”, Gert-Ludwig Ingold
- “Einführung in die wissenschaftliche Programmierung mit Python”, Meik Hellmund
- “NumPy Tutorial”, Bernd Klein

Und diese Serie von Videos ist zu empfehlen:

- “Data Science in Python”, The Morpheus Tutorials

Das Package Matplotlib bildet die Basis für Visualisierung in Python. Andere Packages wie Seaborn bauen darauf auf, weshalb es wichtig ist, die Grundlagen von Matplotlib zu beherrschen. Die Dokumentation unter <https://matplotlib.org/> ist sehr ausführlich und eine erste Übersicht findet man hier:

- Plot types
- Quick start guide

Hier ist nicht das Ziel, alle Funktionalitäten von Numpy und Matplotlib durchzugehen, sondern anhand von Beispielen die Möglichkeiten kennenzulernen, so dass man sich später selbst mithilfe der Dokumentation und weiteren Tutorials zurechtfindet.

Zunächst müssen wir die beiden Packages importieren.

```
import numpy as np
import matplotlib.pyplot as plt
```

Im folgenden schauen wir folgende Themen anhand von Beispielen an:

- Arrays erzeugen
- Daten importieren und exportieren
- Funktionen darstellen
- Shape und Reshaping
- Indizieren (Slicing)
- Filtern
- Rechnen mit Arrays
- Lineare Algebra
- Matrizen darstellen
- Zufallszahlen erzeugen

Arrays erzeugen

Übersicht Einige hilfreiche Funktionen für das Erzeugen von Arrays sind:

Funktion	Resultat
<code>np.array(iterable)</code>	Array aus einem Iterable (Liste, Tuple, ect.)
<code>np.empty(shape)</code> und <code>np.empty_like(arr)</code>	Leeres Array mit angegebener Shape ¹ bzw. gleicher Shape wie ein anderes Array <code>arr</code> . Die Werte werden nicht initialisiert, so dass einfach die Zahlen drin stehen, die im wiederverwendeten Speicher bereits drin standen. Das ist ideal, wenn die Werte auf jeden Fall später neu gesetzt werden.
<code>np.zeros(shape)</code> und <code>np.zeros_like(arr)</code>	Array mit lauter Nullen (0).
<code>np.ones(shape)</code> und <code>np.ones_like(arr)</code>	Array mit lauter Einsen (1).
<code>np.arange(start=0, stop, step=1)</code>	Array mit fortlaufenden Zahlen mit gegebenem Abstand (<code>step</code>), analog zu <code>range()</code>
<code>np.linspace(start, stop, num=50)</code>	Array mit gleichmässig verteilten Zahlen mit gegebener Anzahl (<code>num</code>)
<code>np.logspace(start, stop, num=50)</code>	Array mit logarithmisch gleichmässigem verteilten Zahlen (Exponenten sind linear)

Hier ist die komplette Übersicht in der Dokumentation.

¹ Die *Shape* eines Arrays ist die Anzahl der Elemente in jeder Dimension. Ein Array mit Shape (3, 4) hat zum Beispiel 3 Zeilen und 4 Spalten.

Beispiele

```
a = np.array([4, 7, 2])
a
array([4, 7, 2])
np.empty_like(a)
array([4607182418800017408, 4607182418800017408, 4607182418800017408])
np.ones((2, 3))
array([[1., 1., 1.],
       [1., 1., 1.]])
np.arange(2, 10, 2)
array([2, 4, 6, 8])
```

Daten importieren und exportieren

Hier ist die Übersicht: <https://numpy.org/doc/stable/reference/routines.io.html>.

Um kommaseparierte Werte (CSV) zu lesen und schreiben verwenden wir `np.loadtxt` und `np.savetxt`. Effizientere und speichersparende Methoden existieren, sind hier aber nicht Thema.

Beim Import muss das Dateiformat zuerst angeschaut werden.

```
bac = np.loadtxt("bacteria_nutrient.csv", delimiter=";", skiprows=1)
bac
array([[1.00e+00, 1.00e+02, 1.00e+03],
       [2.00e+00, 1.50e+02, 9.50e+02],
       [3.00e+00, 2.50e+02, 9.00e+02],
       [4.00e+00, 4.00e+02, 8.50e+02],
```

```
[5.00e+00, 6.00e+02, 8.00e+02],
[6.00e+00, 9.00e+02, 7.50e+02],
[7.00e+00, 1.20e+03, 7.00e+02],
[8.00e+00, 1.60e+03, 6.50e+02],
[9.00e+00, 1.90e+03, 6.00e+02],
[1.00e+01, 2.00e+03, 5.50e+02],
[1.10e+01, 2.10e+03, 5.00e+02],
[1.20e+01, 2.05e+03, 4.50e+02],
[1.30e+01, 2.00e+03, 4.00e+02],
[1.40e+01, 1.80e+03, 3.50e+02],
[1.50e+01, 1.60e+03, 3.00e+02],
[1.60e+01, 1.40e+03, 2.50e+02],
[1.70e+01, 1.20e+03, 2.00e+02],
[1.80e+01, 1.00e+03, 1.50e+02],
[1.90e+01, 8.00e+02, 1.00e+02],
[2.00e+01, 7.00e+02, 5.00e+01]]])
```

```
np.savetxt("out.csv", bac)
```

Wenn nur gewisse Spalten benutzt werden sollen, können diese mit dem Parameter `usecols` ausgewählt werden. (Erste Spalte hat Index 0!)

```
weight = np.loadtxt("weight.csv", delimiter=";", skiprows=1, usecols=(1, 2))
weight
```

```
array([[ 0. , 74.3],
       [ 3. , 74.3],
       [ 4. , 74.3],
       [ 5. , 73.5],
       [ 6. , 73.3],
       [ 7. , 73.6],
       [ 8. , 74.1],
       [10. , 73.9],
       [11. , 73.9],
       [12. , 74.2],
       [13. , 74.1],
       [14. , 73.8],
       [16. , 73.8],
       [18. , 74.6],
       [19. , 74.3],
       [20. , 73.8],
       [21. , 73.6],
       [23. , 73.7],
       [25. , 73.4],
       [26. , 74.4],
       [27. , 73.8],
       [29. , 73.9],
       [30. , 74. ],
       [31. , 73.6],
       [32. , 73.6],
       [33. , 73.7],
       [34. , 73.5],
       [35. , 74. ],
       [36. , 73.7],
```

```

[39. , 74.3],
[40. , 73.3],
[41. , 73. ],
[42. , 74. ],
[43. , 73.6],
[44. , 73.7],
[45. , 74. ],
[46. , 73.7],
[47. , 74. ],
[48. , 73.3],
[52. , 74.6],
[54. , 74.9],
[55. , 74.9],
[56. , 74. ],
[57. , 75.6],
[58. , 75.6],
[59. , 75.1],
[60. , 75.4],
[61. , 76.3],
[62. , 74.8],
[63. , 74.8],
[64. , 74.8],
[65. , 74.2],
[66. , 75.2],
[67. , 74.9],
[68. , 74.7],
[69. , 74.5],
[70. , 74.4],
[72. , 74.3],
[74. , 74.4],
[75. , 74.6],
[77. , 74.2],
[78. , 73.3],
[80. , 73.5],
[81. , 74.4],
[82. , 74.7],
[83. , 74.3],
[84. , 73.2],
[85. , 74.3],
[86. , 73.7],
[87. , 74.1],
[89. , 73.6],
[90. , 73.6],
[92. , 73.9],
[94. , 73.7],
[95. , 74.4],
[96. , 73.9]]))

```

Funktionen darstellen

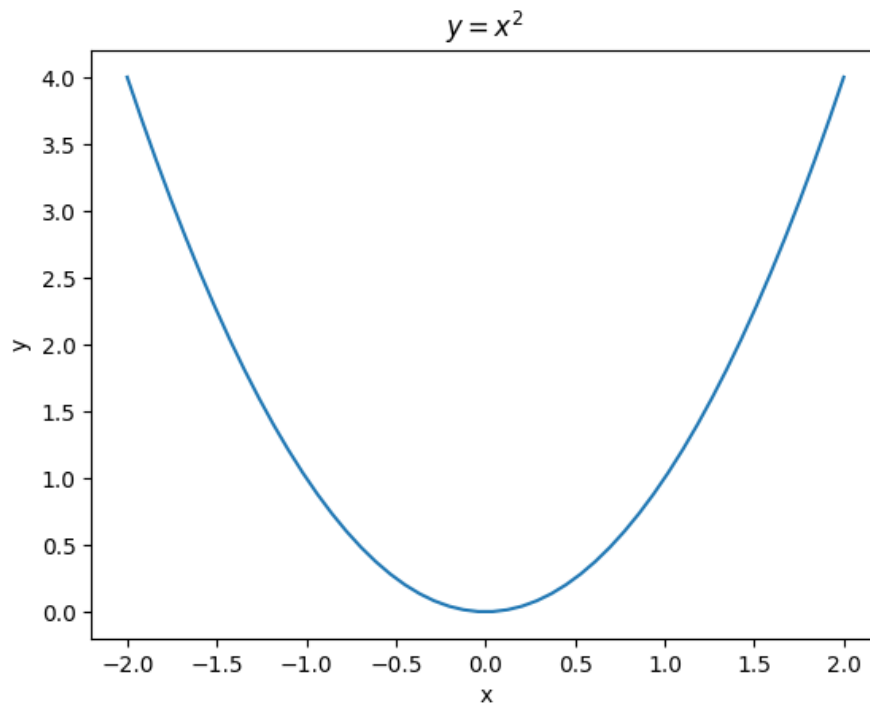
Einfaches Beispiel Wir wollen die Funktion $f(x) = x^2$ im Intervall $x \in [-2, 2]$ darstellen. Dazu erzeugen wir mit `np.linspace` ein Array `x` mit 50 Werten im Intervall $[-2, 2]$ und ein Array `y` mit den Funktionswerten $f(x)$.

Weiter beschriften wir die Achsen und fügen einen Diagrammtitel hinzu. Beachte die Verwendung

von LaTeX-Mathe im Titel.

```
x = np.linspace( -2, 2)
y = x**2

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("$y = x^2$")
plt.show()
```



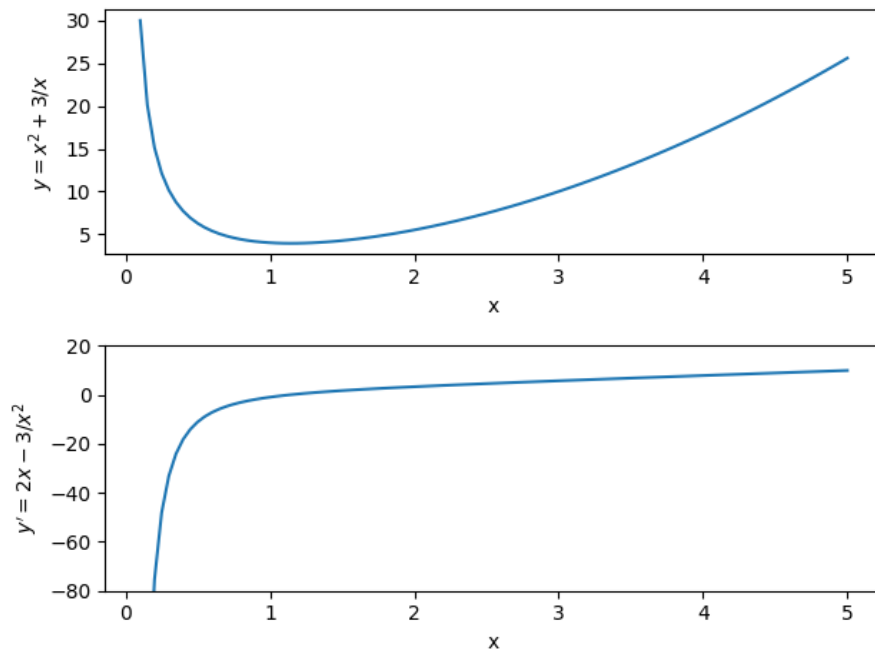
Etwas umfangreicheres Beispiel Nun stellen wir die Funktion $y(x) = x^2 + 3/x$ und ihre Ableitung $y'(x) = 2x - 3/x^2$ in zwei untereinanderliegenden Koordinatensystemen dar.

Dazu wird `plt.subplots` verwendet, um die beiden Koordinatensysteme `ax0` und `ax1` (Axes Objekte) zu erzeugen.

Statt `plt.ylabel` und `plt.ylim` etc. brauchen wir jetzt `ax0.set_ylabel` und `ax0.set_ylim`.

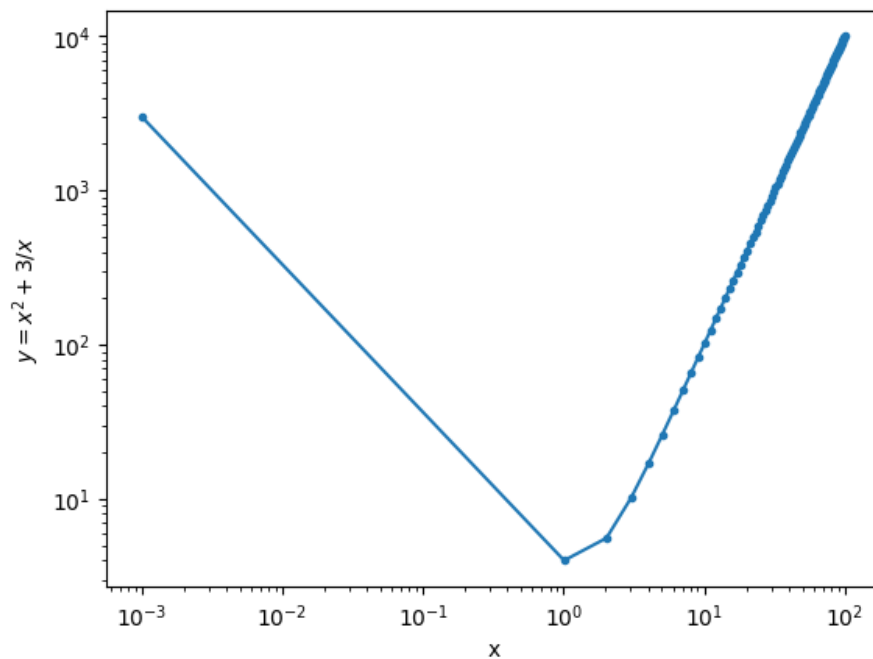
```
x = np.linspace(0.1, 5, num=100)
y = x**2 + 3 / x
yprime = 2 * x - 3 / x**2

fig, (ax0, ax1) = plt.subplots(2, sharex=False)
ax0.plot(x, y)
ax0.set_xlabel("x")
ax0.set_ylabel("$y = x^2 + 3/x$")
ax1.plot(x, yprime)
ax1.set_ylim([ -80, 20])
ax1.set_xlabel("x")
ax1.set_ylabel("$y' = 2x - 3/x^2$")
plt.tight_layout()
plt.show()
```

Logarithmische Darstellung Darstellung in logarithmischer Skala mit `plt.xscale("log")` und `plt.yscale("log")`, oder direkt mit `plt.loglog`.

```
x = np.linspace(0.001, 100, num=100)
y = x**2 + 3 / x
plt.plot(x, y, ". -")
plt.xscale("log")
plt.yscale("log")
# plt.loglog(x, y)
plt.xlabel("x")
plt.ylabel("$y = x^2 + 3/x$")
plt.show()
```



Für logarithmische Skalen bietet sich die Berechnung der Werte mit multiplikativem Abstand an, also mit `np.logspace`.

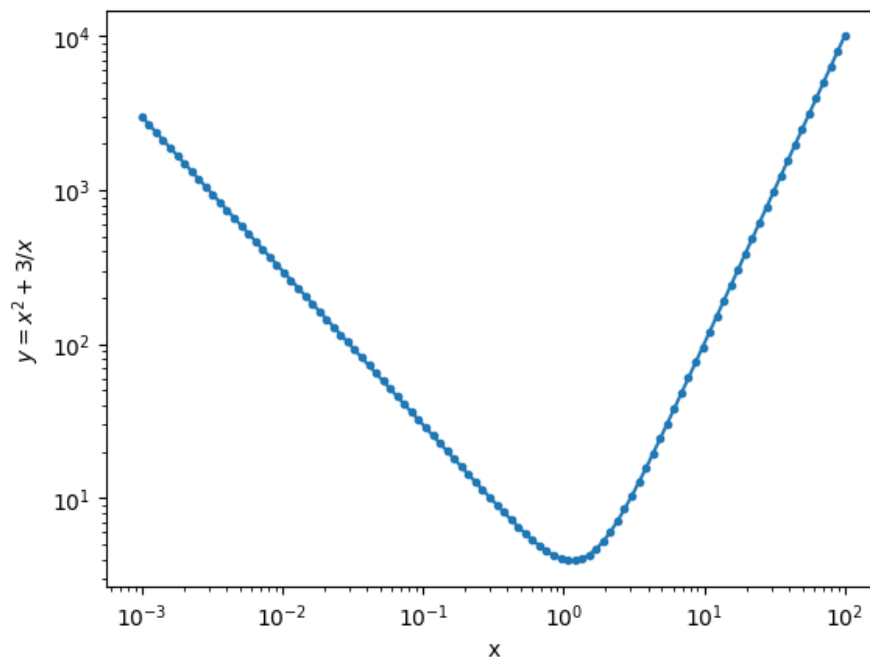
```

np.logspace( -3, 2, num=6)

array([1.e -03, 1.e -02, 1.e -01, 1.e+00, 1.e+01, 1.e+02])

x = np.logspace( -3, 2, num=100)
y = x**2 + 3 / x
plt.plot(x, y, ". -")
plt.xscale("log")
plt.yscale("log")
# plt.loglog(x, y)
plt.xlabel("x")
plt.ylabel("$y = x^2 + 3/x$")
plt.show()

```



Shape und Reshaping

```

# Shape
a = np.arange(50)
a.shape

(50,)

# Reshape in eines 25 x 2 Matrix
a.reshape(25, 2)

array([[ 0,  1],
       [ 2,  3],
       [ 4,  5],
       [ 6,  7],
       [ 8,  9],
       [10, 11],
       [12, 13],
       [14, 15],
       [16, 17],
       [18, 19],

```

```
[20, 21],
[22, 23],
[24, 25],
[26, 27],
[28, 29],
[30, 31],
[32, 33],
[34, 35],
[36, 37],
[38, 39],
[40, 41],
[42, 43],
[44, 45],
[46, 47],
[48, 49]])
```

Reshape verändert das Array selbst nicht, sondern gibt eine neue Sicht auf das alte Array zu
a.shape

```
(50,)
```

Reshape in eine 5 x 10 Matrix.

Die -1 bestimmt hier automatisch, wie gross diese Dimension sein muss, damit es aufgeht.

```
b = a.reshape(5, -1)
```

```
print(b.shape)
```

```
print(b)
```

```
(5, 10)
```

```
[[ 0  1  2  3  4  5  6  7  8  9]
 [10 11 12 13 14 15 16 17 18 19]
 [20 21 22 23 24 25 26 27 28 29]
 [30 31 32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47 48 49]]
```

Transponieren

```
bt = b.T
```

```
print(bt)
```

```
[[ 0 10 20 30 40]
 [ 1 11 21 31 41]
 [ 2 12 22 32 42]
 [ 3 13 23 33 43]
 [ 4 14 24 34 44]
 [ 5 15 25 35 45]
 [ 6 16 26 36 46]
 [ 7 17 27 37 47]
 [ 8 18 28 38 48]
 [ 9 19 29 39 49]]
```

Reshape und Transpose erzeugen nur Sichten (view) auf das ursprüngliche Array!

Sonst np.copy verwenden.

```
bt[0, 0] = 101
```

```
print(bt)
```

```
print(b)
```

```
print(a)
```

```

[[101  10  20  30  40]
 [  1  11  21  31  41]
 [  2  12  22  32  42]
 [  3  13  23  33  43]
 [  4  14  24  34  44]
 [  5  15  25  35  45]
 [  6  16  26  36  46]
 [  7  17  27  37  47]
 [  8  18  28  38  48]
 [  9  19  29  39  49]]
[[101  1  2  3  4  5  6  7  8  9]
 [ 10 11 12 13 14 15 16 17 18 19]
 [ 20 21 22 23 24 25 26 27 28 29]
 [ 30 31 32 33 34 35 36 37 38 39]
 [ 40 41 42 43 44 45 46 47 48 49]]
[101  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
 36 37 38 39 40 41 42 43 44 45 46 47 48 49]

```

Indizieren (Slicing)

```

x = np.array([[3, 5, 1], [6, 7, 9]])
print(x)

[[3 5 1]
 [6 7 9]]

# Genau ein Element extrahieren
x[0, 1]

np.int64(5)

# Zeilen extrahieren (weil ein 2D -Array and Array von Zeilen -Arrays ist)
z0, z1 = x
z0

array([3, 5, 1])

# Spalten extrahieren (zuerst transponieren, damit die Spalten zu Zeilen werden)
s0, s1, s2 = x.T
s0

array([3, 6])

    Etwas direkter mit Slice-Notation

# alle Zeilen (:), 0 -te Spalte
x[:, 0]

array([3, 6])

# 1 -te Zeile, alle Spalten bis und ohne die 2 -te
x[1, :2]

array([6, 7])

```

```

y = np.arange(50).reshape(5, -1)
y

array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
       [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
       [40, 41, 42, 43, 44, 45, 46, 47, 48, 49]])

# Von : Bis -ohne
y[1:3, 2:6]

array([[12, 13, 14, 15],
       [22, 23, 24, 25]])

# Von : Bis -ohne : Schrittweite
y[1:5:2, 0:7:3]

array([[10, 13, 16],
       [30, 33, 36]])

# von Beginn / bis Ende
y[:3, :4]

array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23]])

# von Hinten
y[-1, -2]

np.int64(48)

```

Filtern

```

# Wo sind Werte grösser als 3? => Boolean Array
x > 3

array([[False,  True, False],
       [ True,  True,  True]])

# Filter als Index verwenden, um die Werte zu erhalten, die den Filter erfüllen.
x[x > 3]

array([5, 6, 7, 9])

# Dies kann verwendet werden, um Werte an diesen Stellen zu ändern.
x[x > 3] = -1
x

array([[ 3, -1,  1],
       [-1, -1, -1]])

# Suchen und ersetzen mit np.where lässt x unverändert
np.where(x < 0, 100, x), x

```

```
(array([[ 3, 100,  1],
        [100, 100, 100]]),
array([[ 3, -1,  1],
        [-1, -1, -1]]))

# Gefilterte Werte durch Werte aus einem anderen Array (y) ersetzen.
y = np.full_like(x, 8)
np.where(x < 0, y, x)

array([[3, 8, 1],
       [8, 8, 8]])
```

Rechnen mit Arrays

List-Comprehension vs. NumPy NumPy ist viel schneller

```
l = list(range(1000))
%timeit [x**2 + 3 * x for x in l]
```

```
a = np.arange(1000)
%timeit a**2 + 3 * a
```

```
26.9 \mus \pm 1.13 \mus per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
1.33 \mus \pm 27.8 ns per loop (mean \pm std. dev. of 7 runs, 1,000,000 loops each)
```

Verschiedene Produkte NumPy bietet verschiedene Produkte für Arrays an, siehe hier. Bei Vektoren und Matrizen sind einige davon äquivalent, bei höherdimensionalen Arrays gibt es Unterschiede. Hier sollte man immer die Dokumentation konsultieren und kleine Tests machen, um sicherzustellen, dass das Gewünschte passiert.

```
a = np.arange(1, 4)
b = np.arange(0, 3)
a, b

(array([1, 2, 3]), array([0, 1, 2]))

print("elementweise:\n", a * b)
print("@ (matmul):\n", a @ b)
print("dot:\n", a.dot(b))
print("inner:\n", np.inner(a, b))
print("outer(a):\n", np.outer(a, b))

elementweise:
[0 2 6]
@ (matmul):
8
dot:
8
inner:
8
outer(a):
[[0 1 2]
 [0 2 4]
 [0 3 6]]
```

```

A = np.array([[1, 2], [3, 4]])
B = np.array([[1, 3], [1, 2]])

print("elementweise:\n", A * B)
print("@ (matmul):\n", A @ B)
print("dot:\n", A.dot(B))
print("inner:\n", np.inner(A, B))
print("@ (matmul) mit B^T:\n", A @ B.T)
print("outer(a):\n", np.outer(A, B))

elementweise:
[[1 6]
 [3 8]]
@ (matmul):
[[ 3  7]
 [ 7 17]]
dot:
[[ 3  7]
 [ 7 17]]
inner:
[[ 7  5]
 [15 11]]
@ (matmul) mit B^T:
[[ 7  5]
 [15 11]]
outer(a):
[[ 1  3  1  2]
 [ 2  6  2  4]
 [ 3  9  3  6]
 [ 4 12  4  8]]

```

- Das Matrixprodukt `np.matmul` kann kurz mit dem `@`-Operator geschrieben werden. Bei 1-D Arrays (Vektoren) entspricht es dem Skalarprodukt, bei 2-D Arrays (Matrizen) der Matrixmultiplikation. Für höhere Dimensionen sollte man in der Dokumentation nachschauen, was genau passiert.
- Das “dot”-Produkt `np.dot` entspricht für Vektoren und Matrizen dem Matrixprodukt `np.matmul`, aber für höhere Dimensionen ist es etwas anders definiert, siehe Dokumentation.
- Das innere Produkt `np.inner` entspricht bei zwei Vektoren dem Skalarprodukt und bei zwei Matrizen der Matrizenmultiplikation mit der Transponierten: $A \cdot B^T$. Für gemischte und höhere Dimensionen ist es wieder leicht anders definiert.
- Das äussere Produkt `np.outer` ist das Kreuzprodukt, bekannt aus dem Modul *Daten & Information*.

Lineare Algebra

NumPy deckt viele Bereiche der numerischen linearen Algebra ab, siehe <https://numpy.org/doc/stable/reference/routines.linalg.html>. Hier die wichtigsten Beispiele.

```

# Einheitsmatrix erzeugen
e3 = np.eye(3)
e3

array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])

```

```

# Diagonalmatrix erzeugen
d = np.diag([5, 6, 2])
d

array([[5, 0, 0],
       [0, 6, 0],
       [0, 0, 2]])

# Matrizen multiplizieren
a = np.array(
    [
        [1, 0],
        [0, 1],
        [2, 2],
    ]
)
print("matmul:\n", np.matmul(d, a))
print("@ -Operator (bevorzugen):\n", d @ a)

matmul:
[[5 0]
 [0 6]
 [4 4]]
@ -Operator (bevorzugen):
[[5 0]
 [0 6]
 [4 4]]

# Determinante
b = np.array([[2, 3], [1, 1]])
np.linalg.det(b)

np.float64( -1.0)

# Inverses
np.linalg.inv(b)

array([[ -1.,  3.],
       [ 1., -2.]])

# Eigenwerte und Eigenvektoren
eigenvalues, eigenvectors = np.linalg.eig(b)
# eigenvalues, eigenvectors = np.linalg.eig(np.diag((1, 2, -4)))
print("Eigenwerte:\n", eigenvalues)
print("Eigenvektoren:\n", eigenvectors)

Eigenwerte:
[ 3.30277564 -0.30277564]
Eigenvektoren:
[[ 0.91724574 -0.79325185]
 [ 0.3983218  0.60889368]]

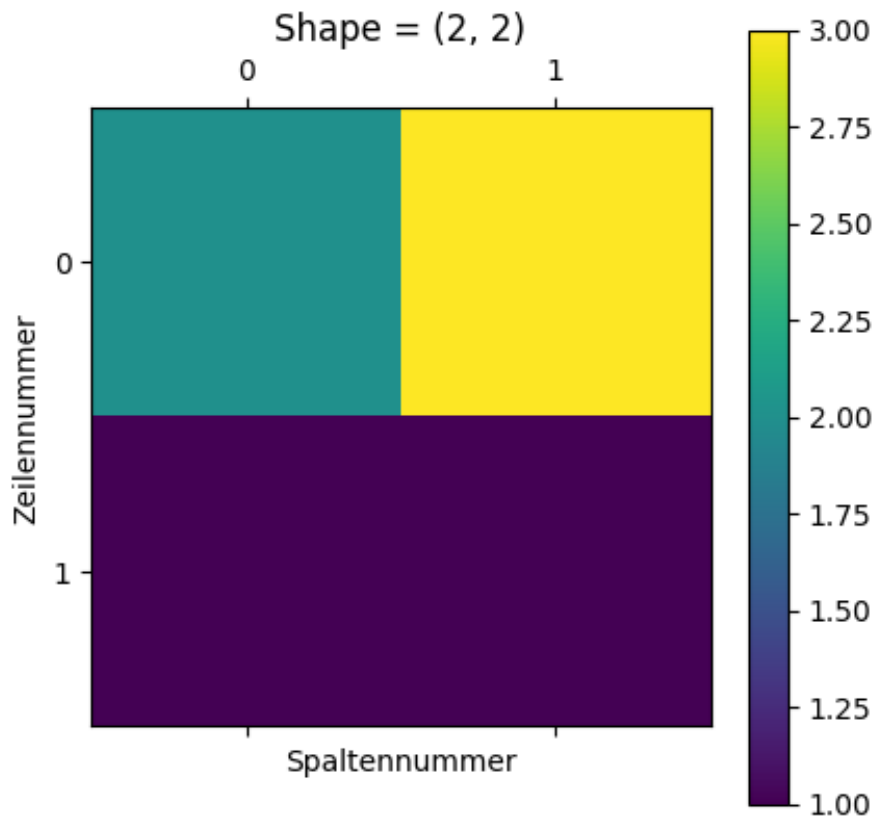
# Die Spalten der Matrix v sind die Eigenvektoren zu den Eigenwerten im 1d -Array w:
for i, lam in enumerate(eigenvalues):
    print(f"Eigenwert {i} ist {lam} mit Eigenvektor {eigenvectors.T[i]}")

Eigenwert 0 ist 3.3027756377319943 mit Eigenvektor [0.91724574 0.3983218 ]
Eigenwert 1 ist -0.3027756377319948 mit Eigenvektor [ -0.79325185  0.60889368]

```


Matrizen darstellen

```
plt.matshow(b)
plt.colorbar()
plt.xlabel("Spaltennummer")
plt.ylabel("Zeilennummer")
plt.title(f"Shape = {b.shape}")
plt.show()
```



Zufallszahlen erzeugen

Übersicht zu Pseudo-Zufallszahlen: <https://numpy.org/doc/stable/reference/random/index.html>

Zuerst mit `np.random.default_rng(seed)` einen Generator initialisieren. Der **Seed** bestimmt, wie der Generator initialisiert wird. Die Zufallszahlen sind dann **reproduzierbar**.

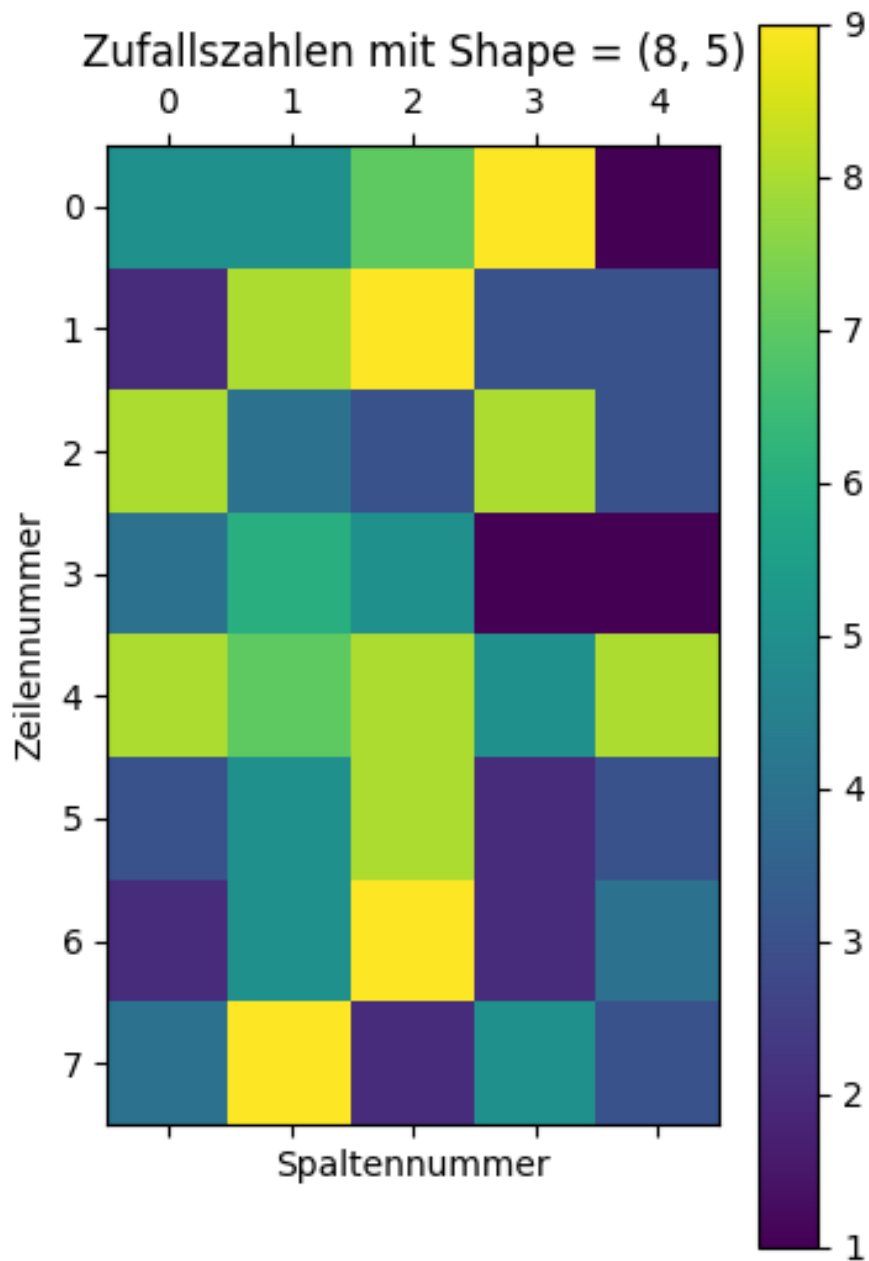
```
rng = np.random.default_rng(seed=1)
```

Ganze Zahlen: Integers

```
r = rng.integers(1, 10, size=(8, 5))
print(r)
```

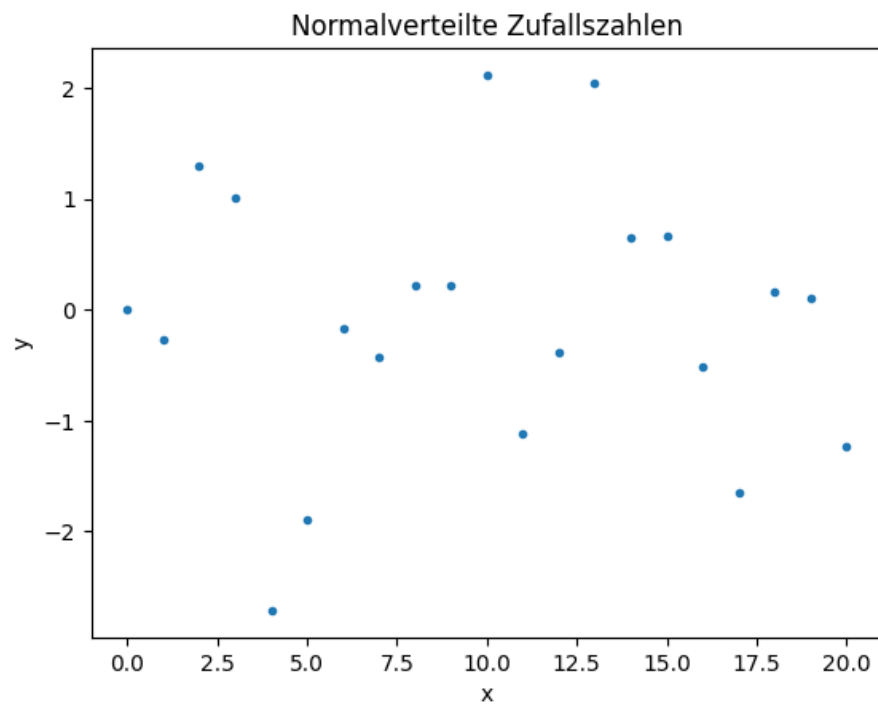
```
[[5 5 7 9 1]
 [2 8 9 3 3]
 [8 4 3 8 3]
 [4 6 5 1 1]
 [8 7 8 5 8]
 [3 5 8 2 3]
 [2 5 9 2 4]
 [4 9 2 5 3]]
```

```
plt.matshow(r)
plt.colorbar()
plt.xlabel("Spaltennummer")
plt.ylabel("Zeilennummer")
plt.title(f"Zufallszahlen mit Shape = {r.shape}")
plt.show()
```



Normalverteilte Zahlen

```
x = np.arange(21)
y = rng.normal(size=x.shape)
plt.plot(x, y, ".")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Normalverteilte Zufallszahlen")
plt.show()
```



0.2.3 NumPy & Matplotlib

Übungen - Lösungen

Für das Einlesen von Daten verwenden wir die Funktion `np.loadtxt` und für die Darstellung meist `plt.plot`, `plt.step` oder `plt.stem`, siehe Beispiele in den Notebooks `01_Numpy_Matplotlib.ipynb`, `01_Grundlagen.ipynb` und `01_Advance_Organizer.ipynb`.

Weiter geben die Tutorials von Matplotlib einen Überblick über die Möglichkeiten, Daten und Funktionen darzustellen.

Bevor wir NumPy und Matplotlib verwenden können, müssen wir diese Bibliotheken importieren. Es reicht, dies zu Beginn des Notebooks zu tun.

```
import numpy as np
import matplotlib.pyplot as plt
```

Aufgabe 1: Daten importieren und darstellen

1.1 Temperatur in Davos a) Daten importieren

Die Datei `davos_temperatur.txt` beinhaltet drei Spalten. Die erste gibt das Kürzel einer Wetterstation an, die zweite einen Zeitstempel (Datum und Stunde: `YYYYMMDDHH`) und die dritte die gemessene Temperatur.

Importiere die dritte Spalte in ein Array namens `temperature`. Gib den Shape des Arrays und die ersten 5 Elemente aus.

```
temperature = np.loadtxt("davos_temperatur.txt", delimiter=";", skiprows=1, usecols=2)
print("shape:", temperature.shape)
print(temperature[:5])
```

```
shape: (48,)
[ -1.5 -2.2 -2.5 -2.8 -2.6]
```

b) Daten darstellen

- Stelle den Temperaturverlauf mit einem Liniendiagramm grafisch dar.
 - Als Zeitachse (horizontal) soll die Anzahl Stunden nach Messbeginn benutzt werden. Diese entspricht einfach dem Index des Temperaturarrays.
- Schränke den Bereich der Zeitachse auf die Stunden 6 - 24 ein. Benutze dazu die Funktion `plt.xlim`.
- Beschrifte die Achsen und gib dem Diagramm einen Titel.

```
# Messzeitpunkte erstellen
time = np.arange(0, len(temperature))

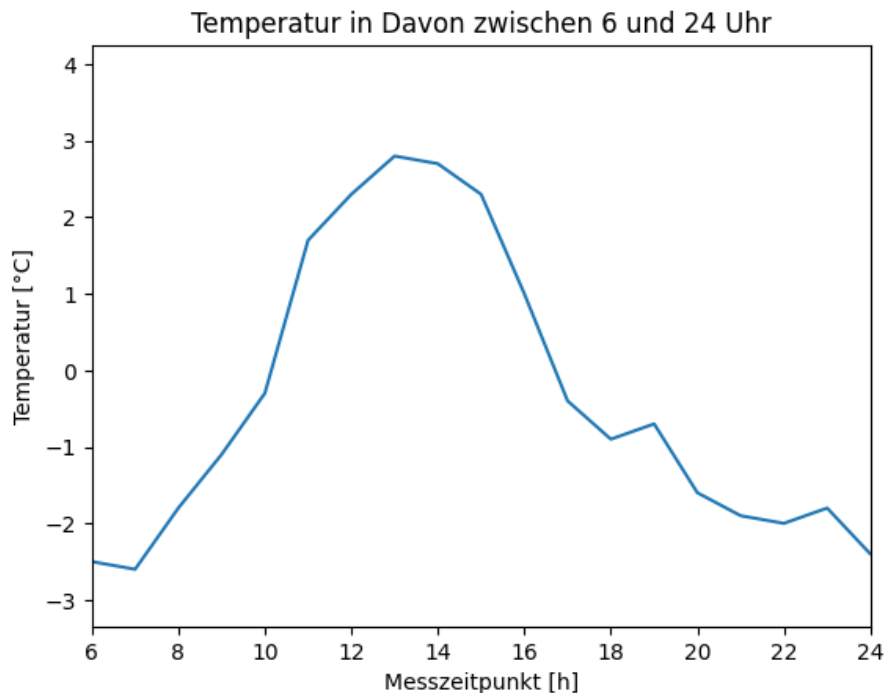
# Temperaturen plotten
plt.plot(time, temperature)

# x -Achse einschränken
plt.xlim(6, 24)

# Achsenbeschriftungen setzen
plt.xlabel("Messzeitpunkt [h]")
plt.ylabel("Temperatur [°C]")

# Titel setzen
plt.title("Temperatur in Davos zwischen 6 und 24 Uhr")
```

```
# Diagramm anzeigen
plt.show()
```



1.2 Bevölkerungsentwicklung von Irland a) Daten importieren

- Importiere die Daten in der Datei `ireland.csv` in ein Array namens `ire_data`. Gib die Shape des Arrays und die ersten 8 Zeilen aus.
- Übertrage die beiden Spalten in entsprechend benannte 1D-Arrays (`year`, `population`).
 - Dazu kannst du entweder Slicing verwenden
 - oder das Array zuerst transponieren und dann die Zeilen in zwei 1-D Arrays “entpacken” mit `x, y = array.T`.
- Bestimme das Minimum und Maximum der Jahreszahlen.
- Bestimme die kleinste, die grösste und die durchschnittliche Bevölkerungszahl.

```
ire_data = np.loadtxt("ireland.csv", delimiter=",", skiprows=1)
print("shape:", ire_data.shape)
print(ire_data[:8])
```

```
shape: (71, 2)
[[1950.    2.91]
 [1951.    2.92]
 [1952.    2.93]
 [1953.    2.92]
 [1954.    2.9 ]
 [1955.    2.89]
 [1956.    2.87]
 [1957.    2.85]]
```

```
# Daten in zwei separate Arrays aufteilen
year, population = ire_data.T
```

```
# Alternative: Slicing
# year = ire_data[:,0]
# population = ire_data[:,1]
```

```

year.min(), year.max()

(np.float64(1950.0), np.float64(2020.0))

population.min(), population.max(), population.mean()

(np.float64(2.81), np.float64(4.95), np.float64(3.551267605633803))

```

b) Daten darstellen

- Stelle die Bevölkerungsentwicklung von Irland grafisch dar.
 - Verwende dazu verbundene kleine Punkte und
 - wähle eine eigene Farbe.
- Schränke den Darstellungsbereich auf Min/Max der Jahreszahlen ein, so dass der Graph bis an die äusseren Ränder des Diagrams geht. Nutze dazu wieder `plt.xlim`.
- Füge mit `plt.grid` ein Gitternetz hinzu.
- Beschrifte die Achsen und gib dem Plot einen Titel.

```

# Plot der Populationsdaten
plt.plot(year, population, "g. -")

# Bereich einschränken
plt.xlim(year.min(), year.max())

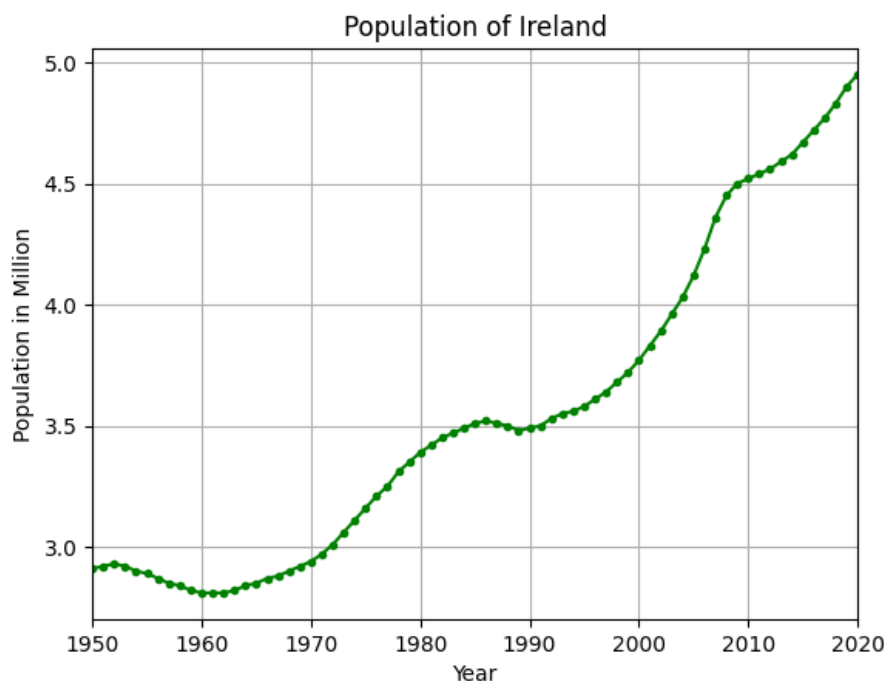
# Achsenbeschriftungen setzen
plt.xlabel("Year")
plt.ylabel("Population in Million")

# Titel setzen
plt.title("Population of Ireland")

# Gitternetz hinzufügen
plt.grid()

# Diagramm anzeigen
plt.show()

```



1.3 Bakterien a) Daten importieren

- Importiere die Daten in der Datei `bacteria_nutrient.csv` in ein Array namens `bac_data`.
- Gib den Shape des Arrays und die ersten 5 Zeilen aus.
- Wie viele Beobachtungen sind es und welche Variablen sind in den Spalten zu finden?
- Übertrage die Spalten in entsprechend benannte 1-D Arrays.

```
# Daten aus der CSV -Datei laden
```

```
bac_data = np.loadtxt("bacteria_nutrient.csv", delimiter=",", skiprows=1)
print("shape:", bac_data.shape)
print(bac_data[:5, :])
```

```
shape: (20, 3)
[[ 1.  100. 1000.]
 [ 2.  150.  950.]
 [ 3.  250.  900.]
 [ 4.  400.  850.]
 [ 5.  600.  800.]
```

```
# Anzahl Beobachtungen
```

```
print(f"n = {bac_data.shape[0]}")
```

```
n = 20
```

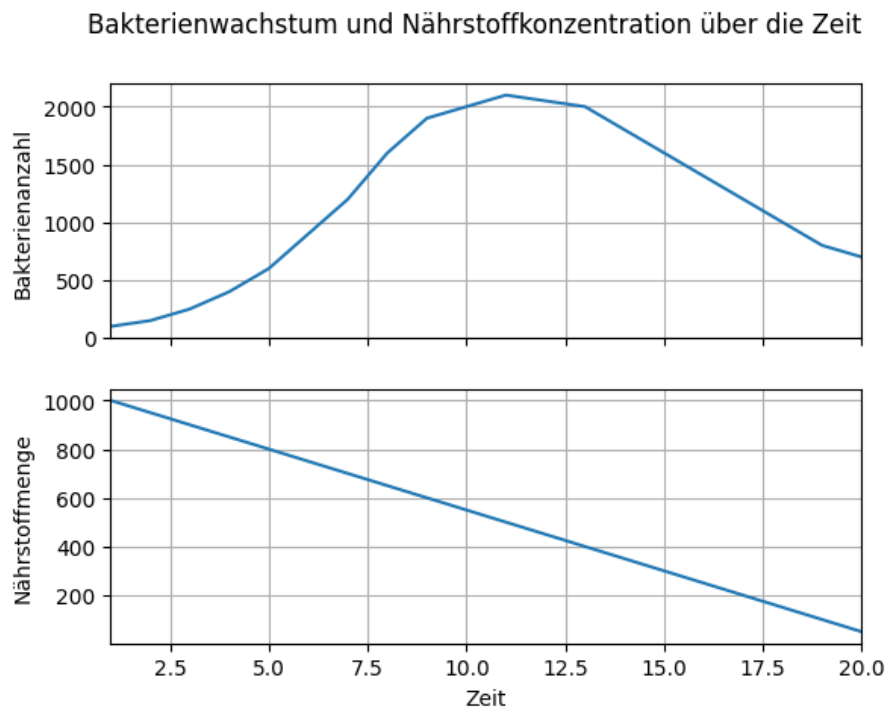
```
# Daten in drei 1 -D Arrays aufteilen
```

```
time, bacteria, nutrient = bac_data.T
```

b) Bakterienanzahl und Nährstoffmenge zusammen darstellen

- Stelle den Verlauf der Bakterienanzahl und der Nährstoffmenge über die Zeit dar. Erzeuge dazu mit `plt.subplots` zwei übereinanderliegende Koordinatensysteme. Stelle die Bakterienanzahl im oberen und die Nährstoffmenge im unteren dar.
- Beschrifte die Achsen und gib dem gesamten Diagramm einen Titel (diesmal mit `plt.suptitle`).
- Schränke den Darstellungsbereich auf Min/Max der Zeitwerte ein. Benutze dazu `ax.set_xlim`.
- Füge in beiden Koordinatensystemen ein Gitternetz hinzu mit `ax.grid`.

```
fig, (ax0, ax1) = plt.subplots(2, 1, sharex=True)
ax0.plot(time, bacteria)
ax0.set_ylabel("Bakterienanzahl")
ax0.grid()
ax1.plot(time, nutrient)
ax1.set_ylabel("Nährstoffmenge")
ax1.set_xlabel("Zeit")
ax1.grid()
ax1.set_xlim(time.min(), time.max())
plt.suptitle("Bakterienwachstum und Nährstoffkonzentration über die Zeit")
plt.show()
```



c) Hinweis

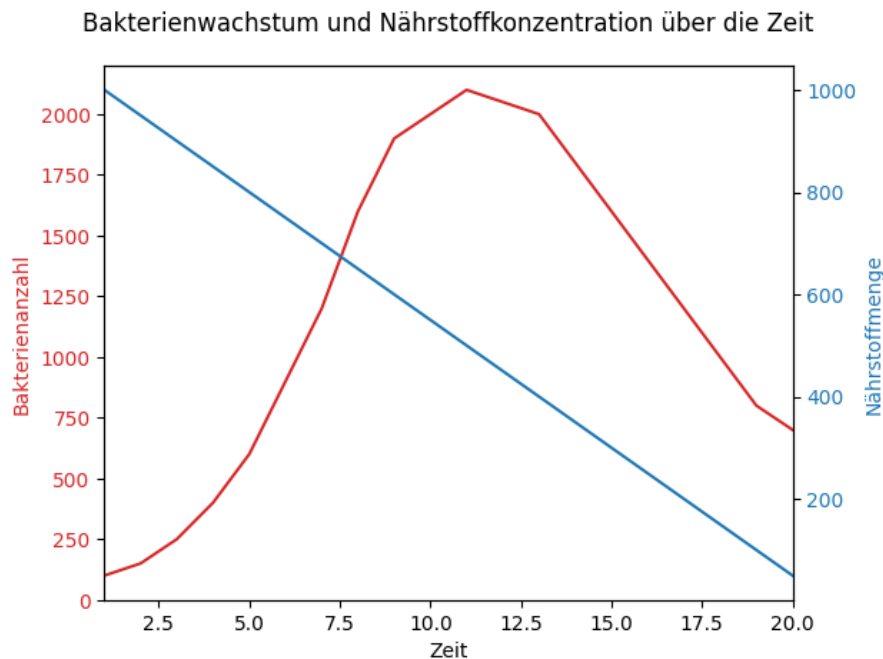
Zwei Graphen unterschiedlicher Einheiten (wie Bakterienanzahl und Nährstoffmenge) sollten wenn möglich nicht im gleichen Diagramm (Koordinatensystem) dargestellt werden. Die Variante mit den zwei Subplots wie oben ist vorzuziehen. Hier wäre zur Vollständigkeit noch eine Variante mit zwei y-Achsen (links und rechts).

```
fig, ax1 = plt.subplots()

# Plot der Bakterienanzahl
color = "tab:red"
ax1.plot(time, bacteria, color=color)
ax1.tick_params(axis="y", labelcolor=color)
ax1.set_ylabel("Bakterienanzahl", color=color)
ax1.set_xlabel("Zeit")
ax1.set_xlim(time.min(), time.max())

# Zweites Koordinatensystem für Nährstoffmenge darüberlegen
color = "tab:blue"
ax2 = ax1.twinx()
ax2.plot(time, nutrient, color=color)
ax2.tick_params(axis="y", labelcolor=color)
ax2.set_ylabel("Nährstoffmenge", color=color)

plt.suptitle("Bakterienwachstum und Nährstoffkonzentration über die Zeit")
plt.tight_layout()
plt.show()
```

Aufgabe 2: Matrizen und Eigenwerte

2.1 Mehrdimensionale Arrays erstellen a) Erstelle ein 1D Array mit den Zahlen ganzen Zahlen -3 bis 4 (mit `np.arange`) und forme es in ein 2D Array mit 2 Zeilen und 4 Spalten um (mit `np.reshape`). Gib das 2D Array aus.

```
a = np.arange(-3, 5).reshape(2, 4)
a
array([[ -3, -2, -1,  0],
       [  1,  2,  3,  4]])
```

b) Erstelle ein 1D Array mit den natürlichen Zahlen 1 bis 12 und forme es in ein 3D Array mit 2 Zeilen, 3 Spalten und 2 Ebenen um. Gib das 3D Array aus.

```
b = np.arange(1, 13).reshape(2, 3, 2)
b
array([[[ 1,  2],
        [ 3,  4],
        [ 5,  6]],
       [[ 7,  8],
        [ 9, 10],
        [11, 12]])])
```

2.2 Matrizen darstellen a) Matrix erzeugen

- Erzeuge ein 1D Array mit den natürlichen Zahlen 1 bis 16 und forme es in ein 2D Array mit 4 Zeilen und 4 Spalten um. Gib die 4×4 Matrix aus.

```
m = np.arange(1, 17).reshape(4, 4)
m
```

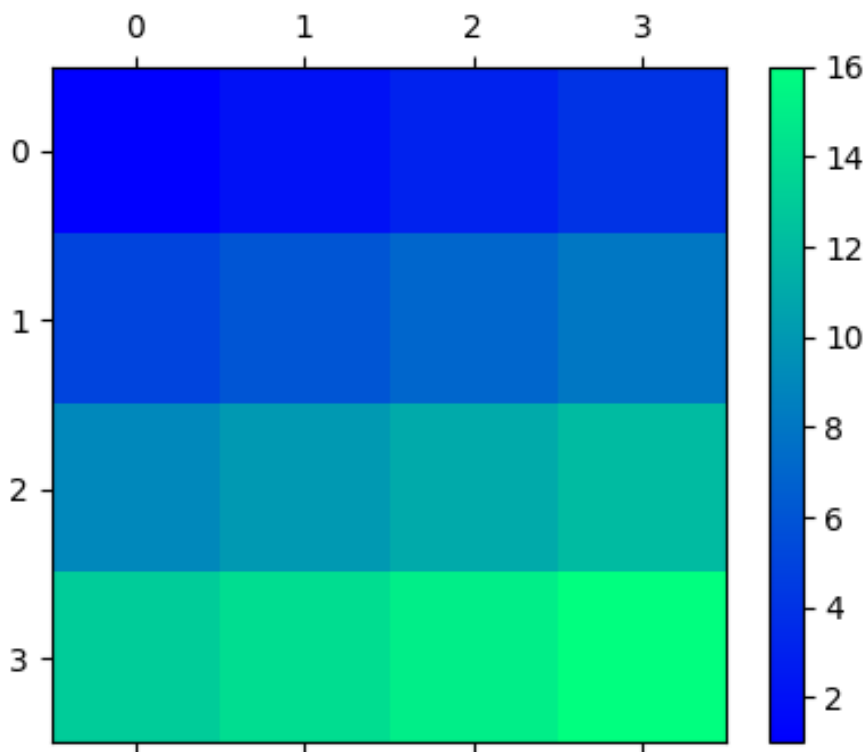
```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16]])
```

b) Matrix darstellen

- Stelle die 4×4 Matrix aus Teil a) als Bild dar. Nutze dazu die Funktion `plt.matshow`.
- Füge eine Farbskala hinzu mit `plt.colorbar`.

Schaue jeweils in die Dokumentation der Funktionen, um zu sehen, welche Argumente sie erwarten und welche Optionen es gibt. Probiere Optionen aus, um zu sehen, was sie bewirken.

```
plt.matshow(m, cmap="winter")
plt.colorbar(shrink=0.8)
plt.show()
```



2.3 Eigenwerte und Eigenvektoren

a) Berechne die Eigenwerte und Eigenvektoren der Matrix

$$\begin{pmatrix} 1 & 2 \\ 1 & 0 \end{pmatrix} \quad (10)$$

Benutze dazu die Funktion `np.linalg.eig`. Welche Eigenwerte und Eigenvektoren gehören jeweils zusammen?

```
evals, evecs = np.linalg.eig([[1, 2], [1, 0]])
```

```
for i, l in enumerate(evals):
    print(f"Eigenwert {i+1}: {l:.3}, \t Eigenvektor {i+1}: {evecs[:,i]}")
```

```
Eigenwert 1: 2.0, Eigenvektor 1: [0.89442719 0.4472136 ]
```

```
Eigenwert 2: -1.0, Eigenvektor 2: [ -0.70710678  0.70710678]
```

b) berechne die Eigenwerte und Eigenvektoren der 4×4 Matrix aus Aufgabe 5a).

```
evals, evecs = np.linalg.eig(m)

for i, l in enumerate(evals):
    print(f"Eigenwert {i+1}: {l:.3f},\t\tEigenvektor {i+1}: {evecs[:,i]}")
```

Eigenwert 1: 36.2, Eigenvektor 1: [-0.15115432 -0.34923733 -0.54732033 -0.74540333]
 Eigenwert 2: -2.21, Eigenvektor 2: [0.72704996 0.28320876 -0.16063243 -0.60447363]
 Eigenwert 3: -2.71e -15, Eigenvektor 3: [0.48456983 -0.83640291 0.21909634 0.13273674]
 Eigenwert 4: 1.34e -16, Eigenvektor 4: [-0.05497296 -0.33288957 0.83069804 -0.4428355]

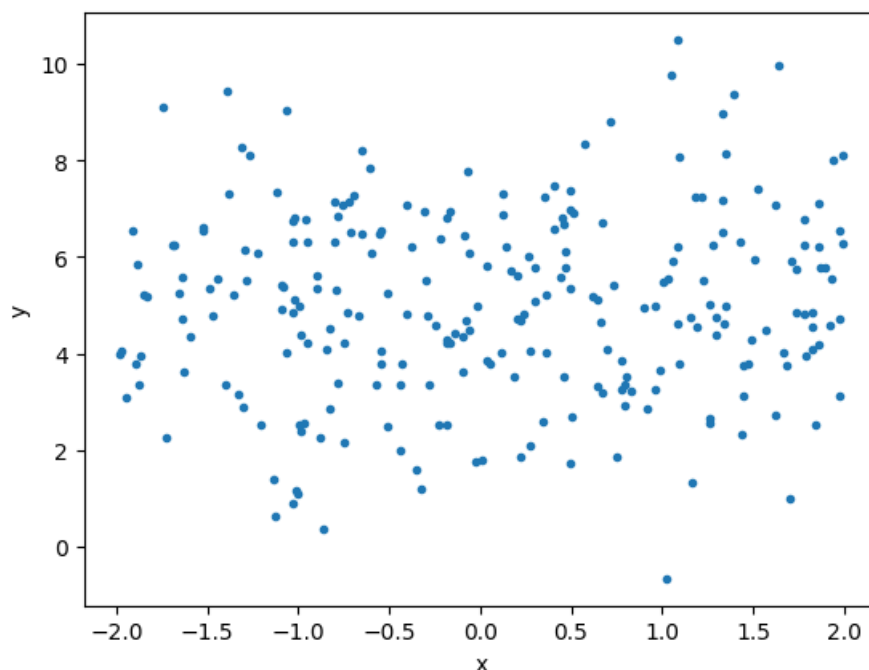
Aufgabe 3: Daten simulieren (Zufallszahlen)

3.1 Zufallszahlen erzeugen

- Erstelle einen Zufallszahlengenerator mit `np.random.default_rng()` mit einem selbstgewählten Seed.
- Erzeuge mit `numpy.random.Generator.uniform` 250 **gleichverteilte** Zufallszahlen zwischen -2 und 2. Speichere sie im Array `x`.
- Erzeuge mit `numpy.random.Generator.normal` 250 **normalverteilte** Zufallszahlen mit Mittelwert 5 und Standardabweichung 2. Speichere sie im Array `y`.
- Stelle `(x,y)` als Punktwolke dar. Nutze dazu die Funktion `plt.scatter`.

```
rng = np.random.default_rng(987234)
x = rng.uniform( -2, 2, 250)
y = rng.normal(5, 2, 250)

plt.scatter(x, y, marker=".")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



3.2 Daten mit Messfehlern (Rauschen, Noise) simulieren Wir betrachten die folgende Sinusfunktion als Signal

$$y = 2 \sin(2\pi t) + 10 \quad (11)$$

und addieren ein zufälliges Rauschen dazu.

a) Signal erzeugen

Definiere für die Zeit `t` mit `np.linspace` ein Array von 0 bis 5. Dann berechne die Funktion `y` in diesem Bereich.

```
t = np.linspace(0, 5)
y = 2 * np.sin(2 * np.pi * t) + 10
```

b) Rauschen hinzufügen

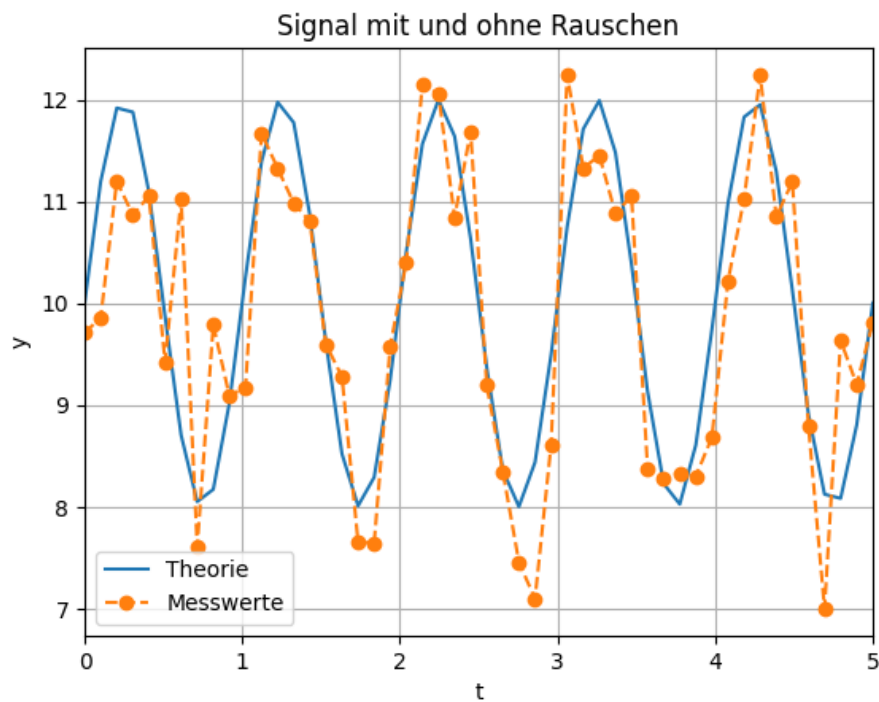
- Erzeuge normalverteilte Messfehler (Zufallszahlen) mit Mittelwert 0 und Standardabweichung 0.8 in einem Array namens `noise`.
- Addiere die Messfehler zu `y` und speichere die simulierten Daten in `noisy`.

```
rng = np.random.default_rng(12345678)
noise = rng.normal(0, 0.8, size=y.shape)
noisy = y + noise
```

c) Signal mit und ohne Rauschen darstellen

- Stelle `y` und `noisy` über die Zeit im gleichen Diagramm dar.
- Die Theorie `y` soll mit einer verbundenen Linie dargestellt werden, die Messwerte `noisy` nur als Punkte.
- Füge mit `plt.legend` eine Legende hinzu.
- Schreibe Achsen an.

```
plt.plot(t, y, label="Theorie")
plt.plot(t, noisy, " - o", label="Messwerte")
plt.xlabel("t")
plt.ylabel("y")
plt.xlim(t.min(), t.max())
plt.legend()
plt.grid()
plt.title("Signal mit und ohne Rauschen")
plt.show()
```



d) Daten exportieren

Speichere die drei Variablen `t`, `y` und `noisy` in eine CSV-Datei `noisy.csv` ab.

- Für das Zusammenfügen mehrerer Spalten in ein 2-D Array gibt es die Funktion `np.column_stack`.
- Benutze `np.savetxt` für den Datenexport.
 - Benutze Strichpunkte als Trennzeichen, damit die Datei auch problemlos in einem deutschsprachigen Excel geöffnet werden kann.
 - Schaue in der Dokumentation nach, wie man in der ersten Zeile der CSV-Datei Spaltenüberschriften hinzufügen kann.

```
data = np.column_stack((t, y, noisy))
np.savetxt("noisy.csv", data, delimiter=";", header="Zeit;Theorie;Messwert")
```

0.3 Funktionen Darstellen

0.3.1 Darstellung von Funktionen mit mehreren Variablen

Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Hier lernst du die Darstellung von Funktionen von mehreren Variablen. Das Ziel ist, Funktionen von mehreren Variablen mithilfe von Matplotlib darstellen zu können. Details und Verfeinerungen folgen in der Vertiefung.

Funktionen mit mehreren Variablen Viele Größen in naturwissenschaftlichen und technischen Kontexten hängen von mehreren Variablen ab:

- Die Temperatur in einem Raum hängt von der 3D-Position (x, y, z) ab, oder auch noch von der Zeit (x, y, z, t) .
- Das Wachstum einer Zellkultur hängt nicht nur von der Zeit t ab, sondern auch von der Menge der vorhandenen Nährstoffe, vom pH-Wert der Umgebung und wohl noch von anderen Variablen.

So gibt es auch in der Mathematik Funktionen, die von mehreren Variablen abhängen. Häufig heissen die unabhängigen Variablen x , y und z , können aber auch mit einem Index durchnummeriert ($x_1, x_2, \dots$) oder in einem Vektor $\vec{x} \in \mathbb{R}^n$ zusammengepackt werden. Ein Beispiel:

$$f(x, y) = x^2 - y^2 \quad (12)$$

Das heisst, jedem Paar (x, y) wird der Wert $x^2 - y^2$ zugeordnet, z.B.

$$f(4, 5) = 4^2 - 5^2 = 16 - 25 = -9 \quad (13)$$

Das Wertepaar (x, y) kann dabei als Vektor oder Punkt in der Ebene verstanden werden: $\vec{x} = (x, y)$. Der Funktionswert kann als Höhe einer Landschaft verstanden werden, die über der Ebene liegt.

Bitte schau dir das folgende Video zur Einführung an:

- “Funktionen mit mehreren Veränderlichen - Einführung”, Georg Hoever

Partielle Funktionen (Isolinien) Wertet man eine Funktion von (x, y) für einen spezifischen, konstanten Wert für x aus und lässt y weiterhin variabel, so führt dies zu einer *partiellen Funktion* von y :

$$p_3(y) = f(3, y) = 3^2 - y^2 = 9 - y^2 \quad (14)$$

$$p_5(y) = f(5, y) = 5^2 - y^2 = 25 - y^2 \quad (15)$$

Funktion für konstantes y auswerten führt zu einer *partiellen Funktion* von x :

$$q_2(x) = f(x, 2) = x^2 - 2^2 = x^2 - 4 \quad (16)$$

$$q_4(x) = f(x, 4) = x^2 - 4^2 = x^2 - 16 \quad (17)$$

Die partiellen Funktionen sind Schnitte entlang konstanter Werte für alle bis auf eine der unabhängigen Variablen. Diese werden auch *Isolinien* genannt.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def f(x, y):
    return x**2 - y**2
```

```

y = np.linspace( -2, 2)
p3 = f(3, y)
p5 = f(5, y)

x = np.linspace( -2, 2)
q2 = f(x, 2)
q4 = f(x, 4)

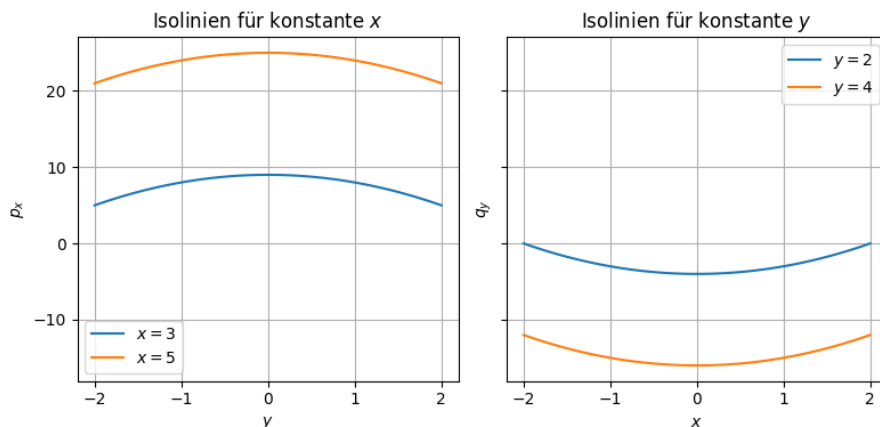
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 4), sharey=True)

ax1.plot(y, p3, label="$x=3$")
ax1.plot(y, p5, label="$x=5$")
ax1.set_xlabel("$y$")
ax1.set_ylabel("$p_x$")
ax1.legend()
ax1.grid()
ax1.set_title("Isolinien für konstante $x$")

ax2.plot(x, q2, label="$y=2$")
ax2.plot(x, q4, label="$y=4$")
ax2.set_xlabel("$x$")
ax2.set_ylabel("$q_y$")
ax2.legend()
ax2.grid()
ax2.set_title("Isolinien für konstante $y$")

plt.tight_layout()
plt.show()

```



Höhenlinien Grafische Darstellungen sind hilfreich, wenn wir mathematische Zusammenhänge richtig verstehen wollen. Bei Funktionen einer Variablen wird einfach der Funktionsgraph im für die Anwendung relevanten Bereich dargestellt.

Hängt eine Funktion von zwei Variablen ab, stellt man meist die Linien konstanter Funktionswerte (also Höhe über der Koordinatenebene = Höhenlinien) dar, manchmal auch eine Fläche im Raum. Bei drei und mehr Variablen ist dies leider nicht mehr möglich.

Schau dir eines der folgenden Videos für die Einführung in die Darstellung von Funktionen von mehreren Variablen an:

- "Hohenlinien/Niveaulinien + Beispiel + 3D Herleitung Einfach Erklärt!", Mathe Peter
- "Höhenlinien, Gradient, Vektorfeld, Vektoranalysis, Unimathematik, Erklärvideo", Daniel Jung

- “Höhenlinien einer Funktion”, Steffen Habermalz

Mach dir ein paar Notizen und schreibe zu klärende Fragen auf.

Optional: Weitere Grundlegende Information findest du hier unter Wikipedia: Isolinie und Spektrum: Isolinien.

Grundsätzliches Vorgehen Es sollen Linien konstanter Funktionswerte z in der x - y -Ebene dargestellt werden.

1. Die Funktionsgleichung $z = f(x, y)$ entweder nach x oder nach y auflösen, je nach dem was einfacher ist. Falls es mehrere Lösungen gibt, z.B. beim Wurzelziehen, dann müssen alle diese Lösungen dargestellt werden.
2. Bereich in x (bzw. y) bestimmen, in dem die Funktion dargestellt werden soll. Dieser hängt stark vom Kontext ab. Array mit x -Werten (bzw. y -Werten) definieren.
3. Zugehörige y -Werte (bzw. x -Werte) für eine Serie von z -Werten berechnen und darstellen.

Notiz Achsenbeschriftungen sind kein nettes Feature sondern ein grundlegender Bestandteil eines Diagramms. Ein sauber dargestellter und aussagekräftiger Graph wird in diesem Kurs erwartet.

Arbeitsauftrag

Aufgabe 1: Per Hand 1. a) Was genau sind Höhenlinien? Nenne ein paar Beispiele für ihre Anwendung.

Beispiele:

- Isobaren (Linien mit konstantem Luftdruck)
- Wanderkarten (Höhenlinien = Linien konstanter Meter über Meer)
- Isoquanten: Produktionsarbeit als Funktion von Arbeit und Kapital

1. b) Reflektiere nochmals das grundsätzliche Vorgehen für die Darstellung von Höhenlinien und notiere dieses hier in eigenen Worten.

1. c) Betrachten wir mal die Funktion

$$z = f(x, y) = 2x^2 + y \quad (18)$$

Funktionsgleichung nach y auflösen:

$$y_z(x) = z - 2x^2 \quad (19)$$

Die Notation $y_z(x)$ soll darauf hinweisen, dass z festgehalten wird und somit y_z nur noch eine Funktion von x ist.

Für die Werte $x \in [0, 3]$ soll die Höhenlinie für $z = 1$ dargestellt werden. Probiere dies zuerst mit Stift und Papier. Nutze hierfür eine Wertetabelle mit 4 x -Werten, die Du hier mithilfe von NumPy oder einer List-Comprehension ausrechnest.

Wertetabelle mit List -Comprehension:

```
z = 1
```

```
[(x, z - 2 * x**2) for x in range(4)]
```

```
[(0, 1), (1, -1), (2, -7), (3, -17)]
```

Wertetabelle mit NumPy:

```
x = np.arange(4)
```

```
y = z - 2 * x**2
```

```
print("      x,   y")
```

```
np.column_stack((x, y))
```

```
x, y
array([[ 0,  1],
       [ 1, -1],
       [ 2, -7],
       [ 3, -17]])
```

In Tabellenform sieht das also so aus:

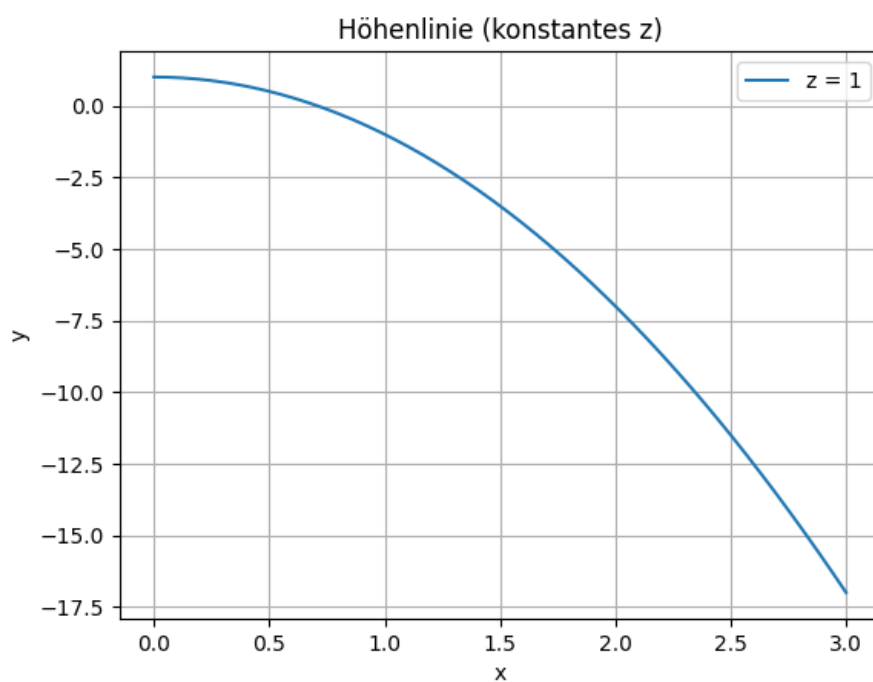
x	y
0	1
1	-1
2	-7
3	-17

Aufgabe 2: Darstellung mit Matplotlib 2. a) Stelle die Höhenlinie zu $z = 1$ im Bereich $x \in [0, 3]$ mit Matplotlib dar.

```
def y_z(z, x):
    return z - 2 * x**2

x = np.linspace(0, 3)
y = y_z(1, x)

plt.plot(x, y, label="z = 1")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.title("Höhenlinie (konstantes z)")
plt.show()
```



2. b) Nun machen wir das gleiche für mehrere z -Werte, sagen wir $z \in \{-5, -3, 0, 3, 5\}$ (**levels**), und stellen diese zusammen in einem Diagramm dar.

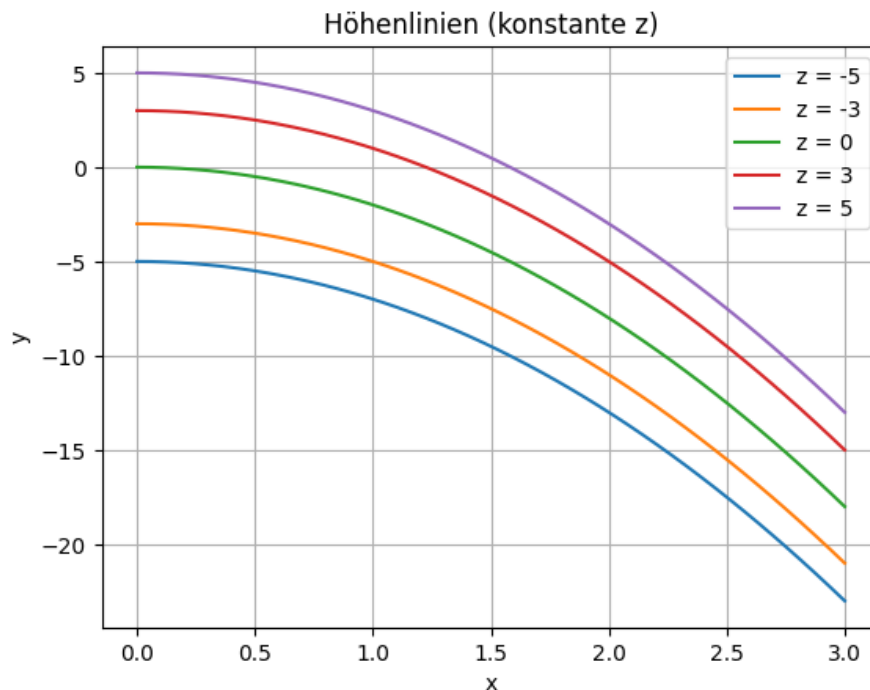
Füge eine Legende mit den zugehörigen Werten von z hinzu.

Aufgabe 2b:

```
levels = (-5, -3, 0, 3, 5)
x = np.linspace(0, 3)

for z in levels:
    y = y_z(z, x)
    plt.plot(x, y, label=f"z = {z}")

plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.title("Höhenlinien (konstante z)")
plt.show()
```



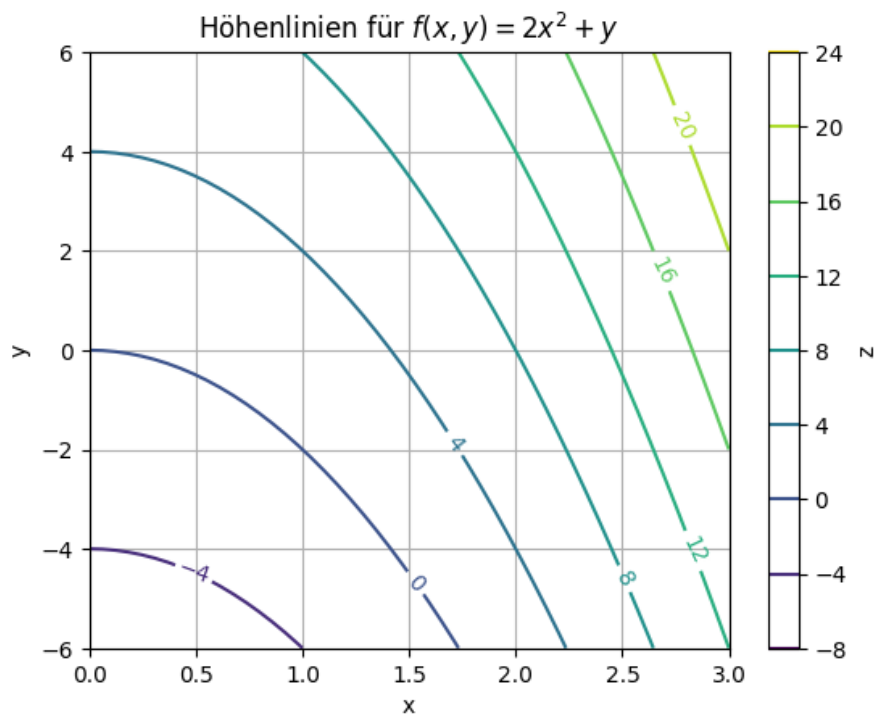
Aufgabe 3: Erstes Beispiel mit `plt.contour` Meist kennt man anstatt der Funktionswerte eher den Bereich in der x - y -Ebene, für den man sich interessiert. Oder man kann die Funktion nicht von Hand nach einer der Variablen auflösen. Oder man hat Daten statt eine analytische Funktion gegeben. Zudem könnte es schwierig sein, die z -Werte (Levels) selbst zu wählen. In all diesen Fällen hilft einem die Funktion `plt.contour`.

Schau dir das erste Beispiel in der Contour Demo an.

Stelle nun die Höhenlinien der Funktion $f(x, y)$ aus Aufgabe 1 im Bereich $x \in [0, 3]$ und $y \in [-6, 6]$ mithilfe von `plt.contour` dar inkl. Beschriftung der Achsen und der Höhenlinien.

```
x = np.linspace(0, 3)
y = np.linspace(-6, 6)
X, Y = np.meshgrid(x, y)
Z = 2 * X**2 + Y
```

```
cs = plt.contour(X, Y, Z)
plt.clabel(cs)
plt.colorbar(label="z")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.title("Höhenlinien für  $f(x, y) = 2x^2 + y$ ")
plt.show()
```



Warum unterscheidet sich diese Darstellung von der in Aufgabe 2?

1. Die dargestellten z -Werte (Levels) sind nicht dieselben. Wir könnten diese mit der Option `levels` in `plt.contour` beeinflussen.
2. Der dargestellte Bereich in y ist nicht der gleiche.

0.3.2 Darstellung von Funktionen mit mehreren Variablen

Übungen - Lösungen

```
import numpy as np
import matplotlib.pyplot as plt
```

Aufgabe 1 Betrachten wir die Funktion

$$g(x, y) = x \cdot e^{-x^2 - y^2} + \frac{1}{6}y \quad (20)$$

im Bereich $x, y \in [-2, 2]$.

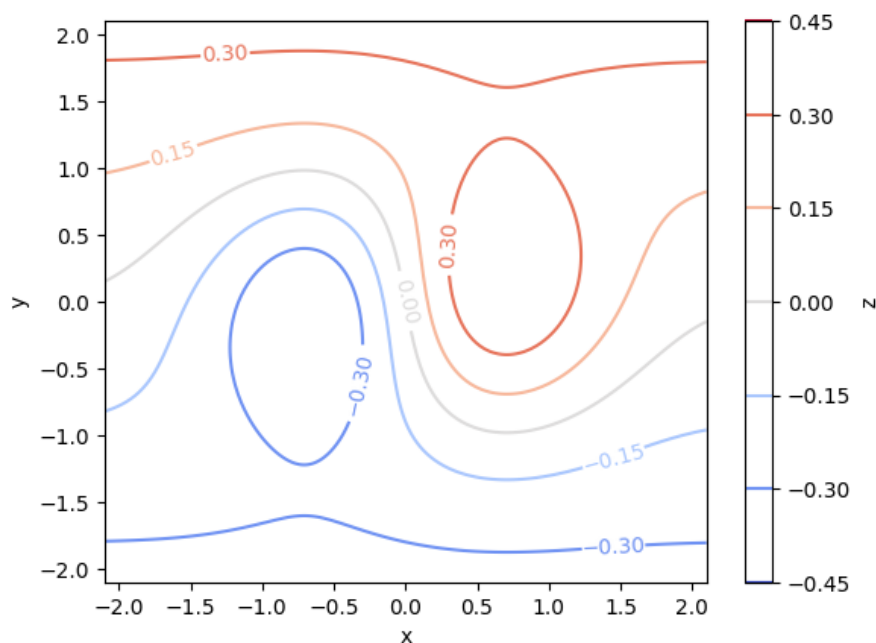
1.1 Höhenlinien Stelle die Höhenlinien der Funktion dar. Probiere unterschiedliche Werte für die Anzahl der Höhenlinien aus. Schreibe die Höhenlinien mit `plt.clabel` an.

Aufgabe 1.1

```
def g(x, y):
    return x * np.exp( -(x**2) - y**2) + y / 6
```

```
n = 500
x = np.linspace( -2.1, 2.1, n)
y = np.linspace( -2.1, 2.1, n)
X, Y = np.meshgrid(x, y)
Z = g(X, Y)
```

```
cs = plt.contour(X, Y, Z, cmap="coolwarm")
plt.clabel(cs)
plt.xlabel("x")
plt.ylabel("y")
plt.colorbar(label="z")
plt.show()
```

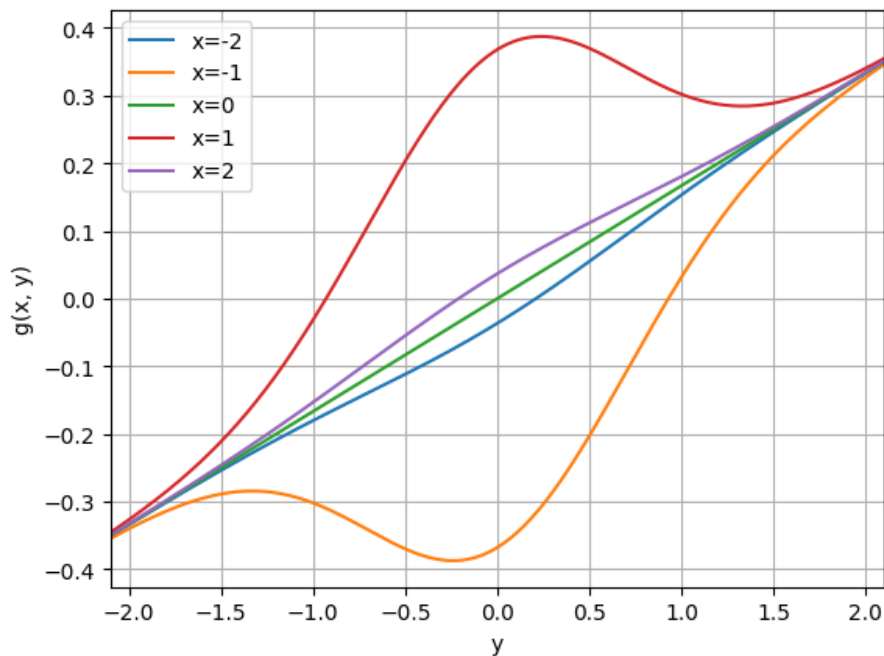


1.2 Isokonturlinien Stelle Isokonturlinien für a) konstante $x \in \{-2, 1, 0, 1, 2\}$ und b) konstante $y \in \{-2, 1, 0, 1, 2\}$ dar (mit korrekten Achsenbeschriftungen).

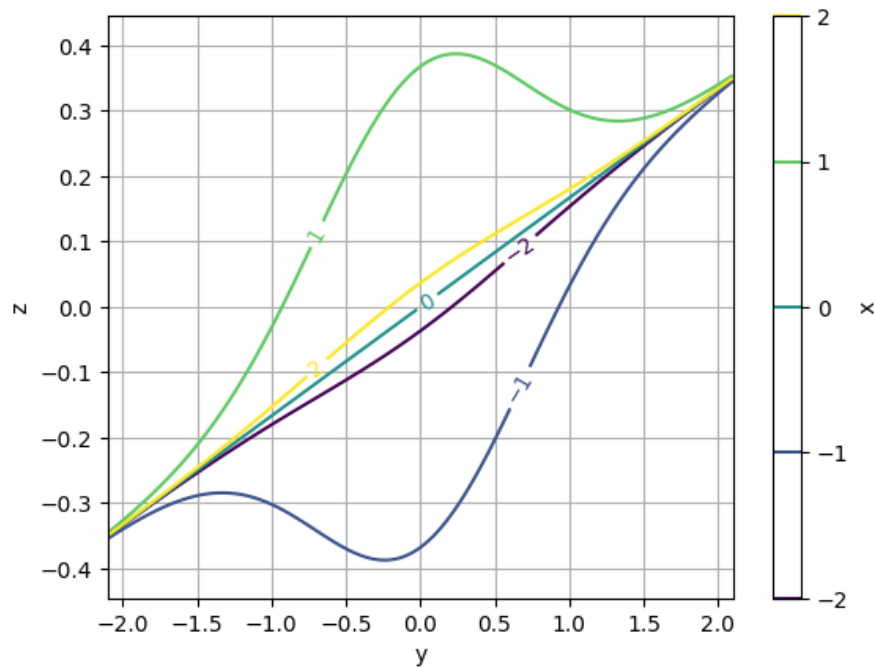
Hinweis: Die Funktionsgleichung von $z = g(x, y)$ kann hier nicht analytisch nach y aufgelöst werden. Also kann man nicht für eine Serie von x -Werten die Du kannst dazu entweder mehrere Graphen mit `plt.plot` erstellen oder auch `plt.contour` nutzen.

```
# Aufgabe 1.2 a)
levels = [-2, -1, 0, 1, 2]

# Lösung mit Plot
for x_level in levels:
    plt.plot(y, g(x_level, y), label=f"x={x_level}")
plt.xlim(y.min(), y.max())
plt.xlabel("y")
plt.ylabel("g(x, y)")
plt.legend()
plt.grid()
plt.show()
```

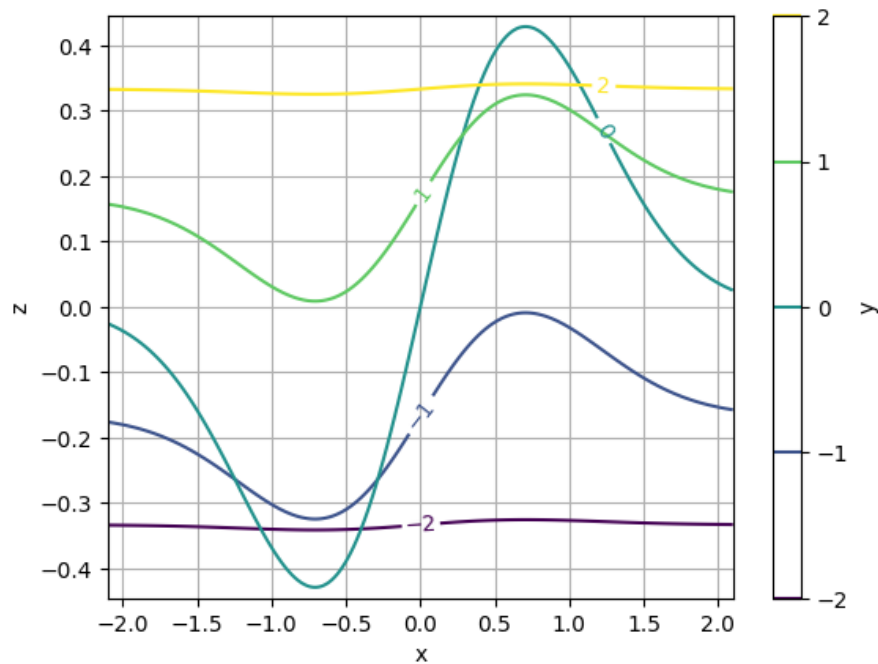


```
# Lösung mit Contour
cs = plt.contour(Y, Z, X, levels=levels)
plt.clabel(cs)
plt.xlabel("y")
plt.ylabel("z")
plt.colorbar(label="x")
plt.grid()
plt.show()
```



Aufgabe 1.2 b)

```
cs = plt.contour(X, Z, Y, levels=levels)
plt.clabel(cs)
plt.xlabel("x")
plt.ylabel("z")
plt.colorbar(label="y")
plt.grid()
plt.show()
```

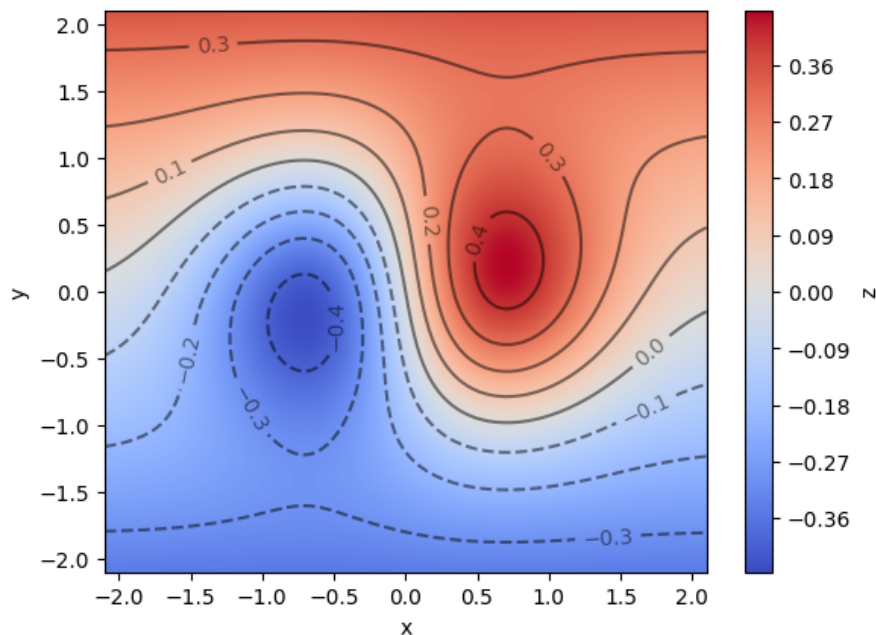


1.3 Gefüllte Höhenlinien Erstelle ein Höhenliniendiagramm mit gefüllten Flächen. Nutze dazu die Funktion `plt.contourf` mit vielen Levels.

Stelle darüber auch noch eine kleinere Anzahl Höhenlinien mit `plt.contour` dar.

Aufgabe 1.3

```
plt.contourf(X, Y, Z, levels=1000, cmap="coolwarm")
plt.colorbar(label="z")
plt.xlabel("x")
plt.ylabel("y")
cs = plt.contour(X, Y, Z, levels=10, colors="black", alpha=0.5)
# Der alpha Wert bestimmt die Transparenz (0 = transparent, 1 = opak)
plt.clabel(cs)
plt.show()
```



1.4 Alles in einem Diagramm Erstelle mit `plt.subplots()` ein Diagramm mit 2×2 Koordinatenachsen. Darin sollen unterschiedliche Darstellungen der Funktion $g(x, y)$ aus Aufgabe 1 eingefügt werden:

- unten links die Höhenlinien in der x-y-Ebene
- oben links die Isolinien für konstante y, mit x in der horizontalen und z in der vertikalen Achse
- unten rechts die Isolinien für konstante x, mit z in der horizontalen und y in der vertikalen Achse mit der Methode `ax.set_axis_off()` kannst du die leeren Koordinatenachsen oben rechts entfernen.

Aufgabe 1.4

```
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(8, 8))
```

```
# Unten links: konstante z
cs = ax3.contour(X, Y, Z)
cs.clabel(inline=True, fontsize=10)
ax3.set_title("constant z")
ax3.set_xlabel("x")
ax3.set_ylabel("y")
```

```
# Unten rechts: konstante x
cs = ax4.contour(Z, Y, X)
cs.clabel(inline=True, fontsize=10)
```



```

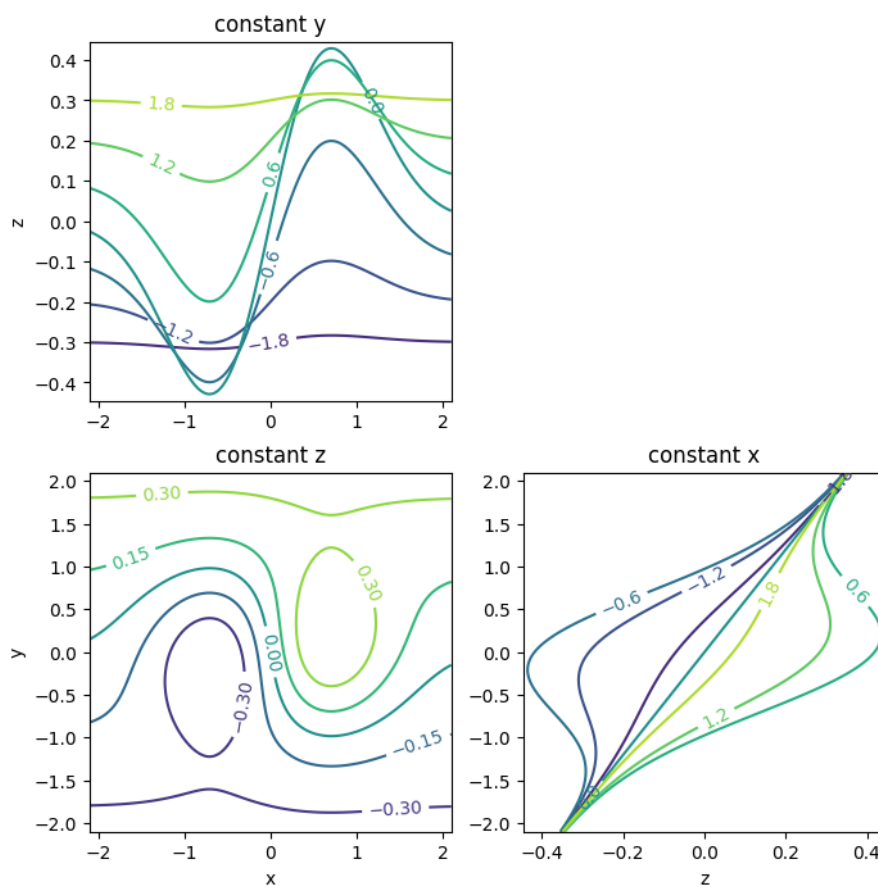
ax4.set_title("constant x")
ax4.set_xlabel("z")
# ax4.set_ylabel("y")

# Oben links: konstante y
cs = ax1.contour(X, Z, Y)
cs.clabel(inline=True, fontsize=10)
ax1.set_title("constant y")
# ax1.set_xlabel("x")
ax1.set_ylabel("z")

# Oben rechts: gar nichts
ax2.set_axis_off()

plt.show()

```



Aufgabe 2

2.1 Daten einlesen In der Datei `xyz_data.csv` befinden sich Messwerte, wobei z als Funktion von x und y interpretiert werden kann.

Lies die Daten ein, entpacke sie in drei Arrays x , y und z . Wie viele Messpunkte sind es?

Aufgabe 2.1

```
x, y, z = np.loadtxt("xyz_data.csv", delimiter=",", skiprows=2).T
```

```
x.shape, y.shape, z.shape
```

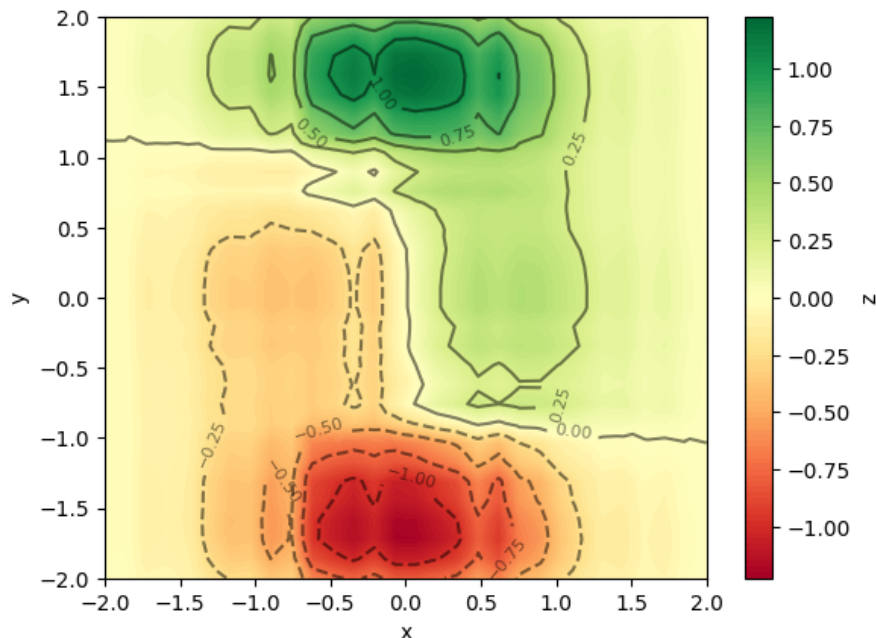
```
((900,), (900,), (900,))
```

2.2 Höhenliniendiagramm

- Erstelle mit den eingelesenen Daten ein Höhenliniendiagramm mit gefüllten Flächen und vielen Levels.
- Beschrifte die Achsen und füge die Farbskala mit entsprechender Beschriftung hinzu.
- Lege darüber nochmals eine kleinere Anzahl Höhenlinien ohne Flächenfüllung und beschrifte die Linien mit deren z -Werten.
- Schau dir die Dokumentation zu Farbskalen in Matplotlib an und probiere für den gefüllten Contour-Plot verschiedene Farbskalen aus.

Aufgabe 2.2

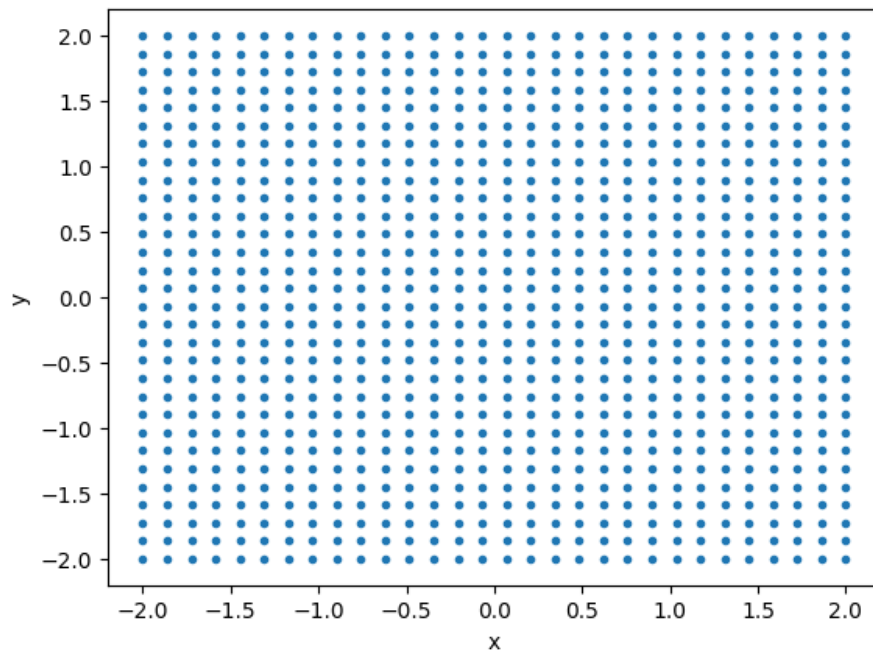
```
plt.tricontourf(x, y, z, levels=100, cmap="RdYlGn")
plt.colorbar(label="z")
cs = plt.tricontour(x, y, z, levels=10, colors="black", alpha=0.5)
plt.clabel(cs, fontsize=8)
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



2.3 Sampling-Punkte Stelle in der Ebene der unabhängigen Variablen alle Punkte dar, an denen die Funktion vermessen wurde.

Aufgabe 2.3

```
plt.plot(x, y, ".")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



Aufgabe 3 Wähle eine der “Test-Functions for Optimization” aus und stelle die Höhenlinien davon dar, wie sie auch auf der Wikipedia-Seite ersichtlich sind.

Hinweis: Die z -Skala ist in gewissen Fällen logarithmiert, siehe genau hin. Um dies auch hinzukriegen, kannst du

- entweder die Funktionswerte vor dem Darstellen zur Basis 10 logarithmieren (`np.log10`),
- oder dieses Beispiel adaptieren, um eine logarithmierte Skala für die Levels zu erhalten: `Contourf` and `log color scale`.

Aufgabe 2

```
def Beale(x, y):
    return (
        (1.5 - x + x * y) ** 2
        + (2.25 - x + x * y**2) ** 2
        + (2.625 - x + x * y**3) ** 2
    )

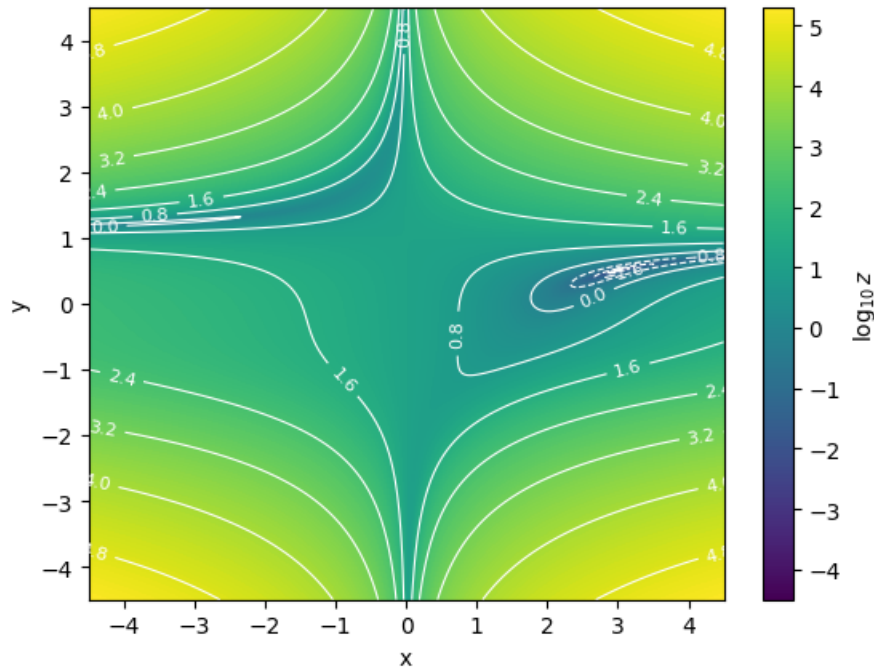
n = 1000
x = y = np.linspace(-4.5, 4.5, n)
X, Y = np.meshgrid(x, y)
Z = Beale(X, Y)

# Statt eine logarithmische Skala zu verwenden, könnten
# einfach die Funktionswerte logarithmieren und dann
# die Höhenlinien davon darstellen:

lgZ = np.log10(Z)

plt.contourf(X, Y, lgZ, 200)
plt.colorbar(label=r"$\log_{10}z$")
cs = plt.contour(X, Y, lgZ, levels=15, colors="white", linewidths=0.8)
plt.clabel(cs, fontsize=8)
```

```
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

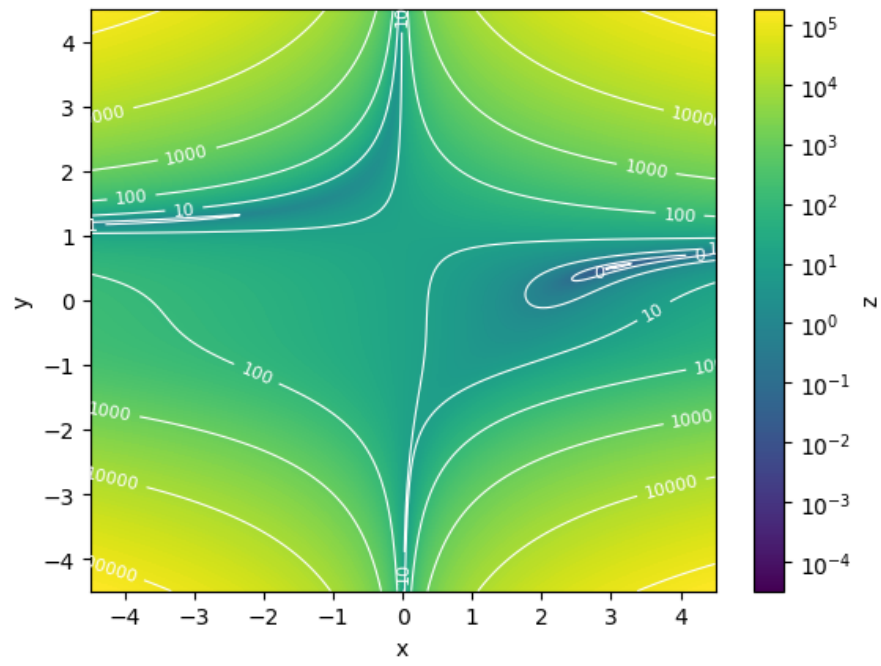


```
# Um wirklich eine logarithmische Skala für die Höhenlinien
# und die Farbgebung zu erhalten, muss man etwas mehr Aufwand
# betreiben, siehe
# https://matplotlib.org/stable/gallery/images_contours_and_fields/contourf_log.html
```

```
from matplotlib.ticker import LogLocator
```

```
levels_fine = np.logspace(lgZ.min(), lgZ.max(), 200)
decades = 10 ** np.arange(np.floor(lgZ.min()), np.ceil(lgZ.max()))
```

```
plt.contourf(X, Y, Z, levels=levels_fine, locator=LogLocator())
plt.colorbar(label="z", ticks=decades)
cs = plt.contour(
    X, Y, Z, levels=decades, locator=LogLocator(), colors="white", linewidths=0.8
)
plt.clabel(cs, fontsize=8)
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



0.3.3 Darstellung von Funktionen mit mehreren Variablen

Vertiefung

Übersicht

1. Test-Funktion für Optimierung
2. Funktion von drei Variablen
3. Contour-Plot aus Messwerten

```
import numpy as np
import matplotlib.pyplot as plt
```

1. Test-Funktionen für Optimierung Auf der Wikipedia-Seite “Test-Functions for Optimization” finden sich viele spannende Funktionen von meist zwei Variablen x und y . Wir betrachten mal die Ackley-Funktion:

$$a(x, y) = -20 \exp \left[-0.2 \sqrt{0.5 (x^2 + y^2)} \right] - \exp \left[0.5 (\cos(2\pi x) + \cos(2\pi y)) \right] + e + 20 \quad (21)$$

und die Himmelblau-Funktion:

$$h(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2 \quad (22)$$

```
def ackley(x, y):
    return (
        -20 * np.exp( -0.2 * np.sqrt(0.5 * (x**2 + y**2)))
        - np.exp(0.5 * (np.cos(2 * np.pi * x) + np.cos(2 * np.pi * y)))
        + np.exp(1)
        + 20
    )

def himmelblau(x, y):
    return (x**2 + y - 11) ** 2 + (x + y**2 - 7) ** 2
```

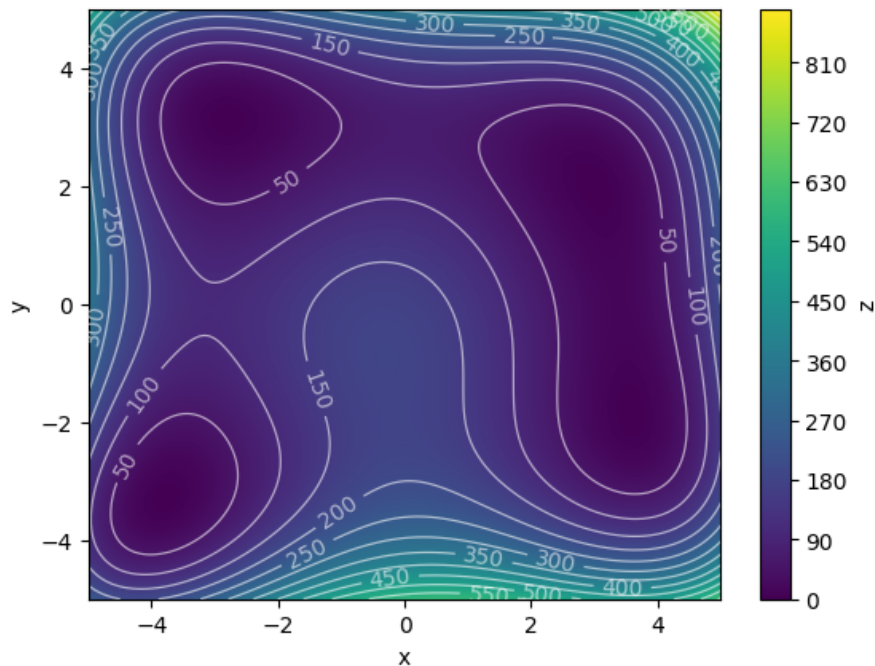
Um ein Gitternetz von über die x - y -Ebene aufzuspannen und an allen Punkten die Funktionswerte zu berechnen, verwenden wir die Funktion `np.meshgrid`.

```
n = 200
x = np.linspace( -5, 5, n)
y = np.linspace( -5, 5, n)
X, Y = np.meshgrid(x, y)

# Z = ackley(X, Y)
Z = himmelblau(X, Y)

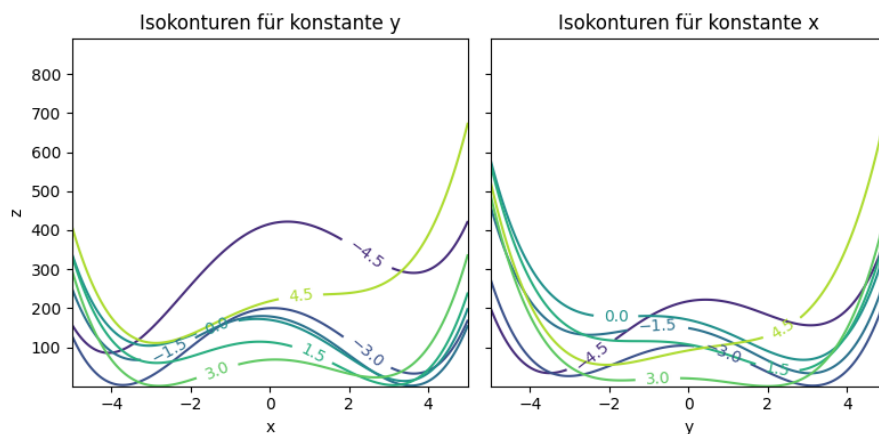
Höhenlinien mit plt.contour. Ausprobieren:
• Parameter levels=25 und höher
• plt.contourf verwenden

plt.contourf(X, Y, Z, levels=1000)
plt.colorbar(label="z")
CS = plt.contour(X, Y, Z, colors="w", linewidths=0.9, alpha=0.6)
plt.clabel(CS)
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



Mit `plt.contour` kann man auch mal konstante x und konstante y Werte anschauen. Nutze `plt.subplots`, um zwei Achsenpaare nebeneinander zu machen:

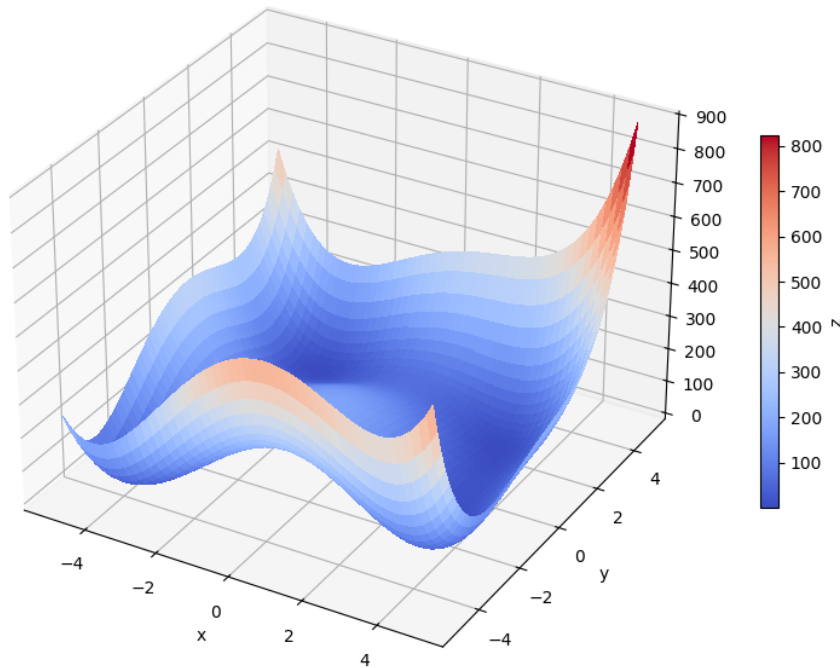
```
fig, (ax1, ax2) = plt.subplots(1, 2, sharey=True, figsize=(8, 4))
cs1 = ax1.contour(X, Z, Y)
ax1.clabel(cs1, inline=True, fontsize=10)
ax1.set_xlabel("x")
ax1.set_ylabel("z")
ax1.set_title("Isokonturen für konstante y")
cs2 = ax2.contour(Y, Z, X)
ax2.clabel(cs2, inline=True, fontsize=10)
ax2.set_xlabel("y")
ax2.set_title("Isokonturen für konstante x")
plt.tight_layout()
plt.show()
```



Auch Flächen im Raum sind eine Möglichkeit der Darstellung.

```
fig, ax = plt.subplots(subplot_kw={"projection": "3d"}, figsize=(10, 8))
surf = ax.plot_surface(X, Y, Z, cmap="coolwarm", antialiased=False)
plt.colorbar(surf, shrink=0.5, label="z")
ax.set_xlabel("x")
ax.set_ylabel("y")
```

```
plt.show()
```



2. Funktion von drei Variablen Wir betrachten die Funktion

$$f(x, y, t) = \sqrt{xy} \cdot \sin(2\pi x) \cdot \sin(2\pi ty) \quad (23)$$

die von drei Variablen abhängt.

Nehmen wir an, x und y sind räumliche Koordinaten und t ist die Zeit. Dann ist es sinnvoll, die Höhenlinien zu verschiedenen Zeiten als Funktion von x und y darzustellen.

```
def f(x, y, t):
    return np.sqrt(x * y) * np.sin(2 * np.pi * x) * np.sin(2 * np.pi * t * y)
```

Gitternetz-Punkte berechnen:

```
n = 200
x = np.linspace(0, 1, n)
y = np.linspace(0, 2, n)
```

```
X, Y = np.meshgrid(x, y)
```

Für jeden Zeitpunkt ein Höhenliniendiagramm darstellen:

```
t_vals = np.linspace(0, 1, 6)
```

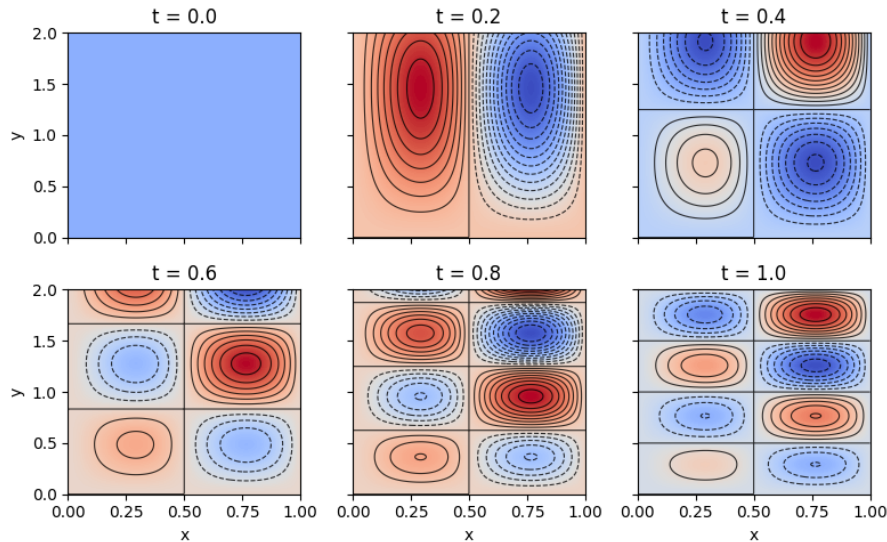
```
fig, axs = plt.subplots(2, len(t_vals) // 2, figsize=(8, 5), sharex=True, sharey=True)
for t, ax in zip(t_vals, axs.flat):
    Z = f(X, Y, t)
    ax.contourf(X, Y, Z, levels=200, cmap="coolwarm")
    ax.contour(X, Y, Z, levels=20, colors="k", linewidths=0.8, alpha=0.8)
    ax.set_title(f"t = {t:.1f}")
```



```

for ax in axs[ -1, :]:
    ax.set_xlabel("x")
for ax in axs[:, 0]:
    ax.set_ylabel("y")
plt.tight_layout()
plt.show()

```



3. Contour-Plot aus Messwerten Um Höhenlinien mit `plt.contour` oder `plt.contourf` darzustellen, müssen die x -, y - und z -Werte in Form von Gitternetzen vorliegen, die normalerweise mit `np.meshgrid` erzeugt werden. Das ist praktisch, wenn man eine Funktion analytisch kennt und an den Gitterpunkten auswerten kann.

Hat man aber Messwerte, die nicht auf einem regelmässigen Gitter liegen, so benutzt man die Funktionen `plt.tricontour` bzw. `plt.tricontourf`. Diese verwenden Triangulation, um die Funktion zwischen den gegebenen Messpunkten zu interpolieren. (Methode nicht Thema dieses Kurses.)

Ein ausführliches Beispiel dazu findet sich hier: [Tricontour Smooth Delaunay](#).

Wir betrachten hier ein einfacheres Beispiel und zwar jenes aus Beispiel 2

$$f(x, y, t) = \sqrt{xy} \cdot \sin(2\pi x) \cdot \sin(2\pi ty) \quad (24)$$

diesmal aber mit simulierten Messwerten.

```

def f(x, y, t):
    return np.sqrt(x * y) * np.sin(2 * np.pi * x) * np.sin(2 * np.pi * t * y)

```

Nun simulieren wir “Messwerte” dieser Funktion an n zufälligen Punkten im Bereich $x \in [0, 1]$ und $y \in [0, 2]$.

```

n = 1000
rng = np.random.default_rng(12345678)
x_data = rng.uniform(0, 1, size=n)
y_data = rng.uniform(0, 2, size=n)

t = 0.6
z_data = f(x_data, y_data, t) # als gemessen gedachte Werte

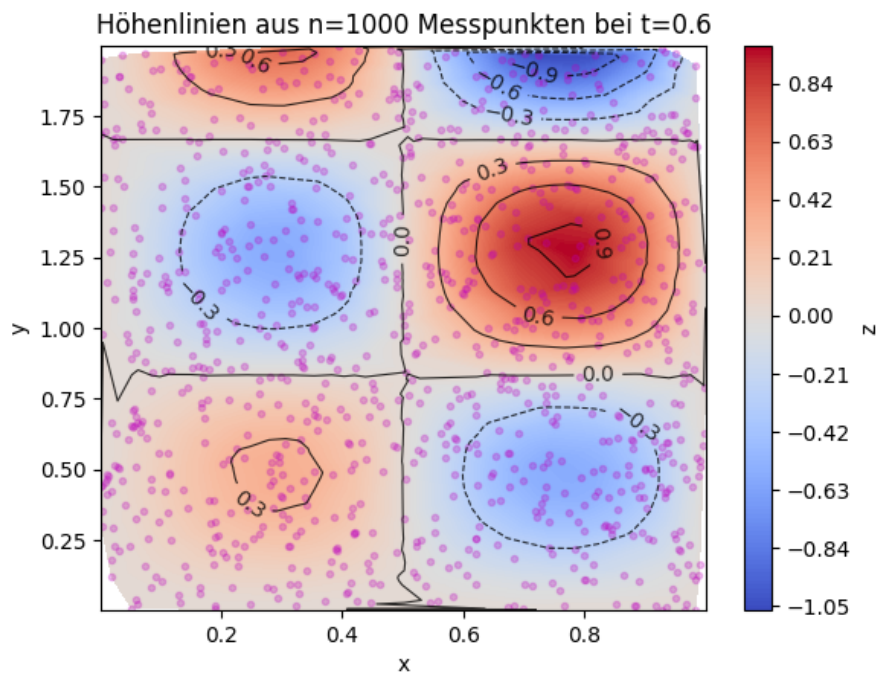
```

Die Darstellung erfolgt mit `plt.tricontour`:

```

plt.tricontourf(x_data, y_data, z_data, levels=200, cmap="coolwarm")
plt.colorbar(label="z")
plt.scatter(x_data, y_data, color="m", marker=".", alpha=0.25)
cs = plt.tricontour(x_data, y_data, z_data, colors="k", linewidths=0.8, alpha=0.8)
plt.clabel(cs)
plt.xlabel("x")
plt.ylabel("y")
plt.title(f"Höhenlinien aus n={n} Messpunkten bei t={t}")
plt.show()

```



0.4 Interpolation

0.4.1 Interpolation

Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Hier lernst du zwei einfache Verfahren zur Interpolation von Funktionswerten zwischen bekannten Funktionswerten kennen. Das Ziel ist, deren Funktionsweise kennenzulernen, indem du sie selbst rudimentär implementierst. Details und Verfeinerungen folgen in der Vertiefung.

```
import numpy as np
import matplotlib.pyplot as plt
```

Arbeitsauftrag

Aufgabe 1: Was ist Interpolation? Studiere die Kapitel 1 des Buchs Walz [2020]. (Das PDF des Buchs liegt auch im GitHub Repo und kann von Springer heruntergeladen werden, wenn man im WLAN der ZHAW oder per VPN verbunden ist.)

a) Formuliere in eigenen Worten, was Interpolation ist:

a) **Lösung** (Andere Formulierungen können auch korrekt sein.)

Interpolation ist eine Methode, Punkte auf einem Funktionsgraph zu bestimmen, wenn nur einzelne Funktionswerte oder Messwerte gegeben sind und nicht die Funktionsgleichung.

b) Nenne ein paar Beispiele, wofür die Interpolation gebraucht wird.

b) **Lösung** (mögliche Beispiele)

- Schätzen von fehlenden Datenpunkten in Messreihen
- Grafische Darstellung von Funktionen anhand weniger berechneter Punkte
- Computergraphik in Games, etc.

Aufgabe 2: Lineare Interpolation Studiere als Einstieg dieses Video: “Lineare Interpolation, Herleitung, Formel” - Mathe by Daniel Jung.

a) Seien wie im Video folgende Punkte gegeben:

$$(x_1, y_1) = (2, 4)$$

$$(x_2, y_2) = (3, 25)$$

wobei $y = f(x)$ mit einer nicht bekannten Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$.

Bestimme die lineare Interpolation bei $x = 2.4$ mithilfe der Gleichung im Video.

a) **Lösung**

Die Gleichung für die lineare Interpolation (Gerade durch zwei Punkte) lautet:

$$y = y_1 + \frac{y_2 - y_1}{x_2 - x_1} \cdot (x - x_1) \quad (25)$$

Setzt man die beiden Punkte und $x = 2.4$ ein, erhält man:

$$\begin{aligned} y &= 4 + \frac{25 - 4}{3 - 2} \cdot (2.4 - 2) \\ &= 4 + 21 \cdot 0.4 \\ &= 12.4 \end{aligned}$$

b) Implementiere die lineare Interpolation zwischen zwei Punkten in Python.

- Definiere dazu eine Funktion `linear_interpolation(x_data, y_data, x)`, die ein Array von y -Werten für die gegebenen x -Werte x zurückgibt.
- `x_data` und `y_data` sind Arrays der Länge 2 mit den bekannten Punkten.

- Teste die Funktion mit den Daten von oben am Punkt $x = 2.4$ und vergleiche mit dem Resultat von a).

Lösung 2.b)

```
def linear_interpolation(x_data, y_data, x):  
    m = (y_data[1] - y_data[0]) / (x_data[1] - x_data[0])  
    return m * (x - x_data[0]) + y_data[0]
```

```
x_data = np.array([2, 3])  
y_data = np.array([4, 25])
```

```
linear_interpolation(x_data, y_data, 2.4)
```

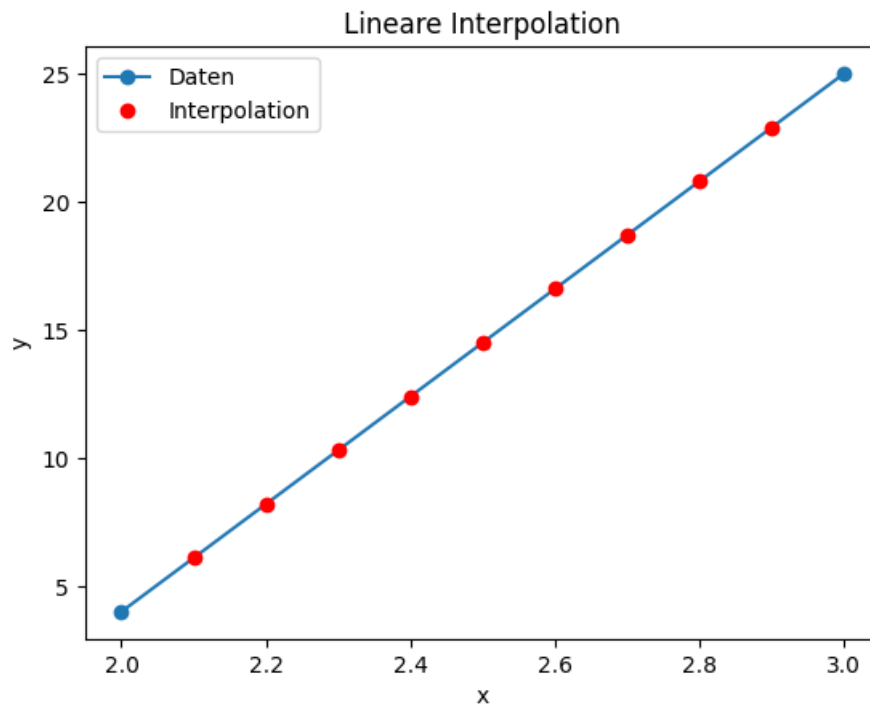
```
np.float64(12.399999999999999)
```

c) Darstellen

- Stelle die beiden gegebenen Punkte inkl. einer Verbindungslinie dar, indem du `plt.plot` mit der Option `"-o"` (verbundene Punkte) verwendest.
- Berechne mit der Funktion aus Teil b) für 9 x -Werte im Intervall $[2.1, 2.9]$ die interpolierten y -Werte, und füge sie dem Diagramm als rote Punkte hinzu.
- Deckt sich das Resultat mit deiner Erwartung?

Lösung 2.c)

```
x = np.linspace(2.1, 2.9, 9)  
y = linear_interpolation(x_data, y_data, x)  
  
plt.plot(x_data, y_data, "-o", label="Daten")  
plt.plot(x, y, "ro", label="Interpolation")  
plt.xlabel("x")  
plt.ylabel("y")  
plt.legend()  
plt.title("Lineare Interpolation")  
plt.show()
```



Aufgabe 3: Interpolation mit Polynomen a) Studiere die Einleitung von Kapitel 2 des Buchs Walz [2020], bis S. 8 ohne Abschnitt 2.1.

Frage: Es seien 8 Datenpunkte (x_i, y_i) für $i = 0, 1, \dots, 7$ gegeben. Diese nennt man auch Stützstellen. Welchen Grad muss dann ein Interpolationspolynom mindestens haben, damit es alle Punkte exakt durchlaufen kann? Begründe kurz.

a) **Lösung:** Der Grad des Polynoms muss mindestens 7 sein, damit es alle 8 Punkte exakt durchlaufen kann, denn ein Polynom vom Grad n hat $n + 1$ Koeffizienten, die durch die 8 Punkte bestimmt werden können.

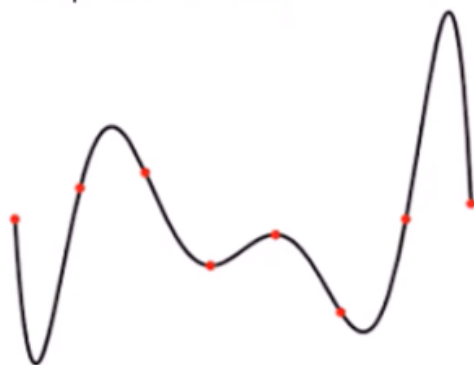
b) Schau dir das Video Kubische Splines mit natürlichen Randbedingungen - Leos Mathekanal bis **Minute 1:40** an.

Frage: Was für Probleme treten auf, wenn man viele Datenpunkte mit einem einzigen Polynom interpoliert?

b) **Lösung:** Das Polynom kann zu stark schwanken, v.a. am Rand des Definitionsbereichs, was zu unnatürlichem Verhalten führt.

Problem

Interpolationspolynome oszillieren mit zunehmender Anzahl an Datenpunkten sehr stark.



Leos Mathekanal. (2020, August 23). *Kubische Splines mit natürlichen Randbedingungen* [Videoaufnahme]. <https://www.youtube.com/watch?v=jWLyZXIEDtg>

Aufgabe 4: Kubische Spline-Interpolation Schau dir folgendes an:

- den Rest des Videos Kubische Splines mit natürlichen Randbedingungen - Leos Mathekanal und
- das Video kubische Splines - Interpolation - Mathe? Logisch! mindestens bis **Minute 1:40**

In beiden Videos wird ist die Mathematik etwas zu schnell erklärt. Keine Angst, wenn du das nicht gleich alles nachvollziehen kannst.

Ein kubischer Spline ist eine Interpolationsmethode, die mehrere Polynome verwendet, um die Datenpunkte zu verbinden. Jedes Polynom verbindet zwei benachbarte Datenpunkte (Stützstellen) und ist so aufgebaut:

$$s_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3 \quad \text{für } j = 0, 1, \dots, n-2 \quad (26)$$

wobei x_j die Stützstellen und a_j, b_j, c_j und d_j die Koeffizienten des Polynoms sind. Hier ist n die Anzahl der Stützstellen.

Fragen:

a) Wie unterscheidet sich ein kubischer Spline durch mehrere Punkte von einem einzelnen Interpolationspolynom?

a) Lösung: Ein kubischer Spline ist eine stückweise definierte Funktion, die in jedem Intervall zwischen zwei benachbarten Punkten ein eigenes kubisches Polynom definiert. Das Interpolationspolynom ist ein einziges Polynom (Grad = Anzahl Punkte + 1), das alle Punkte durchläuft.

b) Wenn wir 8 Datenpunkte (x_i, y_i) für $i = 0, 1, \dots, 7$ gegeben haben, aus wie vielen Polynomen besteht dann der kubische Spline?

b) Lösung: Der kubische Spline besteht aus 7 Polynomen, je eines für jedes Intervall zwischen zwei benachbarten Punkten.

c) Wie viele Koeffizienten gibt es dann in diesem Fall zu bestimmen?

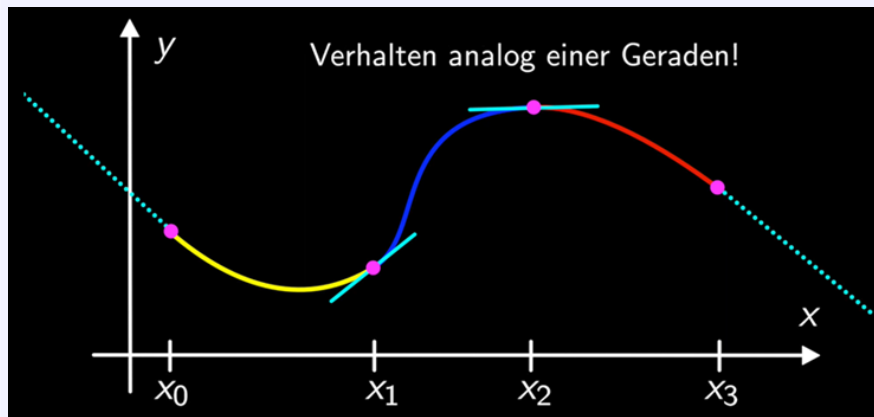
c) Lösung: Für jedes der 7 Polynome gibt es 4 Koeffizienten: $7 \cdot 4 = 28$ Koeffizienten.

d) Wir nehmen natürliche Randbedingungen an, das heisst, die zweite Ableitung soll an den äusseren Randpunkten (x_0 und x_7) gleich Null sein. Welche Bedingungen müssen die Polynome erfüllen? Und wie viele Bedingungen von welchem Typ gibt es?

d) Lösung:

- Jedes Polynom muss durch die beiden benachbarten Punkte gehen:
 - $s_j(x_j) = y_j$ und $s_j(x_{j+1}) = y_{j+1}$ für $j = 0, 1, \dots, 6$
 - Das sind $7 \cdot 2 = 14$ Bedingungen.
- Jeweils die erste und die zweite Ableitung der Polynome müssen an den inneren Punkten übereinstimmen.
 - $s'_j(x_{j+1}) = s'_{j+1}(x_{j+1})$ und $s''_j(x_{j+1}) = s''_{j+1}(x_{j+1})$ für $j = 0, 1, \dots, 5$ (insgesamt 6 innere Punkte)
 - Das sind $(8 - 2) \cdot 2 = 12$ Bedingungen.
- Bei natürlichen Randbedingungen muss die zweite Ableitung an den Rändern 0 sein
 - $s''_0(x_0) = 0$ und $s''_6(x_7) = 0$
 - Das sind 2 Bedingungen.

Spline mit natürlichen Randbedingungen



Mathe? Logisch! (2021, Mai 20). *kubische Splines—Interpolation* [Videoaufnahme].
<https://www.youtube.com/watch?v=pLfifROQ-MM>

0.4.2 Interpolation

Übungen - Lösungen

```
import numpy as np
import matplotlib.pyplot as plt
```

Aufgabe 1

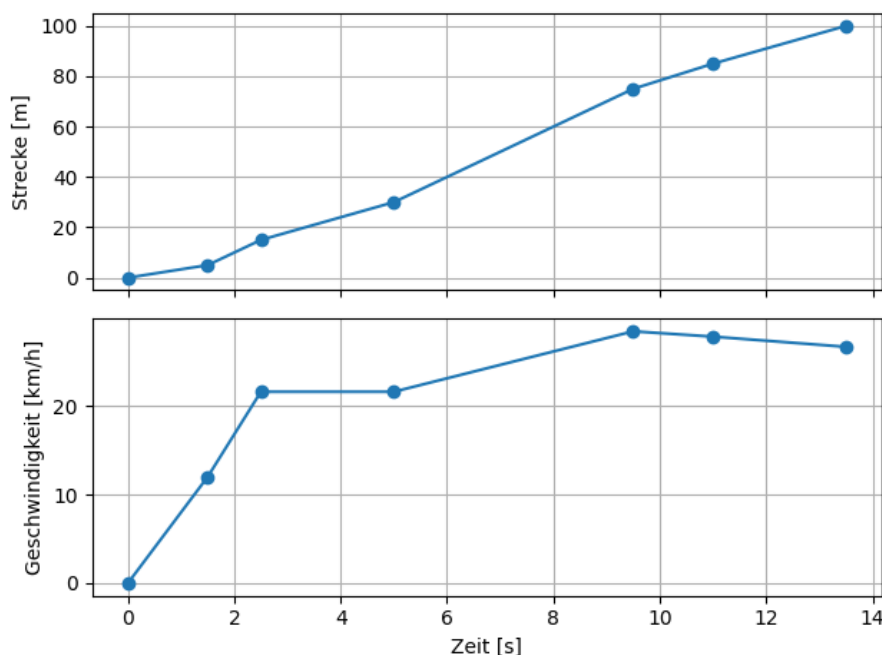
a) Daten einlesen und darstellen

- Lese die Daten in `sprinter.csv` ein und extrahiere die Spalten in die Variablen `x_data`, `t_data` und `v_data`.
- Stelle die zurückgelegte Strecke und die Geschwindigkeit je ein in einem Diagramm als Funktion der Zeit dar. Verwende `plt.subplots` mit der Option `sharex=True`, um dafür zwei übereinanderliegende Koordinatensysteme mit gekoppelten x -Achsen zu erzeugen.

Aufgabe 1 a)

```
sprinter_data = np.loadtxt("sprinter.csv", delimiter=",", skiprows=1)
x_data, t_data, v_data = sprinter_data.T
```

```
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
ax1.plot(t_data, x_data, "o -")
ax1.set_ylabel("Strecke [m]")
ax1.grid()
ax2.plot(t_data, v_data, "o -")
ax2.set_xlabel("Zeit [s]")
ax2.set_ylabel("Geschwindigkeit [km/h]")
ax2.grid()
plt.tight_layout()
plt.show()
```

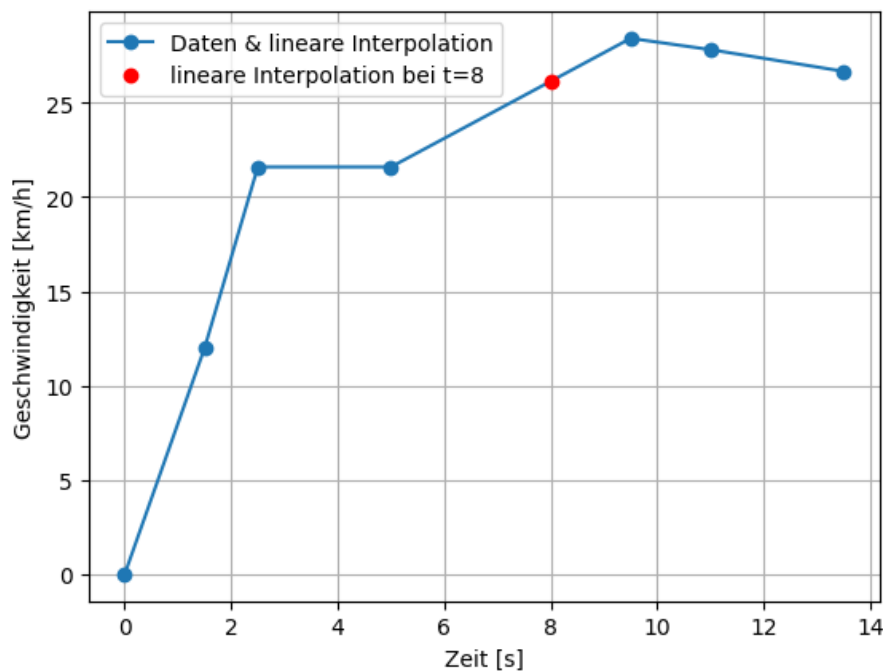


b) Lineare Interpolation Bestimme die linear interpolierte Geschwindigkeit zum Zeitpunkt $t = 8$ s. Füge diesen Punkt in rot ins v - t -Diagramm ein.

Aufgabe 1 b)

```
t0 = 8
i = np.searchsorted(t_data, t0) - 1
m = np.diff(v_data) / np.diff(t_data)
v0 = v_data[i] + m[i] * (t0 - t_data[i])

plt.plot(t_data, v_data, "o -", label="Daten & lineare Interpolation")
plt.plot(t0, v0, "ro", label=f"lineare Interpolation bei t={t0}")
plt.xlabel("Zeit [s]")
plt.ylabel("Geschwindigkeit [km/h]")
plt.legend()
plt.grid()
plt.show()
```



c) Interpolation mit einem Polynom

- Berechne für die Geschwindigkeit als Funktion der Zeit ein Interpolationspolynom von genügend hohem Grad, so dass es exakt durch alle Datenpunkte verläuft.
- Nutze dazu die Klasse `np.polynomial.Polynomial` und deren Methode `fit`.
- Stelle das Polynom mit 200 Punkten im gleichen Zeitraum wie die Daten dar und vergleiche es mit den Datenpunkten.

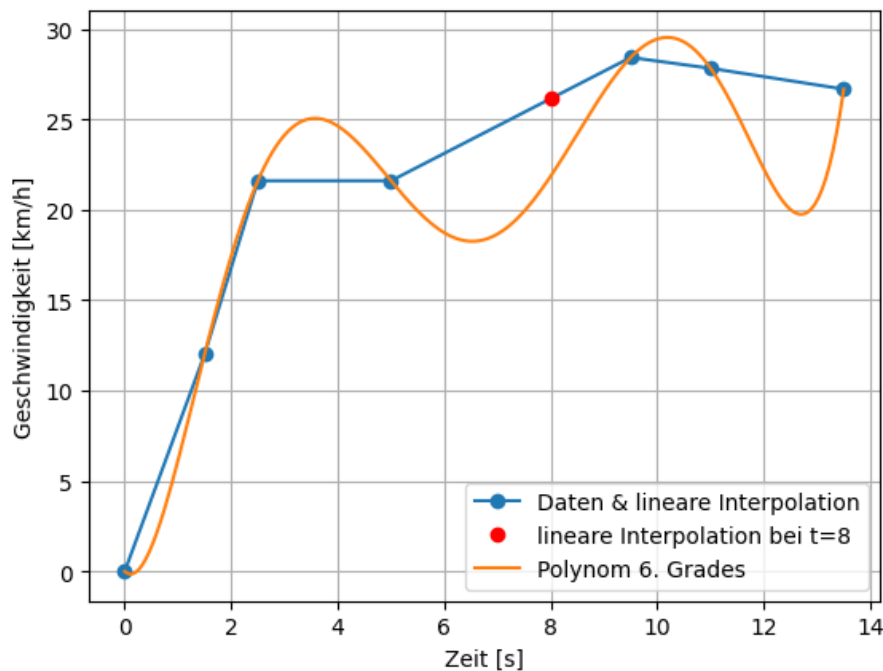
Aufgabe 1.c)

```
deg = len(t_data) - 1
poly = np.polynomial.Polynomial.fit(t_data, v_data, deg)

t = np.linspace(t_data.min(), t_data.max(), 200)
v_poly = poly(t)

plt.plot(t_data, v_data, "o -", label="Daten & lineare Interpolation")
plt.plot(t0, v0, "ro", label=f"lineare Interpolation bei t={t0}")
plt.plot(t, v_poly, label=f"Polynom {deg}. Grades")
plt.xlabel("Zeit [s]")
```

```
plt.ylabel("Geschwindigkeit [km/h]")
plt.legend()
plt.grid()
plt.show()
```



d) Interpolation mit Cubic Spline

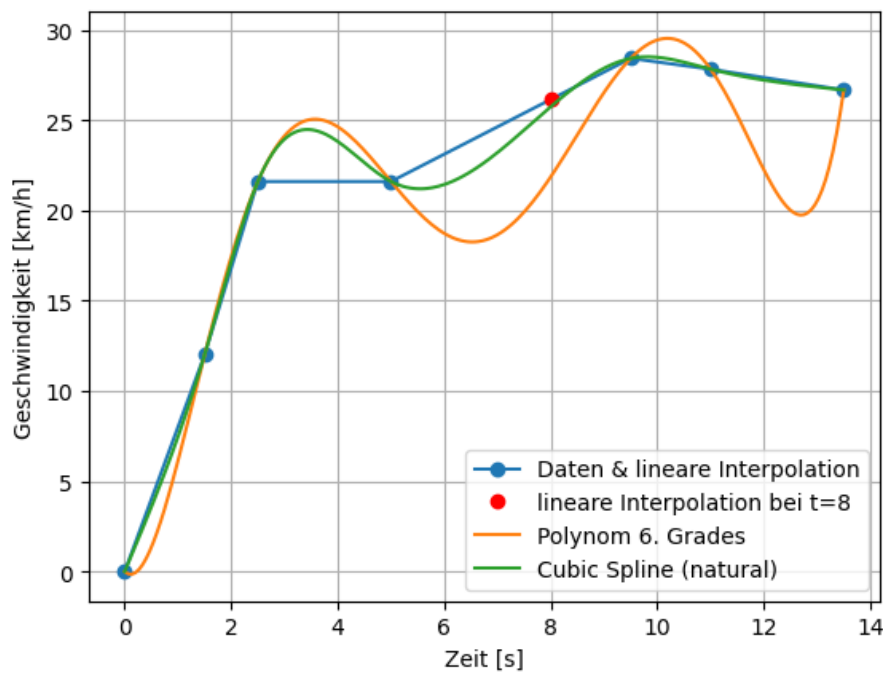
- Bestimme nun eine Cubic-Spline-Interpolation der Geschwindigkeit als Funktion der Zeit. Nutze dazu `scipy.interpolate.make_interp_spline`.
- Benutze die Option `bc_type="natural"`, um natürliche Randbedingungen (*engl. boundary conditions*) zu verwenden. Was bedeutet das nochmal?
- Stelle die Interpolation zusammen mit den Daten und dem Polynom aus Teil c) grafisch dar.

Aufgabe 1.d)

```
from scipy.interpolate import make_interp_spline
```

```
spline = make_interp_spline(t_data, v_data, bc_type="natural")
v_spl = spline(t)
```

```
plt.plot(t_data, v_data, "o -", label="Daten & lineare Interpolation")
plt.plot(t0, v0, "ro", label=f"lineare Interpolation bei t={t0}")
plt.plot(t, v_poly, label=f"Polynom {deg}. Grades")
plt.plot(t, v_spl, label=f"Cubic Spline (natural)")
plt.xlabel("Zeit [s]")
plt.ylabel("Geschwindigkeit [km/h]")
plt.legend()
plt.grid()
plt.show()
```



Aufgabe 2

a) Datenpunkte wählen und interpolieren

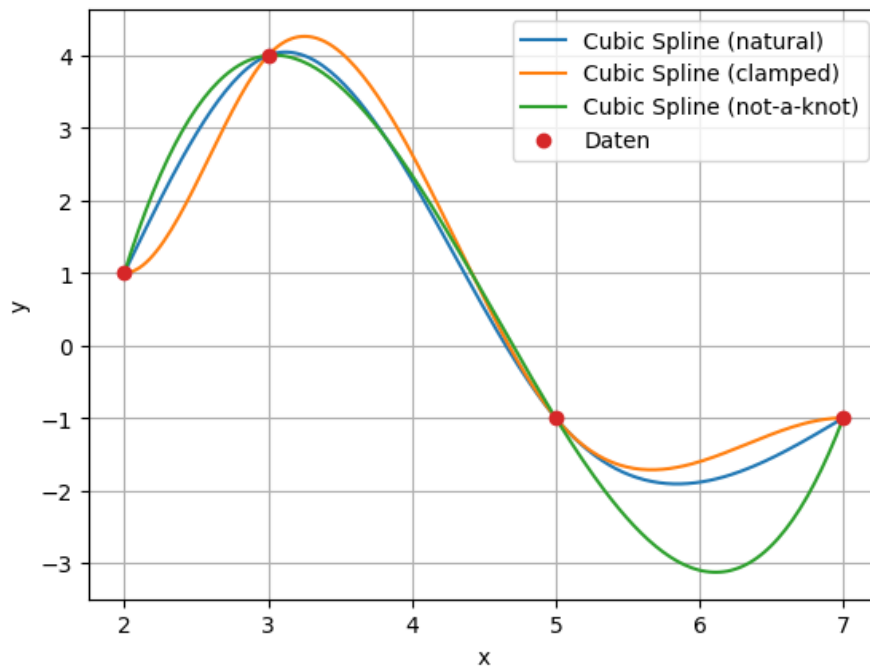
- Wähle selbst 4 Punkte in der x - y -Ebene mit ganzzahligen Koordinaten. Diese müssen in x -Richtung nicht zwingend den Abstand 1 und auch nicht alle den gleichen Abstand haben.
- Bestimme dazu mit SciPy die kubische Spline-Interpolation mit den drei unterschiedlichen Randbedingungen "natural", "clamped" und "not-a-knot".
- Stelle die Daten und die drei Interpolationsfunktionen grafisch dar.
- Verändere die Datenpunkte etwas, um das Verhalten der Interpolation zu untersuchen.

Aufgabe 2.a)

```
x_data = np.array([2, 3, 5, 7])
y_data = np.array([1, 4, -1, -1])
x = np.linspace(x_data.min(), x_data.max(), 200)

for bc_type in ["natural", "clamped", "not -a -knot"]:
    spline = make_interp_spline(x_data, y_data, bc_type=bc_type)
    y = spline(x)
    plt.plot(x, y, label=f"Cubic Spline ({bc_type})")

plt.plot(x_data, y_data, "o", label="Daten")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```



b) Kubischer Spline zwischen 2 Punkten von Hand berechnen Um die Auswirkung der verschiedenen Randbedingungen noch besser zu verstehen, soll ein kubischer Spline zwischen nur zwei Punkten berechnet werden. Wir nehmen die Punkte

$$(x_0, y_0) = (1, 1)$$

$$(x_1, y_1) = (2, 0)$$

Weil es nur zwei Punkte sind, gibt es nur ein einziges kubisches Polynom zu bestimmen, $s_0(x)$.

- Schreibe das Polynom $s_0(x)$ mit den Koeffizienten a_0, b_0, c_0, d_0 auf. (Der Einfachheit halber kannst du den Index 0 weglassen.)
- Bestimme die erste und zweite Ableitung von $s_0(x)$.
- Schreibe alle Bedingungen für $s_0(x)$ und deren Ableitungen auf und löse sie nach den Koeffizienten a_0, a_1, a_2, a_3 auf:

1. Freier Rand: Die zweite Ableitung soll an den Randpunkten 0 sein (Natürliche Randbedingungen).
2. Eingespannter Rand: Die erste Ableitung soll an den Randpunkten gleich 0 sein (Hermite Randbedingungen).

- Stelle schliesslich beide Fälle zusammen mit den Daten grafisch dar.
- Du kannst deine Lösung mithilfe von `make_interp_spline` überprüfen.

b) Lösung:

Spline Funktion und Ableitungen:

$$s(x) = a + b(x - 1) + c(x - 1)^2 + d(x - 1)^3$$

$$s'(x) = b + 2c(x - 1) + 3d(x - 1)^2$$

$$s''(x) = 2c + 6d(x - 1)$$

Funktionswerte an den beiden Punkten:

$$1 \stackrel{!}{=} s(1) = a$$

$$0 \stackrel{!}{=} s(2) = a + b + c + d$$

also $a = 1$ und $b + c + d = -1$.

1. Natürliche Randbedingungen (*natural*):

$$\begin{aligned} 0 &\stackrel{!}{=} s''(1) = 2c \\ 0 &\stackrel{!}{=} s''(2) = 2c + 6d \end{aligned}$$

Daraus folgt $c = 0$ und $d = 0$, und mit der Gleichung von oben $b = -1$. Schliesslich ist

$$s_{\text{natural}}(x) = 1 - (x - 1) = 2 - x \quad (27)$$

2. Hermite Randbedingungen (*clamped*):

$$\begin{aligned} 0 &\stackrel{!}{=} s'(1) = b \\ 0 &\stackrel{!}{=} s'(2) = b + 2c + 3d \end{aligned}$$

Daraus folgt $b = 0$ und es ergibt sich mit der Gleichung von oben das System

$$\begin{aligned} c + d &= -1 \\ 2c + 3d &= 0 \end{aligned}$$

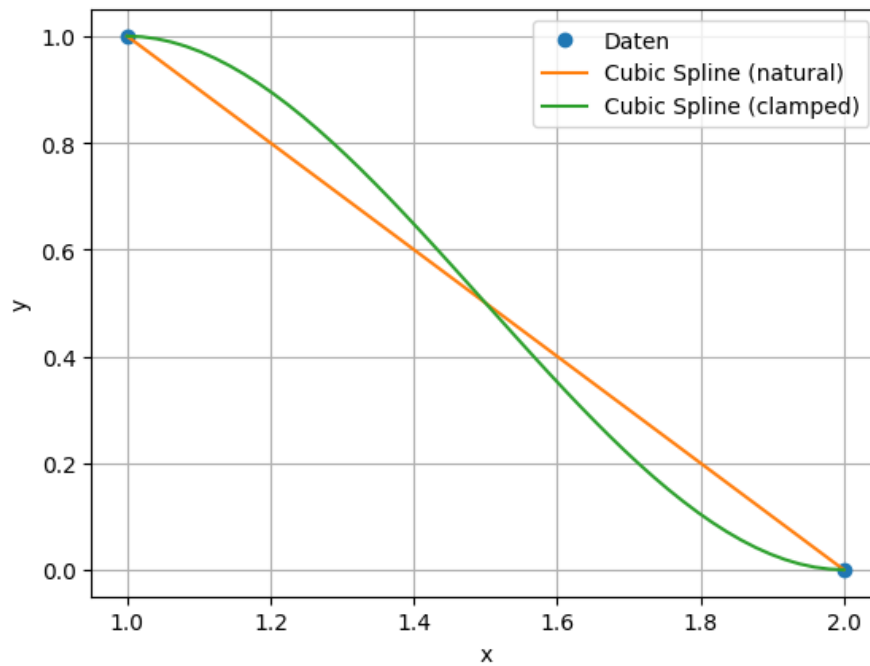
mit den Lösungen $c = -3$ und $d = 2$. Schliesslich ist

$$s_{\text{clamped}}(x) = 1 - 3(x - 1)^2 + 2(x - 1)^3 \quad (28)$$

Aufgabe 2.b) Darstellung der von Hand berechneten kubischen Splines

```
x_data = np.array([1, 2])
y_data = np.array([1, 0])
x = np.linspace(x_data.min(), x_data.max())
y_nat = 2 - x
y_clamp = 1 - 3 * (x - 1) ** 2 + 2 * (x - 1) ** 3

plt.plot(x_data, y_data, "o", label="Daten")
plt.plot(x, y_nat, label=f"Cubic Spline (natural)")
plt.plot(x, y_clamp, label=f"Cubic Spline (clamped)")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```

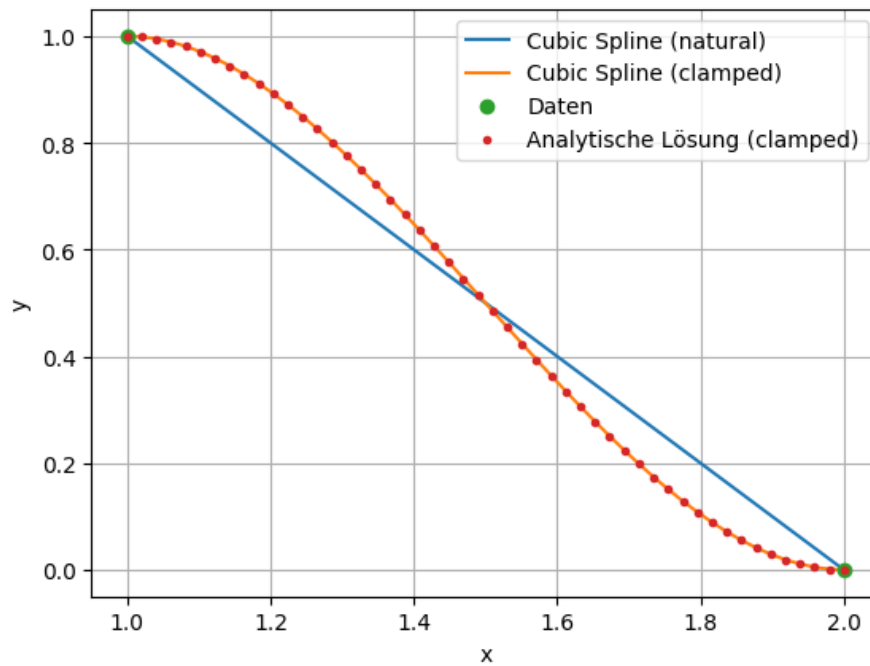


Aufgabe 2.b) Darstellung der mit 'make_interp_spline' berechneten kubischen Splines

```
x_data = np.array([1, 2])
y_data = np.array([1, 0])
x = np.linspace(x_data.min(), x_data.max())

for bc_type in ["natural", "clamped"]:
    spline = make_interp_spline(x_data, y_data, bc_type=bc_type)
    y = spline(x)
    plt.plot(x, y, label=f"Cubic Spline ({bc_type})")

plt.plot(x_data, y_data, "o", label="Daten")
plt.plot(x, y_clamp, ".", label="Analytische Lösung (clamped)")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```



Aufgabe 3

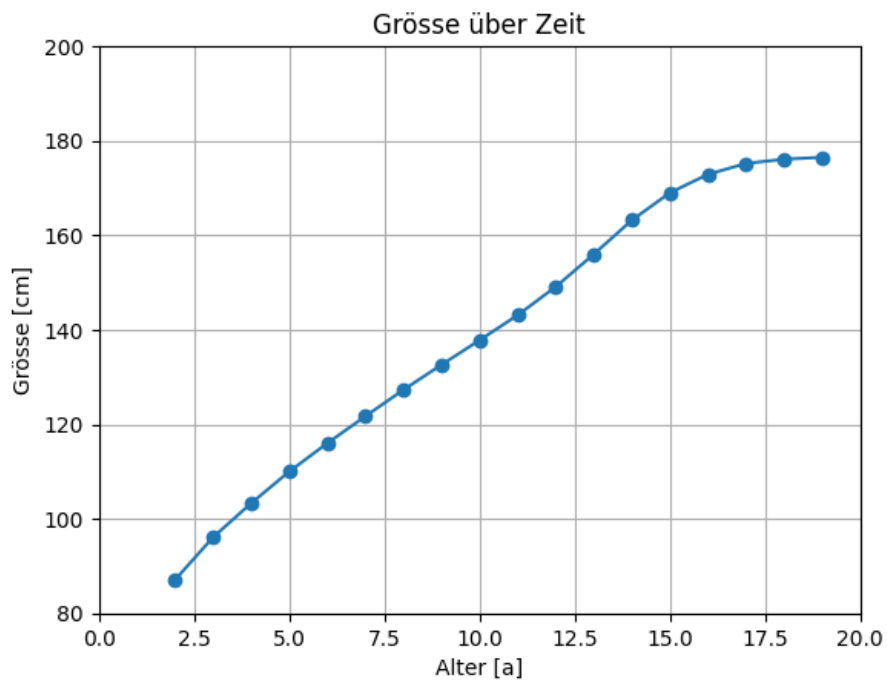
a) Daten einlesen und darstellen

- Lese die Daten in `alter_groesse.csv` ein.
- Stelle die Grösse als Funktion des Alters grafisch dar.
- Wähle den dargestellten Bereich, so dass das Alter von 0 bis 20 Jahren und die Grösse von 80 bis 200 cm reicht.

Aufgabe 3.a)

```
age_data, size_data = np.loadtxt("alter_groesse.csv", delimiter=";", skiprows=1).T

plt.plot(age_data, size_data, "o -")
plt.xlabel("Alter [a]")
plt.ylabel("Grösse [cm]")
plt.title("Grösse über Zeit")
plt.xlim(0, 20)
plt.ylim(80, 200)
plt.grid()
plt.show()
```

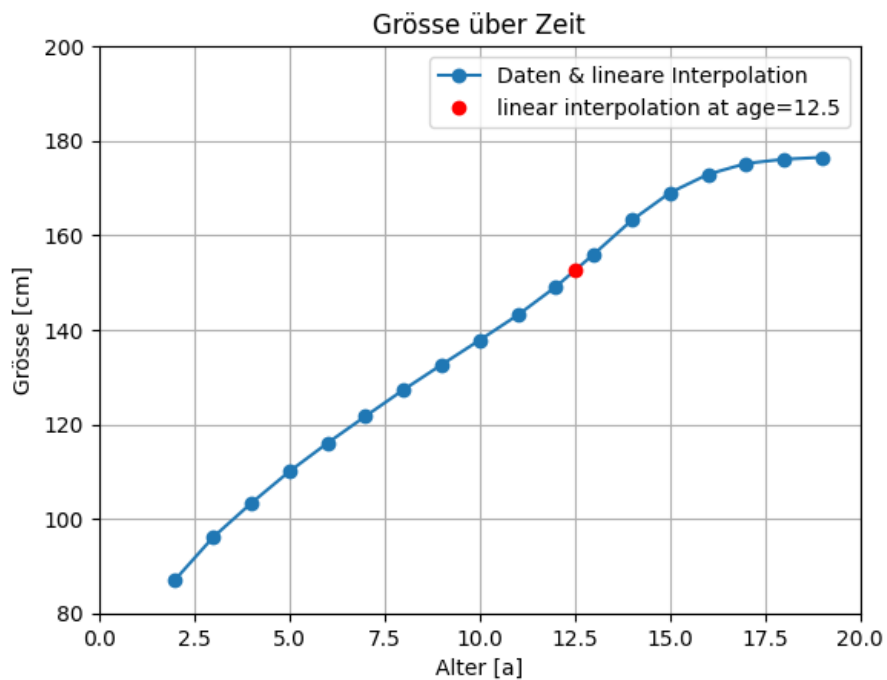
b) Lineare Interpolation Bestimme die Körpergrösse beim Alter 12.5 Jahre mithilfe der linearen Interpolation und füge diesen Punkt in das Diagramm ein.

Aufgabe 3.b)

```
age0 = 12.5
```

```
i = np.searchsorted(age_data, age0) - 1
m = np.diff(size_data) / np.diff(age_data)
size0 = size_data[i] + m[i] * (age0 - age_data[i])
```

```
plt.plot(age_data, size_data, "o -", label="Daten & lineare Interpolation")
plt.plot(age0, size0, "ro", label=f"linear interpolation at age={age0}")
plt.xlabel("Alter [a]")
plt.ylabel("Grösse [cm]")
plt.title("Grösse über Zeit")
plt.xlim(0, 20)
plt.ylim(80, 200)
plt.legend()
plt.grid()
plt.show()
```



c) Interpolationspolynom

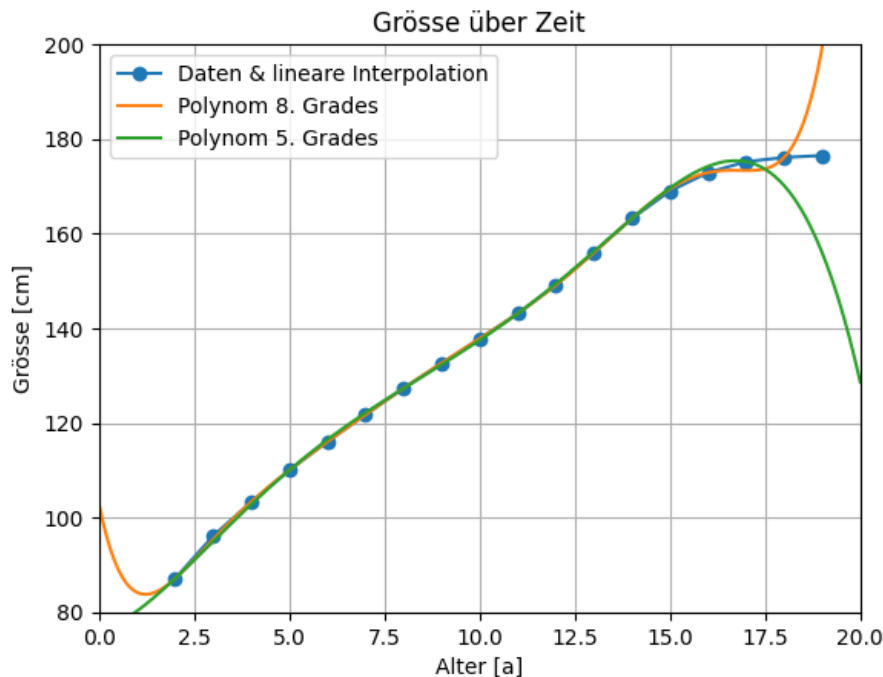
- Bestimme zwei Interpolationspolynome:
 - `poly2` soll nur jeden zweiten Datenpunkt verwenden.
 - `poly3` soll nur jeden dritten Datenpunkt verwenden.
- Benutze dazu die Methode `fit` der Klasse `np.polynomial.Polynomial` und wähle den Grad der beiden Polynome, so dass sie exakt alle verwendeten Datenpunkte interpolieren.
- Stelle die beiden Polynome im Zeitraum $[0, 20]$ dar und vergleiche mit den Datenpunkten.

Aufgabe 3.c)

```
poly2 = np.polynomial.Polynomial.fit(
    age_data[:,2],
    size_data[:,2],
    len(size_data[:,2]) - 1,
)
poly3 = np.polynomial.Polynomial.fit(
    age_data[:,3],
    size_data[:,3],
    len(size_data[:,3]) - 1,
)
age = np.linspace(0, 20, 200)
size_poly2 = poly2(age)
size_poly3 = poly3(age)

plt.plot(age_data, size_data, "o -", label="Daten & lineare Interpolation")
plt.plot(age, size_poly2, label=f"Polynom {len(size_data[:,2]) - 1}. Grades")
plt.plot(age, size_poly3, label=f"Polynom {len(size_data[:,3]) - 1}. Grades")
plt.xlabel("Alter [a]")
plt.ylabel("Grösse [cm]")
plt.title("Grösse über Zeit")
plt.xlim(0, 20)
plt.ylim(80, 200)
plt.legend()
```

```
plt.grid()
plt.show()
```



d) Interpolation mit Splines

- Bestimme einerseits eine lineare und andererseits eine kubische Spline-Interpolation der Grösse als Funktion des Alters. Nutze dazu `scipy.interpolate.make_interp_spline`.
- Erstelle ein Diagramm mit zwei übereinanderliegenden Koordinatensystemen, in denen oben die beiden Interpolationsfunktionen mit den Daten verglichen werden und unten die Differenz zwischen den beiden Interpolationsfunktionen dargestellt wird.
- Wähle für die Darstellung den Bereich, in dem es Daten gibt.

Aufgabe 3.d)

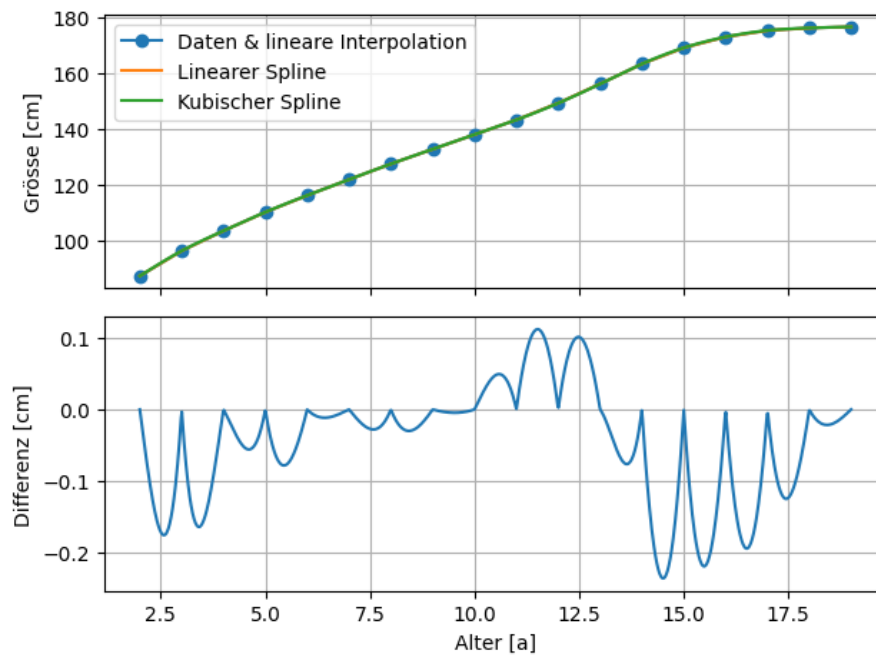
```
spline1 = make_interp_spline(age_data, size_data, k=1)
spline3 = make_interp_spline(age_data, size_data, k=3, bc_type="natural")
age = np.linspace(age_data.min(), age_data.max(), 1000)
size_spl1 = spline1(age)
size_spl3 = spline3(age)

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

ax1.plot(age_data, size_data, "o -", label="Daten & lineare Interpolation")
ax1.plot(age, size_spl1, label=f"Linearer Spline")
ax1.plot(age, size_spl3, label=f"Kubischer Spline")
ax1.set_ylabel("Grösse [cm]")
# ax1.set_xlim(0, 20)
# ax1.set_ylim(60, 200)
ax1.legend()
ax1.grid()

ax2.plot(age, size_spl1 - size_spl3, label="Differenz")
ax2.set_xlabel("Alter [a]")
ax2.set_ylabel("Differenz [cm]")
```

```
# ax2.legend()  
ax2.grid()  
plt.tight_layout()  
plt.show()
```



0.4.3 Interpolation

Vertiefung

Übersicht

1. Interpolationspolynom mit NumPy
2. Implementation der Linearen Interpolation entlang mehrerer Punkte
3. Spline-Interpolation mit SciPy
4. Kubische Spline-Interpolation

```
import numpy as np
import matplotlib.pyplot as plt
```

Vorbereitung: Daten generieren Um ein paar Testdaten zu generieren, verwenden wir eine Cosinus-Funktion und berechnen davon an zufälligen Stellen ein paar Werte.

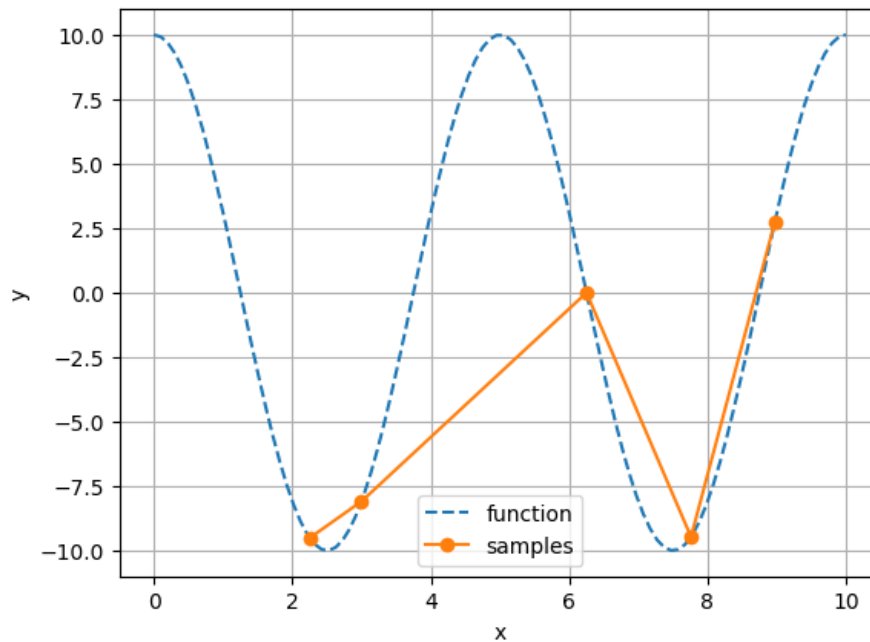
$$f(x) = a \cdot \cos\left(\frac{2\pi x}{T}\right) \quad (29)$$

```
def signal(x, amplitude=10, period=5):
    omega = 2 * np.pi / period
    return amplitude * np.cos(omega * x)

x_signal = np.linspace(0, 10, 100)
y_signal = signal(x_signal)

n_samples = 5
rng = np.random.default_rng(7)
x_data = np.round(np.sort(rng.uniform(low=0, high=10, size=n_samples)), 2)
y_data = np.round(signal(x_data), 2)

plt.plot(x_signal, y_signal, " - -", label="function")
plt.plot(x_data, y_data, "o -", label="samples")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```



1. Interpolationspolynom mit NumPy Mit der Klasse `np.polynomial.Polynomial` können Polynom-Funktionen definiert und damit gerechnet werden.

Z.B. das Polynom $p(x) = 1 + 3x + 2x^2$:

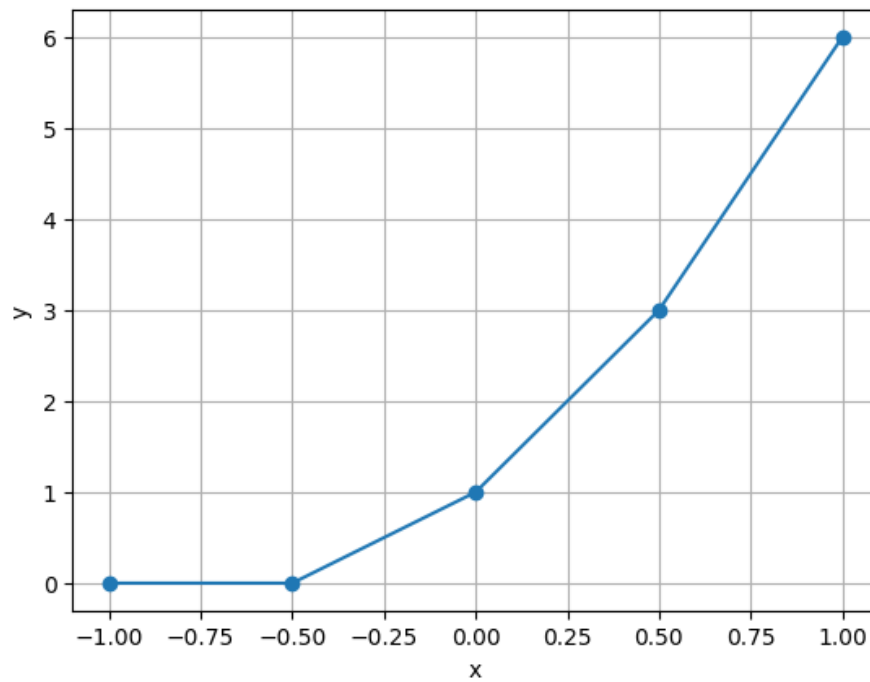
```
p = np.polynomial.Polynomial([1, 3, 2])
p
```

```
Polynomial([1., 3., 2.], domain=[ -1., 1.], window=[ -1., 1.], symbol='x')
```

Das Polynom `p` kann wie eine Funktion verwendet werden, um es an x -Werten auszuwerten:

```
x = np.linspace( -1, 1, 5)
y = p(x)
```

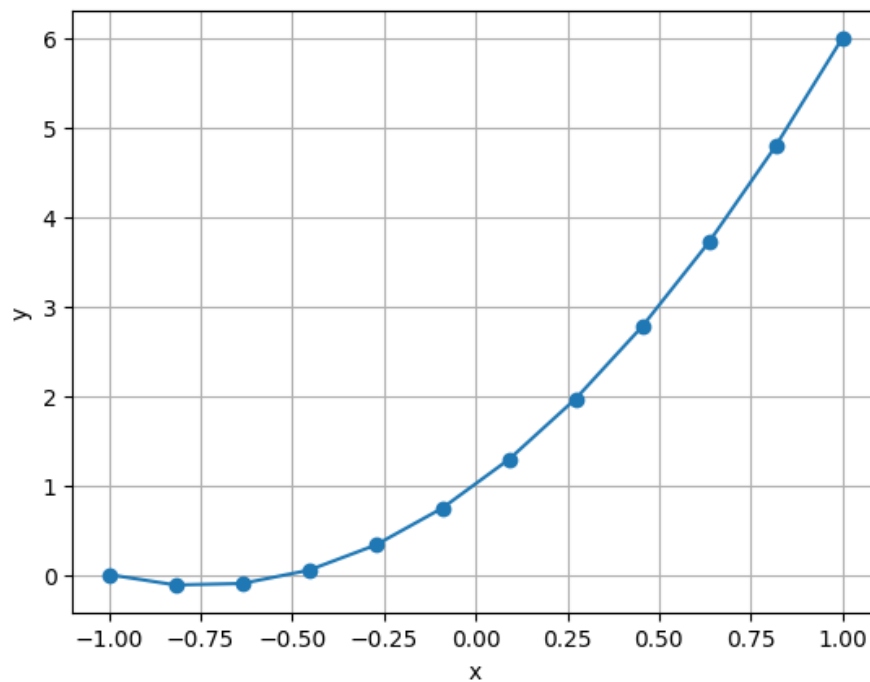
```
plt.plot(x, y, "o -")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.show()
```



Alternativ gibt es die Methode `Polynomial.linspace`, um direkt einen Linspace zu erzeugen und das Polynom dort auszuwerten.

```
x, y = p.linspace(n=12, domain=(-1, 1))
```

```
plt.plot(x, y, "o -")  
plt.xlabel("x")  
plt.ylabel("y")  
plt.grid()  
plt.show()
```



Für die Interpolation von Datenpunkten mit einem einzigen Polynom, wird die Methode `Polynomial.fit` verwendet.

Der Grad des Polynoms muss angegeben werden. Wenn alle Datenpunkte exakt getroffen werden sollen, muss der Grad so gewählt werden, dass es genügend Freiheitsgrade und somit Koeffizienten

gibt. Für jeden Datenpunkt gibt es eine Bedingung (Gleichung) und somit braucht es gleich viele Koeffizienten wie Datenpunkte.

$$\text{Grad} = \text{Anzahl der Datenpunkte} - 1 \quad (30)$$

```
deg = len(x_data) - 1
poly = np.polynomial.Polynomial.fit(x_data, y_data, deg)
poly

Polynomial([ 3.19496996, -10.27737856, -38.38206844, 16.39737856,
            31.79709848], domain=[2.25, 8.97], window=[-1., 1.], symbol='x')
```

Mit der Eigenschaft `Polynomial.coef` können die (ausmultiplizierten) Koeffizienten a_k bzgl. der Potenzen x^k ausgegeben werden.

```
print(poly.coef)
np.polynomial.Polynomial(poly.coef)

[ 3.19496996 -10.27737856 -38.38206844 16.39737856 31.79709848]

Polynomial([ 3.19496996, -10.27737856, -38.38206844, 16.39737856,
            31.79709848], domain=[-1., 1.], window=[-1., 1.], symbol='x')
```

Bemerkung: Es kann auch ein Polynom tieferen Grades an die Daten gefittet werden. Dann kann es aber nicht mehr exakt durch alle Datenpunkte gehen. In diesem Fall wird die Methode der kleinsten Quadrate (*engl. least squares fit*) verwendet und das Resultat ist nicht mehr eine Interpolation sondern eine Regression.

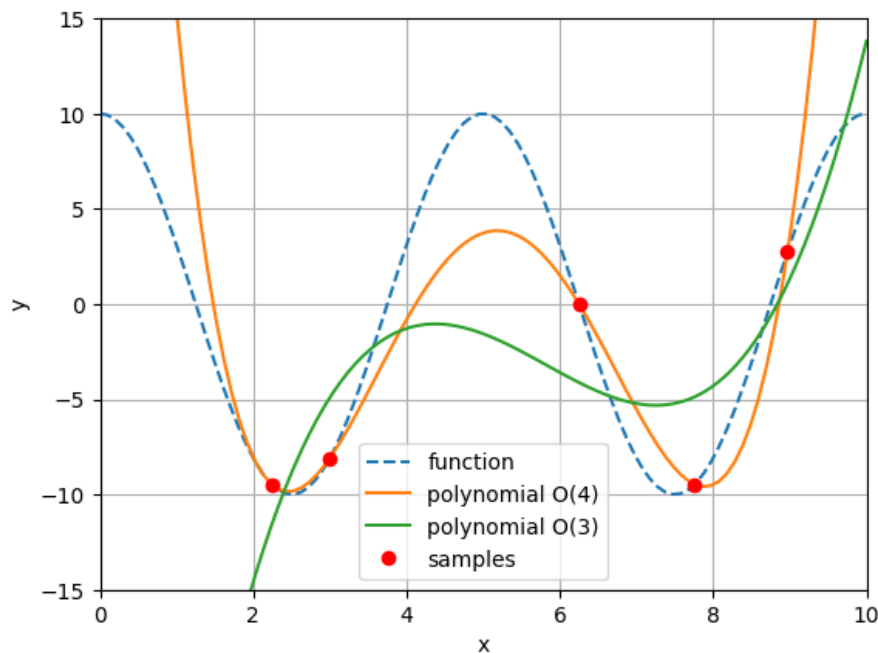
```
poly3 = np.polynomial.Polynomial.fit(x_data, y_data, deg=3)
poly3

Polynomial([ -2.73012492, -7.33687495, -2.48471424, 13.61417005], domain=[2.25, 8.97], window=
```

Darstellung

```
y_poly = poly(x_signal)
y_poly3 = poly3(x_signal)

plt.plot(x_signal, y_signal, "--", label="function")
plt.plot(x_signal, y_poly, label=f"polynomial 0({deg})")
plt.plot(x_signal, y_poly3, label="polynomial 0(3)")
plt.plot(x_data, y_data, "or", label="samples")
plt.xlim(0, 10)
plt.ylim(-15, 15)
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```

2. Implementation der Linearen Interpolation entlang mehrerer Punkte Die lineare Interpolation zwischen zwei Datenpunkten (x_1, y_1) und (x_2, y_2) ist eine Gerade, die durch die beiden Punkte geht. Die Steigung m entspricht dem Differenzenquotienten und dann wird die Gerade so verschoben, dass sie durch den ersten Punkt geht (und somit auch durch den zweiten Punkt).

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$y = y_1 + m \cdot (x - x_1)$$

Haben wir mehrere Datenpunkte, (x_i, y_i) mit $i = 0, 1, \dots, n - 1$, so können wir jeweils zwischen zwei benachbarten Stützstellen linear interpolieren.

Schritte

1. Index i und $i + 1$ der benachbarten Stützstellen suchen: `np.searchsorted`
2. Lineare Interpolation mithilfe der benachbarten Datenpunkte konstruieren

$$m_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

$$y = y_i + m_i \cdot (x - x_i)$$

```
x = 5
i = np.searchsorted(x_data, x) - 1
mi = (y_data[i + 1] - y_data[i]) / (x_data[i + 1] - x_data[i])
y = y_data[i] + mi * (x - x_data[i])
y
np.float64( -3.111538461538461)
```

Nun etwas allgemeiner mithilfe von `np.searchsorted` und `np.diff`:

```
def lin_interp(x_data, y_data, x):
    i = np.searchsorted(x_data, x) - 1
```

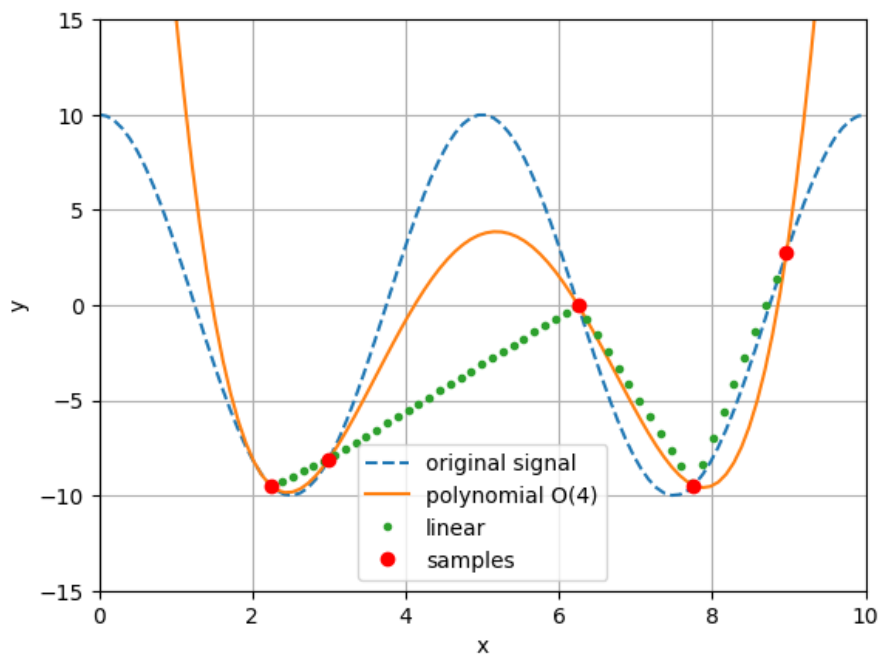
```
i = np.where(i < 0, 0, i)
m = np.diff(y_data) / np.diff(x_data)
return y_data[i] + m[i] * (x - x_data[i])
```

```
lin_interp(x_data, y_data, x)
```

```
np.float64( -3.111538461538461)
```

```
x_interp = np.linspace(x_data.min(), x_data.max())
y_linear = lin_interp(x_data, y_data, x_interp)
```

```
plt.plot(x_signal, y_signal, " - -", label="original signal")
plt.plot(x_signal, y_poly, " -", label=f"polynomial 0({deg})")
plt.plot(x_interp, y_linear, ".", label="linear")
plt.plot(x_data, y_data, "ro", label="samples")
plt.xlim(0, 10)
plt.ylim( -15, 15)
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```



3. Spline-Interpolation mit SciPy

- Übersicht zu Interpolation mit SciPy `scipy.interpolate`.
- Die Funktion `scipy.interpolate.make_interp_spline` ist der wesentliche Einstiegspunkt.
 - Die Option `k` bestimmt den Grad des Polynoms, das für die Interpolation verwendet wird. Standardmässig wird mit `k=3` ein kubisches Spline berechnet. `k=1` entspricht einer linearen Interpolation, `k=2` ist ein quadratischer Spline.
 - Die Option `bc_type` bestimmt die Randbedingungen. Meist liefert `bc_type="natural"` bessere Resultate als die Standardeinstellung `bc_type="not-a-knot"`. Lies dazu die Dokumentation.

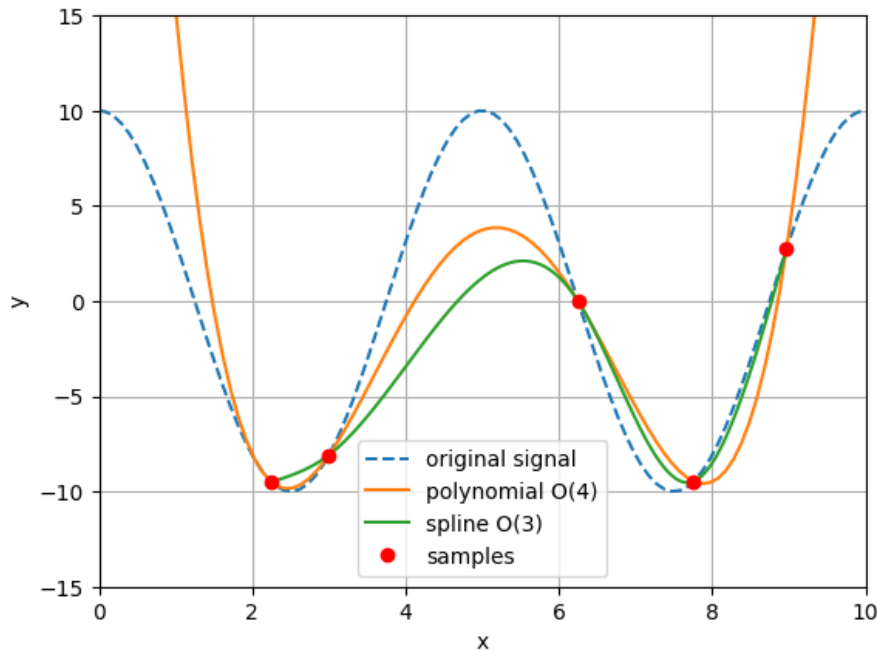
- Viele Beispiele, die man online findet, verwenden die Funktion `scipy.interpolate.interp1d`. Diese Funktion funktioniert sehr ähnlich, bietet aber nicht alle Möglichkeiten von `make_interp_spline` und wird in der Dokumentation als veraltet (legacy) bezeichnet.
- Das Sub-Package `scipy.interpolate` bietet auch Möglichkeiten zur Interpolation mehrdimensionaler Daten.
- Weitere Informationen zu Splines findet man hier
 - “Spline-Interpolation”, Wikipedia
 - “Cubic Spline Interpolation”, Wikiversity.

```
from scipy.interpolate import make_interp_spline

k = 3
spline_clamped = make_interp_spline(x_data, y_data, k, bc_type="clamped")
spline_natural = make_interp_spline(x_data, y_data, k, bc_type="natural")

x_interp = np.linspace(x_data.min(), x_data.max(), 250)
y_spl = spline_natural(x_interp)

plt.plot(x_signal, y_signal, " - -", label="original signal")
plt.plot(x_signal, y_poly, " -", label=f"polynomial O({deg})")
plt.plot(x_interp, y_spl, label=f"spline O({k})")
plt.plot(x_data, y_data, "ro", label="samples")
plt.xlim(0, 10)
plt.ylim(-15, 15)
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```



4. Kubische Spline-Interpolation Bei der kubischen Spline-Interpolation werden je zwei benachbarte Datenpunkte (x_j, y_j) und (x_{j+1}, y_{j+1}) durch ein kubisches Polynom verbunden:

$$s_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3 \quad (31)$$

Dabei wird gefordert, dass

- die Polynome durch die Stützstellen gehen:
 $s_j(x_j) = y_j$ und $s_j(x_{j+1}) = y_{j+1}$
- benachbarte Polynome an den Stützstellen die gleichen Ableitungen haben:
 $s'_j(x_{j+1}) = s'_{j+1}(x_{j+1})$ und $s''_j(x_{j+1}) = s''_{j+1}(x_{j+1})$
- die Polynome an den Randpunkten keiner "externen Kraft" ausgesetzt sind (natürliche Randbedingungen):
 $s''_0(x_0) = s''_{n-1}(x_n) = 0$

Beispiel: Drei Punkte, Zwei Intervalle Gesucht sind die beiden Spline-Polynome durch die folgenden drei Punkte:

$$(x_0, y_0) = (0, 1)$$

$$(x_1, y_1) = (1, 2)$$

$$(x_2, y_2) = (2, 8)$$

Da es 3 Punkte sind, gibt es 2 Intervalle und somit 2 Polynome, deren Koeffizienten bestimmt werden müssen.

Plan:

- Wir schreiben die beiden Polynome auf ($x_0 = 0$ und $x_1 = 1$ einsetzen!).
- Wir berechne deren erste und zweite Ableitungen.
- Wir Schreiben die Bedingungen auf, welche die Polynome und die Ableitungen erfüllen müssen.
- Wir bestimmen daraus ein Gleichungssystem für die Koeffizienten und lösen es.
- Am einfachsten geht dies in der Matrixschreibweise $A \cdot \vec{k} = \vec{c} \Rightarrow \vec{k} = A^{-1} \cdot \vec{c}$, wobei $\vec{k} = (a_0, b_0, c_0, d_0, a_1, b_1, c_1, d_1)$ der Vektor der gesuchten Koeffizienten ist. Das Inverse kann mit NumPy oder Wolfram|Alpha berechnet werden.
- Wir stellen die linear verbundenen Punkte und die kubischen Splines grafisch dar.

Lösung Die gesuchten Polynome sind:

$$s_0(x) = a_0 + b_0x + c_0x^2 + d_0x^3$$

$$s_1(x) = a_1 + b_1(x - 1) + c_1(x - 1)^2 + d_1(x - 1)^3$$

Ableitungen:

$$s'_0(x) = b_0 + 2c_0x + 3d_0x^2$$

$$s''_0(x) = 2c_0 + 6d_0x$$

$$s'_1(x) = b_1 + 2c_1(x - 1) + 3d_1(x - 1)^2$$

$$s''_1(x) = 2c_1 + 6d_1(x - 1)$$

Bedingungen durch Funktionswerte:

$$s_0(x_0) = a_0 = 1$$

$$s_0(x_1) = a_0 + b_0 + c_0 + d_0 = 2$$

$$s_1(x_1) = a_1 = 2$$

$$s_1(x_2) = a_1 + b_1 + c_1 + d_1 = 8$$

Ableitungsbedingungen an den inneren Punkten:

$$\begin{aligned} s'_0(x_1) = s'_1(x_1) &\Rightarrow b_0 + 2c_0 + 3d_0 = b_1 \\ s''_0(x_1) = s''_1(x_1) &\Rightarrow 2c_0 + 6d_0 = 2c_1 \end{aligned}$$

Natürliche Randbedingungen:

$$\begin{aligned} s''_0(x_0) &= 2c_0 = 0 \\ s''_1(x_2) &= 2c_1 + 6d_1 = 0 \end{aligned}$$

Zusammen ergibt sich das Gleichungssystem:

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 0 & -1 & 0 & 0 \\ 0 & 0 & 2 & 6 & 0 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & 6 \end{pmatrix} \cdot \begin{pmatrix} a_0 \\ b_0 \\ c_0 \\ d_0 \\ a_1 \\ b_1 \\ c_1 \\ d_1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 2 \\ 8 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad (32)$$

```
A = np.array(
    [
        [1, 0, 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0],
        [0, 0, 0, 0, 1, 1, 1, 1],
        [0, 1, 2, 3, 0, -1, 0, 0],
        [0, 0, 2, 6, 0, 0, -2, 0],
        [0, 0, 2, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 2, 6],
    ]
)
c = np.array([1, 2, 2, 8, 0, 0, 0, 0])
k = np.linalg.inv(A) @ c
k

array([ 1.   , -0.25,  0.   ,  1.25,  2.   ,  3.5  ,  3.75, -1.25])

s0 = np.polynomial.Polynomial(k[:4])
s1 = np.polynomial.Polynomial(k[4:])

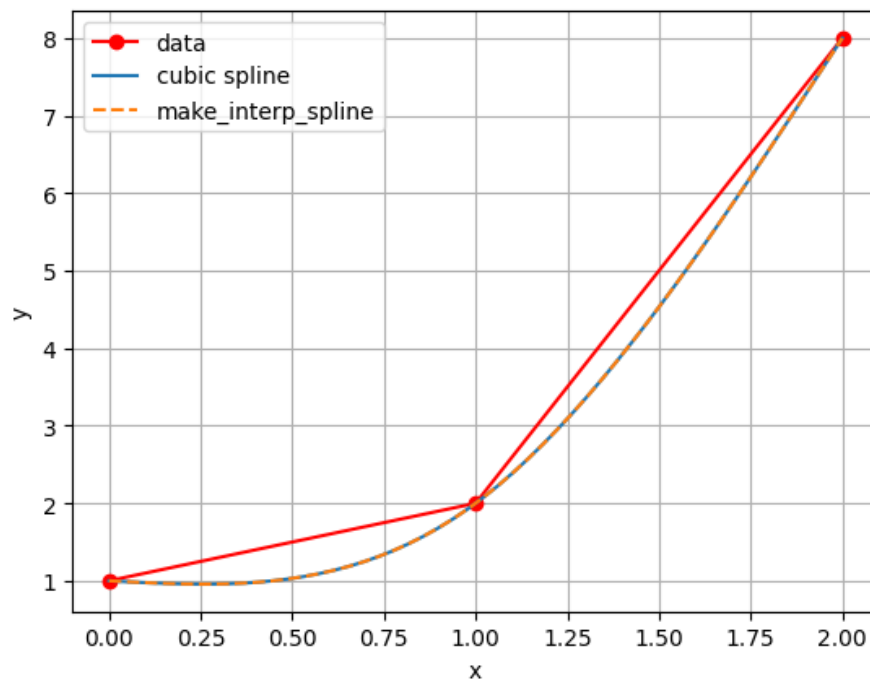
def my_spline(x):
    return np.where(x < 1, s0(x), s1(x - 1))

x_data = np.arange(3)
y_data = np.array([1, 2, 8])
x = np.linspace(0, 2)

# analytisch berechneter Spline
y_spline = my_spline(x)
```

```
# zum Vergleich Spline mit scipy
spline_func = make_interp_spline(x_data, y_data, bc_type="natural")
y_spline_2 = spline_func(x)

plt.plot(x_data, y_data, "ro -", label="data")
plt.plot(x, y_spline, label="cubic spline")
plt.plot(x, y_spline_2, " - -", label="make_interp_spline")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```



0.5 Nullstellen

0.5.1 Nullstellen

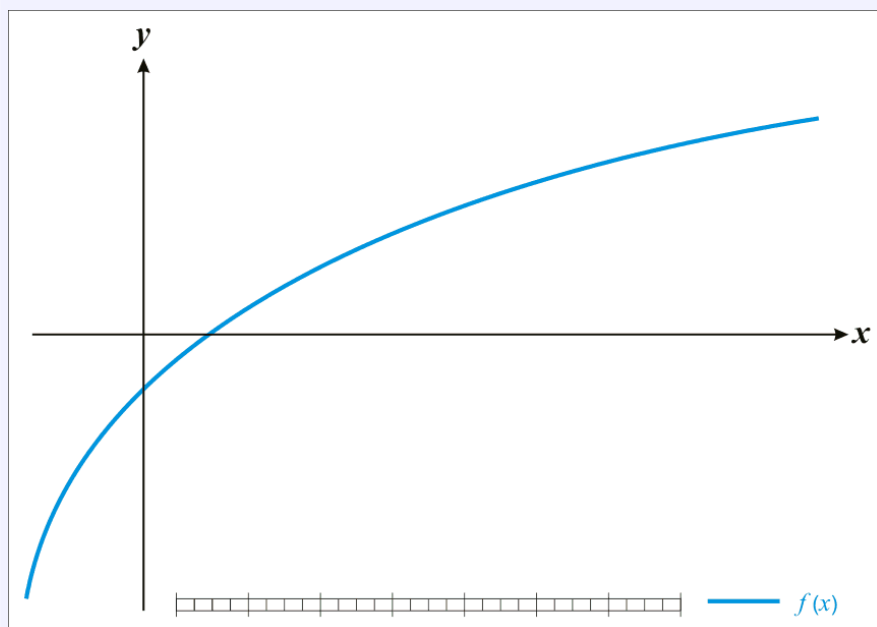
Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Hier lernst du zwei Verfahren zur Berechnung von Nullstellen von Funktionen kennen. Das Ziel ist, deren Funktionsweise kennenzulernen, indem du sie selbst rudimentär implementierst. Details und Verfeinerungen folgen in der Vertiefung.

Bisektionsverfahren Das einfachste Nullstellenverfahren beruht auf dem Zwischenwertsatz der Analysis. Dieser besagt, dass eine stetige Funktion auf einem Intervall $[a, b]$ sicher alle Werte zwischen $f(a)$ und $f(b)$ annimmt. Haben $f(a)$ und $f(b)$ unterschiedliche Vorzeichen, so muss also eine Nullstelle dazwischen liegen. Dies nutzt man, um die Nullstelle durch wiederholtes Halbieren des Intervalls einzuzengen.

Eine kurze Beschreibung des Algorithmus und die folgende Animation findest du hier: “Das Bisektionsverfahren”, Numerik I - Wikiversity.



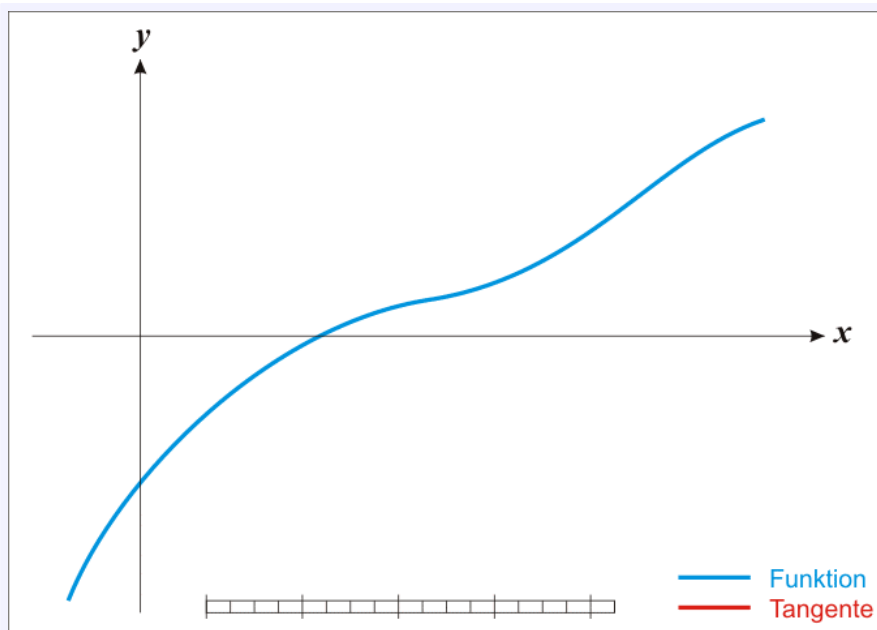
Pfeifer, R. (2005, Mai 22). *Bisektion Ani* [GIF]. Wikimedia Commons (https://commons.wikimedia.org/wiki/File:Bisektion_Ani.gif). CC BY-SA 3.0.

Baue ein grundlegendes Verständnis auf, indem du mindestens eines der folgenden Videos schaust:

- “Bisektionsverfahren, fortgesetzte Bisektion, Intervallhalbierung” - MathePeter
- “Bisektion, Bisektionsverfahren, Intervallhalbierung, Gleichungen lösen” - Daniel Jung

Newton-Verfahren Das Newton-Verfahren nutzt die Linearisierung der Funktion, deren Nullstelle zu berechnen ist. Ausgehend von einer Schätzung der Nullstelle liefert die Nullstelle der Linearisierung eine neue Näherung der gesuchten Nullstelle. Diese Iteration konvergiert in der Regel schnell.

Eine kurze Beschreibung des Algorithmus und die folgende Animation findest du hier: “Das Newton-Verfahren”, Numerik I - Wikiversity.



Pfeifer, R. (2005, April 9). *NewtonIteration Ani* [GIF]. Wikimedia Commons (https://commons.wikimedia.org/wiki/File:NewtonIteration_Ani.gif). CC BY-SA 3.0.

Schaue mindestens eines der folgenden Videos an:

- “Newton-Verfahren (Nullstellen bestimmen)” - Mathe-simpleclub
- “Newton Verfahren” - MathePeter
- Playlist: “Newton-Verfahren, Gleichungen lösen, Newton-Raphson-Verfahren” - Daniel Jung

Gleichungen als Nullstellenprobleme Jegliche Gleichungen, in denen keine Ableitungen oder Integrationen vorkommen, können als Nullstellen von Funktionen umgeschrieben werden. Man nimmt einfach alle Terme auf die linke Seite der Gleichung und erhält so eine Nullstellengleichung einer Funktion.

Beispiel:

$$x^2 - 5 = 2x + \sin(x) \Leftrightarrow f(x) = x^2 - 2x - 5 - \sin(x) \stackrel{!}{=} 0 \quad (33)$$

Bemerkungen:

- Das Finden von Nullstellen wird im Englischen als *root finding* bezeichnet.
- *Fixpunkte* einer Funktion nennt man Punkte x , für die $f(x) = x$ gilt.
 - Fixpunkte sind also Nullstellen der Funktion $g(x) = f(x) - x$.

Arbeitsauftrag

Aufgabe 1 Wofür brauchen wir die Nullstellen einer Funktion?

Lösung (nicht abschliessend)

- Schnittpunkte zwischen zwei Funktionen sind Nullstellen der Differenz der Beiden Funktionen.
- Höhenlinien sind Nullstellen der Differenz der Funktion und des konstanten Levels.
- Optimierung: Nullstellen der Ableitung einer Funktion können Extremstellen sein.

Aufgabe 2 Entscheide dich für die restlichen Aufgaben für eines der beiden Verfahren: **Bisektion oder Newton.** (Das Bisektionsverfahren ist etwas schwieriger zu implementieren.)

Schreibe das grundsätzliche Vorgehen (Ablauf des Algorithmus) für die Berechnung von Nullstellen anhand des gewählten Verfahrens auf.

Lösung:

- Bisektion:

1. Gegeben ein Startintervall $(a[i], b[i])$ mit $i = 0$
 2. Berechne die Funktion an den Grenzen. Falls $f(a[i]) * f(b[i]) > 0$, dann breche mit Fehlermeldung ab.
 3. Bestimme das Zentrum des Intervalls $x[i] = (b[i] - a[i]) / 2$
 4. Falls
 - $f(x[i]) = 0$ dann breche ab mit Erfolg.
 - $f(a[i]) * f(x[i]) < 0$ dann setze $a[i + 1] = a[i]$ und $b[i + 1] = x[i]$
 - sonst setze $a[i + 1] = x[i]$ und $b[i + 1] = b[i]$
 5. Falls Abbruchkriterium nicht erreicht, erhöhe i um 1 und gehe zu Schritte 2.
- Newton:
 1. Neben der Funktion $f(x)$ brauchen wir deren analytische Ableitung $fprime(x)$
 2. Gegeben ein Startpunkt $x[i]$ mit $i = 0$
 3. Berechne die Iterationsvorschrift $x[i + 1] = x[i] - f(x[i]) / fprime(x[i])$
 4. Falls Abbruchkriterium nicht erreicht, erhöhe i um 1 und gehe zu 3.

Bisektion

Startintervall $[a, b]$ wählen

Iteration:

Zentrum des Intervalls $x = (a + b) / 2$

Funktion auswerten $f(x)$

Wenn $f(a) * f(x) > 0$ dann $a = x$

sonst $b = x$

Newton

Newton -Verfahren

Schätzung der Nullstelle x

Iteration:

$x = x - f(x) / f'(x)$

Aufgabe 3 Wir betrachten die Funktion

$$f(x) = 2 - x^2 \tag{34}$$

Sie hat zwei Nullstellen: $x_{\pm} = \pm\sqrt{2}$.

Die irrationale Zahl $\sqrt{2}$ kann nicht direkt von Hand berechnet werden. Daher soll sie hier als Nullstelle der Funktion $f(x)$ näherungsweise numerisch berechnet werden.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
sqrt2 = np.sqrt(2)
print(sqrt2)
```

```
1.4142135623730951
```

a)

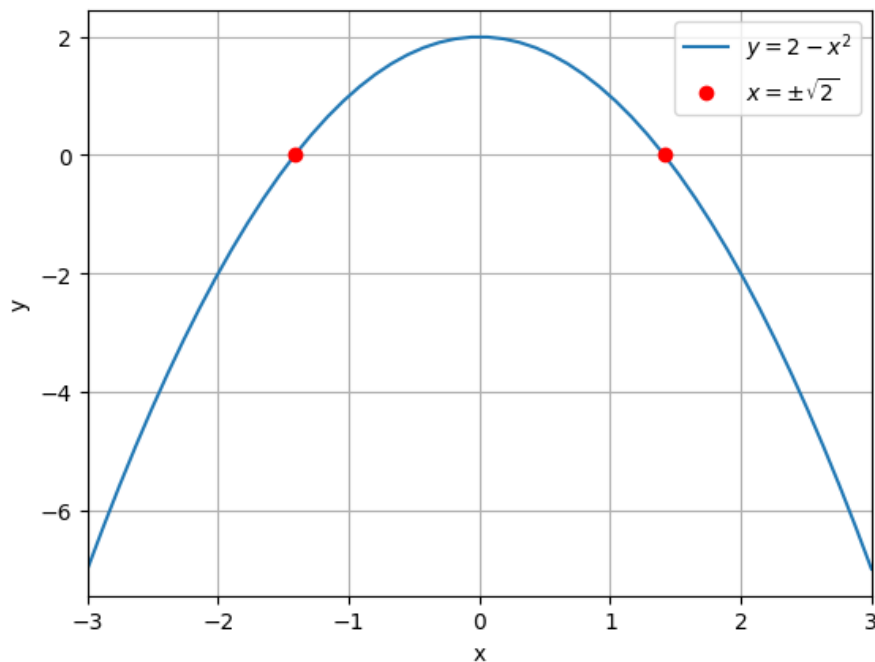
- Definiere die Funktion $f(x) = 2 - x^2$ als Python Funktion $f(x)$.
- Stelle $f(x)$ mithilfe von Matplotlib im Bereich $x \in [-2, 2]$ grafisch dar.
- Stelle zusätzlich die beiden Nullstellen $\pm\sqrt{2}$ als rote Punkte dar.
- Füge Achsenbeschriftungen, eine Legende und ein Gitternetz (Grid) hinzu.

Aufgabe 3.a)

```
def f(x):
    return 2 - x**2

x = np.linspace(-3, 3)
y = f(x)

plt.plot(x, y, label=r"$y=2-x^2$")
plt.plot([-sqrt(2), sqrt(2)], [0, 0], "ro", label=r"$x=\pm\sqrt{2}$")
plt.xlim(x.min(), x.max())
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```



b) Wähle ein geeignetes Start-Intervall $[a_0, b_0]$ für das Bisektionsverfahren bzw. einen geeigneten Startwert x_0 für das Newton-Verfahren, abhängig von der in Aufgabe 2 gewählten Methode. Ziel ist, die positive $\sqrt{2}$ als Nullstelle zu bestimmen.

Lösung:

Beispielsweise:

- Bisektion: $[a_0, b_0] = [1, 2]$
- Newton: $x_0 = 1$

Aufgabe 4 Berechne mit Excel die Näherung der Nullstelle nach 2 Iterationsschritten des gewählten Verfahrens, also $[a_2, b_2]$ bzw. x_2 . Schreibe das Resultat hier auf:

Lösung:

Mit den Startwerten von oben:

- Bisektion: $[a_2, b_2] = [1.25, 1.5] \Rightarrow x_2 = \frac{a_2 + b_2}{2} = 1.375$
- Newton: $x_2 = 1.41\bar{6}$

Wenn du andere Startwerte gewählt hast, wirst du auch andere Werte nach zwei Iterationen erhalten, siehe Excel-Datei `04_Einarbeitung_Loesung.xlsx`.

Aufgabe 5 Schreibe eine Funktion in Python, die das gewählte Verfahren implementiert.

- *Inputs:*
 - Bisektion: Funktion f , Start-Intervall a_0 , b_0 und Anzahl Iterationsschritte n
 - Newton: Funktion f , deren Ableitung f_{prime} , Startwert x_0 und Anzahl Iterationsschritte n
- *Output:* x -Wert der Nullstelle nach der Anzahl Schritte. (Bei Bisektion den Mittelpunkt des Intervalls zurückgeben.)
- *Option:* Es kann auch noch der Funktionswert an der gefundenen Stelle als Qualitätsindikator ausgegeben werden.

Teste deine Funktion, indem Du mit den Resultaten aus Aufgabe 4 und `x_wolfram` vergleichst.

Aufgabe 5

```
def fprime(x):
    """Ableitung von f(x) = 2 - x^2"""

    return -2 * x

def bisection(f, a0, b0, n):
    a, b = a0, b0
    x = (a + b) / 2
    i = 0
    while i < n:
        if f(a) * f(x) > 0:
            a = x
        else:
            b = x
        x = (a + b) / 2
        i += 1
    return x

print("bisection: x_2 =", bisection(f, 1, 2, 2))

bisection: x_2 = 1.375

def newton(f, fprime, x0, n):
    x = x0
    i = 0
    while i < n:
        x = x - f(x) / fprime(x)
        i += 1
    return x

print("newton: x_2 =", newton(f, fprime, 1, 2))

newton: x_2 = 1.4166666666666667
```

Aufgabe 6 Experimentiere mit deiner Lösung aus Aufgabe 5 oder dem Excel-Sheet aus Aufgabe 4, indem du die Anzahl Iterationsschritte erhöhst und den Startwert veränderst. Findest du auch die negative Nullstelle bei $x = -\sqrt{2}$? Notiere deine Beobachtungen.

Zunehmende Anzahl Iterationen

```
for n in 10 * np.arange(1, 6):
    print(f"n = {n}")
    print("bisection: ", bisection(f, 1, 2, n) - sqrt2)
    print("newton:    ", newton(f, fprime, 1, n) - sqrt2)
    print()
```

```
n = 10
bisection: 0.0003372188769048545
newton:    -2.220446049250313e -16
```

```
n = 20
bisection: 9.500605524515038e -08
newton:    -2.220446049250313e -16
```

```
n = 30
bisection: -4.545086529361697e -10
newton:    -2.220446049250313e -16
```

```
n = 40
bisection: -2.1604940059205546e -13
newton:    -2.220446049250313e -16
```

```
n = 50
bisection: 2.220446049250313e -16
newton:    -2.220446049250313e -16
```

Negative Nullstelle?

```
print("bisection: ", bisection(f, -2, -1, 10))
```

```
print("newton:    ", newton(f, fprime, -2, 10))
```

```
bisection: -1.41455078125
newton:    -1.414213562373095
```

Beobachtungen:

1. Das Newton-Verfahren konvergiert schneller als das Bisektionsverfahren.
2. Welche der beiden Nullstellen gefunden wird, hängt von der Wahl des Startintervalls bzw. Startwertes ab.

0.5.2 Nullstellen

Übungen - Lösungen

```
import numpy as np
import matplotlib.pyplot as plt
```

Aufgabe 1: Unterschiede zwischen Bisektionsverfahren und Newtonverfahren Beschreibe und bewerte qualitativ die unterschiedlichen Eigenschaften des Bisektionsverfahrens und des Newtonverfahrens. In welchen Situationen ist welches Verfahren besser geeignet?

Lösung: (nicht abschliessend)

Eigenschaft	Bisektionsverfahren	Newton-Verfahren
Startwert	Intervall	Schätzung
Konvergenz	(-) langsam	(+) schnell
analytische Ableitung	(+) nicht nötig	(-) nötig
Nullstellen der Ableitung	(+) kein Problem	(-) kann ein Problem sein
Fehler in x-Richtung	(+) einfach zu schätzen	(-) schwer zu schätzen
Fehler in y-Richtung	einfach zu schätzen	einfach zu schätzen

Aufgabe 2: Nullstellen einer Funktion

a) Darstellung Wir betrachten folgende Funktion, die Nullstellen bei $x = -1, 3$ und 4 hat:

$$f(x) = x^3 - 6x^2 + 5x + 12 \quad (35)$$

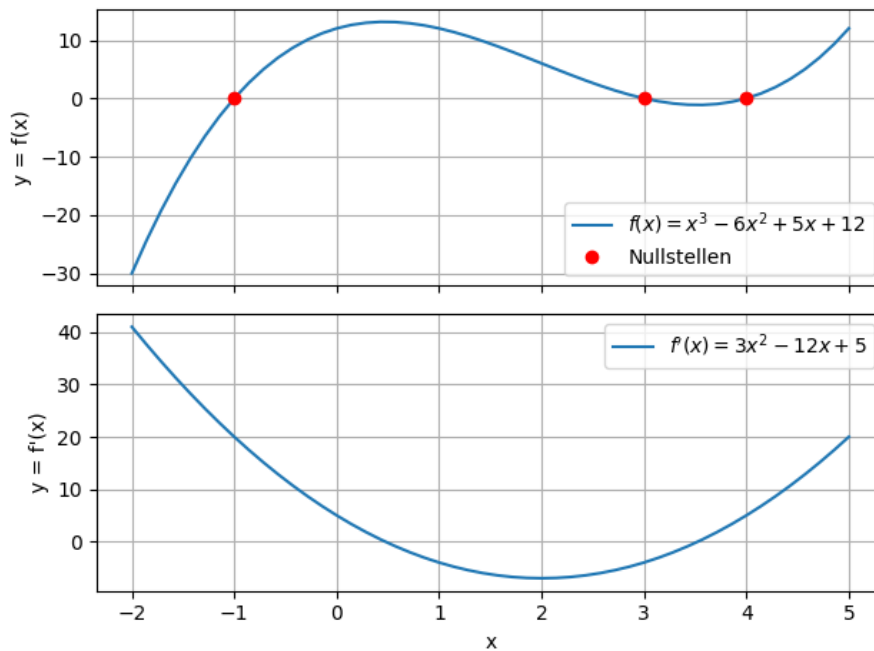
- Berechne die Ableitung von Hand und definiere $f(x)$ und $f'(x)$ als Python Funktionen `f(x)` und `fprime(x)`.
- Stelle beide Funktionen im Bereich $x \in [-2, 5]$ dar und markiere die Nullstellen von $f(x)$.

```
def f(x):
    return x**3 - 6 * x**2 + 5 * x + 12

def fprime(x):
    return 3 * x**2 - 12 * x + 5

x_roots = np.array([-1, 3, 4])
x = np.linspace(-2, 5)

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
ax1.plot(x, f(x), label=r"$f(x) = x^3 - 6x^2 + 5x + 12$")
ax1.plot(x_roots, np.zeros_like(x_roots), "ro", label="Nullstellen")
ax1.set_ylabel("y = f(x)")
ax1.legend()
ax1.grid()
ax2.plot(x, fprime(x), label=r"$f'(x) = 3x^2 - 12x + 5$")
ax2.set_xlabel("x")
ax2.set_ylabel("y = f'(x)")
ax2.legend()
ax2.grid()
plt.tight_layout()
plt.show()
```



b) Nullstellen bestimmen mit SciPy Wie gesagt hat die Funktion 3 Nullstellen in diesem Bereich.

- Benutze die Funktion `scipy.optimize.newton`, um eine der Nullstellen zu berechnen. Gebe der SciPy Funktion `f`, den Startwert `x0=0` und die Ableitung `fprime` mit.
- Probiere auch mal `scipy.optimize.root_scalar` aus.
- Benutze auch die Implementierungen des Newton-Verfahrens und des Bisektionsverfahrens aus der Vertiefung, um die Nullstellen zu berechnen. Vergleiche die Ergebnisse.

Welche der Nullstellen wurde gefunden? Welches Verfahren konvergiert schneller?

```
from scipy import optimize

x0 = 0
optimize.newton(f, x0, fprime)

np.float64( -1.0)

optimize.root_scalar(f, x0=x0, fprime=fprime)

converged: True
      flag: converged
function_calls: 14
      iterations: 7
         root: -1.0
      method: newton

def newton(f, fprime, x0, xtol, ytol, maxiter):
    """Newton -Verfahren aus der Vertiefung"""
    assert xtol > 0
    assert ytol > 0
    assert maxiter >= 0

    x = x0
    y = f(x)
    y_by_yprime = y / fprime(x)
```

```

Dy = abs(y)
Dx = abs(y_by_yprime)
steps = 0
res = [(steps, x, Dx, Dy)]

while (Dx > xtol or Dy > ytol) and steps < maxiter:
    x = x - y_by_yprime
    y = f(x)
    y_by_yprime = y / fprime(x)
    Dy = abs(y)
    Dx = abs(y_by_yprime)
    steps += 1
    res.append((steps, x, Dx, Dy))

return np.array(res)

def bisection(f, a0, b0, xtol, ytol, maxiter):
    """Bisektionsverfahren aus der Vertiefung"""
    assert a0 < b0
    assert f(a0) * f(b0) < 0
    assert xtol > 0
    assert ytol > 0
    assert maxiter >= 0

    a, b = a0, b0
    x = (a + b) / 2
    y = f(x)
    ya = f(a)
    Dx = abs(b - a) / 2
    Dy = abs(y)
    steps = 0
    res = [(steps, x, Dx, Dy)]

    while (Dx > xtol or Dy > ytol) and steps < maxiter:
        if ya * y < 0:
            b = x
        else:
            a = x
            ya = y
        x = (a + b) / 2
        y = f(x)
        Dx = abs(b - a) / 2
        Dy = abs(y)
        steps += 1
        res.append((steps, x, Dx, Dy))

    return np.array(res)

print(newton(f, fprime, x0, 1e -8, 1e -8, 100)[-1])
print(bisection(f, -2, 5, 1e -8, 1e -8, 100)[-1])

[ 6.00000000e+00 -1.00000000e+00  1.90709670e -12  3.81419341e -11]
[ 3.20000000e+01 -1.00000000e+00  8.14907253e -10  2.32830644e -09]

```


c) Mehrere Startwerte für Newton-Verfahren Die Funktion `scipy.optimize.newton` kann auch mit einem Array aus mehreren Startwerten arbeiten. Ausgehend von jedem dieser Startwerte wird eine Nullstelle gesucht. Probiere es aus, indem Du ein paar Startwerte zusammenpackst.

```
x0 = np.array([-2, 0, 5])
optimize.newton(f, x0, fprime)

array([-1., -1.,  4.] )
```

d) Mehrere Nullstellen mit zufälligen Startwerten finden. Nun sollen die Startwerte zufällig ausgewählt werden.

1. Schreibe eine Funktion, die für eine gegebene Anzahl Versuche alle Nullstellen einer Funktion finden soll. Die Startwerte sollen innerhalb eines definierten Intervalls liegen und aus diesem zufällig gewählt werden.

Verwende für das Finden von den Nullstellen die SciPy Funktion `scipy.optimize.newton`.

Funktion `root_finding(f, fprime, low, high, trials, seed=1)`:

- `f`: Funktion, deren Nullstellen gesucht werden sollen
- `fprime`: Die Ableitung von `f`
- `low`: untere Intervallsgrenze
- `high`: obere Intervallsgrenze
- `trials`: Anzahl Versuche, Nullstellen zu finden
- `seed=123456`: Seed des Zufallszahlengenerators mit Default 123456

Die Funktion soll ein Array zurückliefern, wobei in der ersten Spalte die zufälligen Startwerte und in der zweiten Spalte die zugehörigen Nullstellen stehen.

```
def root_finding(f, fprime, low, high, trials, seed=123456):
    rng = np.random.default_rng(seed)
    x0 = rng.uniform(low, high, size=trials)
    x = optimize.newton(f, x0, fprime)
    return np.array([x0, x]).T
```

2. Teste `root_finding` mit der Funktion $f(x)$ aus Teil a) und 3 Startwerten im Intervall $[-2, 5]$. Du sollst die Nullstellen 3, 4 und -1 finden, wenn du den Seed bei 123456 belässt.

```
root_finding(f, fprime, -2, 5, 3)

array([[ 2.45559625,  3.         ],
       [ 0.69368165,  4.         ],
       [-1.66788205, -1.         ]])
```

Wenn du wesentlich mehr Startwerte erzeugst, findest Du natürlich immer wieder die gleichen Nullstellen. Teste dies mit 25 Startwerten oder mehr.

```
root_finding(f, fprime, -2, 5, 25)

array([[ 2.45559625,  3.         ],
       [ 0.69368165,  4.         ],
       [-1.66788205, -1.         ],
       [ 4.68676918,  4.         ],
       [ 4.34235656,  4.         ],
       [ 1.19878688,  3.         ],
       [-0.03626567, -1.         ],
       [ 2.45304162,  3.         ],
       [ 4.86497483,  4.         ],
       [ 3.6870225 ,  4.         ],
```

```
[ 0.13638771, -1.      ],
[ 1.9760767 ,  3.      ],
[ -1.85518979, -1.      ],
[ 2.01170345,  3.      ],
[ 0.88533942,  4.      ],
[ 0.41701228, -1.      ],
[ 3.11316375,  3.      ],
[ -0.02789082, -1.      ],
[ -1.84365879, -1.      ],
[ -0.24095231, -1.      ],
[ -0.32567949, -1.      ],
[ 2.3339966 ,  3.      ],
[ 3.51358924, -1.      ],
[ 0.01176857, -1.      ],
[ 0.44244442, -1.      ]])
```

e) Nur unterschiedliche Nullstellen finden 1. Benutze `root_finding` aus Teil c), um nur unterschiedliche Nullstellen zu finden.

- Schreibe dazu eine neue Funktion `root_finding_unique`, welche `root_finding` benutzt.
- Die Funktion soll nur noch die Nullstellen als Array liefern und nicht mehr die Startwerte.
- Es sollten jetzt keine doppelten Einträge mehr vorkommen. Nutze dazu `np.unique`.

```
def root_finding_unique(f, fprime, low, high, trials, seed=1):
    _, x = root_finding(f, fprime, low, high, trials, seed).T
    return np.unique(x)
```

2. Teste `root_finding_unique` mit der Funktion $f(x)$ und zuerst 5 und dann 50 Startwerten im Intervall $[-2, 5]$.

Benutze weiterhin den Seed 123456, um vergleichbare Ergebnisse zu erhalten.

```
root_finding_unique(f, fprime, -2, 5, 5)

array([-1.,  3.,  4.])

root_finding_unique(f, fprime, -2, 5, 50)

array([-1.,  3.,  3.,  3.,  3.,  4.])

for x in root_finding_unique(f, fprime, -2, 5, 50):
    print(x)

-1.0
2.999999999999998
2.999999999999987
3.0
3.000000000000001
4.0
```

Warum haben wir im Fall `trials=50` trotzdem am Ende noch vier Mal (beinahe) die 3 in der Lösung?

Lösung: Die Funktion `np.unique` entfernt nur doppelte Einträge, wenn sie innerhalb der Maschinenpräzision gleich sind. Da die Nullstellen numerisch bestimmt werden, können die Ergebnisse innerhalb der Toleranz des Newton-Verfahrens leicht variieren. Die Toleranz ist normalerweise grösser als die Maschinenpräzision.

Aufgabe 3: Nullstellen von Daten

a) Daten importieren In der Datei `weight.csv` sind Gewichtsmessungen an verschiedenen Tagen zu finden. Die erste Spalte gibt das Datum, die zweite die Anzahl Tage seit der ersten Messung und die dritte das gemessene Gewicht in kg an.

- Importiere die zweite und dritte Spalte der Datei `weight.csv` in zwei 1D-Arrays namens `day` und `weight`.
- Bestimme wie viele Messerte es gibt und in welchem Zeitraum (Anzahl Tage) die Messungen gemacht wurden.

```
day, weight = np.loadtxt("weight.csv", delimiter=";", usecols=(1, 2), skiprows=1).T
```

```
len(day), float(day.min()), float(day.max())
```

```
# Es gibt 76 Messwerte, die von Tag 0 bis Tag 96 reichen.
```

```
# Also gibt es nicht an jedem Tag einen Messwert. => fehlende Werte.
```

```
(76, 0.0, 96.0)
```

b) Daten interpolieren

- Bestimme mithilfe von SciPy eine kubische Spline-Interpolationsfunktion der Gewichtsdaten mit natürlichen Randbedingungen. Nenne sie `w_spline`.
- Stelle die Interpolationsfunktion zusammen mit den Daten dar.
 - Dazu musste du zuerst die Interpolationsfunktion an vielen Punkten im Bereich der Daten auswerten, z.B. `n=1000` Punkte.
 - Stelle die Interpolationsfunktion als durchgezogene Linie und die Daten als kleine Punkte dar.
 - Füge Achsenbeschriftungen, Legende und ein Gitternetz hinzu.

```
from scipy.interpolate import make_interp_spline
```

```
w_spline = make_interp_spline(day, weight, k=3, bc_type="natural")
```

```
t = np.linspace(day.min(), day.max(), 1000)
```

```
w = w_spline(t)
```

```
plt.plot(t, w, label="Interpolierte Werte")
```

```
plt.plot(day, weight, ".", label="Messwerte")
```

```
plt.xlabel("Tag")
```

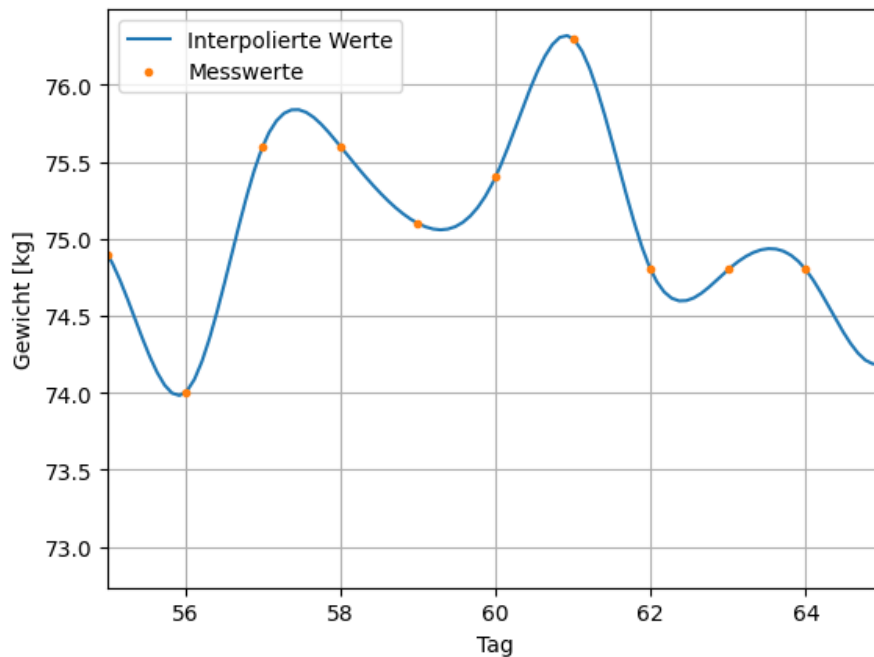
```
plt.ylabel("Gewicht [kg]")
```

```
plt.grid()
```

```
plt.legend()
```

```
plt.xlim(55, 65)
```

```
plt.show()
```



c) Schnittpunkte bestimmen Im Diagramm aus Teil b) wird ersichtlich, dass das Gewicht zwischen den Tagen 55 und 65 insgesamt vier Mal den Schwellenwert 75.5 kg kreuzt.

Das Ziel ist es, die Zeitpunkte dieser vier Schnittpunkte zu bestimmen. Dazu müssen die Nullstellen der Differenz $w_{\text{spline}}(x) - 75.5$ gefunden werden.

- Definiere eine Funktion $f(x)$, welche diese Differenz berechnet.
- Schätze dazu von Auge gute Startwerte bzw. Startintervalle für das Newton- bzw. Bisektionsverfahren.
- Finde alle vier Nullstellen.

Tipp: Folgende Funktionen kannst du für diese Aufgabe benutzen:

- `scipy.optimize.bisect`
- `scipy.optimize.newton`.
- `scipy.optimize.root_scalar`

Bemerkung: Die Funktion `scipy.optimize.newton` kann auch verwendet werden, wenn die Ableitung `fprime` nicht bekannt ist. In diesem Fall wird die Sekantenmethode verwendet, eine leicht angepasste Variante des Newton-Verfahrens, bei dem die Ableitung auch numerisch bestimmt wird.

```
threshold = 75.5
```

```
def f(x):
    return w_spline(x) - threshold

roots = [
    optimize.bisect(f, 56, 57.5),
    optimize.bisect(f, 57.5, 59),
    optimize.newton(f, 60),
    optimize.newton(f, 61.5),
]

print("Die vier Nullstellen befinden sich bei:")
np.array(roots)
```

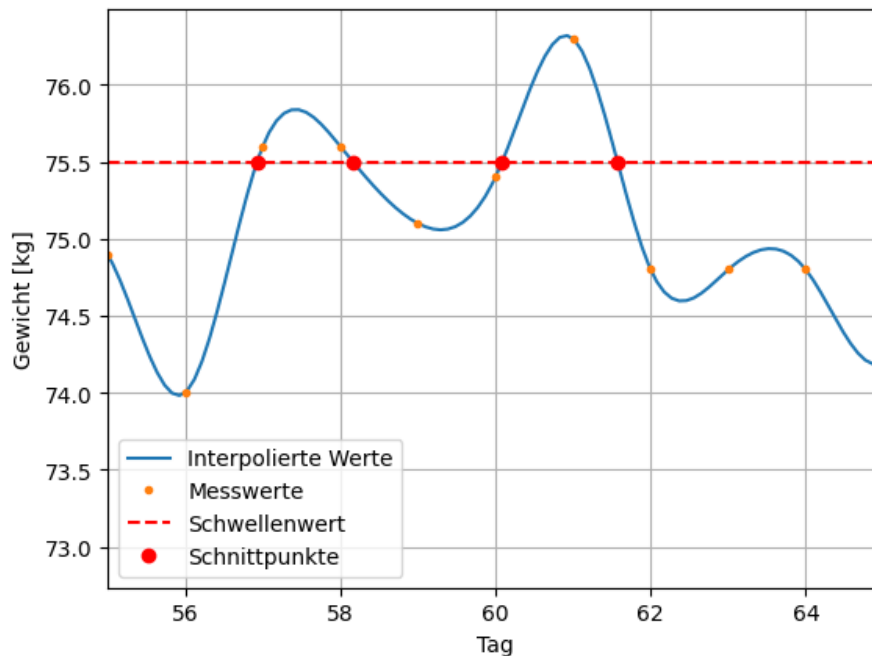
Die vier Nullstellen befinden sich bei:

```
array([56.92735956, 58.16201008, 60.0867981 , 61.57252364])
```

d) Schnittpunkte darstellen

- Plote die Nullstellen im Diagramm mit den interpolierten und gemessenen Daten.
- Stelle die Nullstellen als rote Kreuze dar.

```
plt.plot(t, w, label="Interpolierte Werte")
plt.plot(day, weight, ".", label="Messwerte")
plt.axhline(threshold, color="red", linestyle=" - -", label="Schwellenwert")
plt.plot(roots, np.full_like(roots, threshold), "ro", label="Schnittpunkte")
plt.xlabel("Tag")
plt.ylabel("Gewicht [kg]")
plt.grid()
plt.legend()
plt.xlim(55, 65)
plt.show()
```



0.5.3 Nullstellen

Vertiefung

Übersicht

1. Toleranz als Abbruchkriterium
2. Mehrere Nullstellen
3. Nullstellen bestimmen mit SciPy

```
import numpy as np
import matplotlib.pyplot as plt
```

1. Toleranz als Abbruchkriterium Die folgende Funktion hat nur eine (reelle) Nullstelle und zwar bei $x = 3$:

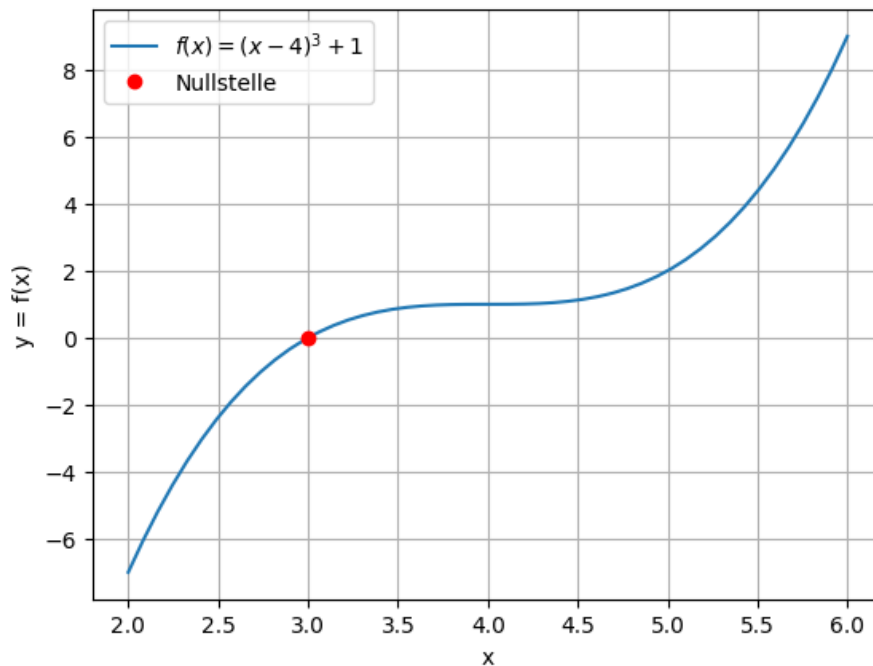
$$f(x) = (x - 4)^3 + 1 \quad (36)$$

```
def f(x):
    return (x - 4) ** 3 + 1
```

```
def fprime(x):
    return 3 * (x - 4) ** 2
```

```
x = np.linspace(2, 6)
y = f(x)
```

```
plt.plot(x, y, label="$f(x) = (x - 4)^3 + 1$")
plt.plot(3, 0, "ro", label="Nullstelle")
plt.ylabel("y = f(x)")
plt.xlabel("x")
plt.grid()
plt.legend()
plt.show()
```



a) Fehlerabschätzung bei Bisektionsverfahren Nach n Schritten, wissen wir, dass die gesuchte Nullstelle x zwischen den Intervallgrenzen a_n und b_n liegt, also $x \in [a_n, b_n]$. Die einfachste Schätzung der Nullstelle ist also die Mitte des Intervalls.

$$x \simeq x_n = \frac{1}{2} (a_n + b_n) \quad (37)$$

Und als Fehlerabschätzung von x können wir die halbe Intervallbreite nehmen

$$\Delta_x = \frac{1}{2} |b_n - a_n| \quad (38)$$

Andererseits kann auch der Funktionswert an der geschätzten Nullstelle als Indikator für die Qualität der Näherung benutzt werden

$$\Delta_y = |f(x_n)| \quad (39)$$

Als Abbruchkriterien können Toleranzen (obere Grenzen) für beide Fehlerabschätzungen benutzt werden.

```
def bisection(f, a0, b0, xtol, ytol, maxiter):
    assert a0 < b0
    assert f(a0) * f(b0) < 0
    assert xtol > 0
    assert ytol > 0
    assert maxiter >= 0

    a, b = a0, b0
    x = (a + b) / 2
    y = f(x)
    ya = f(a)
    Dx = abs(b - a) / 2
    Dy = abs(y)
    steps = 0
    res = [(steps, x, Dx, Dy)]
```

```

while (Dx > xtol or Dy > ytol) and steps < maxiter:
    if ya * y < 0:
        b = x
    else:
        a = x
        ya = y
    x = (a + b) / 2
    y = f(x)
    Dx = abs(b - a) / 2
    Dy = abs(y)
    steps += 1
    res.append((steps, x, Dx, Dy))

```

```

return np.array(res)

```

```

print("Bisection:")
print("\t i \t\t x_i \t\t Dx_i \t\t Dy_i")
bisection(f, 1, 4, 1e -10, 1e -10, 200)

```

Bisection:

```

i      x_i      Dx_i      Dy_i

```

```

array([[0.00000000e+00, 2.50000000e+00, 1.50000000e+00, 2.37500000e+00],
       [1.00000000e+00, 3.25000000e+00, 7.50000000e -01, 5.78125000e -01],
       [2.00000000e+00, 2.87500000e+00, 3.75000000e -01, 4.23828125e -01],
       [3.00000000e+00, 3.06250000e+00, 1.87500000e -01, 1.76025391e -01],
       [4.00000000e+00, 2.96875000e+00, 9.37500000e -02, 9.67102051e -02],
       [5.00000000e+00, 3.01562500e+00, 4.68750000e -02, 4.61463928e -02],
       [6.00000000e+00, 2.99218750e+00, 2.34375000e -02, 2.36210823e -02],
       [7.00000000e+00, 3.00390625e+00, 1.17187500e -02, 1.16730332e -02],
       [8.00000000e+00, 2.99804688e+00, 5.85937500e -03, 5.87082654e -03],
       [9.00000000e+00, 3.00097656e+00, 2.92968750e -03, 2.92682741e -03],
       [1.00000000e+01, 2.99951172e+00, 1.46484375e -03, 1.46555912e -03],
       [1.10000000e+01, 3.00024414e+00, 7.32421875e -04, 7.32243076e -04],
       [1.20000000e+01, 2.99987793e+00, 3.66210938e -04, 3.66255643e -04],
       [1.30000000e+01, 3.00006104e+00, 1.83105469e -04, 1.83094293e -04],
       [1.40000000e+01, 2.99996948e+00, 9.15527344e -05, 9.15555284e -05],
       [1.50000000e+01, 3.00001526e+00, 4.57763672e -05, 4.57756687e -05],
       [1.60000000e+01, 2.99999237e+00, 2.28881836e -05, 2.28883582e -05],
       [1.70000000e+01, 3.00000381e+00, 1.14440918e -05, 1.14440481e -05],
       [1.80000000e+01, 2.99999809e+00, 5.72204590e -06, 5.72205681e -06],
       [1.90000000e+01, 3.00000095e+00, 2.86102295e -06, 2.86102022e -06],
       [2.00000000e+01, 2.99999952e+00, 1.43051147e -06, 1.43051216e -06],
       [2.10000000e+01, 3.00000024e+00, 7.15255737e -07, 7.15255567e -07],
       [2.20000000e+01, 2.99999988e+00, 3.57627869e -07, 3.57627911e -07],
       [2.30000000e+01, 3.00000006e+00, 1.78813934e -07, 1.78813924e -07],
       [2.40000000e+01, 2.99999997e+00, 8.94069672e -08, 8.94069698e -08],
       [2.50000000e+01, 3.00000001e+00, 4.47034836e -08, 4.47034829e -08],
       [2.60000000e+01, 2.99999999e+00, 2.23517418e -08, 2.23517420e -08],
       [2.70000000e+01, 3.00000000e+00, 1.11758709e -08, 1.11758709e -08],
       [2.80000000e+01, 3.00000000e+00, 5.58793545e -09, 5.58793545e -09],
       [2.90000000e+01, 3.00000000e+00, 2.79396772e -09, 2.79396772e -09],
       [3.00000000e+01, 3.00000000e+00, 1.39698386e -09, 1.39698386e -09],

```



```
[3.10000000e+01, 3.00000000e+00, 6.98491931e -10, 6.98491931e -10],
[3.20000000e+01, 3.00000000e+00, 3.49245965e -10, 3.49245965e -10],
[3.30000000e+01, 3.00000000e+00, 1.74622983e -10, 1.74622983e -10],
[3.40000000e+01, 3.00000000e+00, 8.73114914e -11, 8.73114914e -11]])
```

Machen wir noch eine Darstellung davon

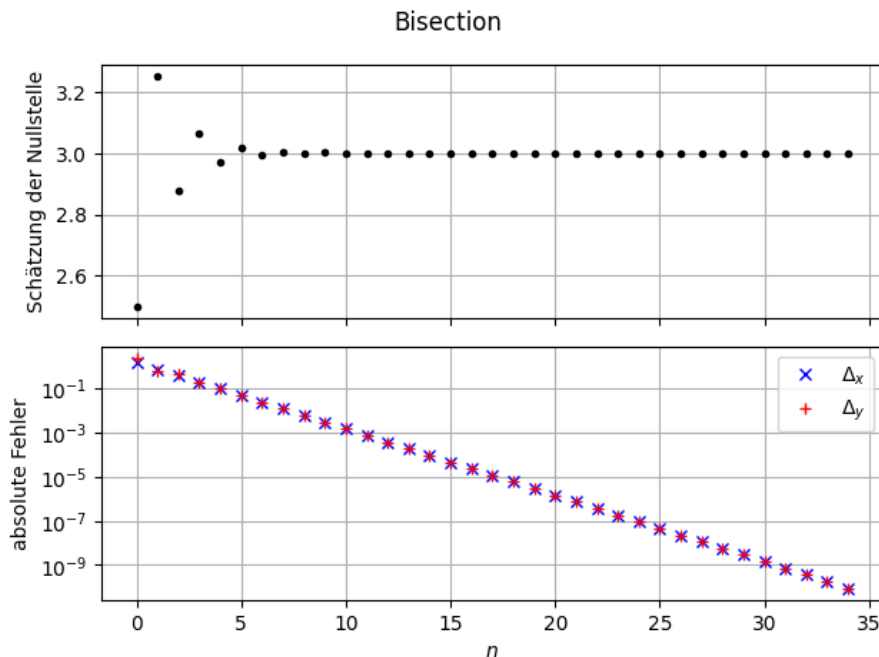
```
res = bisection(f, 1, 4, 1e -10, 1e -10, 200)
steps, x, Dx, Dy = res.T

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

ax1.plot(steps, x, "k.", label="$x_n$")
ax1.set_ylabel("Schätzung der Nullstelle")
ax1.grid()

ax2.set_yscale("log")
ax2.plot(steps, Dx, "bx", label=r"$\Delta_x$")
ax2.plot(steps, Dy, "r+", label=r"$\Delta_y$")
ax2.set_ylabel("absolute Fehler")
ax2.set_xlabel("$n$")
ax2.grid()
ax2.legend()

plt.suptitle("Bisection")
plt.tight_layout()
plt.show()
```



b) Fehlerabschätzung beim Newton-Verfahren Wieder können wir den Funktionswert an der Stelle x_n als Indikator der Qualität der Schätzung der Nullstelle verwenden:

$$\Delta_y = |f(x_n)| \quad (40)$$

Allerdings ist eine Abschätzung des Fehlers in x -Richtung schwieriger. Als Abbruchkriterium können wir aber die Distanz nehmen, die in x -Richtung seit der letzten Iteration zurückgelegt wurde, also

$$\Delta_x \simeq |x_{n+1} - x_n| = \left| \frac{f(x_n)}{f'(x_n)} \right| \quad (41)$$

```
def newton(f, fprime, x0, xtol, ytol, maxiter):
    assert xtol > 0
    assert ytol > 0
    assert maxiter >= 0

    x = x0
    y = f(x)
    y_by_yprime = y / fprime(x)
    Dy = abs(y)
    Dx = abs(y_by_yprime)
    steps = 0
    res = [(steps, x, Dx, Dy)]

    while (Dx > xtol or Dy > ytol) and steps < maxiter:
        x = x - y_by_yprime
        y = f(x)
        y_by_yprime = y / fprime(x)
        Dy = abs(y)
        Dx = abs(y_by_yprime)
        steps += 1
        res.append((steps, x, Dx, Dy))

    return np.array(res)

print("Newton:")
print("\t i \t\t x_i \t\t Dx_i \t\t Dy_i")
newton(f, fprime, 1, 1e -10, 1e -10, 200)

Newton:
  i      x_i      Dx_i      Dy_i

array([[0.00000000e+00, 1.00000000e+00, 9.62962963e -01, 2.60000000e+01],
       [1.00000000e+00, 1.96296296e+00, 5.98681767e -01, 7.45272570e+00],
       [2.00000000e+00, 2.56164473e+00, 3.18332887e -01, 1.97576414e+00],
       [3.00000000e+00, 2.87997762e+00, 1.07620123e -01, 4.05012235e -01],
       [4.00000000e+00, 2.98759774e+00, 1.22509490e -02, 3.76701370e -02],
       [5.00000000e+00, 2.99984869e+00, 1.51288470e -04, 4.54002771e -04],
       [6.00000000e+00, 2.99999998e+00, 2.28905090e -08, 6.86715300e -08],
       [7.00000000e+00, 3.00000000e+00, 4.44089210e -16, 1.33226763e -15]])
```

Darstellung

```
res = newton(f, fprime, 1, 1e -10, 1e -10, 200)
steps, x, Dx, Dy = res.T

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

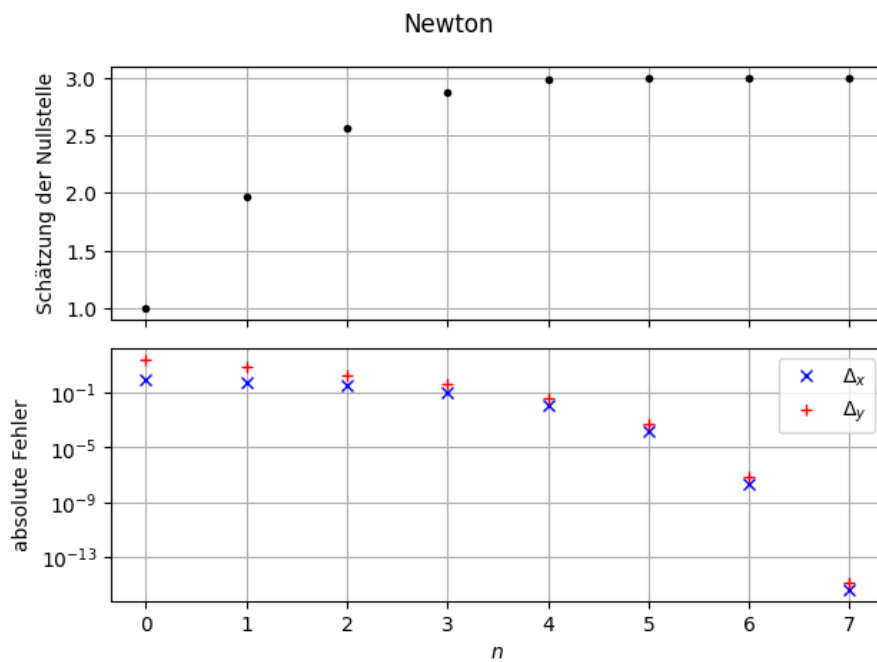
ax1.plot(steps, x, "k.", label="$x_n$")
ax1.set_ylabel("Schätzung der Nullstelle")
ax1.grid()
```

```

ax2.set_yscale("log")
ax2.plot(steps, Dx, "bx", label=r"$\Delta_x$")
ax2.plot(steps, Dy, "r+", label=r"$\Delta_y$")
ax2.set_ylabel("absolute Fehler")
ax2.set_xlabel("$n$")
ax2.legend()
ax2.grid()

plt.suptitle("Newton")
plt.tight_layout()
plt.show()

```



2. Mehrere Nullstellen Sinus- und Cosinus-Funktionen haben unendlich viele Nullstellen

$$g(x) = 2 \cos(x) + 0.5 \quad (42)$$

In diesem Fall können wir sie auch nicht einfach analytisch berechnen:

$$x = \arccos(-0.25) \quad (43)$$

```

def g(x):
    return 2 * np.cos(x) + 0.5

def gprime(x):
    return -2 * np.sin(x)

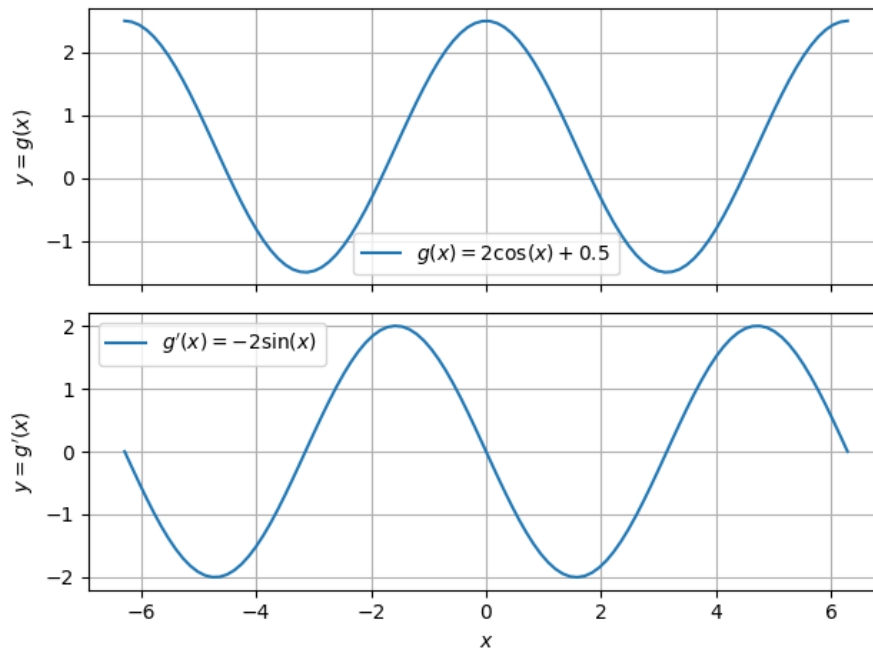
x = np.linspace(-2 * np.pi, 2 * np.pi, 100)
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
ax1.plot(x, g(x), label=r"$g(x) = 2\cos(x) + 0.5$")
ax1.set_ylabel("$y = g(x)$")
ax1.grid()

```

```

ax1.legend()
ax2.plot(x, gprime(x), label=r"$g'(x) = -2\sin(x)$")
ax2.set_ylabel("$y = g'(x)$")
ax2.set_xlabel("$x$")
ax2.grid()
ax2.legend()
plt.tight_layout()
plt.show()

```



Welche Nullstelle von einem Algorithmus gefunden wird, hängt von den Anfangsbedingungen ab.

- Beim Newton-Verfahren ist dies der Startwert x_0 und
- beim Bisektionsverfahren ist es das Start-Intervall $[a_0, b_0]$.

Probieren wir es mal mit Newton aus:

```

xtol = ytol = 1e -15
maxiter = 100
for x0 in [ -4, -2, 2, 4]:
    res = newton(g, gprime, x0, xtol, ytol, maxiter)
    steps, x, Dx, Dy = res.T
    print(f"Newton mit x0 = {x0} findet:", x[ -1])

```

```

Newton mit x0 = -4 findet: -4.459708725242611
Newton mit x0 = -2 findet: -1.8234765819369751
Newton mit x0 = 2 findet: 1.8234765819369751
Newton mit x0 = 4 findet: 4.459708725242611

```

Es ist unmöglich zu wissen, ob man alle Nullstellen gefunden hat. Denn es können sich immer noch mehr Nullstellen zwischen ausprobierten Startwerten befinden.

3. Nullstellen bestimmen mit SciPy Das SciPy Unterpaket `scipy.optimize` bietet verschiedene Funktionen für die Suche von Nullstellen (engl. *root finding*).

Die Funktion `scipy.optimize.root_scalar` ist der übergeordnete Einstiegspunkt. Sie wählt automatisch die beste Methode für das gegebene Problem. Man kann aber auch explizit eine Methode wählen.

Weiter gibt es neben anderen Algorithmen auch `scipy.optimize.bisect` und `scipy.optimize.newton`, um direkt diese Verfahren zu benutzen.

Siehe Beispiel auf dem Übungsblatt.

```
from scipy import optimize

sol = optimize.root_scalar(g, x0=4, fprime=gprime)
sol

converged: True
      flag: converged
function_calls: 10
      iterations: 5
          root: 4.459708725242611
      method: newton

print(type(sol))
sol.root

<class 'scipy.optimize._zeros_py.RootResults'>

np.float64(4.459708725242611)

optimize.newton(g, 1, gprime, full_output=True)

(np.float64(1.8234765819369754),
  converged: True
      flag: converged
function_calls: 10
      iterations: 5
          root: 1.8234765819369754
      method: newton)

optimize.bisect(g, 0, 4, full_output=True)

(1.823476581937939,
  converged: True
      flag: converged
function_calls: 43
      iterations: 41
          root: 1.823476581937939
      method: bisect)
```

Mehrere Nullstellen

```
np.array([optimize.root_scalar(g, x0=x0).root for x0 in [1, -1, 5]])

array([ 1.82347658, -1.82347658,  4.45970873])

optimize.newton(g, [1, -1, 5], gprime)

array([ 1.82347658, -1.82347658,  4.45970873])
```

0.6 Ableitung

0.6.1 Numerische Ableitung

Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Das Ziel ist, eine Näherung der Ableitung zu berechnen, ohne die Funktionsgleichung und somit ihre analytische Ableitung zu kennen. Das Verfahren hierzu wirst du selbst rudimentär implementieren.

In der Vertiefung werden wir sehen, dass man leicht anders vorgehen muss, wenn man nur Messwerte und keine Funktion zur Verfügung hat, die man überall auswerten kann.

Numerische Ableitung ausgehend von einer Funktion

Es sei eine Funktion $f : D \rightarrow \mathbb{R}$ mit $D \subseteq \mathbb{R}$ gegeben, ihre Ableitung kennen wir aber nicht. Darunter können wir uns eine Funktion vorstellen, die z.B. das Resultat einer numerischen Simulation ist, oder die aus Messwerten via Interpolation gewonnen wird.

Die Ableitung an einer Stelle $x \in D$ ist definiert als der Grenzwert des Differenzenquotienten

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (44)$$

Als numerische Schätzung für $f'(x)$ können wir den Differenzenquotienten für ein kleines h verwenden ($0 < h \ll 1$), also

$$f'(x) \simeq \frac{f(x+h) - f(x)}{h} \quad (45)$$

Wie gross h zu wählen ist, muss man ausprobieren, und hängt vom Verhalten der Funktion ab.

Zudem könnten wir statt einen Schritt um h nach vorne zu machen auch einen Schritt zurück machen, so dass ein “rückwärtsgerichteter” Differenzenquotient berechnet wird. Eine gute Idee ist, den Mittelwert zwischen dem vorwärtsgerichteten und dem rückwärtsgerichteten Differenzenquotienten zu nehmen. Dieser wird als **zentraler Differenzenquotient** bezeichnet:

$$f'(x) \simeq \frac{1}{2} \left(\frac{f(x+h) - f(x)}{h} + \frac{f(x) - f(x-h)}{h} \right) = \frac{f(x+h) - f(x-h)}{2h} \quad (46)$$

Dieses kurze Video erklärt das Wesentliche:

- “Numerisches Differenzieren” - Christian Karpfinger

Die folgenden Videos können fürs weitere Verständnis angeschaut werden:

- “20B.2 zentrale Differenzformeln; Ableitung numerisch” - Jörn Loviscach
- “Differenzialrechnung - Numerische Ableitung und Approximation” - Georg Hoever

Arbeitsauftrag

```
import numpy as np
import matplotlib.pyplot as plt
```

Aufgabe 1: Beispielfunktionen darstellen Wir betrachten die folgende Funktion

$$f(x) = \cos\left(2\pi\left(x+x^2\right)\right) \quad (47)$$

- Bestimme die Ableitung $f'(x)$ zuerst analytisch, damit du später eine Vergleichsmöglichkeit hast.
 - Benutze dazu die Kettenregel und die Ableitungsregel für den Cosinus: $\frac{d}{dx} \cos(x) = -\sin(x)$.
 - Überprüfe dein Resultat mit Wolfram|Alpha.
- Definiere $f(x)$ als Python-Funktion `f(x)` und die Ableitung $f'(x)$ als `fprime(x)`.
- Stelle die beiden Funktionen in zwei übereinanderliegenden Koordinatensystemen mit 200 Punkten im Bereich $[-1, 1]$ dar.

Lösung:

Die Ableitung von $f(x)$ ist

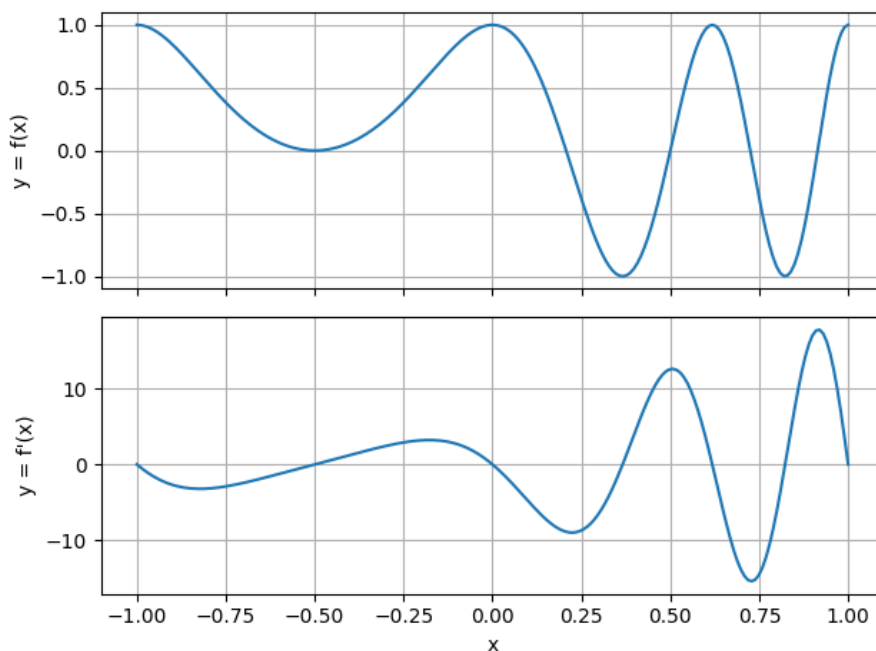
$$f'(x) = -\sin\left(2\pi\left(x + x^2\right)\right) \cdot 2\pi(1 + 2x) \quad (48)$$

```
def f(x):
    return np.cos(2 * np.pi * (x + x**2))

def fprime(x):
    return -np.sin(2 * np.pi * (x + x**2)) * 2 * np.pi * (1 + 2 * x)

x = np.linspace(-1, 1, 200)

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
ax1.plot(x, f(x))
ax1.grid()
ax1.set_ylabel("y = f(x)")
ax2.plot(x, fprime(x))
ax2.grid()
ax2.set_xlabel("x")
ax2.set_ylabel("y = f'(x)")
plt.tight_layout()
plt.show()
```



Aufgabe 2: Numerische Ableitung an einer Stelle a) Schreibe eine Funktion `deriv(f, x, h)`, welche die Ableitung an einer Stelle x anhand des zentralen Differenzenquotienten schätzt:

$$f'(x) \simeq \frac{f(x+h) - f(x-h)}{2h} \quad (49)$$

- Inputs:
 - **f**: die Funktion, die abgeleitet wird
 - **x**: die Stelle, an der abgeleitet wird (kann auch ein Array sein)

- `h`: die Schrittweite der Schätzung
- Output:
 - Die Schätzung der Ableitung an der Stelle `x` bzw. an den Stellen des Arrays `x`

```
def deriv(f, x, h):
    return (f(x + h) - f(x - h)) / (2 * h)
```

b) Teste `deriv` an der Stelle $x = 0.25$ mit $h = 0.01$. Du solltest -8.69220992831535 erhalten.

```
deriv(f, 0.25, 0.01)
```

```
np.float64( -8.69220992831535)
```

c) Berechne den absoluten Fehler Δ_y : Die Differenz zwischen der numerischen Schätzung der Ableitung und der analytischen Ableitung aus Aufgabe 1. Benutze wieder $x = 0.25$ und $h = 0.01$. Du solltest 0.0151495281029419 erhalten.

```
deriv(f, 0.25, 0.01) - fprime(0.25)
```

```
np.float64(0.0151495281029419)
```

Aufgabe 3: Numerische Ableitung darstellen

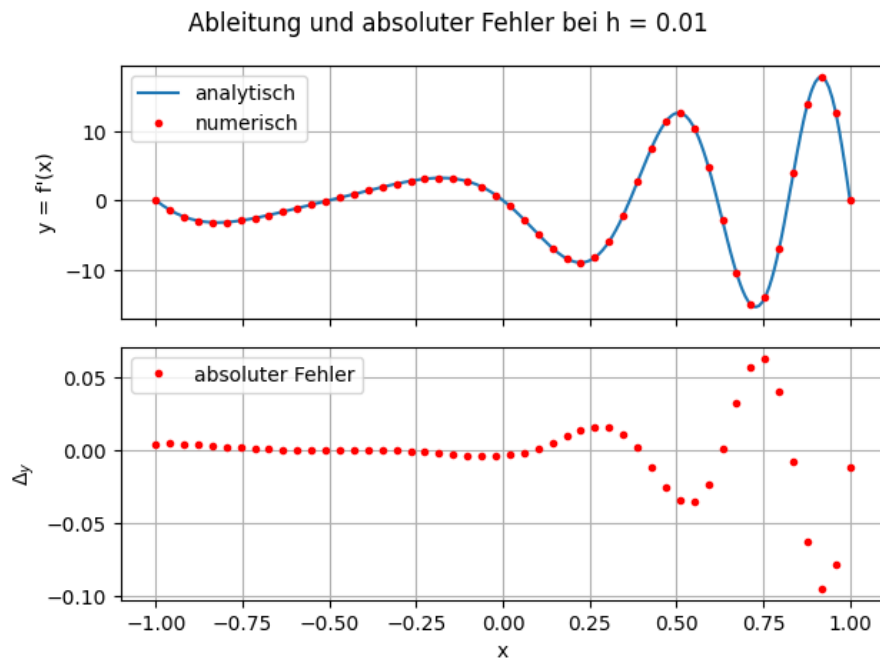
- Stelle die analytische Ableitung `fprime` und deine numerische Ableitung `deriv` zusammen in einem Diagramm.
- Wähle dazu 50 Punkte im Bereich $[-1, 1]$.
- Die Schätzungen mittels `deriv` sollen als rote Punkte dargestellt werden.
- Stelle in einem zweiten Diagramm (darunter und mit gleicher x-Achse) den absoluten Fehler Δ_y dar: Dies ist die Differenz zwischen der numerischen Schätzung der Ableitung und der analytischen Ableitung, so dass dieser positiv ist, wenn die Ableitung überschätzt wird, und negativ, wenn sie unterschätzt wird.

Benutze zuerst $h = 0.1$ und verkleinere h dann sukzessive jeweils um einen Faktor $\frac{1}{10}$. Beobachte wie sich das Resultat ändert.

```
h = 1e -2
x = np.linspace( -1, 1, 200)
yprime = fprime(x)

x2 = np.linspace( -1, 1, 50)
yderiv = deriv(f, x2, h)
Dy = yderiv - fprime(x2)

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
ax1.plot(x, fprime(x), label="analytisch")
ax1.plot(x2, yderiv, "r.", label="numerisch")
ax1.set_ylabel("y = f'(x)")
ax1.grid()
ax1.legend()
ax2.plot(x2, Dy, "r.", label="absoluter Fehler")
ax2.set_xlabel("x")
ax2.set_ylabel(r"$\Delta_y$")
ax2.grid()
ax2.legend()
plt.suptitle(f"Ableitung und absoluter Fehler bei h = {h}")
plt.tight_layout()
plt.show()
```



Aufgabe 4: Interpretation der Fehler Welche Aussagen kannst über den Fehler und die Abhängigkeit von h und der Form von $f'(x)$ machen?

Antwort

Beobachtungen:

- Je grösser die Schwankung in der Ableitung, desto grösser der Fehler der Näherung der Ableitung.
 - Das bedeutet: Je grösser die zweite Ableitung $f''(x)$, desto grösser die Fehler in der Berechnung von $f'(x)$.
- Je kleiner h , desto kleiner die Fehler in der numerischen Ableitung.
 - Dabei ändert sich aber nur die Amplitude des Fehlers und nicht die Abhängigkeit von x . Das heisst, die Form der “Fehlerkurve” bleibt gleich auch bei sehr kleinen h .
 - Nimmt man sehr kleine h , also z.B. $h = 10^{-10}$, so ändert sich die Abhängigkeit des Fehlers von x und die Fehler werden wieder grösser.

0.6.2 Numerische Ableitung

Übungen - Lösungen

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import make_interp_spline
from scipy.optimize import root_scalar

def deriv(f, x, h):
    """
    Ableitung einer Funktion f an der/n Stelle/n x mit Schrittweite h,
    berechnet mit dem zentralen Differenzenquotienten.
    """
    return (f(x + h) - f(x - h)) / (2 * h)

def deriv_data(x_data, y_data):
    """
    Ableitung einer Funktion, die durch Datenpunkte gegeben ist.
    """
    dydx = np.diff(y_data) / np.diff(x_data)
    yprime = np.empty_like(y_data)
    yprime[0] = dydx[0]
    yprime[-1] = dydx[-1]
    yprime[1:-1] = (dydx[: -1] + dydx[1:]) / 2
    return yprime
```

Aufgabe 1: Jogging In der Datei `jogging.csv` sind die Daten einer Joggingrunde gespeichert. Die Spalten enthalten folgende Daten:

1. Zeit der Messung in Sekunden
2. Zurückgelegte Distanz in Metern
3. Geschwindigkeit (*engl. pace*) in Minuten pro Meile (1 Meile = 1609.344 Meter)
4. Herzfrequenz in Anzahl Schläge pro Minute

Ziel ist es, die Geschwindigkeit aus der zurückgelegten Distanz und der Zeit zu berechnen und diese mit den Geschwindigkeitsdaten zu vergleichen.

Vorgehen:

1. Lade die ersten drei Spalten aus der Datei `jogging.csv` in die 1D-Arrays `time`, `distance`, `pace`. Die letzte Spalte kann ignoriert werden.
2. Berechne die Geschwindigkeit als Ableitung der zurückgelegten Distanz nach der Zeit. Nutze dazu die passende Funktion aus der Vertiefung, siehe oben.
3. Rechne die berechneten und die gemessenen Geschwindigkeiten in die gleichen Einheiten um: entweder beide in *min/km* oder beide in *m/s*.
4. Stelle die beiden Geschwindigkeiten im gleichen Diagramm dar. Wähle geeignete Darstellungsbereiche und schreibe die Achsen an (inkl. Einheiten).

Wie gut stimmen die beiden Geschwindigkeiten überein?

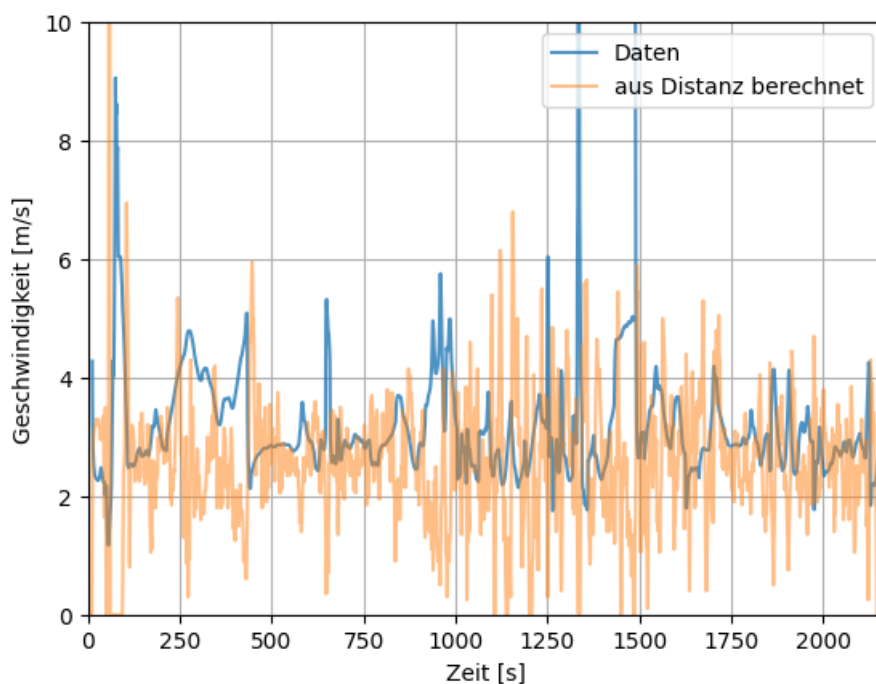
```
jogging = np.loadtxt("jogging.csv", delimiter=";", skiprows=1, usecols=(0, 1, 2))
time, distance, pace = jogging.T
```

```
speed_from_distance = deriv_data(time, distance)
speed_from_pace = 1 / pace * 1609.344 / 60
```

```
plt.plot(time, speed_from_pace, label="Daten", alpha=0.8)
plt.plot(time, speed_from_distance, label="aus Distanz berechnet", alpha=0.5)
plt.ylabel("Geschwindigkeit [m/s]")
plt.xlabel("Zeit [s]")
plt.xlim(time.min(), time.max())
plt.ylim(0, 10)
plt.legend()
plt.grid()
plt.show()
```

```
print("Mittlere Geschwindigkeiten:")
speed_from_pace[speed_from_pace < np.inf].mean(), speed_from_distance.mean()
```

```
/var/folders/p7/c2v89_m12k7dbrfs0m28h1640000gr/T/ipykernel_67865/2418746585.py:5: RuntimeWarni
speed_from_pace = 1 / pace * 1609.344 / 60
```



Mittlere Geschwindigkeiten:

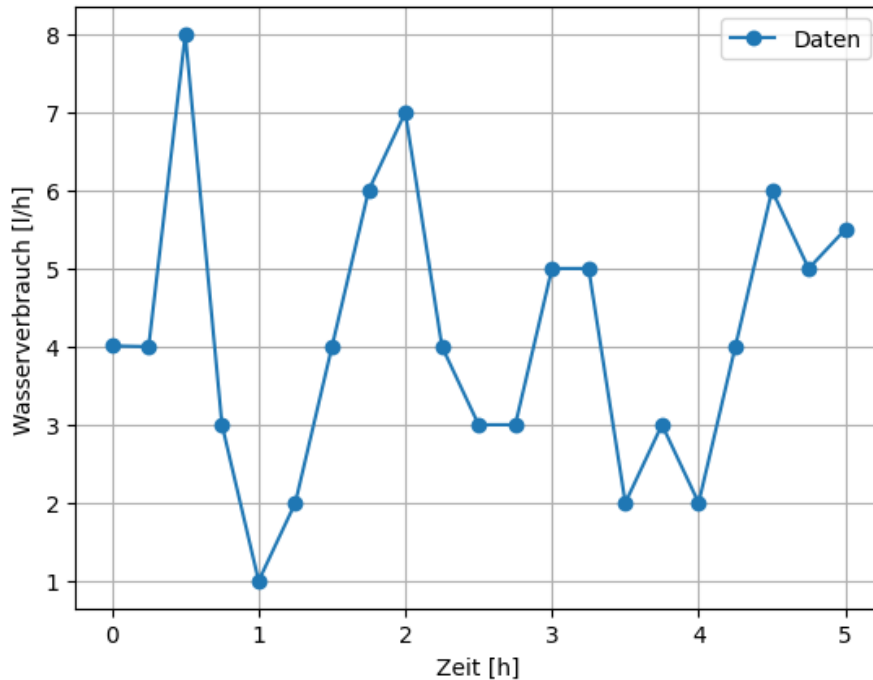
```
(np.float64(3.180133365144171), np.float64(2.560018579935653))
```

Aufgabe 2: Wasserverbrauch Wir schauen Daten zum Wasserverbrauch an. Das Ziel ist hier die Ableitung (Änderungsrate des Wasserverbrauchs) einerseits direkt aus den Daten zu berechnen und andererseits, die Daten zuerst zu interpolieren und dann die Ableitung der Interpolationsfunktion zu berechnen.

a) Daten importieren und darstellen Importiere die Daten in der Datei `wasserverbrauch.csv`. Stelle den Wasserverbrauch als Funktion der Zeit dar.

```
x_data, y_data = np.loadtxt("wasserverbrauch.csv", delimiter=";", skiprows=1).T

plt.plot(x_data, y_data, "o -", label="Daten")
plt.xlabel("Zeit [h]")
plt.ylabel("Wasserverbrauch [l/h]")
plt.legend()
plt.grid()
plt.show()
```



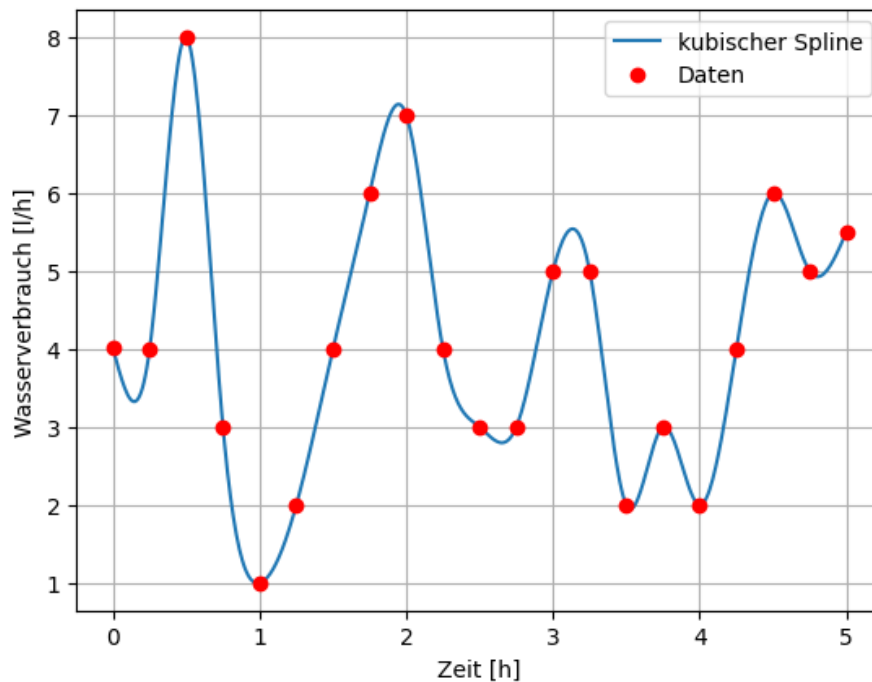
b) Interpolieren Nun interpolieren wir die Daten an 500 Stellen im Bereich $x \in [0, 5]$.

Für die kubische Spline-Interpolation verwenden wir `scipy.interpolate.make_interp_spline` mit natürlichen Randbedingungen.

Stelle nun die interpolierten Werte (als verbundene Linie) und die Messwerte (als rote Punkte) zusammen dar.

```
spline = make_interp_spline(x_data, y_data, bc_type="natural")
x_spline = np.linspace(0, 5, 500)
y_spline = spline(x_spline)

plt.plot(x_spline, y_spline, label="kubischer Spline")
plt.plot(x_data, y_data, "ro", label="Daten")
plt.xlabel("Zeit [h]")
plt.ylabel("Wasserverbrauch [l/h]")
plt.legend()
plt.grid()
plt.show()
```



c) Ableiten Hier soll die Änderungsrate (Ableitung) des Wasserverbrauchs berechnet werden. Die beiden Funktionen `deriv` und `deriv_data` sind aus der Einarbeitung und Vertiefung bekannt. Sie sind zuoberst im Übungsblatt bereits definiert. Schaue sie nochmals an und mache dir klar, wie sie funktionieren und angewendet werden.

1. Berechne die numerische Ableitung der Interpolationsfunktion mit einer sinnvollen Schrittweite (siehe Diskussion in Vertiefung).
2. Berechne die Ableitung ausgehend von den Originaldaten.
3. Stelle den Wasserverbrauch (Spline und Daten wie in Teil b) und darunter dessen Ableitung (aus Spline und aus Daten) grafisch dar.

Benutze auch mal einen linearen oder quadratischen Spline (ohne Angabe des `bc_type`) und beobachte, wie sich die Ableitung ändert.

```
cubic_spline = make_interp_spline(x_data, y_data, bc_type="natural")
linear_spline = make_interp_spline(x_data, y_data, k=1)
x_spline = np.linspace(0, 5, 500)
y_cubic_spline = cubic_spline(x_spline)
y_linear_spline = linear_spline(x_spline)
```

```
yprime_cubic_spline = deriv(cubic_spline, x_spline, 5e -6)
yprime_linear_spline = deriv(linear_spline, x_spline, 5e -6)
yprime_data = deriv_data(x_data, y_data)
```

```
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
```

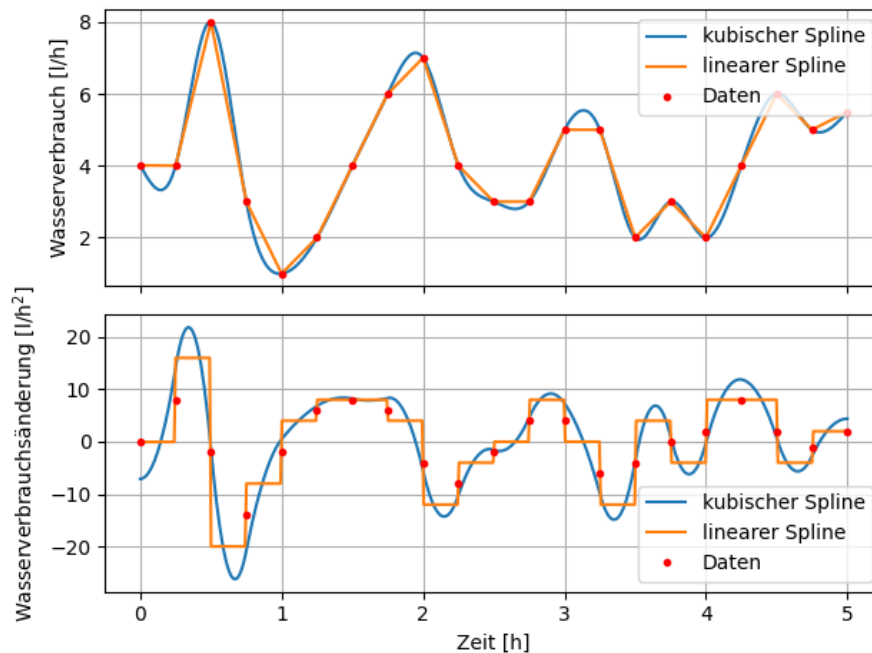
```
ax1.plot(x_spline, y_cubic_spline, label="kubischer Spline")
ax1.plot(x_spline, y_linear_spline, label="linearer Spline")
ax1.plot(x_data, y_data, "r.", label="Daten")
ax1.set_ylabel("Wasserverbrauch [l/h]")
ax1.legend()
ax1.grid()
```

```

ax2.plot(x_spline, yprime_cubic_spline, label="kubischer Spline")
ax2.plot(x_spline, yprime_linear_spline, label="linearer Spline")
ax2.plot(x_data, yprime_data, "r.", label="Daten")
ax2.set_xlabel("Zeit [h]")
ax2.set_ylabel("Wasserverbrauchsänderung [l/h$^2$]")
ax2.grid()
ax2.legend()

plt.tight_layout()
plt.show()

```



d) Analytische Ableitung der Spline-Interpolation Eine Spline-Interpolation ist ja eine stückweise definierte Funktion. Die Teilstücke sind Polynome, deren Ableitung analytisch berechnet werden können und auch wieder Polynome sind. Also ist die Ableitung eines kubischen Splines ein quadratischer Spline, die Ableitung eines quadratischen Splines ein linearer Spline und die Ableitung eines linearen Splines eine stückweise konstante Funktion (Spline vom Grad 0).

Das `BSpline` Objekt, das von der Funktion `make_interp_spline` erzeugt wird, bietet die Methode `BSpline.derivative`. Diese liefert wieder ein `BSpline` Objekt, das nun die Ableitung des ursprünglichen Splines repräsentiert.

Bestimme für den kubischen Spline aus Teil b) die analytische Ableitung und vergleiche diese mit der numerischen Ableitung aus Teil c). Stelle darunter den absoluten Fehler der numerischen Ableitung ggü. der Ableitung des Splines dar.

```

yprime_analytic = cubic_spline.derivative()
yprime_from_analytic = yprime_analytic(x_spline)
D_yprime_numeric = yprime_cubic_spline - yprime_from_analytic

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

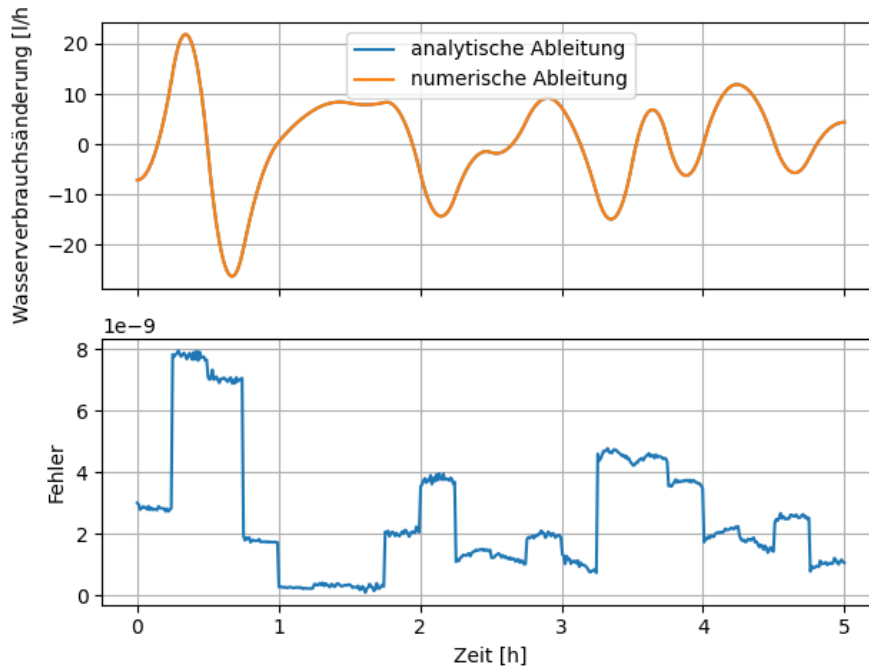
ax1.plot(x_spline, yprime_from_analytic, label="analytische Ableitung")
ax1.plot(x_spline, yprime_cubic_spline, label="numerische Ableitung")
ax1.set_ylabel("Wasserverbrauchsänderung [l/h$^2$]")
ax1.legend()

```

```
ax1.grid()

ax2.plot(x_spline, np.abs(D_yprime_numeric))
ax2.set_xlabel("Zeit [h]")
ax2.set_ylabel("Fehler")
ax2.grid()

plt.tight_layout()
plt.show()
```



Aufgabe 3: Bolts Geschwindigkeit Hierfür schauen wir uns die Daten von Usain Bolts Weltrekord von 2009 an. Diese findet ihr in der Datei `bolt100m.csv`.

Das Ziel hier ist aus Messungen des Weg-Zeit-Diagramms die Geschwindigkeit zu berechnen. Daraus bestimmen wir den Zeitpunkt der maximalen Geschwindigkeit, indem wir die Nullstelle der Beschleunigung bestimmen.

a) Daten importieren Lade die Daten. Gemessen wurde die Zeit t in s (zweite Spalte) zu spezifischen Distanzen d in m (erste Spalte).

```
d_data, t_data = np.loadtxt("bolt100m.csv", delimiter=",", skiprows=1).T
```

b) Geschwindigkeit (Ableitung) berechnen

1. Berechne die Geschwindigkeit v in m/s, indem Du die numerische Ableitung $v = d'(t)$ mit `np.gradient` berechnest.
 - *Vorsicht:* Die Funktion `np.gradient` erwartet als Argumente zuerst die Funktionswerte und dann die Werte der unabhängigen Variable, siehe Beispiel in der Vertiefung und lese die Dokumentation.
2. Stelle d und v als Funktionen von t sinnvoll dar.

```
v_data = np.gradient(d_data, t_data)
```

```
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
```



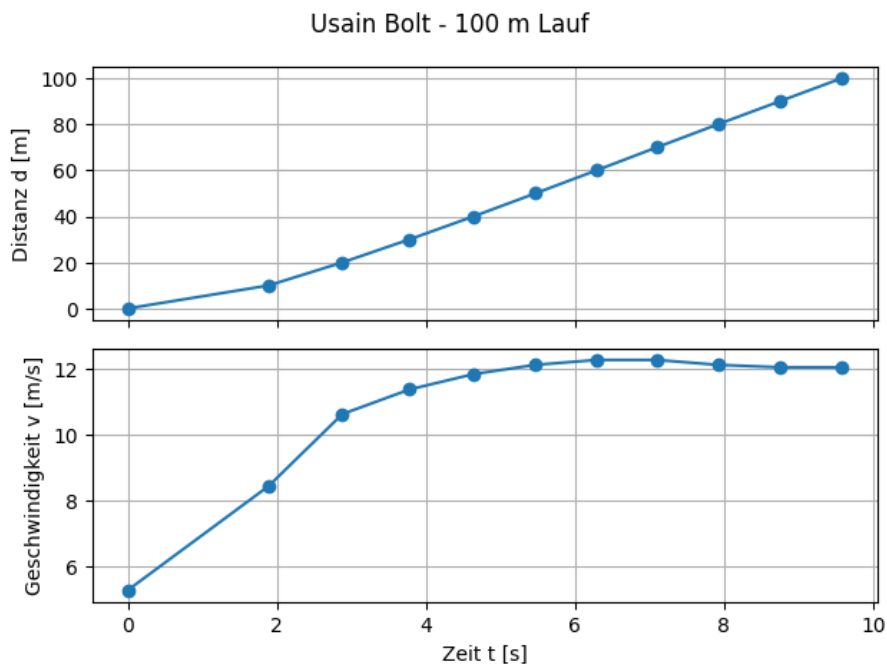
```

ax1.plot(t_data, d_data, "o -")
ax1.set_ylabel("Distanz d [m]")
ax1.grid()

ax2.plot(t_data, v_data, "o -")
ax2.set_xlabel("Zeit t [s]")
ax2.set_ylabel("Geschwindigkeit v [m/s]")
ax2.grid()

plt.suptitle("Usain Bolt - 100 m Lauf")
plt.tight_layout()
plt.show()

```



c) Geschwindigkeit interpolieren Als nächstes ist das Ziel, herauszufinden, wann Bolt seine maximale Geschwindigkeit erreicht hat. Dazu brauchen wir die Ableitung der Geschwindigkeit, also die Beschleunigung a . Davon bestimmen wir die Nullstelle. (Erinnere Dich an die Kurvendiskussion in der *Analysis*.)

Nullstellen können wir mit SciPy nur von Funktionen und nicht direkt von Daten berechnen. Daher konstruieren wir zuerst eine kubische Spline-Interpolationsfunktion der Geschwindigkeit und nehmen davon die Ableitung. (Alternativ könnten wir die Geschwindigkeit zuerst numerisch ableiten und dann die Beschleunigung interpolieren.)

1. Konstruiere mit `scipy.interpolate.make_interp_spline` eine Interpolationsfunktion der Geschwindigkeit.
2. Leite diese Funktion mit der Methode `BSpline.derivative` ab (siehe Aufgabe 2 d) und definiere damit die Beschleunigungsfunktion.
3. Finde die Nullstelle der Beschleunigung mit einer passenden Methode aus `scipy.optimize`.
4. Stelle die Geschwindigkeit (Daten und Spline) und die Beschleunigung (Spline) in übereinanderliegenden Diagrammen dar. Zeichne die Nullstelle der Beschleunigung und das somit gefundene Maximum der Geschwindigkeit ein.

```
v_spline = make_interp_spline(t_data, v_data, bc_type="natural")
a_spline = v_spline.derivative()
a_root = root_scalar(a_spline, bracket=[5, 8])
a_root

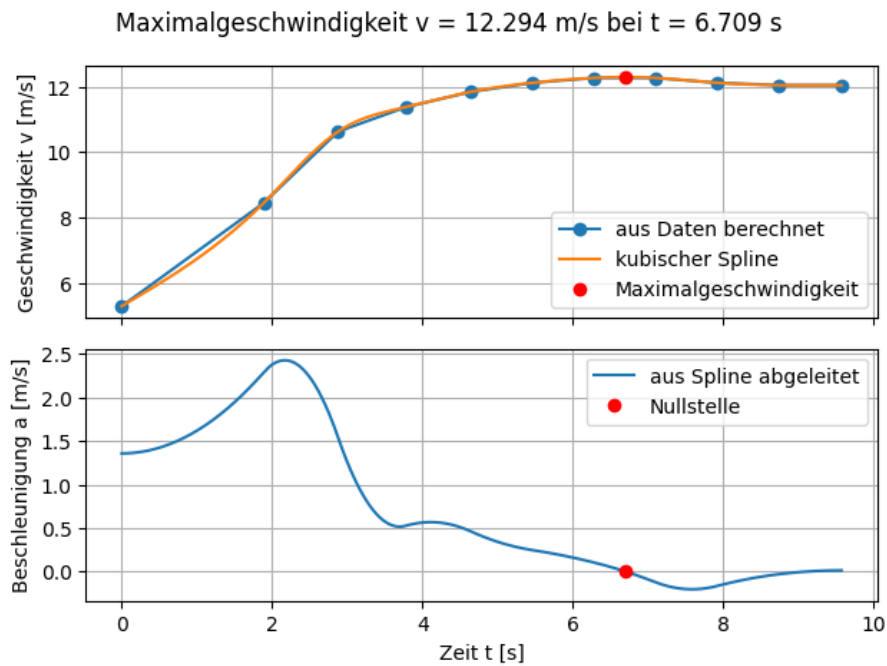
converged: True
    flag: converged
function_calls: 10
    iterations: 9
        root: 6.709476943128238
    method: brentq

t_max = a_root.root
v_max = v_spline(t_max)

t = np.linspace(t_data.min(), t_data.max(), 500)
v = v_spline(t)
a = a_spline(t)

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

ax1.plot(t_data, v_data, "o -", label="aus Daten berechnet")
ax1.plot(t, v, " -", label="kubischer Spline")
ax1.plot(t_max, v_max, "ro", label="Maximalgeschwindigkeit")
ax1.set_ylabel("Geschwindigkeit v [m/s]")
ax1.legend()
ax1.grid()
ax2.plot(t, a, " -", label="aus Spline abgeleitet")
ax2.plot(t_max, 0, "ro", label="Nullstelle")
ax2.set_xlabel("Zeit t [s]")
ax2.set_ylabel("Beschleunigung a [m/s²]")
ax2.legend()
ax2.grid()
plt.suptitle(f"Maximalgeschwindigkeit v = {v_max:.3f} m/s bei t = {t_max:.3f} s")
plt.tight_layout()
plt.show()
```



d) Geschwindigkeit numerisch ableiten und Beschleunigung interpolieren (optional)

Leite die Geschwindigkeit zuerst numerisch ab und interpoliere die resultierende Beschleunigung. Bestimme davon die Nullstelle. Stelle wiederum Geschwindigkeit und Beschleunigung dar und zeichne die Nullstelle ein. Vergleiche das Ergebnis mit dem aus c).

```
a_data = np.gradient(v_data, t_data)
a_spline = make_interp_spline(t_data, a_data, bc_type="natural")
a_root = root_scalar(a_spline, bracket=[5, 8])
a_root

converged: True
      flag: converged
function_calls: 10
      iterations: 9
         root: 6.669833311440452
      method: brentq

t_max = a_root.root

v_spline = make_interp_spline(t_data, v_data, bc_type="natural")
v_max = v_spline(t_max)

t = np.linspace(t.min(), t.max(), 500)
v = v_spline(t)
a = a_spline(t)

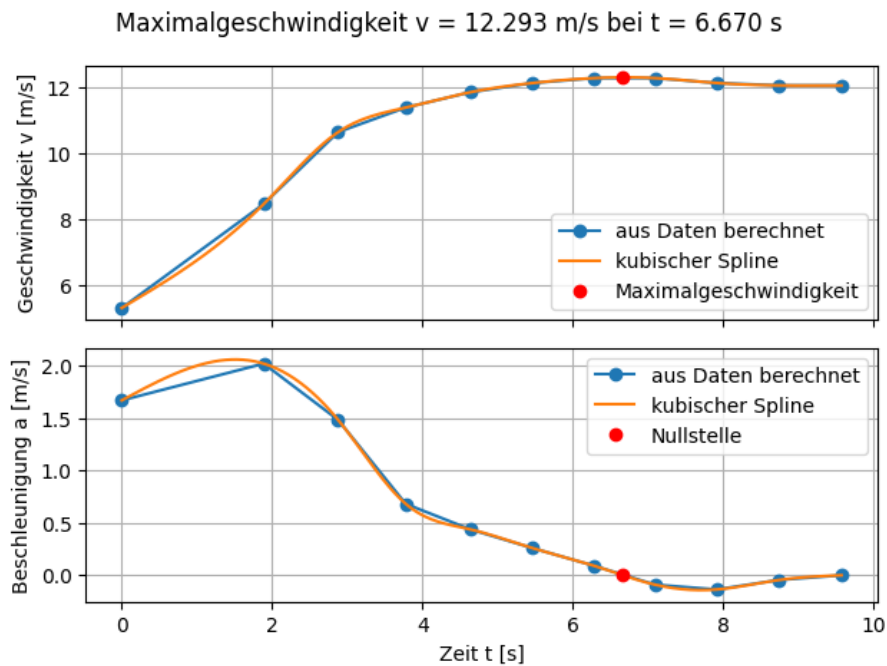
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

ax1.plot(t_data, v_data, "o -", label="aus Daten berechnet")
ax1.plot(t, v, " -", label="kubischer Spline")
ax1.plot(t_max, v_spline(t_max), "ro", label="Maximalgeschwindigkeit")
ax1.set_ylabel("Geschwindigkeit v [m/s]")
ax1.legend()
ax1.grid()
```

```

ax2.plot(t_data, a_data, "o -", label="aus Daten berechnet")
ax2.plot(t, a, " -", label="kubischer Spline")
ax2.plot(t_max, 0, "ro", label="Nullstelle")
ax2.set_xlabel("Zeit t [s]")
ax2.set_ylabel("Beschleunigung a [m/s²]")
ax2.legend()
ax2.grid()
plt.suptitle(f"Maximalgeschwindigkeit v = {v_max:.3f} m/s bei t = {t_max:.3f} s")
plt.tight_layout()
plt.show()

```



Die beiden Vorgehen liefern beinahe das gleiche Ergebnis. Sie unterscheiden sich ab der zweiten Stelle nach dem Komma in der Zeit, und in der dritte Stelle nach dem Komma in der Geschwindigkeit.

0.6.3 Numerische Ableitung

Vertiefung

Übersicht

1. Fehler: Abhängigkeit und Schätzung
2. Ableitung ausgehend von (ungleichmässig verteilten) Daten
3. Vergleich mit NumPy Gradient

```
import numpy as np
import matplotlib.pyplot as plt
```

1. Fehler: Abhängigkeit und Schätzung In der Einarbeitung haben wir gesehen, dass der Fehler der numerischen Ableitung von der Schrittweite h abhängt. Wir untersuchen dies hier noch etwas genauer.

Dazu betrachten wir die Funktion

$$f(x) = x^5 + 4x^4 + x^3 - 10x^2 - 4x + 8 \quad (50)$$

deren Ableitung

$$f'(x) = 5x^4 + 16x^3 + 3x^2 - 20x - 4 \quad (51)$$

ist. Es sind beides Polynome, so dass wir sie mithilfe von `np.polynomial.Polynomial` repräsentieren und berechnen können.

```
f = np.polynomial.Polynomial([8, -4, -10, 1, 4, 1])
f
```

```
Polynomial([ 8., -4., -10., 1., 4., 1.], domain=[-1., 1.], window=[-1., 1.], symbol='x')
```

Solche NumPy Polynome können mit der Methode `Polynomial.deriv()` analytisch abgeleitet werden

```
fprime = f.deriv()
fprime
```

```
Polynomial([-4., -20., 3., 16., 5.], domain=[-1., 1.], window=[-1., 1.], symbol='x')
```

Wir schätzen die Ableitung einmal mit dem Vorwärtsdifferenzenquotienten und einmal mit dem zentralen Differenzenquotienten.

```
def deriv_forward(f, x, h):
    return (f(x + h) - f(x)) / h
```

```
def deriv_central(f, x, h):
    return (f(x + h) - f(x - h)) / (2 * h)
```

und vergleichen die Ergebnisse mit der analytischen Ableitung.

```

h = 1e -3

x = np.linspace( -3, 1.5)
yprime = fprime(x)
yprime_forward = deriv_forward(f, x, h)
yprime_central = deriv_central(f, x, h)

D_forward = yprime_forward - yprime
D_central = yprime_central - yprime

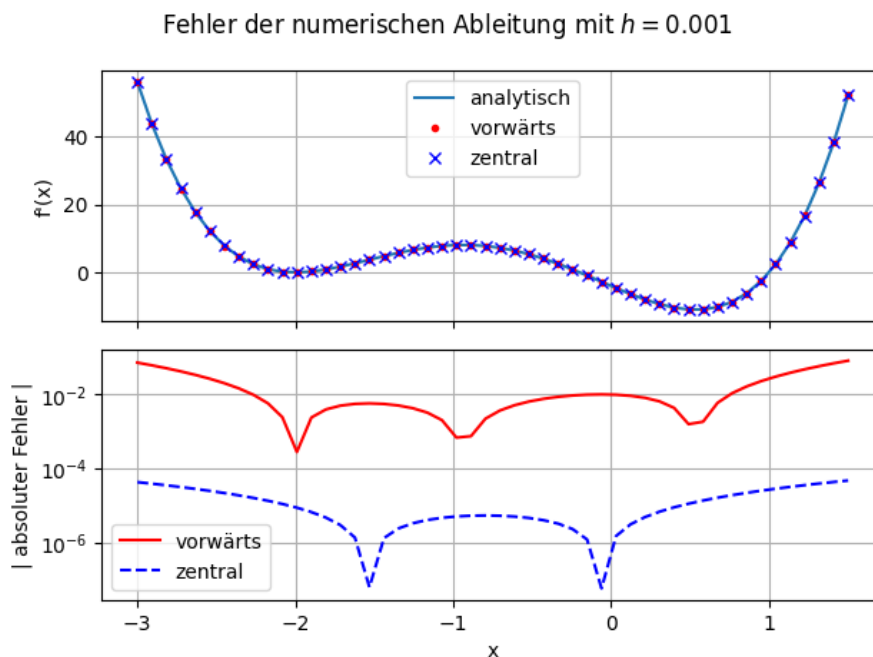
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

ax1.plot(x, yprime, label="analytisch")
ax1.plot(x, yprime_forward, "r.", label="vorwärts")
ax1.plot(x, yprime_central, "bx", label="zentral")
ax1.set_ylabel("f'(x)")
ax1.legend()
ax1.grid()

ax2.plot(x, np.abs(D_forward), "r", label="vorwärts")
ax2.plot(x, np.abs(D_central), "b - -", label="zentral")
ax2.set_xlabel("x")
ax2.set_ylabel("| absoluter Fehler |")
ax2.set_yscale("log")
ax2.grid()
plt.legend()

plt.suptitle(f"Fehler der numerischen Ableitung mit $h = {h}$")
plt.tight_layout()
plt.show()

```



Der zentrale Differenzenquotient schätzt die Ableitung also viel besser als der Vorwärtsdifferenzenquotient. Machen wir einen Vergleich an einer Stelle x_0 und variieren die Schrittweite h .

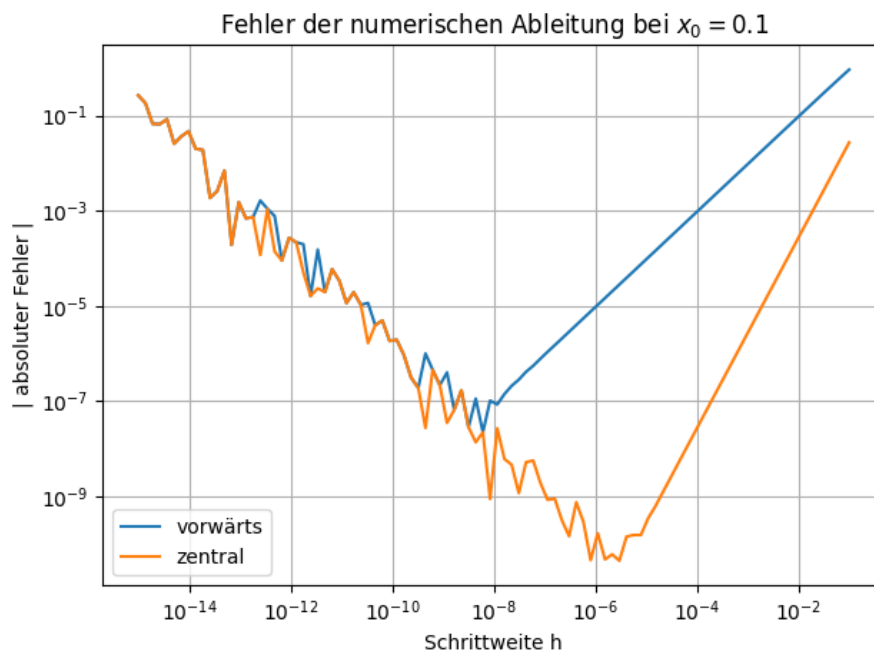
```
h = np.logspace( -15, -1, 100)
```

```
# h = np.linspace(1e -15, 0.1, 100)
x0 = 0.1

yprime = fprime(x0)
yprime_forward = deriv_forward(f, x0, h)
yprime_central = deriv_central(f, x0, h)

D_forward = yprime_forward - yprime
D_central = yprime_central - yprime

plt.plot(h, np.abs(D_forward), label="vorwärts")
plt.plot(h, np.abs(D_central), label="zentral")
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Schrittweite h")
plt.ylabel("| absoluter Fehler |")
plt.grid()
plt.legend()
plt.title(f"Fehler der numerischen Ableitung bei $x_0={x0}$")
plt.tight_layout()
plt.show()
```



Beobachtungen:

- Der Fehler des Vorwärtsdifferenzenquotienten ist linear in h (Gerade mit Steigung 1 im doppelt logarithmischen Diagramm).

$$\frac{f(x_0 + h) - f(x_0)}{h} - f'(x_0) = O(h) \quad (52)$$

- Der Fehler des zentralen Differenzenquotienten ist quadratisch in h (Gerade mit Steigung 2 im doppelt logarithmischen Diagramm) und ergibt eine wesentlich bessere Schätzung.

$$\frac{f(x_0 + h) - f(x_0 - h)}{2h} - f'(x_0) = O(h^2) \quad (53)$$

- Beide Differenzenquotienten stossen bei sehr kleinen h an numerische Grenzen. Grund ist die Auslöschung bei der Subtraktion von fast gleich grossen Zahlen: $f(x - h) - f(x)$.

Wie h idealerweise gewählt wird, hängt von der Form der Funktion ab. Für den zentralen Differenzenquotienten gibt es die Faustregel

$$h \approx \sqrt[3]{\varepsilon} \quad (54)$$

wobei ε die Maschinenpräzision ist. Für 64-bit Gleitkommazahlen ist ergibt dies $h \approx 5 \times 10^{-6}$.

Bemerkung: Es gibt auch eine bessere Methode, die mithilfe der Taylorentwicklung der Funktion, deren Ableitung mit einem Fehler der Ordnung $O(h^4)$ schätzt, siehe Numerische Differentiation - Wikipedia.

2. Ableitung ausgehend von Daten Wie gehen wir vor, wenn wir keine analytische Funktion gegeben haben, sondern nur Datenpunkte?

Gegeben sind also n Datenpunkte (x_i, y_i) , $i = 0, 1, \dots, n-1$.

a) Gleichmässig verteilte Punkte Sind alle x -Werte gleich weit voneinander entfernt, $x_{i+1} = x_i + h$ (äquidistant), so können wir analog zur Methode für die Ableitung von Funktionen den *zentralen Differenzenquotienten* nehmen:

$$y'_i = \frac{y_{i+1} - y_{i-1}}{2h}, \quad i = 1, 2, \dots, n-2 \quad (55)$$

Am Rand, also für $i = 0$ und $i = n-1$ klappt dies nicht mehr und man beschränkt sich eben auf den vorwärts- bzw. rückwärtsgerichteten Differenzenquotienten:

$$y'_0 = \frac{y_1 - y_0}{h}, \quad y'_{n-1} = \frac{y_{n-1} - y_{n-2}}{h} \quad (56)$$

b) Ungleichmässig verteilte Punkte (allgemeiner Fall) Sind die Punkte nicht gleichmässig verteilt, können wir trotzdem den Mittelwert des vorwärts- und rückwärtsgerichteten Differenzenquotienten nehmen.

$$y'_i = \frac{1}{2} \left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i} + \frac{y_i - y_{i-1}}{x_i - x_{i-1}} \right), \quad i = 1, 2, \dots, n-2 \quad (57)$$

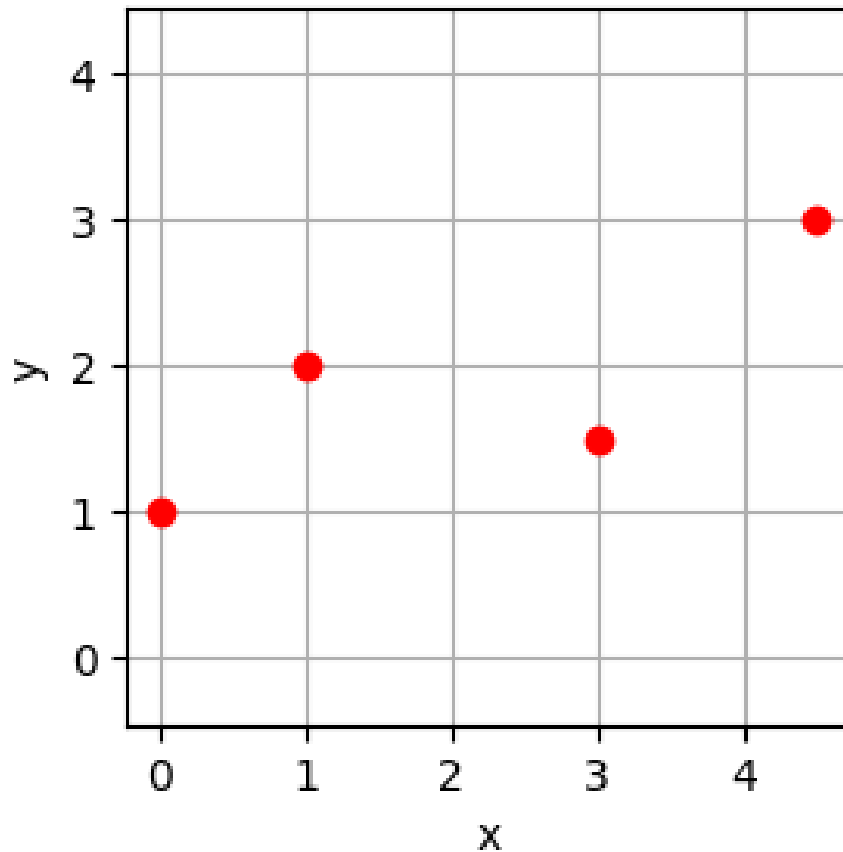
und an den Rändern entsprechend

$$y'_0 = \frac{y_1 - y_0}{x_1 - x_0}, \quad y'_{n-1} = \frac{y_{n-1} - y_{n-2}}{x_{n-1} - x_{n-2}} \quad (58)$$

Beispiel

```
x = np.array([0, 1, 3, 4.5])
y = np.array([1, 2, 1.5, 3])

plt.figure(figsize=(3, 3))
plt.plot(x, y, "ro")
plt.axis("equal") # equal scaling in x and y
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.show()
```

```

dx = np.diff(x)
dy = np.diff(y)
dydx = dy / dx
dydx

array([ 1.   , -0.25,  1.   ])

yprime = np.array(
    [
        dydx[0],
        (dydx[0] + dydx[1]) / 2,
        (dydx[1] + dydx[2]) / 2,
        dydx[-1],
    ]
)
yprime

array([1.   , 0.375, 0.375, 1.   ])

```

Implementation

- Wir berechnen die Differenzen benachbarter Punkte mit `np.diff` und daraus die Differenzenquotienten.
- Dann verschieben wir diese gegenüber einander und addieren sie, um den zentralen Differenzenquotienten zu erhalten.

```

def deriv_data(x, y):
    dydx = np.diff(y) / np.diff(x)
    yprime = np.empty_like(y)

```

```
yprime[0] = dydx[0]
yprime[-1] = dydx[-1]
yprime[1:-1] = (dydx[: -1] + dydx[1:]) / 2
return yprime
```

Für einen Test sampeln wir zufällige Punkte aus der Funktion $f(x)$ und schätzen die Ableitung.

```
# Messdaten erzeugen
rng = np.random.default_rng(123456)
x_data = np.sort(rng.uniform(-3, 1.5, 20))
y_data = f(x_data)

# numerische Ableitung und ihren Fehler berechnen
yprime_data = deriv_data(x_data, y_data)
D_yprime = yprime_data - fprime(x_data)

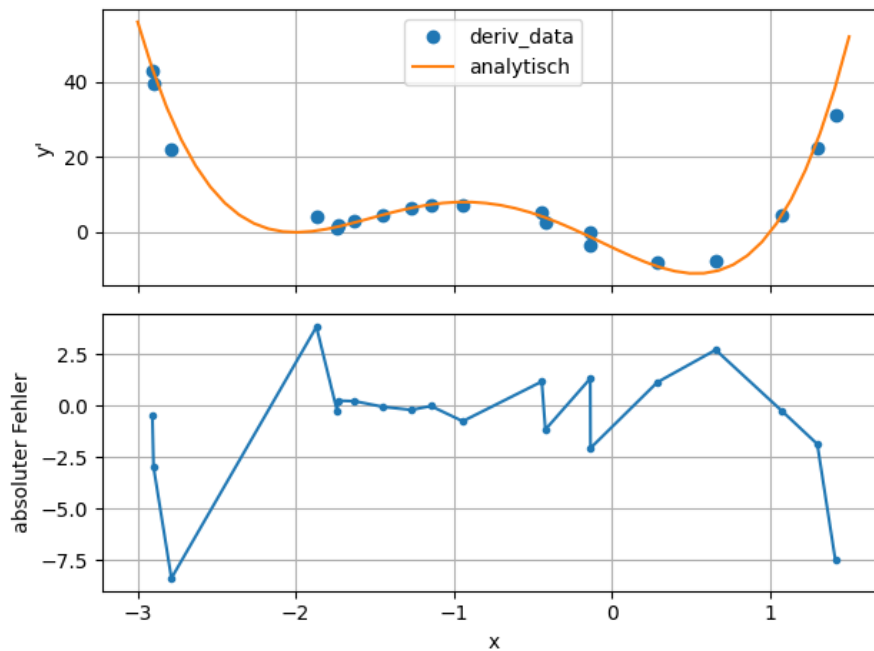
# analytische Ableitung
x = np.linspace(-3, 1.5)
yprime = fprime(x)

# Darstellung
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

# ... der Ableitungen
ax1.plot(x_data, yprime_data, "o", label="deriv_data")
ax1.plot(x, yprime, label="analytisch")
ax1.set_ylabel("y'")
ax1.legend()
ax1.grid()

# ... des Fehlers
ax2.plot(x_data, D_yprime, ". -")
ax2.set_xlabel("x")
ax2.set_ylabel("absoluter Fehler")
ax2.grid()

plt.tight_layout()
plt.show()
```



3. Vergleich mit NumPy Gradient NumPy bietet die Funktion `np.gradient` für die Berechnung von Ableitungen an.

Der Parameter `edge_order` bestimmt, wie die Ableitung an den Rändern berechnet wird. Mit `edge_order=1` wird einfach der vorwärts- bzw. rückwärtsgerichtete Differenzenquotient zum nächsten Punkt verwendet, mit `edge_order=2` wird auch noch der übernächste Punkt miteinbezogen. Ausprobieren!

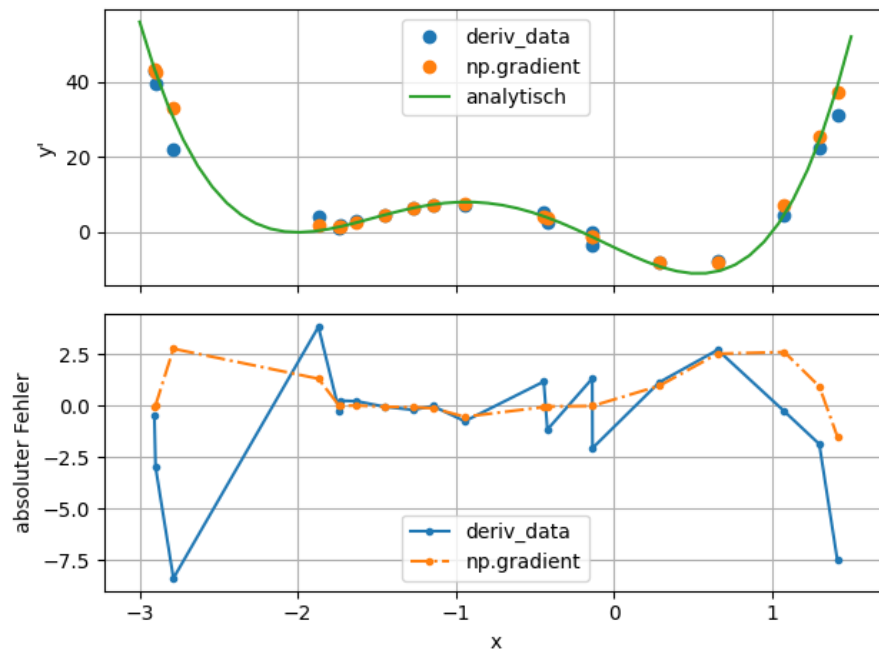
```
# Ableitung mit np.gradient und Fehler der Schätzung berechnen
grad = np.gradient(y_data, x_data, edge_order=2)
D_grad = grad - fprime(x_data)
```

```
# Darstellung
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
```

```
# ... der Ableitungen
ax1.plot(x_data, yprime_data, "o", label="deriv_data")
ax1.plot(x_data, grad, "o", label="np.gradient")
ax1.plot(x, yprime, label="analytisch")
ax1.set_ylabel("y'")
ax1.legend()
ax1.grid()
```

```
# ... des Fehlers
ax2.plot(x_data, D_yprime, ". -", label="deriv_data")
ax2.plot(x_data, D_grad, ". -", label="np.gradient")
ax2.set_xlabel("x")
ax2.set_ylabel("absoluter Fehler")
ax2.legend()
ax2.grid()
```

```
plt.tight_layout()
plt.show()
```



0.7 Ableitung Mehrere Variablen

0.7.1 Ableitung in mehreren Variablen

Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Hier lernst du die Grundlagen der Analysis in mehreren Variablen und konkret die **partielle Ableitung** kennen.

In der Vertiefung werden wir solche Ableitungen numerisch berechnen und weiterverwenden.

Partielle Ableitung Ableiten nach mehreren Variablen beschreibt den Prozess, bei dem man die Änderung einer Funktion in Bezug auf die Änderungen mehrerer unabhängiger Variablen untersucht. Dies wird häufig beim Modellieren von dynamischen Systemen und in der Optimierung verwendet. Um dies zu tun, verwendet man die sogenannten **partiellen Ableitungen**, welche die Änderung einer Funktion in Bezug auf eine einzelne unabhängige Variable beschreiben.

Konsultiere nochmals das Material zum Thema 02 *Darstellung von Funktionen mit mehreren Variablen* für die Einführung der Idee, dass Funktionen von mehreren unabhängigen Variablen abhängen können.

Schaue nun das folgende Video als Einführung in die partielle Ableitung an:

- Studflix.de - Partielle Ableitung

Weitere Erklärungen findest du falls nötig auf diesen Websites

- Mathebibel.de - Partielle Ableitung
- studysmarter.de - Partielle Ableitung

und in diesen Videos

- “Partielle Ableitung” - Georg Hoever
- “ABLEITUNG mehrere Variablen - partiell ableiten mehrdimensional”, MathemaTrick
- “Partielle Ableitung Basics” - Mathe by Daniel Jung
- “Partielle Ableitung, mehrdimensionale Funktionen ableiten, Variablen” - Kochrezepte für Mathematik

Arbeitsauftrag

1. Theorie Schreibe für dich eine Zusammenfassung der vorgehensweise bei einer partiellen Ableitung auf.

Lösung:

- Bei einer partiellen Ableitung leitet man nur eine Variable einer Funktion mit mehreren Variablen ab.
- Bei der partiellen Ableitung wird nach einer der unabhängigen Variablen abgeleitet (zum Beispiel x oder y). Die anderen Variablen werden in diesem Prozess wie Konstanten behandelt.

2. Ableitungsregeln Bei der partiellen Ableitung gelten die gleichen Regeln wie bei der Ableitung in einer Variablen.

Fasse die Ableitungsregeln für Funktionen von einer Variablen kurz für dich zusammen. (Siehe Unterlagen aus der Analysis.)

Lösung:

- Summen- und Faktorregel:

$$f(x) = a \cdot g(x) + b \cdot h(x) \quad \Rightarrow \quad f'(x) = a \cdot g'(x) + b \cdot h'(x) \quad (59)$$

- Produktregel:

$$f(x) = g(x) \cdot h(x) \quad \Rightarrow \quad f'(x) = g'(x) \cdot h(x) + g(x) \cdot h'(x) \quad (60)$$

- Quotientenregel:

$$f(x) = \frac{g(x)}{h(x)} \quad \Rightarrow \quad f'(x) = \frac{g'(x) \cdot h(x) - g(x) \cdot h'(x)}{(h(x))^2} \quad (61)$$

- Kettenregel:

$$f(x) = v(w(x)) \quad \Rightarrow \quad f'(x) = v'(w(x)) \cdot w'(x) \quad (62)$$

3. Elementare Ableitungen Die Ableitungen der folgenden Funktionen sind wichtig und sollten bekannt sein:

- Potenzfunktionen: $[x^p]' = p \cdot x^{p-1}$
- Exponentialfunktionen: $[b^x]' = \ln(b) \cdot b^x$
- natürliche Exponentialfunktion: $[e^x]' = e^x$
- Logarithmusfunktionen: $[\log_b(x)]' = \frac{1}{\ln(b) \cdot x}$
- natürlicher Logarithmus: $[\ln(x)]' = \frac{1}{x}$
- Sinus: $[\sin(x)]' = \cos(x)$
- Cosinus: $[\cos(x)]' = -\sin(x)$

4. Erste Beispiele Im Folgenden sind einige Beispiele für partielle Ableitungen gegeben. Bestimme welche der Ableitungsregeln benutzt wurden.

a) $f(x_1, x_2) = 3x_1^3 + 4x_1x_2^2 + 4x_2$

$$\frac{\partial f}{\partial x_1}(x_1, x_2) = 9x_1^2 + 4x_2^2$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) = 8x_1x_2 + 4$$

a) **Lösung:** Summenregel und Faktorregel

b) $g(x, y) = 5x^3 \cdot \sin(x) \cdot 4y^2 \cdot \cos(y)$

$$\frac{\partial g}{\partial x}(x, y) = (15x^2 \cdot \sin(x) + 5x^3 \cdot \cos(x)) \cdot 4y^2 \cdot \cos(y)$$

$$\frac{\partial g}{\partial y}(x, y) = 5x^3 \cdot \sin(x) \cdot (8y \cdot \cos(y) - 4y^2 \cdot \sin(y))$$

b) **Lösung:** Faktorregel und Produktregel

c) $h(x, y) = \frac{x^2 + 3y}{5xy}$

$$\frac{\partial h}{\partial x}(x, y) = \frac{2x \cdot 5xy - (x^2 + 3y) \cdot 5y}{(5xy)^2}$$

$$\frac{\partial h}{\partial y}(x, y) = \frac{3 \cdot 5xy - (x^2 + 3y) \cdot 5x}{(5xy)^2}$$

c) **Lösung:** Quotientenregel, Summenregel und Faktorregel

d) $N(t, x) = \exp(-t \cdot x^2)$

$$\frac{\partial N}{\partial t}(t, x) = \exp(-t \cdot x^2) \cdot (-x^2)$$

$$\frac{\partial N}{\partial x}(t, x) = \exp(-t \cdot x^2) \cdot (-2tx)$$

d) **Lösung:** Kettenregel und Faktorregel

5. Notation Partielle Ableitungen schreibt man normalerweise mit dem Zeichen ∂ statt d . Oder man verwendet eine Kurznotation, bei der man die Variable, nach der abgeleitet wird, tiefgestellt neben den Funktionsnamen schreibt (Subskript). Somit gibt es drei Schreibweisen:

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x} f = f_x \quad (63)$$

Bemerke: Keine dieser Schreibweisen beinhalten einen Strich, wie bei der Ableitung in einer Variablen, $f'(x)$.

Bei zweite und höheren Ableitungen wird ∂ bzw. ∂x entsprechend potenziert. Beispiel:

$$\frac{\partial}{\partial x} \left(\frac{\partial}{\partial x} f \right) = \frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x} f_x = f_{xx} \quad (64)$$

und

$$\frac{\partial}{\partial x} \left(\frac{\partial}{\partial y} f \right) = \frac{\partial^2 f}{\partial x \partial y} = \frac{\partial}{\partial y} f_x = f_{xy} \quad (65)$$

6. Berechnungen von Hand Berechne folgende partielle Ableitung von Hand. Du kannst die Resultate mit Wolfram|Alpha überprüfen. Beispiel:

Wolfram|Alpha: d/dy sin(xy)y

a) Berechne die partiellen Ableitungen nach x und y von $f(x, y) = 3x^2 \cdot y$.

a) **Lösung:**

$$\frac{\partial f}{\partial x} = 6x \cdot y$$

$$\frac{\partial f}{\partial y} = 3x^2$$

b) Berechne alle partiellen Ableitungen von $z(x, y) = 3^{2x^2+5y}$

b) **Lösung:**

$$\frac{\partial f}{\partial x} [3^{2x^2+5y}] = 4x \ln(3) \cdot 3^{2x^2+5y}$$

$$\frac{\partial f}{\partial y} [3^{2x^2+5y}] = 5 \ln(3) \cdot 3^{2x^2+5y}$$

c) Berechne $\frac{\partial}{\partial x} \left(\frac{\partial}{\partial y} \left[\frac{ay}{x} \right] \right)$ und umgekehrt $\frac{\partial}{\partial y} \left(\frac{\partial}{\partial x} \left[\frac{ay}{x} \right] \right)$. Vergleiche die beiden Resultate. Was fällt auf?

c) **Lösung:**

$$\begin{aligned}\frac{\partial}{\partial x} \left(\frac{\partial}{\partial y} \left[\frac{ay}{x} \right] \right) &= \frac{\partial}{\partial x} \left(\frac{a}{x} \right) \\ &= -\frac{a}{x^2}\end{aligned}$$

$$\begin{aligned}\frac{\partial}{\partial y} \left(\frac{\partial}{\partial x} \left[\frac{ay}{x} \right] \right) &= \frac{\partial}{\partial y} \left(-\frac{ay}{x^2} \right) \\ &= -\frac{a}{x^2}\end{aligned}$$

Die Reihenfolge der Ableitungen spielt bei höheren Ableitungen keine Rolle.

d) Gegeben ist die Funktion $g(r, s, t) = 4r^2s + (r + 7s)^2 \ln(t)$. Berechne $g_r(r, s, t)$, $g_s(r, s, t)$ und $g_t(r, s, t)$.

d) Lösung:

$$g_r(r, s, t) = 8rs + 2(r + 7s) \ln(t)$$

$$g_s(r, s, t) = 4r^2 + 14(r + 7s) \ln(t)$$

$$g_t(r, s, t) = (r + 7s)^2 \cdot t^{-1}$$

7. Interpretation In den folgenden Aufgaben geht es um das Verständnis des Ergebnisses einer partiellen Ableitung.

a) Körpergewicht

Angenommen dein Gewicht m in kg ist eine Funktion $m(c, t)$ der täglich eingenommenen Kilokalorien c und der Minuten t , die du täglich trainierst.

Interpretiere die folgenden Angaben, indem du Einheiten für m , c und t verwendest:

$$\frac{\partial m}{\partial c}(2500, 30) = 0.02, \quad \frac{\partial m}{\partial t}(2500, 30) = -0.025 \quad (66)$$

a) Lösung:

- $\frac{\partial m}{\partial c}$ hat die Einheit kg/kcal. Also würde man, wenn man täglich 30 min trainiert und 2500 kcal zu sich nimmt, pro zusätzliche Kilokalorie 0.02 kg zunehmen.
- $\frac{\partial m}{\partial t}$ hat die Einheit kg/min. Also würde man, wenn man täglich 30 min trainiert und 2500 kcal zu sich nimmt, pro zusätzlicher Trainingsminute 0.025 kg abnehmen.

b) Ansteckungsrisiko

Nehmen wir an, die Virenbelastung der Luft in der Nähe einer erkrankten Person nehme wie

$$v(d) = \exp\left(-\frac{1}{5}d^2\right) \quad (67)$$

mit der Distanz d von der infektiösen Person ab. (Das ist ein einfaches Modell, das nicht den Anspruch hat, die Realität korrekt abzubilden.)

Zudem nehme das Ansteckungsrisiko r linear mit der Virenbelastung v und der ihr ausgesetzten Zeit t zu, also

$$r(t, d) = t \cdot v(d) = t \cdot \exp\left(-\frac{1}{5}d^2\right) \quad (68)$$

(Bemerkung: Diese Funktion kann nicht direkt als Wahrscheinlichkeitsdichte interpretiert werden, da sie nicht korrekt normiert ist.)

Ein Bild sagt wohl mehr als die Funktionsgleichung:

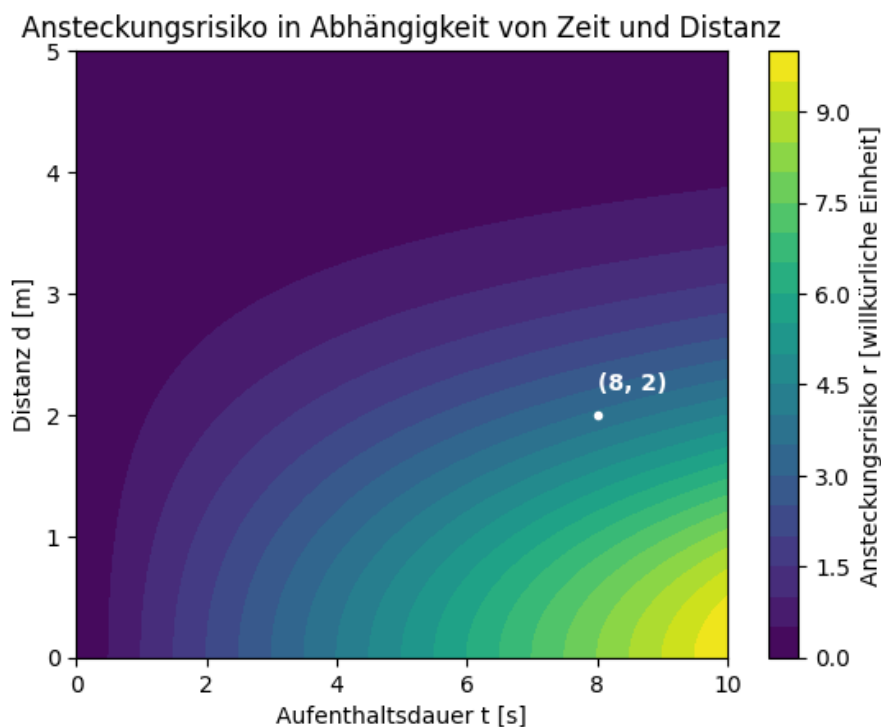
```

import numpy as np
import matplotlib.pyplot as plt

def r(t, d):
    return t * np.exp( -(d**2) / 5)

t = np.linspace(0, 10, 100)
d = np.linspace(0, 5, 100)
T, D = np.meshgrid(t, d)
R = r(T, D)
plt.contourf(T, D, R, 20)
plt.plot(8, 2, "w.")
plt.text(8, 2.2, "(8, 2)", color="white", weight="bold")
plt.xlabel("Aufenthaltsdauer t [s]")
plt.ylabel("Distanz d [m]")
plt.xlim(t.min(), t.max())
plt.ylim(d.min(), d.max())
plt.colorbar(label="Ansteckungsrisiko r [willkürliche Einheit]")
plt.title("Ansteckungsrisiko in Abhängigkeit von Zeit und Distanz")
plt.show()

```



Hier nochmals die Funktion: $r(t, d) = t \cdot \exp(-d^2/5)$. Das Risiko sei eine einheitslose Grösse.

1. Berechne $\frac{\partial r}{\partial t}$ und $\frac{\partial r}{\partial d}$.
2. Werte die Funktion r und die partiellen Ableitungen an der Stelle $(t, d) = (8, 2)$ aus. (Dieser Punkt ist im Diagramm eingezeichnet.)
3. Bestimme die Einheiten der partiellen Ableitungen. (Notation: $[x]$ bedeutet "die Einheit der Variable x ".)
4. Beschreibe die Bedeutung dieser Grössen.

Lösungen:**1. \$\$**

$$\frac{\partial r}{\partial t} = \exp\left(-\frac{1}{5}d^2\right)$$

$$\frac{\partial r}{\partial d} = t \cdot \exp\left(-\frac{1}{5}d^2\right) \cdot \left(-\frac{2}{5}d\right)$$

\$\$

2. \$\$

$$r(8, 2) = 8 \cdot \exp\left(-\frac{1}{5}2^2\right) \approx 3.59$$

$$\frac{\partial r}{\partial t}(8, 2) \approx 0.45$$

$$\frac{\partial r}{\partial d}(8, 2) \approx -2.9$$

\$\$

3. \$\$

$$\left[\frac{\partial r}{\partial t}\right] = \frac{[r]}{[t]} = \text{s}^{-1}$$

$$\left[\frac{\partial r}{\partial d}\right] = \frac{[r]}{[d]} = \text{m}^{-1}$$

\$\$

4.

- $\frac{\partial r}{\partial t}(8, 2) \approx 0.45 \text{ s}^{-1}$ bedeutet, dass sich das Ansteckungsrisiko bei $(t, d) = (8, 2)$ um 0.45 (oder 12.5%) pro Sekunde erhöht, wenn man stehen bleibt.
- $\frac{\partial r}{\partial d}(8, 2) \approx -2.9 \text{ m}^{-1}$ bedeutet, dass sich das Ansteckungsrisiko bei $(t, d) = (8, 2)$ um 2.9 (oder 80.8%) oder pro Meter verringert, wenn man sich instantan weiter weg bewegen würde.

Weiterführendes Material (optional)

1. Gradient In der Vertiefung wird das Konzept des Gradienten aufgegriffen. Damit alle ein grundsätzliches Verständnis davon haben, empfehlen wir diese beiden Videos:

- “Gradient: Definition” - Georg Hoever
- “Gradient: Bedeutung” - Georg Hoever

oder auch eines von diesen hier:

- “GRADIENT berechnen – mehrdimensionale Funktion” - MathemaTrick
- “Gradient + Richtung des steilsten Anstiegs” - MathePeter

2. Logarithmische Darstellung Für den folgenden Präsenzünterricht ist es hilfreich ein Verständnis der logarithmischen Darstellungen zu haben. Die folgenden Videos können dir dabei helfen:

- “Logarithmische Skalierung” - Georg Hoever
- “Logarithmische Achsen – Grundlagen (horizontale Achse)” - simpleclub
- “Logarithmische Achsen – Was man beachten muss” - simpleclub
- “Logarithmische Achsen – vertikale Achse” - simpleclub

0.7.2 Ableitung mehrere Variablen

Übungen - Lösungen

```
import numpy as np
import matplotlib.pyplot as plt
```

Aufgabe 1: Partielle Ableitungen berechnen Hier geht es um die Funktion $f(x, y) = x^2 - 2y^2$.

a) Von Hand Berechne von Hand die beiden partiellen Ableitungen $f_x(x, y)$ und $f_y(x, y)$ und werte beide an der Stelle $(x, y) = (1, 3)$ aus.

$$\begin{aligned} f_x(x, y) &= 2x, & f_x(1, 3) &= 2 \cdot 1 = 2 \\ f_y(x, y) &= -4y, & f_y(1, 3) &= -4 \cdot 3 = -12 \end{aligned}$$

b) Numerisch Berechne die beiden partiellen Ableitungen an der Stelle $(x, y) = (1, 3)$ mithilfe der numerischen Approximation (siehe Vertiefung).

```
def f(x, y):
    return x**2 - 2 * y**2

def partial_x(f, x, y, h=1e-6):
    return (f(x + h, y) - f(x - h, y)) / (2 * h)

def partial_y(f, x, y, h=1e-6):
    return (f(x, y + h) - f(x, y - h)) / (2 * h)

x, y = 1, 3

f_x = partial_x(f, x, y)
f_y = partial_y(f, x, y)

print(f"f_x{(x, y)} = {f_x}")
print(f"f_y{(x, y)} = {f_y}")

f_x(1, 3) = 2.000000000279556
f_y(1, 3) = -12.000000001677336
```

Aufgabe 2: Gradient numerisch berechnen

a) Implementierung Implementiere eine Funktion `gradient_2d(f, x, y, h=5e-6)`, die den Gradienten einer beliebigen Funktion $f(x, y)$ an der Stelle (x, y) berechnet. Die Funktion soll die beiden partiellen Ableitungen numerisch berechnen und den Gradienten als NumPy Array zurückgeben.

```
def gradient_2d(f, x, y, h=5e-6):
    return np.array(
        [
            (f(x + h, y) - f(x - h, y)) / (2 * h),
            (f(x, y + h) - f(x, y - h)) / (2 * h),
        ]
    )
```

b) Test Teste die Funktion mit der Funktion $f(x, y) = x^2 - 2y^2$ an der Stelle $(x, y) = (1, 3)$ und vergleiche mit der analytischen Rechnung (von Hand).

```
gradient_2d(f, 1, 3)
```

```
array([ 2., -12.])
```

c) Darstellung Stelle den Gradienten der Funktion $f(x, y) = x^2 - 2y^2$ im Bereich $x \in [-2, 2]$ und $y \in [-1.5, 1.5]$ dar. Verwende

- `np.linspace` und `np.meshgrid` für das Erstellen der x - und y -Gitterpunkte,
- `gradient_2d` für das Berechnen des Gradienten an den Gitterpunkten und
- `plt.quiver` für das Darstellen des Gradienten als Pfeile.

Hinweis: Erzeuge nicht zu viele Gitterpunkte, da die Gradientenpfeile sonst zu dicht und klein werden.

Option 1: Als fünftes Argument von `plt.quiver` kann die Farbe der Pfeile angegeben werden. Verwende die Funktionswerte $z = f(x, y)$ als Farbe der Pfeile.

Option 2: Ersetze `plt.quiver` durch `plt.streamplot`, um den Gradienten als Stömungsfeld zu visualisieren.

```
n = 21
x = np.linspace(-2, 2, n)
y = np.linspace(-1.5, 1.5, n)
X, Y = np.meshgrid(x, y)

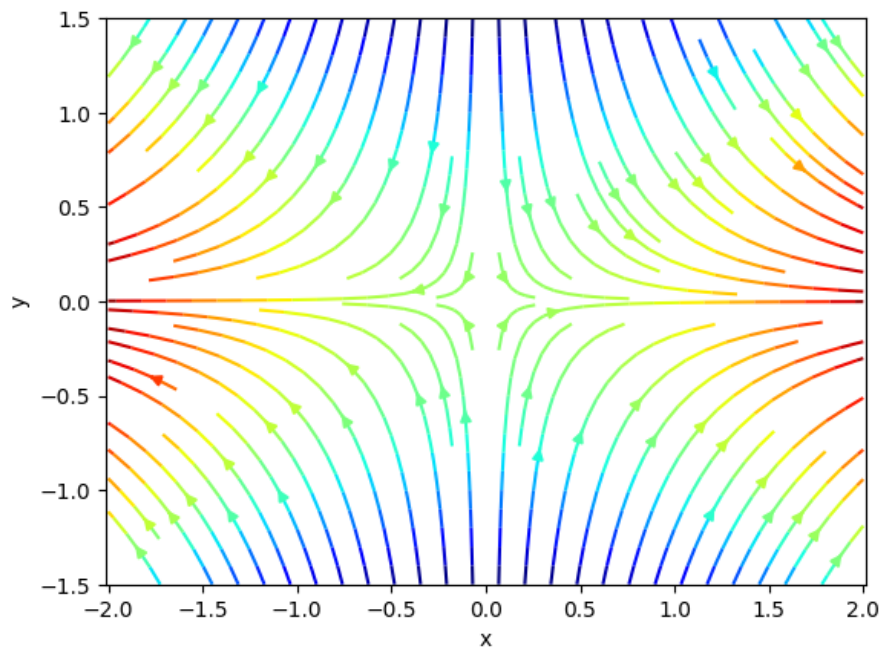
Z = f(X, Y)
gradx, grady = gradient_2d(f, X, Y)

# Version gleicher Farbe für alle Pfeile
# plt.quiver(X, Y, gradx, grady, color="r", angles="xy")

# Option 1: Version mit Farbverlauf abhängig von Z
# plt.quiver(X, Y, gradx, grady, Z, cmap="jet", angles="xy")

# Option 2:
plt.streamplot(X, Y, gradx, grady, color=Z, cmap="jet")

plt.xlabel("x")
plt.ylabel("y")
plt.axis("equal")
plt.show()
```



Aufgabe 3: Höhenlinien und Gradient zusammen darstellen Gegeben sei die Funktion

$$g(x, y) = y \cdot e^{-x^2} \quad (69)$$

1. Stelle die Höhenlinien im Bereich $x, y \in [-2, 2]$ mit 200×200 Punkten dar.
2. Zeichne zusätzlich den Gradienten an 15×15 ein.

Hierfür solltest du folgende Funktion verwenden:

- `np.linspace`
- `np.meshgrid`
- `plt.contourf`
- `np.gradient`
- `plt.quiver`

Wenn du weniger Punkte für den `np.linspace` verwendest, dann bekommst du auch weniger Gradienten-Pfeile und sie werden etwas länger (und somit grösser) dargestellt. Probiere es aus.

```
def g(x, y):
    return y * np.exp( -(x**2))

n = 200
x = np.linspace( -2, 2, n)
y = np.linspace( -2, 2, n)
X, Y = np.meshgrid(x, y)
Z = g(X, Y)
plt.contourf(X, Y, Z, cmap="Spectral")
plt.colorbar(label="f(x,y)")

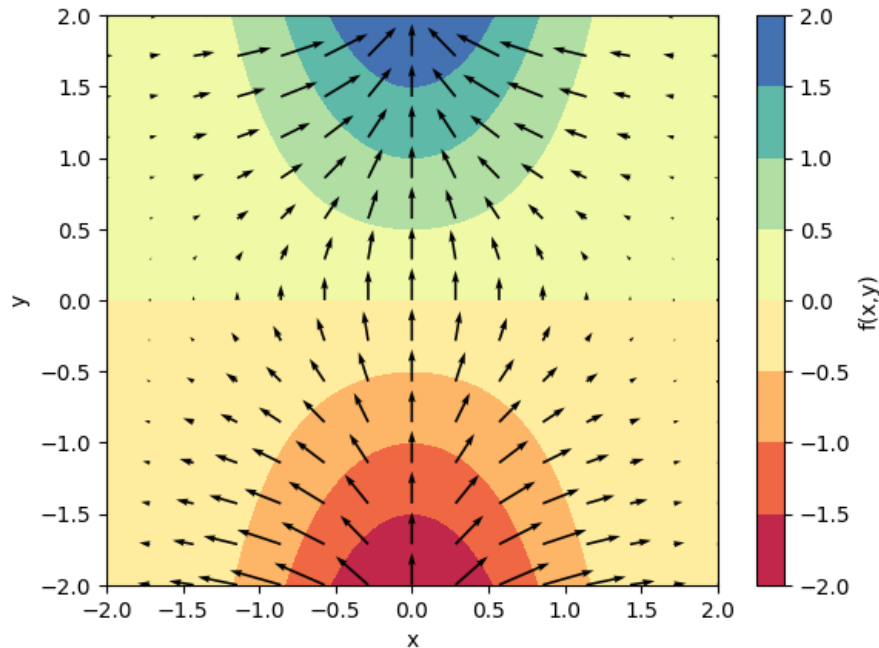
n2 = 15
x2 = np.linspace( -2, 2, n2)
y2 = np.linspace( -2, 2, n2)
X2, Y2 = np.meshgrid(x2, y2)
Z2 = g(X2, Y2)
```

```

grady, gradx = np.gradient(Z2, y2, x2)
plt.quiver(X2, Y2, gradx, grady, angles="xy")

plt.xlabel("x")
plt.ylabel("y")
plt.show()

```



Aufgabe 4: Gradientenverfahren in 2D Hier ist die Implementation des Gradientenverfahrens aus der Vertiefung:

```

def gradient_descent_2d(grad_f, x0, y0, alpha, tol, max_steps):
    """
    Performs gradient descent optimization in 2D.

    Parameters:
    - 'grad_f': The gradient function that calculates the gradient at a given point.
    - 'x0': The initial x -coordinate.
    - 'y0': The initial y -coordinate.
    - 'alpha': The learning rate or step size.
    - 'tol': The tolerance value to stop the optimization.
    - 'max_steps': The maximum number of steps to perform.

    Returns:
    - 'res': A list of tuples containing the x -coordinate, y -coordinate, and error at each step.
    """
    x, y = x0, y0
    grad = grad_f(x, y)
    err = np.sqrt(np.sum(grad**2))
    res = [(x, y, err)]
    while len(res) < max_steps and err > tol:
        x = x - alpha * grad[0]
        y = y - alpha * grad[1]
        grad = grad_f(x, y)
        err = np.sqrt(np.sum(grad**2))

```

```

        res.append((x, y, err))
    return res

```

Nun soll ein Minimum der folgenden Funktion bestimmt werden:

$$f(x, y) = x^4 + \frac{1}{2}y^4 - 2x^2 - 3y^2 + x(y - 1) + 10 \quad (70)$$

1. Bestimme den Gradienten der Funktion analytisch von Hand. Überprüfe dein Resultat mithilfe von Wolfram|Alpha. Beispiel: Wolfram|Alpha: gradient of $x\hat{y}$
2. Nutze die Funktion `gradient_descent_2d` von oben, um das Minimum (bzw. eines der Minima) zu finden.
3. Stelle die Höhenlinien im Bereich $x \in [-2, 2]$ und $y \in [-3, 3]$ dar.
4. Visualisiere den Verlauf des Gradientenverfahrens (analog zur Vertiefung)
5. Experimentiere mit verschiedenen Startwerten und Lernraten.

Tipp: Mithilfe von Wolfram|Alpha kannst du zum Vergleich das Minimum der Funktion auch bestimmen (*minimize ...*).

Optional: Stelle den Gradienten der Funktion auch noch dar.

```

def f(x, y):
    return x**4 + y**4 / 2 - 2 * x**2 - 3 * y**2 + x * (y - 1) + 10

def grad_f(x, y):
    return np.array(
        [
            4 * x**3 - 4 * x + y - 1,
            2 * y**3 - 6 * y + x,
        ]
    )

# Parameter für das Gradientenverfahren
x0 = 0
y0 = 0.1
alpha = 1e -2
tol = 1e -6
max_steps = 1000

# Gradientenverfahren durchführen
res = gradient_descent_2d(grad_f, x0, y0, alpha, tol, max_steps)
xs, ys, err = np.array(res).T

# Daten für Darstellung Höhenlinien und theoretisches Minimum
n = 200
x = np.linspace( -2, 2, n)
y = np.linspace( -3, 3, n)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

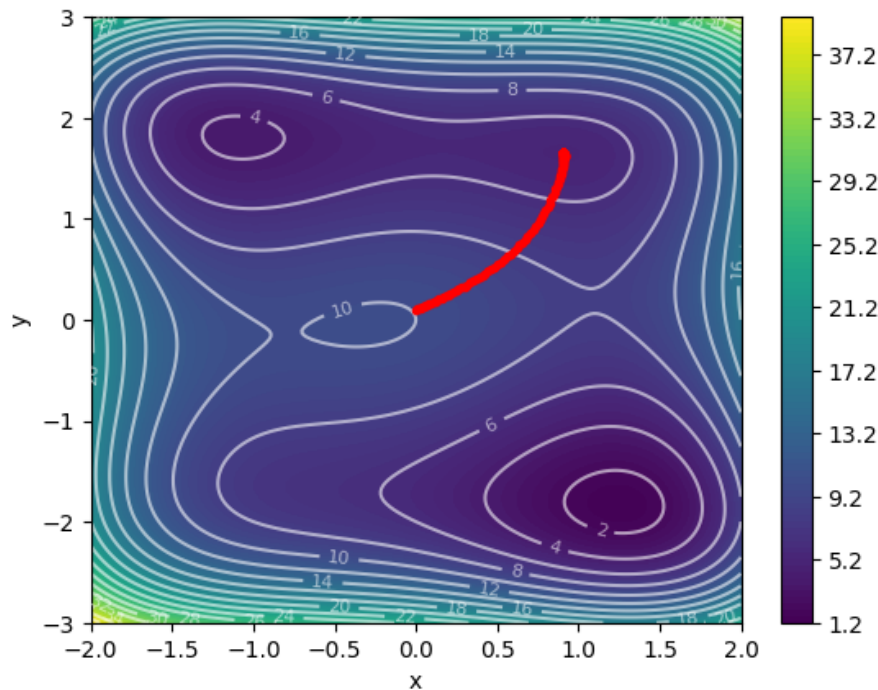
# Darstellung
plt.contourf(X, Y, Z, levels=100)

```



```
plt.colorbar()
cs = plt.contour(X, Y, Z, levels=20, colors="white", alpha=0.6)
plt.clabel(cs, fontsize=8)
plt.plot(xs, ys, "r.")
plt.xlabel("x")
plt.ylabel("y")
plt.show()

print(f"found (x, y) = ({xs[ -1]}, {ys[ -1]})")
print(f"in {len(res)} steps (err={err[ -1]:.2e})")
```



```
found (x, y) = (0.9057128493201139, 1.6509702585046229)
in 277 steps (err=1.00e -06)
```

Aufgabe 5: Gradientenverfahren in 1D (optional)

a) Wie funktioniert das Gradientenverfahren? Schreibe Schritt für Schritt auf, wie das Gradientenverfahren funktioniert und was die Rollen der Parameter sind. Analysiere dazu die Implementierung in der Funktion `gradient_descent_2d`.

Antwort:

Das Gradientenverfahren wird verwendet, um das Minimum einer Zielfunktion zu finden. Dabei wird der Gradient (steilster Anstieg) der Funktion an der aktuellen Stelle berechnet und dann die Position in entgegengesetzter Richtung des Gradienten verschoben.

1. Initialisiere die Variablen mit Startwert (x_0, y_0) .
2. Berechne den Gradienten der Funktion an der aktuellen Position der Variablen.
3. Aktualisiere die Variablen in die entgegengesetzte Richtung des Gradienten mit einem Schrittgrößenfaktor (`alpha`, Lernrate).
4. Wiederhole die Schritte 2 und 3 bis die Änderung der Zielfunktion kleiner ist als die Toleranz (`tol`) oder eine maximale Anzahl von Iterationen (`max_steps`) erreicht ist.

Um das Maximum einer Funktion zu finden, kann einfach in Richtung des Gradienten statt in die Gegenrichtung gewandert werden. Ansonsten ist der Ablauf der gleiche.

Es ist wichtig darauf hinzuweisen, dass das Gradientenverfahren nur dann garantiert das globale Extremum findet, wenn die Funktion konvex ist. In anderen Fällen kann es leider in einem lokalen Extremum steckenbleiben.

Beschreibung der Parameter:

Der Startwert (`x0,y0`), auch bekannt als Anfangswert oder Initialisierungswert, bezieht sich auf die Anfangswerte der Variablen, die der Algorithmus verwendet, um die Optimierung zu beginnen. Der Startwert hat grossen Einfluss darauf, wo der Algorithmus im Suchraum beginnt und kann daher auch Einfluss darauf haben, welches Minimum er findet. Ein geeigneter Startwert kann die Konvergenz des Algorithmus beschleunigen und das Risiko verringern, in ein lokales Minimum zu fallen.

Der Alpha-Wert, auch bekannt als Lernrate, bestimmt, wie schnell der Algorithmus sich durch den Suchraum bewegt. Ein kleiner Alpha-Wert führt zu kleinen Schritten und kann die Konvergenz des Algorithmus verlangsamen, während ein grosser Alpha-Wert zu grösseren Schritten und schnellerer Konvergenz führen kann. Andererseits besteht bei grossem Alpha das Risiko, das optimale Minimum zu überspringen. Ein geeigneter Alpha-Wert hängt von der spezifischen Anwendung und dem zugrunde liegenden Problem ab und muss in der Regel experimentell bestimmt werden.

b) Implementierung in 1D Implementiere das Gradientenverfahren in 1D. Dazu kannst Du die Funktion `gradient_descent_2d` von oben als Vorlage nutzen. Nenne die neue Funktion `gradient_descent_1d`. Diese soll wieder eine Liste der Zwischenwerte und deren Fehlerabschätzung zurückgeben.

```
def gradient_descent_1d(grad_f, x0, alpha, tol, max_steps):
    x = x0
    grad = grad_f(x)
    err = np.abs(grad)
    res = [(x, err)]
    while len(res) < max_steps and err > tol:
        x = x - alpha * grad
        grad = grad_f(x)
        err = np.abs(grad)
        res.append((x, err))
    return res
```

c) Test Teste die Implementierung mit der Funktion

$$f(x) = x^2 - 6x \quad (71)$$

Probiere unterschiedliche Startwerte `x0` im Bereich von $[-10, 10]$ und `alpha`-Werte im Intervall $]0, 1[$ aus. Beschreibe welchen Einfluss `x0` und `alpha` auf das Verhalten des Algorithmus haben.

Um den Verlauf des Gradientenverfahrens zu visualisieren, kannst du z.B. die Folge der x -Werte mit `plt.plot` oder `plt.stem` darstellen.

```
def f(x):
    return x**2 - 6 * x

def grad_f(x):
    return 2 * x - 6

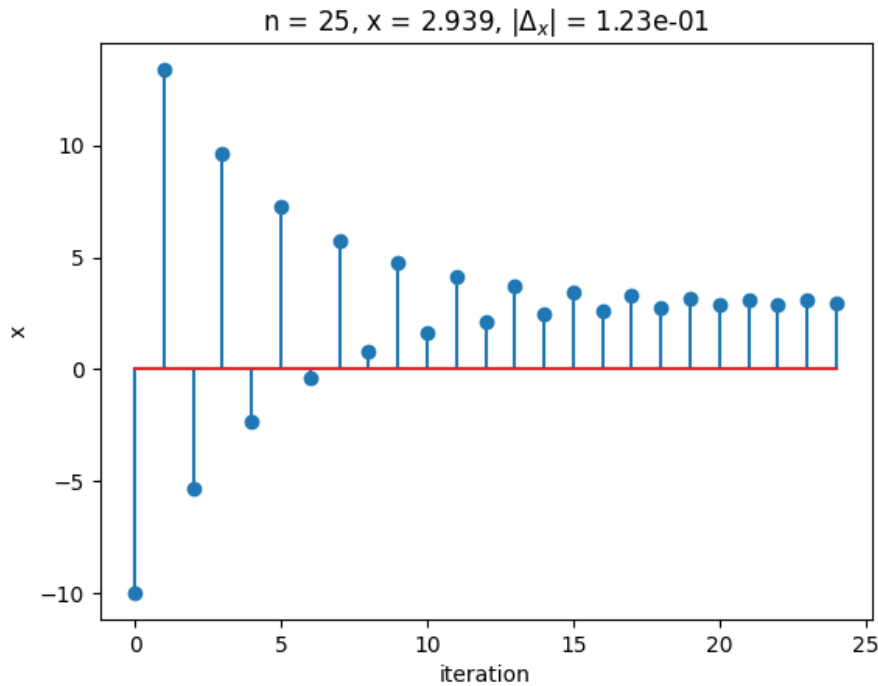
x0 = -10
alpha = 0.9
```

```

tol = 1e -4
max_steps = 25
res = gradient_descent_1d(grad_f, x0, alpha, tol, max_steps)

xs, errs = np.array(res).T
plt.stem(xs)
plt.xlabel("iteration")
plt.ylabel("x")
plt.title(f"n = {len(xs)}, x = {xs[ -1]:.3f},  $|\Delta_x| = {errs[ -1]:.2e}")$ 
plt.show()

```



Eine spannende Darstellung könnte auch so aussehen. Hier sind **xs** die x -Werte aus dem Resultat des Gradientenverfahrens.

Dann wird die Funktion $f(x)$ dargestellt und darauf die Punkte $(x_i, f(x_i))$, die beim Gradientenverfahren durchlaufen werden, markiert, nummeriert und mit Pfeilen verbunden.

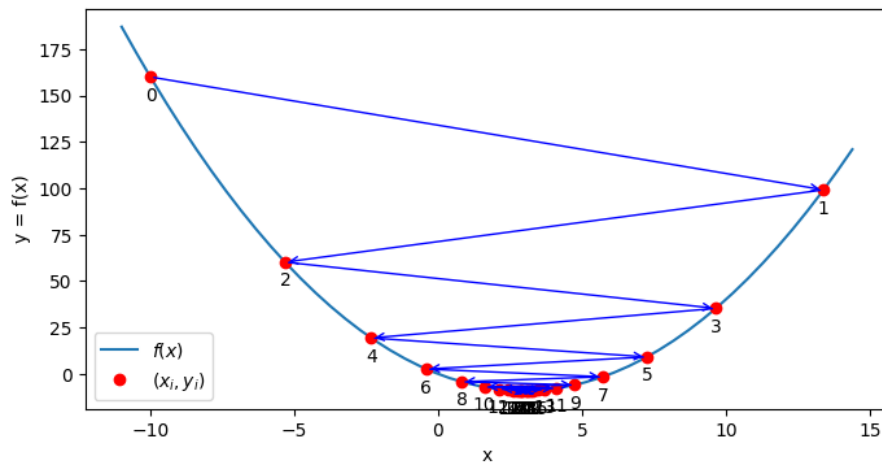
```

ys = f(xs)
x = np.linspace(xs.min() - 1, xs.max() + 1)
y = f(x)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, y, label="$f(x)$")
ax.plot(xs, ys, "ro", label="$x_i, y_i$")
for i, (xi, yi) in enumerate(zip(xs, ys)):
    ax.text(xi, yi - 5, f"{i}", horizontalalignment="center", verticalalignment="top")
    if i > 0:
        ax.annotate(
            "",
            xy=(xi, yi),
            xycoords="data",
            xytext=(xs[i - 1], ys[i - 1]),
            textcoords="data",
            arrowprops=dict(arrowstyle="->", connectionstyle="arc3", color="b"),
        )

```

```
ax.set_xlabel("x")
ax.set_ylabel("y = f(x)")
ax.legend()
plt.show()
```



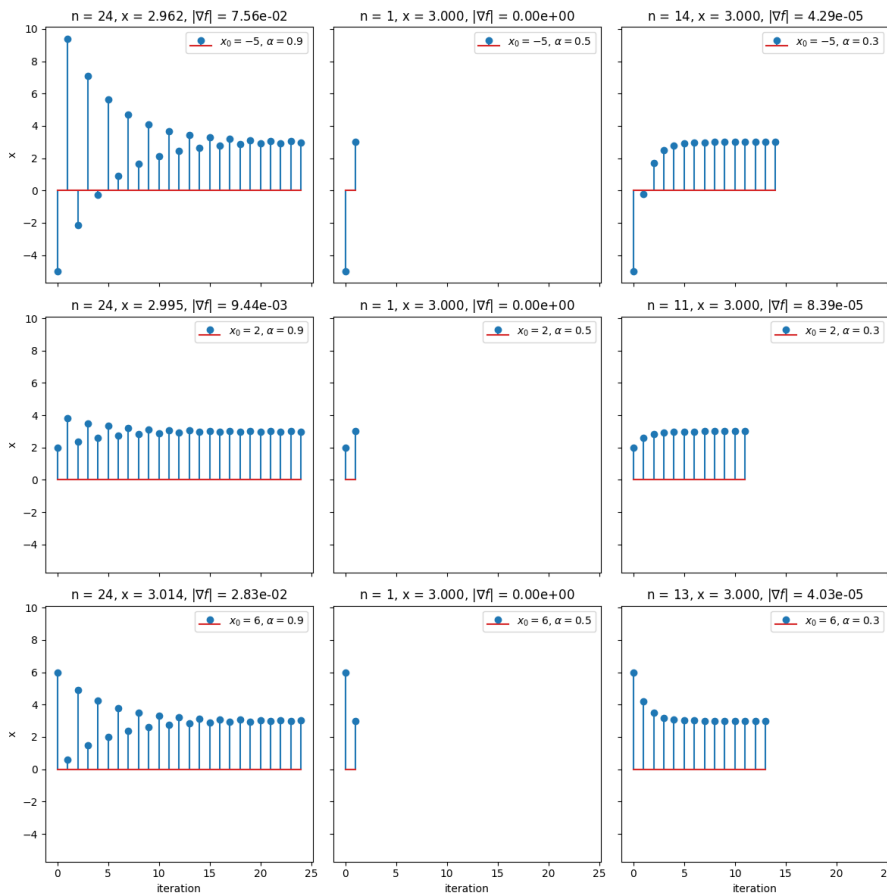
d) Verfahren mehrmals durchführen und Resultate vergleichen Nun soll das Gradientenverfahren mehrmals durchgeführt werden und die Resultate verglichen werden.

Dazu soll eine Liste von x_0 -Werten und eine Liste von unterschiedlichen α -Werten gegeben werden. Für jede Kombination soll das Gradientenverfahren ausgeführt und die Resultate in einem Subplot dargestellt werden.

Entscheide selbst, ob du die Darstellung des Verlaufs der x -Werte (mit `plt.stem`) oder die Darstellung der Punkte in der Zielfunktion (mit `plt.plot`) bevorzugst.

```
x0_list = [ -5, 2, 6]
alpha_list = [0.9, 0.5, 0.3]
```

```
fig, axs = plt.subplots(
    len(x0_list), len(alpha_list), figsize=(12, 12), sharey=True, sharex=True
)
for i, x0 in enumerate(x0_list):
    for j, alpha in enumerate(alpha_list):
        res = gradient_descent_1d(grad_f, x0, alpha, tol, max_steps)
        xs, errs = np.array(res).T
        ax = axs[i, j]
        ax.stem(xs, label=f"$x_0 = {x0}$, $\alpha = {alpha}$")
        ax.legend()
        ax.set_title(
            f"$n = {len(xs) - 1}$, $x = {xs[-1]:.3f}$, $||\nabla f|| = {errs[-1]:.2e}$"
        )
        if i == len(x0_list) - 1:
            ax.set_xlabel("iteration")
        if j == 0:
            ax.set_ylabel("x")
plt.tight_layout()
plt.show()
```



```
fig, axs = plt.subplots(
    len(x0_list), len(alpha_list), figsize=(12, 12), sharex=True, sharey=True
)
```

```
for i, x0 in enumerate(x0_list):
```

```
    for j, alpha in enumerate(alpha_list):
```

```
        res = gradient_descent_1d(grad_f, x0, alpha, tol, max_steps)
        xs, errs = np.array(res).T
```

```
        ax = axs[i, j]
        ax.plot(x, y, label="$f(x)$")
        ys = f(xs)
        ax.plot(xs, ys, "ro", label="$ (x_i, y_i) $")
```

```
        for k, (xi, yi) in enumerate(zip(xs, ys)):
            ax.text(
                xi,
                yi - 5,
                f"{k}",
                horizontalalignment="center",
                verticalalignment="top",
            )
```

```
        if k > 0:
            ax.annotate(
                "",
                xy=(xi, yi),
```

```

        xycoords="data",
        xytext=(xs[k - 1], ys[k - 1]),
        textcoords="data",
        arrowprops=dict(arrowstyle="->", connectionstyle="arc3", color="b"),
    )
    ax.text(
        0,
        130,
        f"n = {len(xs) - 1}",
        horizontalalignment="center",
        verticalalignment="top",
    )
    ax.text(
        0,
        115,
        f"x = {xs[-1]:.4f}",
        horizontalalignment="center",
        verticalalignment="top",
    )
    ax.text(
        0,
        100,
        f"$|\\nabla f| = {errs[-1]:.2e}",
        horizontalalignment="center",
        verticalalignment="top",
    )

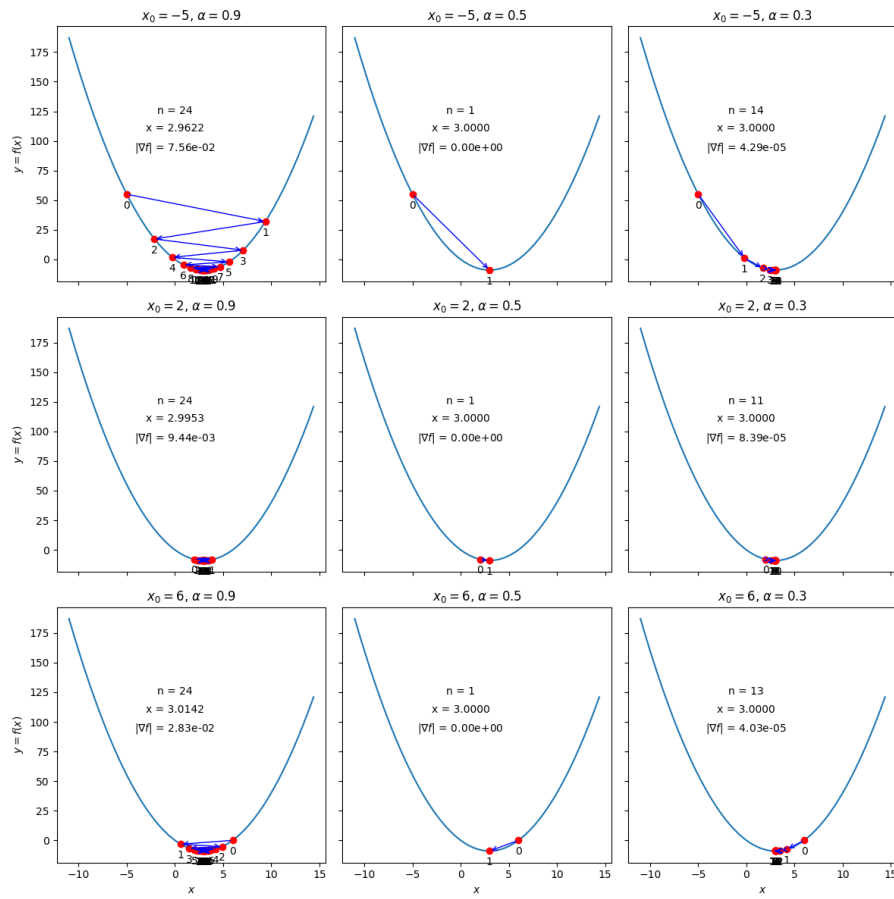
    ax.set_title(f"$x_0 = {x0}$, $\\alpha = {alpha}$")

    if i == len(x0_list) - 1:
        ax.set_xlabel("$x$")

    if j == 0:
        ax.set_ylabel("$y = f(x)$")

plt.tight_layout()
plt.show()

```



0.7.3 Ableitung mit mehreren Variablen

Vertiefung

```
import numpy as np
import matplotlib.pyplot as plt
```

Übersicht

1. Partielle Ableitungen und Gradient darstellen
2. Gradient numerisch berechnen
3. Gradientenverfahren

1. Partielle Ableitungen und Gradient darstellen Als Beispiel nehmen wir die Funktion

$$z = f(x, y) = -3x^2 + 5xy + y^3 \quad (72)$$

und betrachten zuerst mal die Höhenlinien und eine 3D-Darstellung davon.

```
def f(x, y):
    return -3 * x**2 + 5 * x * y + y**3

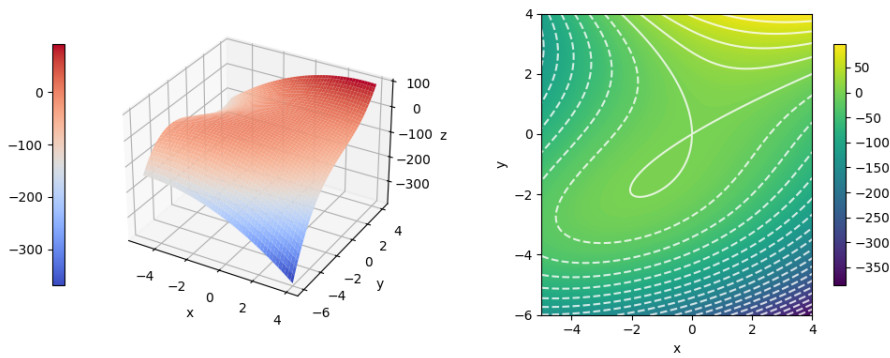
n = 200
x = np.linspace(-5, 4, n)
y = np.linspace(-6, 4, n)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

fig = plt.figure(figsize=(10, 4))

# Fläche im Raum
ax1 = fig.add_subplot(121, projection="3d")
surf = ax1.plot_surface(X, Y, Z, cmap="coolwarm")
plt.colorbar(surf, ax=ax1, shrink=0.8, location="left")
ax1.set_xlabel("x")
ax1.set_ylabel("y")
ax1.set_zlabel("z")

# Höhenlinien
ax2 = fig.add_subplot(122)
c = ax2.contourf(X, Y, Z, levels=200)
ax2.contour(X, Y, Z, levels=25, colors="w", alpha=0.8)
ax2.set_aspect("equal")
plt.colorbar(c, ax=ax2, shrink=0.8)
ax2.set_xlabel("x")
ax2.set_ylabel("y")

plt.tight_layout()
plt.show()
```

a) Partielle Ableitungen berechnen Die beiden partiellen Ableitungen von $f(x, y) = -3x^2 + 5xy + y^3$ sind

$$\frac{\partial f}{\partial x} = -6x + 5y$$

$$\frac{\partial f}{\partial y} = 5x + 3y^2$$

Dies sind beides wieder Funktionen von x und y . Wir können sie also auch wieder an einer Stelle (x, y) auswerten, z.B.

$$f(2, 1) = -3 \cdot 2^2 + 5 \cdot 2 \cdot 1 + 1^3 = -1$$

$$\frac{\partial f}{\partial x}(2, 1) = -6 \cdot 2 + 5 \cdot 1 = -7$$

$$\frac{\partial f}{\partial y}(2, 1) = 5 \cdot 2 + 3 \cdot 1^2 = 13$$

b) Partielle Ableitungen darstellen Zuerst definieren wir die analytischen Ableitungen:

```
def dfdx(x, y):
    return -6 * x + 5 * y
```

```
def dfdy(x, y):
    return 5 * x + 3 * y**2
```

Als Visualisierung stellen wir mal die Isokonturlinien für konstante x bzw. y , jeweils mit der partiellen Ableitung nach x bzw. y darunter dar.

```
x_levels = [ -2, 0, 2]
y_levels = [ -2, 0, 2]
```

```
fig, axs = plt.subplots(2, 2, sharex="col", figsize=(8, 6))
```

```
# Isokonturen x=konst und partielle Ableitung nach y
for xi in x_levels:
    z = f(xi, y)
    z_x = dfdy(xi, y)
    axs[0, 0].plot(y, z, label=f"$x={xi}$")
    axs[1, 0].plot(y, z_x, label=f"$x={xi}$")
```

```

axs[0, 0].set_title("Isokonturen konstante x")
axs[0, 0].set_ylabel(r"$f(x, y)$")
axs[0, 0].legend()
axs[0, 0].grid()

axs[1, 0].set_title("Partielle Ableitung nach y")
axs[1, 0].set_xlabel(r"$y$")
axs[1, 0].set_ylabel(r"$\partial f / \partial y (x,y)$")
axs[1, 0].grid()

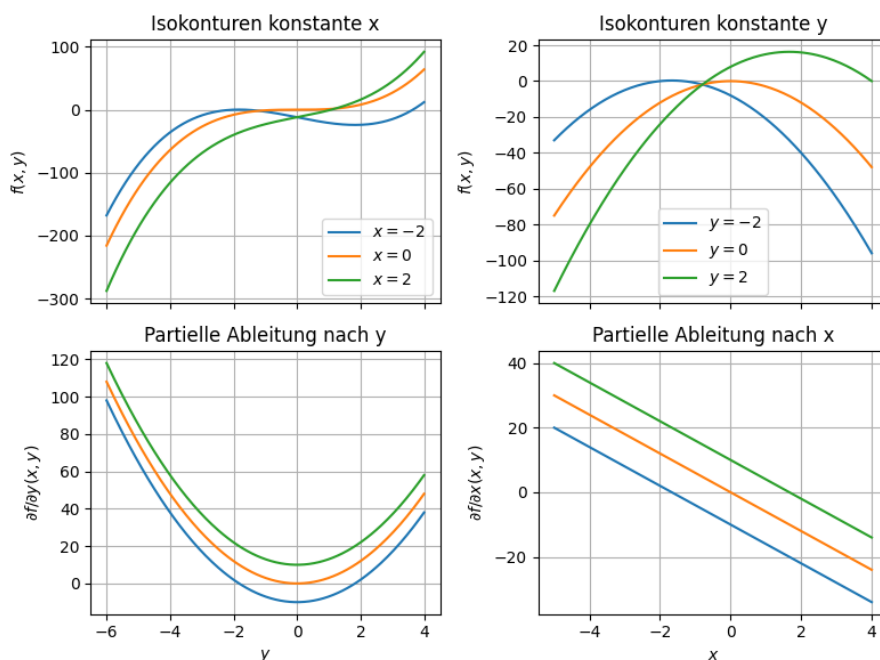
# Isokonturen y=konst und partielle Ableitung nach x
for yi in y_levels:
    z = f(x, yi)
    z_y = dfdx(x, yi)
    axs[0, 1].plot(x, z, label=f"$y={yi}$")
    axs[1, 1].plot(x, z_y, label=f"$y={yi}$")

axs[0, 1].set_title("Isokonturen konstante y")
axs[0, 1].set_ylabel(r"$f(x, y)$")
axs[0, 1].legend()
axs[0, 1].grid()

axs[1, 1].set_title("Partielle Ableitung nach x")
axs[1, 1].set_xlabel(r"$x$")
axs[1, 1].set_ylabel(r"$\partial f / \partial x (x,y)$")
axs[1, 1].grid()

plt.tight_layout()
plt.show()

```



Beobachtungen:

- **x konstant (links)**
 - f ist ein Polynom 3. Grades in y
 - Die partielle Ableitung nach y ist ein Polynom 2. Grades (quadratische Funktion).

- **y konstant (rechts)**

- f ist ein Polynom 2. Grades in x
- Die partielle Ableitung nach x ist ein Polynom 1. Grades (lineare Funktion).

Allerdings gibt diese Darstellung nur beschränkt Auskunft über die partiellen Ableitungen. Diese sind ja auch Funktionen von x und y und nicht nur von der einen der beiden Variablen.

Mithilfe des Gradienten können wir die lokalen Änderungsraten viel einfacher darstellen.

c) Gradient berechnen Der Gradient einer Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$, ist eine Funktion, die jedem Punkt $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$ den **Zeilenvektor aller partiellen Ableitungen** zuordnet:

$$\text{grad}f(\vec{x}) = \left(\frac{\partial f}{\partial x_1}(\vec{x}), \frac{\partial f}{\partial x_2}(\vec{x}), \dots, \frac{\partial f}{\partial x_n}(\vec{x}) \right) \quad (73)$$

In 2 Dimensionen, d.h. für Funktionen von 2 Variablen heisst das:

$$\text{grad}f(x, y) = \left(\frac{\partial f}{\partial x}(x, y), \frac{\partial f}{\partial y}(x, y) \right) \quad (74)$$

Häufig wird der Gradient auch mit dem Symbol $\vec{\nabla}$ geschrieben, um zu betonen, dass es sich um einen Vektor handelt:

$$\vec{\nabla}f(\vec{x}) = \text{grad}f(\vec{x}) \quad (75)$$

Das Symbol $\vec{\nabla}$ wird “Nabla” ausgesprochen.

Der Gradient von $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ist also eine **vektorwertige Funktion** $\vec{\nabla}f : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Sie ordnet jedem Vektor $\vec{x} \in \mathbb{R}^n$ einen Vektor $\vec{\nabla}f(\vec{x}) \in \mathbb{R}^n$ zu. Solche Funktionen werden (v.a. in der Physik) auch Vektorfelder genannt.

In folgenden Videos wird der Gradient nochmal erklärt (optional):

- “Gradient: Definition” - Georg Hoever
- “Gradient: Bedeutung” - Georg Hoever
- “GRADIENT berechnen – mehrdimensionale Funktion” - MathemaTrick
- “Gradient + Richtung des steilsten Anstiegs” - MathePeter

Für unser Beispiel $f(x, y) = -3x^2 + 5xy + y^3$ bedeutet dies konkret:

$$\text{grad}f(x, y) = \left(-6x + 5y, \quad 5x + 3y^2 \right) \quad (76)$$

Wir erhalten zum Beispiel für $(x, y) = (1, 3)$:

$$\text{grad}f(1, 3) = \left(-6 \cdot 1 + 5 \cdot 3, \quad 5 \cdot 1 + 3 \cdot 3^2 \right) = \left(9, \quad 32 \right) \quad (77)$$

d) Gradient darstellen Für eine grafische Darstellung, können wir den Gradienten an einem Gitternetz von Punkten berechnen und als Vektorpfeile darstellen.

Dazu definieren wir zuerst eine Funktion, die den Gradienten berechnet:

```
def grad_f(x, y):
    return np.array([dfdx(x, y), dfdy(x, y)])
```

Dann stellen wir die Vektorpfeile mithilfe von `plt.quiver` dar. Am anschaulichsten ist es, wenn wir die Höhenlinien der Funktion darunter darstellen.

In `plt.quiver` sollte man für die Darstellung von Gradienten die Option `angles="xy"` setzen, damit die Pfeile in die richtige Richtung relativ zum x - y -Koordinatensystem zeigen.

Alternativ kann man ein Vektorfeld wie den Gradienten auch mithilfe von `plt.streamplot` darstellen.

```
x_min, x_max = -4, 4
y_min, y_max = -4, 4
n = 250

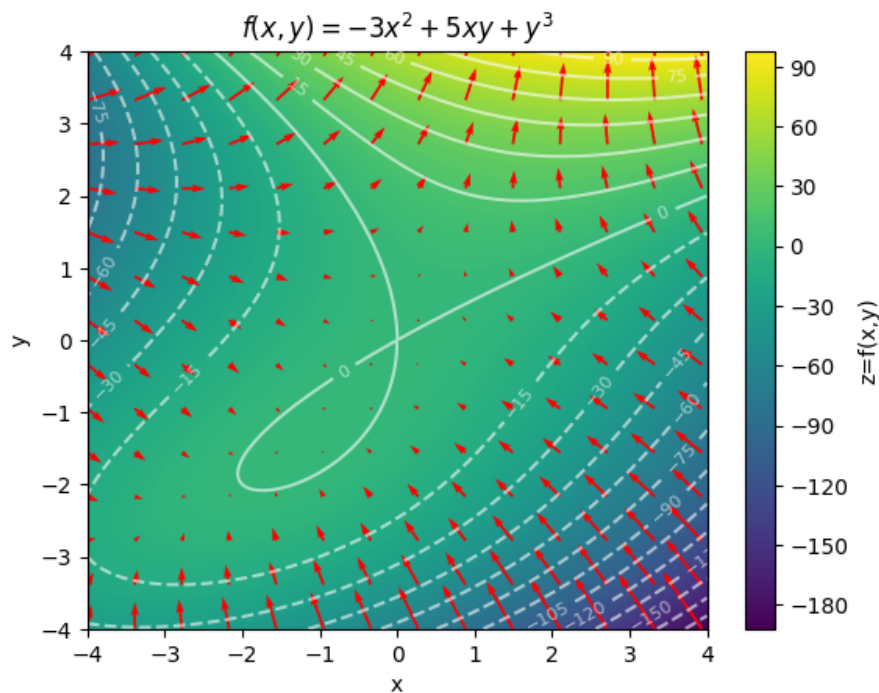
# Funktionswerte für Höhenlinienplot berechnen
x = np.linspace(x_min, x_max, n)
y = np.linspace(y_min, y_max, n)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
grad_x, grad_y = grad_f(X, Y)

# Höhenlinienplot
plt.contourf(X, Y, Z, levels=200)
plt.colorbar(label="z=f(x,y)")
cs = plt.contour(X, Y, Z, levels=25, colors="w", alpha=0.6)
plt.clabel(cs, fontsize=8)

# Gradient als Vektorfeld darstellen
step = n // 13
plt.quiver(
    X[::step, ::step],
    Y[::step, ::step],
    grad_x[::step, ::step],
    grad_y[::step, ::step],
    angles="xy",
    color="r",
)

# Alternativ: Gradient als Strömungsfeld darstellen
# plt.streamplot(X, Y, grad_x, grad_y, color="r", linewidth=0.75)

plt.xlabel("x")
plt.ylabel("y")
plt.title(r"$f(x,y) = -3x^2 + 5xy + y^3$")
plt.show()
```



Wichtige Beobachtungen:

- Der Gradient ist ein *Vektorfeld*, d.h. er beschreibt an jedem Punkt des Definitionsbereichs der Funktion einen Vektor.
- Der Gradient zeigt in die *Richtung des steilsten Anstiegs* der Funktion.
- Der Gradient ist *senkrecht zu den Höhenlinien* der Funktion.
- Die Länge des Gradientenvektors ist proportional zur Steigung der Funktion.
- Mögliche *Extremstellen* oder *Sattelpunkte* befinden sich dort, wo der Gradient gleich dem Nullvektor ist.

2. Gradient numerisch berechnen

a) Numerische Berechnung des Gradienten mit zentralen Differenzen Wie die Ableitung in einer Variablen können wir auch die partielle Ableitung mit dem zentralen Differenzenquotienten approximieren:

$$\frac{\partial f}{\partial x_j}(\vec{x}) \simeq \frac{f(x_1, \dots, x_j + h, \dots, x_n) - f(x_1, \dots, x_j - h, \dots, x_n)}{2h} \quad (78)$$

wobei f eine Funktion von $\vec{x} \in \mathbb{R}^n$ ist und h eine kleine Schrittweite, z.B. $h = 5 \times 10^{-6}$.

Damit kann natürlich auch der Gradient numerisch berechnet werden.

$$\text{grad}f(\vec{x}) \simeq \left(\frac{f(x_1 + h, x_2, \dots, x_n) - f(x_1 - h, x_2, \dots, x_n)}{2h}, \dots, \frac{f(x_1, x_2, \dots, x_n + h) - f(x_1, x_2, \dots, x_n - h)}{2h} \right) \quad (79)$$

Wir implementieren das mal nur für Funktionen, die von 2 Variablen x und y abhängen.

```
def grad_2d(f, x, y, h=5e -6):
    return np.array(
        [
            (f(x + h, y) - f(x - h, y)) / (2 * h),
            (f(x, y + h) - f(x, y - h)) / (2 * h),
        ]
    )
```

Testen können wir dies am Beispiel aus Teil 1: $f(x, y) = -3x^2 + 5xy + y^3$.

Da hatten wir berechnet:

$$\text{grad}f(x, y) = (-6x + 5y, \quad 5x + 3y^2)$$

$$\text{grad}f(1, 3) = (9, \quad 32)$$

```
grad_2d(f, 1, 3), grad_f(1, 3)

(array([ 9., 32.]), array([ 9, 32]))
```

b) Numerische Berechnung des Gradienten mit `np.gradient` `np.gradient` kann man benutzen, um den Gradienten auf Basis von Funktionswerten (Daten) zu berechnen, nicht so wie die oben implementierte Funktion `grad_2D`. Für den Vergleich müssen wir also ein paar Datenpunkte generieren.

Vorsicht: `np.gradient` berechnet den Gradienten in der Reihenfolge der Achsen, also Zeilen vor Spalten, und somit kommt die y-Richtung zuerst und dann die x-Richtung.

```
x0, y0 = 1, 3
h = 1e -6

x = [x0 - h, x0, x0 + h]
y = [y0 - h, y0, y0 + h]

X, Y = np.meshgrid(x, y)
Z = f(X, Y)

gradient_y, gradient_x = np.gradient(Z, y, x)

print("x -Komponente von grad f:")
print(gradient_x)
print("y -Komponente von grad f:")
print(gradient_y)

x -Komponente von grad f:
[[8.99999799 8.999995    8.99999201]
 [9.000003    9.         8.999997   ]
 [9.000008    9.000005    9.000002   ]]
y -Komponente von grad f:
[[31.999986 31.999991 31.999996]
 [31.999995 32.         32.000005]
 [32.000004 32.000009 32.000014]]
```

c) Beispiel: Himmelblau Funktion Hier schauen wir uns nochmals ein Beispiel der Testfunctions for Optimization an, und zwar die Himmelblau Funktion:

```
def himmelblau(x, y):
    return (x**2 + y - 11) ** 2 + (x + y**2 - 7) ** 2

x_min = np.array([3, -3.779310, -2.805118, 3, 3.584428])
y_min = np.array([2, -3.283186, 3.131312, 2, -1.848126])
```

```

n = 500
x = np.linspace( -5, 5, n)
y = np.linspace( -5, 5, n)

X, Y = np.meshgrid(x, y)
Z = himmelblau(X, Y)
grad_y, grad_x = np.gradient(Z, y, x)

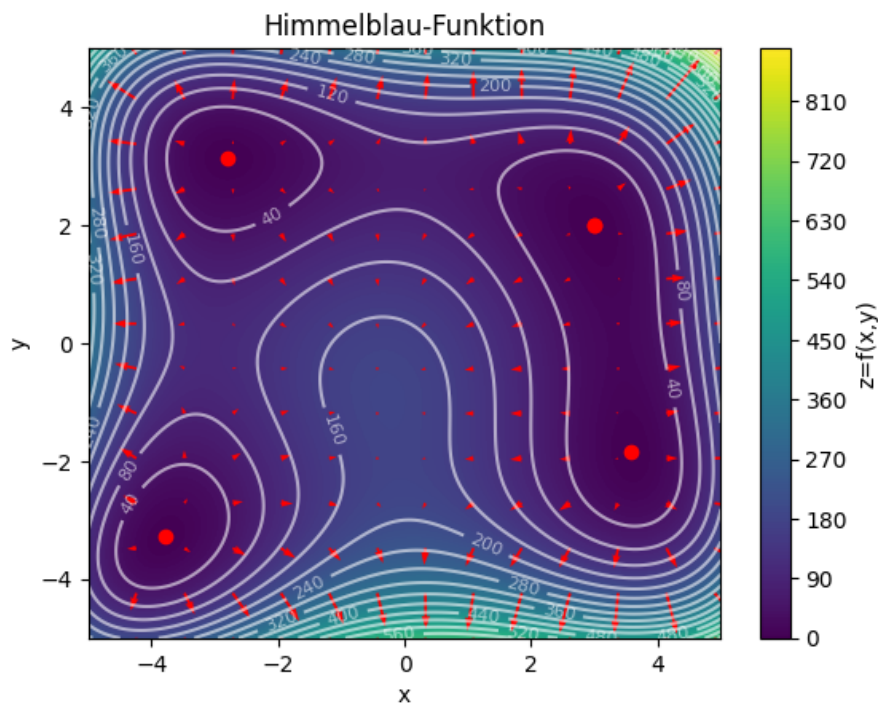
plt.contourf(X, Y, Z, levels=200)
plt.colorbar(label="z=f(x,y)")
cs = plt.contour(X, Y, Z, levels=25, colors="w", alpha=0.6)
plt.clabel(cs, fontsize=8)
plt.scatter(x_min, y_min, c="r")

step = n // 13
plt.quiver(
    X[::step, ::step],
    Y[::step, ::step],
    grad_x[::step, ::step],
    grad_y[::step, ::step],
    angles="xy",
    color="r",
)

# plt.streamplot(X, Y, grad_x, grad_y, color="r", linewidth=0.75)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Himmelblau -Funktion")
plt.show()

```



Hier beobachten wir nochmals schön, dass der Gradient weg von lokalen Minima zeigt, hin zu lokalen oder globalen Maxima. Wenn wir also ein Minimum finden wollen, müssen wir in die entgegengesetzte Richtung des Gradienten gehen.

3. Gradientenverfahren Das Gradientenverfahren (engl. *Gradient Descent*) ist ein Verfahren, um die lokalen Extremstellen einer Funktion zu finden. Es ist ein iteratives Verfahren, das mit einem Startwert $\vec{x}_0$ beginnt und dann immer wieder die Iterationsvorschrift

$$\vec{x}_{i+1} = \vec{x}_i - \alpha_i \cdot \text{grad}f(\vec{x}_i) \quad (80)$$

anwendet, wobei α_i ein Schrittweitenparameter (auch Lernrate genannt) ist.

Das folgende Video zeigt, wie das Gradientenverfahren funktioniert:

- “Gradientenverfahren”, Georg Hoever

```
def gradient_descent_2d(f, x0, y0, alpha, tol, max_steps, h=5e -6):
    x, y = x0, y0
    grad = grad_2d(f, x, y, h)
    err = np.linalg.norm(grad)
    res = [(x, y, err)]
    while len(res) < max_steps and err > tol:
        x = x - alpha * grad[0]
        y = y - alpha * grad[1]
        grad = grad_2d(f, x, y, h)
        err = np.sqrt(np.sum(grad**2))
        res.append((x, y, err))
    return np.array(res)
```

Beispiel Himmelblau Funktion

```
res = gradient_descent_2d(
    f=himmelblau,
    x0=0,
    y0=0,
    alpha=1e -2,
    tol=1e -8,
    max_steps=1000,
)
x_gd, y_gd, err = res.T

print(f"Minimum gefunden: (x, y) = ({x_gd[ -1]}, {y_gd[ -1]})")
print(f"Anzahl Schritte: {len(res)}")
print(f"Fehlerschätzung: err = {err[ -1] :.2e}")
print("Bekannte Minima:")
print(np.array([x_min, y_min]).T)

plt.contourf(X, Y, Z, levels=200)
plt.colorbar(label="z=f(x,y)")
cs = plt.contour(X, Y, Z, levels=25, colors="w", alpha=0.4)
# plt.clabel(cs, fontsize=8)
plt.scatter(x_min, y_min, c="r")

plt.scatter(x_gd, y_gd, c="w", marker=".")

step = n // 13
plt.quiver(
    X[::step, ::step],
    Y[::step, ::step],
    grad_x[::step, ::step],
```



```

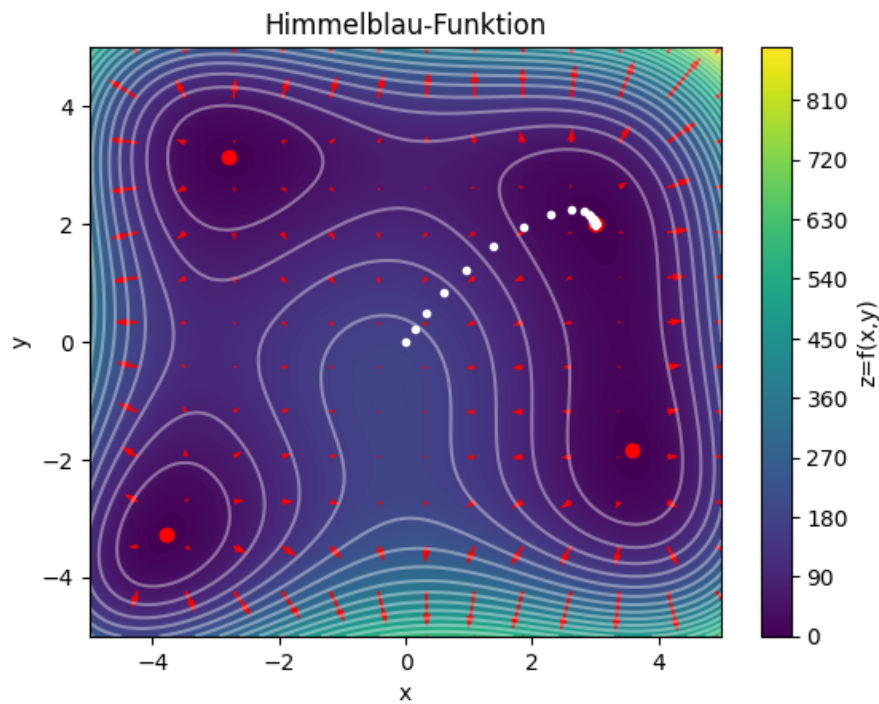
    grad_y[:,::step, ::step],
    angles="xy",
    color="r",
)

# plt.streamplot(X, Y, grad_x, grad_y, color="r", linewidth=0.75)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Himmelblau -Funktion")
plt.show()

Minimum gefunden: (x, y) = (2.9999999998484674, 2.0000000003545986)
Anzahl Schritte: 78
Fehlerschätzung: err = 9.99e -09
Bekannte Minima:
[[ 3.         2.         ]
 [ -3.77931  -3.283186]
 [ -2.805118  3.131312]
 [ 3.         2.         ]
 [ 3.584428  -1.848126]]

```



0.8 Integration

0.8.1 Numerische Integration

Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Hier lernst du die numerische Integration kennen. Ziel ist es, das Integral einer analytisch gegebenen Funktion numerisch zu berechnen. Dies wirst du rudimentär implementieren.

In der Vertiefung werden wir den Fall betrachten, dass nur Daten anstatt einer analytischen Funktion gegeben sind, und weitere Details diskutieren.

Grundsätzliches Vorgehen

Das **bestimmte Integral** ist aus dem Modul **Analysis & Algebra** (1. Semester) bekannt. Eine kurze Wiederholung: Das unbestimmte Integral kehrt die Ableitung um, d.h. die Integration dient der Rekonstruktion einer Funktion aus ihrer Ableitung bis auf eine Konstante C :

$$F'(x) = f(x) \quad \Leftrightarrow \quad \int f(x) dx = F(x) + C \quad (81)$$

Das bestimmte Integral $\int_a^b f(x) dx$ kann weiter als die gerichtete Fläche zwischen dem Graphen von $f(x)$ und der x -Achse im Bereich $x \in [a, b]$ interpretiert werden. Der **Hauptsatz der Differential- und Integralrechnung** lässt uns ein bestimmtes Integral berechnen, wenn wir eine Stammfunktion $F(x)$ des Integranden kennen, also

$$F'(x) = f(x) \quad \Rightarrow \quad \int_a^b f(x) dx = F(b) - F(a) \quad (82)$$

Die numerische Integration kommt dann zur Anwendung, wenn

- der Integrand zwar als Funktion gegeben ist, dessen Stammfunktion jedoch nicht analytisch bestimmt werden kann. Beispiele:
 - $f(x) = \frac{\sin(x)}{x}$
 - $f(x) = \exp(-x^2)$ (Gauss-Funktion)
- das Integral im Prinzip analytisch integrierbar ist, die analytische Auswertung jedoch unverhältnismässig viel Rechenaufwand erfordert.
- der Integrand nur punktweise gegeben ist, also als Datenreihe $(x_i, f(x_i))$, $i = 0, \dots, n-1$.

Trapezregel Eine einfache aber bereits recht gute Näherung des bestimmten Integrals erhält man mit der Trapezregel. Schaue dir eines der Erklärvideos an:

- “TRAPEZREGEL (Numerischer Integration)” - Mathe Peter
- “Näherungsformel zum Trapezverfahren, Numerische Integration” - Daniel Jung

Arbeitsauftrag

```
import numpy as np
import matplotlib.pyplot as plt
```

Aufgabe 1

a) Die Trapezregel Notiere zuerst das grundsätzliche Vorgehen für die Anwendung der Trapezregel zur numerischen Integration. Schreibe auch die Formel für die Trapezregel auf, wie sie im Video von Mathe Peter gegeben ist.

Lösung:

- Zerlege das Intervall $[a, b]$ in n gleich breite Intervalle.

- Berechne Funktionswerte an den Intervallgrenzen und verbinde diese mit Geraden, so dass $n - 1$ Trapeze entstehen.
- Berechne die Flächen der Trapeze.
- Die Summe aller Flächen ergibt eine Näherung des Integrals.
- Je grösser n desto genauer das Ergebnis.

$$T_n = \frac{h}{2} \left(f(a) + 2 \sum_{i=1}^{n-1} f(a + ih) + f(b) \right), \quad \text{wobei } h = \frac{b-a}{n} \quad (83)$$

b) Trapezregel implementieren Schreibe eine Funktion `trapezregel(f, a, b, n)` mit:

- Argumente:
 - **f**: die zu integrierende Funktion
 - **a**: die untere Integrationsgrenze
 - **b**: die obere Integrationsgrenze
 - **n**: die Anzahl Trapeze.
- Rückgabe:
 - Näherungswert des bestimmten Integrals nach der Trapezregel.

```
def trapezregel(func, a, b, n):
    h = (b - a) / n
    s = sum([func(a + i * h) for i in range(1, n)])
    return (h / 2) * (func(a) + 2 * s + func(b))
```

c) Implementation testen Berechne das folgende Integral mit deiner Funktion `trapezregel` und $n=4$ Trapezen

$$\int_0^1 e^{x^2} dx \quad (84)$$

Genau dieses Integral wird im Video vom MathePeter berechnet. Er findet das Resultat ≈ 1.4907 . Du auch?

```
def f(x):
    return np.exp(x**2)
```

```
trapezregel(f, 0, 1, 4)
```

```
np.float64(1.4906788616988553)
```

d) Darstellung Hier soll eine Darstellung der bei der Trapezregel berechneten Trapez-Flächen und der eigentlichen Integral-Fläche entstehen.

1. Stelle $f(x) = \exp(x^2)$ im Bereich $x \in [-0.25, 1.25]$ grafisch dar.
2. Benutze die Funktion `plt.fill_between`, um die Fläche zwischen $y = f(x)$ und der x -Achse im integrierten Bereich $x \in [0, 1]$ einzufärben.
3. Füge die 5 linear verbundenen Punkte hinzu, die bei der Berechnung der Trapezregel mit $n=4$ benutzt werden. (Dazu kannst du `plt.plot` und `plt.stem` kombinieren.)

```
a = 0
b = 1
n = 4
```

```

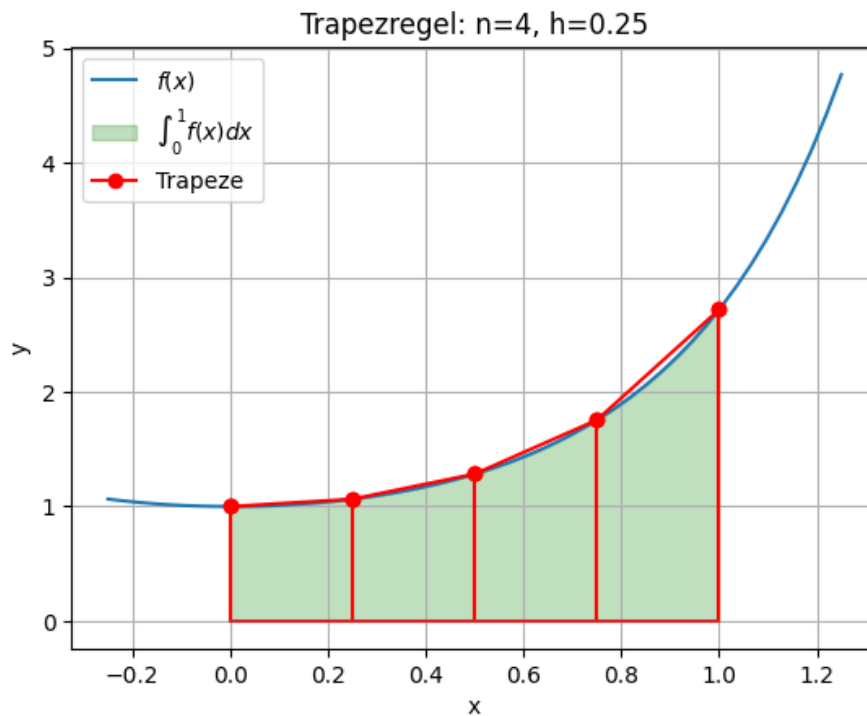
x = np.linspace(a - 0.25, b + 0.25)
y = f(x)

x_ab = np.linspace(a, b)
y_ab = f(x_ab)

x_trapez = np.linspace(a, b, n + 1)
y_trapez = f(x_trapez)

plt.plot(x, y, label="$f(x)$")
plt.fill_between(x_ab, y_ab, alpha=0.25, color="green", label=r"$\int_0^1 f(x)dx$")
plt.stem(x_trapez, y_trapez, "ro -")
plt.plot(x_trapez, y_trapez, "ro -", label="Trapeze")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.title(f"Trapezregel: n={4}, h={1/n}")
plt.show()

```



Aufgabe 2

a) Berechnung mit SciPy Das Modul `scipy.integrate` stellt Funktionen fürs Integrieren bereit. Wir benutzen `scipy.integrate.quad` für die Integration von Funktionen.

Berechne das Integral mit `scipy.integrate.quad`. Informiere dich über die Argumente und die Rückgabewerte in der Dokumentation.

```

from scipy.integrate import quad

quad(f, a, b)

(1.4626517459071817, 1.6238696453143372e -14)

```

b) Fehleranalyse Berechne den absoluten Fehler, also die Differenz zwischen der selbst implementierten Funktion `trapezregel` und `scipy.integrate.quad`. Der absolute Fehler soll positiv sein, wenn das Resultat von `trapezregel` grösser ist als das von SciPy.

- Berechne diesen Fehler in Abhängigkeit von n für $1 \leq n \leq 100$.
- Benutze `plt.loglog`, um die Fehler in einem doppelt logarithmischen Diagramm darzustellen. Alternativ kannst du auch `plt.plot` mit `plt.xscale("log")` und `plt.yscale("log")` benutzen.

```
n_range = np.arange(1, 100)
```

```
reference = quad(f, a, b)[0]
```

```
error = np.array([trapezregel(f, a, b, n) - reference for n in n_range])
```

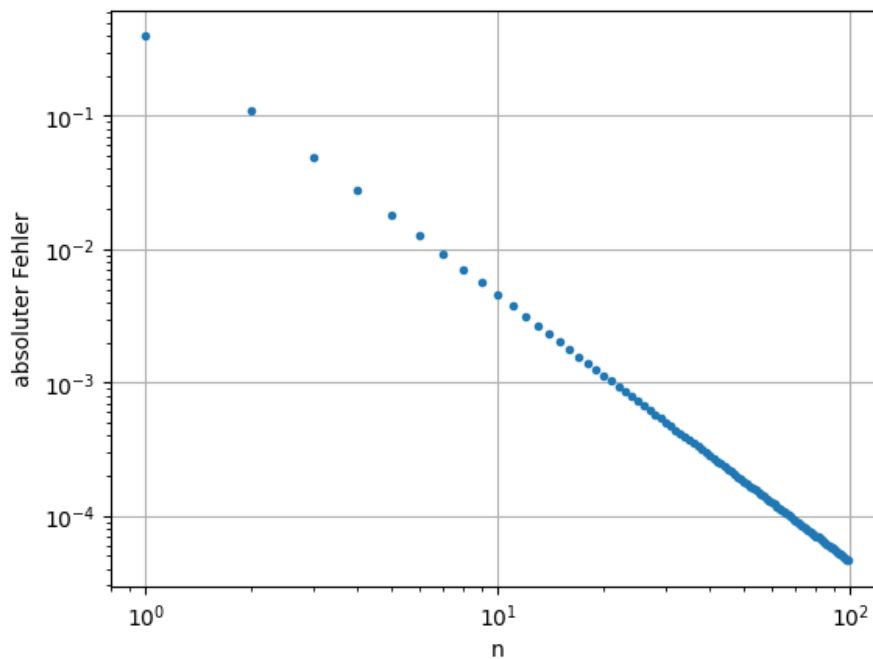
```
plt.loglog(n_range, error, ".")
```

```
plt.grid()
```

```
plt.xlabel("n")
```

```
plt.ylabel("absoluter Fehler")
```

```
plt.show()
```



0.8.2 Numerische Integration

Übungen - Lösungen

```
import numpy as np
import matplotlib.pyplot as plt

def trapz(func, a, b, n):
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = func(x)
    return (h / 2) * (y[0] + 2 * np.sum(y[1: -1]) + y[-1])

def trapz_data(x, y):
    return np.sum((x[1:] - x[:-1]) * (y[1:] + y[:-1])) / 2
```

Aufgabe 1 Die Funktion

$$f(x) = \cos\left(2\pi\left(x + x^2\right)\right) \quad (85)$$

deren Ableitung in der Einarbeitung zum Thema “Ableitung” bereits numerisch berechnet wurde, soll nun integriert werden:

$$S = \int_{-1}^1 f(x) dx \quad (86)$$

1. Berechne S mit Wolfram|Alpha.
2. Berechne S mit der passenden Funktion aus `scipy.integrate`.
3. Berechne S mit der passenden Variante der oben implementierten Trapezregel (`trapz()` oder `trapz_data()`) für unterschiedliche Anzahl Intervalle n . Bestimme n , so dass der absolute Fehler verglichen mit dem Resultat von `scipy.integrate` kleiner als 10^{-10} ist.

```
# 1. Wolfram|Alpha

# https://www.wolframalpha.com/input?i=int+cos%282pi%28x%2Bx%5E2%29%29+from+ -1+to+1
s1 = 0.467286

# 2. SciPy
from scipy.integrate import quad

def f(x):
    return np.cos(2 * np.pi * (x + x**2))

s2, s2_error = quad(f, -1, 1)
s2, s2_error
(0.46728607317886517, 6.878545121594632e -13)

# 3. Trapezregel

s3 = trapz(f, -1, 1, 100)
s3
```

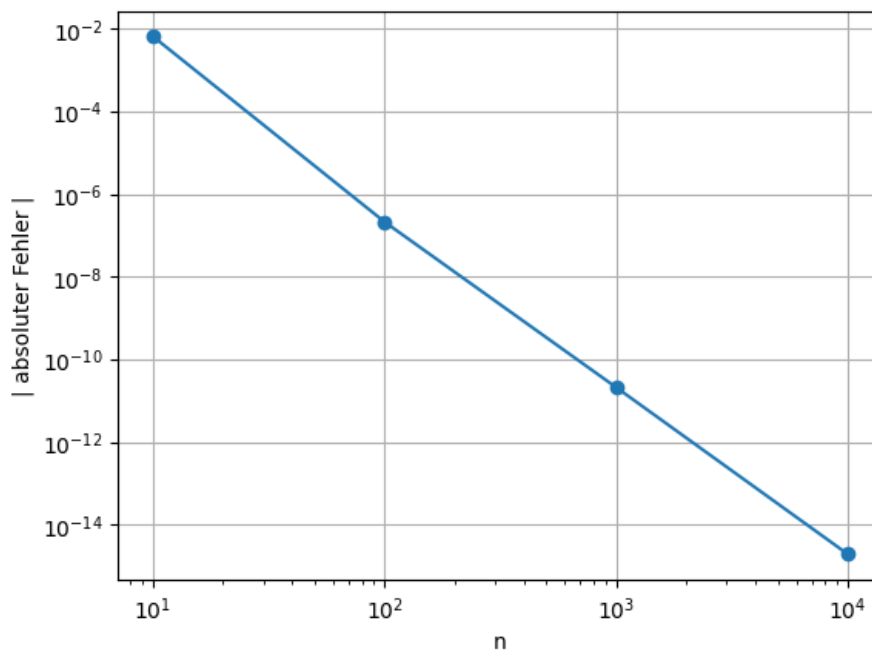
```

np.float64(0.467286285590903)

# Darstellung
ns = np.logspace(1, 4, 4, dtype=int)
s3s = np.array([trapz(f, -1, 1, n) for n in ns])
s3_errors = np.abs(s3s - s2)

plt.loglog(ns, s3_errors, "o -")
plt.xlabel("n")
plt.ylabel("| absoluter Fehler |")
plt.grid()
plt.show()

```



Also muss $n \approx 10^3$ gewählt werden.

Aufgabe 2: Wasserverbrauch

a) Daten einlesen, darstellen, verstehen

1. Lies die Daten in `wasserverbrauch.csv` ein.
2. Stelle den Wasserverbrauch in Abhängigkeit der Zeit dar.
3. Schreibe die Achsen mit den korrekten Variablen und deren Einheiten an.
4. Bestimme die Einheit des Integrals über den Wasserverbrauch. Also was ist die Bedeutung der Fläche unter dieser Kurve?

```

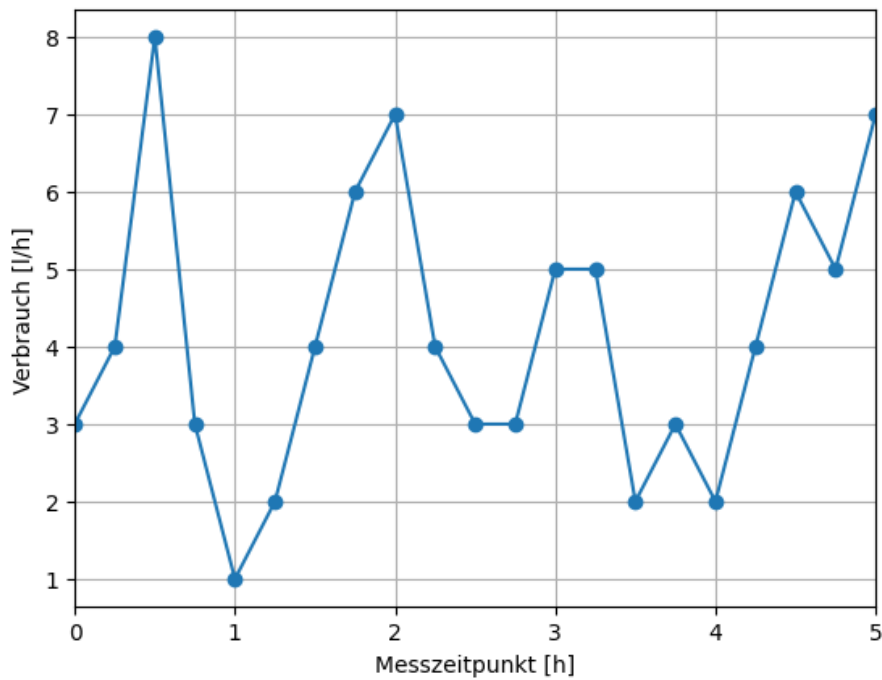
wasserverbrauch = np.loadtxt("wasserverbrauch.csv", delimiter=";", skiprows=1)
zeit, verbrauch = wasserverbrauch.T

```

```

plt.plot(zeit, verbrauch, "o -")
plt.xlim(zeit.min(), zeit.max())
plt.xlabel("Messzeitpunkt [h]")
plt.ylabel("Verbrauch [l/h]")
plt.grid()
plt.show()

```

Das Integral über den Verbrauch (in l/h) ist die Wassermenge in Litern, die im integrierten Zeitraum verbraucht wurde.

b) Daten integrieren Bestimme den Wasserverbrauch von $t = 1$ bis und mit $t = 4$.

Benutze dazu einmal die passende Trapezregel-Funktion (zuoberst bereits definiert) und einmal `scipy.integrate`.

```
a, b = 1, 4
```

```
bereich = (a <= zeit) & (zeit <= b)
```

```
trapez_data(zeit[bereich], verbrauch[bereich])
```

```
np.float64(11.375)
```

```
from scipy.integrate import trapezoid
```

```
trapezoid(verbrauch[bereich], zeit[bereich])
```

```
np.float64(11.375)
```

c) Integrierten Bereich darstellen Stelle die Fläche unter dem Graphen zwischen $t = 1$ und $t = 4$ mithilfe von `plt.fill_between` dar.

```
plt.plot(zeit, verbrauch, "o -")
```

```
plt.fill_between(zeit[bereich], verbrauch[bereich], alpha=0.25, color="green")
```

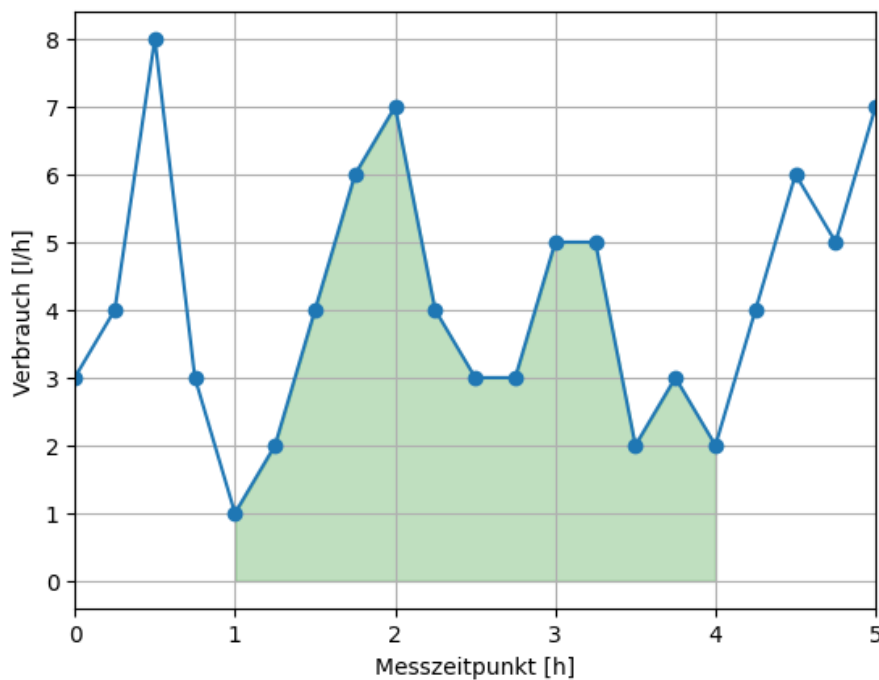
```
plt.xlim(zeit.min(), zeit.max())
```

```
plt.xlabel("Messzeitpunkt [h]")
```

```
plt.ylabel("Verbrauch [l/h]")
```

```
plt.grid()
```

```
plt.show()
```



Aufgabe 3: Jogging-Rekonstruktion Wir benutzen nochmals die Jogging-Daten, die im Übungsblatt zum Thema Ableitung bereits verwendet wurden. Hier stellen wir die Frage, ob die zurückgelegte Distanz aus der Geschwindigkeit rekonstruiert werden kann.

Die Geschwindigkeit $v(t)$ ist die Ableitung der zurückgelegten Distanz $x(t)$, also $x'(t) = v(t)$. Somit ist die zurückgelegte Distanz $x(t)$ das Integral der Geschwindigkeit $v(t)$ über die Zeit:

$$x(t) = \int_0^t v(t') dt' \quad (87)$$

Schritte:

1. Lies die ersten drei Spalten aus der Datei `jogging.csv` ein:
 - Zeit (in Sekunden)
 - Distanz (in Metern)
 - Pace (in Minuten pro Meile)
2. Filtere die ersten paar Datenpunkte heraus, wo die Pace = 0 ist.
3. Rechne die Pace in die Geschwindigkeit in Metern pro Sekunde um (1 Meile = 1609.44 Meter).
4. Integriere die Geschwindigkeit über die Zeit, um die zurückgelegte Distanz zu bestimmen.
5. Vergleiche das Resultat mit der Distanz, die ganz am Schluss gemessen wurde.

Da die Geschwindigkeit und die Distanz von der GPS-Uhr gemessen wurden, sind die Daten nicht perfekt. Daher wird die Distanz, die am Schluss gemessen wurde, nicht ganz mit der Distanz aus der aufintegrierten Geschwindigkeit übereinstimmen.

1. -3. Import, Bereinigung & Umrechnung

```
jogging = np.loadtxt("jogging.csv", delimiter=";", skiprows=1, usecols=(0, 1, 2))
time, distance, pace = jogging.T
```

```
idx = np.isfinite(pace) & (pace > 0)
```

```
t_data = time[idx]
```

```
x_data = distance[idx]
```

```
v_data = 1 / pace[idx] * 1609.344 / 60
```

```

dist_from_v = trapz_data(t_data, v_data)

print("rekonstruierte Distanz:\t", dist_from_v)
print("gemessene Distanz:\t", x_data[ -1])
print("absoluter Unterschied:\t", dist_from_v - x_data[ -1])
print("relativer Unterschied:\t", dist_from_v / x_data[ -1] - 1)

rekonstruierte Distanz:  6860.088525670482
gemessene Distanz:  5544.60009765625
absoluter Unterschied:  1315.4884280142323
relativer Unterschied:  0.23725578127271985

```

Aufgabe 4: Temperaturen in Davos Hier werden wir Daten zuerst interpolieren, um die Nullstellen bestimmen zu können. Dann wird zwischen den Nullstellen integriert.

a) Daten einlesen, darstellen, verstehen

- Lies die Daten in `temperatur_weihnachten.csv` ein. Diese enthalten den Temperaturverlauf vom 24. und 25. Dezember in Davos.
- Stelle die Temperatur in Abhängigkeit der Zeit dar mit den korrekten Achsenbeschriftungen inklusive Einheiten.

```

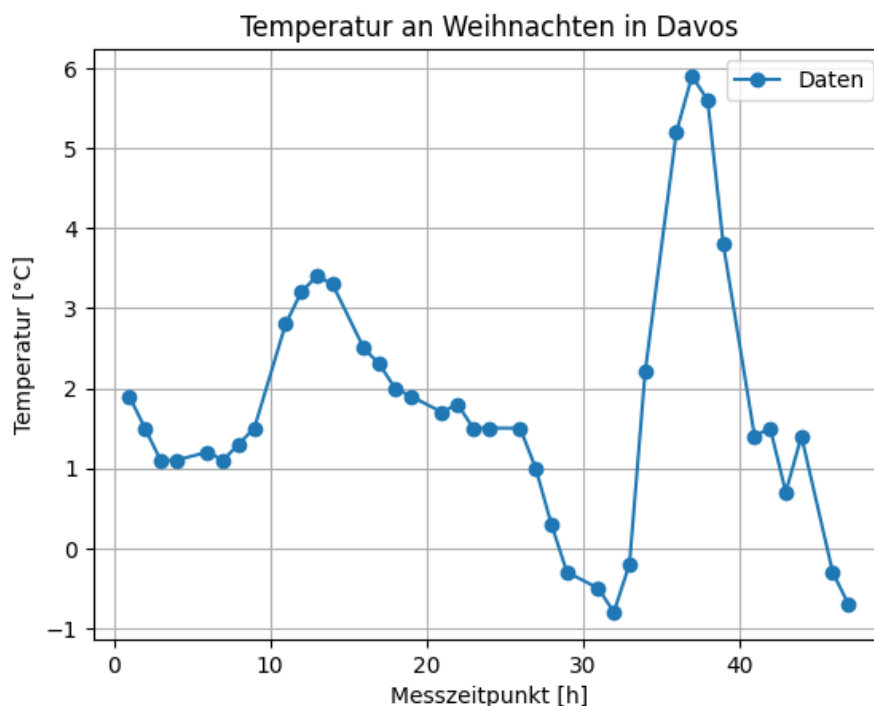
davos = np.loadtxt("temperatur_weihnachten.csv", delimiter=";", skiprows=1)
t_data, temp_data = davos.T

```

```

plt.plot(t_data, temp_data, "o -", label="Daten")
plt.xlabel("Messzeitpunkt [h]")
plt.ylabel("Temperatur [°C]")
plt.grid()
plt.legend()
plt.title("Temperatur an Weihnachten in Davos")
plt.show()

```



b) Interpolation

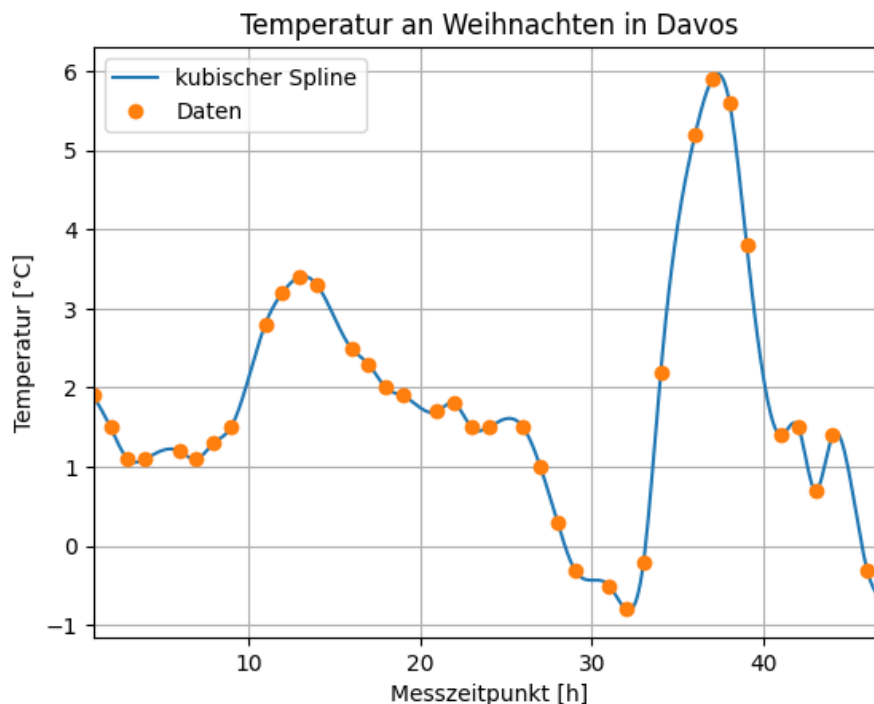
- Konstruiere eine kubische Spline-Interpolation der Temperatur mit natürlichen Randbedingungen.
- Stelle die Interpolationsfunktion mittels 500 Punkten im gleichen Zeitraum wie die Daten zusammen mit den Daten grafisch dar.

```
from scipy.interpolate import make_interp_spline

temp_spline = make_interp_spline(t_data, temp_data, bc_type="natural")

t = np.linspace(t_data.min(), t_data.max(), 500)
temp = temp_spline(t)

plt.plot(t, temp, label="kubischer Spline")
plt.plot(t_data, temp_data, "o", label="Daten")
plt.xlim(t_data.min(), t_data.max())
plt.xlabel("Messzeitpunkt [h]")
plt.ylabel("Temperatur [°C]")
plt.legend()
plt.grid()
plt.title("Temperatur an Weihnachten in Davos")
plt.show()
```



c) Nullstellen bestimmen Im Diagramm von b) ist ersichtlich, dass die Temperatur an 3 Stellen durch 0 geht.

- Finde alle drei Nullstellen.
- Stelle sie im Diagramm als rote Punkte dar.

Tipp: Folgende Funktionen kannst du für diese Aufgabe benutzen:

- `scipy.optimize.newton`
- `scipy.optimize.bisect`

- `scipy.optimize.root_scalar`

```
# Variante mit Newton
from scipy.optimize import newton

low = 25
high = t.max()
n = 1000

roots = np.unique(
    [
        newton(
            temp_spline,
            x0=x0,
            fprime=temp_spline.derivative(),
            fprime2=temp_spline.derivative().derivative(),
        )
        for x0 in np.linspace(25, t.max(), 1000)
    ]
)
roots
```

 RuntimeError Traceback (most recent call last)

Cell In[15], line 10

```
5 high = t.max()
6 n = 1000
8 roots = np.unique(
9     [
- - -> 10         newton(
11             temp_spline,
12             x0=x0,
13             fprime=temp_spline.derivative(),
14             fprime2=temp_spline.derivative().derivative(),
15         )
16         for x0 in np.linspace(25, t.max(), 1000)
17     ]
18 )
19 roots
```

File ~/Repos/NGDS -DZP/.venv/lib/python3.14/site -packages/scipy/optimize/_zeros_py.py:392, in

390 if disp:

391 msg = f"Failed to converge after {itr + 1} iterations, value is {p}."

- -> 392 raise RuntimeError(msg)

394 return _results_select(full_output, (p, funcalls, itr + 1, _ECONVERR), method)

RuntimeError: Failed to converge after 50 iterations, value is 28.430820615192008.

```
# Variante mit Bisektionsverfahren
from scipy.optimize import bisect
from itertools import pairwise
```

```
low = 25
high = t.max()
```

```

n = 1000

roots = np.unique(
    [
        bisect(temp_spline, a, b)
        for a, b in pairwise(np.linspace(low, high, n))
        if temp_spline(a) * temp_spline(b) < 0
    ]
)
roots

plt.plot(t, temp, label="kubischer Spline")
plt.plot(roots, np.zeros_like(roots), "or", label="Nullstellen")
plt.xlim(t.min(), t.max())
plt.xlabel("Messzeitpunkt [h]")
plt.ylabel("Temperatur [°C]")
plt.title("Interpolierte Temperaturen mit Nullstellen")
plt.legend()
plt.grid()
plt.show()

```

d) Mittlere Temperatur bestimmen Berechnen wir die durchschnittliche Temperatur zwischen der 1. und 3. Nullstelle.

Der Mittelwert einer Funktion $f(x)$ zwischen a und b ist definiert als

$$\frac{1}{b-a} \int_a^b f(x) dx \quad (88)$$

- Schreibe eine Funktion `mean_func(func, a, b)`, welche die den Mittelwert einer Funktion `func` zwischen zwei werten `a` und `b` mithilfe von SciPy berechnet.
- Benutze diese, um die mittlere Temperatur zwischen der 1. und 3. Nullstelle zu berechnen.

```
from scipy.integrate import quad
```

```

def mean_func(func, a, b):
    integral, _ = quad(func, a, b)
    return integral / (b - a)

```

```
r1, r3 = roots[0], roots[-1]
```

```

mean_temp = mean_func(temp_spline, r1, r3)
mean_temp

```

e) Darstellung

- Stelle das Integral in dem Diagramm aus der Aufgabe c) mithilfe von `plt.fill_between` dar.
- Plote dazu den Mittelwert zwischen der 1. und 3. Nullstelle als konstante Funktion.

```
ind = (r1 <= t) & (t <= r3)
```

```

plt.plot(t, temp, label="Temperaturverlauf")
plt.fill_between(t[ind], temp[ind], alpha=0.3)
plt.plot(
    roots, np.full_like(roots, mean_temp), "m", label=f"Mittelwert: {mean_temp:.2f}°C"
)

```

```
)  
plt.fill_between(roots, np.full_like(roots, mean_temp), color="m", alpha=0.2)  
plt.plot(roots, np.zeros_like(roots), "or", label="Nullstellen")  
plt.xlim(t.min(), t.max())  
plt.xlabel("Messzeitpunkt [h]")  
plt.ylabel("Temperatur [°C]")  
plt.title("Interpolierte Temperaturen mit Nullstellen und Mittelwert")  
plt.legend()  
plt.grid()  
plt.show()
```

0.8.3 Numerische Integration

Vertiefung

```
import numpy as np
import matplotlib.pyplot as plt
```

Übersicht

1. Trapezformel vektorisiert implementieren
2. Fehlerabschätzung
3. Trapezformel für Daten mit ungleichmässigen Abständen
4. Integration mit SciPy

1. Trapezformel vektorisiert implementieren Vorgehen beim Integrieren mit der Trapezregel:

- Zerlege das Intervall $[a, b]$ in n gleich breite Intervalle.
- Berechne Funktionswerte an den Intervallgrenzen und verbinde diese mit Geraden, so dass n Trapeze entstehen.
- Berechne die Flächen der Trapeze.
- Die Summe aller Flächen ergibt eine Näherung des Integrals.
- Je grösser n desto genauer das Ergebnis.

$$T_n = \frac{h}{2} \left(f(a) + 2 \sum_{i=1}^{n-1} f(a + ih) + f(b) \right), \quad \text{wobei} \quad h = \frac{b-a}{n} \quad (89)$$

```
def trapez_comprehension(func, a, b, n):
    h = (b - a) / n
    s = sum([func(a + i * h) for i in range(1, n)])
    return (h / 2) * (func(a) + 2 * s + func(b))

def trapez_vectorized(func, a, b, n):
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = func(x)
    return (h / 2) * (y[0] + 2 * np.sum(y[1: -1]) + y[-1])
```

Test mit der Funktion

$$f(x) = \sin(x) \quad (90)$$

Analytisches Resultat:

$$\int_0^\pi \sin(x) dx = \left[-\cos(x) \right]_0^\pi = -\cos(\pi) + \cos(0) = 2 \quad (91)$$

Beide Implementationen vergleichen:

```
n = 1000
T1 = trapez_comprehension(np.sin, 0, np.pi, n)
T2 = trapez_vectorized(np.sin, 0, np.pi, n)
T1, T2, T1 - T2

(np.float64(1.9999983550656624),
 np.float64(1.9999983550656628),
 np.float64(-4.440892098500626e-16))
```


Laufzeit vergleichen mit %timeit:

```
%timeit trapz_comprehension(np.sin, 0, np.pi, n)
%timeit trapz_vectorized(np.sin, 0, np.pi, n)
```

82.1 μs $\pm$ 548 ns per loop (mean $\pm$ std. dev. of 7 runs, 10,000 loops each)

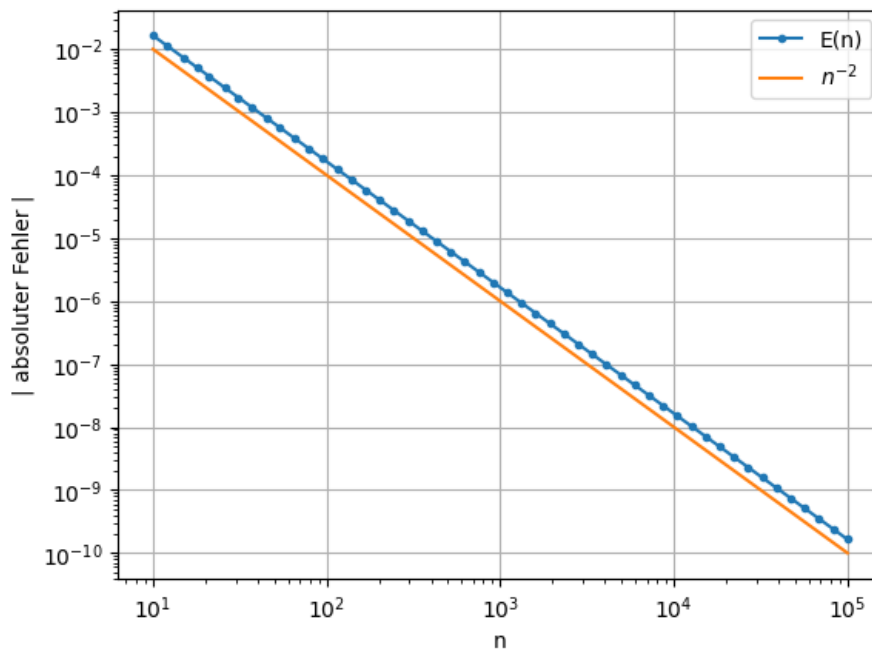
6.58 μs $\pm$ 36.9 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

2. Fehlerabschätzung

Explorativ Abhängigkeit des Fehlers von n . Beispiel von oben.

```
ns = np.logspace(1, 5).round().astype(int)
# ns = np.array([10, 100, 1000, 10000, 100000])
errors = np.array([abs(trapz_vectorized(np.sin, 0, np.pi, n) - 2) for n in ns])

plt.loglog(ns, errors, ". -", label="E(n)")
plt.loglog(ns, ns.astype(float) ** (-2), " -", label="$n^{-2}$")
plt.xlabel("n")
plt.ylabel("| absoluter Fehler |")
plt.grid()
plt.legend()
plt.show()
```



Der Fehler $E(n)$ ist eine Gerade im Log-Log-Plot. Das heisst, es handelt sich um eine Potenzfunktion

$$E(n) = C \cdot n^p \quad (92)$$

und p ist die Steigung der Geraden im Log-Log-Plot. Berechnen wir also mal die Steigung zwischen je zwei Punkten und mitteln diese:

```
np.mean(np.diff(np.log10(errors)) / np.diff(np.log10(ns)))
np.float64( -2.000176435455864)
```

Trapezregel Abschätzung Eine analytische Fehlerabschätzung ist in Wikipedia zu finden. Der Fehler

$$E_n = T_n - \int_a^b (x) dx \quad (93)$$

ist beschränkt durch

$$|E_n| \leq \frac{(b-a)}{12} \cdot M_2 \cdot h^2 = \frac{(b-a)^3}{12} \cdot M_2 \cdot n^{-2} \quad (94)$$

wobei

$$M_2 = \max_{a \leq x \leq b} |f''(x)| \quad (95)$$

Das heisst, der Fehler ist proportional zu h^2 bzw. n^{-2} , wie wir oben bereits gesehen haben.

Zudem ist der Fehler auch proportional zum Maximum der zweiten Ableitung der Funktion, M_2 . Das heisst, je stärker die Funktion im Intervall $[a, b]$ schwankt (grössere Krümmung), desto grösser ist der Fehler durch die Näherung mit der Trapezregel.

Folgende Videos gehen auf die Fehlerabschätzung ein (optional):

- Fehler der Trapezregel für numerische Integration - Jörn Loviscach
- Analysis Numerische Integration: Trapezformel Abschätzung des Fehlers (Trapezregel) - Henning Dierks

3. Trapezformel für Daten mit ungleichmässigen Abständen

Beispiel-Daten erzeugen Wir generieren ein paar Datenpunkte, die nicht gleichmässig verteilt sind. Die x -Werte werden zufällig gleichverteilt über das Intervall $[0, 3]$ generiert. Die y -Werte sind die Funktionswerte der Funktion

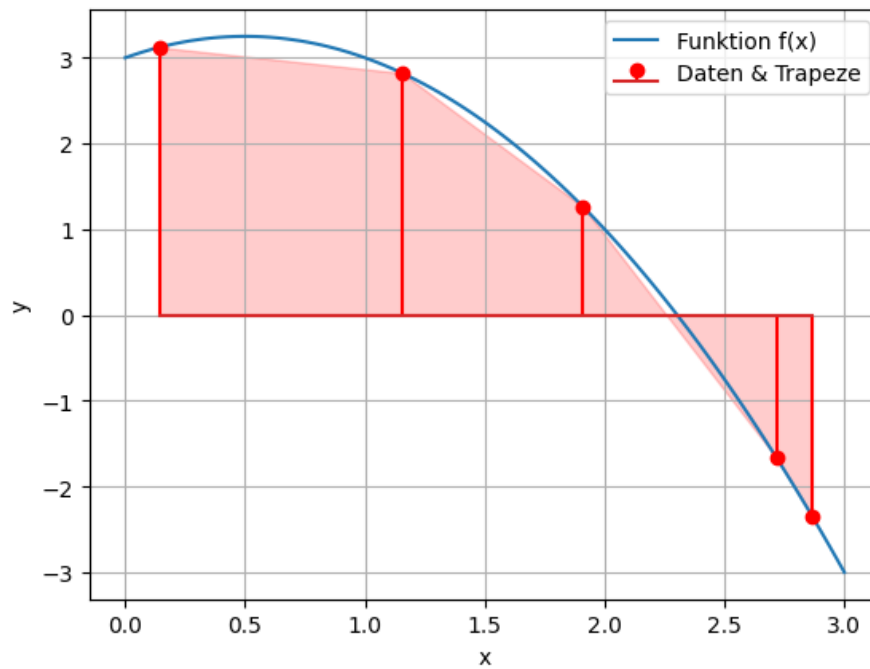
$$f(x) = 1 - (x + 1)(x - 2) \quad (96)$$

```
def f(x):
    return 1 - (x + 1) * (x - 2)

x = np.linspace(0, 3, 100)
y = f(x)

n_samples = 5
rng = np.random.default_rng(123456)
x_data = np.sort(rng.uniform(low=x.min(), high=x.max(), size=n_samples))
y_data = f(x_data)

plt.plot(x, y, label="Funktion f(x)")
plt.stem(x_data, y_data, "r", label="Daten & Trapeze")
plt.fill_between(x_data, y_data, 0, alpha=0.2, color="r")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```



Trapezformel für Daten Die Trapezformel für n Datenpunkte mit (x_i, y_i) mit $i = 0, \dots, n-1$ ungleichmässigen Abständen lautet:

$$T_n = \frac{1}{2} \sum_{k=1}^{n-1} (x_k - x_{k-1}) (y_k + y_{k-1}) \quad (97)$$

Hier ist

- $\Delta x_k = x_k - x_{k-1}$ die Breite des k -ten Trapezes und
- $\frac{1}{2} (y_k + y_{k-1})$ die mittlere Höhe des k -ten Trapezes.

Die Formel wird hier erklärt:

- Analysis Numerische Integration: Trapezformel #13-55 - Henning Dierks

Implementation als Comprehension

```
def trapez_data_comprehension(x, y):
    terms = [(x[k] - x[k - 1]) * (y[k] + y[k - 1]) for k in range(1, len(x))]
    return sum(terms) / 2
```

```
trapez_data_comprehension(x_data, y_data)
```

```
np.float64(4.089099472806035)
```

Vektorisierte Implementation

```
def trapez_data_vectorized(x, y):
    return np.sum((x[1:] - x[:-1]) * (y[1:] + y[:-1])) / 2
```

```
trapez_data_vectorized(x_data, y_data)
```

```
np.float64(4.089099472806035)
```

Vergleich mit analytischen Resultat Das analytische Resultat ist:

$$\begin{aligned}\int_0^3 1 - (x+1)(x-2) dx &= \int_0^3 -x^2 + x + 3 dx \\ &= \left[-\frac{1}{3}x^3 + \frac{1}{2}x^2 + 3x \right]_0^3 \\ &= -\frac{27}{3} + \frac{9}{2} + 3 \cdot 3 \\ &= 4.5\end{aligned}$$

Siehe auch Wolfram|Alpha: `int 1-(x+1)(x-2) from 0 to 3`.

Vergleich der Performance

```
%timeit trapez_data_comprehension(x_data, y_data)
%timeit trapez_data_vectorized(x_data, y_data)
```

885 ns $\pm$ 14.8 ns per loop (mean $\pm$ std. dev. of 7 runs, 1,000,000 loops each)
 1.68 μ s $\pm$ 13.4 ns per loop (mean $\pm$ std. dev. of 7 runs, 1,000,000 loops each)

4. Integration mit SciPy

- Übersichtsseite: SciPy-Integration
- Tutorial: SciPy-Integration
- Integration von Funktionen mit `scipy.integrate.quad`
- Integration von Daten mit `scipy.integrate.trapezoid`

```
from scipy.integrate import quad, trapezoid
```

Integration von Funktionen

```
quad(np.sin, 0, np.pi)

(2.0, 2.220446049250313e -14)
```

`n = 10_000_000`

```
trapez_vectorized(np.sin, 0, np.pi, n) - quad(np.sin, 0, np.pi)[0]

np.float64( -1.6431300764452317e -14)
```

```
%timeit trapez_vectorized(np.sin, 0, np.pi, n)
%timeit quad(np.sin, 0, np.pi)[0]
```

28.1 ms $\pm$ 125 μ s per loop (mean $\pm$ std. dev. of 7 runs, 10 loops each)
 2.17 μ s $\pm$ 6.75 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

Dem Integranden können auch zusätzliche Argumente übergeben werden. Beispiel:

```
def g(t, omega):
    return np.sin(omega * t)

[
    (omega, quad(g, 0, np.pi, args=(omega,))[0])
    for omega in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
]
```

```
[(1, 2.0),
 (2, 1.1107507412563885e -16),
 (3, 0.6666666666666665),
 (4, -6.408156246354975e -17),
 (5, 0.3999999999999999),
 (6, 6.484247472472079e -18),
 (7, 0.28571428571428553),
 (8, -4.580085487159364e -16),
 (9, 0.22222222222222238),
 (10, -7.281558691778114e -16)]
```

Auch uneigentliche Integrale (mit unendlichem Intervall) können näherungsweise berechnet werden:

$$\begin{aligned}\int_0^\infty e^{-x} dx &= \lim_{b \rightarrow \infty} \int_0^b e^{-x} dx \\ &= \lim_{b \rightarrow \infty} \left[-e^{-x} \right]_0^b \\ &= \lim_{b \rightarrow \infty} (1 - e^{-b}) \\ &= 1\end{aligned}$$

```
quad(lambda x: np.exp( -x), 0, np.inf)
```

```
(1.0, 5.842605965544164e -11)
```

Integration über Daten

```
trapezoid(y_data, x_data)
```

```
np.float64(4.089099472806035)
```

```
trapz_data_vectorized(x_data, y_data) - trapezoid(y_data, x_data)
```

```
np.float64(0.0)
```

```
%timeit trapezoid(y_data, x_data)
```

```
%timeit trapz_data_vectorized(x_data, y_data)
```

```
3.49 \mus \pm 30.2 ns per loop (mean \pm std. dev. of 7 runs, 100,000 loops each)
```

```
1.72 \mus \pm 9.57 ns per loop (mean \pm std. dev. of 7 runs, 1,000,000 loops each)
```

0.9 Anwendung Integration

0.9.1 Anwendungen der Integration

Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Hier lernst du ein paar Anwendungen der Integration im Kontext der Data Sciences kennen. Der Fokus liegt dabei auf dem gewichteten Mittelwert und dem gleitenden Mittelwert (Faltung).

Die Lernziele dieses Abschnitts sind:

- Mittelwert einer Funktion in einem Bereich berechnen.
- Gewichteter Mittelwert einer Funktion für gegebene Gewichtungsfunktion berechnen.
- Definition des Skalarprodukts zweier Funktionen auf einem geschlossenen Intervall angeben/erkennen und für einfache Funktionen berechnen.
- Gleitendes, gewichtetes Mittel einer Funktion berechnen.
- Definition der Faltung zweier Funktionen angeben.
- Gleitendes, gewichtetes Mittel einer Funktion als Glättung und Faltung erklären und berechnen.

In diesem Kapitel stellen wir uns als Anwendungsfall eine zeitabhängige skalare Funktion $f(t)$ vor, deren Eigenschaften wir untersuchen. Dabei könnte es sich z.B. um ein gemessenes Signal handeln:

- Temperatur und Druck in einem Bioreaktor
- Auslastung einer Blutanalysestation oder Länge der Warteschlange davor
- Feinstaubbelastung und Lautstärke entlang einer Strasse
- Blutdruck und Puls einer Patient:in

```
import numpy as np
import matplotlib.pyplot as plt
```

Arithmetisches Mittel einer Funktion

Input Für eine Messreihe $\vec{y} = (y_0, \dots, y_{n-1})$ mit n Messwerten y_i gleichmässig über die Zeit verteilt (also $t_i = t_0 + ih$) ist das arithmetische Mittel

$$\bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i \quad (98)$$

eine Schätzung für den Mittelwert der Funktion $y = f(t)$, die der Messreihe zugrunde liegt.

Analog dazu ist das arithmetische Mittel dieser Funktion $f : D \rightarrow \mathbb{R}$ in einem abgeschlossenen Intervall $[a, b] \subseteq D$ definiert als

$$\bar{f} = \frac{1}{b-a} \int_a^b f(t) dt \quad (99)$$

Vergleiche dazu Aufgabe 4.d) aus dem Übungsblatt 07 Integration.

Arbeitsauftrag Falls nötig, schaue dir eines der folgenden Videos an

- Mittelwert einer Funktion | Integralrechnung - Quatematik
- Integral & Mittelwert von Funktionen (Herleitung) - Simpleclub (ab 1:50)

und beantworte dann die folgenden Fragen.

1. Von Hand berechnen Berechne den Mittelwert $\bar{f}$ der Funktion $f(t) = \sin(t)$ im Bereich $[0, 4]$ von Hand.

Lösung

$$\begin{aligned}
 \bar{f} &= \frac{1}{4-0} \int_0^4 \sin(t) \, dt \\
 &= \frac{1}{4-0} \left[-\cos(t) \right]_0^4 \\
 &= \frac{1}{4} (-\cos(4) + \cos(0)) \\
 &= \frac{1}{4} (1 - \cos(4)) \\
 &\approx 0.41341
 \end{aligned}$$

2. Mit SciPy Berechne den Mittelwert $\bar{f}$ mit SciPy.

```
# 2. Berechnung mit SciPy
from scipy.integrate import quad
```

```
f = np.sin
a, b = 0, 4
fbar = quad(f, a, b)[0] / (b - a)
fbar
```

```
0.41341090521590296
```

3. Darstellung der Funktion und des Mittelwerts Stelle die Funktion $f(t)$ und die konstante Funktion $y = \bar{f}$ in diesem Bereich mit Matplotlib dar. Färbe die bei der Mittelwertberechnung integrierte Fläche zwischen $f(t)$ und der t -Achse mithilfe von `plt.fill_between` ein.

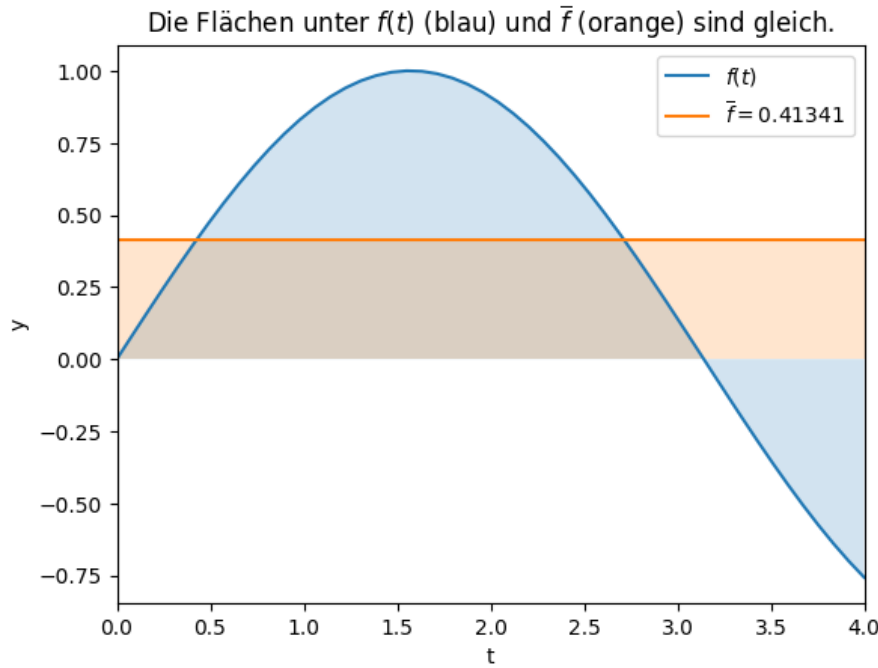
```
# 3. Darstellung
```

```
t = np.linspace(a, b)
y = f(t)
```

```
plt.fill_between(t, y, 0, alpha=0.2)
plt.fill_between(t, fbar, 0, alpha=0.2)
```

```
plt.plot(t, y, label="$f(t)$")
plt.plot(t, np.full_like(t, fbar), label=f"$\\overline{{f}}$={fbar:.5f}$")
```

```
plt.xlim(a, b)
plt.xlabel("t")
plt.ylabel("y")
plt.legend()
plt.title(r"Die Flächen unter $f(t)$ (blau) und $\overline{f}$ (orange) sind gleich.")
plt.show()
```

Gewichtetes arithmetisches Mittel einer Funktion

Input Für eine Messreihe $\vec{y} = (y_0, \dots, y_{n-1})$ mit n Messwerten y_i ist das **gewichtete arithmetische Mittel**

$$\bar{y} = \sum_{i=0}^{n-1} w_i y_i \quad (100)$$

wobei die Gewichte w_i normalerweise normiert werden, so dass $\sum_{i=0}^{n-1} w_i = 1$. Sind sie dies noch nicht, so normiert man die Gewichte, indem man sie durch die Summe aller Gewichte teilt:

$$w_i \rightarrow \frac{w_i}{\sum_{i=0}^{n-1} w_i} \quad (101)$$

Beispiel: Zwei Prüfungen während des Semesters zählen je 20% der Modulnote und die abgesetzte Prüfung zählt 60%. Die Gewichtung der Prüfungsnoten y_i sind somit $w_0 = 20$, $w_1 = 20$, $w_2 = 60$. Die Normierung der Gewichte ergibt $\sum_{i=0}^{n-1} w_i = 100$. Die normierten Gewichte sind also $w_0 = 0.2$, $w_1 = 0.2$, $w_2 = 0.6$.

Beobachtung: Das gleichgewichtete (normale) arithmetische Mittel erhält man mit $w_i = \frac{1}{n}$ für $i = 0, \dots, n-1$.

Für die kontinuierliche Funktion $f(x)$ kann analog dazu das gewichtete arithmetische Mittel im Intervall $[a, b]$ definiert werden:

$$\bar{f} = \int_a^b w(t) f(t) dt \quad (102)$$

Die Gewichtungsfunktion (*engl.* weight function) $w(t)$ ist auch hier meist normiert, so dass

$$\int_a^b w(t) dt = 1 \quad (103)$$

gilt. Ist die Gewichtung noch nicht normiert, kann sie wie folgt normiert werden:

$$w(t) \rightarrow \frac{w(t)}{\int_a^b w(t) dt} \quad (104)$$

Die Gewichtungsfunktion kann je nach Anwendung etwas anders interpretiert werden, siehe Vertiefung.

Arbeitsauftrag

1. Wir benutzen die Gewichtungsfunktion $w(t) = t$, bei der spätere Messungen höher gewichtet werden. Bestimme die normierte Variante dieser Gewichtungsfunktion im Bereich $[0, 4]$ (von Hand).
2. Schreibe das Integral für das gewichtete Mittel $\bar{f}$ der Funktion $f(t) = \sin(t)$ im Bereich $[0, 4]$ mit der unter 1. bestimmten Gewichtungsfunktion $w(t)$ auf.
3. Berechne dieses gewichtete Mittel numerisch mit SciPy.
4. Stelle die Funktion $f(t)$, die normierte Gewichtungsfunktion $w(t)$, das Produkt der beiden Funktionen $w(t)f(t)$ und die Konstante Funktion $y = \bar{f}$ im Bereich $[0, 4]$ mit Matplotlib dar. Färbe die Fläche, die beim gewichteten Mittel berechnet ein.

Lösung

1. Normierte Gewichtungsfunktion:

$$\int_0^4 t \, dt = \left[\frac{1}{2} t^2 \right]_0^4 = \frac{1}{2} (4^2 - 0^2) = 8 \quad (105)$$

$$\Rightarrow w(t) = \frac{1}{8} t \quad (106)$$

2. Gewichtetes Mittel:

$$\bar{f} = \int_0^4 w(t)f(t) \, dt = \int_0^4 \frac{1}{8} t \cdot \sin(t) \, dt \quad (107)$$

```
# 3. Integration mit SciPy
f = np.sin
w = lambda t: t / 8
wf = lambda t: w(t) * f(t)
```

```
a, b = 0, 4
fbar, _ = quad(wf, a, b)
fbar
```

```
0.2322214985183149
```

```
# 4. Darstellung
t = np.linspace(a, b)
y_f = f(t)
y_w = w(t)
y_wf = wf(t)
```

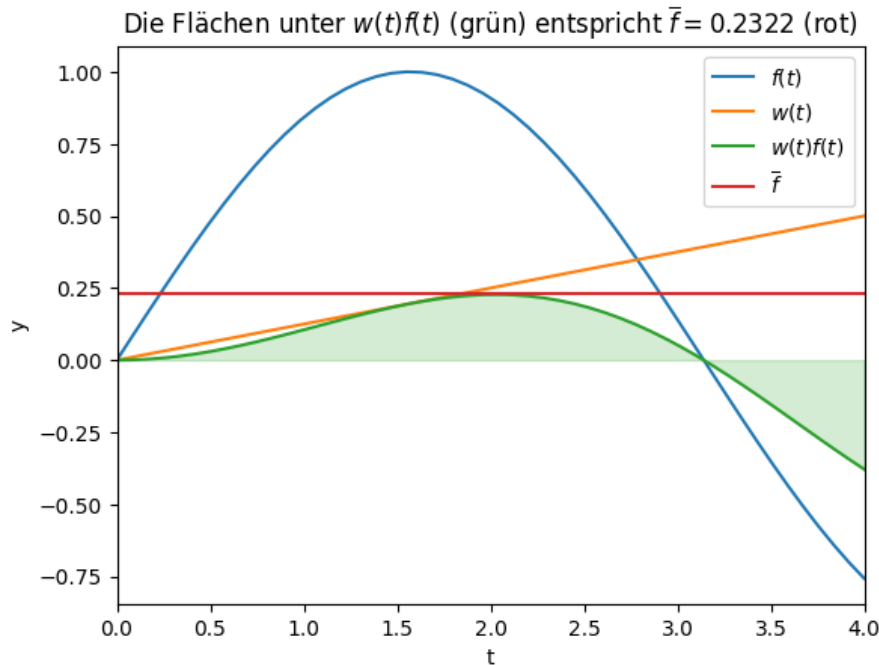
```
plt.plot(t, y_f, label="$f(t)$")
plt.plot(t, y_w, label="$w(t)$")
```

```
p = plt.plot(t, y_wf, label="$w(t)f(t)$")
plt.fill_between(t, y_wf, color=p[0].get_color(), alpha=0.2)
```

```
plt.plot(t, np.full_like(t, fbar), label=r"$\overline{f}$")
```

```
plt.xlim(a, b)
plt.xlabel("t")
```

```
plt.ylabel("y")
plt.legend()
plt.title(
    f"Die Flächen unter  $w(t)f(t)$  (grün) entspricht  $\overline{f} = \bar{f}$  (rot)"
)
plt.show()
```

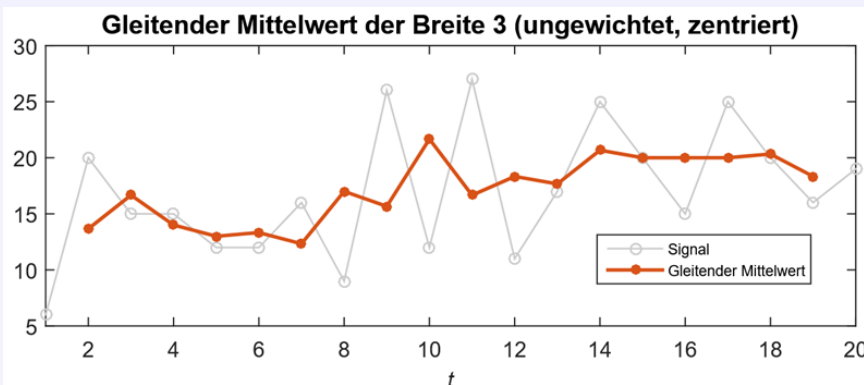


Gleitender Mittelwert

Input Wir beginnen wieder mit einer Messreihe $\vec{y} = (y_0, \dots, y_{n-1})$ mit n Messwerten y_i gleichmässig über die Zeit verteilt ($t_i = t_0 + ih$). Um die Messwerte zu glätten, berechnet man häufig den gleitenden Mittelwert über die letzten m Messwerte (*engl.* simple moving average, SMA). Aus Sicht eines Zeitpunkts t_i mit $i \geq m$ heisst das, man mittelt über diesen und die $m-1$ vorhergehenden Messwerte:

$$\text{SMA}_i^{(m)} = \frac{1}{m} (y_i + y_{i-1} + \dots + y_{i-(m-1)}) = \frac{1}{m} \sum_{k=0}^{m-1} y_{i-k} \quad \text{für } m-1 \leq i < n \quad (108)$$

Die folgende Darstellung zeigt die Messreihe $\vec{y}$ und die Folge der gleitenden Mittelwerte $\text{SMA}^{(3)}$. (Vorsicht, hier wurden die Werte $\text{SMA}_i^{(3)}$ jeweils auf die Mitte des gemittelten Tripels zentriert, was nicht ganz der obigen Definition entspricht).



Gut informiert (2012, Dezember 4). *ittelwertSignalvergleich3* [SVG]. Wikimedia Commons (<https://commons.wikimedia.org/wiki/File:MittelwertSignalvergleich3.svg>). Gemeinfrei.

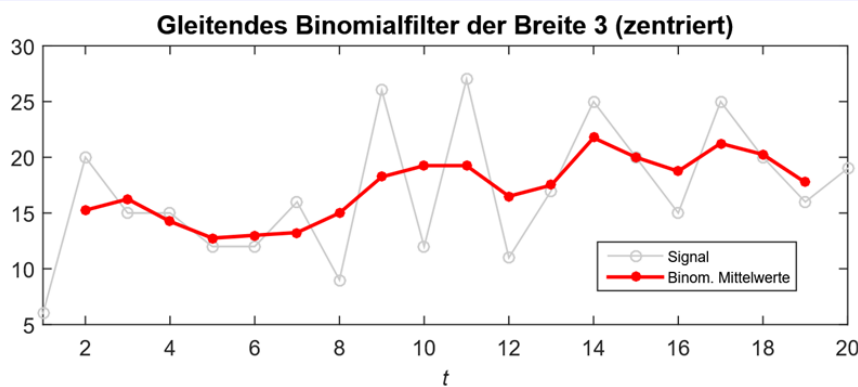
Nun können wir auch hier die zu mittelnden Messwerte mit Gewichten w_k versehen, so dass sich der gewichtete gleitende Mittelwert (*engl.* weighted moving average, WMA) ergibt:

$$\text{WMA}_i^{(m)} = \sum_{k=0}^{m-1} w_k y_{i-k} \quad (109)$$

Im folgenden Beispiel mit $m = 3$ sind die Gewichte “Dreiecksförmig” (Binomialfilter) gewählt:

$$\vec{w} = \frac{1}{4} (1, 2, 1) \quad \Rightarrow \quad \text{WMA}_i^{(3)} = \frac{1}{4} (y_i + 2y_{i-1} + y_{i-2}) \quad (110)$$

Beobachtung: Mit dieser Wahl der Gewichte ist die Überkompensation bei starken Schwankungen (hier bei $8 < t < 12$) wesentlich weniger ausgeprägt.



Gut informiert (2012, Dezember 4). *ittelwertSignalvergleich3* [SVG]. Wikimedia Commons (<https://commons.wikimedia.org/wiki/File:MittelwertSignalvergleich3.svg>). Gemeinfrei.

Arbeitsauftrag Gegeben ist die Messreihe

$$\vec{y} = (1, 2, 1, 3, 5, 4, 6, 3) \quad (111)$$

mit $n = 8$ Messwerten.

1. Berechne $\text{SMA}_i^{(3)}$ für $m - 1 \leq i < n$ von Hand.
2. Berechne $\text{SMA}_i^{(3)}$ für $m - 1 \leq i < n$ mit einer List-Comprehension.
3. Stelle die Daten und die gleitenden Mittelwerte mit Matplotlib dar.

Lösung

1. Von Hand:

$$\begin{aligned} \text{SMA}^{(3)} &= \left(\frac{1+2+1}{3}, \frac{2+1+3}{3}, \frac{1+3+5}{3}, \frac{3+5+4}{3}, \frac{5+4+6}{3}, \frac{4+6+3}{3} \right) \\ &= (1.\bar{3}, 2, 3, 4, 5, 4.\bar{3}) \end{aligned}$$

```
# 2. mit List -Comprehension
```

```
y = np.array([1, 2, 1, 3, 5, 4, 6, 3])
```

```
m = 3
```

```
ind = np.arange(m - 1, len(y))
```

```
# sma = [sum(y[i - k] for k in range(m)) / m for i in ind]
```

```
sma = [np.mean(y[i - (m - 1) : i + 1]) for i in ind]
```

```
sma = np.array(sma)
```

```
sma
```

```
array([1.33333333, 2.          , 3.          , 4.          , 5.          ,  
       4.33333333])
```

```
# 3. Darstellung
```

```
plt.plot(y, "o -", label="Daten")
```

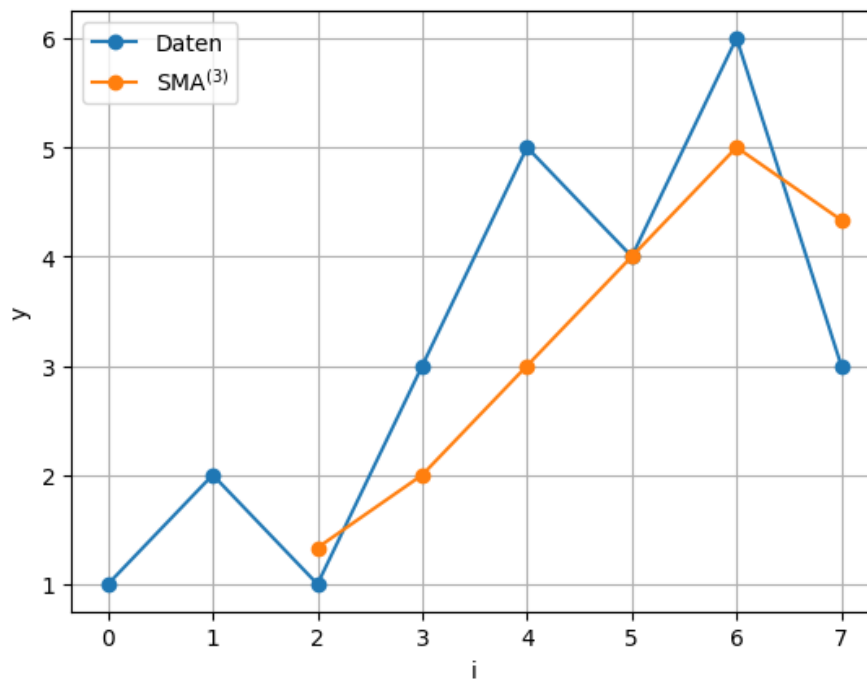
```
plt.plot(ind, sma, "o -", label=r"SMA $\hat{=}$ {(3)}$")
```

```
plt.xlabel("i")
```

```
plt.ylabel("y")
```

```
plt.legend()
```

```
plt.grid()
```



0.9.2 Anwendungen der Integration

Übungen - Lösungen

In diesem Übungsblatt gibt es etwas mehr Aufgaben als sonst. Die Erwartung ist nicht, dass du alle Aufgaben in einer Woche löst, sondern einfach ein paar pro Aufgabentyp. Den Rest kannst du als Vorbereitung auf die abgesetzte Modulprüfung bearbeiten.

```
import numpy as np
import matplotlib.pyplot as plt
```

Aufgabe 1: Mittelwert einer Funktion Berechne die Mittelwerte der folgenden Funktionen im angegebenen Intervall.

1. $f(x) = x^2$ für $x \in [1, 3]$.
2. $g(t) = e^{-t}$ für $t \in [-1, 1]$.
3. $h(x) = x^3 - x$ für $x \in [2, 4]$.
4. $\sin(t)$ für $t \in [0, 2\pi]$.
5. $f(t) = 3t^2 - 2$ für $t \in [-2, 2]$.
6. $g(t) = e^{-t}$ für $t \in [0, \infty[$. (*Hinweis:* Berechne den Mittelwert in $[0, x]$ und lasse x gegen ∞ gehen.)

Lösungen:

1.

$$\bar{f} = \frac{1}{3-1} \int_1^3 x^2 dx = \frac{1}{2} \left[\frac{1}{3} x^3 \right]_1^3 = \frac{1}{6} (27 - 1) = \frac{13}{3} = 4.\bar{3} \quad (112)$$

2.

$$\bar{g} = \frac{1}{1 - (-1)} \int_{-1}^1 e^{-t} dt = \frac{1}{2} \left[-e^{-t} \right]_{-1}^1 = \frac{1}{2} (-e^{-1} + e^1) \approx 1.175 \quad (113)$$

3.

$$\bar{h} = \frac{1}{4-2} \int_2^4 x^3 - x dx = \frac{1}{2} \left[\frac{1}{4} x^4 - \frac{1}{2} x^2 \right]_2^4 = \frac{1}{8} (4^4 - 2^4) - \frac{1}{4} (4^2 - 2^2) = 27 \quad (114)$$

4.

$$\overline{\sin} = \frac{1}{2\pi - 0} \int_0^{2\pi} \sin(t) dt = \frac{1}{2\pi} \left[-\cos(t) \right]_0^{2\pi} = \frac{1}{2\pi} (-1 - (-1)) = 0 \quad (115)$$

5.

$$\bar{f} = \frac{1}{2 - (-2)} \int_{-2}^2 3t^2 - 2 dt = \frac{1}{4} \left[t^3 - 2t \right]_{-2}^2 = \frac{1}{4} (2^3 - (-2)^3 - 2(2 - (-2))) = 2 \quad (116)$$

6.

$$\overline{g_x} = \frac{1}{x-0} \int_0^x e^{-t} dt = \frac{1}{x} \left[-e^{-t} \right]_0^x = \frac{1}{x} (1 - e^{-x}) \quad (117)$$

$$\bar{g} = \lim_{x \rightarrow \infty} \overline{g_x} = \lim_{x \rightarrow \infty} \frac{1}{x} (1 - e^{-x}) = 0 \quad (118)$$

Aufgabe 2: Gewichteter Mittelwert

2.1) Gewichtungsfunktion normieren Normiere die folgenden Gewichtungsfunktion über das angegebene Intervall.

1. $w(x) = x$ im Intervall $[1, 3]$
2. $w(t) = -t(t - 1)$ im Intervall $[0, 1]$
3. $w(t) = e^{-t}$ im Intervall $[0, \infty[$. (*Hinweis:* Berechne das Integral im Intervall $[0, x]$ und nehme dann den Grenzwert für $x \rightarrow \infty$.)

Lösungen:

1.

$$\int_1^3 x \, dx = \frac{1}{2} \left[x^2 \right]_1^3 = \frac{1}{2} (9 - 1) = 4 \quad (119)$$

$$\Rightarrow w(x) = \frac{1}{4}x \quad (120)$$

2.

$$\int_0^1 -t(t - 1) \, dt = \int_0^1 t - t^2 \, dt = \left[\frac{1}{2}t^2 - \frac{1}{3}t^3 \right]_0^1 = \frac{1}{2} - \frac{1}{3} = \frac{1}{6} \quad (121)$$

$$\Rightarrow w(t) = 6(t - t^2) \quad (122)$$

3.

$$\int_0^\infty e^{-t} \, dt = \lim_{x \rightarrow \infty} \left[-e^{-t} \right]_0^x = \lim_{x \rightarrow \infty} 1 - e^{-x} = 1 \quad (123)$$

Die Funktion $w(t) = e^{-t}$ ist bereits normiert.

2.2) Gewichteter Mittelwert einer Funktion berechnen

1. Berechne das gewichtete Mittel von $f(x) = x^2$ im Intervall $[1, 3]$ mit der normierten Gewichtungsfunktion $w(x) = x$ Aufgabe 1.
2. Berechne das gewichtete Mittel von $g(t) = 2t^2 + 3t$ im Intervall $[0, 1]$ mit der normierten Gewichtungsfunktion $w(t) = -t(t - 1)$ aus Aufgabe 1.
3. Berechne das gewichtete Mittel von $h(t) = \sin(t)$ im Intervall $[0, \infty[$ mit der normierten Gewichtungsfunktion $w(t) = e^{-t}$ aus Aufgabe 1.

Hinweis: Laut Wolfram|Alpha ist die Stammfunktion von $e^{-t} \sin(t)$:

$$-\frac{1}{2}e^{-t}(\sin(t) + \cos(t)) + C \quad (124)$$

Lösungen:

1.

$$\begin{aligned}
\int_1^3 w(x)f(x) \, dx &= \int_1^3 \frac{1}{4}x \cdot x^2 \, dx \\
&= \int_1^3 \frac{1}{4}x^3 \, dx \\
&= \left[\frac{1}{16}x^4 \right]_1^3 \\
&= \frac{1}{16}(3^4 - 1^4) \\
&= 5
\end{aligned}$$

2.

$$\begin{aligned}
\int_0^1 w(t)g(t) \, dt &= \int_0^1 6(t - t^2)(2t^2 + 3t) \, dt \\
&= 6 \int_0^1 2t^3 + 3t^2 - 2t^4 - 3t^3 \, dt \\
&= 6 \int_0^1 -2t^4 - t^3 + 3t^2 \, dt \\
&= 6 \left[-\frac{2}{5}t^5 - \frac{1}{4}t^4 + t^3 \right]_0^1 \\
&= 6 \left(-\frac{2}{5} - \frac{1}{4} + 1 \right) \\
&= \frac{21}{10}
\end{aligned}$$

3.

$$\begin{aligned}
\int_0^x w(t)h(t) \, dt &= \int_0^x e^{-t} \sin(t) \, dt \\
&= \left[-\frac{1}{2}e^{-t}(\sin(t) + \cos(t)) \right]_0^x \\
&= -\frac{1}{2}e^{-x}(\sin(x) + \cos(x)) + \frac{1}{2}e^{-0}(\sin(0) + \cos(0)) \\
&= -\frac{1}{2}e^{-x}(\sin(x) + \cos(x)) + \frac{1}{2}
\end{aligned}$$

$$\int_0^\infty w(t)h(t) \, dt = \lim_{x \rightarrow \infty} \left(-\frac{1}{2}e^{-x}(\sin(x) + \cos(x)) + \frac{1}{2} \right) = \frac{1}{2} \quad (125)$$

2.3 Darstellung Stelle zu den Aufgaben in 2.1 und 2.2 jeweils die zu mittelnde Funktion, die Gewichtungsfunktion und deren Produkt dar, und färbe die Fläche ein, die dem gewichteten Mittel entspricht.

Für Beispiel 3 kannst du den Bereich $t \in [0, 5]$ verwenden.

Lösungen

1. $f(x) = x^2$ im Intervall $[1, 3]$ mit der normierten Gewichtungsfunktion $w(x) = \frac{1}{4}x$

```
x = np.linspace(1, 3)
f = x**2
```

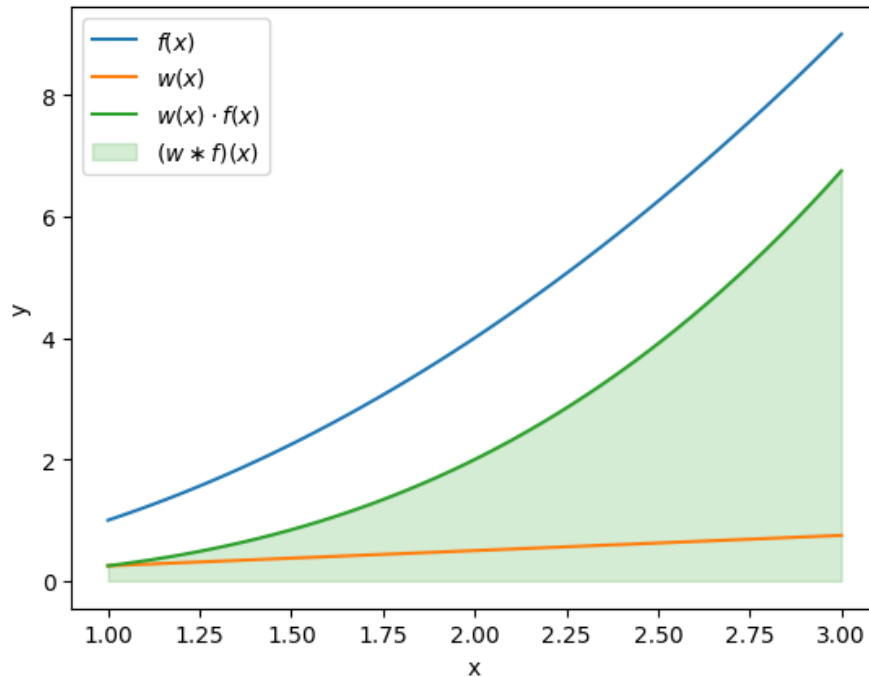


```

w = x / 4
wf = w * f

plt.plot(x, f, label=r"$f(x)$")
plt.plot(x, w, label=r"$w(x)$")
p = plt.plot(x, wf, label=r"$w(x) \cdot f(x)$")
plt.fill_between(x, wf, color=p[0].get_color(), alpha=0.2, label=r"$ (w \ast f)(x)$")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()

```



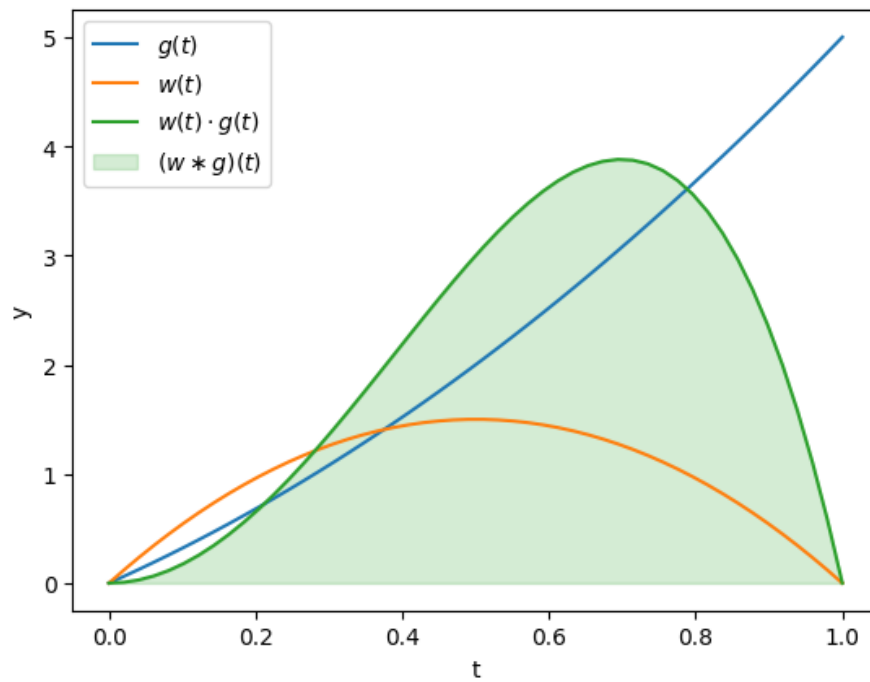
2. $g(t) = 2t^2 + 3t$ im Intervall $[0, 1]$ mit der normierten Gewichtungsfunktion $w(t) = 6(t - t^2)$.

```

t = np.linspace(0, 1)
g = 2 * t**2 + 3 * t
w = 6 * (t - t**2)
wg = w * g

plt.plot(t, g, label=r"$g(t)$")
plt.plot(t, w, label=r"$w(t)$")
p = plt.plot(t, wg, label=r"$w(t) \cdot g(t)$")
plt.fill_between(t, wg, color=p[0].get_color(), alpha=0.2, label=r"$ (w \ast g)(t)$")
plt.xlabel("t")
plt.ylabel("y")
plt.legend()
plt.show()

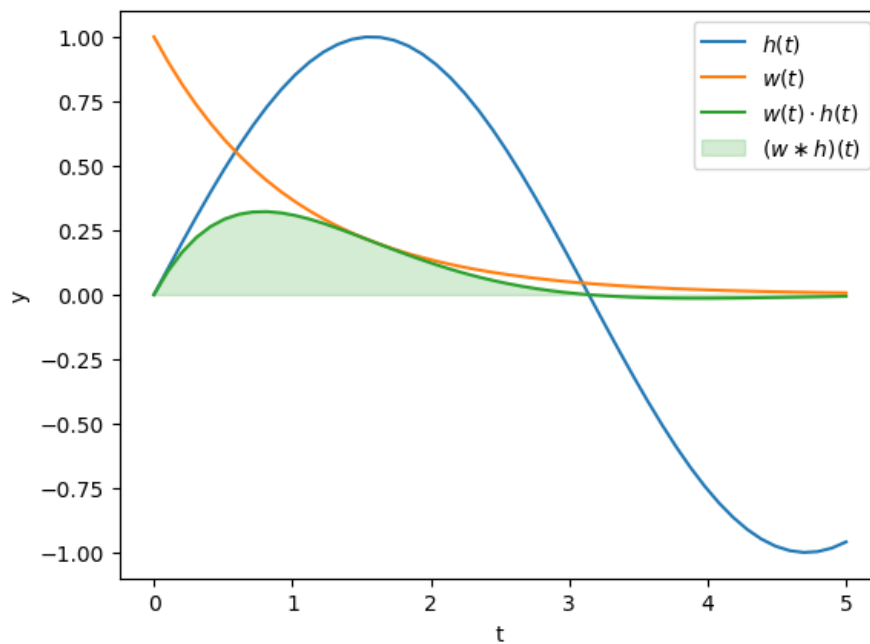
```



3. $h(t) = \sin(t)$ im Intervall $[0, \infty[$ mit der normierten Gewichtungsfunktion $w(t) = e^{-t}$. Darstellung im Bereich $[0, 5]$.

```
t = np.linspace(0, 5)
h = np.sin(t)
w = np.exp( -t)
wh = w * h

plt.plot(t, h, label=r"$h(t)$")
plt.plot(t, w, label=r"$w(t)$")
p = plt.plot(t, wh, label=r"$w(t) \cdot h(t)$")
plt.fill_between(t, wh, color=p[0].get_color(), alpha=0.2, label=r"$(w \ast h)(t)$")
plt.xlabel("t")
plt.ylabel("y")
plt.legend()
plt.show()
```



Aufgabe 3: Skalarprodukt zweier Funktionen

3.1) Eigenschaften des Skalarprodukts

1. Zeige, dass das Skalarprodukt zweier Funktionen

$$\langle f, g \rangle = \int_a^b f(x)g(x) \, dx \quad (126)$$

unter der Annahme, dass das Integral konvergiert (berechenbar ist), die folgenden Eigenschaften erfüllt:

1. Kommutativität: $\langle f, g \rangle = \langle g, f \rangle$
2. Linearität (a konstant): $\langle f + a \cdot g, h \rangle = \langle f, h \rangle + a \langle g, h \rangle$

Lösungen:

1. Folgt aus der Kommutativität (Vertauschbarkeit) des Produkts von reellen Zahlen: $x \cdot y = y \cdot x$ für alle $x, y \in \mathbb{R}$.

$$\langle f, g \rangle = \int_a^b f(x) \cdot g(x) \, dx = \int_a^b g(x) \cdot f(x) \, dx = \langle g, f \rangle \quad (127)$$

2. Folgt aus der Linearität des Integrals:

$$\langle f + a \cdot g, h \rangle = \int_a^b (f(x) + a \cdot g(x)) \cdot h(x) \, dx = \int_a^b f(x) \cdot h(x) \, dx + a \int_a^b g(x) \cdot h(x) \, dx = \langle f, h \rangle + a \langle g, h \rangle \quad (128)$$

Übrigens gilt die Linearität auch für die rechte Seite des Skalarprodukts:

$$\langle f, g + a \cdot h \rangle = \langle f, g \rangle + a \langle f, h \rangle \quad (129)$$

Das Skalarprodukt ist also linear in beiden Argumenten, was man auch als *bilinear* bezeichnet.

3.2) Skalarprodukt von Sinus und Cosinus Die beiden Funktionen

$$f_n(t) = \sin(n\omega_0 t) \quad \text{und} \quad g_n(t) = \cos(n\omega_0 t) \quad (130)$$

beschreiben harmonische Schwingungen mit der Kreisfrequenz $\omega_n = n\omega_0$, wobei $n \in \mathbb{N}$ eine natürliche Zahl grösser Null und $\omega_0 \in \mathbb{R}_{>0}$ eine positive reelle Konstante ist. Die Kreisfrequenz ω_0 ist die Grundschiwingung und für $n > 1$ ist ω_n eine Oberschiwingung.

Aufgabe: Zeige, dass die beiden Funktionen bezüglich des Skalarprodukts über eine Periode der Grundschiwingung, also $\left[0, \frac{2\pi}{\omega_0}\right]$, orthogonal sind.

Hinweis: Verwende das Additionstheorem

$$\sin(\alpha) \cos(\beta) + \cos(\alpha) \sin(\beta) = \sin(\alpha + \beta) \quad (131)$$

mit $\alpha = \beta$, um den Integranden zu vereinfachen, bevor zu die Stammfunktion bestimmst.

Lösung:

$$\begin{aligned} \langle f_n, g_n \rangle &= \int_0^{2\pi/\omega_0} \sin(n\omega_0 t) \cdot \cos(n\omega_0 t) \, dt \\ &= \frac{1}{2} \int_0^{2\pi/\omega_0} \sin(2n\omega_0 t) \, dt \\ &= \frac{1}{2} \left[-\frac{1}{2n\omega_0} \cos(2n\omega_0 t) \right]_0^{2\pi/\omega_0} \\ &= -\frac{1}{4n\omega_0} (\cos(4\pi n) - \cos(0)) \\ &= -\frac{1}{4n\omega_0} (1 - 1) \\ &= 0 \end{aligned}$$

Aufgabe 4: Faltung von Funktionen Berechne die Faltung $(f * g)(t)$ von

1. der Rechteckfunktion, $f(t) = \begin{cases} 1 & \text{für } t \in [0, 1] \\ 0 & \text{sonst} \end{cases}$, und der Funktion $g(t) = \sqrt{t}$.
2. $f(t) = e^{-t^2}$ und $g(t) = \sin(t)$ mithilfe von
 - a) Wolfram|Alpha analytisch
 - b) SciPy numerisch
 - c) und vergleiche die beiden im Bereich $t \in [-2\pi, 2\pi]$.

Lösungen

1.

$$\begin{aligned} \int_{-\infty}^{\infty} f(\tau) g(t - \tau) \, d\tau &= \int_0^1 (t - \tau)^{1/2} \, d\tau \\ &= \left[-\frac{2}{3} (t - \tau)^{3/2} \right]_0^1 \\ &= -\frac{2}{3} (t - 1)^{3/2} + \frac{2}{3} t^{3/2} \\ &= \frac{2}{3} (t^{3/2} - (t - 1)^{3/2}) \end{aligned}$$

2.a)

Das Integral kann mit Wolfram|Alpha: `integrate exp(-x2)sin(t-x) dx from -inf to inf` berechnet werden.

$$\begin{aligned}\int_{-\infty}^{\infty} f(\tau)g(t-\tau) d\tau &= \int_{-\infty}^{\infty} e^{-\tau^2} \sin(t-\tau) d\tau \\ &= \frac{\sqrt{\pi}}{\sqrt[4]{e}} \cdot \sin(t) \\ &\approx 1.380 \cdot \sin(t)\end{aligned}$$

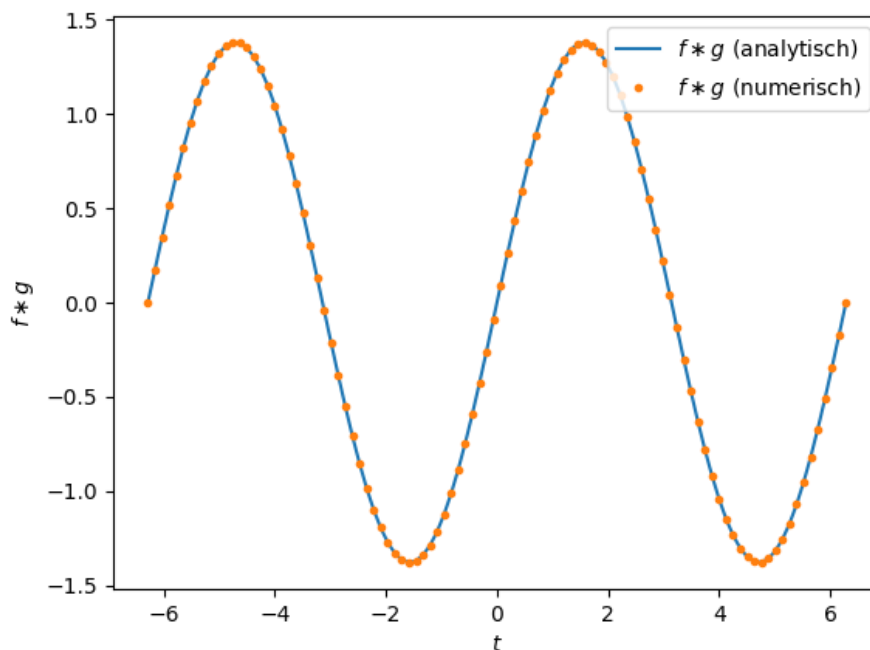
```
# 2.b)
from scipy.integrate import quad

f = lambda t: np.exp( -(t**2))
g = np.sin

conf_fg = np.vectorize(
    lambda t: quad(lambda tau: f(tau) * g(t - tau), -np.inf, np.inf)[0]
)

t = np.linspace( -2 * np.pi, 2 * np.pi, 100)
cn = conf_fg(t)
ca = np.sqrt(np.pi) * np.exp( -1 / 4) * np.sin(t)

plt.plot(t, ca, label=r"$f \ast g$ (analytisch)")
plt.plot(t, cn, ".", label=r"$f \ast g$ (numerisch)")
plt.xlabel(r"$t$")
plt.ylabel(r"$f \ast g$")
plt.legend()
plt.show()
```

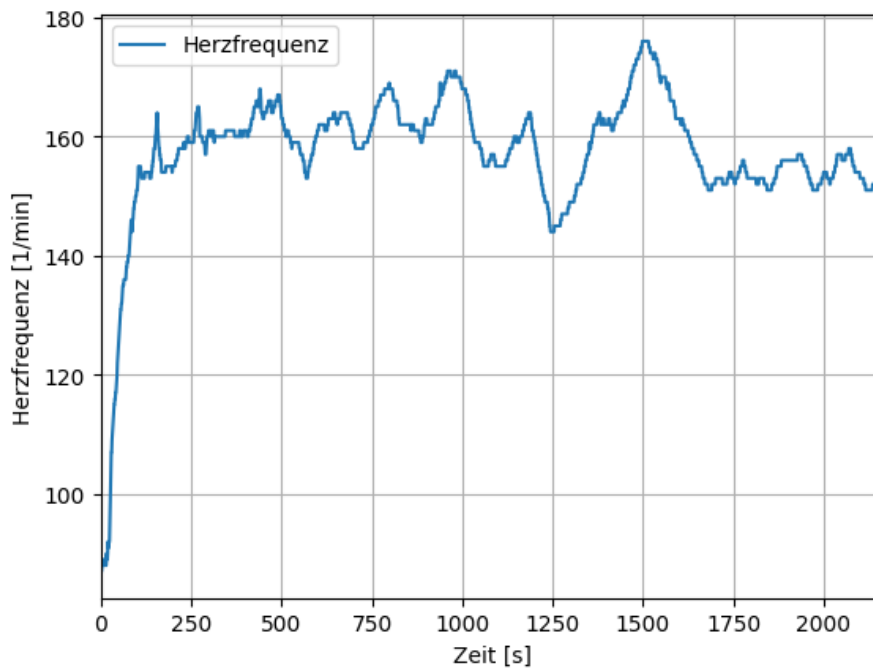


Aufgabe 5: Glättung von Daten mit der Faltung

5.1) Daten importieren und darstellen Importiere die Messzeitpunkte und die Herzfrequenz aus der Datei `jogging.csv` in die Variablen `t_data` und `hr_data`. Stelle die Herzfrequenz gegen die Zeit dar.

```
t_data, hr_data = np.loadtxt("jogging.csv", delimiter=";", skiprows=1, usecols=(0, 3)).T

plt.plot(t_data, hr_data, label="Herzfrequenz")
plt.xlim(t_data.min(), t_data.max())
plt.xlabel("Zeit [s]")
plt.ylabel("Herzfrequenz [1/min]")
plt.legend()
plt.grid()
plt.show()
```



5.2) Faltung mit einer Rechteckfunktion Nun soll die Herzfrequenz geglättet werden. Dazu wird die Faltung der Herzfrequenz mit einer Rechteckfunktion durchgeführt. Die Breite der Rechteckfunktion soll als Parameter `width=30` definiert werden, so dass sie später einfach verändert werden kann.

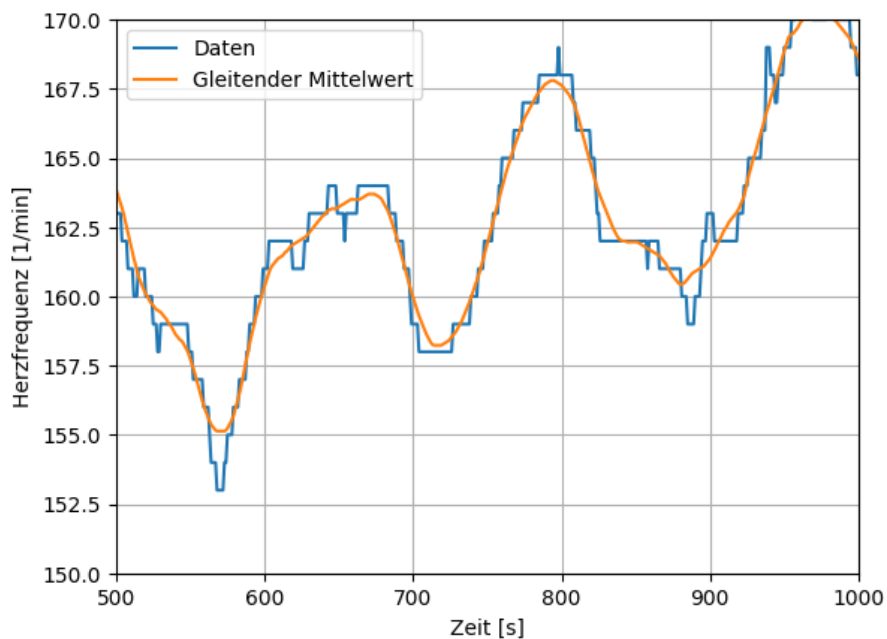
a) Benutze die Funktion `np.convolve` mit `mode="valid"`, um die diskrete Faltung zu berechnen. Siehe Beispiel in der Vertiefung. Stelle die Messdaten und die geglätteten Daten dar.

5.2.a)

```
width = 30
window = np.full(width, 1 / width)
hr_smooth = np.convolve(hr_data, window, mode="valid")
t_smooth = t_data[width // 2 : -width // 2 + 1]

plt.plot(t_data, hr_data, label="Daten")
# plt.xlim(t_data.min(), t_data.max())
plt.xlim(500, 1000)
plt.ylim(150, 170)
plt.plot(t_smooth, hr_smooth, label="Gleitender Mittelwert")
plt.xlabel("Zeit [s]")
```

```
plt.ylabel("Herzfrequenz [1/min]")
plt.legend()
plt.grid()
plt.show()
```



b) Berechne nun die Faltung mittels Integration. Erstelle dazu zuerst eine Interpolationsfunktion der Herzfrequenz und die Rechteckfunktion. Definiere dann eine Funktion `hr_convolution(t)`, welche die Faltung der Rechteckfunktion mit der Herzfrequenz-Interpolation berechnet. Stelle diese Funktion auch noch zusammen mit den Plots aus Teil a) da. (Die beiden Resultate sollten übereinstimmen.)

```
from scipy.interpolate import make_interp_spline

half_width = width / 2

rect = lambda t: np.where(np.abs(t) <= half_width, 1 / width, 0)

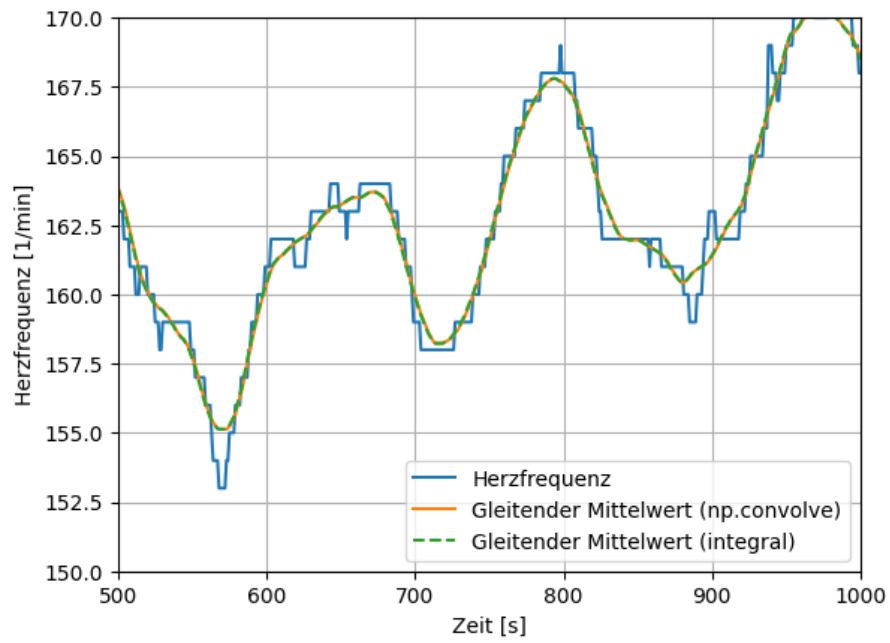
hr_spline = make_interp_spline(t_data, hr_data)

def hr_convolution(t, limit=100):
    integrand = lambda tau: rect(tau) * hr_spline(t - tau)
    return quad(integrand, -half_width, half_width, limit=limit)[0]

hr_smooth_2 = np.vectorize(hr_convolution)(t_smooth)

plt.plot(t_data, hr_data, label="Herzfrequenz")
plt.plot(t_smooth, hr_smooth, label="Gleitender Mittelwert (np.convolve)")
plt.plot(t_smooth, hr_smooth_2, " - -", label="Gleitender Mittelwert (integral)")
# plt.xlim(t_data.min(), t_data.max())
plt.xlim(500, 1000)
plt.ylim(150, 170)
plt.xlabel("Zeit [s]")
plt.ylabel("Herzfrequenz [1/min]")
plt.legend()
plt.grid()
```

```
plt.show()
```



0.9.3 Anwendungen der Integration

Vertiefung

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad, trapezoid
```

Matplotlib is building the font cache; this may take a moment.

Übersicht

1. Mittelwert einer Funktion
2. Gewichteter Mittelwert und Skalarprodukt
3. Gleitender Mittelwert und Faltung von Daten
4. Faltung von Funktionen

1. Mittelwert einer Funktion

Jogging: Mittlere Geschwindigkeit Wir benutzen nochmals die Jogging-Daten. Wie im Übungsblatt zur Integration laden wir die Daten und rechnen die Pace in die Geschwindigkeit in m/s um.

```
jogging = np.loadtxt("jogging.csv", delimiter=";", skiprows=1, usecols=(0, 1, 2))
time, distance, pace = jogging.T
```

```
t_max = 120
my_filter = (pace > 0) & (time <= t_max)
```

```
t_data = time[my_filter]
v_data = (1609.344 / 60) / pace[my_filter]
```

Nun ist die mittlere Geschwindigkeit gleich der zurückgelegten Strecke geteilt durch die Zeit:

$$\bar{v} = \frac{s(t_{\text{end}}) - s(t_{\text{start}})}{t_{\text{end}} - t_{\text{start}}} \quad (132)$$

Weil die zurückgelegte Strecke $s(t)$ eine Stammfunktion der Geschwindigkeit $v(t)$ ist, können wir die mittlere Geschwindigkeit auch als Integral berechnen:

$$s(t) = \int v(t) dt \quad \Rightarrow \quad \bar{v} = \frac{1}{t_{\text{end}} - t_{\text{start}}} \int_{t_{\text{start}}}^{t_{\text{end}}} v(t) dt \quad (133)$$

```
total_distance = trapezoid(v_data, t_data)
v_mean = total_distance / (t_data.max() - t_data.min())
v_mean
```

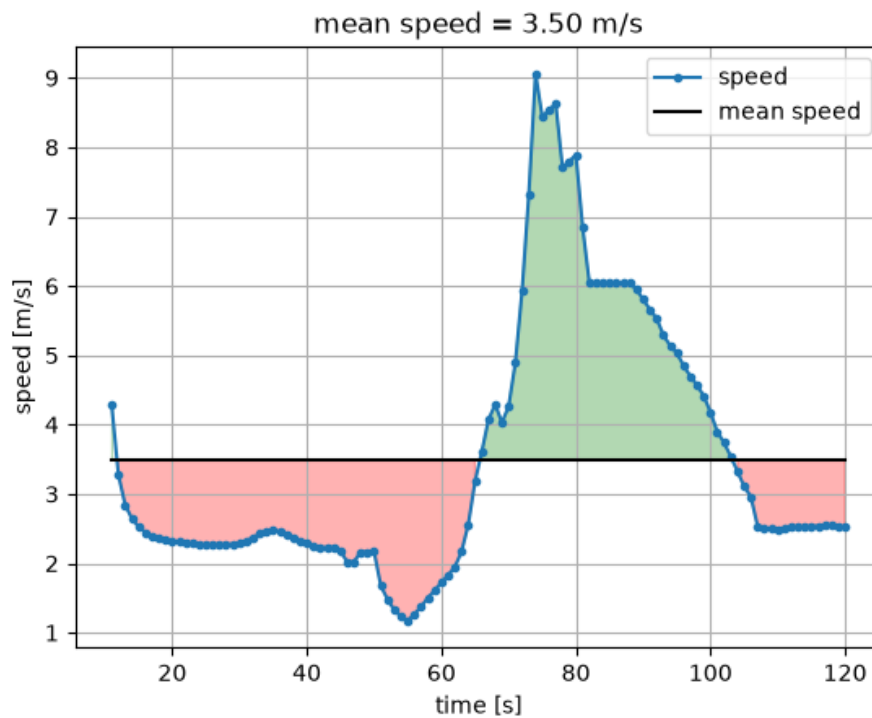
```
np.float64(3.495530561927936)
```

```
plt.plot(t_data, v_data, ". -", label="speed")
plt.plot(t_data, np.full_like(t_data, v_mean), "k", label="mean speed")
plt.fill_between(
    t_data,
    v_data,
    v_mean,
```

```

    where=v_data > v_mean,
    alpha=0.3,
    color="g",
)
plt.fill_between(
    t_data,
    v_data,
    v_mean,
    where=v_data < v_mean,
    alpha=0.3,
    color="r",
)
plt.xlabel("time [s]")
plt.ylabel("speed [m/s]")
plt.title(f"mean speed = {v_mean:.2f} m/s")
plt.legend()
plt.grid()
plt.show()

```



Unregelmässige Daten Mittelwert via Integration der Daten

`v_mean`

```
np.float64(3.495530561927936)
```

Mittelwert der Daten direkt stimmt praktisch überein, weil die Daten gleichmässig verteilt sind.

`v_data.mean()`

```
np.float64(3.4946686541818943)
```

Mittelwert über einen Teil der Daten stimmt nicht überein

```

rng = np.random.default_rng(42234098234)
sample_index = rng.choice([True, False], size=len(t_data))

v_data[sample_index].mean()

np.float64(3.8372847546527575)

```

Aber die Berechnung mithilfe des Integrals ist stabiler gegenüber ungleichmässiger Daten.

```

dist = trapezoid(v_data[sample_index], t_data[sample_index])
dist / (t_data[sample_index].max() - t_data[sample_index].min())

np.float64(3.5071185803425253)

```

2. Gewichteter Mittelwert und Skalarprodukt

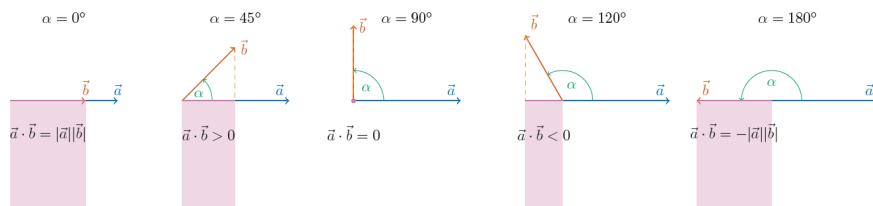
Allgemein Das gewichtete Mittel von Daten $\vec{y} \in \mathbb{R}^n$ mit normierten Gewichten $\vec{w} \in \mathbb{R}^n$ kann als Skalarprodukt geschrieben werden:

$$\bar{y} = \sum_{i=0}^{n-1} w_i y_i = \vec{w} \cdot \vec{y} \quad (134)$$

Man erinnere sich an die Definition des Skalarprodukts in der Linearen Algebra. Dort zeigt man, dass es einen Zusammenhang zur Norm (Länge) der beiden Vektoren und zum Zwischenwinkel α gibt:

$$\vec{w} \cdot \vec{y} = |\vec{w}| |\vec{y}| \cos(\alpha) \quad (135)$$

Die Norm selbst kann man auch verstehen als das Skalarprodukt eines Vektors mit sich selbst: $|\vec{a}| = \sqrt{\vec{a} \cdot \vec{a}}$.



Das Skalarprodukt ist also die Projektion des einen Vektors auf den anderen multipliziert mit der Länge des anderen Vektors. Diese Projektion ist

- maximal, wenn die beiden Vektoren in die gleiche Richtung zeigen ($\alpha = 0$),
- minimal, wenn sie in entgegengesetzte Richtung zeigen ($\alpha = \pi = 180^\circ$) und
- null, wenn sie orthogonal sind ($\alpha = \frac{\pi}{2} = 90^\circ$).

Man kann also auch sagen, dass das Skalarprodukt **die Ähnlichkeit der beiden Vektoren** misst.

Analog kann das Integral über ein Produkt zweier Funktionen als Skalarprodukt (auch inneres Produkt) interpretiert werden:

$$\langle f, g \rangle = \int_a^b f(t)g(t) dt \quad (136)$$

Bei Funktionen benutzt man die Schreibweise mit den Klammern $\langle \cdot, \cdot \rangle$ statt einem Punkt. Dies ist eine Verallgemeinerung des Skalarprodukts aus der linearen Algebra.

In diesem Sinn ist das Skalarprodukt zweier Funktionen ein **Mass für die Ähnlichkeit** der beiden Funktionen und kann als Projektion der einen Funktion auf die andere verstanden werden. Wenn es 0 ist, sind die Funktionen orthogonal, d.h. sie sind in einem gewissen Sinn unabhängig voneinander.

Bemerkung: Entsprechend kann man eine Norm (Länge), die L^2 -Norm, einer Funktion definieren:

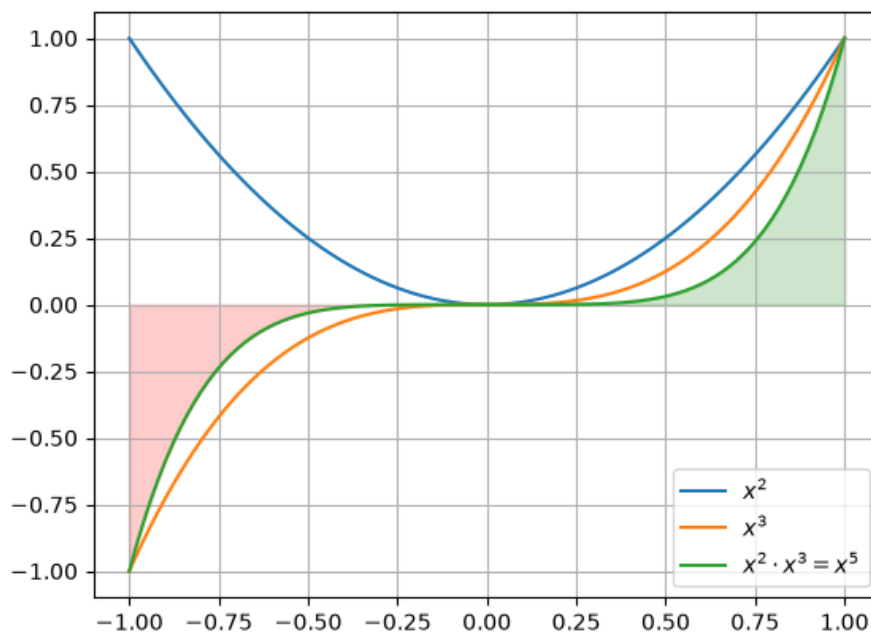
$$\|f\|_2 = \sqrt{\langle f, f \rangle} = \sqrt{\int_a^b f(t)^2 dt} \quad (137)$$

Beispiel 1 Das Skalarprodukt von $f(x) = x^2$ und $g(x) = x^3$ im Bereich $[-a, a]$ ist:

$$\begin{aligned}\langle f, g \rangle &= \int_{-a}^a x^2 x^3 dx \\ &= \int_{-a}^a x^5 dx \\ &= \left[\frac{1}{6} x^6 \right]_{-a}^a \\ &= \frac{1}{6} (a^6 - (-a)^6) \\ &= 0\end{aligned}$$

$f(x) = x^2$ und $g(x) = x^3$ sind also orthogonal. Allgemein sind gerade und ungerade Funktionen orthogonal, solange das Skalarprodukt auf einem symmetrischen Intervall $[-a, a]$ betrachtet wird.

```
x = np.linspace( -1, 1, 100)
f = x**2
g = x**3
fg = f * g
plt.plot(x, f, label=r"$x^2$")
plt.plot(x, g, label=r"$x^3$")
plt.plot(x, fg, label=r"$x^2 \cdot x^3 = x^5$")
plt.fill_between(x, fg, alpha=0.2, color="g", where=fg > 0)
plt.fill_between(x, fg, alpha=0.2, color="r", where=fg < 0)
plt.legend()
plt.grid()
plt.show()
```



Beispiel 2 Das Skalarprodukt von $f(t) = \sin(t)$ und $g(t) = \cos(t)$ über eine Periode $[0, 2\pi]$ ist:

$$\begin{aligned}
\langle f, g \rangle &= \int_0^{2\pi} \sin(t) \cos(t) \, dt \\
&= \int_0^{2\pi} \frac{1}{2} \sin(2t) \, dt \\
&= \left[-\frac{1}{4} \cos(2t) \right]_0^{2\pi} \\
&= -\frac{1}{4} (\cos(4\pi) - \cos(0)) \\
&= -\frac{1}{4} (1 - 1) \\
&= 0
\end{aligned}$$

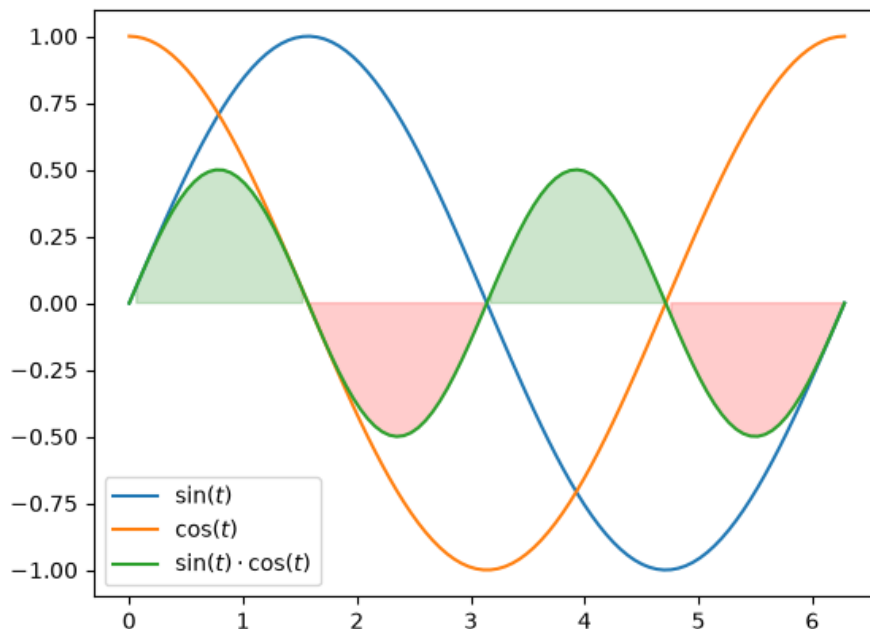
wobei wir ein Additionstheorem der trigonometrischen Funktionen benutzt haben: $\sin(t) \cos(t) = \frac{1}{2} \sin(2t)$.

```

t = np.linspace(0, 2 * np.pi, 100)
f = np.sin(t)
g = np.cos(t)
fg = f * g

plt.plot(t, f, label=r"$\sin(t)$")
plt.plot(t, g, label=r"$\cos(t)$")
plt.plot(t, fg, label=r"$\sin(t) \cdot \cos(t)$")
plt.fill_between(t, fg, alpha=0.2, color="g", where=fg > 0)
plt.fill_between(t, fg, alpha=0.2, color="r", where=fg < 0)
plt.legend()
plt.show()

```



Beispiel 3 Bei Funktionen, die für $x \rightarrow \pm\infty$ gegen Null gehen kann man sagen, dass das Skalarprodukt den Überlapp der beiden Funktionen misst.

Als Beispiel betrachten wir die Normalverteilung mit Erwartungswert μ und Standardabweichung $\sigma = 1$:

$$f(x, \mu) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(x-\mu)^2} \quad (138)$$

Wir berechnen das Skalarprodukt zwischen zwei dieser Normalverteilungen, eine mit $\mu = 0$ und eine mit $\mu \neq 0$.

Das Integral kann grundsätzlich analytisch berechnet werden, siehe Wolfram|Alpha, doch die Rechnung ist nicht relevant für uns:

$$\begin{aligned} \langle f(x, 0), f(x, \mu) \rangle &= \int_{-\infty}^{+\infty} f(x, 0) f(x, \mu) \, dx \\ &= \int_{-\infty}^{+\infty} \frac{1}{2\pi} e^{-\frac{1}{2}x^2} e^{-\frac{1}{2}(x-\mu)^2} \, dx \\ &= \frac{1}{2\pi} \int_{-\infty}^{+\infty} e^{-\frac{1}{2}(x^2+(x-\mu)^2)} \, dx \\ &= \frac{1}{2\pi} \int_{-\infty}^{+\infty} e^{-(x^2-x\mu+\frac{1}{2}\mu^2)} \, dx \\ &= \frac{1}{2\pi} \int_{-\infty}^{+\infty} e^{-(x^2-x\mu+\frac{1}{4}\mu^2)-\frac{1}{4}\mu^2} \, dx \\ &= \frac{1}{2\pi} e^{-\frac{1}{4}\mu^2} \int_{-\infty}^{+\infty} e^{-(x-\frac{1}{2}\mu)^2} \, dx \\ &= \frac{1}{2\pi} e^{-\frac{1}{4}\mu^2} \sqrt{\pi} \\ &= \frac{1}{2\sqrt{\pi}} e^{-\frac{1}{4}\mu^2} \end{aligned}$$

Im vorletzten Schritt wurde ein bekanntes Standardintegral verwendet: $\int_{-\infty}^{+\infty} e^{-x^2} \, dx = \sqrt{\pi}$.

Wir schauen uns das lieber graphisch an:

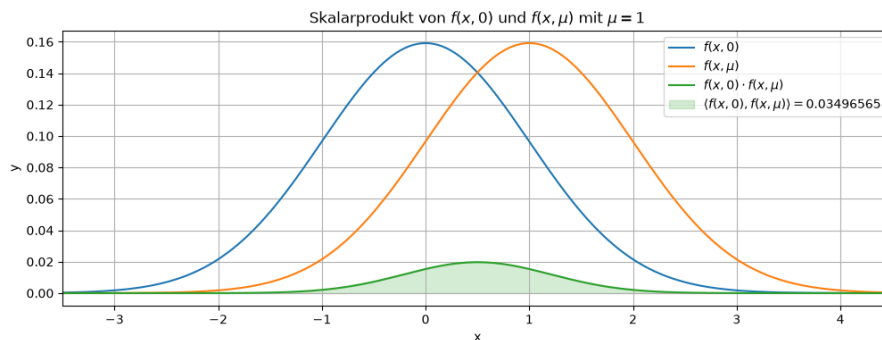
```
mu = 1
margin = 3.5
x = np.linspace(min(0, mu) - margin, max(0, mu) + margin, 200)

f = lambda x, mu: np.exp( -0.5 * (x - mu) ** 2) / (2 * np.pi)
y0 = f(x, 0)
y1 = f(x, mu)
y01 = y0 * y1

skalarprodukt, err = quad(lambda x: f(x, 0) * f(x, mu), -np.inf, np.inf)

plt.figure(figsize=(12, 4))
plt.plot(x, y0, label=r"$f(x,0)$")
plt.plot(x, y1, label=r"$f(x,\mu)$")
p = plt.plot(x, y01, label=r"$f(x,0) \cdot f(x,\mu)$")
plt.fill_between(
    x,
    y01,
    alpha=0.2,
    color=p[0].get_color(),
    label=f"$\\langle f(x,0), f(x,\mu) \\rangle = \{skalarprodukt:.8f\}$",
)
plt.legend()
plt.grid()
```

```
plt.xlabel("x")
plt.ylabel("y")
plt.title(f"Skalarprodukt von $f(x,0)$ und $f(x,\mu)$ mit $\mu=\{\mu\}$")
plt.xlim(x.min(), x.max())
plt.show()
```



3. Gleitender Mittelwert und Faltung von Daten

Allgemein Der gleitende Mittelwert über die letzten m Messwerte (*engl.* simple moving average, SMA) ist (siehe Einarbeitung)

$$\text{SMA}_i^{(m)} = \frac{1}{m} (y_i + y_{i-1} + \dots + y_{i-(m-1)}) = \frac{1}{m} \sum_{k=0}^{m-1} y_{i-k} \quad \text{für } m-1 \leq i < n \quad (139)$$

Mit Gewichten w_k definiert man auch den gewichteten gleitenden Mittelwert (*engl.* weighted moving average, WMA):

$$\text{WMA}_i^{(m)} = \sum_{k=0}^{m-1} w_k y_{i-k} \quad \text{für } m-1 \leq i < n \quad (140)$$

Typischerweise werden die Gewichte dreiecksförmig gewählt oder so, dass sie mit zunehmendem Abstand vom aktuellen Zeitpunkt i abnehmen.

Beispiel Wir vergleichen eine eigene Implementation mit der Funktion `np.convolve`.

```
t = np.linspace(0, 25, 50)
y = np.sin(t) + 2

m = 7
w = np.full(m, 1 / m)

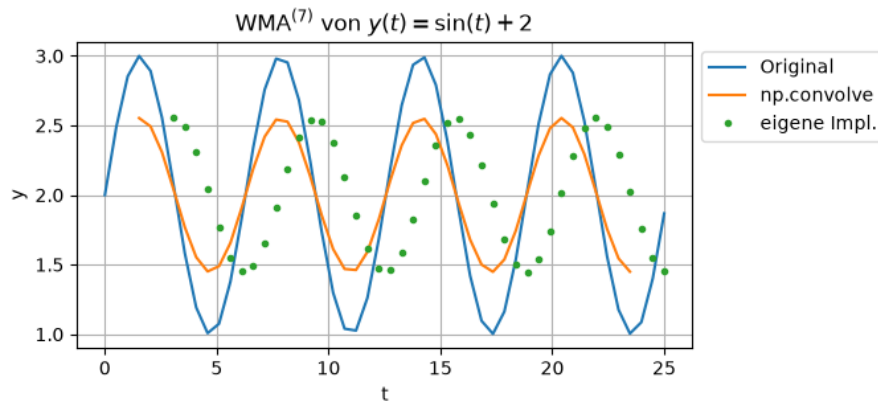
# Implementation von WMA wie in der Einarbeitung
wma = np.array([np.sum(w * y[i - (m - 1) : i + 1]) for i in range(m - 1, len(y))])

# Berechnung von WMA mit np.convolve
wma_conv = np.convolve(y, w, mode="valid")

t_conv = t[(m - 1) // 2 : -(m - 1) // 2]

plt.figure(figsize=(6, 3))
plt.plot(t, y, label="Original")
plt.plot(t_conv, wma_conv, label="np.convolve")
plt.plot(t[m - 1 :], wma, ".", label=f"eigene Impl.")
```

```
plt.legend(bbox_to_anchor=(1, 1))
plt.xlabel("t")
plt.ylabel("y")
plt.title(f"WMA(7) von  $y(t) = \sin(t) + 2$ ")
plt.grid()
plt.show()
```



Beobachtung: Die Extrema des ursprünglichen Signals werden geglättet aber auch verschoben.

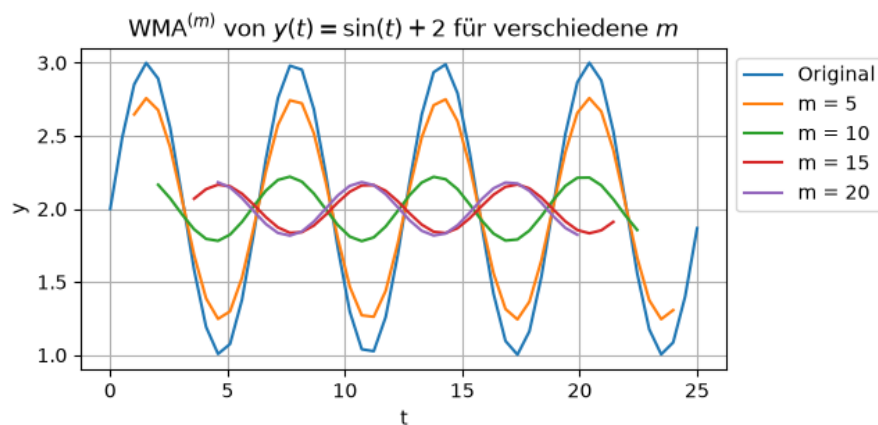
- Das kommt daher, dass deren Einfluss auf den gleitenden Mittelwert etwas verspätet eintritt.
- Deshalb wird der gleitende Mittelwert oft um die halbe Fensterbreite zurück verschoben.

Abhängigkeit von der Fensterbreite: Je grösser m , desto stärker wird das Signal geglättet.

```
plt.figure(figsize=(6, 3))
plt.plot(t, y, label="Original")

for m in (5, 10, 15, 20):
    w = np.full(m, 1 / m)
    y_conv = np.convolve(y, w, mode="valid")
    # plt.plot(t[m - 1 :], y_conv, label=f"m = {m}")
    plt.plot(t[(m - 1) // 2 : -(m - 1) // 2], y_conv, label=f"m = {m}")

plt.legend(bbox_to_anchor=(1, 1))
plt.xlabel("t")
plt.ylabel("y")
plt.title(r"WMA(m) von  $y(t) = \sin(t) + 2$  für verschiedene  $m$ ")
plt.grid()
plt.show()
```



4. Faltung von Funktionen

Allgemein Überträgt man das Konzept des gewichteten gleitenden Mittelwerts

$$\text{WMA}_i^{(m)} = \sum_{k=0}^{m-1} w_k y_{i-k} \quad (141)$$

auf kontinuierliche Funktionen, so erhält man die Faltung zweier Funktionen $w(t)$ und $f(t)$:

$$(w * f)(t) = \int_{-\infty}^{\infty} w(\tau) f(t - \tau) d\tau \quad (142)$$

Bemerkungen:

- Beim Analogieübergang von der diskreten Messreihe zu den kontinuierlichen Funktionen wurde wie folgt vorgegangen:
 - Der i -te Zeitpunkt wurde durch die Variable t ersetzt.
 - Der k -te Zeitpunkt wurde durch die Variable τ ersetzt.
 - Die Summierung über k wurde durch die Integration über τ ersetzt.
 - Die Gewichte w_k wurden durch die Funktion $w(\tau)$ ersetzt.
 - Die um k verschobenen Messwerte y_{i-k} wurden durch die verschobene Funktion $f(t - \tau)$ ersetzt.
 - Die Integrationsgrenzen wurden auf $\pm\infty$ erweitert. Das heisst, dass die Funktionen $w(t)$ und $f(t)$ für alle $t \in \mathbb{R}$ definiert sein müssen, bzw. ausserhalb eines interessierenden Bereichs den Wert 0 annehmen müssen.
- Die Faltung $w * f$ ist wieder eine Funktion und die Notation mit dem Stern ist üblich. Dies ist der Grund, weshalb man die Multiplikation auf keinen Fall mit einem Stern schreiben sollte.
- Die Funktion w nimmt hier die Rolle der Gewichtungsfunktion (Fenster oder Filterfunktion) ein und f diejenige der gemessenen Funktion (Signal).
- Die Faltung kommutativ, d.h., allgemein gilt:

$$f * g = g * f \quad (143)$$

und somit die Interpretation der Rollen der Funktionen austauschbar.

Interpretation:

- In $f * g$ wird die Funktion $g(t)$ geglättet, indem rückwärtsblickend der gewichtete Mittelwert über τ berechnet wird.
- Die Gewichtungsfunktion $f(\tau)$ gibt an, wie stark der von t aus um τ zurückliegende Wert gewichtet werden.

Beispiel 1: Faltung einer Sägezahnfunktion mit einer Rechteckfunktion Als Signal betrachten wir eine Sägezahnfunktion $f(t)$

$$f(t) = \begin{cases} 1 - 2t & \text{wenn } 0 \leq t \leq \frac{1}{2} \\ 0 & \text{sonst} \end{cases} \quad (144)$$

und als Gewichtungsfunktion eine Rechteckfunktion $w(t)$:

$$w(t) = \begin{cases} 1 & \text{wenn } |t| \leq \frac{1}{2} \\ 0 & \text{sonst} \end{cases} \quad (145)$$

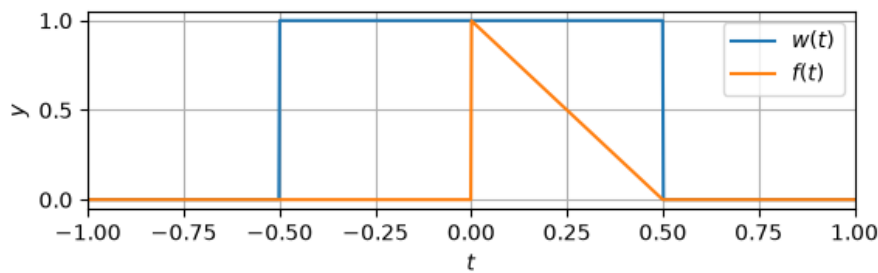
Machen wir uns zuerst mal ein Bild von den beiden Funktionen.

```
w = lambda t: np.where(np.abs(t) <= 0.5, 1, 0)
f = lambda t: np.where((0 <= t) & (t <= 0.5), 1 - 2 * t, 0)

t = np.linspace(-1, 1, 1000)
yw = w(t)
```

```
yf = f(t)
```

```
fig, ax1 = plt.subplots(1, 1, figsize=(6, 2))
ax1.plot(t, yw, label="$w(t)$")
ax1.plot(t, yf, label="$f(t)$")
ax1.set_xlabel("$t$")
ax1.set_ylabel("$y$")
ax1.legend()
ax1.grid()
ax1.set_xlim(t.min(), t.max())
plt.tight_layout()
plt.show()
```



Die Berechnung des Faltungsintegrals ist in diesem Fall von Hand möglich aber nicht so einfach. Wir berechnen es numerisch und stellen das Resultat dar.

```
width = 1
half_width = width / 2
height = 1 / width

w = lambda t: np.where(np.abs(t) <= half_width, height, 0)
f = lambda t: np.where((0 <= t) & (t <= 0.5), 1 - 2 * t, 0)

def convolution(t):
    wf = lambda tau: w(tau) * f(t - tau)
    return quad(wf, -half_width, half_width)[0]

t = np.linspace(-1, 1, 1000)
yw = w(t)
yf = f(t)
yconv = np.vectorize(convolution)(t)

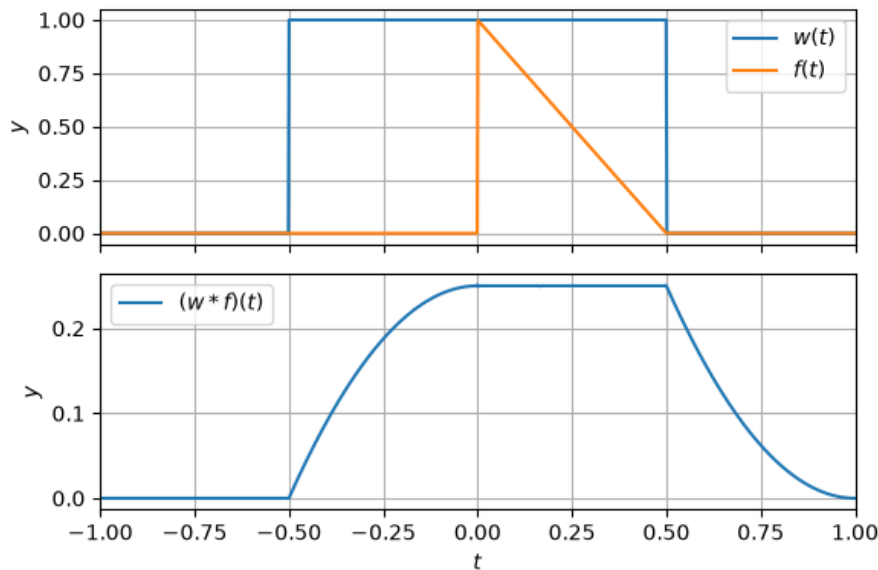
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(6, 4))

ax1.plot(t, yw, label="$w(t)$")
ax1.plot(t, yf, label="$f(t)$")
ax1.set_ylabel("$y$")
ax1.legend()
ax1.grid()
ax1.set_xlim(t.min(), t.max())

ax2.plot(t, yconv, label="$ (w * f)(t) $")
ax2.set_xlabel("$t$")
ax2.set_ylabel("$y$")
```

```
ax2.legend()
ax2.grid()

plt.tight_layout()
plt.show()
```



Aber was passiert denn während der Faltung?

```
from matplotlib.animation import FuncAnimation, FFMpegWriter
from IPython.display import HTML

tau = t # = np.linspace( -1, 1, 1000)

fig, (ax1, ax2, ax3) = plt.subplots(3, 1, sharex=True, figsize=(6, 5))

ax1.plot(t, yw, label="$w(t)$")
ax1.plot(t, yf, label="$f(t)$")
ax1.set_xlabel("$t$")
ax1.set_ylabel("$y$")
ax1.legend()
ax1.grid()

(p_w,) = ax2.plot([], [], label=r"$w(\tau)$")
(p_f,) = ax2.plot([], [], label=r"$f(t - \tau)$")
(p_wf,) = ax2.plot([], [], label=r"$w(\tau) \cdot f(t - \tau)$")
text = ax2.text(t.min() + 0.05, max(w(t).max(), f(t).max()) * 0.8, "")
ax2.set_xlim(t.min(), t.max())
ax2.set_ylim(min(w(t).min(), f(t).min()) - 0.1, max(w(t).max(), f(t).max()) + 0.1)
ax2.set_xlabel(r"$\tau$")
ax2.set_ylabel("$y$")
ax2.legend()
ax2.grid()

(p_conv,) = ax3.plot([], [], label="$f * g(t)$")
ax3.set_xlim(t.min(), t.max())
ax3.set_ylim(yconv.min() - 0.05, yconv.max() + 0.05)
ax3.set_xlabel("$t$")
ax3.set_ylabel("$y$")
```

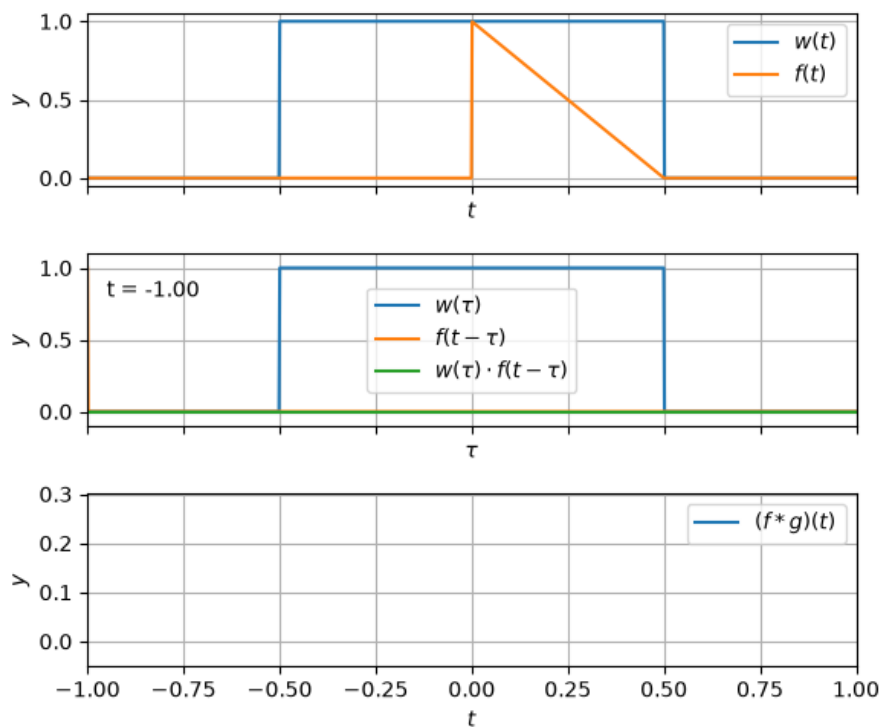
```
ax3.legend()
ax3.grid()
```

```
plt.tight_layout()
```

```
def update(t):
    p_w.set_data(tau, w(tau))
    p_f.set_data(tau, f(t - tau))
    p_wf.set_data(tau, w(tau) * f(t - tau))
    p_conv.set_data(tau[tau <= t], yconv[tau <= t])
    text.set_text(f"t = {t:.2f}")
    return (p_w, p_f, p_wf, p_conv, text)
```

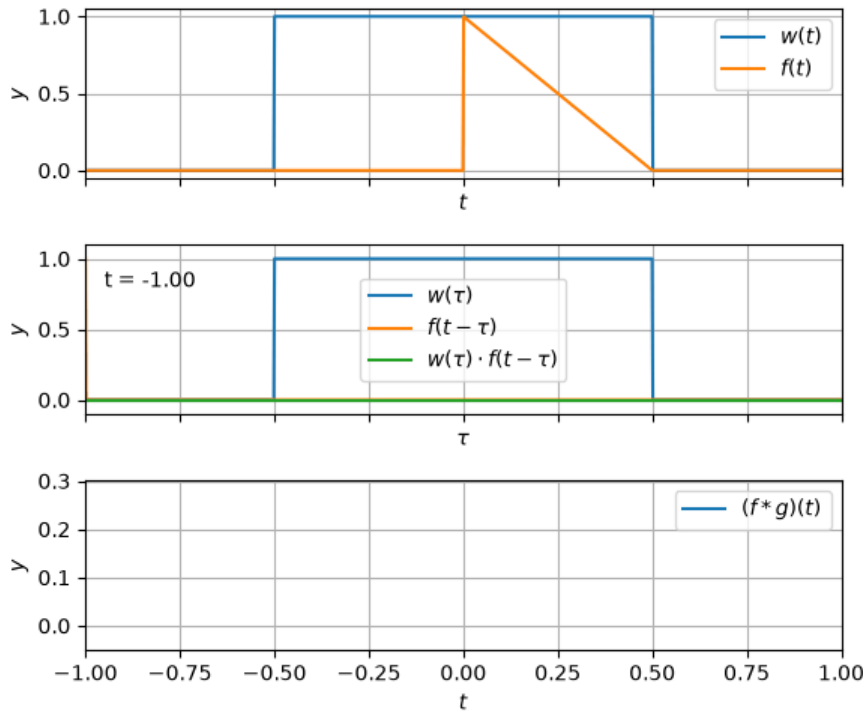
```
t_frames = np.linspace(-1, 1, 200)
```

```
ani = FuncAnimation(fig, update, frames=t_frames, interval=50, blit=True)
```



```
# Animation als GIF speichern und anzeigen (funktioniert in HTML und PDF)
from IPython.display import Image
```

```
ani.save("convolution_ani.gif", writer="pillow", fps=20)
plt.close(fig)
Image(filename="convolution_ani.gif")
```



Beispiel 2: Glättung des Sinus mit einer symmetrischen Rechteckfunktion Als Gewichtungsfunktion nehmen wir wieder symmetrische Rechteckfunktion mit Länge 1:

$$f(t) = \text{rect}(t) = \begin{cases} 1 & \text{wenn } |t| \leq \frac{1}{2} \\ 0 & \text{sonst} \end{cases} \quad (146)$$

$$\begin{aligned} (\text{rect} * \sin)(t) &= \int_{-\infty}^{\infty} \text{rect}(\tau) \sin(t - \tau) d\tau \\ &= \int_{-\frac{1}{2}}^{\frac{1}{2}} \sin(t - \tau) d\tau \\ &= \left[\cos(t - \tau) \right]_{-\frac{1}{2}}^{\frac{1}{2}} \\ &= \cos\left(t - \frac{1}{2}\right) - \cos\left(t + \frac{1}{2}\right) \\ &= 2 \sin\left(\frac{1}{2}\right) \sin(t) \end{aligned}$$

wobei die letzte Gleichheit aus dem Additionstheorem für den Cosinus folgt:

$$\cos(\alpha \pm \beta) = \cos(\alpha) \cos(\beta) \mp \sin(\alpha) \sin(\beta) \quad (147)$$

Der Sinus wird durch die Faltung mit der symmetrischen Rechteckfunktion einfach um einen Faktor $2 \sin(\frac{1}{2}) \approx 0.9589$ gestaucht.

Für die numerische Berechnung mit `scipy.integrate.quad` brauchen wir noch die Funktion `np.vectorize`, um die Faltung auf ein Array anwenden zu können.

```
half_width = 0.5
f = lambda t: np.where(np.abs(t) > half_width, 0, 1 / (2 * half_width))
g = np.sin
```

```
def convolution(t):
```

```

fg = lambda tau: f(tau) * g(t - tau)
return quad(fg, -half_width, half_width)[0]

```

```

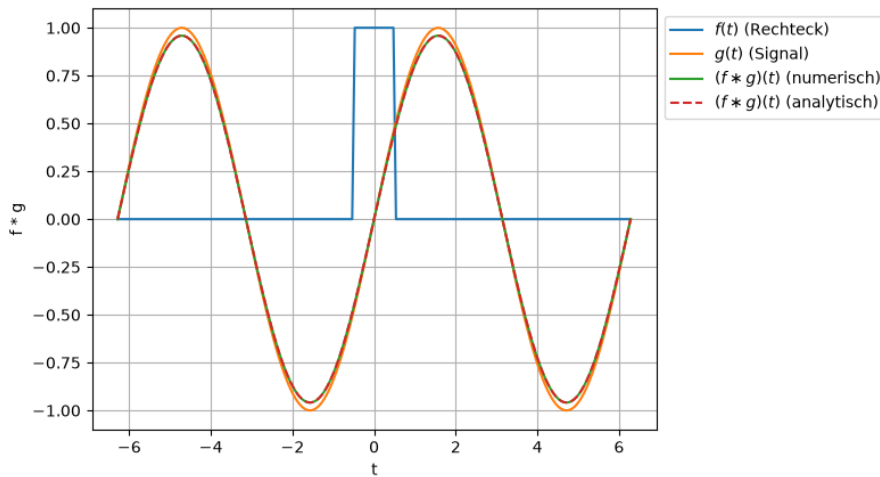
t = np.linspace(-2 * np.pi, 2 * np.pi, 200)
y = np.vectorize(convolution)(t)

```

```

plt.plot(t, f(t), label=r"$f(t)$ (Rechteck)")
plt.plot(t, g(t), label=r"$g(t)$ (Signal)")
plt.plot(t, y, label=r"$f \ast g(t)$ (numerisch)")
plt.plot(t, 2 * np.sin(1 / 2) * np.sin(t), "--", label=r"$f \ast g(t)$ (analytisch)")
plt.xlabel("t")
plt.ylabel("f * g")
plt.legend(bbox_to_anchor=(1, 1))
plt.grid()
plt.show()

```



Beispiel 3: Glättung des Sinus mit einer rückwärtsblickenden Rechteckfunktion Analog zum diskreten Beispiel weiter oben sei nun die Gewichtungsfunktion eine Rechteckfunktion der Länge Δt :

$$f(\tau) = \frac{1}{\Delta t} \begin{cases} 1 & \text{für } 0 \leq \tau \leq \Delta t \\ 0 & \text{sonst} \end{cases} \quad (148)$$

$$\begin{aligned}
 (f * \sin)(t) &= \int_{-\infty}^{\infty} f(\tau) \sin(t - \tau) d\tau \\
 &= \frac{1}{\Delta t} \int_0^{\Delta t} \sin(t - \tau) d\tau \\
 &= \frac{1}{\Delta t} \left[\cos(t - \tau) \right]_0^{\Delta t} \\
 &= \frac{1}{\Delta t} (\cos(t - \Delta t) - \cos(t))
 \end{aligned}$$

In diesem Fall wird der Sinus nicht nur gestaucht, sondern auch etwas verschoben. D.h. die Wellenberge kommen etwas verzögert an.

```

t = np.linspace(0, 25, 200)
y = np.sin(t)

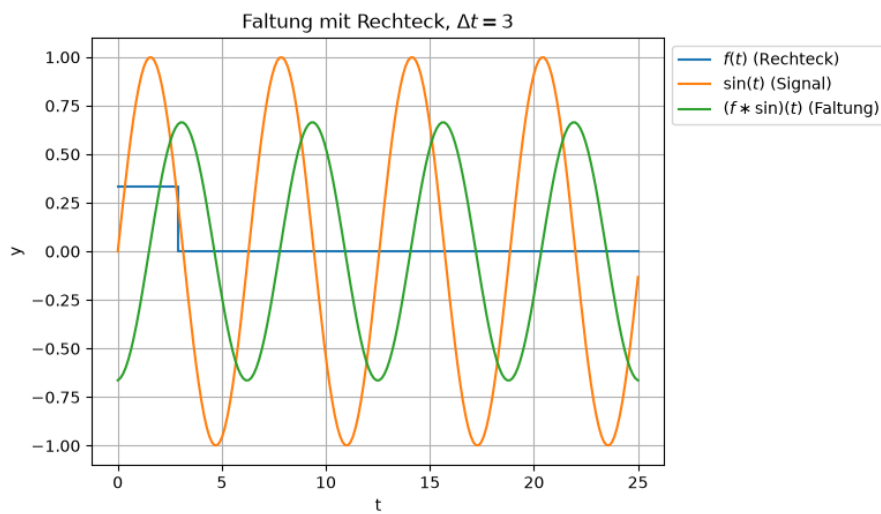
```

```

Dt = 3
f = np.where(t <= Dt, 1 / Dt, 0)
fy = (np.cos(t - Dt) - np.cos(t)) / Dt

plt.step(t, f, label=r"$f(t)$ (Rechteck)")
plt.plot(t, y, label=r"$\sin(t)$ (Signal)")
plt.plot(t, fy, label=r"$f \ast \sin(t)$ (Faltung)")
plt.xlabel("t")
plt.ylabel("y")
plt.legend(bbox_to_anchor=(1, 1))
plt.grid()
plt.title(f"Faltung mit Rechteck, $\Delta t = \{Dt\}$")
plt.show()

```



0.10 Best Practices Performance

0.10.1 Best Practices und Performance

Übungen - Lösungen

```
import numpy as np
```

Aufgabe 1: Best Practices Im GitHub Repository des Kurses findest du die Datei `09_BestPracticesPerformance/ngds.py`, in der ich am Freitag im Unterricht damit begonnen habe, ein Modul mit allen selbst-implementierten numerischen Methoden zu erstellen. Bisher sind nur die lineare Interpolation und das Bisektionsverfahren hinzugefügt.

Deine Aufgabe ist es, das Modul zu vervollständigen, indem du alle von uns im Unterricht besprochenen Implementierungen von numerischen Methoden hinzufügst und sauber nach den Best Practices dokumentierst und mit Assertions sicherstellst, dass die Eingabeparameter den Erwartungen entsprechen.

Aufgabe 2: Zeitmessung mit `timeit` Vergleiche die Laufzeit der Berechnung von

$$y = 2x^2 - 5x + 2 \quad (149)$$

für $x \in \{0, 1, 2, \dots, n-1\}$ zwischen einer Implementierung mit einer Schleife (List Comprehension) und mit NumPy. Ab welchem Wert von n ist NumPy schneller?

```
def list_comprehension(n):
    return [2 * x**2 - 5 * x + 2 for x in range(n)]
```

```
def numpy_array(n):
    x = np.arange(n)
    return 2 * x**2 - 5 * x + 2
```

```
n = 10
%timeit list_comprehension(n)
%timeit numpy_array(n)
```

```
283 ns \pm 2.71 ns per loop (mean \pm std. dev. of 7 runs, 1,000,000 loops each)
2.08 \mus \pm 452 ns per loop (mean \pm std. dev. of 7 runs, 100,000 loops each)
```

```
n = 50
%timeit list_comprehension(n)
%timeit numpy_array(n)
```

```
1.55 \mus \pm 15 ns per loop (mean \pm std. dev. of 7 runs, 1,000,000 loops each)
1.54 \mus \pm 12.3 ns per loop (mean \pm std. dev. of 7 runs, 1,000,000 loops each)
```

```
n = 100
%timeit list_comprehension(n)
%timeit numpy_array(n)
```

```
3.37 \mus \pm 39.5 ns per loop (mean \pm std. dev. of 7 runs, 100,000 loops each)
1.6 \mus \pm 10 ns per loop (mean \pm std. dev. of 7 runs, 1,000,000 loops each)
```

Beobachtungen:

- Für ca. $n < 50$ ist die List Comprehension schneller.
- Für ca. $n > 50$ ist NumPy schneller.

Aufgabe 3: Performance Profiling mit cProfile bzw. snakeviz In der untenstehenden Zelle ist eine einfache Lösung des Problem des Handlungsreisenden (*engl. Traveling Salesperson Problem*) implementiert. Dabei wird der kürzeste Weg durch eine Liste von Städten mit x-y-Koordinaten bestimmt.

Das Beispiel ist adaptiert von *Exercise 3* von diesem Tutorial der University of Sheffield, wo es auch eine ausführliche Erklärung von cProfile und SnakeViz gibt.

a) Führe den Code unten für 5 und 8 Städte aus und betrachte die Visualisierung und den Code, um zu verstehen, was eigentlich berechnet wurde. Du kannst die Anzahl Städte auch weiter erhöhen, doch Vorsicht, bei 12 Städten dauerte es bei mir bereits **fast 100 Minuten**.

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import permutations

def distance(city1, city2):
    """Compute the Euclidean distance between two cities."""
    return np.sqrt((city2[0] - city1[0]) ** 2 + (city2[1] - city1[1]) ** 2)

def total_distance(cities, sequence):
    """Compute the total distance to visit all the cities in the given sequence."""
    total = 0
    for i in range(len(sequence) - 1):
        total += distance(cities[sequence[i]], cities[sequence[i + 1]])
    return total + distance(cities[sequence[-1]], cities[sequence[0]])

def traveling_salesman_brute_force(cities):
    """Find the shortest path to visit all the cities exactly once by brute force."""
    min_distance = np.inf
    min_sequence = None
    for sequence in permutations(range(len(cities))):
        d = total_distance(cities, sequence)
        if d < min_distance:
            min_distance = d
            min_sequence = sequence
    return min_sequence, min_distance

def generate_cities(n, seed=12345):
    """Generate n random city coordinates."""
    rng = np.random.default_rng(seed)
    cities = []
    for _ in range(n):
        cities.append((rng.uniform(-1, 1), rng.uniform(-1, 1)))
    return cities

def plot_path(cities, sequence, min_distance):
    """Plot the path that visits the cities in the given sequence."""
    nodes = [cities[i] for i in sequence]
    nodes.append(nodes[0])
    x, y = np.array(nodes).T
```

```

plt.plot(x, y)
plt.plot(x, y, "ko")
for i, city in enumerate(cities):
    plt.annotate(
        str(i),
        city,
        textcoords="offset points",
        xytext=(0, 5),
        ha="center",
        va="bottom",
        annotation_clip=False,
    )
plt.axis("square")
plt.xlim( -1.05, 1.05)
plt.ylim( -1.05, 1.05)
plt.grid()
plt.title(f"path={sequence}, dist={min_distance:.3f}")
plt.show()

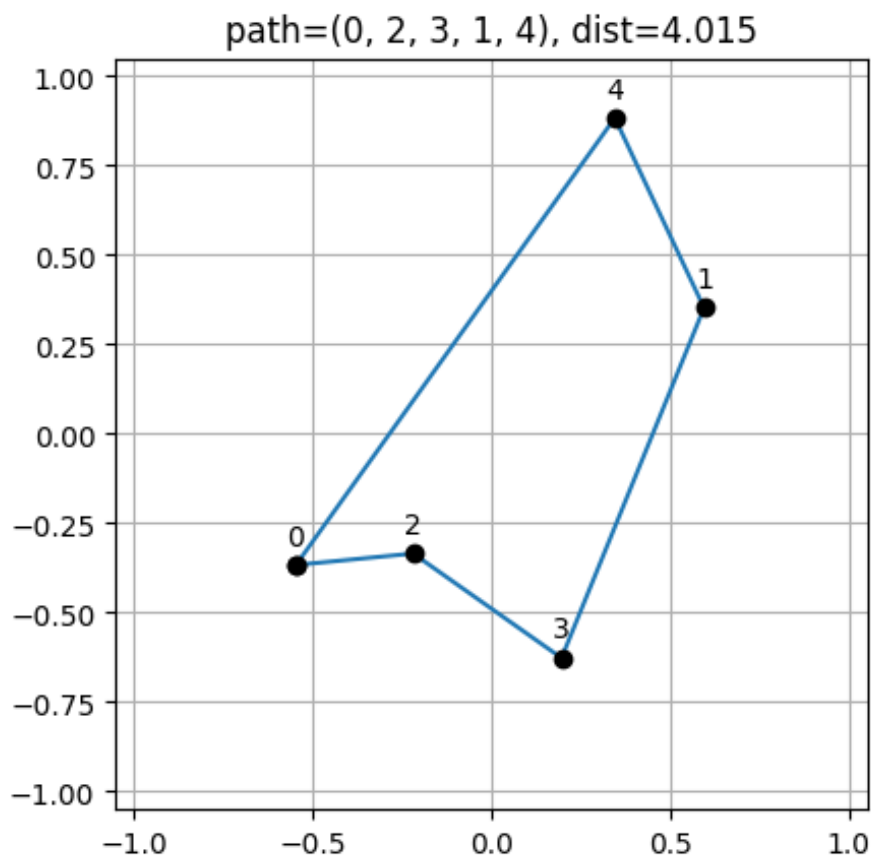
```

```

n_cities = 5
cities = generate_cities(n_cities)
shortest_path, min_distance = traveling_salesman_brute_force(cities)

plot_path(cities, shortest_path, min_distance)

```



b) Führe nun mit `cProfile` und/oder `snakeviz` ein Profiling der Funktionen `generate_cities()` und `traveling_salesman_brute_force()` durch. Du kannst entweder den Code in ein eigenes Script speichern und dort mit `cProfile` arbeiten (siehe `profiling.py`) oder die `snakeviz`-Extension in

Jupyter verwenden (siehe Beispiel am Ende von `09_Best_Practices_und_Performance.ipynb`).

- Identifiziere die Teile der Funktionen, die am meisten Zeit in Anspruch nehmen.
- Variiere dazu die Anzahl Städte im Bereich 3-6 und betrachte die Auswirkungen auf die Laufzeiten der beiden Funktionen.
- Notiere deine Beobachtungen.

```
%load_ext snakeviz
```

```
%snakeviz -t cities = generate_cities(1000)
```

```
*** Profile stats marshalled to file '/var/folders/p7/c2v89_m12k7dbrfs0m28h1640000gr/T/tmpkb56
Opening SnakeViz in a new tab...
```

```
snakeviz web server started on 127.0.0.1:8080; enter Ctrl -C to exit
```

```
http://127.0.0.1:8080/snakeviz/%2Fvar%2Ffolders%2Fp7%2Fc2v89_m12k7dbrfs0m28h1640000gr%2FT%2Ftm
```

```
%%snakeviz -t
```

```
cities = generate_cities(6)
```

```
shortest_path, min_distance = traveling_salesman_brute_force(cities)
```

```
*** Profile stats marshalled to file '/var/folders/p7/c2v89_m12k7dbrfs0m28h1640000gr/T/tmpsosl
Opening SnakeViz in a new tab...
```

```
snakeviz web server started on 127.0.0.1:8080; enter Ctrl -C to exit
```

```
http://127.0.0.1:8080/snakeviz/%2Fvar%2Ffolders%2Fp7%2Fc2v89_m12k7dbrfs0m28h1640000gr%2FT%2Ftm
```

```
cities = generate_cities(9)
```

```
%snakeviz -t shortest_path, min_distance = traveling_salesman_brute_force(cities)
```

```
*** Profile stats marshalled to file '/var/folders/p7/c2v89_m12k7dbrfs0m28h1640000gr/T/tmpyz_j
Opening SnakeViz in a new tab...
```

```
snakeviz web server started on 127.0.0.1:8080; enter Ctrl -C to exit
```

```
http://127.0.0.1:8080/snakeviz/%2Fvar%2Ffolders%2Fp7%2Fc2v89_m12k7dbrfs0m28h1640000gr%2FT%2Ftm
```

Beobachtungen:

- Die Laufzeit Funktion `generate_cities()` wird bei einer kleinen Anzahl Städte dominiert von der Initialisierung des Random-Number-Generators `np.random.default_rng()`. Bei einer grossen Anzahl Städte beginnt dann das Erzeugen der Zufallszahlen auch mehr Zeit zu benötigen.
- Die Funktion `generate_cities()` braucht wesentlich weniger Zeit als `traveling_salesman_brute_force()`. Je mehr Städte, desto grösser ist der Anteil der Zeit, die in `traveling_salesman_brute_force()` verbracht wird.
- Die Laufzeit der Funktion `traveling_salesman_brute_force()` wird durch `total_distance()` dominiert und diese wiederum durch die vielen Aufrufe der Funktion `distance()`.

Das heisst, eine Optimierung sollte darauf fokussieren, die Distanzen effizienter zu berechnen und wieder zu verwenden.

c) (optional) Versuche die Laufzeit von `generate_cities()` und `traveling_salesman_brute_force` zu verbessern, indem du das Profiling nutzt, die langsamsten Teile des Codes identifizierst und (mithilfe von NumPy) optimierst.

Lösung:

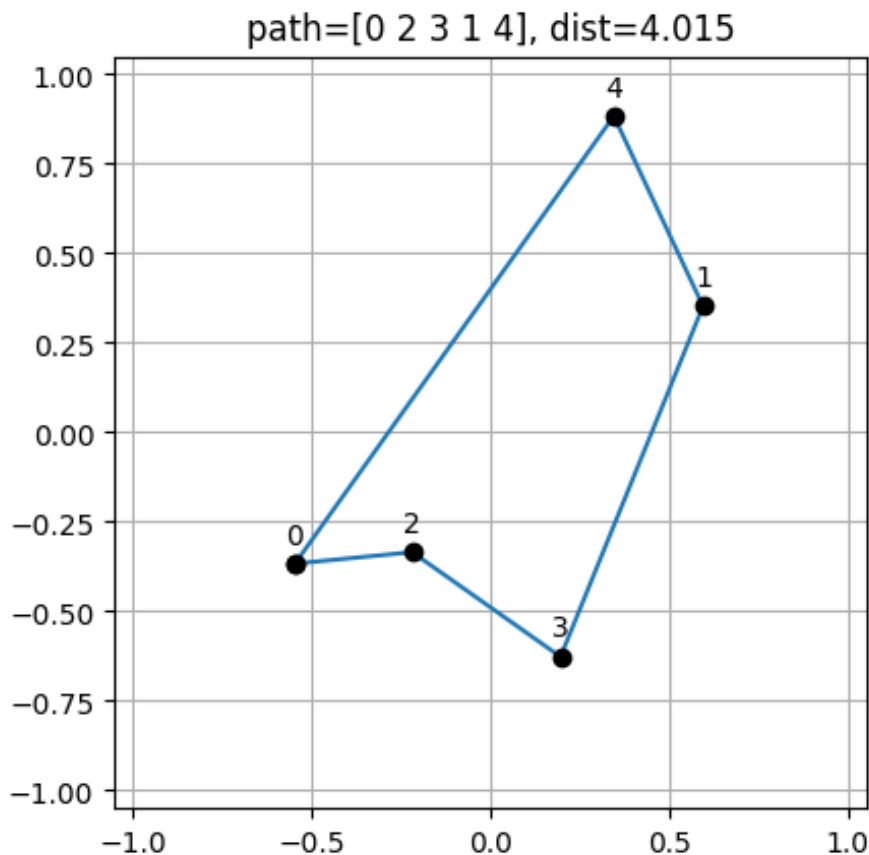
`traveling_salesman_brute_force()` kann auf verschiedenste Weisen optimiert werden.

Eine Möglichkeit ist, alle Anordnungen der Städte zu generieren und dann die Länge der Route mit NumPy zu berechnen, ohne explizit über die Pfade zu iterieren. Dies ist viel schneller. Allerdings könnte es bei einer grossen Anzahl Städte zu einem Speicherproblem führen, da alle Anordnungen im Speicher gehalten werden müssen.

```
def traveling_salesman_brute_force_vectorized(cities):
    """Find the shortest path to visit all the cities exactly once by brute force."""
    cities = np.array(cities)
    n_cities = cities.shape[0]
    # Generate all permutations of city indices
    sequences = np.array(list(permutations(range(n_cities))))
    # Create all possible paths
    paths = cities[sequences]
    # Compute the vectors between consecutive cities
    diffs = np.diff(paths, axis=1)
    # Compute the distances between consecutive cities
    distances = np.linalg.norm(diffs, axis=2)
    # Compute the distance to return to the starting city
    return_distances = np.linalg.norm(paths[:, 0] - paths[:, -1], axis=1)
    # Compute the total lengths of each path
    lengths = np.sum(distances, axis=1) + return_distances
    # Find the minimum length and corresponding sequence
    min_index = np.argmin(lengths)
    min_length = lengths[min_index]
    min_sequence = sequences[min_index]
    return min_sequence, min_length
```

```
n_cities = 5
cities = generate_cities(n_cities)
shortest_path, min_distance = traveling_salesman_brute_force_vectorized(cities)

plot_path(cities, shortest_path, min_distance)
```



```
cities = generate_cities(9)
```

```
%snakeviz -t shortest_path, min_distance = traveling_salesman_brute_force_vectorized(cities)
```

```
*** Profile stats marshalled to file '/var/folders/p7/c2v89_m12k7dbrfs0m28h1640000gr/T/tmpmja2
```

```
Opening SnakeViz in a new tab...
```

```
snakeviz web server started on 127.0.0.1:8080; enter Ctrl -C to exit
```

```
http://127.0.0.1:8080/snakeviz/%2Fvar%2Ffolders%2Fp7%2Fc2v89_m12k7dbrfs0m28h1640000gr%2FT%2Ftm
```

0.10.2 Best Practices und Performance

Vertiefung

Übersicht

1. Ziele bei der Programmierung von numerischen Algorithmen
2. Best Practices
3. Dokumentation
4. Performance
5. Profiling mit cProfile

```
import numpy as np
import matplotlib.pyplot as plt
```

1. Ziele bei der Programmierung von numerischen Algorithmen

- **Korrektheit**
 - Numerische Algorithmen müssen korrekt implementiert sein, d.h. sie müssen die mathematischen Konzepte und Theorien, auf denen sie basieren, genau widerspiegeln.
- **Genauigkeit & Stabilität**
 - Numerische Algorithmen müssen in der Lage sein, Ergebnisse zu liefern, die innerhalb einer geforderten Toleranz genau sind. Dazu gehört auch, dass der Code numerische Instabilitäten vermeidet und robust gegenüber Störungen und Rundungsfehlern ist.
- **Effizienz**
 - Numerische Verfahren arbeiten oft mit grossen Datenmengen und müssen daher effizient sein, um innerhalb eines angemessenen Zeitrahmens ausgeführt werden zu können. Dazu gehört auch, dass der Code optimiert wird, um die bestmögliche Leistung aus der zugrundeliegenden Hardware herauszuholen.
- **Modularität**
 - Die Methoden bestehen oft aus mehreren Schritten oder Teilproblemen, die modular und wiederverwendbar sein sollten. Eine gute Modularität erleichtert auch das Testen und Debuggen des Codes.
- **Verständlichkeit**
 - Numerische Algorithmen können komplex sein. Also ist es wichtig, dass der Code so geschrieben ist, dass er leicht verständlich ist. Dazu gehört auch, dass der Code gut dokumentiert und strukturiert ist.

2. Best Practices

2.1 Ziel Korrektheit

- Exakte Beschreibung der mathematischen Konzepte
- Exakte Implementierung der Algorithmen
- Fehlerhafte und inkonsistente Inputs abfangen und rückmelden
- Implementation anhand mehrerer Beispiele validieren (z.B. Vergleich mit anderen Methoden oder analytischen Lösungen)
- “Edge-Cases” auch in den Tests abdecken (z.B. Division durch Null, etc.)

2.2 Ziel Genauigkeit & Stabilität

- Bestehende Lösungen (Packages) verwenden (aber nur vertrauenswürdige und gut gewartete Quellen)
- Numerische Instabilitäten vermeiden: 64-bit Fließkommazahlen verwenden, Subtraktion von ähnlich grossen Zahlen (Auslöschung) und Division durch sehr kleine Zahlen vermeiden, etc.
- Toleranzen als Parameter bereitstellen und einhalten (z.B. bei der Konvergenz von Iterationsverfahren)
- Tests mit verschiedenen Toleranzen durchführen, sinnvolle Defaults angeben und ggf. Konvergenzgeschwindigkeit in der Dokumentation kommunizieren

2.3 Ziel Effizienz

- Vermeiden von IO-Operationen (Console, Disk, Netzwerk, etc.)
- Vektorisierung (z.B. mit Slicing und `np.vectorize`)
- Parallelisierung (z.B. mit `multiprocessing`)
- C-Code Kompilierung (z.B. mit Numba oder Cython)
- Manchmal durch Mutationen (z.B. in-place Operationen auf Arrays)

2.4 Ziel Modularität

- Funktionen und Methoden schreiben, die nur eine Aufgabe erfüllen.
- Gleichartige Funktionalität in ein Modul oder wenn nötig in eine Klasse packen.
- Klare Schnittstellen für Funktionen, Module und Klassen definieren, um Abhängigkeiten zu minimieren und transparent zu machen.
- Schnittstellen sollen sich an Standards orientieren (z.B. NumPy, SciPy, Pandas, etc.)

2.5 Ziel Verständlichkeit

- Klare, konsistente und aussagekräftige Namen für Funktionen, Module, Klassen und Variablen verwenden.
- Funktionen, Module und Klasse mit aussagekräftiger Dokumentationen versehen, z.B. auch Referenzen zu implementierten Methode angeben.
- Tests schreiben, um die Funktionalität zu testen und um die Verständlichkeit zu erhöhen, indem sie gleich Beispielcode liefern.
- Automatische Code-Formatierung verwenden, um den Code zu standardisieren (z.B. Black, `autopep8`), siehe Formatting Python in VS Code.

3. Dokumentation

- Funktionen in einem Modul zusammenstellen
- Jede Funktion mit einem Docstring versehen
 - Was macht die Funktion?
 - Welche Annahmen werden gemacht?
 - Welche Parameter werden erwartet?
 - Welche Rückgabewerte werden erzeugt?
 - Beispiel

In der Datei `ngds.py` ist ein Modul definiert, das selbstgeschriebene Funktionen für numerische Berechnungen enthält. Das Modul kann hier importiert werden. Die Docstrings der Funktionen werden angezeigt, wenn man mit der Maus über dem Funktionsnamen ist, oder können mit `help()` aufgerufen werden.

```
import ngds
```

```
# Modul neu laden, wenn man Änderungen gemacht hat. Ohne Reload wird die alte Version verwendet
from importlib import reload
```

```
reload(ngds)
```



```
# Docstring anzeigen
help(ngds.interpolate_linear)
```

Help on function interpolate_linear in module ngds:

```
interpolate_linear(x_data, y_data, x)
```

Perform linear interpolation to estimate the value of a function at a given point.

Parameters:

`x_data` (array -like): A sequence of x -coordinates (must be sorted in ascending order)
`y_data` (array -like): A sequence of y -coordinates corresponding to `x_data`.
`x` (array -like): The x -coordinates at which to interpolate.

Returns:

float: The interpolated y -coordinate corresponding to the input x.

Raises:

AssertionError: If any of the following conditions are not met:

- `x_data` and `y_data` are array -like structures.
- `x_data` and `y_data` have the same length.
- `x_data` and `y_data` contain at least two points.
- `x_data` is sorted in ascending order.

Notes:

- This function assumes that `x_data` is sorted in ascending order.
- The interpolation is performed using the formula:

$$y = y_data[i] + m[i] * (x - x_data[i]),$$
 where `m[i]` is the slope between consecutive points in `x_data` and `y_data`.

Und natürlich können die Funktionen auch direkt verwendet werden.

```
ngds.find_root_bisection(np.sin, -1.2, 1, 1e -6, 1e -6, 100)
```

```
array([[ 0.00000000e+00, -1.00000000e -01, -9.98334166e -02,
        1.10000000e+00,  9.98334166e -02],
       [ 1.00000000e+00,  4.50000000e -01,  4.34965534e -01,
        5.50000000e -01,  4.34965534e -01],
       [ 2.00000000e+00,  1.75000000e -01,  1.74108138e -01,
        2.75000000e -01,  1.74108138e -01],
       [ 3.00000000e+00,  3.75000000e -02,  3.74912116e -02,
        1.37500000e -01,  3.74912116e -02],
       [ 4.00000000e+00, -3.12500000e -02, -3.12449140e -02,
        6.87500000e -02,  3.12449140e -02],
       [ 5.00000000e+00,  3.12500000e -03,  3.12499491e -03,
        3.43750000e -02,  3.12499491e -03],
       [ 6.00000000e+00, -1.40625000e -02, -1.40620365e -02,
        1.71875000e -02,  1.40620365e -02],
       [ 7.00000000e+00, -5.46875000e -03, -5.46872274e -03,
        8.59375000e -03,  5.46872274e -03],
       [ 8.00000000e+00, -1.17187500e -03, -1.17187473e -03,
        4.29687500e -03,  1.17187473e -03],
       [ 9.00000000e+00,  9.76562500e -04,  9.76562345e -04,
        2.14843750e -03,  9.76562345e -04],
       [ 1.00000000e+01, -9.76562500e -05, -9.76562498e -05,
        1.07421875e -03,  9.76562498e -05],
       [ 1.10000000e+01,  4.39453125e -04,  4.39453111e -04,
```

```

5.37109375e -04, 4.39453111e -04],
[ 1.20000000e+01, 1.70898438e -04, 1.70898437e -04,
 2.68554688e -04, 1.70898437e -04],
[ 1.30000000e+01, 3.66210938e -05, 3.66210937e -05,
 1.34277344e -04, 3.66210937e -05],
[ 1.40000000e+01, -3.05175781e -05, -3.05175781e -05,
 6.71386719e -05, 3.05175781e -05],
[ 1.50000000e+01, 3.05175781e -06, 3.05175781e -06,
 3.35693359e -05, 3.05175781e -06],
[ 1.60000000e+01, -1.37329102e -05, -1.37329102e -05,
 1.67846680e -05, 1.37329102e -05],
[ 1.70000000e+01, -5.34057617e -06, -5.34057617e -06,
 8.39233398e -06, 5.34057617e -06],
[ 1.80000000e+01, -1.14440918e -06, -1.14440918e -06,
 4.19616699e -06, 1.14440918e -06],
[ 1.90000000e+01, 9.53674316e -07, 9.53674316e -07,
 2.09808350e -06, 9.53674316e -07],
[ 2.00000000e+01, -9.53674316e -08, -9.53674316e -08,
 1.04904175e -06, 9.53674316e -08],
[ 2.10000000e+01, 4.29153442e -07, 4.29153442e -07,
 5.24520874e -07, 4.29153442e -07]])

```

“Premature optimization is the root of all evil.” (Donald Knuth)

4. Performance

4.1 Einfache Zeitmessung mit `timeit` Für ein erstes Beispiel für die Zeitmessung mit `timeit` wandeln wir ein NumPy Array mit Integers in Strings um und vergleichen verschiedene Lösungen

```
numbers = np.arange(1_000)
```

```
print("Numpy astype")
%timeit numbers.astype(str)
```

```
print("List comprehension")
%timeit np.array([str(n) for n in numbers])
```

```
print("Generator expression")
%timeit np.array(list(str(n) for n in numbers))
```

```
print("Map")
%timeit np.array(list(map(str, numbers)))
```

Numpy astype

65.7 \mus \pm 1.25 \mus per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)

List comprehension

98.5 \mus \pm 1.83 \mus per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)

Generator expression

105 \mus \pm 1.15 \mus per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)

Map

99 \mus \pm 589 ns per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)

In NumPy bleiben ist also etwa doppelt so schnell wie eine Comprehension oder Map. Die Generator Expression ist immer ein Bisschen weniger effizient als die Comprehension. Meist ist Map auch minimal langsamer als die Comprehension.

Als nächstes benutzen wir die Funktion `str.join`, die eine Sequenz von Strings mit einem Trennzeichen zu einem einzigen String zusammenfügt.

```
print("Numpy astype")
%timeit " -".join(numbers.astype(str))

print("List comprehension")
%timeit " -".join([str(n) for n in numbers])

print("Generator expression")
%timeit " -".join(str(n) for n in numbers)

print("Map")
%timeit " -".join(map(str, numbers))

Numpy astype
169 \mus \pm 1.02 \mus per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
List comprehension
58.1 \mus \pm 234 ns per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
Generator expression
63.9 \mus \pm 435 ns per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
Map
58.5 \mus \pm 469 ns per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
```

Interessanterweise ist hier Map am schnellsten, gefolgt von der Comprehension und dann der Generator Expression. Die Lösung mit NumPy ist wesentlich langsamer - weshalb ist nicht ganz klar.

Klar ist, dass es sich lohnt, verschiedene Lösungen zu vergleichen!

Zum Vergleich noch eine eigene Implementation, ohne die Funktion `str.join` zu benutzen.

```
def join_numbers_by_hyphen(numbers):
    if len(numbers) == 0:
        return ""
    else:
        res = str(numbers[0])
        if len(numbers) > 1:
            for n in numbers[1:]:
                res += f" -{n}"
        return res

%timeit join_numbers_by_hyphen(numbers)

125 \mus \pm 5.68 \mus per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
```

Gar nicht so übel, aber doch nicht besser.

Die folgenden Beispiele zeigen, dass es sich lohnt, NumPy Funktionen zu verwenden und nicht die generischen Python Funktionen.

```
n = 1_000_000
rng = np.random.default_rng(12345)
numbers = rng.random(n)

%timeit sum(numbers)
%timeit np.sum(numbers)
```

```
31.2 ms \pm 258 \mus per loop (mean \pm std. dev. of 7 runs, 10 loops each)
116 \mus \pm 252 ns per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
```

```
%timeit np.sum(numbers) / len(numbers)
%timeit np.mean(numbers)
```

```
116 \mus \pm 130 ns per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
118 \mus \pm 1.35 \mus per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
```

```
# Mittelwert pro Block
```

```
block_size = 1_000
```

```
%timeit np.array([np.mean(numbers[pos : pos + block_size]) for pos in range(0, n, block_size)])
%timeit numbers.reshape(-1, block_size).mean(axis=1)
```

```
1.45 ms \pm 17.7 \mus per loop (mean \pm std. dev. of 7 runs, 1,000 loops each)
124 \mus \pm 1.26 \mus per loop (mean \pm std. dev. of 7 runs, 10,000 loops each)
```

Hier zeigt sich, dass eine vektorisierte Lösung der Problemstellung mithilfe von Array-Denkweise gefunden werden kann und dann oft viel schneller als die Lösung mit Schleifen ist. GitHub Copilot kann hierbei helfen, die richtige Formulierung zu finden. Immer testen!

4.2 Profiling mit cProfile Video: “Using “cProfile” to analyze Python code performance” - IDG TECHtalk

Als Beispiel banutzen wir unsere Implementation der linearen Interpolation.

Es ist besser, Profiling in einem Script statt in Jupyter zu machen, weil Jupyter einigen Overhead produziert, den man bei der Interpretation des Profilings herausfiltern muss. Siehe Script `profiling.py`.

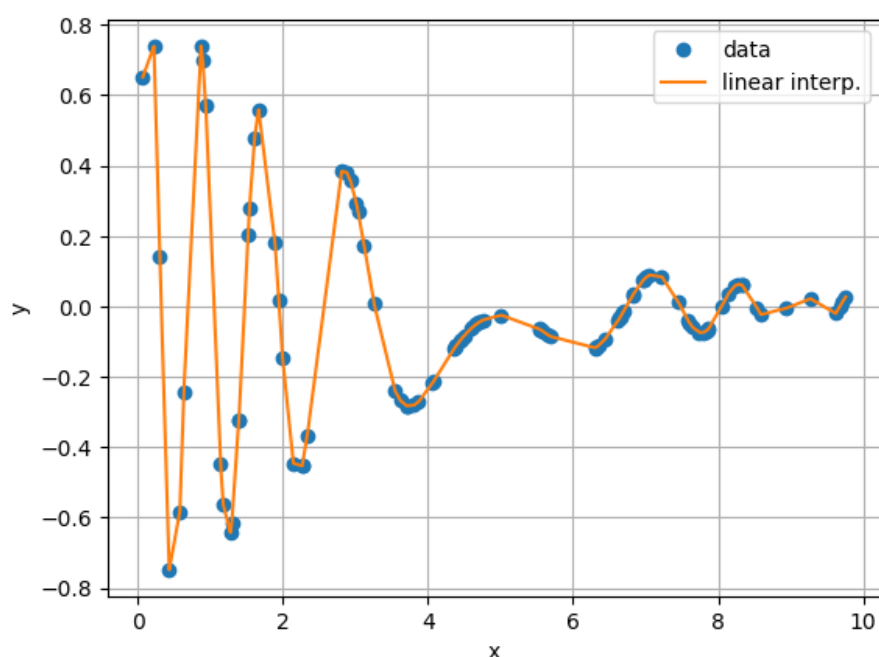
```
def interpolate_linear(x_data, y_data, x):
    i = np.searchsorted(x_data, x) - 1
    i = np.where(i < 0, 0, i)
    m = np.diff(y_data) / np.diff(x_data)
    return y_data[i] + m[i] * (x - x_data[i])

n_data = 100
rng = np.random.default_rng(42)

x_data = np.sort(rng.uniform(low=0, high=10, size=n_data))
y_data = np.sin(x_data * (10 - x_data)) * np.exp(-x_data / 3)

x = np.linspace(x_data.min(), x_data.max(), 1_000_000)
y = interpolate_linear(x_data, y_data, x)

plt.plot(x_data, y_data, "o", label="data")
plt.plot(x, y, label="linear interp.")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```



```
from cProfile import Profile
from pstats import Stats
```

```
with Profile() as pr:
    y = interpolate_linear(x_data, y_data, x)
stats = Stats(pr)
stats.strip_dirs()
stats.sort_stats("tottime")
stats.print_stats()
```

677 function calls (671 primitive calls) in 0.010 seconds

Ordered by: internal time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.005	0.005	0.005	0.005	{method 'searchsorted' of 'numpy.ndarray' object}
2/1	0.003	0.002	0.010	0.010	2823643808.py:1(interpolate_linear)
2	0.001	0.000	0.001	0.000	{method '__exit__' of 'sqlite3.Connection' object}
1	0.000	0.000	0.000	0.000	{method 'execute' of 'sqlite3.Connection' object}
1	0.000	0.000	0.002	0.002	history.py:100(only_when_enabled)
4	0.000	0.000	0.000	0.000	attrsettr.py:66(_get_attr_opt)
14	0.000	0.000	0.000	0.000	socket.py:623(send)
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}
2	0.000	0.000	0.000	0.000	iostream.py:348(<lambda>)
2	0.000	0.000	0.000	0.000	zmqstream.py:546(_run_callback)
60	0.000	0.000	0.000	0.000	enum.py:1575(_get_value)
175/171	0.000	0.000	0.000	0.000	{built-in method builtins.isinstance}
2	0.000	0.000	0.000	0.000	_function_base_impl.py:1402(diff)
2	0.000	0.000	0.000	0.000	socket.py:700(send_multipart)
2	0.000	0.000	0.000	0.000	socket.py:771(recv_multipart)
14	0.000	0.000	0.000	0.000	enum.py:1582(__or__)
1	0.000	0.000	0.000	0.000	{method 'control' of 'select.kqueue' objects}
2	0.000	0.000	0.000	0.000	iostream.py:350(_really_send)
26	0.000	0.000	0.000	0.000	enum.py:676(__call__)

2	0.000	0.000	0.000	0.000	<frozen abc>:121(__subclasscheck__)
1	0.000	0.000	0.000	0.000	decorator.py:200(fix)
1	0.000	0.000	0.000	0.000	inspect.py:3087(_bind)
4	0.000	0.000	0.000	0.000	typing.py:184(_type_check)
4	0.000	0.000	0.000	0.000	attrsettr.py:43(__getattr__)
2/1	0.000	0.000	0.005	0.005	fromnumeric.py:1421(searchsorted)
6	0.000	0.000	0.000	0.000	enum.py:1593(__and__)
26	0.000	0.000	0.000	0.000	enum.py:1145(__new__)
1	0.000	0.000	0.001	0.001	history.py:1055(writeout_cache)
2	0.000	0.000	0.000	0.000	zmqstream.py:573(_handle_events)
2	0.000	0.000	0.000	0.000	traitlets.py:3631(set)
2	0.000	0.000	0.000	0.000	iostream.py:229(_handle_event)
1	0.000	0.000	0.000	0.000	cProfile.py:120(__exit__)
1	0.000	0.000	0.000	0.000	inspect.py:3229(bind)
1	0.000	0.000	0.000	0.000	traitlets.py:1527(_notify_observers)
2	0.000	0.000	0.000	0.000	zmqstream.py:653(_rebuild_io_state)
2	0.000	0.000	0.000	0.000	events.py:92(_run)
1	0.000	0.000	0.000	0.000	base_events.py:1966(_run_once)
1	0.000	0.000	0.000	0.000	asyncio.py:231(add_callback)
2	0.000	0.000	0.000	0.000	zmqstream.py:676(_update_handler)
32	0.000	0.000	0.000	0.000	typing.py:1355(__eq__)
2	0.000	0.000	0.000	0.000	traitlets.py:718(_validate)
2	0.000	0.000	0.000	0.000	traitlets.py:689(set)
4	0.000	0.000	0.000	0.000	traitlets.py:676(__get__)
1	0.000	0.000	0.000	0.000	inspect.py:2891(apply_defaults)
1	0.000	0.000	0.005	0.005	fromnumeric.py:48(_wrapfunc)
2	0.000	0.000	0.000	0.000	traitlets.py:3474(validate)
2	0.000	0.000	0.000	0.000	{method 'run' of '_contextvars.Context' objects}
4	0.000	0.000	0.000	0.000	<frozen importlib._bootstrap>:1401(_handle_froml
8	0.000	0.000	0.000	0.000	{built -in method builtins.next}
1	0.000	0.000	0.001	0.001	history.py:1039(_writeout_input_cache)
2	0.000	0.000	0.000	0.000	typing.py:1583(__subclasscheck__)
2	0.000	0.000	0.000	0.000	zmqstream.py:614(_handle_recv)
1	0.000	0.000	0.000	0.000	selectors.py:540(select)
2	0.000	0.000	0.000	0.000	traitlets.py:3624(validate_elements)
4	0.000	0.000	0.000	0.000	traitlets.py:629(get)
9	0.000	0.000	0.000	0.000	{built -in method builtins.hasattr}
2	0.000	0.000	0.000	0.000	traitlets.py:727(_cross_validate)
1	0.000	0.000	0.000	0.000	traitlets.py:1512(_notify_trait)
1	0.000	0.000	0.000	0.000	traitlets.py:1523(notify_change)
6	0.000	0.000	0.000	0.000	{built -in method builtins.getattr}
1	0.000	0.000	0.000	0.000	base_events.py:846(_call_soon)
1	0.000	0.000	0.000	0.000	threading.py:325(_acquire_restore)
12	0.000	0.000	0.000	0.000	typing.py:1361(__hash__)
2	0.000	0.000	0.000	0.000	queue.py:112(empty)
1	0.000	0.000	0.000	0.000	inspect.py:2838(args)
2	0.000	0.000	0.000	0.000	selector_events.py:744(_process_events)
1	0.000	0.000	0.000	0.000	contextlib.py:303(helper)
1	0.000	0.000	0.000	0.000	events.py:41(__init__)
2	0.000	0.000	0.000	0.000	typing.py:1292(__instancecheck__)
2	0.000	0.000	0.000	0.000	{built -in method builtins.issubclass}
2	0.000	0.000	0.000	0.000	{built -in method _abc._abc_subclasscheck}
2	0.000	0.000	0.000	0.000	traitlets.py:708(__set__)

1	0.000	0.000	0.000	0.000	base_events.py:1951(_add_callback)
1	0.000	0.000	0.002	0.002	decorator.py:232(fun)
1	0.000	0.000	0.000	0.000	contextlib.py:108(__init__)
1	0.000	0.000	0.000	0.000	contextlib.py:136(__enter__)
4	0.000	0.000	0.000	0.000	typing.py:396(inner)
2	0.000	0.000	0.000	0.000	traitlets.py:2304(validate)
1	0.000	0.000	0.000	0.000	base_events.py:817(call_soon)
6	0.000	0.000	0.000	0.000	{method '__enter__' of '_thread.lock' objects}
2	0.000	0.000	0.000	0.000	iostream.py:285(_is_master_process)
20	0.000	0.000	0.000	0.000	typing.py:2300(cast)
7	0.000	0.000	0.000	0.000	{method 'append' of 'list' objects}
12	0.000	0.000	0.000	0.000	{built -in method builtins.len}
1	0.000	0.000	0.000	0.000	history.py:1047(_writeout_output_cache)
3	0.000	0.000	0.000	0.000	base_events.py:766(time)
5	0.000	0.000	0.000	0.000	{method 'get' of 'dict' objects}
2	0.000	0.000	0.000	0.000	threading.py:316(__exit__)
2	0.000	0.000	0.000	0.000	zmqstream.py:532(sending)
6	0.000	0.000	0.000	0.000	{method '__exit__' of '_thread.lock' objects}
12	0.000	0.000	0.000	0.000	{built -in method builtins.hash}
4	0.000	0.000	0.000	0.000	{method 'upper' of 'str' objects}
2	0.000	0.000	0.000	0.000	threading.py:313(__enter__)
1	0.000	0.000	0.000	0.000	inspect.py:2861(kwargs)
1	0.000	0.000	0.000	0.000	ioloop.py:750(_run_callback)
1	0.000	0.000	0.000	0.000	threading.py:322(_release_save)
2	0.000	0.000	0.000	0.000	iostream.py:288(_check_mp_mode)
3	0.000	0.000	0.000	0.000	{method 'acquire' of '_thread.lock' objects}
1	0.000	0.000	0.000	0.000	asyncio.py:206(_handle_events)
1	0.000	0.000	0.000	0.000	contextlib.py:145(__exit__)
4	0.000	0.000	0.000	0.000	typing.py:175(_type_convert)
1	0.000	0.000	0.000	0.000	{method 'values' of 'mappingproxy' objects}
1	0.000	0.000	0.000	0.000	threading.py:641(clear)
2	0.000	0.000	0.000	0.000	history.py:1100(hold)
2	0.000	0.000	0.000	0.000	traitlets.py:3486(validate_elements)
3	0.000	0.000	0.000	0.000	{method 'items' of 'mappingproxy' objects}
2	0.000	0.000	0.000	0.000	{built -in method numpy.asanyarray}
2	0.000	0.000	0.000	0.000	queue.py:266(_qsize)
4	0.000	0.000	0.000	0.000	{method 'popleft' of 'collections.deque' objects}
3	0.000	0.000	0.000	0.000	{built -in method time.monotonic}
10	0.000	0.000	0.000	0.000	inspect.py:2744(kind)
2	0.000	0.000	0.000	0.000	{built -in method numpy.lib.array_utils.normaliz
4	0.000	0.000	0.000	0.000	zmqstream.py:528(receiving)
2	0.000	0.000	0.000	0.000	{built -in method builtins.max}
3	0.000	0.000	0.000	0.000	{method 'append' of 'collections.deque' objects}
2	0.000	0.000	0.000	0.000	{built -in method posix.getpid}
2	0.000	0.000	0.000	0.000	iostream.py:327(closed)
4	0.000	0.000	0.000	0.000	inspect.py:2732(name)
2	0.000	0.000	0.000	0.000	{method '__enter__' of 'sqlite3.Connection' obje
1	0.000	0.000	0.000	0.000	zmqstream.py:684(<lambda>)
2	0.000	0.000	0.000	0.000	{built -in method builtins.iter}
1	0.000	0.000	0.000	0.000	threading.py:328(_is_owned)
4	0.000	0.000	0.000	0.000	inspect.py:3041(parameters)
1	0.000	0.000	0.000	0.000	{built -in method _contextvars.copy_context}
1	0.000	0.000	0.000	0.000	{built -in method _asyncio.get_running_loop}

2	0.000	0.000	0.000	0.000	{method 'extend' of 'list' objects}
1	0.000	0.000	0.000	0.000	{built -in method _thread.allocate_lock}
2	0.000	0.000	0.000	0.000	_function_base_impl.py:1398(_diff_dispatcher)
1	0.000	0.000	0.000	0.000	inspect.py:2830(__init__)
1	0.000	0.000	0.000	0.000	base_events.py:2064(get_debug)
1	0.000	0.000	0.000	0.000	multiarray.py:403(where)
1	0.000	0.000	0.000	0.000	fromnumeric.py:1417(_searchsorted_dispatcher)
1	0.000	0.000	0.000	0.000	base_events.py:548(_check_closed)
1	0.000	0.000	0.000	0.000	{method 'release' of '_thread.lock' objects}

<pstats.Stats at 0x1156aee40>

Vergleichen wir mit einer nicht-vektorierten Implementierung.

```
def interpolate_linear_slow1(x_data, y_data, x):
    y = []
    for x_k in x:
        i = 0
        while i < len(x_data) and x_data[i] < x_k:
            i += 1
        if i == 0:
            i = 1
        elif i == len(x_data):
            i = len(x_data) - 1
        x_lower = x_data[i - 1]
        x_upper = x_data[i]
        y_lower = y_data[i - 1]
        y_upper = y_data[i]
        y_k = (y_upper - y_lower) / (x_upper - x_lower) * (x_k - x_lower) + y_lower
        y.append(y_k)
    return np.array(y)
```

```
with Profile() as pr:
    y = interpolate_linear_slow1(x_data, y_data, x)
stats = Stats(pr)
stats.strip_dirs()
stats.sort_stats("cumtime")
stats.print_stats()
```

53493788 function calls (53493783 primitive calls) in 6.517 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
5	0.000	0.000	6.342	1.268	base_events.py:1966(_run_once)
5	0.004	0.001	6.341	1.268	selectors.py:540(select)
1	2.951	2.951	4.214	4.214	3124833435.py:1(interpolate_linear_slow1)
4	1.511	0.378	2.124	0.531	{method 'control' of 'select.kqueue' objects}
52493049	1.836	0.000	1.836	0.000	{built -in method builtins.len}
5	0.000	0.000	0.087	0.017	events.py:92(_run)
5	0.000	0.000	0.087	0.017	{method 'run' of '_contextvars.Context' objects}
2	0.000	0.000	0.087	0.043	zmqstream.py:573(_handle_events)
1	0.000	0.000	0.087	0.087	asyncio.py:206(_handle_events)

2	0.000	0.000	0.087	0.043	zmqstream.py:614(_handle_recv)
2	0.000	0.000	0.087	0.043	zmqstream.py:546(_run_callback)
2	0.000	0.000	0.087	0.043	iostream.py:229(_handle_event)
2	0.000	0.000	0.087	0.043	iostream.py:348(<lambda>)
14	0.057	0.004	0.075	0.005	socket.py:623(send)
2	0.000	0.000	0.069	0.035	iostream.py:350(_really_send)
2	0.000	0.000	0.069	0.035	socket.py:700(send_multipart)
1000005	0.045	0.000	0.045	0.000	{method 'append' of 'list' objects}
2/1	0.087	0.043	0.017	0.017	decorator.py:232(fun)
1	0.000	0.000	0.017	0.017	history.py:100(only_when_enabled)
1	0.000	0.000	0.017	0.017	history.py:1055(writeout_cache)
1	0.000	0.000	0.017	0.017	history.py:1039(_writeout_input_cache)
1	0.016	0.016	0.016	0.016	{built -in method numpy.array}
2	0.004	0.002	0.006	0.003	{method '__exit__' of 'sqlite3.Connection' object}
1	0.004	0.004	0.005	0.005	{method 'execute' of 'sqlite3.Connection' object}
2	0.000	0.000	0.000	0.000	ioloop.py:750(_run_callback)
4	0.000	0.000	0.000	0.000	attrsettr.py:43(__getattr__)
14	0.000	0.000	0.000	0.000	enum.py:1582(__or__)
1	0.000	0.000	0.000	0.000	zmqstream.py:684(<lambda>)
4	0.000	0.000	0.000	0.000	attrsettr.py:66(_get_attr_opt)
2	0.000	0.000	0.000	0.000	zmqstream.py:653(_rebuild_io_state)
2	0.000	0.000	0.000	0.000	socket.py:771(recv_multipart)
60	0.000	0.000	0.000	0.000	enum.py:1575(_get_value)
2	0.000	0.000	0.000	0.000	zmqstream.py:676(_update_handler)
1	0.000	0.000	0.000	0.000	cProfile.py:120(__exit__)
175/171	0.000	0.000	0.000	0.000	{built -in method builtins.isinstance}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}
2	0.000	0.000	0.000	0.000	traitlets.py:708(__set__)
2	0.000	0.000	0.000	0.000	traitlets.py:3631(set)
1	0.000	0.000	0.000	0.000	iostream.py:190(_run_event_pipe_gc)
5	0.000	0.000	0.000	0.000	selector_events.py:744(_process_events)
2	0.000	0.000	0.000	0.000	traitlets.py:689(set)
26	0.000	0.000	0.000	0.000	enum.py:676(__call__)
1	0.000	0.000	0.000	0.000	decorator.py:200(fix)
1	0.000	0.000	0.000	0.000	base_events.py:1951(_add_callback)
6	0.000	0.000	0.000	0.000	enum.py:1593(__and__)
4	0.000	0.000	0.000	0.000	typing.py:184(_type_check)
2	0.000	0.000	0.000	0.000	tasks.py:687(sleep)
2	0.000	0.000	0.000	0.000	traitlets.py:718(_validate)
1	0.000	0.000	0.000	0.000	inspect.py:3229(bind)
2	0.000	0.000	0.000	0.000	typing.py:1292(__instancecheck__)
1	0.000	0.000	0.000	0.000	inspect.py:3087(_bind)
1	0.000	0.000	0.000	0.000	iostream.py:682(_flush)
26	0.000	0.000	0.000	0.000	enum.py:1145(__new__)
1	0.000	0.000	0.000	0.000	traitlets.py:1512(_notify_trait)
1	0.000	0.000	0.000	0.000	futures.py:319(_set_result_unless_cancelled)
1	0.000	0.000	0.000	0.000	iostream.py:199(_event_pipe_gc)
2	0.000	0.000	0.000	0.000	typing.py:1583(__subclasscheck__)
1	0.000	0.000	0.000	0.000	asyncio.py:231(add_callback)
2	0.000	0.000	0.000	0.000	base_events.py:817(call_soon)
1	0.000	0.000	0.000	0.000	iostream.py:776(_flush_buffers)
1	0.000	0.000	0.000	0.000	traitlets.py:1523(notify_change)
2	0.000	0.000	0.000	0.000	traitlets.py:3474(validate)

32	0.000	0.000	0.000	0.000	typing.py:1355(__eq__)
1	0.000	0.000	0.000	0.000	base_events.py:775(call_later)
9	0.000	0.000	0.000	0.000	base_events.py:766(time)
2	0.000	0.000	0.000	0.000	base_events.py:846(_call_soon)
1	0.000	0.000	0.000	0.000	traitlets.py:1527(_notify_observers)
1	0.000	0.000	0.000	0.000	contextlib.py:303(helper)
4	0.000	0.000	0.000	0.000	traitlets.py:676(__get__)
1	0.000	0.000	0.000	0.000	iostream.py:784(_rotate_buffers)
2	0.000	0.000	0.000	0.000	iostream.py:288(_check_mp_mode)
2	0.000	0.000	0.000	0.000	zmqstream.py:532(sending)
1	0.000	0.000	0.000	0.000	{method 'set_result' of 'asyncio.Future' object
4	0.000	0.000	0.000	0.000	<frozen importlib._bootstrap>:1401(_handle_froml
1	0.000	0.000	0.000	0.000	base_events.py:799(call_at)
3	0.000	0.000	0.000	0.000	events.py:41(__init__)
2	0.000	0.000	0.000	0.000	traitlets.py:727(_cross_validate)
2	0.000	0.000	0.000	0.000	{built -in method builtins.issubclass}
2	0.000	0.000	0.000	0.000	queue.py:112(empty)
4	0.000	0.000	0.000	0.000	typing.py:396(inner)
2	0.000	0.000	0.000	0.000	iostream.py:285(_is_master_process)
1	0.000	0.000	0.000	0.000	history.py:1047(_writeout_output_cache)
9	0.000	0.000	0.000	0.000	{built -in method builtins.hasattr}
8	0.000	0.000	0.000	0.000	{built -in method builtins.next}
1	0.000	0.000	0.000	0.000	contextlib.py:108(__init__)
1	0.000	0.000	0.000	0.000	inspect.py:2891(apply_defaults)
2	0.000	0.000	0.000	0.000	<frozen abc>:121(__subclasscheck__)
1	0.000	0.000	0.000	0.000	threading.py:325(_acquire_restore)
5	0.000	0.000	0.000	0.000	{built -in method builtins.getattr}
3	0.000	0.000	0.000	0.000	threading.py:1177(is_alive)
12	0.000	0.000	0.000	0.000	typing.py:1361(__hash__)
2	0.000	0.000	0.000	0.000	traitlets.py:3624(validate_elements)
1	0.000	0.000	0.000	0.000	events.py:190(cancel)
1	0.000	0.000	0.000	0.000	contextlib.py:145(__exit__)
1	0.000	0.000	0.000	0.000	events.py:146(__init__)
1	0.000	0.000	0.000	0.000	base_events.py:457(create_future)
7	0.000	0.000	0.000	0.000	{method '__enter__' of 'thread.lock' objects}
4	0.000	0.000	0.000	0.000	traitlets.py:629(get)
2	0.000	0.000	0.000	0.000	traitlets.py:2304(validate)
7	0.000	0.000	0.000	0.000	{method 'popleft' of 'collections.deque' objects}
4	0.000	0.000	0.000	0.000	typing.py:175(_type_convert)
9	0.000	0.000	0.000	0.000	{built -in method time.monotonic}
1	0.000	0.000	0.000	0.000	contextlib.py:136(__enter__)
2	0.000	0.000	0.000	0.000	{built -in method _abc._abc_subclasscheck}
1	0.000	0.000	0.000	0.000	inspect.py:2838(args)
1	0.000	0.000	0.000	0.000	threading.py:641(clear)
7	0.000	0.000	0.000	0.000	{method '__exit__' of 'thread.lock' objects}
2	0.000	0.000	0.000	0.000	threading.py:316(__exit__)
5	0.000	0.000	0.000	0.000	{built -in method builtins.max}
2	0.000	0.000	0.000	0.000	{built -in method _heapq.heappop}
6	0.000	0.000	0.000	0.000	{method 'append' of 'collections.deque' objects}
2	0.000	0.000	0.000	0.000	queue.py:266(_qsize)
2	0.000	0.000	0.000	0.000	threading.py:313(__enter__)
5	0.000	0.000	0.000	0.000	{method 'get' of 'dict' objects}
1	0.000	0.000	0.000	0.000	{method 'values' of 'mappingproxy' objects}

```

3      0.000      0.000      0.000      0.000 {method 'acquire' of '_thread.lock' objects}
20     0.000      0.000      0.000      0.000 typing.py:2300(cast)
1      0.000      0.000      0.000      0.000 threading.py:322(_release_save)
2      0.000      0.000      0.000      0.000 {built -in method posix.getpid}
2      0.000      0.000      0.000      0.000 traitlets.py:3486(validate_elements)
4      0.000      0.000      0.000      0.000 {method 'upper' of 'str' objects}
1      0.000      0.000      0.000      0.000 inspect.py:2861(kwargs)
12     0.000      0.000      0.000      0.000 {built -in method builtins.hash}
1      0.000      0.000      0.000      0.000 {method '__enter__' of '_thread.RLock' objects}
2      0.000      0.000      0.000      0.000 {built -in method _contextvars.copy_context}
2      0.000      0.000      0.000      0.000 {method 'items' of 'dict' objects}
1      0.000      0.000      0.000      0.000 threading.py:328(_is_owned)
2      0.000      0.000      0.000      0.000 iostream.py:327(closed)
4      0.000      0.000      0.000      0.000 zmqstream.py:528(receiving)
10     0.000      0.000      0.000      0.000 inspect.py:2744(kind)
2      0.000      0.000      0.000      0.000 {method 'extend' of 'list' objects}
3      0.000      0.000      0.000      0.000 {method 'items' of 'mappingproxy' objects}
2      0.000      0.000      0.000      0.000 {built -in method builtins.iter}
2      0.000      0.000      0.000      0.000 {built -in method _asyncio.get_running_loop}
1      0.000      0.000      0.000      0.000 events.py:78(cancel)
2      0.000      0.000      0.000      0.000 history.py:1100(hold)
5      0.000      0.000      0.000      0.000 base_events.py:2064(get_debug)
1      0.000      0.000      0.000      0.000 {built -in method _heapq.heappush}
3      0.000      0.000      0.000      0.000 {method 'is_done' of '_thread._ThreadHandle' obj
1      0.000      0.000      0.000      0.000 {built -in method math.isnan}
1      0.000      0.000      0.000      0.000 {method '__exit__' of '_thread.RLock' objects}
4      0.000      0.000      0.000      0.000 inspect.py:2732(name)
2      0.000      0.000      0.000      0.000 {method '__enter__' of 'sqlite3.Connection' obje
3      0.000      0.000      0.000      0.000 threading.py:615(is_set)
1      0.000      0.000      0.000      0.000 {built -in method _thread.allocate_lock}
4      0.000      0.000      0.000      0.000 inspect.py:3041(parameters)
3      0.000      0.000      0.000      0.000 base_events.py:548(_check_closed)
1      0.000      0.000      0.000      0.000 {method 'cancelled' of '_asyncio.Future' objects}
1      0.000      0.000      0.000      0.000 base_events.py:1961(_timer_handle_cancelled)
1      0.000      0.000      0.000      0.000 {method 'release' of '_thread.lock' objects}
1      0.000      0.000      0.000      0.000 inspect.py:2830(__init__)

```

<pstats.Stats at 0x1155d9450>

Die Funktion `len` verbraucht 3.421 von total 12.716 Sekunden, kumuliert über 52'493'051 aufrufe. Versuchen wir das zu verbessern.

```

def interpolate_linear_slow2(x_data, y_data, x):
    n = len(x_data)
    y = []
    for x_k in x:
        i = 0
        while i < n and x_data[i] < x_k:
            i += 1
        if i == 0:
            i = 1
        elif i == n:
            i = n - 1
        x_lower = x_data[i - 1]

```

```

    x_upper = x_data[i]
    y_lower = y_data[i - 1]
    y_upper = y_data[i]
    yi = (y_upper - y_lower) / (x_upper - x_lower) * (x_k - x_lower) + y_lower
    y.append(yi)
return np.array(y)

```

```

with Profile() as pr:
    y = interpolate_linear_slow2(x_data, y_data, x)
stats = Stats(pr)
stats.strip_dirs()
stats.sort_stats("cumtime")
stats.print_stats()

```

1000684 function calls (1000680 primitive calls) in 2.536 seconds

Ordered by: cumulative time

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
3	0.000	0.000	2.467	0.822	base_events.py:1966(_run_once)
3	0.004	0.001	2.467	0.822	selectors.py:540(select)
1	2.282	2.282	2.334	2.334	559756653.py:1(interpolate_linear_slow2)
2	0.124	0.062	0.129	0.064	{method 'control' of 'select.kqueue' objects}
3	0.000	0.000	0.068	0.023	events.py:92(_run)
3	0.000	0.000	0.068	0.023	{method 'run' of '_contextvars.Context' objects}
2	0.000	0.000	0.068	0.034	zmqstream.py:573(_handle_events)
1	0.000	0.000	0.068	0.068	asyncio.py:206(_handle_events)
2	0.000	0.000	0.068	0.034	zmqstream.py:614(_handle_recv)
2	0.000	0.000	0.068	0.034	zmqstream.py:546(_run_callback)
2	0.000	0.000	0.068	0.034	iostream.py:229(_handle_event)
2	0.006	0.003	0.068	0.034	iostream.py:348(<lambda>)
14	0.050	0.004	0.055	0.004	socket.py:623(send)
1000005	0.048	0.000	0.048	0.000	{method 'append' of 'list' objects}
2	0.000	0.000	0.043	0.022	iostream.py:350(_really_send)
2	0.000	0.000	0.043	0.022	socket.py:700(send_multipart)
1	0.000	0.000	0.018	0.018	decorator.py:232(fun)
1	0.000	0.000	0.018	0.018	history.py:100(only_when_enabled)
1	0.000	0.000	0.018	0.018	history.py:1055(writeout_cache)
1	0.000	0.000	0.018	0.018	history.py:1039(_writeout_input_cache)
1	0.015	0.015	0.015	0.015	{built-in method numpy.array}
2	0.006	0.003	0.006	0.003	{method '__exit__' of 'sqlite3.Connection' objects}
2	0.000	0.000	0.000	0.000	ioloop.py:750(_run_callback)
1	0.000	0.000	0.000	0.000	zmqstream.py:684(<lambda>)
4	0.000	0.000	0.000	0.000	attrsettr.py:43(__getattr__)
4	0.000	0.000	0.000	0.000	attrsettr.py:66(_get_attr_opt)
2	0.000	0.000	0.000	0.000	zmqstream.py:653(_rebuild_io_state)
14	0.000	0.000	0.000	0.000	enum.py:1582(__or__)
2	0.000	0.000	0.000	0.000	socket.py:771(recv_multipart)
2	0.000	0.000	0.000	0.000	zmqstream.py:676(_update_handler)
2	0.000	0.000	0.000	0.000	traitlets.py:708(__set__)
1	0.000	0.000	0.000	0.000	cProfile.py:120(__exit__)
2	0.000	0.000	0.000	0.000	traitlets.py:3631(set)
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

60	0.000	0.000	0.000	0.000	enum.py:1575(_get_value)
175/171	0.000	0.000	0.000	0.000	{built -in method builtins.isinstance}
2	0.000	0.000	0.000	0.000	traitlets.py:689(set)
6	0.000	0.000	0.000	0.000	enum.py:1593(__and__)
1	0.000	0.000	0.000	0.000	decorator.py:200(fix)
26	0.000	0.000	0.000	0.000	enum.py:676(__call__)
4	0.000	0.000	0.000	0.000	typing.py:184(_type_check)
2	0.000	0.000	0.000	0.000	traitlets.py:718(_validate)
1	0.000	0.000	0.000	0.000	inspect.py:3229(bind)
2	0.000	0.000	0.000	0.000	typing.py:1292(__instancecheck__)
1	0.000	0.000	0.000	0.000	inspect.py:3087(_bind)
1	0.000	0.000	0.000	0.000	{method 'execute' of 'sqlite3.Connection' object}
1	0.000	0.000	0.000	0.000	iostream.py:682(_flush)
2	0.000	0.000	0.000	0.000	traitlets.py:3474(validate)
1	0.000	0.000	0.000	0.000	traitlets.py:1512(_notify_trait)
2	0.000	0.000	0.000	0.000	typing.py:1583(__subclasscheck__)
1	0.000	0.000	0.000	0.000	asyncio.py:231(add_callback)
26	0.000	0.000	0.000	0.000	enum.py:1145(__new__)
1	0.000	0.000	0.000	0.000	iostream.py:776(_flush_buffers)
1	0.000	0.000	0.000	0.000	traitlets.py:1523(notify_change)
32	0.000	0.000	0.000	0.000	typing.py:1355(__eq__)
2	0.000	0.000	0.000	0.000	zmqstream.py:532(sending)
1	0.000	0.000	0.000	0.000	traitlets.py:1527(_notify_observers)
2	0.000	0.000	0.000	0.000	{built -in method builtins.issubclass}
1	0.000	0.000	0.000	0.000	iostream.py:784(_rotate_buffers)
1	0.000	0.000	0.000	0.000	base_events.py:817(call_soon)
2	0.000	0.000	0.000	0.000	queue.py:112(empty)
4	0.000	0.000	0.000	0.000	<frozen importlib._bootstrap>:1401(_handle_froml
5	0.000	0.000	0.000	0.000	base_events.py:766(time)
4	0.000	0.000	0.000	0.000	traitlets.py:676(__get__)
2	0.000	0.000	0.000	0.000	traitlets.py:727(_cross_validate)
1	0.000	0.000	0.000	0.000	contextlib.py:303(helper)
4	0.000	0.000	0.000	0.000	typing.py:396(inner)
3	0.000	0.000	0.000	0.000	selector_events.py:744(_process_events)
1	0.000	0.000	0.000	0.000	base_events.py:846(_call_soon)
2	0.000	0.000	0.000	0.000	traitlets.py:3624(validate_elements)
8	0.000	0.000	0.000	0.000	{built -in method builtins.next}
9	0.000	0.000	0.000	0.000	{built -in method builtins.hasattr}
5	0.000	0.000	0.000	0.000	{built -in method builtins.getattr}
2	0.000	0.000	0.000	0.000	<frozen abc>:121(__subclasscheck__)
13	0.000	0.000	0.000	0.000	{built -in method builtins.len}
1	0.000	0.000	0.000	0.000	history.py:1047(_writeout_output_cache)
1	0.000	0.000	0.000	0.000	inspect.py:2891(apply_defaults)
2	0.000	0.000	0.000	0.000	traitlets.py:2304(validate)
1	0.000	0.000	0.000	0.000	threading.py:325(_acquire_restore)
2	0.000	0.000	0.000	0.000	iostream.py:288(_check_mp_mode)
12	0.000	0.000	0.000	0.000	typing.py:1361(__hash__)
1	0.000	0.000	0.000	0.000	contextlib.py:145(__exit__)
1	0.000	0.000	0.000	0.000	contextlib.py:108(__init__)
2	0.000	0.000	0.000	0.000	iostream.py:285(_is_master_process)
1	0.000	0.000	0.000	0.000	contextlib.py:136(__enter__)
6	0.000	0.000	0.000	0.000	{method '__enter__' of 'thread.lock' objects}
4	0.000	0.000	0.000	0.000	traitlets.py:629(get)

```

1    0.000    0.000    0.000    0.000 threading.py:641(clear)
1    0.000    0.000    0.000    0.000 base_events.py:1951(_add_callback)
1    0.000    0.000    0.000    0.000 events.py:41(__init__)
4    0.000    0.000    0.000    0.000 typing.py:175(_type_convert)
2    0.000    0.000    0.000    0.000 {built -in method _abc._abc_subclasscheck}
5    0.000    0.000    0.000    0.000 {method 'get' of 'dict' objects}
2    0.000    0.000    0.000    0.000 threading.py:316(__exit__)
1    0.000    0.000    0.000    0.000 inspect.py:2838(args)
6    0.000    0.000    0.000    0.000 {method '__exit__' of '_thread.lock' objects}
3    0.000    0.000    0.000    0.000 {method 'acquire' of '_thread.lock' objects}
2    0.000    0.000    0.000    0.000 threading.py:313(__enter__)
5    0.000    0.000    0.000    0.000 {method 'popleft' of 'collections.deque' objects}
2    0.000    0.000    0.000    0.000 traitlets.py:3486(validate_elements)
20   0.000    0.000    0.000    0.000 typing.py:2300(cast)
1    0.000    0.000    0.000    0.000 threading.py:322(_release_save)
5    0.000    0.000    0.000    0.000 {built -in method time.monotonic}
4    0.000    0.000    0.000    0.000 {method 'upper' of 'str' objects}
4    0.000    0.000    0.000    0.000 {method 'append' of 'collections.deque' objects}
2    0.000    0.000    0.000    0.000 queue.py:266(_qsize)
3    0.000    0.000    0.000    0.000 {built -in method builtins.max}
1    0.000    0.000    0.000    0.000 {method 'values' of 'mappingproxy' objects}
1    0.000    0.000    0.000    0.000 {method '__enter__' of '_thread.RLock' objects}
1    0.000    0.000    0.000    0.000 inspect.py:2861(kwargs)
12   0.000    0.000    0.000    0.000 {built -in method builtins.hash}
1    0.000    0.000    0.000    0.000 threading.py:328(_is_owned)
2    0.000    0.000    0.000    0.000 {method 'extend' of 'list' objects}
2    0.000    0.000    0.000    0.000 history.py:1100(hold)
1    0.000    0.000    0.000    0.000 {built -in method _heapq.heappop}
2    0.000    0.000    0.000    0.000 {built -in method posix.getpid}
1    0.000    0.000    0.000    0.000 {built -in method _contextvars.copy_context}
10   0.000    0.000    0.000    0.000 inspect.py:2744(kind)
3    0.000    0.000    0.000    0.000 {method 'items' of 'mappingproxy' objects}
4    0.000    0.000    0.000    0.000 zmqstream.py:528(receiving)
2    0.000    0.000    0.000    0.000 {built -in method builtins.iter}
4    0.000    0.000    0.000    0.000 inspect.py:2732(name)
1    0.000    0.000    0.000    0.000 {built -in method _asyncio.get_running_loop}
1    0.000    0.000    0.000    0.000 {method '__exit__' of '_thread.RLock' objects}
2    0.000    0.000    0.000    0.000 iostream.py:327(closed)
4    0.000    0.000    0.000    0.000 inspect.py:3041(parameters)
2    0.000    0.000    0.000    0.000 {method '__enter__' of 'sqlite3.Connection' objects}
1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}
1    0.000    0.000    0.000    0.000 inspect.py:2830(__init__)
1    0.000    0.000    0.000    0.000 {built -in method _thread.allocate_lock}
1    0.000    0.000    0.000    0.000 {method 'release' of '_thread.lock' objects}
1    0.000    0.000    0.000    0.000 base_events.py:548(_check_closed)
1    0.000    0.000    0.000    0.000 base_events.py:2064(get_debug)

```

<pstats.Stats at 0x1156ba490>

Das hat sehr viel geholfen. Der grösste Anteil der total noch 4.606 s wird jetzt in der Methode `append` der Liste `y` verbraucht (0.105 s kumuliert über 1'000'000 Aufrufe).

Da wir die Länge dieses Arrays kennen, können wir es vorher mit `np.empty_like` initialisieren.

```
def interpolate_linear_slow3(x_data, y_data, x):
```

```

n = len(x_data)
y = np.empty_like(x)
for k, x_k in enumerate(x):
    i = 0
    while i < n and x_data[i] < x_k:
        i += 1
    if i == 0:
        i = 1
    elif i == n:
        i = n - 1
    x_lower = x_data[i - 1]
    x_upper = x_data[i]
    y_lower = y_data[i - 1]
    y_upper = y_data[i]
    y[k] = (y_upper - y_lower) / (x_upper - x_lower) * (x_k - x_lower) + y_lower
return y

```

```

with Profile() as pr:
    y = interpolate_linear_slow3(x_data, y_data, x)
stats = Stats(pr)
stats.strip_dirs()
stats.sort_stats("cumtime")
stats.print_stats()

```

684 function calls (680 primitive calls) in 2.444 seconds

Ordered by: cumulative time

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
3	0.000	0.000	2.350	0.783	base_events.py:1966(_run_once)
3	0.000	0.000	2.350	0.783	selectors.py:540(select)
1	2.247	2.247	2.247	2.247	3589582048.py:1(interpolate_linear_slow3)
2	0.103	0.051	0.103	0.051	{method 'control' of 'select.kqueue' objects}
3	0.000	0.000	0.094	0.031	events.py:92(_run)
3	0.000	0.000	0.094	0.031	{method 'run' of '_contextvars.Context' objects}
2	0.000	0.000	0.094	0.047	zmqstream.py:573(_handle_events)
1	0.000	0.000	0.094	0.094	asyncio.py:206(_handle_events)
2	0.000	0.000	0.094	0.047	zmqstream.py:614(_handle_recv)
2	0.000	0.000	0.094	0.047	zmqstream.py:546(_run_callback)
2	0.000	0.000	0.094	0.047	iostream.py:229(_handle_event)
2	0.006	0.003	0.094	0.047	iostream.py:348(<lambda>)
14	0.074	0.005	0.074	0.005	socket.py:623(send)
2	0.000	0.000	0.068	0.034	iostream.py:350(_really_send)
2	0.000	0.000	0.068	0.034	socket.py:700(send_multipart)
1	0.000	0.000	0.019	0.019	decorator.py:232(fun)
1	0.000	0.000	0.019	0.019	history.py:100(only_when_enabled)
1	0.000	0.000	0.019	0.019	history.py:1055(writeout_cache)
1	0.000	0.000	0.019	0.019	history.py:1039(_writeout_input_cache)
1	0.006	0.006	0.006	0.006	{method 'execute' of 'sqlite3.Connection' object}
2	0.006	0.003	0.006	0.003	{method '__exit__' of 'sqlite3.Connection' object}
4	0.000	0.000	0.000	0.000	attrsettr.py:43(_getattr__)
2	0.000	0.000	0.000	0.000	ioloop.py:750(_run_callback)
4	0.000	0.000	0.000	0.000	attrsettr.py:66(_get_attr_opt)

1	0.000	0.000	0.000	0.000	zmqstream.py:684(<lambda>)
2	0.000	0.000	0.000	0.000	zmqstream.py:653(_rebuild_io_state)
14	0.000	0.000	0.000	0.000	enum.py:1582(__or__)
2	0.000	0.000	0.000	0.000	socket.py:771(recv_multipart)
2	0.000	0.000	0.000	0.000	zmqstream.py:676(_update_handler)
1	0.000	0.000	0.000	0.000	cProfile.py:120(__exit__)
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}
60	0.000	0.000	0.000	0.000	enum.py:1575(_get_value)
175/171	0.000	0.000	0.000	0.000	{built -in method builtins.isinstance}
2	0.000	0.000	0.000	0.000	traitlets.py:708(__set__)
2	0.000	0.000	0.000	0.000	traitlets.py:3631(set)
2	0.000	0.000	0.000	0.000	traitlets.py:689(set)
1	0.000	0.000	0.000	0.000	decorator.py:200(fix)
26	0.000	0.000	0.000	0.000	enum.py:676(__call__)
4	0.000	0.000	0.000	0.000	typing.py:184(_type_check)
6	0.000	0.000	0.000	0.000	enum.py:1593(__and__)
2	0.000	0.000	0.000	0.000	traitlets.py:718(_validate)
1	0.000	0.000	0.000	0.000	inspect.py:3229(bind)
2	0.000	0.000	0.000	0.000	typing.py:1292(__instancecheck__)
1	0.000	0.000	0.000	0.000	inspect.py:3087(_bind)
1	0.000	0.000	0.000	0.000	iostream.py:682(_flush)
2	0.000	0.000	0.000	0.000	typing.py:1583(__subclasscheck__)
1	0.000	0.000	0.000	0.000	asyncio.py:231(add_callback)
1	0.000	0.000	0.000	0.000	traitlets.py:1512(_notify_trait)
26	0.000	0.000	0.000	0.000	enum.py:1145(__new__)
32	0.000	0.000	0.000	0.000	typing.py:1355(__eq__)
2	0.000	0.000	0.000	0.000	traitlets.py:3474(validate)
1	0.000	0.000	0.000	0.000	iostream.py:776(_flush_buffers)
1	0.000	0.000	0.000	0.000	traitlets.py:1523(notify_change)
2	0.000	0.000	0.000	0.000	zmqstream.py:532(sending)
1	0.000	0.000	0.000	0.000	base_events.py:817(call_soon)
4	0.000	0.000	0.000	0.000	traitlets.py:676(__get__)
1	0.000	0.000	0.000	0.000	traitlets.py:1527(_notify_observers)
2	0.000	0.000	0.000	0.000	{built -in method builtins.issubclass}
1	0.000	0.000	0.000	0.000	contextlib.py:303(helper)
1	0.000	0.000	0.000	0.000	iostream.py:784(_rotate_buffers)
3	0.000	0.000	0.000	0.000	selector_events.py:744(_process_events)
2	0.000	0.000	0.000	0.000	queue.py:112(empty)
4	0.000	0.000	0.000	0.000	<frozen importlib._bootstrap>:1401(_handle_from_l)
2	0.000	0.000	0.000	0.000	traitlets.py:727(_cross_validate)
5	0.000	0.000	0.000	0.000	base_events.py:766(time)
4	0.000	0.000	0.000	0.000	typing.py:396(inner)
1	0.000	0.000	0.000	0.000	base_events.py:846(_call_soon)
2	0.000	0.000	0.000	0.000	<frozen abc>:121(__subclasscheck__)
2	0.000	0.000	0.000	0.000	iostream.py:288(_check_mp_mode)
9	0.000	0.000	0.000	0.000	{built -in method builtins.hasattr}
5	0.000	0.000	0.000	0.000	{built -in method builtins.getattr}
1	0.000	0.000	0.000	0.000	inspect.py:2891(apply_defaults)
1	0.000	0.000	0.000	0.000	threading.py:325(_acquire_restore)
2	0.000	0.000	0.000	0.000	traitlets.py:3624(validate_elements)
1	0.000	0.000	0.000	0.000	contextlib.py:108(__init__)
8	0.000	0.000	0.000	0.000	{built -in method builtins.next}
12	0.000	0.000	0.000	0.000	typing.py:1361(__hash__)

13	0.000	0.000	0.000	0.000	{built -in method builtins.len}
1	0.000	0.000	0.000	0.000	history.py:1047(_writeout_output_cache)
2	0.000	0.000	0.000	0.000	iostream.py:285(_is_master_process)
1	0.000	0.000	0.000	0.000	contextlib.py:136(__enter__)
4	0.000	0.000	0.000	0.000	traitlets.py:629(get)
1	0.000	0.000	0.000	0.000	contextlib.py:145(__exit__)
6	0.000	0.000	0.000	0.000	{method '__enter__' of '_thread.lock' objects}
2	0.000	0.000	0.000	0.000	{built -in method _abc._abc_subclasscheck}
1	0.000	0.000	0.000	0.000	inspect.py:2838(args)
1	0.000	0.000	0.000	0.000	events.py:41(__init__)
1	0.000	0.000	0.000	0.000	base_events.py:1951(_add_callback)
2	0.000	0.000	0.000	0.000	threading.py:316(__exit__)
4	0.000	0.000	0.000	0.000	typing.py:175(_type_convert)
2	0.000	0.000	0.000	0.000	traitlets.py:2304(validate)
6	0.000	0.000	0.000	0.000	{method '__exit__' of '_thread.lock' objects}
1	0.000	0.000	0.000	0.000	threading.py:641(clear)
2	0.000	0.000	0.000	0.000	traitlets.py:3486(validate_elements)
4	0.000	0.000	0.000	0.000	{method 'append' of 'collections.deque' objects}
5	0.000	0.000	0.000	0.000	{method 'get' of 'dict' objects}
5	0.000	0.000	0.000	0.000	{method 'popleft' of 'collections.deque' objects}
3	0.000	0.000	0.000	0.000	{method 'acquire' of '_thread.lock' objects}
5	0.000	0.000	0.000	0.000	{built -in method time.monotonic}
5	0.000	0.000	0.000	0.000	{method 'append' of 'list' objects}
2	0.000	0.000	0.000	0.000	threading.py:313(__enter__)
1	0.000	0.000	0.000	0.000	{built -in method _heapq.heappop}
1	0.000	0.000	0.000	0.000	threading.py:322(_release_save)
20	0.000	0.000	0.000	0.000	typing.py:2300(cast)
1	0.000	0.000	0.000	0.000	inspect.py:2861(kwargs)
2	0.000	0.000	0.000	0.000	queue.py:266(_qsize)
12	0.000	0.000	0.000	0.000	{built -in method builtins.hash}
4	0.000	0.000	0.000	0.000	{method 'upper' of 'str' objects}
3	0.000	0.000	0.000	0.000	{built -in method builtins.max}
1	0.000	0.000	0.000	0.000	{method 'values' of 'mappingproxy' objects}
2	0.000	0.000	0.000	0.000	{method 'extend' of 'list' objects}
2	0.000	0.000	0.000	0.000	{built -in method posix.getpid}
1	0.000	0.000	0.000	0.000	{method '__enter__' of '_thread.RLock' objects}
2	0.000	0.000	0.000	0.000	history.py:1100(hold)
1	0.000	0.000	0.000	0.000	threading.py:328(_is_owned)
10	0.000	0.000	0.000	0.000	inspect.py:2744(kind)
3	0.000	0.000	0.000	0.000	{method 'items' of 'mappingproxy' objects}
4	0.000	0.000	0.000	0.000	zmqstream.py:528(receiving)
2	0.000	0.000	0.000	0.000	{built -in method builtins.iter}
1	0.000	0.000	0.000	0.000	{method 'items' of 'dict' objects}
1	0.000	0.000	0.000	0.000	{built -in method _contextvars.copy_context}
1	0.000	0.000	0.000	0.000	{built -in method _asyncio.get_running_loop}
1	0.000	0.000	0.000	0.000	{method '__exit__' of '_thread.RLock' objects}
4	0.000	0.000	0.000	0.000	inspect.py:3041(parameters)
4	0.000	0.000	0.000	0.000	inspect.py:2732(name)
2	0.000	0.000	0.000	0.000	{method '__enter__' of 'sqlite3.Connection' objects}
1	0.000	0.000	0.000	0.000	{built -in method _thread.allocate_lock}
2	0.000	0.000	0.000	0.000	iostream.py:327(closed)
1	0.000	0.000	0.000	0.000	base_events.py:548(_check_closed)
1	0.000	0.000	0.000	0.000	base_events.py:2064(get_debug)

```

1      0.000      0.000      0.000      0.000 {method 'release' of '_thread.lock' objects}
1      0.000      0.000      0.000      0.000 multiarray.py:115(empty_like)
1      0.000      0.000      0.000      0.000 inspect.py:2830(__init__)

```

```
<pstats.Stats at 0x1156e0050>
```

Allerdings wird nun die restliche Zeit durch die Rechenoperationen, den for-loop und den while-loop verbraucht. Die nächste Verbesserung wäre also, `np.searchsorted` statt den while-loop zu benutzen.

```

def interpolate_linear_slow4(x_data, y_data, x):
    n = len(x_data)
    y = np.empty_like(x)
    for k, x_k in enumerate(x):
        i = np.searchsorted(x_data, x_k)
        if i == 0:
            i = 1
        elif i == n:
            i = n - 1
        x_lower = x_data[i - 1]
        x_upper = x_data[i]
        y_lower = y_data[i - 1]
        y_upper = y_data[i]
        y[k] = (y_upper - y_lower) / (x_upper - x_lower) * (x_k - x_lower) + y_lower
    return y

```

```

with Profile() as pr:
    y = interpolate_linear_slow4(x_data, y_data, x)
stats = Stats(pr)
stats.strip_dirs()
stats.sort_stats("cumtime")
stats.print_stats()

```

5000754 function calls (5000749 primitive calls) in 1.332 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
2/1	0.605	0.303	1.324	1.324	2669866335.py:1(interpolate_linear_slow4)
1000000	0.171	0.000	0.687	0.000	fromnumeric.py:1421(searchsorted)
1000000	0.162	0.000	0.516	0.000	fromnumeric.py:48(_wrapfunc)
1000000	0.312	0.000	0.312	0.000	{method 'searchsorted' of 'numpy.ndarray' object}
1000005	0.042	0.000	0.042	0.000	{built-in method builtins.getattr}
1000000	0.038	0.000	0.038	0.000	fromnumeric.py:1417(_searchsorted_dispatcher)
2	0.001	0.001	0.011	0.005	iostream.py:348(<lambda>)
5	0.000	0.000	0.008	0.002	events.py:92(_run)
5	0.000	0.000	0.008	0.002	{method 'run' of '_contextvars.Context' objects}
2	0.000	0.000	0.008	0.004	zmqstream.py:573(_handle_events)
1	0.000	0.000	0.008	0.008	asyncio.py:206(_handle_events)
2	0.000	0.000	0.008	0.004	zmqstream.py:614(_handle_recv)
2	0.000	0.000	0.008	0.004	zmqstream.py:546(_run_callback)
2	0.000	0.000	0.008	0.004	iostream.py:229(_handle_event)
1	0.000	0.000	0.005	0.005	decorator.py:232(fun)
1	0.000	0.000	0.002	0.002	history.py:100(only_when_enabled)

4	0.000	0.000	0.000	0.000	base_events.py:1966(_run_once)
2	0.000	0.000	0.000	0.000	ioloop.py:750(_run_callback)
4	0.000	0.000	0.000	0.000	attrsettr.py:43(__getattr__)
2	0.000	0.000	0.000	0.000	zmqstream.py:653(_rebuild_io_state)
1	0.000	0.000	0.000	0.000	zmqstream.py:684(<lambda>)
4	0.000	0.000	0.000	0.000	attrsettr.py:66(_get_attr_opt)
2	0.000	0.000	0.000	0.000	zmqstream.py:676(_update_handler)
14	0.000	0.000	0.000	0.000	socket.py:623(send)
2	0.000	0.000	0.000	0.000	socket.py:771(recv_multipart)
1	0.000	0.000	0.000	0.000	history.py:1055(writeout_cache)
1	0.000	0.000	0.000	0.000	cProfile.py:120(__exit__)
14	0.000	0.000	0.000	0.000	enum.py:1582(__or__)
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}
1	0.000	0.000	0.000	0.000	iostream.py:190(_run_event_pipe_gc)
2	0.000	0.000	0.000	0.000	traitlets.py:708(__set__)
2	0.000	0.000	0.000	0.000	traitlets.py:3631(set)
2	0.000	0.000	0.000	0.000	traitlets.py:689(set)
60	0.000	0.000	0.000	0.000	enum.py:1575(_get_value)
175/171	0.000	0.000	0.000	0.000	{built -in method builtins.isinstance}
1	0.000	0.000	0.000	0.000	decorator.py:200(fix)
4	0.000	0.000	0.000	0.000	typing.py:184(_type_check)
6	0.000	0.000	0.000	0.000	enum.py:1593(__and__)
2	0.000	0.000	0.000	0.000	tasks.py:687(sleep)
26	0.000	0.000	0.000	0.000	enum.py:676(__call__)
4	0.000	0.000	0.000	0.000	selectors.py:540(select)
1	0.000	0.000	0.000	0.000	asyncio.py:231(add_callback)
2	0.000	0.000	0.000	0.000	traitlets.py:718(_validate)
1	0.000	0.000	0.000	0.000	inspect.py:3229(bind)
1	0.000	0.000	0.000	0.000	futures.py:319(_set_result_unless_cancelled)
1	0.000	0.000	0.000	0.000	iostream.py:682(_flush)
1	0.000	0.000	0.000	0.000	inspect.py:3087(_bind)
1	0.000	0.000	0.000	0.000	iostream.py:199(_event_pipe_gc)
2	0.000	0.000	0.000	0.000	base_events.py:817(call_soon)
4	0.000	0.000	0.000	0.000	{method 'control' of 'select.kqueue' objects}
1	0.000	0.000	0.000	0.000	traitlets.py:1512(_notify_trait)
1	0.000	0.000	0.000	0.000	base_events.py:775(call_later)
32	0.000	0.000	0.000	0.000	typing.py:1355(__eq__)
2	0.000	0.000	0.000	0.000	iostream.py:350(_really_send)
2	0.000	0.000	0.000	0.000	base_events.py:846(_call_soon)
2	0.000	0.000	0.000	0.000	traitlets.py:3474(validate)
1	0.000	0.000	0.000	0.000	traitlets.py:1523(notify_change)
2	0.000	0.000	0.000	0.000	typing.py:1292(__instancecheck__)
1	0.000	0.000	0.000	0.000	iostream.py:776(_flush_buffers)
1	0.000	0.000	0.000	0.000	{method 'set_result' of '_asyncio.Future' object}
9	0.000	0.000	0.000	0.000	base_events.py:766(time)
2	0.000	0.000	0.000	0.000	socket.py:700(send_multipart)
26	0.000	0.000	0.000	0.000	enum.py:1145(__new__)
2	0.000	0.000	0.000	0.000	typing.py:1583(__subclasscheck__)
1	0.000	0.000	0.000	0.000	traitlets.py:1527(_notify_observers)
1	0.000	0.000	0.000	0.000	history.py:1039(_writeout_input_cache)
1	0.000	0.000	0.000	0.000	base_events.py:799(call_at)
2	0.000	0.000	0.000	0.000	{method '__exit__' of 'sqlite3.Connection' object}
3	0.000	0.000	0.000	0.000	events.py:41(__init__)

4	0.000	0.000	0.000	0.000	traitlets.py:676(__get__)
1	0.000	0.000	0.000	0.000	iostream.py:784(_rotate_buffers)
4	0.000	0.000	0.000	0.000	<frozen importlib._bootstrap>:1401(_handle_froml
3	0.000	0.000	0.000	0.000	threading.py:1177(is_alive)
2	0.000	0.000	0.000	0.000	zmqstream.py:532(sending)
5	0.000	0.000	0.000	0.000	selector_events.py:744(_process_events)
2	0.000	0.000	0.000	0.000	iostream.py:288(_check_mp_mode)
8	0.000	0.000	0.000	0.000	{built -in method builtins.next}
2	0.000	0.000	0.000	0.000	traitlets.py:727(_cross_validate)
1	0.000	0.000	0.000	0.000	{method 'execute' of 'sqlite3.Connection' object
2	0.000	0.000	0.000	0.000	{built -in method builtins.issubclass}
1	0.000	0.000	0.000	0.000	inspect.py:2891(apply_defaults)
4	0.000	0.000	0.000	0.000	typing.py:396(inner)
2	0.000	0.000	0.000	0.000	queue.py:112(empty)
12	0.000	0.000	0.000	0.000	typing.py:1361(__hash__)
1	0.000	0.000	0.000	0.000	contextlib.py:303(helper)
1	0.000	0.000	0.000	0.000	events.py:190(cancel)
2	0.000	0.000	0.000	0.000	traitlets.py:3624(validate_elements)
1	0.000	0.000	0.000	0.000	events.py:146(__init__)
17	0.000	0.000	0.000	0.000	{built -in method builtins.len}
1	0.000	0.000	0.000	0.000	contextlib.py:145(__exit__)
2	0.000	0.000	0.000	0.000	iostream.py:285(_is_master_process)
1	0.000	0.000	0.000	0.000	history.py:1047(_writeout_output_cache)
2	0.000	0.000	0.000	0.000	<frozen abc>:121(__subclasscheck__)
7	0.000	0.000	0.000	0.000	{method '__enter__' of '_thread.lock' objects}
1	0.000	0.000	0.000	0.000	inspect.py:2838(args)
1	0.000	0.000	0.000	0.000	contextlib.py:136(__enter__)
9	0.000	0.000	0.000	0.000	{built -in method builtins.hasattr}
4	0.000	0.000	0.000	0.000	traitlets.py:629(get)
1	0.000	0.000	0.000	0.000	threading.py:325(_acquire_restore)
1	0.000	0.000	0.000	0.000	threading.py:641(clear)
1	0.000	0.000	0.000	0.000	base_events.py:457(create_future)
2	0.000	0.000	0.000	0.000	traitlets.py:2304(validate)
4	0.000	0.000	0.000	0.000	typing.py:175(_type_convert)
1	0.000	0.000	0.000	0.000	contextlib.py:108(__init__)
2	0.000	0.000	0.000	0.000	threading.py:316(__exit__)
7	0.000	0.000	0.000	0.000	{method 'popleft' of 'collections.deque' objects}
9	0.000	0.000	0.000	0.000	{built -in method time.monotonic}
1	0.000	0.000	0.000	0.000	base_events.py:1951(_add_callback)
7	0.000	0.000	0.000	0.000	{method '__exit__' of '_thread.lock' objects}
6	0.000	0.000	0.000	0.000	{method 'append' of 'collections.deque' objects}
2	0.000	0.000	0.000	0.000	{built -in method _abc._abc_subclasscheck}
1	0.000	0.000	0.000	0.000	threading.py:322(_release_save)
2	0.000	0.000	0.000	0.000	threading.py:313(__enter__)
2	0.000	0.000	0.000	0.000	{built -in method _heapq.heappop}
1	0.000	0.000	0.000	0.000	inspect.py:2861(kwargs)
20	0.000	0.000	0.000	0.000	typing.py:2300(cast)
5	0.000	0.000	0.000	0.000	{method 'get' of 'dict' objects}
5	0.000	0.000	0.000	0.000	{method 'append' of 'list' objects}
5	0.000	0.000	0.000	0.000	{built -in method builtins.max}
12	0.000	0.000	0.000	0.000	{built -in method builtins.hash}
3	0.000	0.000	0.000	0.000	{method 'acquire' of '_thread.lock' objects}
1	0.000	0.000	0.000	0.000	{method '__enter__' of '_thread.RLock' objects}

```

2      0.000      0.000      0.000      0.000 queue.py:266(_qsize)
2      0.000      0.000      0.000      0.000 traitlets.py:3486(validate_elements)
1      0.000      0.000      0.000      0.000 {method 'values' of 'mappingproxy' objects}
2      0.000      0.000      0.000      0.000 {built -in method _contextvars.copy_context}
2      0.000      0.000      0.000      0.000 {method 'items' of 'dict' objects}
2      0.000      0.000      0.000      0.000 {built -in method _asyncio.get_running_loop}
10     0.000      0.000      0.000      0.000 inspect.py:2744(kind)
2      0.000      0.000      0.000      0.000 {method 'extend' of 'list' objects}
4      0.000      0.000      0.000      0.000 {method 'upper' of 'str' objects}
3      0.000      0.000      0.000      0.000 {method 'items' of 'mappingproxy' objects}
2      0.000      0.000      0.000      0.000 history.py:1100(hold)
1      0.000      0.000      0.000      0.000 {built -in method math.isnan}
2      0.000      0.000      0.000      0.000 {built -in method posix.getpid}
1      0.000      0.000      0.000      0.000 events.py:78(cancel)
3      0.000      0.000      0.000      0.000 {method 'is_done' of '_thread._ThreadHandle' obj
1      0.000      0.000      0.000      0.000 threading.py:328(_is_owned)
1      0.000      0.000      0.000      0.000 {built -in method _heapq.heappush}
4      0.000      0.000      0.000      0.000 zmqstream.py:528(receiving)
2      0.000      0.000      0.000      0.000 {built -in method builtins.iter}
1      0.000      0.000      0.000      0.000 {method '__exit__' of '_thread.RLock' objects}
4      0.000      0.000      0.000      0.000 inspect.py:3041(parameters)
3      0.000      0.000      0.000      0.000 base_events.py:548(_check_closed)
5      0.000      0.000      0.000      0.000 base_events.py:2064(get_debug)
4      0.000      0.000      0.000      0.000 inspect.py:2732(name)
2      0.000      0.000      0.000      0.000 {method '__enter__' of 'sqlite3.Connection' obje
2      0.000      0.000      0.000      0.000 iostream.py:327(closed)
1      0.000      0.000      0.000      0.000 {method 'cancelled' of '_asyncio.Future' objects
1      0.000      0.000      0.000      0.000 {built -in method _thread.allocate_lock}
1      0.000      0.000      0.000      0.000 {method 'release' of '_thread.lock' objects}
3      0.000      0.000      0.000      0.000 threading.py:615(is_set)
1      0.000      0.000      0.000      0.000 inspect.py:2830(__init__)
1      0.000      0.000      0.000      0.000 multiarray.py:115(empty_like)
1      0.000      0.000      0.000      0.000 base_events.py:1961(_timer_handle_cancelled)

```

```
<pstats.Stats at 0x115587e10>
```

Und dann noch den for-loop durch eine vektorisierte Implementierung ersetzen. Das ist genau das, was wir in der schnellen Version `interpolate_linear` bereits gemacht haben.

4.3 Visualisierung mit snakeviz Mit dem Package `snakeviz` können wir die Profiling-Daten visualisieren. Zuerst installieren

```
pip install snakeviz
```

und dann können wir Snakeviz direkt mit `%load_ext snakeviz` aktivieren und mit `%snakeviz -t` pro Zeile oder `%%snakeviz -t` pro Zelle im Jupyter Notebook benutzen. Wegen der Option `-t` wird die Visualisierung im Browser geöffnet. (Ohne `-t` müsste die Visualisierung eigentlich direkt im Notebook dargestellt werden, funktioniert bei mir aber nicht).

Mehr infos z.B. hier:

- <https://jiffyclub.github.io/snakeviz/>
- <https://coderzcolumn.com/tutorials/python/snakeviz-visualize-profiling-results-in-python>

```
%load_ext snakeviz
```

```
%snakeviz -t interpolate_linear_slow2(x_data, y_data, x)
```

```
*** Profile stats marshalled to file '/var/folders/p7/c2v89_m12k7dbrfs0m28h1640000gr/T/tmpjqos
Opening SnakeViz in a new tab...
snakeviz web server started on 127.0.0.1:8080; enter Ctrl -C to exit
http://127.0.0.1:8080/snakeviz/%2Fvar%2Ffolders%2Fp7%2Fc2v89_m12k7dbrfs0m28h1640000gr%2FT%2Ftm
%%snakeviz -t
```

```
interpolate_linear_slow1(x_data, y_data, x)
```

```
*** Profile stats marshalled to file '/var/folders/p7/c2v89_m12k7dbrfs0m28h1640000gr/T/tmpzmk0
Opening SnakeViz in a new tab...
snakeviz web server started on 127.0.0.1:8080; enter Ctrl -C to exit
http://127.0.0.1:8080/snakeviz/%2Fvar%2Ffolders%2Fp7%2Fc2v89_m12k7dbrfs0m28h1640000gr%2FT%2Ftm
```

Leider eignet sich dieses Beispiel nicht so gut für die Visualisierung. Profiling ist dann sinnvoll, wenn man komplexere Algorithmen oder Anwendungen nach Performance-Engpässen analysieren möchte. Die Visualisierung ist dann sehr hilfreich, um die Engpässe zu finden und die Performance zu verbessern.

0.11 Zufallszahlen Erzeugen

0.11.1 Zufallszahlen Erzeugen

Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Das Ziel ist, ein einfaches Verständnis für die Erzeugung von Zufallszahlen zu bekommen. Dazu soll die “Middle-Square-Methode” implementiert und untersucht werden.

```
import numpy as np
import matplotlib.pyplot as plt
```

Pseudozufallszahlen

Die Erzeugung von Zufallszahlen ist ein wichtiger Bestandteil vieler Anwendungen:

- **Simulationen** von biologischen, chemischen oder physikalischen Prozessen, aber auch von Wirtschaftssystemen benötigen Zufallszahlen.
- **Optimierungsverfahren** verwenden Zufallszahlen, um verschiedene Startwerte zu erzeugen und im Suchraum umherzuspringen.
- **Verifikation** von Algorithmen, Software und Hardware wird mithilfe von Testdaten gemacht, die auf Zufallszahlen basieren.
- **Statistische Analysen** benötigen Zufallszahlen, beispielsweise um Signifikanz und Konfidenzintervalle zu schätzen.
- In der **Kryptographie** werden Schlüssel für die Verschlüsselung von Daten mithilfe von Zufallszahlen erzeugt werden.
- In **Computerspielen** werden zufällige Ereignisse und Ergebnisse mit Zufallszahlen generiert.

Echte Zufallszahlen werden durch natürliche Phänomene wie Atmosphärenrauschen, Radioaktivität oder die stochastische Natur der Quantenphysik erzeugt. Diese Zufallszahlen sind wirklich zufällig und können nicht vorhergesagt werden. Sie sind für viele Anwendungen nicht geeignet, da sie nicht praktikabel sind, zu viel Zeit pro Zufallszahl benötigen und nicht reproduzierbar sind. Allerdings können sie in der Kryptographie verwendet werden, um Schlüssel zu erzeugen, die nicht vorhergesagt werden können. Quantencomputer können also wirklich zufällige Zahlen erzeugen, weshalb Quantencomputer im Begriff sind, die Kryptographie zu revolutionieren.

Digitale Computer hingegen können keine wirklich zufälligen Zahlen erzeugen, da sie deterministisch arbeiten. (*Deterministisch* bedeutet, dass für dieselbe Eingabe immer die gleiche Ausgabe erzeugt wird. Das Gegenteil ist *stochastisch*, was bedeutet, dass die Ausgabe nicht vorhersagbar ist.)

Pseudozufallszahlen werden durch deterministische Algorithmen generiert, die jedoch zufällig genug für viele Anwendungen sind. Sie sind reproduzierbar und können daher für Simulationen, Tests und statistische Analysen verwendet werden.

Es gibt verschiedene Algorithmen zur Erzeugung von Pseudozufallszahlen, darunter die Middle-Square Methode (MSM), der Linear Congruential Generator (LCG) und der Mersenne Twister.

Bei der Erzeugung von Zufallszahlen ist es wichtig, dass der Algorithmus eine ausreichend lange Sequenz von Zahlen erzeugt, die sich nicht wiederholt, *Periode* genannt. Ausserdem müssen die generierten Zahlen statistisch unabhängig und gleichverteilt sein, um ein zufälliges Verhalten zu gewährleisten.

In der Programmierung können Zufallszahlen durch die Verwendung von speziellen Funktionen oder Bibliotheken erzeugt werden. Diese verwenden unterschiedliche Algorithmen, die unterschiedliche Eigenschaften haben. Die Wahl des Algorithmus hängt von der Anwendung ab.

- Random sampling mit `numpy.random`
- Zufallsvariablen in ScyPy mit `scipy.stats`

Middle-Square-Methode

Die Middle-Square-Methode (MSM) ist ein einfacher Algorithmus zur Generierung von Pseudozufallszahlen. Er wurde erstmals 1946 von John von Neumann vorgestellt und ist ein Beispiel für einen

deterministischen Algorithmus, der auf einem Anfangswert (engl. *seed*) basiert.

Die MSM verwendet eine einfache Formel, um den nächsten Wert in der Zahlenfolge zu berechnen. Zunächst wird der Seed quadriert und das Ergebnis dann in der Mitte abgeschnitten, um eine Zahl mit der gleichen Anzahl an Stellen wie der Seed zu erhalten. Diese Zahl wird dann wiederum verwendet, um auf die gleiche Weise den nächsten Wert in der Folge zu generieren.

Die MSM ist zwar einfach zu implementieren, hat aber einige Nachteile. Je nach Seed ist die Periode (Länge der Sequenz bis sich die Zahlen wiederholen) vergleichsweise kurz. Daher folgen die Zahlen einem einfach verhersehbaren Muster und sind für moderne Anwendungen nicht genügend pseudo-zufällig.

Dennoch ist die MSM ein interessantes Beispiel dafür, wie einfache mathematische Formeln zur Erzeugung von Pseudozufallszahlen verwendet werden können.

Das folgende Video zeigt, wie der MSM Algorithmus funktioniert:

- “Pseudorandom number generators | Computer Science | Khan Academy” - Khan Academy Labs

Weiteres interessantes Video:

- “How to Generate Pseudorandom Numbers | Infinite Series” - PBS Infinite Series

Die Wikipedia erklärt die Implementierung der Middle-Square-Methode:

- “middle-square method” - Wikipedia

Und für wirklich motivierte noch ein weiteres Video:

- “Wie kommt der Zufall in den Computer? (Teil 1 von 2)” - Weitz / HAW Hamburg

Arbeitsauftrag

Aufgabe 1: Implementierung der Middle-Square-Methode Wir implementieren die Middle-Square-Methode für 4-stellige Zahlen.

a) Implementierung Schritt 1 Wir implementieren die Middle-Square-Methode für 4-stellige Zahlen.

Programmiere zuerst eine Funktion namens `msm_next(number)`, die eine ganze Zahl `number` mit 4 Stellen als Parameter nimmt. Sie soll die nächste Zahl in der Zahlenfolge nach dem MSM Algorithmus zurück geben. Validiere deine Implementierung mit den folgenden Beispielen und weiteren Fällen, die du dir selbst ausdenkst:

- $3333 \rightarrow 3333^2 = 11108889 \rightarrow 1088$
- $1234 \rightarrow 1234^2 = 1522756 = 01522756 \rightarrow 5227$

Hinweise:

- Mit `str()` und `int()` wandelt man Zahlen in Strings und umgekehrt um.
- Ein Teil eines Strings kann mit Slicing extrahiert werden.
- Das Quadrat einer 4-stelligen Zahl hat nicht immer 8 Stellen, wie im Beispiel mit 1234 zu sehen ist. Führende Nullen können mit `str.zfill()` hinzugefügt werden.

Links:

- Python String `zfill()` Method - w3schools
- Python String Methods - Python docs

```
def msm_next(number):
    """Ein Schritt der Middle -Square -Method für 4 -stellige Zahlen."""
    n2_padded = str(number**2).zfill(8)
    return int(n2_padded[2: -2])

(
    msm_next(3333) == 1088,
    msm_next(1234) == 5227,
    msm_next(3839) == 7379,
```

)

(True, True, True)

b) Implementierung Schritt 2 Nun schreibst du eine Funktion namens `msm(seed, n)`, die einen Startwert `seed` und eine Anzahl an Schritten `n` als Parameter nimmt. Sie soll `msm_next()` benutzen, um ausgehend vom Startwert ein NumPy Array von `n` Pseudozufallszahlen nach der MSM zu erzeugen. Das heisst, der Seed kommt nicht im resultierenden Array vor, sondern nur die `n` darauffolgenden Zahlen.

```
def msm(seed, n):
    """Middle -Square -Method mit Seed und Anzahl Schritten n"""
    number = seed
    sequence = []
    for _ in range(n):
        number = msm_next(number)
        sequence.append(number)
    return np.array(sequence)
```

c) Testen der Funktion Teste die Funktion `msm()` mit einem Seed von 1547 und erzeuge 100 Zahlen in einem Array namens `rnd_numbers`.

Prüfe, dass die Summe der Zahlen 510531 ergibt.

Schaue die Zahlen an. Was fällt dir auf?

```
rnd_numbers = msm(1547, 100)
print(rnd_numbers)
np.sum(rnd_numbers) == 510531
```

```
[3932 4606 2152 6311 8287 6743 4680 9024 4325 7056 7871 9526 7446 4429
 6160 9456 4159 2972 8327 3389 4853 5516 4262 1646 7093 3106 6472 8867
 6236 8876 7833 3558 6593 4676 8649 8052 8347 6724 2121 4986 8601 9772
 4919 1965 8612 1665 7722 6292 5892 7156 2083 3388 4785 8962 3174 742
 5505 3050 3025 1506 2680 1824 3269 6863 1007 140 196 384 1474 1726
 9790 8441 2504 2700 2900 4100 8100 6100 2100 4100 8100 6100 2100 4100
 8100 6100 2100 4100 8100 6100 2100 4100 8100 6100 2100 4100 6100
 2100 4100]
```

```
np.True_
```

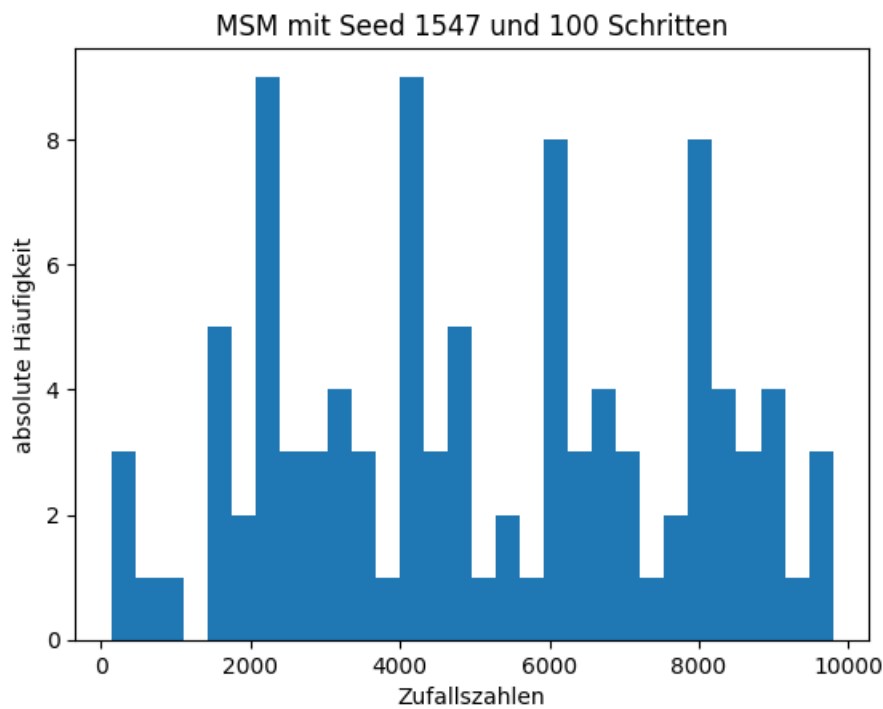
Beobachtung: Ca. nach 2/3 der Zahlen wiederholt sich die Sequenz [4100 8100 6100 2100]. Das heisst, dass die Periode der MSM für den Seed 1547 kleiner als 100 ist und die Sequenz dann in einen Zyklus der Länge übergeht.

d) Histogramm Erzeuge ein Histogramm der Zufallszahlen `rnd_numbers`. Verwende dazu die Funktion `plt.hist()`.

- Matplotlib-Tutorial: Histogramme

Erhöhe die Anzahl Bins auf (Option `bins`) auf 20 - 30. Was fällt dir auf? Schau dir die Zahlenfolge nochmals an. Kannst du erklären, warum das Histogramm so aussieht?

```
plt.hist(rnd_numbers, bins=30)
plt.xlabel("Zufallszahlen")
plt.ylabel("absolute Häufigkeit")
plt.title("MSM mit Seed 1547 und 100 Schritten")
plt.show()
```



Beobachtung: Das Histogramm zeigt, dass die Zahlen nicht gleichverteilt sind. Es gibt vier Zahlen (2100, 4100, 6100, 8100), die sich wiederholen und deswegen wesentlich häufiger vorkommen als die anderen Zahlen.

Aufgabe 2: Periode bei Zufallszahlen Die Periode bei der Erzeugung von Pseudozufallszahlen gibt an, nach wie vielen Schritten sich die Zahlenfolge wiederholt.

Die Periode ist ein Mass für die Qualität eines Zufallszahlengenerators. Eine kurze Periode führt dazu, dass die Zahlen schnell vorhersehbar werden, was schlecht ist. Eine ausreichend lange Periode ist daher wichtig. Im Idealfall ist sie sogar länger als die Anzahl der Zahlen, die der Generator jemals generieren wird.

Es gibt verschiedene Methoden, um die Periode zu erhöhen, z.B. durch Verbesserung des Algorithmus, Kombination mehrerer Methoden oder Erhöhung der Grösse des Zustandsraums (lange Seeds). Die Periode hängt auch von der Wahl des Seeds ab. Daher ist es wichtig, einen geeigneten Seed zu wählen, um sicherzustellen, dass die Periode möglichst gross ist.

a) Funktion zur Messung der Periode Schreibe eine Funktion `msm_period(seed)`, die einen Seed als Parameter erhält. Sie soll die Anzahl Schritte zurückgeben, die der Generator `msm_next()` benötigt, bis er sich zum ersten Mal wiederholt. Nutze dazu eine Menge (`set`), um die bereits generierten Zahlen zu speichern und neue damit zu vergleichen.

```
def msm_period(seed):
    sequence = set()
    number = msm_next(seed)
    while number not in sequence:
        sequence.add(number)
        number = msm_next(number)
    period = len(sequence) + 1
    return period, number
```

b) Verifikation der Periodenmessung Mit dem Seed 1234 sollte die Periode 57 sein. Die Zahl 0 beginnt sich zu wiederholen. Überprüfe dies mit der Funktion `msm_period()`.

```
msm_period(1234)
```

```
(57, 0)
```

c) Maximale Periode finden Berechne die Periode für alle Seeds von 1000 bis 9999 mit einer List Comprehension und finde den Seed mit der längsten Periode mithilfe der Funktion `max()`. Wie lang ist diese längste Periode?

```
periods = [(msm_period(seed)[0], seed) for seed in range(1000, 10000)]
max_period, max_seed = max(periods)

print(f"Die maximale Periode ist {max_period} und wird erreicht mit Seed {max_seed}.")
```

Die maximale Periode ist 111 und wird erreicht mit Seed 6239.

Alternativ kann man auch `np.argmax()` verwenden, um den Index der längsten Periode zu finden:

```
seeds = np.arange(1000, 10000)
periods, _ = np.vectorize(msm_period)(seeds)
i_max = np.argmax(periods)

print(
    f"Die maximale Periode ist {periods[i_max]} und wird erreicht mit Seed {seeds[i_max]}."
)
```

Die maximale Periode ist 111 und wird erreicht mit Seed 6239.

Weitere Informationen (optional) Zwei interessante Blogposts zum Thema:

- <http://bit-player.org/2022/the-middle-of-the-square>
- <https://hbfs.wordpress.com/2017/11/21/the-middle-square-method-generating-random-sequences-viii/>

0.11.2 Zufallszahlen Erzeugen

Übungen - Lösungen

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import scipy
```

Aufgabe 1: Allgemeine Gleichverteilung Implementiere eine Funktion `random_uniform(a, b, n, seed)`, die `n` Zufallszahlen gleichverteilt im Intervall $[a, b) \in \mathbb{R}$ mithilfe der Middle-Square-Methode mit 10-stelligen Zahlen erzeugt.

Vorgehen

- Übernehme die Middle-Square-Methode aus der Vertiefung und passe sie so an, dass sie 10-stellige Zahlen erzeugt.
- Skaliere die erzeugten Zahlen zuerst so, dass sie im Intervall $[0, 1)$ liegen, siehe Vertiefung.
- Skaliere die Zahlen dann so, dass sie im Intervall $[a, b)$ liegen. (Mit $b - a$ skalieren und um a verschieben.)

Teste die Funktion, indem du 10'000 Zufallszahlen im Intervall $[-2, 5)$ erzeugst und in einem Histogramm darstellst. Prüfe von Auge, ob die Zahlen wirklich gleichverteilt im Intervall $[-2, 5)$ liegen.

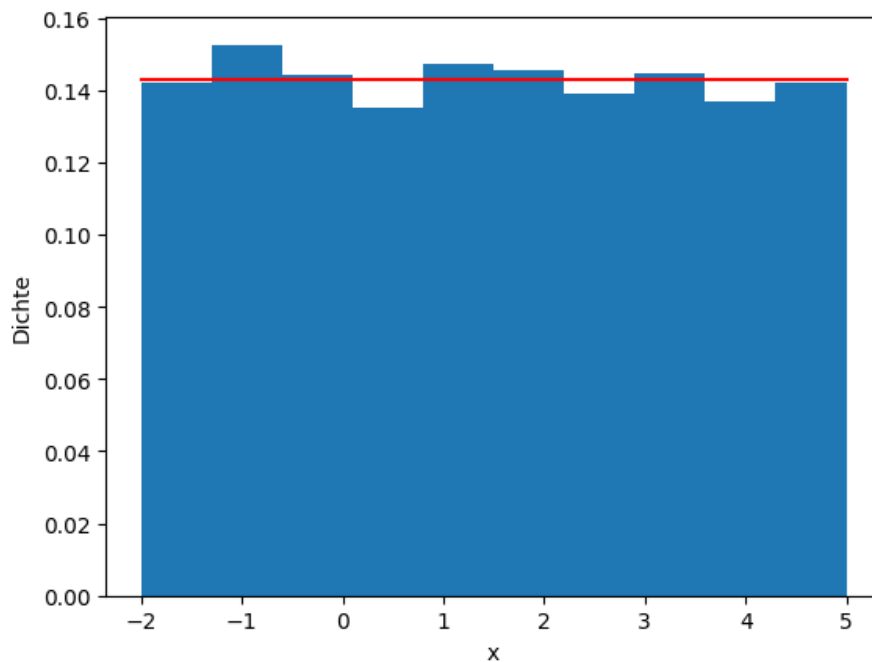
```
def msm_next_10(number):
    """Ein Schritt der Middle -Square -Method für 10 -stellige Zahlen."""
    n2_padded = str(number**2).zfill(20)
    return int(n2_padded[5: -5])

def msm_10(seed, n):
    """Middle -Square -Method mit 10 -stelligen Zahlen mit Seed und Anzahl Schritten n"""
    assert len(str(seed)) == 10, "Seed muss 16 -stellig sein."
    number = seed
    sequence = []
    for _ in range(n):
        number = msm_next_10(number)
        sequence.append(number)
    return np.array(sequence)

def random_uniform(a, b, n, seed):
    """Erzeugt n Zufallszahlen gleichverteilt im Intervall [a, b)."""
    numbers = msm_10(seed, n)
    u01 = numbers / 10 ** len(str(seed))
    return (b - a) * u01 + a

x = random_uniform(-2, 5, 10000, 1234567890)

plt.hist(x, density=True)
plt.plot([-2, 5], [1 / 7, 1 / 7], "r")
plt.xlabel("x")
plt.ylabel("Dichte")
plt.show()
```



Aufgabe 2: Normalverteilung würfeln Nutze den Random Generator von NumPy, um 10'000 Zufallszahlen aus einer Normalverteilung (`rng.normal`) mit Erwartungswert 4 und Standardabweichung 2 zu erzeugen. Stelle die Häufigkeitsdichte der erzeugten Zahlen in einem Histogramm mit 60 bins dar.

Weshalb 60 bins? Hier habe ich die Freedman-Diaconis Faustregel benutzt. Eine gute Binbreite ist $w = 2 \cdot IQR \cdot n^{-1/3}$, wobei IQR der Interquartilsabstand ist. Die Anzahl bins ist dann

$$n_{\text{bins}} = \frac{(\max(x) - \min(x)) n^{1/3}}{2 \cdot IQR} \quad (150)$$

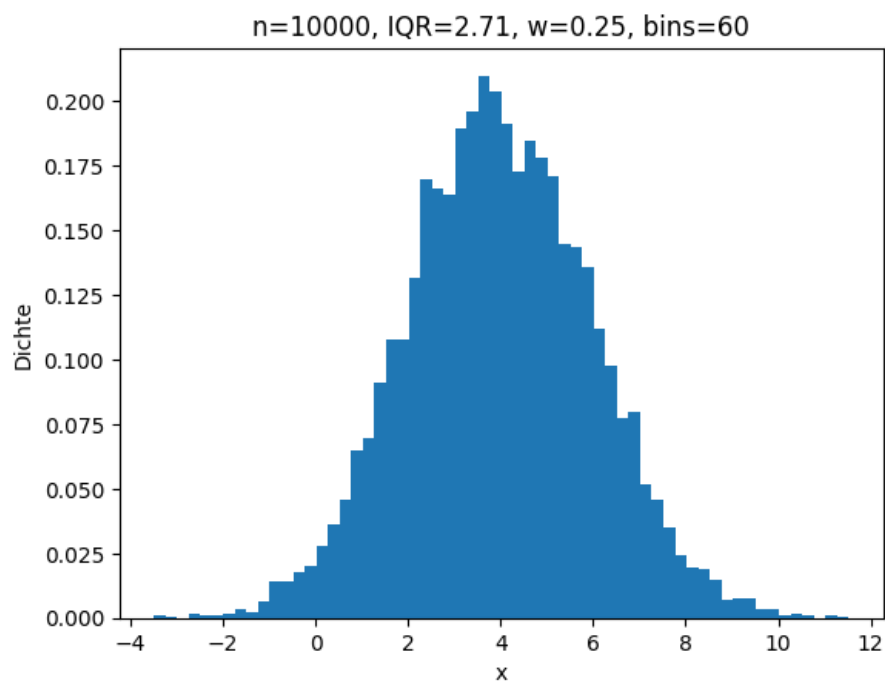
Die Option `bins` von `plt.hist` kann übrigens auch ein String sein, der eine der Methoden angibt, wie die Anzahl der bins bestimmt werden soll. Zum Beispiel `bins='fd'` für die Freedman-Diaconis Faustregel. Siehe Details dazu hier: https://numpy.org/doc/stable/reference/generated/numpy.histogram_bin_edges.html.

```
rng = np.random.default_rng()

x = rng.normal(4, 2, 10000)

IQR = scipy.stats.iqr(x)
w = 2 * IQR * len(x) ** (-1 / 3)
bins = int((x.max() - x.min()) / w) + 1

plt.hist(x, bins=bins, density=True)
plt.xlabel("x")
plt.ylabel("Dichte")
plt.title(f"n={len(x)}, IQR={IQR:.2f}, w={w:.2f}, bins={bins}")
plt.show()
```



Aufgabe 3: Zahlenlotto

a) Lotto Simulation Schreibe eine Funktion `lotto_ziehen(rng)`, die mit einem gegebenen Zufallszahlengenerator `rng` 6 verschiedene Zahlen zwischen 1 und 42 zieht und zusätzlich eine Glückszahl zwischen 1 und 6. Der Rückgabewert soll ein `tuple` mit den 6 Lottozahlen in aufsteigender Reihenfolge als `set` und der Glückszahl als `int` sein.

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36
37	38	39	40	41	42

Glückszahlen



Swiss Lotto | Einzeltipps spielen | Swisslos. (o. J.). Abgerufen 27. Januar 2026, von <https://www.swisslos.ch/de/konto/spielkonto/uebersicht.html>

```
def lotto_ziehen(rng):  
    lotto = rng.choice(range(1, 43), 6, replace=False)  
    glueckszahl = rng.choice(range(1, 7))  
    return set(map(int, lotto)), int(glueckszahl)
```

```
rng = np.random.default_rng(42)  
ziehung = lotto_ziehen(rng)  
ziehung
```


{4, 18, 26, 30, 37, 41}, 6)

b) Bewertung Schreibe nun eine Funktion `lotto_bewerten(tipp, ziehung)`, die einen Tipp (`tuple[set[int], int]`) und eine Ziehung (`tuple[set[int], int]`) als Argumente entgegennimmt und die Anzahl der Übereinstimmungen zurückgibt. Dabei soll die Glückszahl halb (0.5) gezählt werden. Z.B.: Stimmen 3 Lottozahlen überein und die Glückszahl auch, so gibt die Funktion 3.5 zurück.

Hinweis: Nutze die Methode `set.intersection`.

```
def lotto_bewerten(tipp, ziehung):
    """Bewertet den Tipp."""
    return len(tipp[0].intersection(ziehung[0])) + 0.5 * (tipp[1] == ziehung[1])
```

```
tipp = ({4, 18, 26, 27, 28, 29}, 6)
lotto_bewerten(tipp, ziehung)
```

3.5

c) Simulation 1. Simuliere nun die Ergebnisse von 50 Ziehungen mit dem immer gleichen Tipp. Schreibe dazu eine Funktion `lotto_simulieren(tipp, n, seed=None)`, die `n` Ziehungen simuliert, den Tipp auswertet und alle Bewertungen als Array zurückgibt. Nutze dazu die Funktion `lotto_ziehen` aus Aufgabe 2a) und die Funktion `lotto_auswerten` aus Aufgabe 2b). Die Funktion soll zuerst einen Zufallszahlengenerator mit dem gegebenen `seed` initialisieren.

Zum Überprüfen: mit dem Tipp {4, 18, 26, 27, 28, 29}, der Glückszahl 6 und dem Seed 42 sollte die Summe der Bewertungen 50.5 ergeben.

```
def lotto_simulieren(tipp, n, seed=None):
    """Simuliert n Ziehungen und wertet jeweils den gleichen Tipp aus."""
    rng = np.random.default_rng(seed)
    return np.array([lotto_bewerten(tipp, lotto_ziehen(rng)) for _ in range(n)])
```

```
bewertungen = lotto_simulieren(tipp, 50, seed=42)
bewertungen.sum()
```

```
np.float64(50.5)
```

d) Erfolg? (optional) Schreibe eine Funktion `ziehungen_bis_zum_sechser(tipp, bewertung, seed=None)`, die simuliert, wie viele Ziehungen nötig sind, um eine 6 im Lotto zu haben.

Mit dem Tipp von Teil c) und dem Seed 42 sollte es 14'556'468 Ziehungen brauchen, um eine 6 im Lotto zu haben. (Die Simulation dauerte bei mir ca. 4 Minuten.)

Vorsicht: Diese Funktion kann grundsätzlich unendlich lange laufen, weil Erfolg nicht garantiert ist. Zur Vermeidung von Endlosschleifen könnte man z.B. eine maximale Anzahl an Ziehungen (`maxiter`) festlegen.

```
def ziehungen_bis_zum_sechser(tipp, bewertung, seed=None):
    """Zählt die Ziehungen bis zum ersten Sechser."""
    rng = np.random.default_rng(seed)
    n = 0
    bewertung = 0
    while bewertung < 6:
        bewertung = lotto_bewerten(tipp, lotto_ziehen(rng))
        n += 1
```

```
return n
```

```
ziehungen_bis_zum_sechser(tipp, 0, seed=42)
```

```
14556468
```

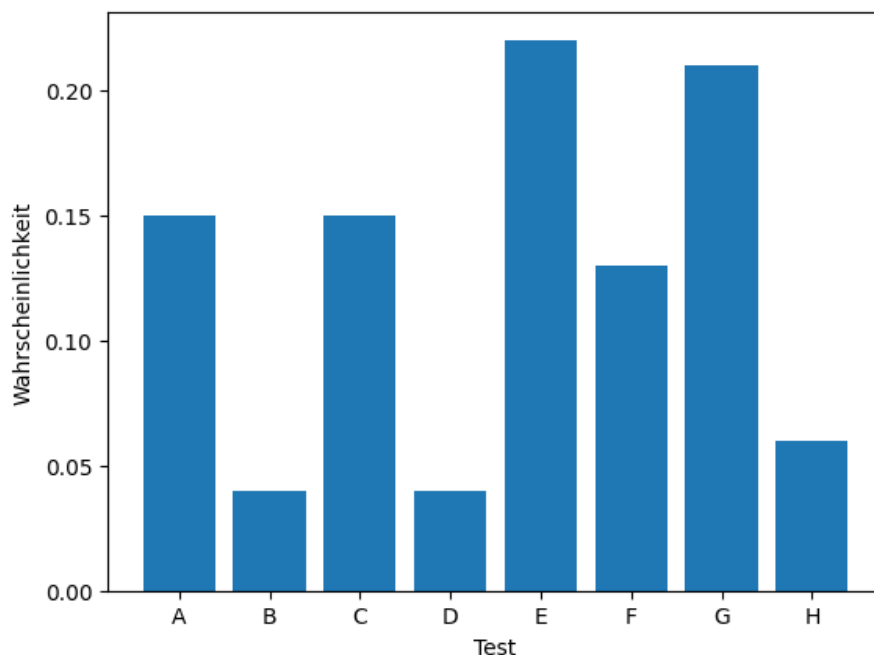
Aufgabe 4: Diskrete Zufallsvariable In einem Blutanalyse-Labor werden 8 verschiedene Bluttests durchgeführt. Die Statistik über das letzte Jahr besagt, dass die Tests, die wir hier einfach mit Buchstaben bezeichnen, folgende relative Häufigkeiten haben:

```
tests = np.array(list("ABCDEFGH"))
rel_frequencies = np.array([0.15, 0.04, 0.15, 0.04, 0.22, 0.13, 0.21, 0.06])
```

Im Folgenden gehen wir davon aus, dass die Tests unabhängig voneinander auftreten und dass die Häufigkeiten konstant sind. Entsprechend können wir die Wahrscheinlichkeiten der Tests gleich deren relativen Häufigkeiten setzen.

a) Wahrscheinlichkeiten darstellen Stelle die Wahrscheinlichkeitsverteilung der Zufallsvariable T dar, die den Test-Typ angibt. Nutze dazu ein Säulendiagramm (`plt.bar`).

```
plt.bar(tests, rel_frequencies)
plt.xlabel("Test")
plt.ylabel("Wahrscheinlichkeit")
plt.show()
```



b) Tests simulieren Für die Planung einer neuen automatisierten Analyse-Anlage soll nun ein dynamisches Simulationsmodell mit zufällig auftretenden Test-Aufträgen gefüttert werden. Schreibe eine Funktion `generate_tests(n, seed)`, die eine Anzahl n Tests zufällig erzeugt und als Array zurückgibt. Die Tests sollen dabei als Buchstaben von A bis H repräsentiert werden und der Wahrscheinlichkeitsverteilung aus der Statistik in a) folgen. Der Parameter `seed` soll den Standard-Zufallsgenerator von NumPy initialisieren.

```
# Variante 1: Inversionsmethode

cdf = rel_frequencies.cumsum()

def generate_tests(n, seed):
    rng = np.random.default_rng(seed)
    u = rng.random(n)
    i = np.searchsorted(cdf, u)
    return tests[i]

print(generate_tests(10, 12345678))

['H' 'G' 'D' 'C' 'E' 'B' 'A' 'B' 'H' 'G']

# Variante 2: Choice

def generate_tests_choice(n, seed):
    rng = np.random.default_rng(seed)
    return rng.choice(tests, n, p=rel_frequencies)

print(generate_tests_choice(10, 12345678))

['H' 'G' 'D' 'C' 'E' 'B' 'A' 'B' 'H' 'G']
```

c) Resultate auswerten Ziehe mithilfe der Funktion aus b) 500 zufällige Tests und stelle die relativen Häufigkeiten in einem Säulendiagramm dar.

Dazu hast du drei Möglichkeiten:

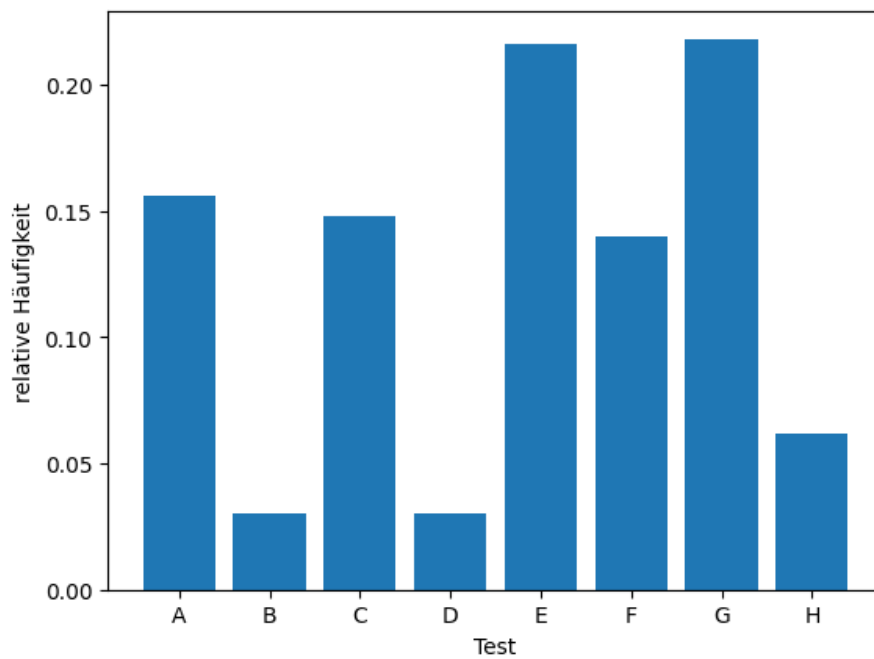
- Verwende die Klasse `collections.Counter`, um die Häufigkeiten der Tests zu bestimmen, und die Funktion `plt.bar` für das Säulendiagramm.
- Verwende eine Pandas Series, um die Häufigkeiten der Tests zu bestimmen, und die Methode `Series.plot` für das Säulendiagramm.
- Verwende direkt `sns.countplot` oder `sns.histplot` von Seaborn.

```
n = 500
seed = 1234567890
data = generate_tests(n, seed)

# Variante 1: Counter und Barplot
from collections import Counter

counts = Counter(data)
freqs = np.array([counts[t] for t in tests]) / n

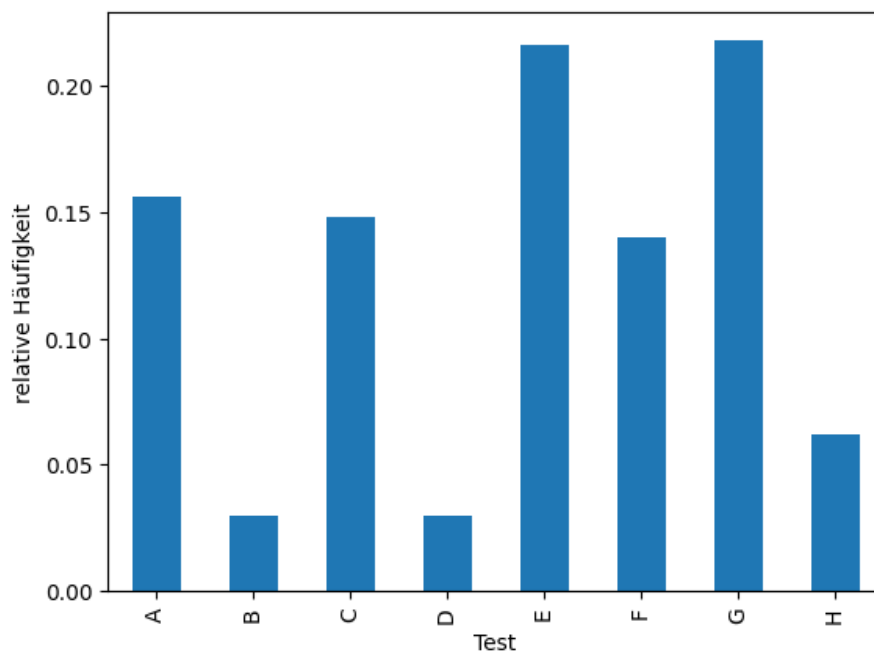
plt.bar(tests, freqs)
plt.xlabel("Test")
plt.ylabel("relative Häufigkeit")
plt.show()
```



Variante 2: Pandas

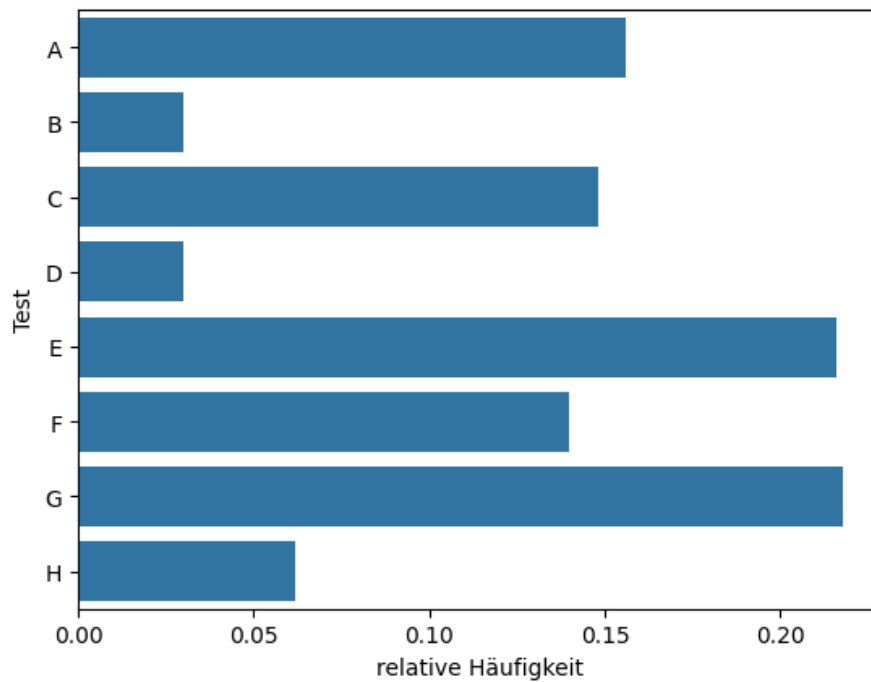
```
counts = pd.Series(data).value_counts(normalize=True, sort=False).sort_index()
```

```
counts.plot(kind="bar")  
plt.xlabel("Test")  
plt.ylabel("relative Häufigkeit")  
plt.show()
```

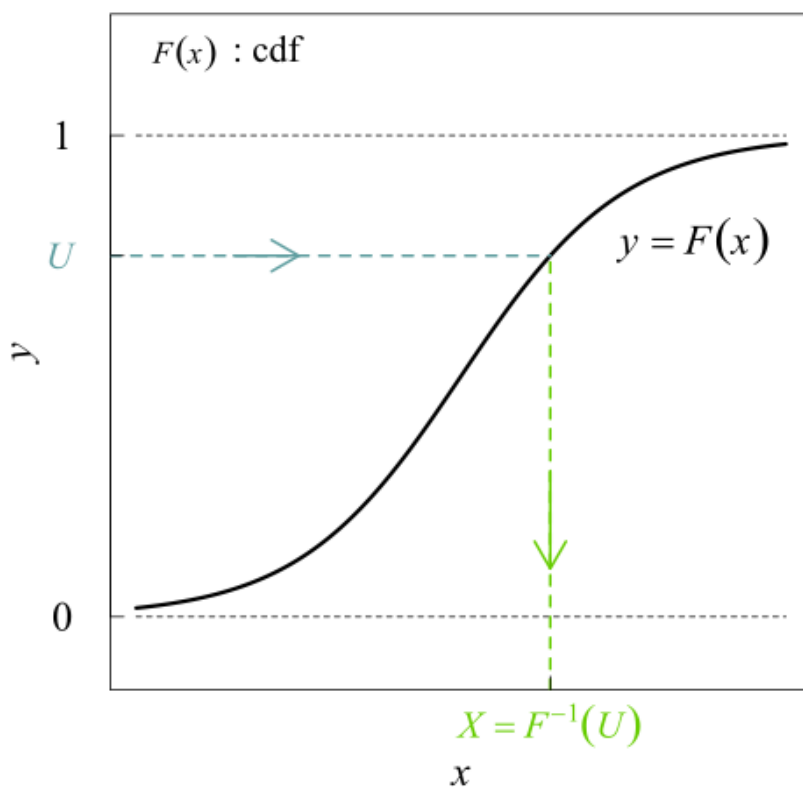


Variante 3: Seaborn

```
sns.countplot(pd.Series(data), order=tests, stat="probability")  
plt.ylabel("Test")  
plt.xlabel("relative Häufigkeit")  
plt.show()
```



Aufgabe 5: Inversionsmethode für kontinuierliche Zufallsvariablen (optional) Mithilfe der Inversionsmethode können Zufallszahlen aus einer beliebigen Wahrscheinlichkeitsverteilung erzeugt werden. Dazu zieht man Zahlen u aus der Gleichverteilung $U(0,1)$ und wendet die Umkehrfunktion der kumulativen Verteilungsfunktion F der gewünschten Verteilung darauf an: $x = F^{-1}(u)$. Dieses Diagramm zeigt das Prinzip:



- “Inversionsmethode” - Wikipedia

Theorie Die kumulative Verteilungsfunktion $F(x)$ (engl. *cumulative distribution function*, CDF) gibt die Wahrscheinlichkeit an, dass eine Zufallsvariable X einen Wert kleiner oder gleich x annimmt, also

$$F(x) = P(X \leq x) \quad (151)$$

Daher ist $F(x)$ das Integral der Wahrscheinlichkeitsdichte $f(x)$ (engl. *probability density function*, PDF):

$$F(x) = \int_{-\infty}^x f(x') dx' \quad (152)$$

Die Umkehrfunktion $F^{-1}(u)$ ordnet nun jeder Zahl $u \in [0, 1)$ den Wert x zu, für den $F(x) = u$ gilt.

Vorgehen Die Inversionsmethode funktioniert also so:

1. Ziehe eine Zahl u aus der Gleichverteilung $U(0, 1)$.
2. Berechne $x = F^{-1}(u)$. x ist nun eine Zufallszahl zur gewünschten Wahrscheinlichkeit $F(x)$.

Auftrag In dieser Übung sollen Zufallszahlen der Beta-Verteilung erzeugt werden. Die Wahrscheinlichkeitsdichte der Beta-Verteilung kann mithilfe der Funktion `scipy.stats.beta.pdf` berechnet werden. Wir gehen so vor:

- Die kumulative Verteilungsfunktion $F(x)$ wird mit numerischer Integration an 100 Punkten berechnet.
- Daraus wird eine Interpolationsfunktion erstellt, die $F(x)$ für beliebige x berechnen kann.
- Diese wird anschliessend benutzt, um die Umkehrung $F^{-1}(u)$ numerisch zu berechnen, indem die Gleichung $F(x) = u$ nach x gelöst wird.
- Zum Schluss wird alles zusammengesetzt, um aus einer gleichverteilten Zufallszahl u eine Zufallszahl x aus der Beta-Verteilung zu erzeugen.

a) Wahrscheinlichkeitsdichte der Beta-Verteilung berechnen und darstellen Die Beta-Verteilung ist definiert für $x \in [0, 1]$ und hat zwei Parameter α und β . Die Wahrscheinlichkeitsdichte $f(x)$ kann mit `scipy.stats.beta.pdf` berechnet werden.

Berechne `pdf = f(x)` für $\alpha = 2$ und $\beta = 5$ an 100 Punkten im Intervall $x \in [0, 1]$ und stelle diese in einem Diagramm dar.

- “Betaverteilung” - Wikipedia

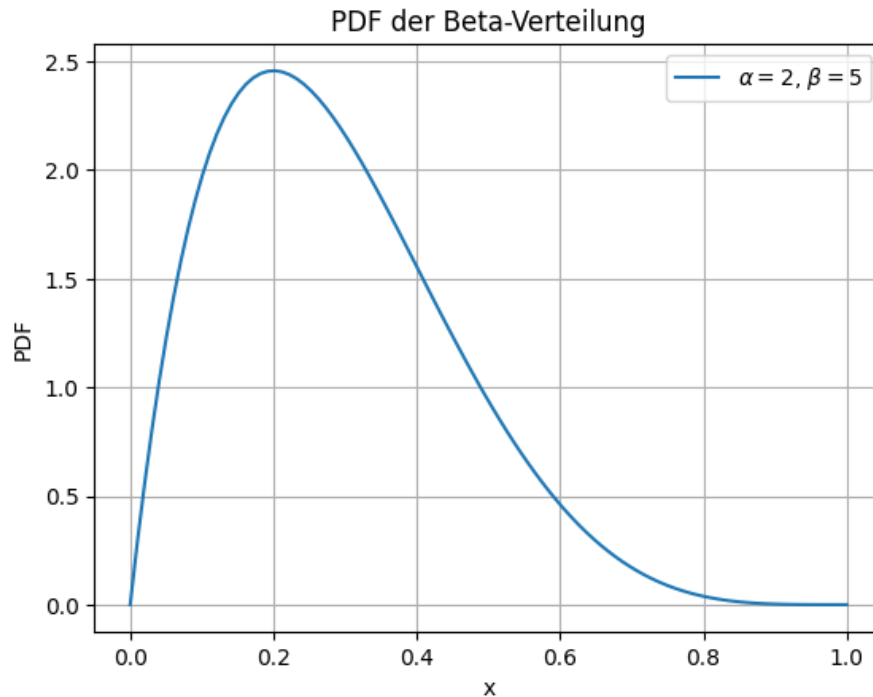
```
# Parameter der Beta -Verteilung (alpha und beta)
alpha, beta = 2, 5

# Erstelle ein Array von x -Werten im Bereich von 0 bis 1
x = np.linspace(0, 1, 100)

# Berechne die PDF für jeden x -Wert
pdf = scipy.stats.beta.pdf(x, alpha, beta)

# Plote die PDF
plt.plot(x, pdf, label=f"$\\alpha={alpha}$, $\\beta={beta}$")
plt.xlabel("x")
plt.ylabel("PDF")
plt.title(f"PDF der Beta -Verteilung")
```

```
plt.legend()
plt.grid()
plt.show()
```



b) Verteilungsfunktion der Beta-Verteilung numerisch berechnen Schreibe eine Funktion $F(x, \alpha, \beta)$, welche die kumulative Verteilungsfunktion der Beta-Verteilung numerisch berechnet. Da es sich um das Integral

$$F(x) = \int_0^x f(x') dx' \quad (153)$$

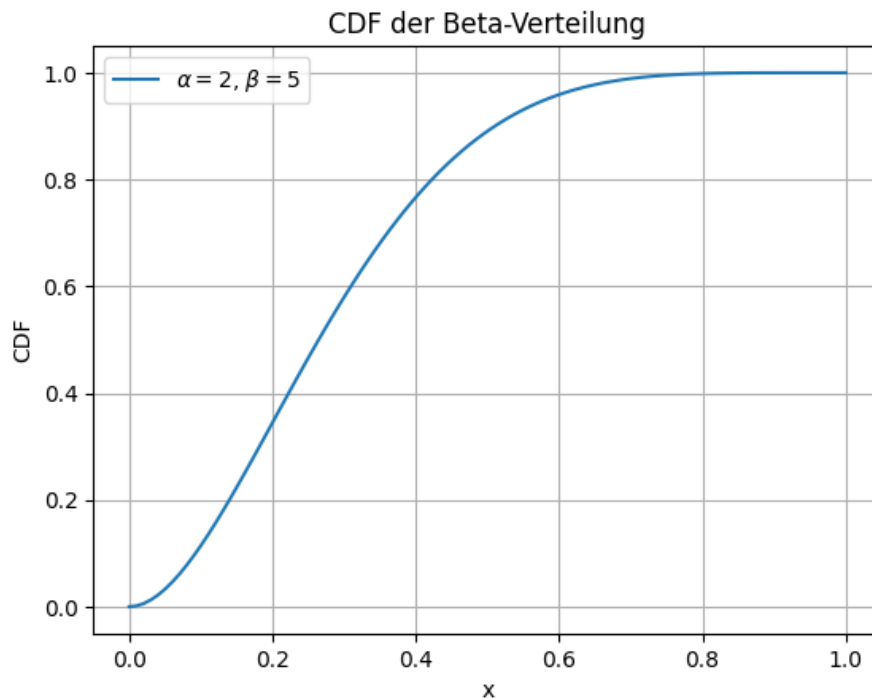
handelt, sollst du dazu die passende Funktion aus dem Modul `scipy.integrate` verwenden.

Benutze deine Funktion, um $\text{cdf} = F(x)$ für $\alpha = 2$ und $\beta = 5$ an 100 Punkten im Intervall $x \in [0, 1]$ zu berechnen und stelle diese in einem Diagramm dar.

```
def F(x, alpha, beta):
    f = lambda t: scipy.stats.beta.pdf(t, alpha, beta)
    return scipy.integrate.quad(f, 0, x)[0]

cdf = np.vectorize(F)(x, alpha, beta)

plt.plot(x, cdf, label=f"$\\alpha={alpha}$, $\\beta={beta}$")
plt.xlabel("x")
plt.ylabel("CDF")
plt.title(f"CDF der Beta -Verteilung")
plt.legend()
plt.grid()
plt.show()
```



c) Interpolationsfunktion der Verteilungsfunktion Erstelle eine lineare Interpolationsfunktion der berechneten Werte `cdf` mithilfe der passenden Funktion aus dem Modul `scipy.interpolate`. Nenne diese `F_interp`.

Teste deine Funktion mit dem Wert $x = 0.5$. Dies soll 0.8905532623114158 ergeben.

```
F_interp = scipy.interpolate.make_interp_spline(x, cdf, k=1)

print(F_interp(0.5))

0.890553262311416
```

d) Wahrscheinlichkeitsfunktion invertieren Nun soll zu jedem $u \in [0, 1]$ das x mit $F(x) = u$ gefunden werden, also $x = F^{-1}(u)$.

Schreibe dazu eine Funktion `F_inv(u)`, welche das Newton-Verfahren aus `scipy.optimize` und `F_interp(x)` von vorhin nutzt, um die Gleichung $F(x) - u = 0$ zu lösen, und x zurückgibt.

Nutze `x0 = 0.5` als Startwert und erhöhe die maximale Anzahl an Iterationen auf 1000.

Teste deine Funktion mit dem Wert $u = 0.5$. Dies sollte $F^{-1}(0.5) = 0.26446285938954506$ geben.

```
def F_inv(u, x0=0.5, maxiter=1000):
    return scipy.optimize.newton(lambda x: F_interp(x) - u, x0, maxiter=maxiter)

print(F_inv(0.5))

0.26446285938954506
```

e) Zufallszahlen aus der Beta-Verteilung erzeugen Schreibe zum Abschluss eine Funktion `random_beta(n, seed=None)`, die n Zufallszahlen aus der Beta-Verteilung erzeugt.

Benutze hierfür den Default Random Generator von NumPy und `rng.uniform`, um n zufällige u -Werte zu generieren. Diese Werte werden dann mit `F_inv(u)` in x -Werte transformiert. Diese Funktion soll ein Array mit den x -Werten zurückgeben.

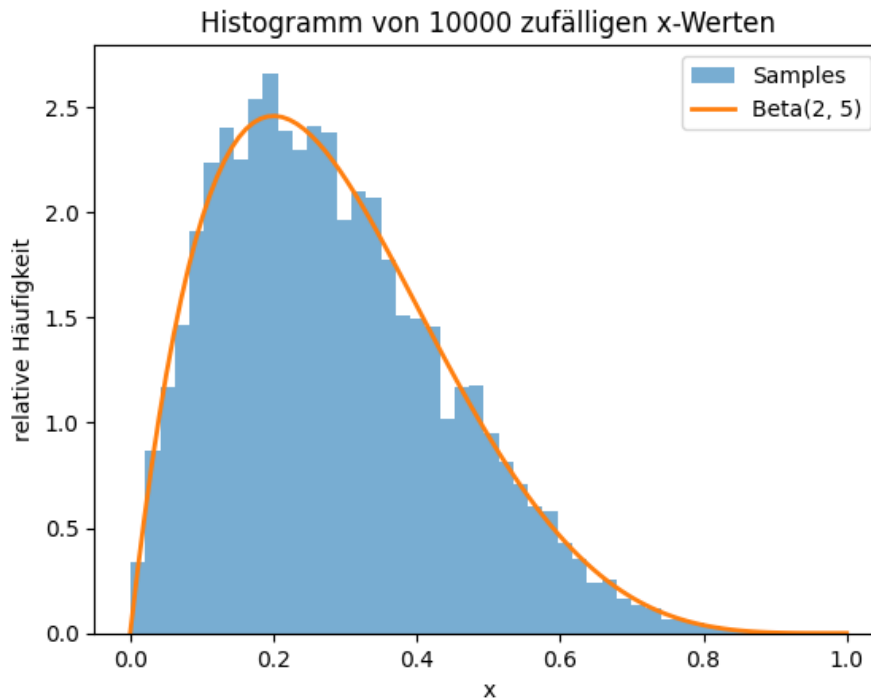
Erzeuge mit dieser Funktion 10'000 Beta-verteilte Zahlen. Stelle die relative Häufigkeit davon in einem Histogramm dar. Benutze hierfür `plt.hist` mit `density=True` und `bins="fd"` (für die

Freedman-Diaconis Faustegel zur Bestimmung der Binbreite). Plote dazu die Wahrscheinlichkeitsdichte $f(x)$ aus a).

```
def random_beta(n, seed=None):
    rng = np.random.default_rng(seed)
    u = rng.uniform(0, 1, n)
    return np.vectorize(F_inv)(u)

n = 10_000
random_beta_values = random_beta(n)

plt.hist(random_beta_values, bins="fd", density=True, alpha=0.6, label="Samples")
plt.plot(x, pdf, label=f"Beta({alpha}, {beta})", linewidth=2)
plt.xlabel("x")
plt.ylabel("relative Häufigkeit")
plt.title(f"Histogramm von {n} zufälligen x -Werten")
plt.legend()
plt.show()
```



0.11.3 Zufallszahlen Erzeugen

Vertiefung

Übersicht

1. Kontinuierliche Gleichverteilung mit Middle-Square-Methode
2. NumPy Random
3. Inversionsmethode (Inverse Transform Sampling)

```
import numpy as np
import matplotlib.pyplot as plt
```

1. Kontinuierliche Gleichverteilung mit Middle-Square-Methode

a) Grössere Anzahl Stellen In der Einarbeitung haben wir die Middle-Square-Methode mit 4-stelligen Zahlen kennengelernt. Diese Methode lässt sich auch auf eine beliebige Anzahl Stellen verallgemeinern. Es muss aber eine gerade Anzahl sein.

Generell gilt: Je mehr Stellen die Zahlen haben, desto grösser kann die maximale Periode der Methode sein.

Wir implementieren die MSM mal für 16-stellige Zahlen.

```
def msm_next_4(number):
    """Ein Schritt der Middle -Square -Method für 4 -stellige Zahlen. (Aus der Einarbeitung)"""
    n2_padded = str(number**2).zfill(8)
    return int(n2_padded[2: -2])

def msm_next_16(number):
    """Ein Schritt der Middle -Square -Method für 16 -stellige Zahlen."""
    n2_padded = str(number**2).zfill(32)
    return int(n2_padded[8: -8])

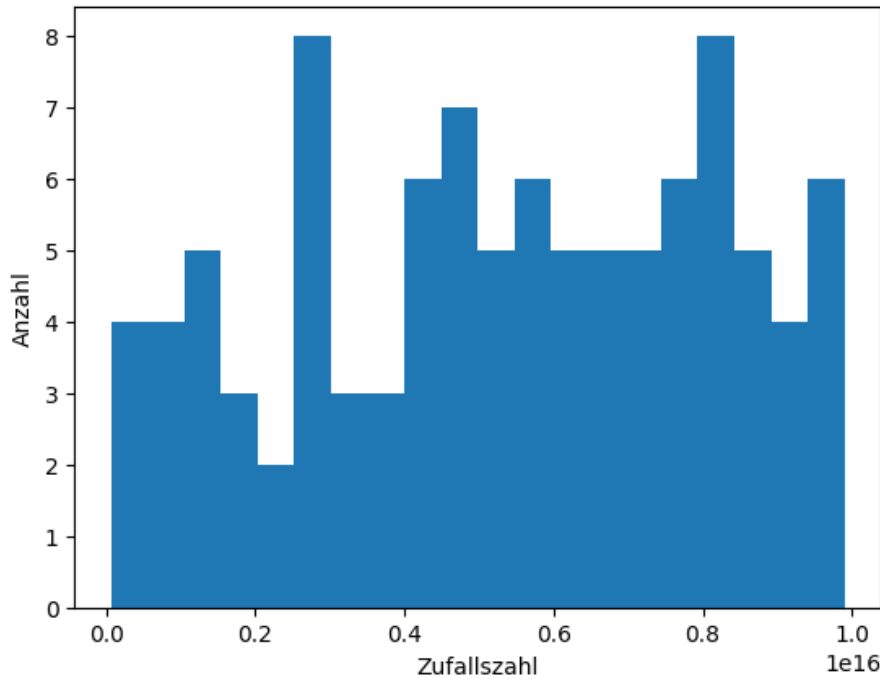
msm_next_16(3451234683710473)

8420461286073748

def msm_16(seed, n):
    """Middle -Square -Method mit 16 -stelligen Zahlen mit Seed und Anzahl Schritten n"""
    assert len(str(seed)) == 16, "Seed muss 16 -stellig sein."
    number = seed
    sequence = []
    for _ in range(n):
        number = msm_next_16(number)
        sequence.append(number)
    return np.array(sequence)

x = msm_16(3451234683710473, 100)

plt.hist(x, bins=20)
plt.xlabel("Zufallszahl")
plt.ylabel("Anzahl")
plt.show()
```



b) Mapping auf das Einheitsintervall Der MSM Algorithmus (wie bessere Algorithmen auch) erzeugt ganzzahlige Zufallszahlen, die Gleichverteilt sind. Diese müssen für viele Anwendungen zuerst auf das reelle Einheitsintervall von 0 bis 1 abgebildet werden. Je nach Konvention wird das Intervall entweder inklusive oder exklusive der Grenzen betrachtet. Wir folgen hier der Konvention von NumPy und nehmen das halb-offene Intervall $[0, 1)$, siehe `numpy.random.Generator.uniform`.

$$X \sim U(0, 1) \quad \Rightarrow \quad f(x) = \begin{cases} 1 & \text{für } x \in [0, 1) \\ 0 & \text{sonst} \end{cases} \quad (154)$$

Hier steht $U(0, 1)$ für die Gleichverteilung auf dem Intervall $[0, 1)$ und $f(x)$ für die Wahrscheinlichkeitsdichte.

Die MSM-Methode erzeugt natürliche Zahlen $0 \leq z < 10^m$, wobei m die Anzahl Stellen des Seeds ist. Durch division mit 10^n erhalten wir reelle Zufallszahlen im Intervall $[0, 1)$:

$$x = \frac{z}{10^m} \quad (155)$$

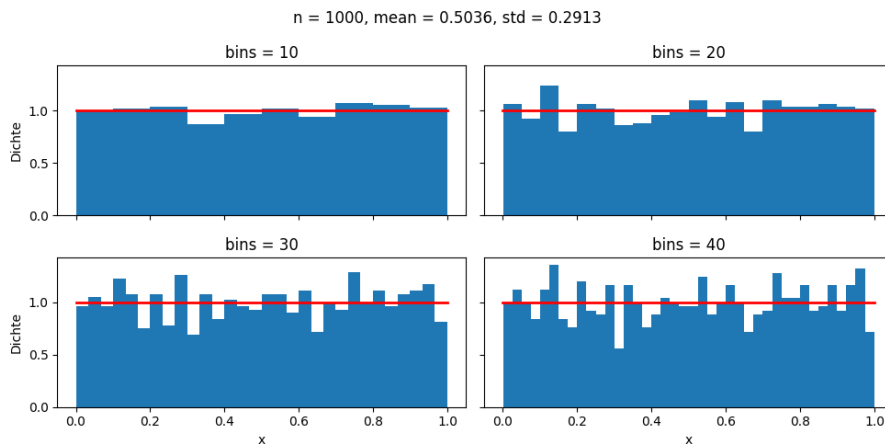
```
def uniform(seed, n):
    numbers = msm_16(seed, n)
    upper_bound = 10 ** len(str(seed))
    return numbers / upper_bound

u = uniform(3451234683710473, 1000)

fig, axes = plt.subplots(2, 2, figsize=(10, 5), sharex=True, sharey=True)
for ax, bins in zip(axes.flat, (10, 20, 30, 40)):
    ax.hist(u, bins=bins, density=True)
    ax.plot([0, 1], [1, 1], color="red", linewidth=2)
    ax.set_title(f"bins = {bins}")

for ax in axes[-1, :]:
    ax.set_xlabel("x")
for ax in axes[:, 0]:
    ax.set_ylabel("Dichte")
```

```
plt.suptitle(f"n = {len(u)}, mean = {u.mean():.4f}, std = {u.std():.4f}")
plt.tight_layout()
plt.show()
```



2. NumPy Random Der Einstiegspunkt in die Dokumentation ist hier: <https://numpy.org/doc/stable/reference/random/index.html>

Grundsätzlich wird zuerst ein Random Generator mit einem Seed instanziiert und dann für die Erzeugung von Pseudozufallszahlen verwendet

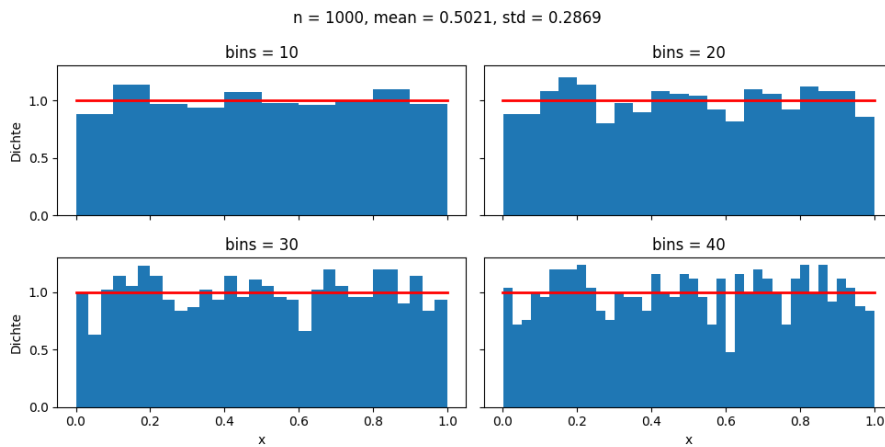
a) Gleichverteilung

```
rng = np.random.default_rng(3451234683710473)
u = rng.uniform(0, 1, 1000)

fig, axes = plt.subplots(2, 2, figsize=(10, 5), sharex=True, sharey=True)
for ax, bins in zip(axes.flat, (10, 20, 30, 40)):
    ax.hist(u, bins=bins, density=True)
    ax.plot([0, 1], [1, 1], color="red", linewidth=2)
    ax.set_title(f"bins = {bins}")

for ax in axes[-1, :]:
    ax.set_xlabel("x")
for ax in axes[:, 0]:
    ax.set_ylabel("Dichte")

plt.suptitle(f"n = {len(u)}, mean = {u.mean():.4f}, std = {u.std():.4f}")
plt.tight_layout()
plt.show()
```



b) Weitere Verteilungen Der NumPy Random Generator bietet eine grosse Auswahl Verteilungen an. Scipy bietet noch weitere Verteilungen an, unter anderem auch multivariate Verteilungen.

Beispiel: Gammaverteilung mit `rng.gamma`

```
rng = np.random.default_rng()
```

```
n = 1000
```

```
g1 = rng.gamma(1, 1, n)
```

```
g2 = rng.gamma(2, 1, n)
```

```
plt.hist(g1, bins=20, density=True, alpha=0.5, label="shape=1, scale=1")
```

```
plt.hist(g2, bins=20, density=True, alpha=0.5, label="shape=2, scale=1")
```

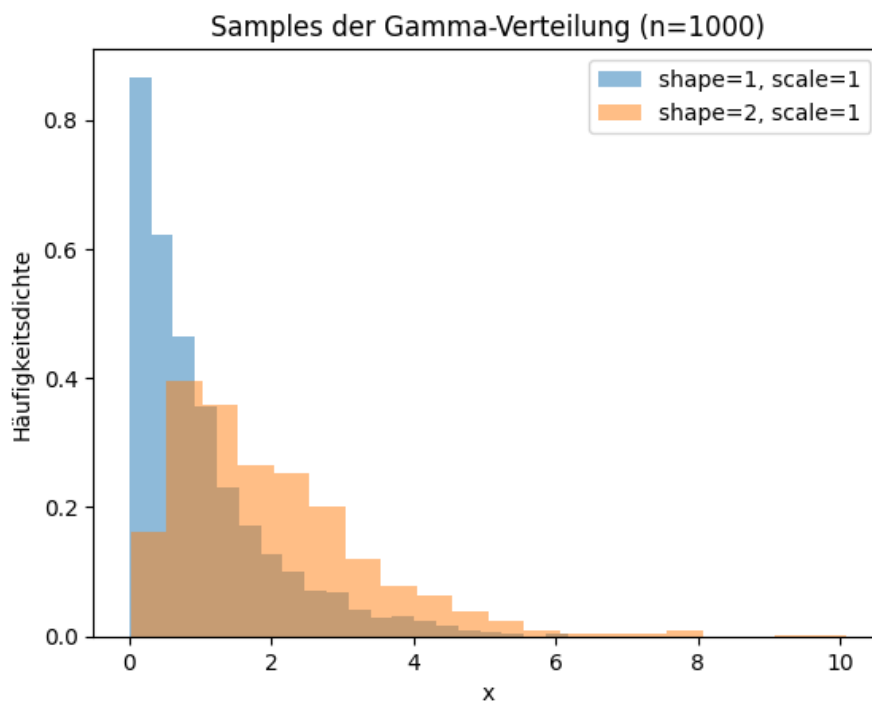
```
plt.legend()
```

```
plt.xlabel("x")
```

```
plt.ylabel("Häufigkeitsdichte")
```

```
plt.title(f"Samples der Gamma -Verteilung (n={n})")
```

```
plt.show()
```



c) Samples aus Arrays ziehen Samples aus einem Array zieht man mit `rng.choice` entweder mit oder ohne zurücklegen `replace=True/False`.

Mit Zurücklegen (default):

```
rng = np.random.default_rng()
x = np.arange(8)

print("Mit Zurücklegen:", rng.choice(x, size=20))

print("Ohne Zurücklegen:", rng.choice(x, size=4, replace=False))

Mit Zurücklegen: [0 1 7 5 5 1 4 2 5 1 7 5 6 2 7 4 4 0 0 5]
Ohne Zurücklegen: [7 4 5 6]
```

Zusätzlich kann man mit der Option `p=...` die Wahrscheinlichkeiten der Elemente angeben. (Die Summe der Wahrscheinlichkeiten muss 1 ergeben.)

```
x = np.arange(3)
p = [0.1, 0.6, 0.3]

y = rng.choice(x, 1000, p=p)

from collections import Counter

counter = Counter(y)

np.array([counter[xi] for xi in x]) / len(y)

array([0.1   , 0.616, 0.284])
```

d) Arrays Mischen Zum Mischen von Arrays gibt es 3 Funktionen:

- `rng.shuffle`
- `rng.permutation`
- `rng.permuted`

Vorsicht: Dokumentation genau lesen.

- `rng.shuffle` verändert das Array inplace (Mutation!), `rng.permuted` und `rng.permutation` geben ein neues Array zurück.
- `rng.shuffle` und `rng.permutation` mischen das Array entlang einer Achse (default `axis=0`). Das heisst, es werden nur Zeilen (bzw. Spalten) vertauscht aber nicht die Elemente innerhalb einer Zeile (bzw. Spalte).
- `rng.permuted` mischt das ganze Array.

```
rng = np.random.default_rng(42)
x = np.arange(8).reshape(4, 2)

print("x before\n", x)
print("shuffle\n", rng.shuffle(x))
print("x after\n", x)
```

```
x before
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
```

```
shuffle
None
x after
[[6 7]
 [4 5]
 [2 3]
 [0 1]]

rng = np.random.default_rng(42)
x = np.arange(8).reshape(4, 2)

print("x before\n", x)
print("permutation\n", rng.permutation(x))
print("x after\n", x)

x before
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
permutation
[[6 7]
 [4 5]
 [2 3]
 [0 1]]
x after
[[0 1]
 [2 3]
 [4 5]
 [6 7]]

rng = np.random.default_rng(42)
x = np.arange(8).reshape(4, 2)

print("x before\n", x)
print("permuted\n", rng.permuted(x))
print("x after\n", x)

x before
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
permuted
[[3 4]
 [2 7]
 [6 1]
 [5 0]]
x after
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
```

3. Inversionsmethode (Inverse Transform Sampling) Wie werden nun Verteilungen erzeugt, die nicht Gleichverteilungen sind? Die einfachsten Verfahren sind:

- Inversionsmethode
- Verwerfungsmethode

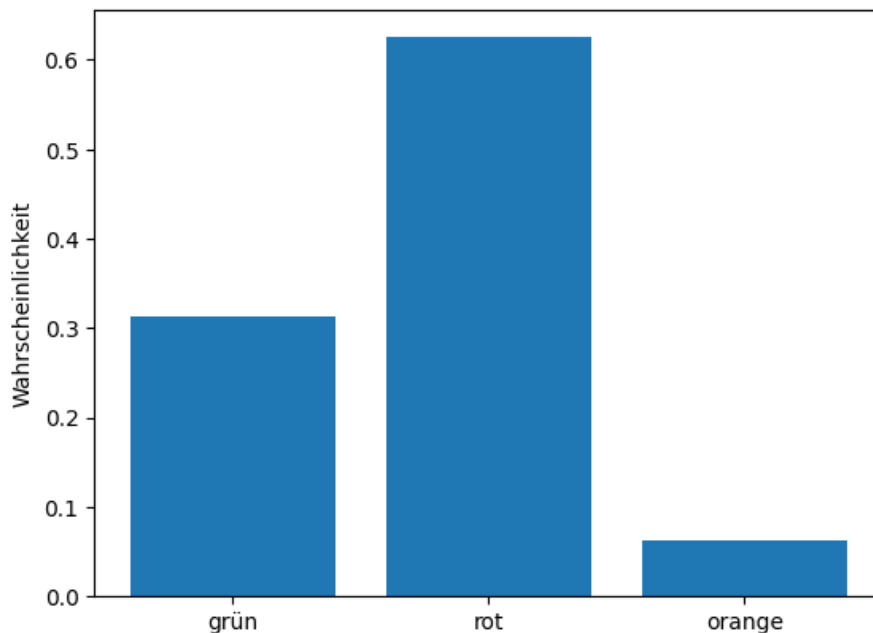
Hier diskutieren und implementieren wir die Inversionsmethode für diskrete Verteilungen. Die Inversionsmethode für stetige Verteilungen wird im Übungsblatt implementiert und kam bereits im Modul “Statistik & Wahrscheinlichkeit” vor (Serie 3, Aufgabe 3c).

a) Beispiel: Ampel-Farben Als Beispiel schauen wir an, welche Farbe wohl eine Verkehrsampel anzeigt, wenn man bei ihr ankommt. Wenn die Ampel immer 10 Sekunden auf grün steht, 20 Sekunden auf rot und dazwischen je 1 Sekunden auf Orange, dann ist die Wahrscheinlichkeit folgendermassen

$$P(\text{grün}) = \frac{10}{32}, \quad P(\text{rot}) = \frac{20}{32}, \quad P(\text{orange}) = \frac{2}{32}$$

```
x = np.array(["grün", "rot", "orange"])
p = np.array([10, 20, 2])
p = p / p.sum()

plt.bar(x, p)
plt.ylabel("Wahrscheinlichkeit")
plt.show()
```



Das Ziel ist nun, Realisierungen der Zufallsvariable mit Wahrscheinlichkeit $P(k)$ und $k \in \{\text{grün}, \text{rot}, \text{orange}\}$ zu erzeugen.

b) Implementation der Inversionsmethode Erster Schritt: Grün oder nicht grün?

Ist eine gleichverteilte Zufallszahl $u \sim U(0, 1)$ kleiner oder gleich $P(\text{grün})$, dann ist die Ampel grün, sonst nicht.

```
rng = np.random.default_rng()

u = rng.uniform()
```



```

if u < p[0]:
    print(f"u={u} -> grün")
else:
    print(f"u={u} -> nicht grün")

u=0.06930910570835769 -> grün

```

Zweiter Schritt: Wie bestimmt man, welches Licht an ist, wenn nicht das grüne?
Dazu schauen wir die kumulative Verteilungsfunktion (CDF) an.

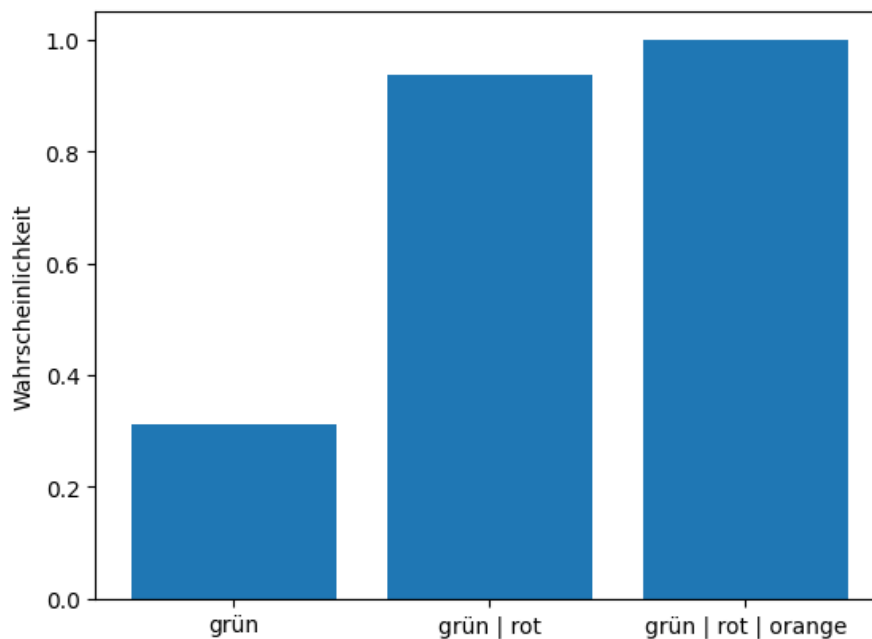
```

# Kumulative Verteilungsfunktion
# p = [10, 20, 2] / 32
# cdf = [10, 10+20, 10+20+2] / 32

cdf = np.cumsum(p)
xx = ["grün", "grün | rot", "grün | rot | orange"]

plt.bar(xx, cdf)
plt.ylabel("Wahrscheinlichkeit")
plt.show()

```



Nun können wir auf die CDF würfeln und erhalten eine Realisierung der Zufallsvariable.

```

u = rng.uniform(0, 1)
i = np.searchsorted(cdf, u)

print(cdf)
print(f"{u} -> {i} -> {x[i]}")

[0.3125 0.9375 1.    ]
0.19478549059355943 -> 0 -> grün

n = 1000
u = rng.uniform(0, 1, n)
i = np.searchsorted(cdf, u)
lights = x[i]
lights[:20]

```

```
array(['rot', 'grün', 'rot', 'rot', 'grün', 'grün', 'orange', 'rot',
      'rot', 'rot', 'grün', 'grün', 'grün', 'rot', 'rot', 'rot', 'rot',
      'grün', 'orange', 'grün'], dtype='<U6')
```

Die Klasse `collections.Counter` ist ein nützliches Werkzeug, um die Häufigkeit von Diskreten Werten in einer Liste zu zählen. Wir können damit auch die relativen Häufigkeiten berechnen und in einem Säulendiagramm darstellen.

```
from collections import Counter

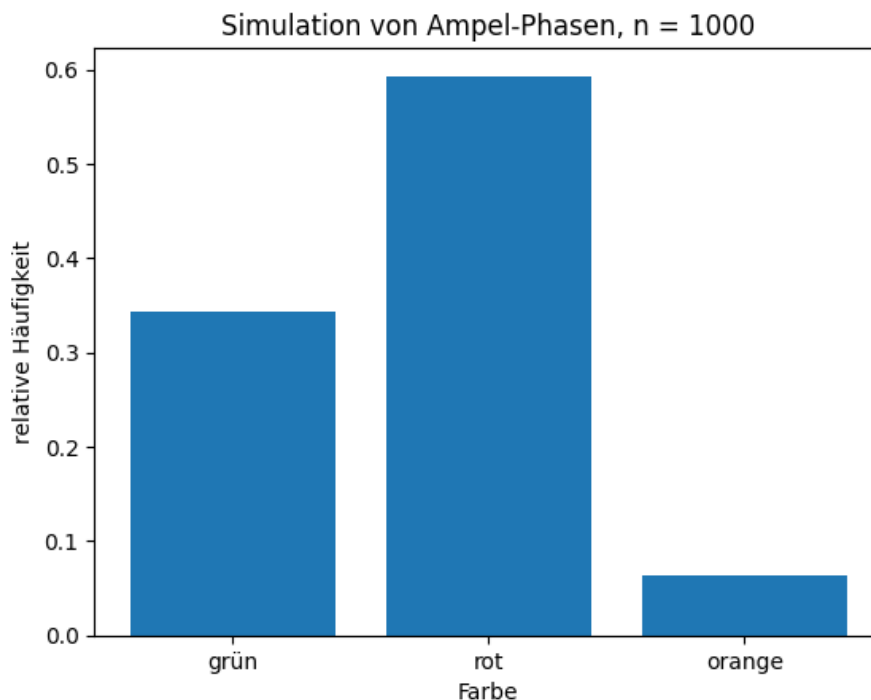
counter = Counter(lights)
counter

Counter({np.str_('rot'): 593, np.str_('grün'): 343, np.str_('orange'): 64})
```

Der `Counter` kann wie ein Dictionary verwendet werden, um die Häufigkeiten der einzelnen Werte auszugeben.

```
frequencies = np.array([counter[xi] for xi in x]) / len(lights)

plt.bar(x, frequencies)
plt.xlabel("Farbe")
plt.ylabel("relative Häufigkeit")
plt.title(f"Simulation von Ampel -Phasen, n = {n}")
plt.show()
```



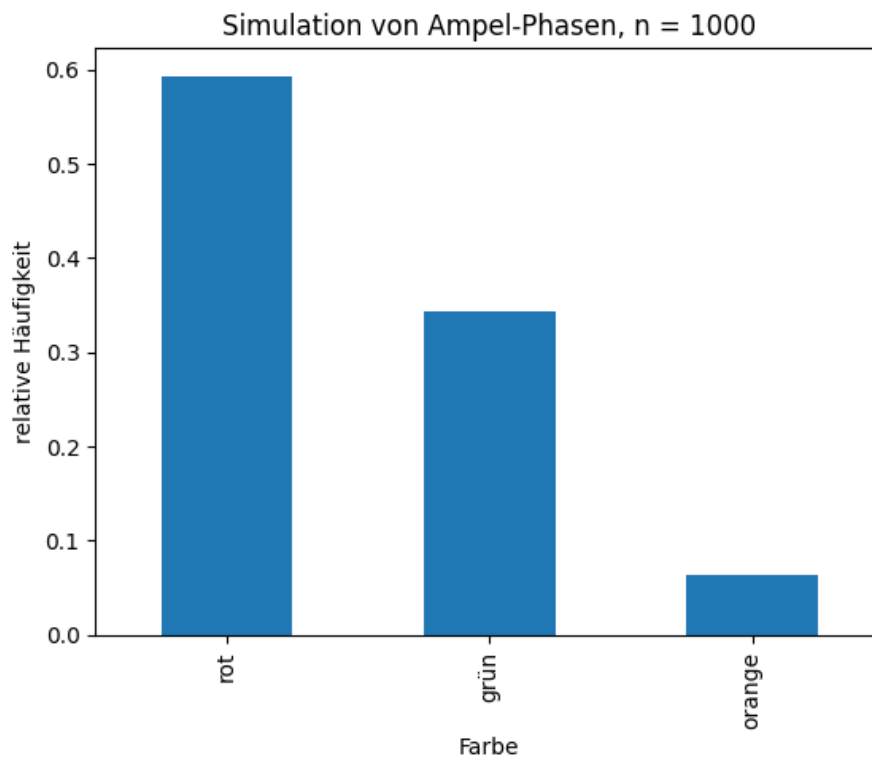
Alternative mit Pandas:

```
import pandas as pd

pd_freq = pd.Series(lights).value_counts(normalize=True)
pd_freq

rot      0.593
grün     0.343
orange   0.064
Name: proportion, dtype: float64
```

```
pd_freq.plot(kind="bar")
plt.xlabel("Farbe")
plt.ylabel("relative Häufigkeit")
plt.title(f"Simulation von Ampel -Phasen, n = {n}")
plt.show()
```

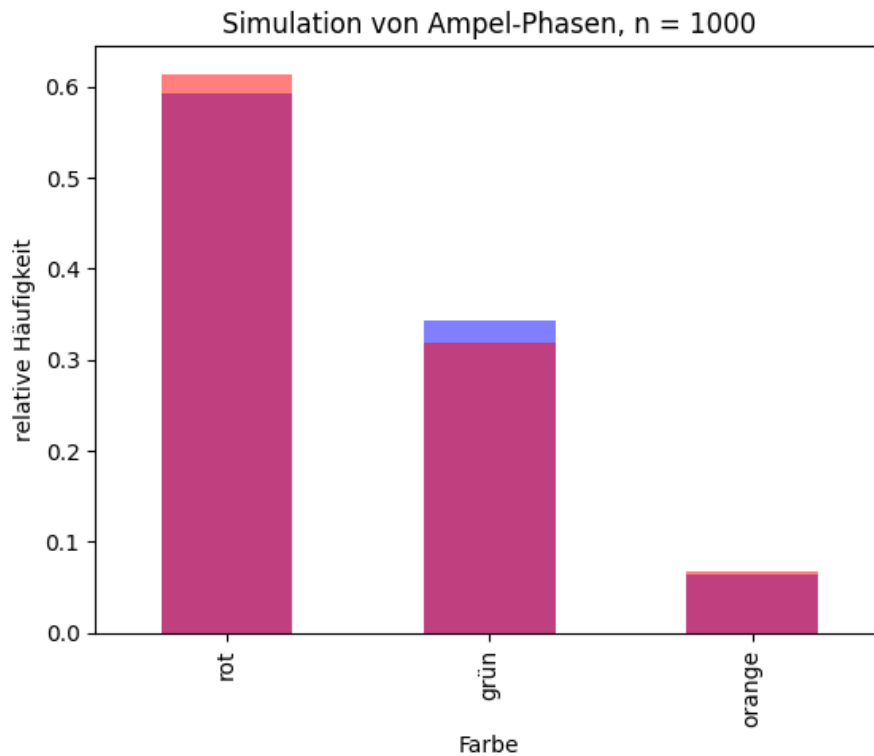


c) Direkte Variante mit `rng.choice`

```
lights2 = rng.choice(x, size=n, p=p)
pd_freq2 = pd.Series(lights2).value_counts(normalize=True)

pd_freq.plot(kind="bar", color="b", alpha=0.5)
pd_freq2.plot(kind="bar", color="r", alpha=0.5)

plt.xlabel("Farbe")
plt.ylabel("relative Häufigkeit")
plt.title(f"Simulation von Ampel -Phasen, n = {n}")
plt.show()
```



d) Alternative: Verwerfungsmethode (Rejection Sampling) (optional) Erklärung:

- “Zufallszahlen generieren: Verwerfungsmethode 01: Die Idee” - LernenInVerschiedenenFormen
- “Zufallszahlen generieren: Verwerfungsmethode 02: Algorithmus” - LernenInVerschiedenenFormen

Übung: Implementiere die Verwerfungsmethode für die kontinuierliche Dreiecksverteilung im Intervall $[0, 1]$ mit der Wahrscheinlichkeitsdichte ($a = 0, b = 1$):

$$f_c(x) = \begin{cases} \frac{2x}{c} & \text{wenn } 0 \leq x < c \\ \frac{2(1-x)}{1-c} & \text{wenn } c \leq x \leq 1 \\ 0 & \text{sonst} \end{cases} \quad (156)$$

0.11.4 Bootstrapping

```
import numpy as np
import matplotlib.pyplot as plt
```

Der Standardfehler ist die Standardabweichung der Stichprobe geteilt durch die Wurzel der Anzahl der Beobachtungen.

Wenn die Anzahl der Beobachtungen klein ($n \leq 30$) und die zugrundeliegende Verteilung fern von normal ist, dann ist der Standardfehler manchmal kein gutes Mass für den Fehler der Schätzung des Erwartungswerts.

Weiter ist es bei anderen Statistiken (z.B. Median, Varianz, Quantile, Korrelationskoeffizienten) nicht immer analytisch möglich, den Standardfehler zu bestimmen.

Eine Alternative bietet das **Bootstrapping**: Dabei zieht man wiederholt Stichproben mit Zurücklegen aus den Daten und berechnet für jede dieser Stichproben die Statistik, deren Fehler zu bestimmen ist. Die Standardabweichung über diese Schätzungen ist ein guter Indikator für den Fehler der ursprünglichen Statistik.

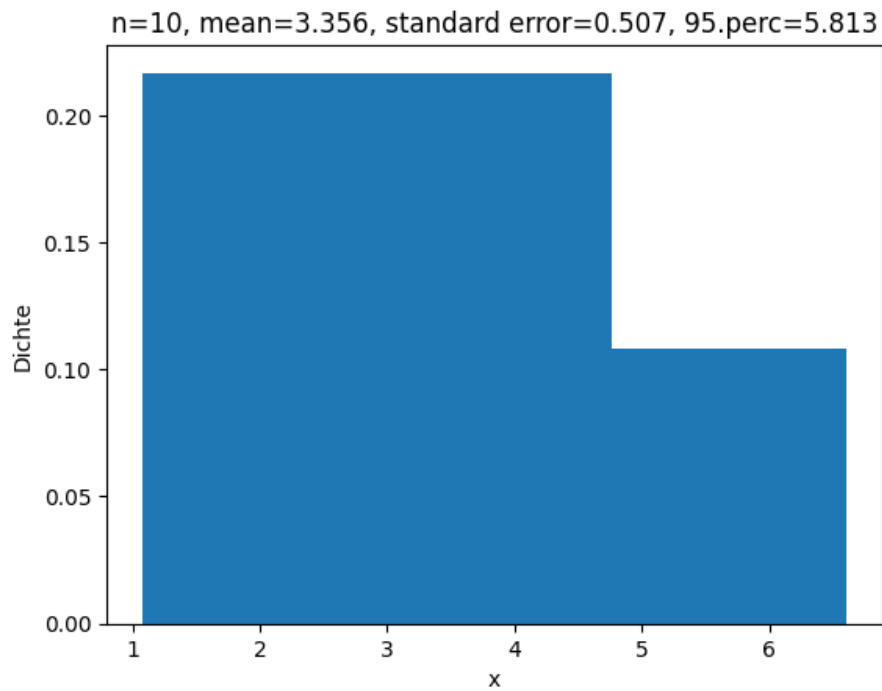
1. Messdaten simulieren

```
rng = np.random.default_rng()

n_data = 10
data = rng.gamma(1, 1, n_data)
data = rng.triangular(0, 0.5, 7, n_data)
# data = np.array([0.1, 0.2, 0.15, 0.25, 0.11, 0.08, 0.31])

data_mean = np.mean(data)
data_std = np.std(data)
data_se = data_std / np.sqrt(n_data)
data_95perc = np.percentile(data, 95)

plt.hist(data, bins="fd", density=True)
plt.xlabel("x")
plt.ylabel("Dichte")
plt.title(
    f"n={len(data)}, mean={data_mean:.3f}, standard error={data_se:.3f}, 95.perc={data_95perc:.3f}"
)
plt.show()
```



2. Bootstrapping

```
# Anzahl Subsamples sollte sehr gross sein
n_subsamples = 10_000
# Anzahl Samples pro Subsample sollte gleich der Anzahl Beobachtungen sein
subsample_size = len(data)

rng = np.random.default_rng()
subsamples = rng.choice(data, (n_subsamples, subsample_size), replace=True)
subsamples.shape

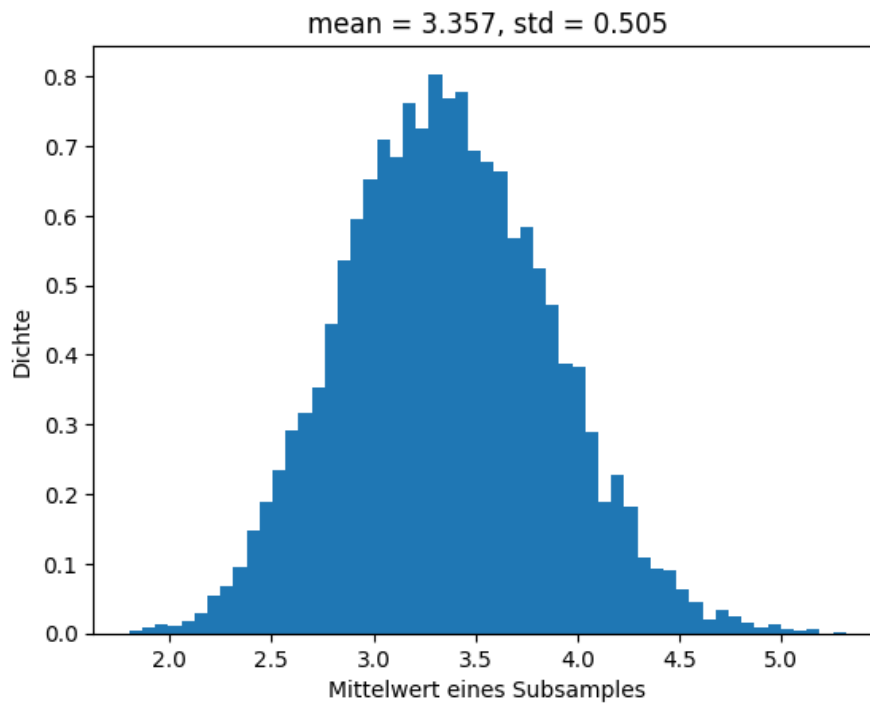
(10000, 10)
```

3. Fehler des Mittelwerts schätzen

```
# Für jedes Subsample (Zeile) den Mittelwert berechnen
means = subsamples.mean(axis=1)

means_mean = means.mean()
means_std = means.std()

plt.hist(means, bins="fd", density=True)
plt.xlabel("Mittelwert eines Subsamples")
plt.ylabel("Dichte")
plt.title(f"mean = {means_mean:.3f}, std = {means_std:.3f}")
plt.show()
```



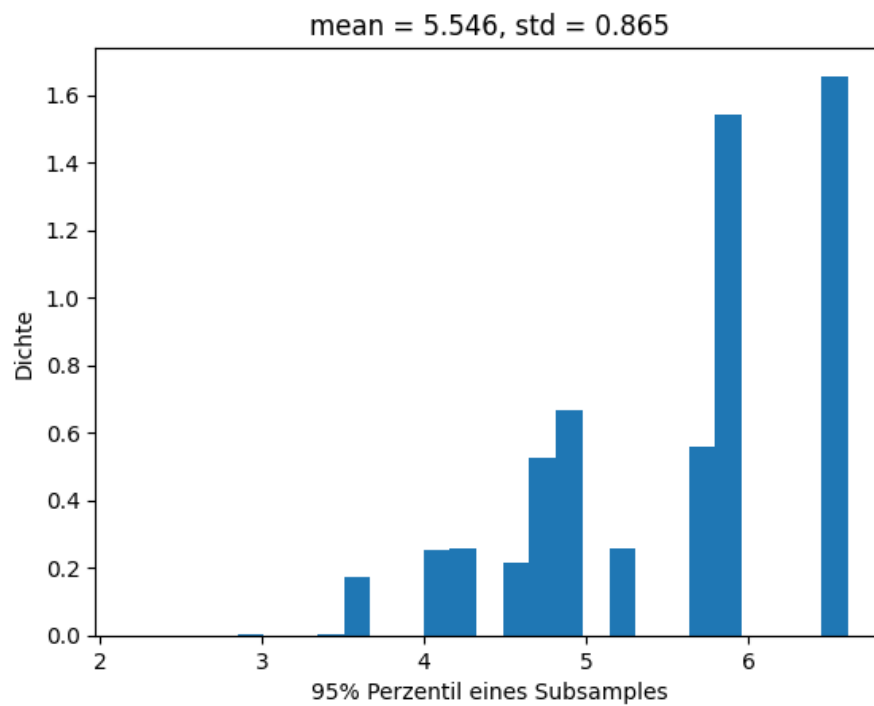
Die Standardabweichung der Mittelwerte aller Subsamples `means_std` ist nun eine gute Schätzung für den Standardfehler des Mittelwerts der Originalstichprobe. Im Fall des Mittelwerts ist allerdings die Formel $SE = \sigma / \sqrt{n}$ in den meisten Fällen bereits eine sehr gute Schätzung für den Standardfehler.

3. Bootstrapping für das 95. Perzentil

```
# Für jedes Subsample (Zeile) das 95. Perzentil berechnen
percentiles = np.percentile(subsamples, 95, axis=1)
```

```
percentiles_mean = percentiles.mean()
percentiles_std = percentiles.std()
```

```
plt.hist(percentiles, bins="fd", density=True)
plt.xlabel("95% Perzentil eines Subsamples")
plt.ylabel("Dichte")
plt.title(f"mean = {percentiles_mean:.3f}, std = {percentiles_std:.3f}")
plt.show()
```



Hier liefert das Bootstrapping mit der Standardabweichung über die Subsamples `percentiles_std` eine Schätzung des Fehlers des 95. Perzentsils der Originalstichprobe. Das Bootstrapping ist hier besonders nützlich, weil es keine Annahmen über die zugrundeliegende Verteilung macht.

0.12 Stochastische Prozesse

0.12.1 Zufallszahlen Simulieren

Arbeitsblatt zur Einarbeitung - Lösungen

Dieses Arbeitsblatt soll durchgearbeitet werden, bevor mit der Vertiefung begonnen wird. Die Lösungen werden zwischen Freitag Abend und Montag Morgen hochgeladen.

Das Ziel ist, ein einfaches Verständnis für den Nutzen von Zufallszahlen in der Simulation von stochastischen Prozessen zu erlangen.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Stochastische Prozesse

Ein stochastischer Prozess ist ein Modell, das die zeitliche Entwicklung eines Systems oder Phänomens beschreibt. Dabei wird berücksichtigt, dass Zufallseinflüsse eine Rolle spielen können. Im Gegensatz zu deterministischen Prozessen, bei denen der zukünftige Zustand eines Systems eindeutig aus dem aktuellen Zustand bestimmt werden kann, können bei stochastischen Prozessen zufällige Ereignisse auftreten, welche die zukünftige Entwicklung unsicher machen.

Stochastische Prozesse sind in vielen wissenschaftlichen Disziplinen von grosser Bedeutung, wie zum Beispiel in der Physik, Chemie, Biologie, Ökonomie, Informatik und Statistik. Sie werden häufig verwendet, um natürliche Phänomene, wirtschaftliche Entwicklungen oder technische Systeme zu modellieren, bei denen Unsicherheit oder zufällige Fluktuationen eine Rolle spielen. Beispiele für stochastische Prozesse:

- Wienerprozess (Brownsche Bewegung)
- Poisson-Prozess
- Random Walk

Markow-Ketten sind ein spezieller Typ von stochastischen Prozessen, bei denen die Zustandsänderung des Systems in einem Zeitschritt Δt nur vom aktuellen Zustand (zur Zeit t) abhängt und nicht von den vorherigen Zuständen (Historie). Sie haben also ein "kurzes Gedächtnis".

Man kann sich das als ein Spiel vorstellen, bei dem man eine Entscheidung nur abhängig von der aktuellen Spielsituation und nicht von vorherigen Entscheidungen trifft. Die Wahrscheinlichkeiten, von einem Zustand des Systems in einen anderen Zustand überzugehen, sind konstant und werden in einer sogenannten Übergangsmatrix dargestellt.

Schau dir dazu eines der beiden Videos an:

- "Markoff Kette, Markov Kette, Übergangsprozess, stochastischer Prozess" - Daniel Jung
- "Markov Chains Clearly Explained! Part - 1" - Normalized Nerd

Weitere Videos (optional):

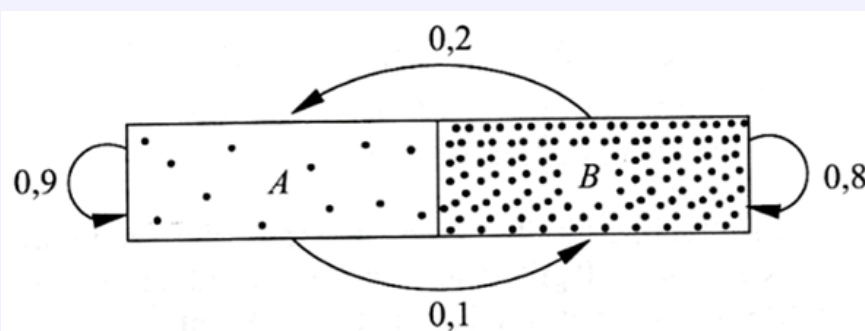
- "Stochastische Prozesse, Übergangsgraph, Übergangsdiagramm, Zustände" - Fit im Mathe-Abi"
- "Markoff Kette, Markov Kette, Übergangsprozess, stochastischer Prozess" - Mathe by Daniel Jung
- "Introducing Markov Chains" - Harvard Online
- "Intro to Markov Chains & Transition Diagrams" - Dr. Trefor Bazett

Diffusion

Als erstes Beispiel betrachten wir ein einfaches Modell der Diffusion.

Unter Diffusion versteht man in der Physik einen Vorgang, bei dem ein Stoff aufgrund molekularer Bewegung in einen anderen eindringt oder ihn ganz durchdringt.

Modell: Ein mit Teilchen gefüllter Kasten ist durch eine durchlässige Wand in zwei Hälften A und B geteilt. Die Verteilung der Teilchen auf die beiden Hälften ändert sich jeweils nach Ablauf einer festen Zeiteinheit Δt : 10 % der sich in A befindlichen Teilchen gelangen nach B (die restlichen 90 % bleiben in A) und 20 % der sich in B befindlichen Teilchen gelangen nach A (die restlichen 80 % bleiben in B).



Meyer, J. (o. J.). *Modellieren und Rechnen mit Matrizen* [Skript]. Mathe-Sport-Lehrer. Abgerufen 27. Januar 2026, von <https://www.maspole.de/index.php/mathematikunterricht/jahrgang-13-g9-q2-phase/modellieren-mit-matrizen/download/88-modellieren-und-rechnen-mit-matrizen-sse/1036-13-modellieren-und-rechnen-mit-matrizen-skript>

Das Modell ist ein Markow-Prozess, da der Übergang von einem Zustand in den nächsten nur vom aktuellen Zustand (Anzahl Teilchen in A oder B) abhängt und nicht von den vorherigen Zuständen.

Arbeitsauftrag

Aufgabe 1: Vorbereitungen

a) Repetition `rng.choice` Bevor wir direkt mit der Implementierung der Diffusion beginnen, kommt hier eine kleine Repetition der Funktion `rng.choice`. Hierfür wird das folgende Beispiel verwendet:

Wir werfen eine unfaire Münze, welche zu 60 % Kopf und zu 40 % Zahl zeigt. Schreibe hierfür eine Funktion `toss_coin(n, rng)`, welche das Ergebnis von `n` Münzwürfen zurückgibt. Als Argument nimmt sie einen NumPy Random Generator `rng`. Benutze hierfür die Funktion `rng.choice`.

Teste deine Funktion, indem du 5 Würfe simulierst. Mit dem Default Random Generator und dem Seed 42 sollte das Ergebnis ["Zahl", "Kopf", "Zahl", "Zahl", "Kopf"] sein.

```
rng = np.random.default_rng(42)

# Die möglichen Ergebnisse eines Münzwurfs
outcomes = ["Kopf", "Zahl"]

# Die Wahrscheinlichkeiten für die beiden Ergebnisse
probabilities = [0.6, 0.4]

def toss_coin(n, rng):
    return rng.choice(outcomes, p=probabilities, size=n)

toss_coin(5, rng)

array(['Zahl', 'Kopf', 'Zahl', 'Zahl', 'Kopf'], dtype='<U4')
```

b) Repetition Häufigkeiten zählen und darstellen mit Pandas Führe 1000 Münzwürfe mit der Funktion `toss_coin` aus Aufgabe 1 a) durch und speichere das Ergebnis in einer Variablen `coins`.

Bestimme die absoluten Häufigkeit der Ergebnisse mit der Funktion `pd.Series.value_counts`.

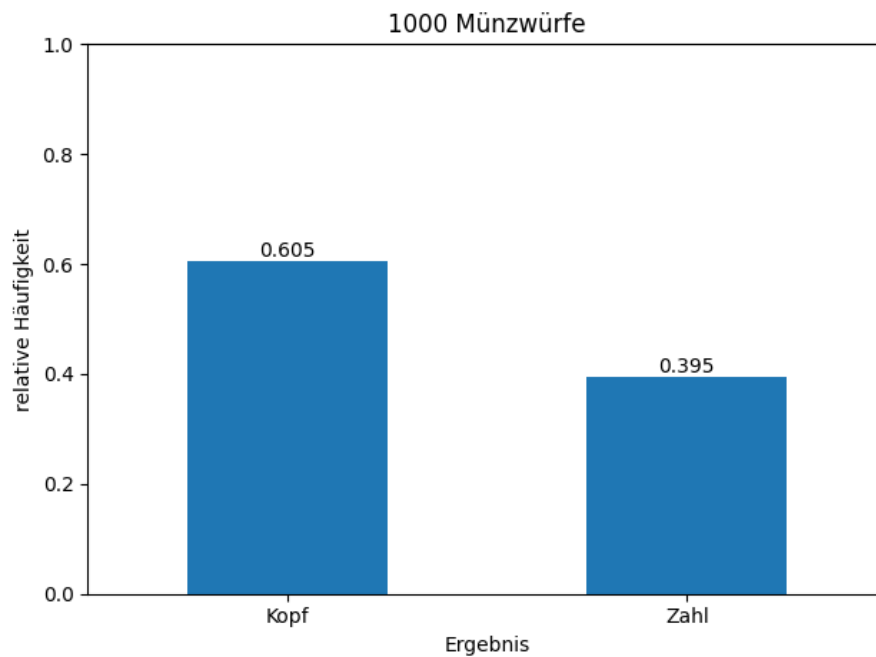
Dann bestimme die relativen Häufigkeiten mit der Funktion `pd.Series.value_counts` und dem Argument `normalize=True` und stelle diese mit der Funktion `pd.Series.plot` dar.

```
n = 1000
coins = toss_coin(n, rng)

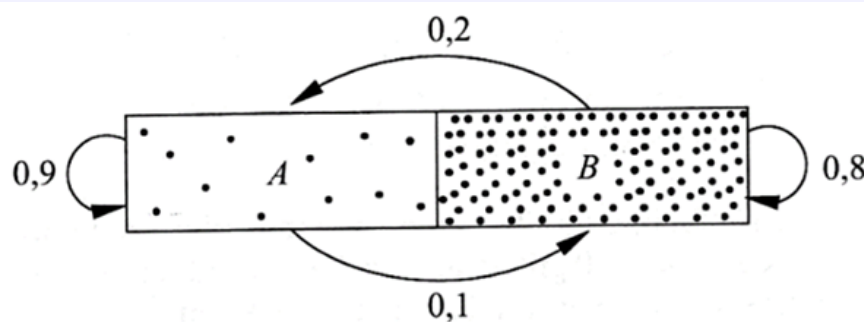
pd.Series(coins).value_counts()

Kopf    605
Zahl    395
Name: count, dtype: int64

ax = (
    pd.Series(coins)
    .value_counts(normalize=True)
    .plot.bar(
        title=f"{n} Münzwürfe",
        xlabel="Ergebnis",
        ylabel="relative Häufigkeit",
        rot=0,
    )
)
ax.bar_label(ax.containers[0])
ax.set_ylim(0, 1)
plt.tight_layout()
plt.show()
```



Aufgabe 2: Implementierung der Diffusion Wir implementieren eine einfache Version des Diffusionsmodells. Hier ist nochmals das Prozessdiagramm:



Meyer, J. (o. J.). *Modellieren und Rechnen mit Matrizen* [Skript]. Mathe-Sport-Lehrer. Abgerufen 27. Januar 2026, von <https://www.maspole.de/index.php/mathematikunterricht/jahrgang-13-g9-q2-phase/modellieren-mit-matrizen/download/88-modellieren-und-rechnen-mit-matrizen/1036-13-modellieren-und-rechnen-mit-matrizen-skript>

a) Implementierung Schritt 1 Programmiere zuerst eine Funktion namens `markov_next(x, rng)`, die den aktuellen Zustand `x` eines Teilchens und einen Random Generator `rng` als Parameter nimmt. Die Funktion soll den nächsten Zustand zurückgeben. Die Übergangswahrscheinlichkeiten sind $p_{AB} = 0.1$ und $p_{BA} = 0.2$. Benutze hierfür die Funktion `rng.choice()`.

Verwende `A = 0` und `B = 1` für die möglichen Zustände, anstatt mit den Strings "A" und "B" zu arbeiten.

Test die Funktion mit dem Anfangszustand `x = A` und dem Seed 12345678. Das Ergebnis sollte 1 sein (also Zustand B).

```
A, B = 0, 1
p_AB = 0.1
p_BA = 0.2
```

```
def markov_next(x, rng):
    if x == A:
        return rng.choice((A, B), p=(1 - p_AB, p_AB))
    else:
        return rng.choice((A, B), p=(p_BA, 1 - p_BA))
```

```
rng = np.random.default_rng(12345678)
markov_next(A, rng)

np.int64(1)
```

b) Implementierung des gesamten Prozesses Nun schreibst du eine Funktion `markov_process(n, x0, seed=12345678)`, die eine Anzahl Schritte `n`, einen Startwert `x0` und den Seed mit Standardwert 12345678 als Parameter nimmt. Sie soll einen Default Random Generator und die Funktion `markov_next()` benutzen, um ausgehend vom Startwert eine Liste von `n` Schritten in der Markov-Kette zu erzeugen. Die Funktion soll den Verlauf der Zustände inkl. des Anfangszustandes als Array zurückgeben.

```
def markov_process(n, x0, seed=12345678):
    rng = np.random.default_rng(seed)
    x = x0
    xs = [x]
    for _ in range(n):
```

```

x = markov_next(x, rng)
xs.append(x)
return np.array(xs)

```

c) Testen der Funktion Teste die Funktion `markov_process()` mit der Anzahl Schritte $n=10$, dem Startwert $x_0=A$, einem Random Generator mit dem Seed 12345678.

Das Resultat sollte so aussehen: [0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1].

```

markov_process(10, A)

array([0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1])

```

d) Verlauf mit Stemplot darstellen Um sich einen Überblick über die zeitliche Veränderung in der Markov-Kette zu machen, plotten wir nun die Zustände in Abhängigkeit der Zeit (= Schritt). Hierfür verwenden wir die Funktion `plt.stem`. Wir nehmen den gleichen Seed und Startwert wie in Aufgabe 2c), jedoch erhöhen wir die Anzahl Schritte auf $n = 30$.

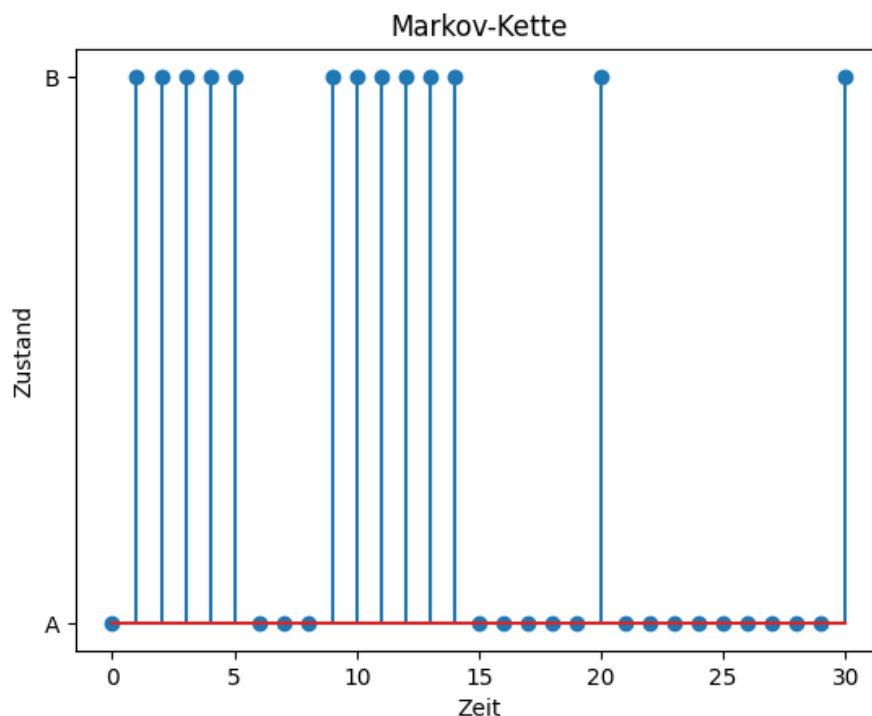
Verändere den Seed und beobachte, wie sich der Verlauf der Zustände ändert.

```

y = markov_process(30, A, 12345678)

plt.stem(y)
plt.xlabel("Zeit")
plt.ylabel("Zustand")
plt.yticks((A, B), ("A", "B"))
plt.title("Markov -Kette")
plt.show()

```



e) Statistik mit Barplot darstellen Erzeuge nun eine Markov-Kette `chain` mit $n = 1000$ Schritten und dem Seed 12345678.

Zähle die absoluten Häufigkeiten mit Pandas und gib das Resultat aus.

Dann erstelle einen Barplot der relativen Häufigkeit mit Pandas.

Was fällt dir auf? Beschreibe die Verteilung der Zustände. Und wie hängt diese von Seed und Anzahl Schritten ab?

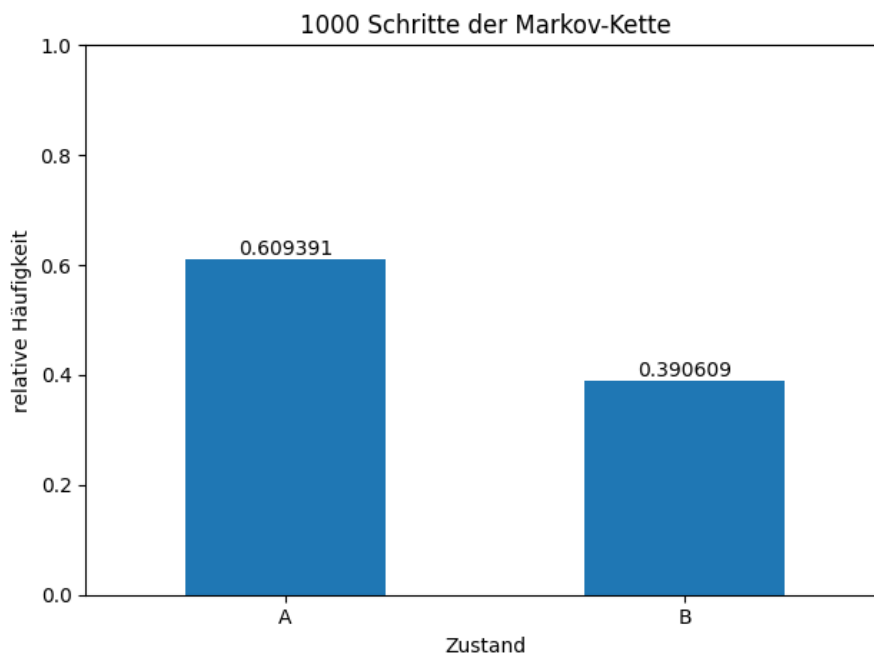
```

n = 1000
chain = markov_process(n, A)
pd.Series(chain).value_counts()

0    610
1    391
Name: count, dtype: int64

ax = (
    pd.Series(chain)
    .value_counts(normalize=True)
    .rename({A: "A", B: "B"})
    .plot(
        kind="bar",
        title=f"{n} Schritte der Markov -Kette",
        xlabel="Zustand",
        ylabel="relative Häufigkeit",
        rot=0,
    )
)
ax.bar_label(ax.containers[0])
ax.set_ylim(0, 1)
plt.tight_layout()
plt.show()

```



Die relative Häufigkeit der Zustände sollte sich bei genügend vielen Schritten einem Gleichgewicht annähern. Dieses Gleichgewicht wird als stationäre Verteilung bezeichnet. Und die stationäre Verteilung hängt nicht vom Startwert und dem Random Seed ab.

Für dieses Beispiel könnte die stationäre Verteilung analytisch berechnet werden. Wir können sie aber auch durch Simulation bestimmen, indem wir die Markov-Kette für eine grosse Anzahl Seeds simulieren und dann die Verteilung der Häufigkeit der Zustände pro Seed bestimmen anschauen.

```

def get_relative_frequency_A(chain):
    return pd.Series(chain).value_counts(normalize=True)[A]

```

```

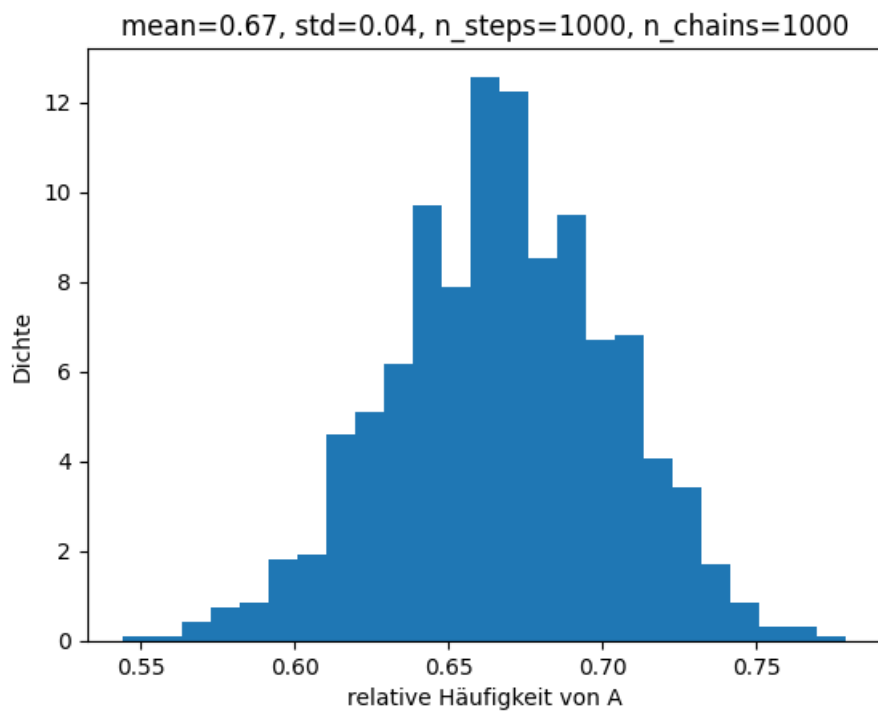
n_steps = 1000
n_chains = 1000
chains = [markov_process(n_steps, A, None) for _ in range(n_chains)]
freqs = np.array([get_relative_frequency_A(chain) for chain in chains])

freqs[:10]

array([0.67932068, 0.66233766, 0.64335664, 0.67732268, 0.6013986 ,
       0.67432567, 0.67032967, 0.64335664, 0.6973027 , 0.62937063])

plt.hist(freqs, density=True, bins="fd")
plt.xlabel("relative Häufigkeit von A")
plt.ylabel("Dichte")
plt.title(
    f"mean={np.mean(freqs):.2f}, std={np.std(freqs):.2f}, n_steps={n_steps}, n_chains={n_chains}"
)
plt.show()

```



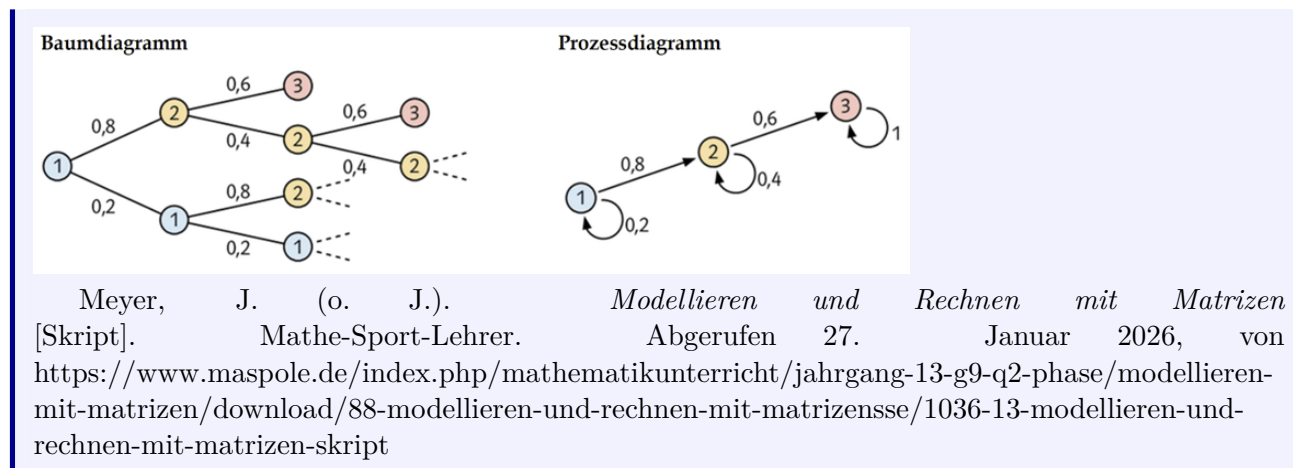
0.12.2 Stochastischer Prozess

Übungen - Lösungen

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Aufgabe 1: Computerspiel Bei einem Computerspiel gibt es drei Zustände: Level 1, Level 2 und Level 3. Eine Spielerin beginnt auf Level 1 und bewältigt die Übergänge von Level 1 nach Level 2 mit einer Wahrscheinlichkeit von 80 % und von 2 nach 3 mit einer Wahrscheinlichkeit von 60 %. Ein Level zurückfallen ist nicht möglich. Sie spielt so lange, bis sie Level 3 erreicht hat.

Der Ablauf (von Level zu Level) ist eine Markow-Kette. Sie kann durch ein “unendliches” Baumdiagramm oder ein Prozessdiagramm mit drei Zuständen beschrieben werden:



Schreibe eine Funktion `play_game(seed=None)`, die den Spielverlauf simuliert und die Anzahl der benötigten Versuche zurückgibt, bis die Spielerin das Level 3 erreicht und noch nicht gespielt hat. Der Parameter `seed` ist dabei der Seed für den Default Random Generator von NumPy und der Standardwert `None` bedeutet, dass der Seed “zufällig” gewählt wird.

```
p_12 = 0.8
p_23 = 0.6
```

```
def play_game(seed=None):
    rng = np.random.default_rng(seed)
    level = 1
    attempts = 0
    while level < 3:
        attempts += 1
        p = p_12 if level == 1 else p_23
        if rng.random() < p:
            level += 1
    return attempts
```

```
print(f"Die Spielerin erreichte Level 3 nach {play_game()} Versuchen.")
```

Die Spielerin erreichte Level 3 nach 4 Versuchen.

Alternative Implementation mit der Übergangsmatrix (engl. transition matrix):

```

levels = (1, 2, 3)
transition_matrix = (
    None,
    (1 - p_12, p_12, 0),
    (0, 1 - p_23, p_23),
    (0, 0, 1),
)

def play_game2(seed=None):
    rng = np.random.default_rng(seed)
    level = 1
    attempts = 0
    while level < 3:
        attempts += 1
        level = rng.choice(levels, p=transition_matrix[level])
    return attempts

```

```
print(f"Die Spielerin erreichte Level 3 nach {play_game2()} Versuchen.")
```

Die Spielerin erreichte Level 3 nach 3 Versuchen.

b) Mittlere Spieldauer mit Monte-Carlo-Simulation schätzen Simuliere das Spiel `play_game()` nun 10'000 Mal (Monte-Carlo-Simulation) mit unterschiedlichen Seeds (oder immer `seed=None`) und sammle die benötigten Versuche in einem Array.

Berechne den Mittelwert und die Standardabweichung der benötigten Versuche.

```

n = 10_000
attempts = np.array([play_game() for _ in range(n)])
attempts.mean(), attempts.std()

(np.float64(2.8817), np.float64(1.1399583808192297))

```

c) Relative Häufigkeiten der Anzahl Versuche Berechne die relative Häufigkeit, dass die Spielerin nach n Versuchen Level 3 erreicht hat und stelle sie mit einem Bar-Plot dar. Verwende dazu die Resultate der 10'000 Simulationen aus Teil b).

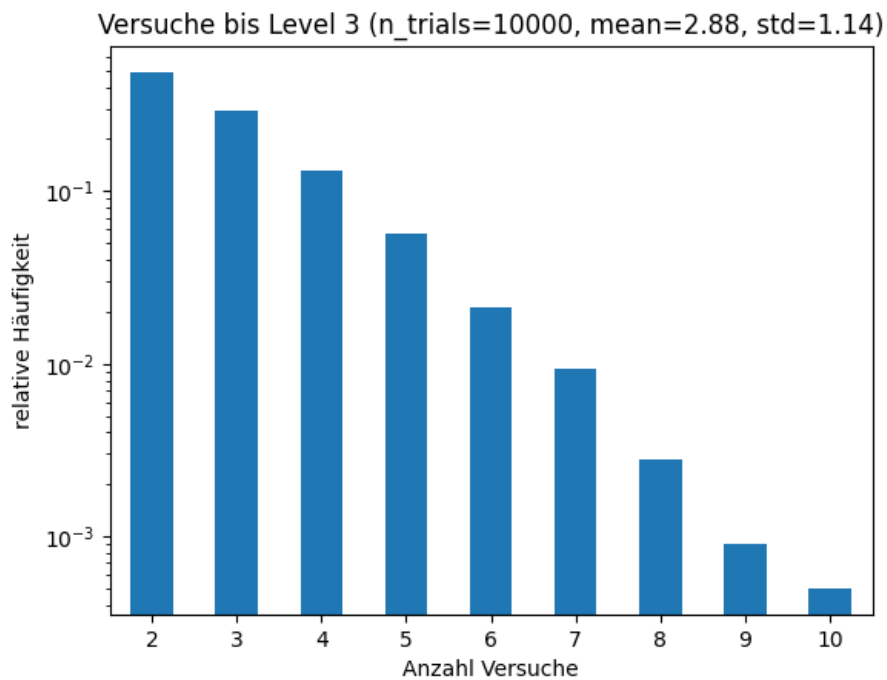
Hierfür kannst du entweder Pandas `pd.Series.values_counts`, `Counter` oder `np.bincount` verwenden.

Was fällt dir auf? Wenn du die y-Achse logarithmisch darstellst, siehst du, wie die Häufigkeit von der Anzahl Versuche abhängt. Was kannst du dazu sagen?

```

ax = (
    pd.Series(attempts)
    .value_counts(normalize=True)
    .sort_index()
    .plot(
        kind="bar",
        title=f"Versuche bis Level 3 (n_trials={n}, mean={attempts.mean():.2f}, std={attempts.st
        xlabel="Anzahl Versuche",
        ylabel="relative Häufigkeit",
        rot=0,
        logy=True,
    )
)

```



Aufgabe 2: Random Walk in 1D Der Random Walk (dt. “zufälliger Spaziergang”) ist ein Konzept aus der Stochastik, das in vielen Bereichen der Physik, Mathematik und Informatik Anwendung findet. Er beschreibt eine mathematische Idee, bei der ein Teilchen auf einem Gitter oder in einem Raum eine zufällige Bewegung ausführt.

Ein Random Walk kann auf verschiedene Weise durchgeführt werden, aber im Allgemeinen besteht er aus einer Abfolge von Schritten, die zufällig in verschiedene Richtungen durchgeführt werden. Jeder Schritt kann eine feste Länge haben oder eine zufällige Länge zwischen einem bestimmten Minimum und Maximum.

Random Walks sind in vielen Bereichen nützlich, da sie helfen, das Verhalten von Partikeln oder Molekülen in einer Flüssigkeit oder Gas zu modellieren, das Wachstum von Zellen in einem Gewebe zu simulieren oder das Verhalten von Tieren bei der Futtersuche zu untersuchen. Sie können auch in der Finanzmathematik verwendet werden, um den Preis von Wertpapieren oder Optionen zu modellieren.

Ein wichtiger Aspekt des Random Walks ist die Wahrscheinlichkeitsverteilung der Schritte, die das Teilchen durchführt. Wenn die Schritte zufällig sind und die Wahrscheinlichkeitsverteilung der Schrittlängen symmetrisch ist, d.h. sie hat das gleiche Gewicht für positive und negative Werte, dann wird der Random Walk als “einfacher Random Walk” bezeichnet.

Eine weitere wichtige Eigenschaft von Random Walks ist die Tatsache, dass sie dazu neigen, sich auszubreiten oder zu diffundieren. Das bedeutet, dass sich das Teilchen im Durchschnitt mit der Wurzel aus der Anzahl der durchgeführten Schritte von seinem ursprünglichen Ausgangspunkt entfernt. Dies ist ein grundlegendes Prinzip der Diffusionsphysik und wird in vielen Anwendungen wie der Molekularbiologie, der Physik von Flüssigkeiten und Gasen sowie der Finanzmathematik verwendet.

Optionales Material:

- “What is a Random Walk? | Infinite Series” - PBS Infinite Series
- “Random walks in 2D and 3D are fundamentally different (Markov chains approach)” - Mathemaniac
- “Random Walk” - Wikipedia

a) Random Walk simulieren Schreibe eine Funktion `random_walk_1d(n_steps, seed=None)`, die einen Random Walk mit `n` Schritten in 1D simuliert. Die Funktion soll einen NumPy Default Random Generator mit dem gegebenen Seed nutzen und die abgelaufenen Positionen x_i in einem Array zurückgeben.

- Startposition: $x_0 = 0$.

- Iterationsvorschrift: $x_{n+1} = x_n + d_{n+1}$.
- Der Schritt d_{n+1} wird zufällig aus der Menge $\{-1, 1\}$ gewählt. Das heisst, die Schrittlänge ist immer 1, die Richtung ist zufällig.

Verwende dazu die Funktion `rng.choice`.

Mit dem Seed 42 und 10 Schritten sollte die Ausgabe wie folgt aussehen: `[0, -1, 0, 1, 0, -1, 0, -1, 0, -1, -2]`

```
def random_walk_1d(n_steps, seed=None):
    rng = np.random.default_rng(seed)
    x = 0
    walk = [x]
    for _ in range(n_steps):
        x += rng.choice([-1, 1])
        walk.append(x)
    return np.array(walk)
```

```
random_walk_1d(10, 42)
```

```
array([ 0, -1,  0,  1,  0, -1,  0, -1,  0, -1, -2])
```

Alternative Implementierung mit NumPy

```
def random_walk_1d(n_steps, seed=None):
    rng = np.random.default_rng(seed)
    steps = rng.choice([-1, 1], n_steps)
    steps = np.hstack([0, steps])
    return np.cumsum(steps)
```

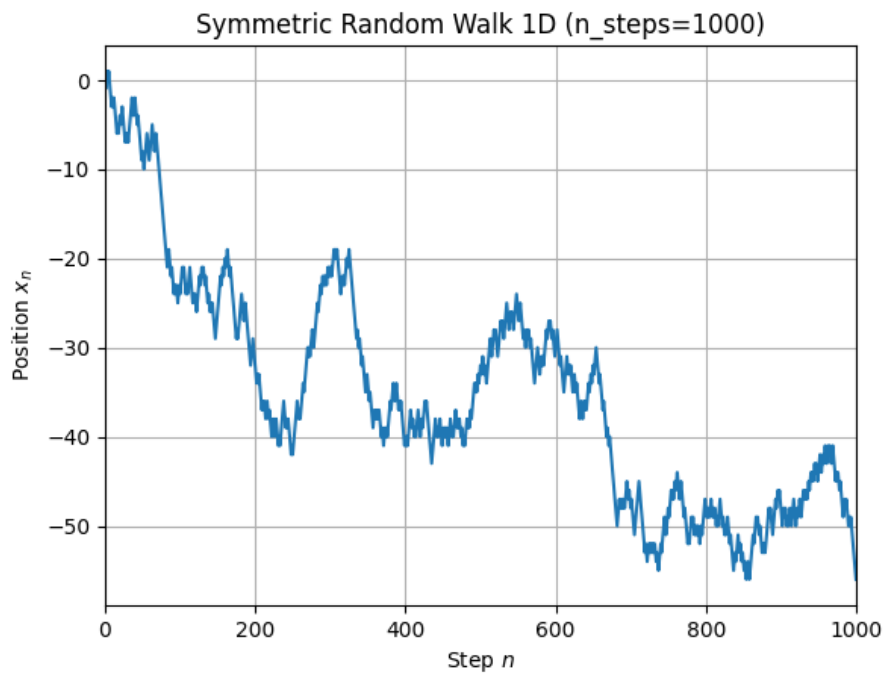
```
random_walk_1d(10, 42)
```

```
array([ 0, -1,  0,  1,  0, -1,  0, -1,  0, -1, -2])
```

b) Darstellung des Random Walks Simuliere einen `random_walk_1d()` mit 1000 Schritten und stelle den Verlauf dar.

```
n_steps = 1000
walk = random_walk_1d(n_steps)

plt.plot(walk)
plt.xlabel("Step $n$")
plt.ylabel("Position $x_n$")
plt.title(f"Symmetric Random Walk 1D (n_steps={n_steps})")
plt.grid()
plt.xlim(0, n_steps)
plt.show()
```

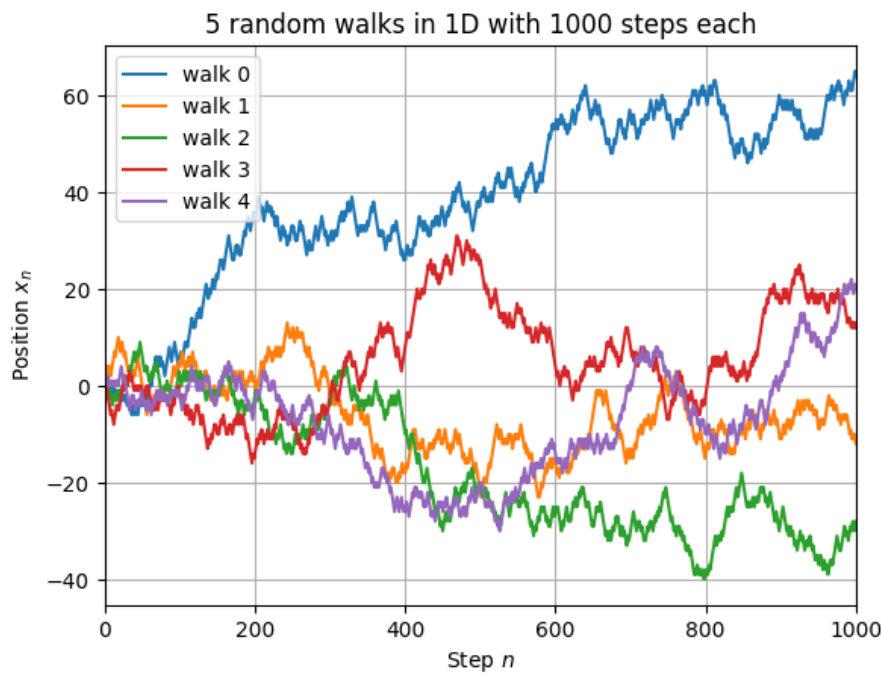


c) Darstellen mehrerer Random Walks Stelle 5 Random Walks mit je 1000 Schritten und unterschiedlichen Seeds in einem Plot dar. Verwende dazu die Funktion `random_walk_1d()` aus Aufgabe b).

```
n_steps = 1000
n_walks = 5

for i in range(n_walks):
    walk = random_walk_1d(n_steps)
    plt.plot(walk, label=f"walk {i}")

plt.xlabel("Step $n$")
plt.ylabel("Position $x_n$")
plt.legend()
plt.title(f"{n_walks} random walks in 1D with {n_steps} steps each")
plt.grid()
plt.xlim(0, n_steps)
plt.show()
```



d) Varianz des Random Walks Wie man in Aufgabe c) sehen kann, driften die Random Walks in 1D mit der Zeit (Anzahl Schritte) auseinander. Man kann zeigen, dass die Varianz der Aufenthaltsverteilung der Walks mit der Anzahl Schritte linear ansteigt:

$$\text{Var}(X_n) = n \quad (157)$$

Also ist die Standardabweichung nach n Schritten:

$$\sigma_n = \sqrt{\text{Var}(X_n)} = \sqrt{n} \quad (158)$$

Das Ziel ist nun, dieses theoretische Resultat mit einer Monte-Carlo-Simulation zu verifizieren.

- Erzeuge 500 Random Walks mit je 1000 Schritten und unterschiedlichen Seeds mithilfe von `random_walk_1d()`.
- Packe die resultierenden 500 Walks in ein Array zu einem 2D Array mit Shape (500, 1001). Das heisst jede Zeile ist ein Walk und die i -te Spalte entspricht den Positionen der Walks nach i Schritten.
- Berechne die Standardabweichung mit `np.std` pro Spalte, also jeweils über alle Zeilen. (Nutze die Option `axis`.)
- Stelle die Standardabweichung als Funktion des Schritts dar und vergleiche diese mit der theoretischen Standardabweichung $\sigma_n = \sqrt{n}$.
- Stelle dazu auch noch den Mittelwert der Walks dar. Was fällt dir auf? Warum ist das so?

```
n_steps = 1000
n_walks = 500
```

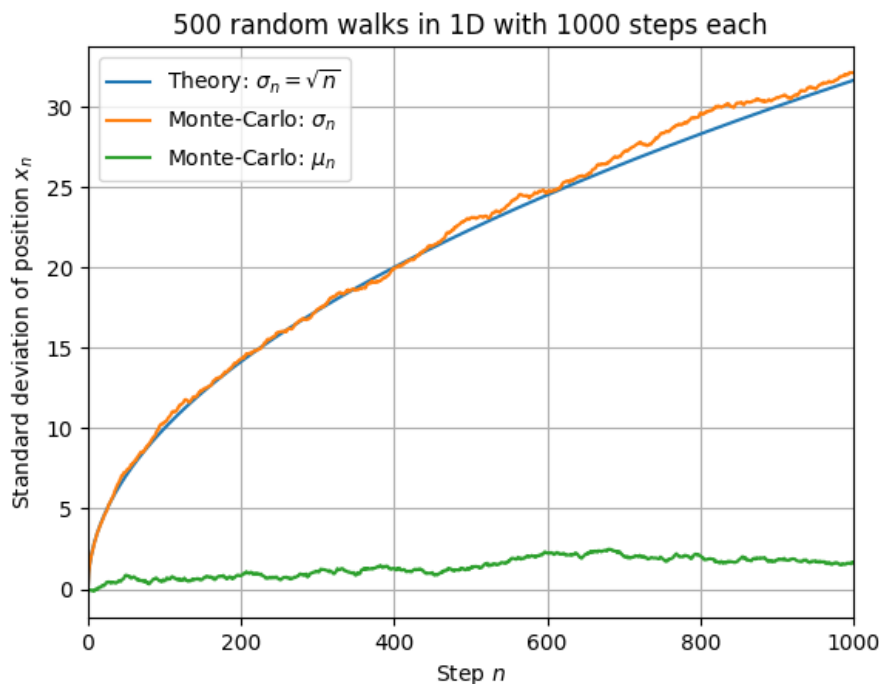
```
walks = np.array([random_walk_1d(n_steps) for _ in range(n_walks)])
walks.shape
```

```
(500, 1001)
```

```
sigma_theory = np.sqrt(np.arange(n_steps))
sigma_walks = np.std(walks, axis=0)
mean_walks = np.mean(walks, axis=0)
```

```
plt.plot(sigma_theory, label=r"Theory: $\sigma_n = \sqrt{n}$")
```

```
plt.plot(sigma_walks, label=r"Monte -Carlo:  $\sigma_n$ ")
plt.plot(mean_walks, label=r"Monte -Carlo:  $\mu_n$ ")
plt.xlabel("Step  $n$ ")
plt.ylabel("Standard deviation of position  $x_n$ ")
plt.title(f"{n_walks} random walks in 1D with {n_steps} steps each")
plt.grid()
plt.legend()
plt.xlim(0, n_steps)
plt.show()
```



Beobachtungen:

- Die Standardabweichung stimmt mit der theoretischen Vorhersage überein.
- Der Mittelwert der Walks ist ≈ 0 . Grund dafür ist, dass die Wahrscheinlichkeit für einen Schritt nach links und einen Schritt nach rechts gleich ist, und somit die Schritte symmetrisch verteilt sind.

Aufgabe 3: Random Walk in 2D

a) Random Walk 2D Funktion Schreibe eine Funktion `random_walk_2d(n_steps, seed=None)`, die einen Random Walk der Länge `n_steps` in 2D durchführt. Die Funktion soll mit dem angegebenen Seed einen NumPy Default Random Generator erstellen und ein Array der abgelaufenen x und y Positionen zurückgeben.

- Startposition $(x_0, y_0) = (0, 0)$.
- Es sind nur Schritte der Länge 1 erlaubt, entweder in positive oder negative x - oder in positive oder negative y -Richtung. Diagonale Schritte sind nicht erlaubt.
- Die Richtung soll zufällig gewählt werden, wobei alle Richtungen gleich wahrscheinlich sind. Verwende dazu die Funktion `rng.choice`.

```
def random_walk_2d(steps, seed=None):
    rng = np.random.default_rng(seed)
    possible_steps = np.array([(1, 0), (-1, 0), (0, 1), (0, -1)])
    x, y = 0, 0
    walk = [(x, y)]
    for _ in range(steps):
```

```

    dx, dy = rng.choice(possible_steps)
    x += dx
    y += dy
    walk.append((x, y))
return np.array(walk)

```

b) Darstellen des Random Walks in 2D Stelle den `random_walk_2d` aus Aufgabe a) mit 1000 Schritten für den Seed 42 in der x-y-Ebene dar.

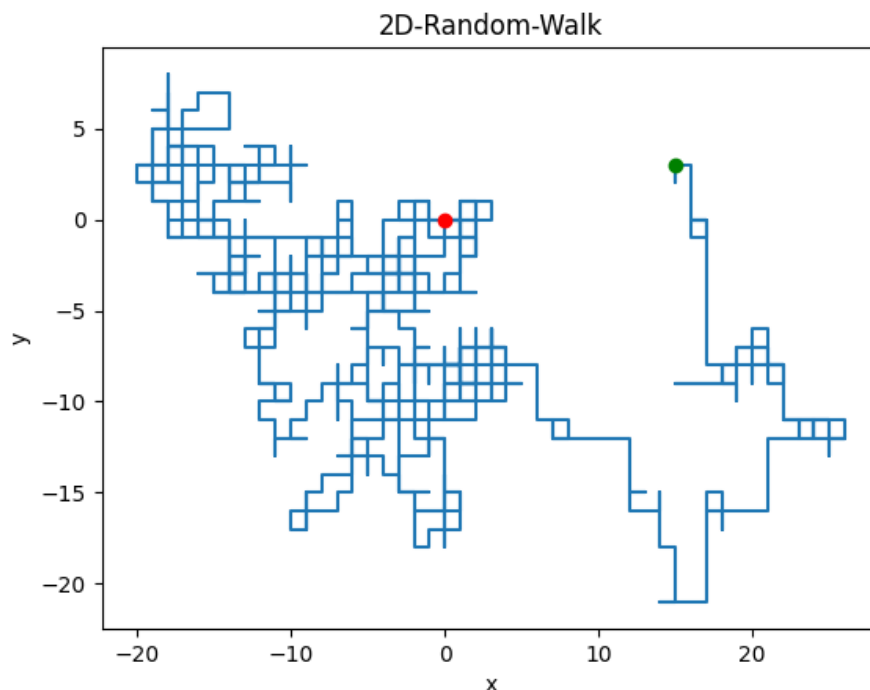
Markiere den Startpunkt mit einem grünen Punkt und den Endpunkt mit einem roten Punkt.

```

steps = 1000
x, y = random_walk_2d(steps, 42).T

plt.plot(x, y)
plt.plot(x[0], y[0], "ro", label="Start")
plt.plot(x[-1], y[-1], "go", label="Ende")
plt.xlabel("x")
plt.ylabel("y")
plt.title("2D -Random -Walk")
plt.show()

```



“A drunk man will find his way home, but a drunk bird may get lost forever.” - Shizuo Kakutani

c) Rückkehrzeit des Random Walks in 2D Das nächste Ziel ist es herauszufinden, nach wie vielen Schritten der 2D Random Walk wieder zum Ausgangspunkt zurückkehrt. Dies nennt man die **Rückkehrzeit** (engl. first return time)

Schreibe dazu eine Funktion `first_return_time(max_steps=1000, seed=None)`, die einen Random Walk in 2D durchführt bis er wieder am Ausgangspunkt angekommen ist. `max_steps` soll hierbei als Abbruchkriterium für die maximale Anzahl Schritte dienen. Die Funktion soll die Anzahl Schritte bis zur ersten Rückkehr zum Ursprung zurückgeben. Ansonsten gelten die gleichen Regeln wie in Aufgabe a).

- “Drunkard’s Walk” - Wikipedia

- “Random walks in 2D and 3D are fundamentally different (Markov chains approach)” - Mathematician

Probiere es mit verschiedenen Seeds aus.

```
def first_return_time(max_steps=1000, seed=None):
    rng = np.random.default_rng(seed)
    possible_steps = ((1, 0), (-1, 0), (0, 1), (0, -1))
    x, y = 0, 0
    steps = 0
    while steps < max_steps:
        steps += 1
        dx, dy = rng.choice(possible_steps)
        x += dx
        y += dy
        if x == 0 and y == 0:
            break

    return steps
```

```
first_return_time()
```

2

d) Darstellen der Verteilung der Rückkehrzeit Ähnlich wie in Aufgabe 2d) sollen nun viele Simulationen berechnet und jeweils die erste Rückkehrzeit bestimmt werden. Also wieder eine Monte-Carlo-Simulation.

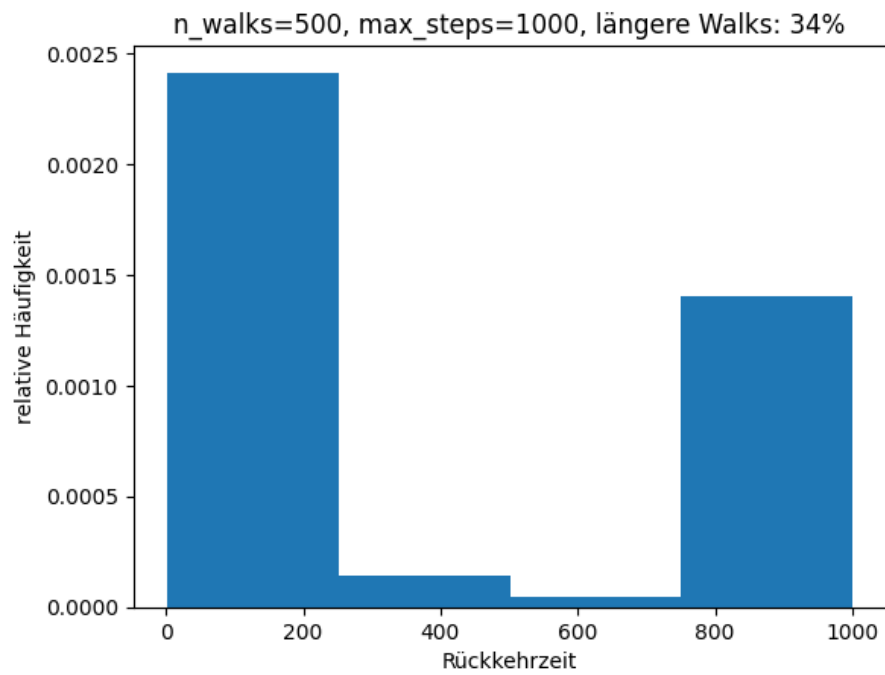
- Bestimme die erste Rückkehrzeit mit `first_return_time()` und `max_steps=1000` für 500 Walks mit unterschiedlichen Seeds.
- Stelle die (relative) Häufigkeitsdichte der ersten Rückkehrzeit mit einem Histogramm dar. Verwende dazu die Funktion `plt.hist()` mit der Option `bins="fd"`, damit die Freedman-Diaconis-Regel für die Bestimmung der Anzahl Bins verwendet wird.
- Schreibe die Anzahl Simulationen und Mittelwert und Standardabweichung der Rückkehrzeit in den Titel des Diagramms.

Erhöhe die maximale Anzahl Schritte `max_steps` und beobachte, wie sich die Verteilung verändert.

```
max_steps = 1000
n_walks = 500
```

```
return_times = np.array([first_return_time(max_steps) for _ in range(n_walks)])
```

```
plt.hist(return_times, bins="fd", density=True)
plt.xlabel("Rückkehrzeit")
plt.ylabel("relative Häufigkeit")
plt.title(
    f"n_walks={n_walks}, max_steps={max_steps}, längere Walks: {np.sum(return_times >= max_steps)}
)
plt.show()
```



```
np.sum(return_times == max_steps) / n_walks
```

```
np.float64(0.338)
```

Beobachtungen:

- Die Rückkehrzeit ist meist eher kurz (<200 Schritte).
- Doch kommen sehr viele sehr sehr lange Rückkehrzeiten (~ 1000 Schritte) vor.
- Es sind ca 1/3 der Walks, die mehr als 1000 Schritte benötigen.
- Erhöht man die maximale Anzahl Schritte, so wird die Verteilung immer breiter und die Wahrscheinlichkeit für sehr lange Rückkehrzeiten nimmt nur leicht ab.

0.12.3 Stochastische Prozesse

Vertiefung

Übersicht

1. Monte-Carlo-Simulation
2. Simulated Annealing

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

1. Monte-Carlo-Simulation Monte-Carlo-Simulation ist eine grosse Klasse von Methoden, die Fragestellungen mithilfe einer grossen Anzahl Zufallsexperimenten beantworten.

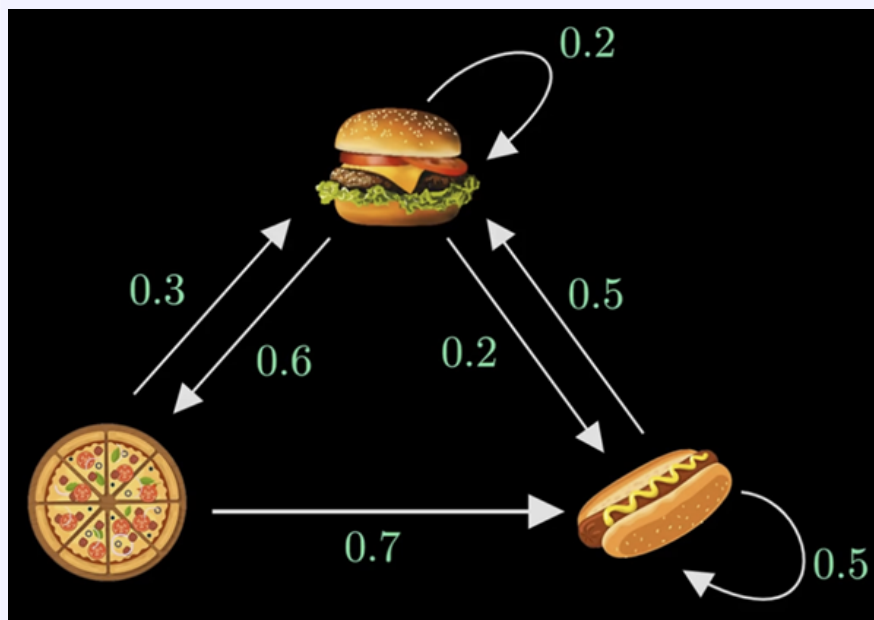
Beispiele:

- Bestimmen von statistische Eigenschaften von Zufallsprozessen, z.B. Markow-Chain-Monte-Carlo-Verfahren (MCMC).
- Resampling von Daten, um statistische Eigenschaften zu bestimmen, z.B. Bootstrapping.
- Stochastisches Sampling der Zielfunktion eines Optimierungsproblems, siehe Simulated Annealing im Teil 2.

a) Markow-Ketten mit diskreten Zuständen

- Zustandsverlauf
- Häufigkeitsverteilung der Zustände über einen langen Zeitraum
- Häufigkeitsverteilung über viele Realisierungen

Wir betrachten das folgende Markow-Ketten-Modell mit 3 Zuständen:



Normalized Nerd. (2020, Oktober 25). *Markov Chains Clearly Explained! Part—1* [Videoaufnahme]. <https://www.youtube.com/watch?v=i3AkTO9HLXo>

Ziele

1. Schätzung der stationären Verteilung durch Simulation
2. Bestimmung der Aufenthaltswahrscheinlichkeit in den einzelnen Zuständen durch Monte-Carlo-Simulation

Für die Simulation des Markow-Ketten-Modells nutzen wir eine Übergangsmatrix, welche die Wahrscheinlichkeiten der Übergänge zwischen den Zuständen beschreibt:

$$M = \begin{pmatrix} 0.2 & 0.6 & 0.2 \\ 0.3 & 0 & 0.7 \\ 0.5 & 0 & 0.5 \end{pmatrix} \quad (159)$$

M_{ij} ist die Wahrscheinlichkeit, dass das System in einem Zeitschritt Δt vom Zustand i in den Zustand j wechselt.

```
states = np.array([0, 1, 2])
state_names = np.array(["Burger", "Pizza", "Hotdog"])
transition_matrix = np.array(
    [
        [0.2, 0.6, 0.2],
        [0.3, 0.0, 0.7],
        [0.5, 0.0, 0.5],
    ]
)

def markov_next(current_state, rng):
    return rng.choice(states, p=transition_matrix[current_state])

def markov_process(n, initial_state=None, seed=None):
    rng = np.random.default_rng(seed)
    state = initial_state if initial_state is not None else rng.choice(states)
    chain = [state]
    for _ in range(n):
        state = markov_next(state, rng)
        chain.append(state)
    return np.array(chain)

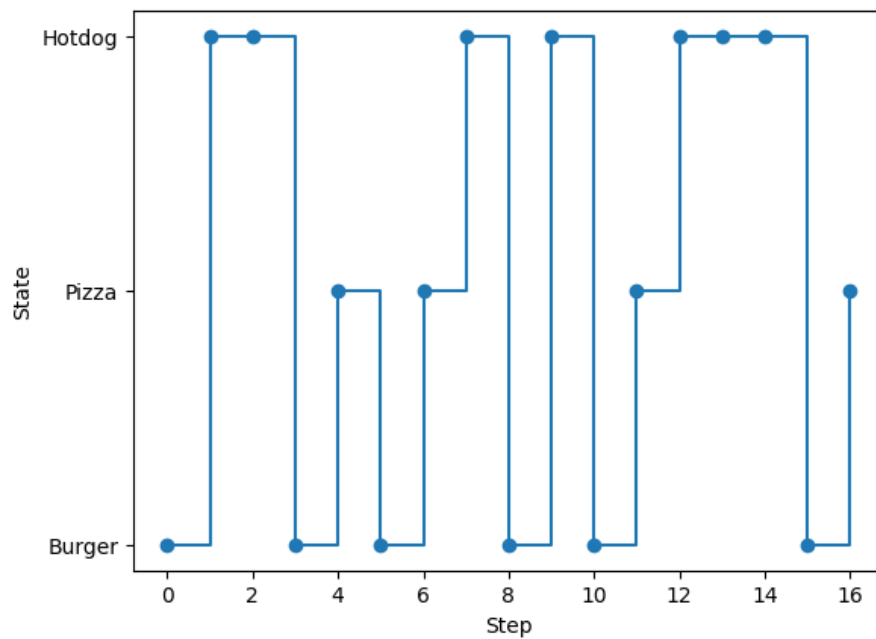
markov_process(10, initial_state=0)

array([0, 0, 0, 1, 0, 1, 2, 0, 1, 2, 2])
```

Zustandsverlauf

```
chain = markov_process(16)

plt.step(chain, "o -", where="post")
plt.yticks(states, state_names)
plt.xlabel("Step")
plt.ylabel("State")
plt.show()
```

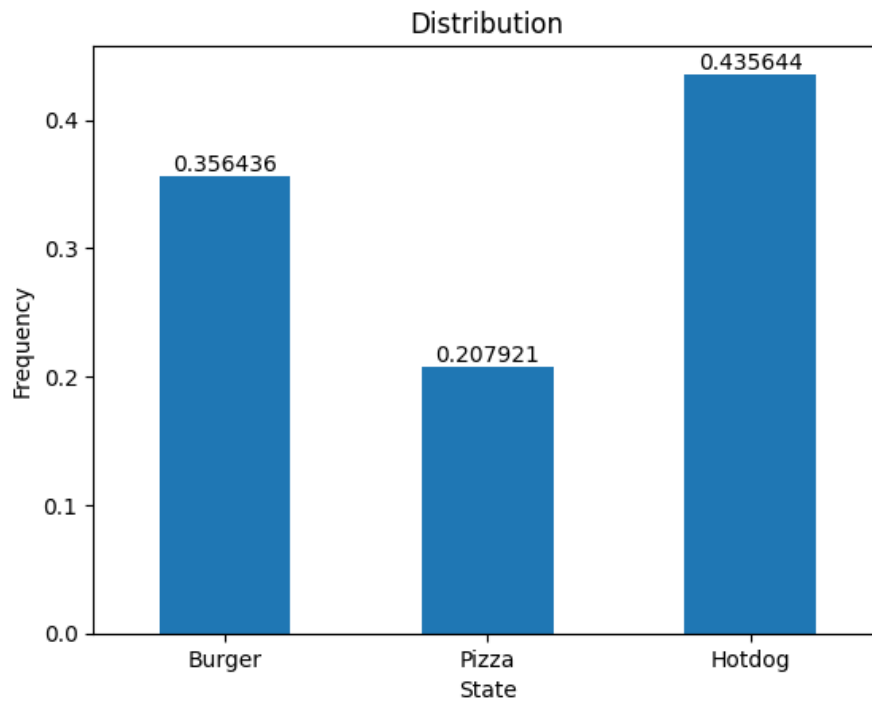


Aufenthaltswahrscheinlichkeit über einen langen Verlauf

```
n_steps = 100
chain = markov_process(n_steps)
frequencies = pd.Series(chain).value_counts(normalize=True).sort_index()
frequencies
```

```
0    0.356436
1    0.207921
2    0.435644
Name: proportion, dtype: float64
```

```
ax = frequencies.plot(kind="bar", rot=0)
ax.set_ylabel("Frequency")
ax.set_xlabel("State")
ax.set_xticks(states)
ax.set_xticklabels(state_names)
ax.set_title("Distribution")
ax.bar_label(ax.containers[0])
plt.show()
```



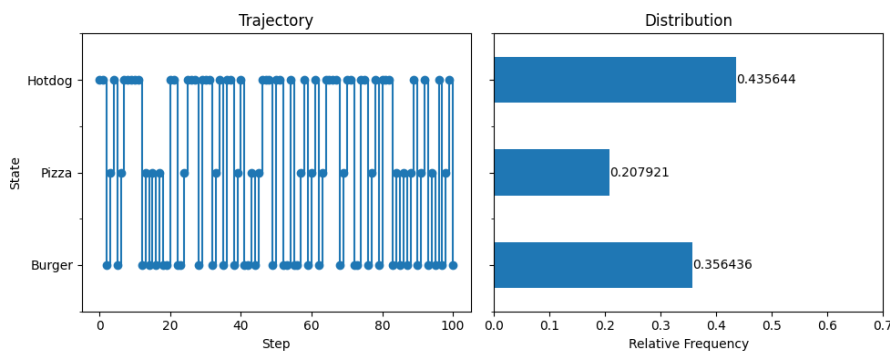
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
```

```
ax1.step(chain, "o -", where="post")
ax1.set_title("Trajectory")
ax1.set_xlabel("Step")
ax1.set_ylabel("State")
ax1.set_yticks(states, state_names)
```

```
frequencies.plot(kind="barh", ax=ax2)
ax2.set_title("Distribution")
ax2.set_xlabel("Relative Frequency")
ax2.set_yticks(states, state_names)
ax2.set_ylabel("State")
```

```
ax2.bar_label(ax2.containers[0])
ax2.set_xlim(0, 0.7)
```

```
plt.tight_layout()
plt.show()
```



Aufenthaltswahrscheinlichkeit nach t Schritten Nun bestimmen wir die Wahrscheinlichkeit, dass sich das System nach t Schritten in einem der Zustände befindet. Dazu führen wir

eine Monte-Carlo-Simulation durch, indem wir viele Markow-Ketten simulieren.

```
n_steps = 10
n_chains = 10000
initial_state = 0

chains = np.array([markov_process(n_steps, initial_state) for _ in range(n_chains)])

array([[0, 1, 2, ..., 0, 1, 2],
       [0, 0, 1, ..., 2, 0, 2],
       [0, 1, 2, ..., 2, 0, 2],
       ...,
       [0, 2, 0, ..., 2, 0, 1],
       [0, 1, 2, ..., 0, 1, 2],
       [0, 1, 2, ..., 1, 0, 1]], shape=(10000, 11))
```

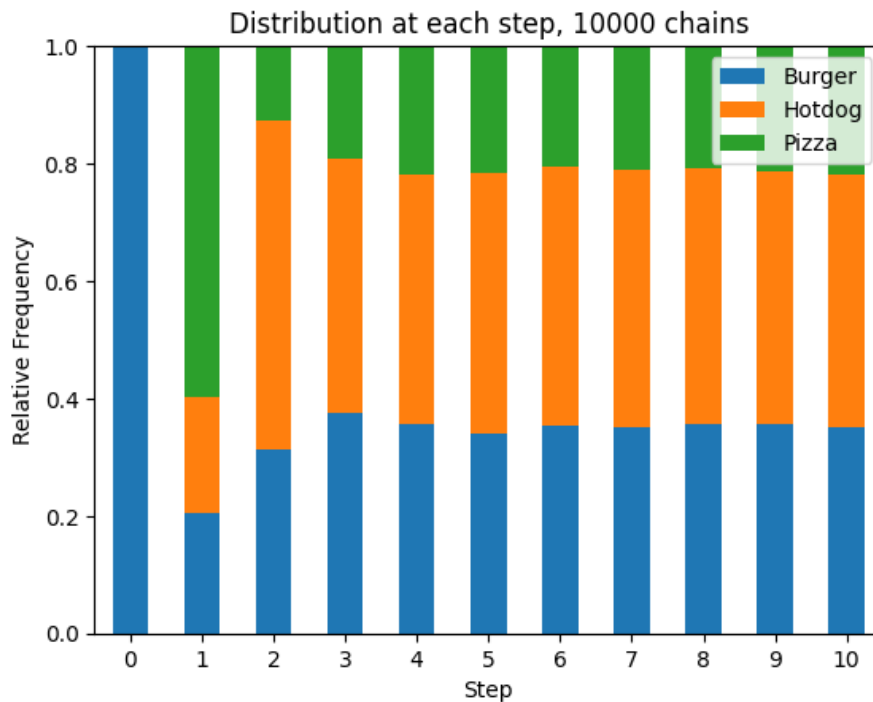
Dann bestimmen wir die relativen Häufigkeiten der Zustände für alle Zeitschritte t .

```
df_frequencies = pd.DataFrame(
    {
        step: pd.Series(state_names[chains[:, step]]).value_counts(normalize=True)
        for step in range(chains.shape[1])
    }
).T
df_frequencies
```

	Burger	Hotdog	Pizza
0	1.0000	NaN	NaN
1	0.2046	0.1990	0.5964
2	0.3133	0.5604	0.1263
3	0.3759	0.4334	0.1907
4	0.3551	0.4248	0.2201
5	0.3406	0.4427	0.2167
6	0.3550	0.4410	0.2040
7	0.3521	0.4365	0.2114
8	0.3558	0.4367	0.2075
9	0.3577	0.4289	0.2134
10	0.3504	0.4303	0.2193

Diese stellen wir dar.

```
df_frequencies.plot(
    kind="bar",
    stacked=True,
    title=f"Distribution at each step, {n_chains} chains",
    xlabel="Step",
    ylabel="Relative Frequency",
    rot=0,
)
plt.show()
```



Man sieht, dass die Verteilung nach wenigen Schritten um einen Gleichgewichtszustand schwankt.

Die Verteilung konvergiert gegen die stationäre Verteilung. Je mehr Markow-Ketten man simuliert, desto schneller konvergiert die Verteilung.

In einfachen Fällen kann man die stationäre Verteilung analytisch bestimmen. In komplexeren Systemen kann man sie mit Hilfe der Monte-Carlo-Simulation bestimmen.

b) Markow-Ketten mit kontinuierlichen Zuständen Der Zustandsraum einer Markow-Kette kann natürlich auch kontinuierlich sein. Ausgehend von einem Startwert $x_0 \in \mathbb{R}$, soll nun ein Random Walk simuliert werden, der in jedem Schritt eine Zufallszahl aus einer Normalverteilung mit Mittelwert $\mu=0$ und Standardabweichung $\sigma=1$ zum letzten Zustand addiert.

```
def random_walk(n, x0=0, seed=None, mu=0, sigma=1):
    rng = np.random.default_rng(seed)
    steps = rng.normal(mu, sigma, n)
    steps = np.hstack((x0, steps))
    return np.cumsum(steps)
```

```
n_steps = 200
walk = random_walk(n_steps)

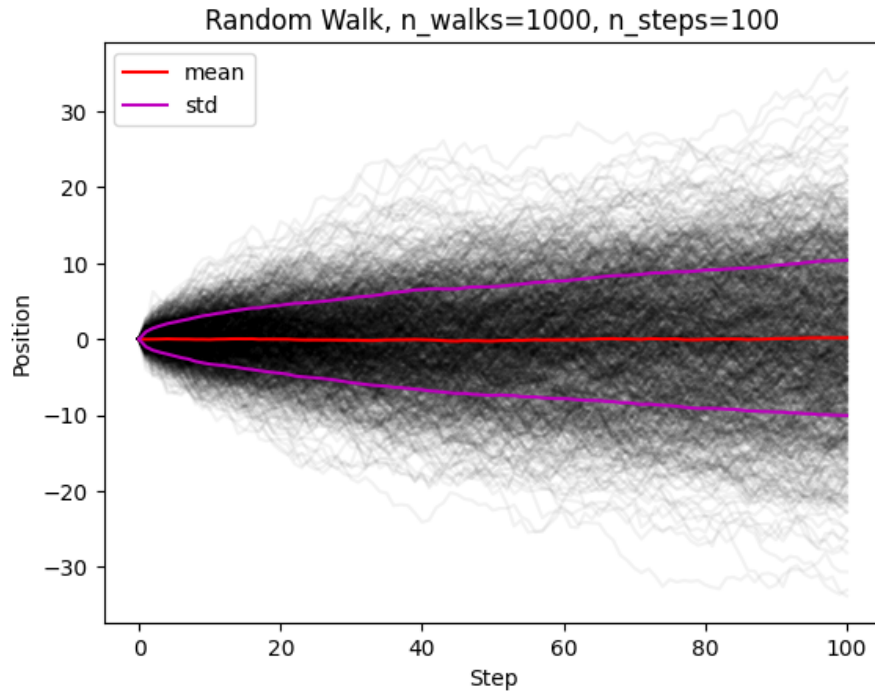
plt.plot(walk)
plt.xlabel("Step")
plt.ylabel("Position")
plt.title(f"Random Walk, n_steps={n_steps}")
plt.show()
```




Auch hier können wir die Aufenthaltswahrscheinlichkeit nach t Schritten bestimmen, indem wir viele Realisierungen des Random Walks simulieren.

```
n_steps = 100
n_walks = 1000
walks = np.array([random_walk(n_steps) for _ in range(n_walks)])

plt.plot(walks.T, "k", alpha=0.05)
plt.plot(walks.mean(axis=0), "r", label="mean")
plt.plot(walks.mean(axis=0) + walks.std(axis=0), "m", label="std")
plt.plot(walks.mean(axis=0) - walks.std(axis=0), "m")
plt.xlabel("Step")
plt.ylabel("Position")
plt.title(f"Random Walk, n_walks={n_walks}, n_steps={n_steps}")
plt.legend()
plt.show()
```



2. Simulated Annealing Simulated Annealing ist ein stochastisches Optimierungsverfahren, das auf verschiedenste Probleme angewendet werden kann. Stochastisch bedeutet hier, dass bei der Suche des Optimums Zufallszahlen verwendet werden, um den Lösungsraum D zu durchsuchen. Als eine Konsequenz kann man nicht sicher sein, ob man je das globale Optimum wirklich findet. Solche Verfahren werden dann genutzt, wenn exakte Methoden nicht anwendbar sind, weil der Lösungsraum zu gross ist oder die Zielfunktion zu komplex oder nicht differenzierbar ist.

Das gesuchte Optimum kann ein globales Minimum oder Maximum einer (skalaren) Zielfunktion f sein. Hier formulieren wir es als ein Minimierungsproblem, ansonsten könnten wir einfach das Minimum von $g(x) = -f(x)$ suchen.

$$\text{Suche } x_{\text{opt}} = \underset{x \in D}{\operatorname{argmin}} f(x) \quad \text{für } f : D \rightarrow \mathbb{R} \quad (160)$$

Der Lösungsraum D kann eine Teilmenge des Vektorraums $\mathbb{R}^n$ sein, aber auch eine Menge von diskreten Werten. Die Funktion f muss nicht stetig oder differenzierbar sein.

Vorgehen:

1. Startpunkt $x_i \in D$ für $i = 0$ wählen, evtl. zufällig.
2. Neuen Punkt x_* zufällig aus der Umgebung von x_i ziehen.
3. Falls $f(x_*) < f(x_i)$, dann x_* als neuen Punkt wählen, also $x_{i+1} = x_*$. Sonst mit Wahrscheinlichkeit p als neuen Punkt wählen, oder verwerfen.
4. Falls Abbruchkriterium erfüllt, dann x_i als Lösung zurückgeben, sonst mit $i = i + 1$ zu 2.

Die Akzeptanzwahrscheinlichkeit p wird häufig als Boltzmann-Verteilung gewählt:

$$p = \exp\left(-\frac{f(x_*) - f(x_i)}{T}\right) \quad (161)$$

- Je schlechter der Lösungsvorschlag x_* , d.h. je grösser die Differenz $f(x_*) - f(x_i)$, desto kleiner die Akzeptanzwahrscheinlichkeit
- Je höher die Temperatur T , desto grösser die Akzeptanzwahrscheinlichkeit
- Die Temperatur wird mit der Zeit (Anzahl Iterationen) abgesenkt, um die Akzeptanzwahrscheinlichkeit zu verringern.

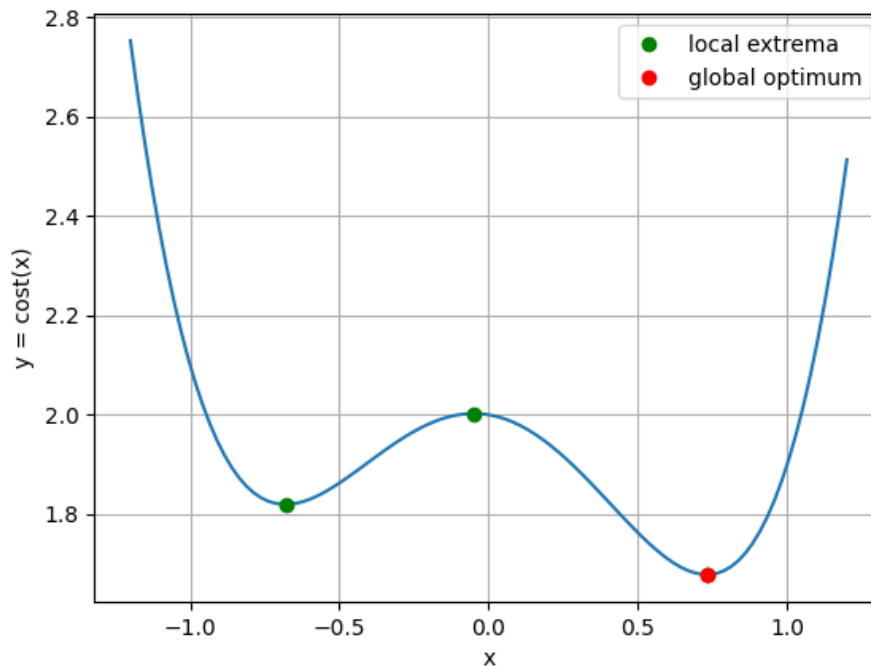
Die Qualität und Effizienz des Verfahrens hängt stark von der Wahl der Umgebung des Lösungsvorschlags x_i ab. Die Umgebung sollte so gewählt werden, dass die Wahrscheinlichkeit, einen besseren Punkt zu finden, nicht zu klein ist, und gleichzeitig die Möglichkeit besteht, den gesamten Lösungsraum zu durchsuchen und so lokalen Extremstellen zu entkommen.

Videos:

- “Simulated Annealing” - Andreas Müller

Beispiel 1: Minimierung einer Funktion in 1D

```
def f(x):  
    return x**4 - x**2 - 0.1 * x + 2  
  
x = np.linspace(-1.2, 1.2, 100)  
y = f(x)  
  
x_extrema = np.array((-0.680639, -0.0502538, 0.730893))  
y_extrema = f(x_extrema)  
x_optimum = x_extrema[np.argmin(y_extrema)]  
y_optimum = f(x_optimum)  
  
plt.plot(x, y)  
plt.plot(x_extrema, y_extrema, "go", label="local extrema")  
plt.plot(x_optimum, y_optimum, "ro", label="global optimum")  
plt.xlabel("x")  
plt.ylabel("y = cost(x)")  
plt.grid()  
plt.legend()  
plt.show()
```



Schritt 1: Random Walk im Lösungsraum

```
rng = np.random.default_rng(123456789)  
n_steps = 1000
```

```

sigma = 0.1

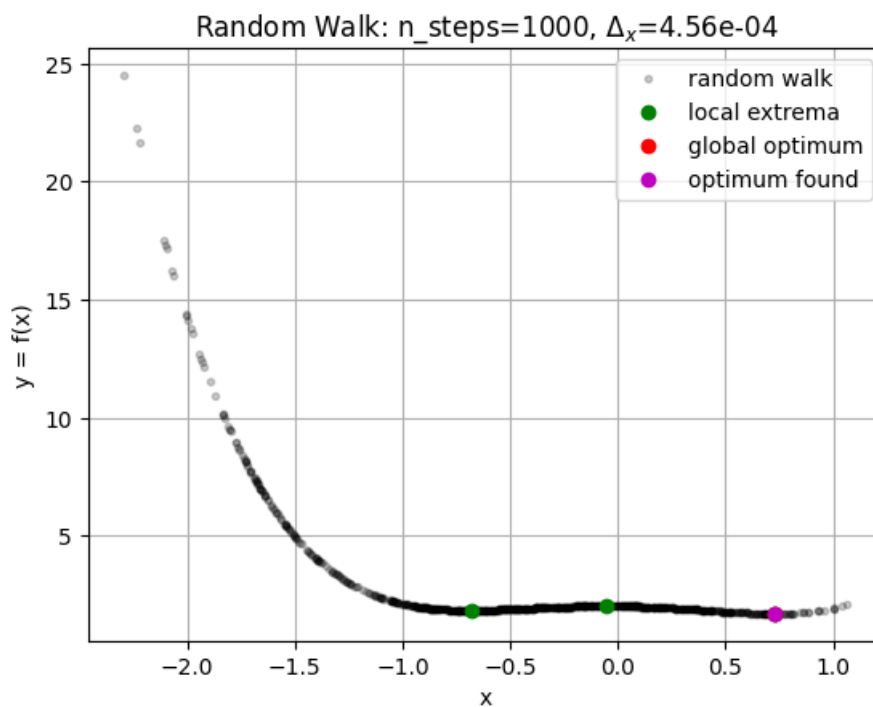
xi = -0.5
yi = f(xi)
res = [(xi, yi)]

# Random Walk
for _ in range(n_steps):
    xi = rng.normal(xi, sigma)
    yi = f(xi)
    res.append((xi, yi))

x, y = np.array(res).T
i_best = np.argmin(y)
x_best = x[i_best]
y_best = y[i_best]
Dx = np.abs(x_best - x_optimum)

plt.plot(x, y, "k.", alpha=0.2, label="random walk")
plt.plot(x_extrema, y_extrema, "go", label="local extrema")
plt.plot(x_optimum, y_optimum, "ro", label="global optimum")
plt.plot(x_best, y_best, "mo", label="optimum found")
plt.xlabel("x")
plt.ylabel("y = f(x)")
plt.title(f"Random Walk: n_steps={n_steps},  $\Delta_x={Dx:.2e}$ ")
plt.grid()
plt.legend()
plt.show()

```



Beobachtung: Der Random Walk im Lösungsraum bewegt sich zwar dort, wo die Funktion f lokale Extrema hat, aber er konvergiert nicht zum Minimum. Ändert man den Startpunkt, so findet er das Minimum womöglich nicht.

Er muss also etwas in die richtige Richtung gelenkt werden.

Schritt 2: Nur bessere Punkte akzeptieren Nun werden nur diejenigen Punkte akzeptiert, die besser sind als der aktuelle Punkt. Der Random Walk bewegt sich nun in die Richtung des Minimums.

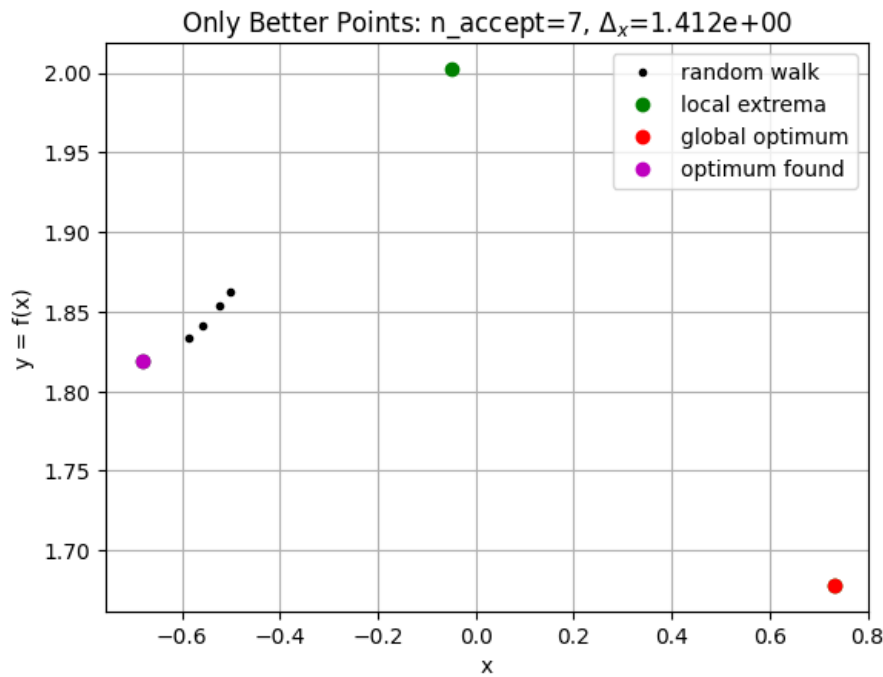
```
rng = np.random.default_rng(123456789)
n_steps = 1000
sigma = 0.1

xi = -0.5
yi = f(xi)
res = [(xi, yi)]

# Only accept better points
for _ in range(n_steps):
    x_trial = rng.normal(xi, sigma)
    y_trial = f(x_trial)
    if y_trial < yi:
        xi = x_trial
        yi = y_trial
        res.append((xi, yi))

x, y = np.array(res).T
i_best = np.argmin(y)
x_best = x[i_best]
y_best = y[i_best]
Dx = np.abs(x_best - x_optimum)
n_accept = len(x) - 1

plt.plot(x, y, "k.", label="random walk")
plt.plot(x_extrema, y_extrema, "go", label="local extrema")
plt.plot(x_optimum, y_optimum, "ro", label="global optimum")
plt.plot(x_best, y_best, "mo", label="optimum found")
plt.xlabel("x")
plt.ylabel("y = f(x)")
plt.grid()
plt.legend()
plt.title(f"Only Better Points: n_accept={n_accept},  $\Delta x = {Dx:.3e}")
plt.show()$ 
```



Beobachtung: Der Random Walk kommt nur sehr langsam vorwärts, weil zu viele Schritte nicht akzeptiert werden. Es besteht die Gefahr, dass der Random Walk in dem lokalen Minimum bei $x = -0.680639$ stecken bleibt. Erhöht man die Schrittweite σ , so kann es sein, dass ein lokales Maximum übersprungen und das globale Maximum gefunden wird.

Schritt 3: Akzeptanzwahrscheinlichkeit Schliesslich soll mit einer gewissen Wahrscheinlichkeit, ein Vorschlag auch dann akzeptiert werden, wenn er eigentlich schlechter ist, als der letzte. Die Akzeptanzwahrscheinlichkeit wird mit der Temperatur T gesteuert, so dass sie mit der Zeit abnimmt, damit der Algorithmus konvergiert.

```
def acceptance_probability(new_cost, old_cost, T):
    return np.exp( -(new_cost - old_cost) / T)

rng = np.random.default_rng(123456789)
n_steps = 1000
sigma = 0.1

Ti = 10
xi = -0.5
yi = f(xi)
x_best = xi
y_best = yi
res = [(xi, yi, x_best, y_best, Ti)]

# Simulated Annealing (with acceptance probability)
for _ in range(n_steps):
    Ti = Ti / 1.1
    x_trial = rng.normal(xi, sigma)
    y_trial = f(x_trial)
    if y_trial < yi or rng.random() < acceptance_probability(y_trial, yi, Ti):
        xi = x_trial
        yi = y_trial
        if yi < y_best:
```

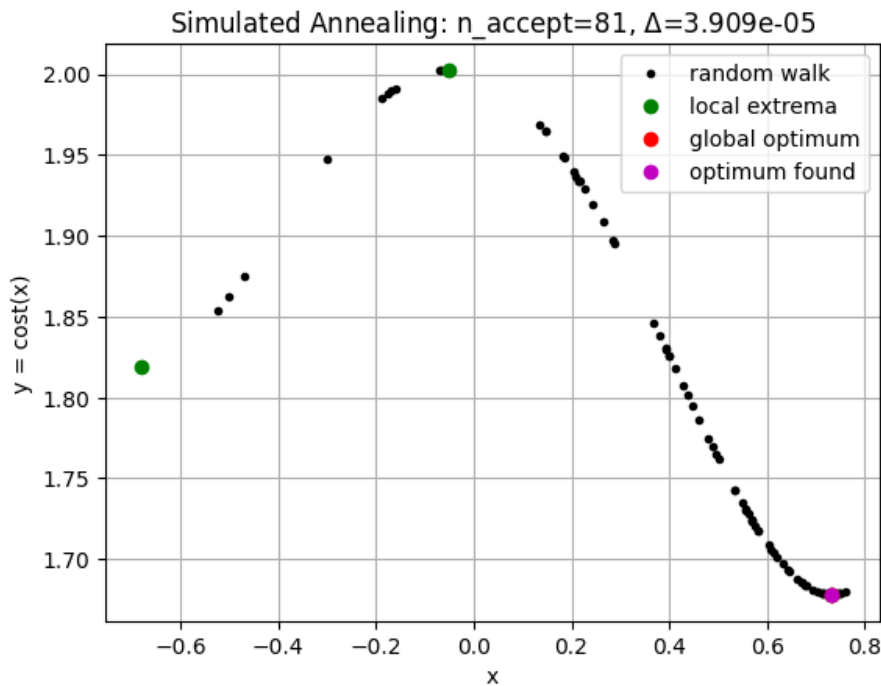
```

        x_best = xi
        y_best = yi
        res.append((xi, yi, x_best, y_best, Ti))

x, y, x_best, y_best, Ti = np.array(res).T
delta = np.abs(x_best[-1] - x_optimum)
n_accept = len(x) - 1

plt.plot(x, y, "k.", label="random walk")
plt.plot(x_extrema, y_extrema, "go", label="local extrema")
plt.plot(x_optimum, y_optimum, "ro", label="global optimum")
plt.plot(x_best[-1], y_best[-1], "mo", label="optimum found")
plt.xlabel("x")
plt.ylabel("y = cost(x)")
plt.grid()
plt.legend()
plt.title(f"Simulated Annealing: n_accept={n_accept},  $\Delta$ = $\{delta:.3e\}$ ")
plt.show()

```

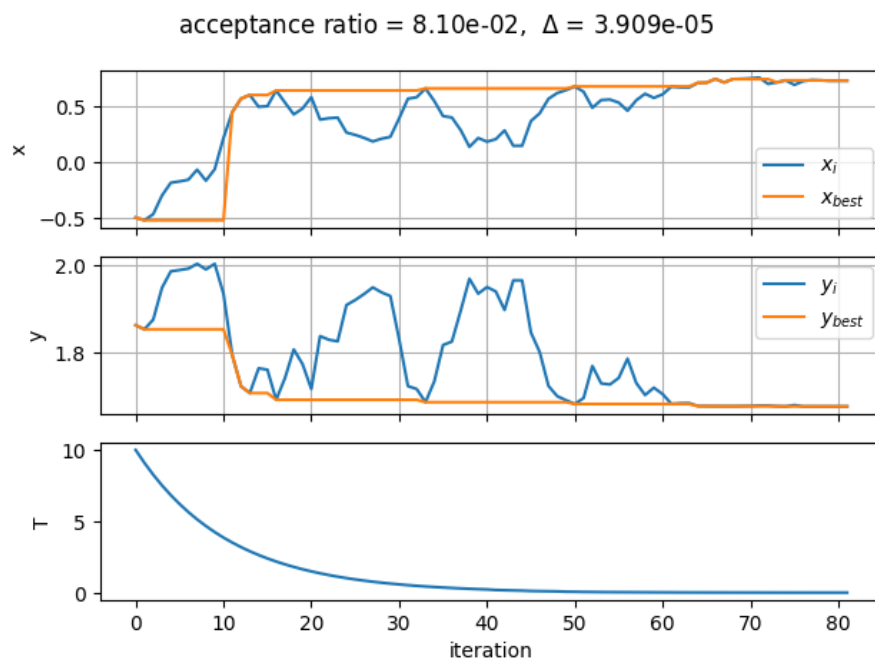


```

fig, (ax1, ax2, ax3) = plt.subplots(3, 1, sharex=True)
ax1.plot(x, label="$x_i$")
ax1.plot(x_best, label="$x_{best}$")
ax1.set_ylabel("x")
ax1.legend()
ax1.grid()
ax2.plot(y, label="$y_i$")
ax2.plot(y_best, label="$y_{best}$")
ax2.set_ylabel("y")
ax2.legend()
ax2.grid()
ax3.plot(Ti, label="T")
ax3.set_ylabel("T")

```

```
ax3.set_xlabel("iteration")
plt.suptitle(f"acceptance ratio = {n_accept / n_steps:.2e},  $\Delta$  = {delta:.3e}")
plt.tight_layout()
plt.show()
```



Bibliography

G. Walz. *Interpolation von Daten und Funktionen: Klartext für Nichtmathematiker*. Springer Fachmedien Wiesbaden, 2020. ISBN 9783658306588. doi: 10.1007/978-3-658-30658-8. URL <http://dx.doi.org/10.1007/978-3-658-30658-8>.